adt

tree

preliminaries

- search
  - one of the central problems in computing
  - basis of many computing applications
    - search engines
    - bank accounts
    - books in library
    - on-line shopping
    - streaming tv series or movies (yay)
    - spotify
    - and many, many more



preliminaries

- search in lists
  - O(n) or linear - arbitrary
  - O(log n) or logarithmic - sorted
    - but sorting takes O(n log n)
- search in queues?
- search in stacks?

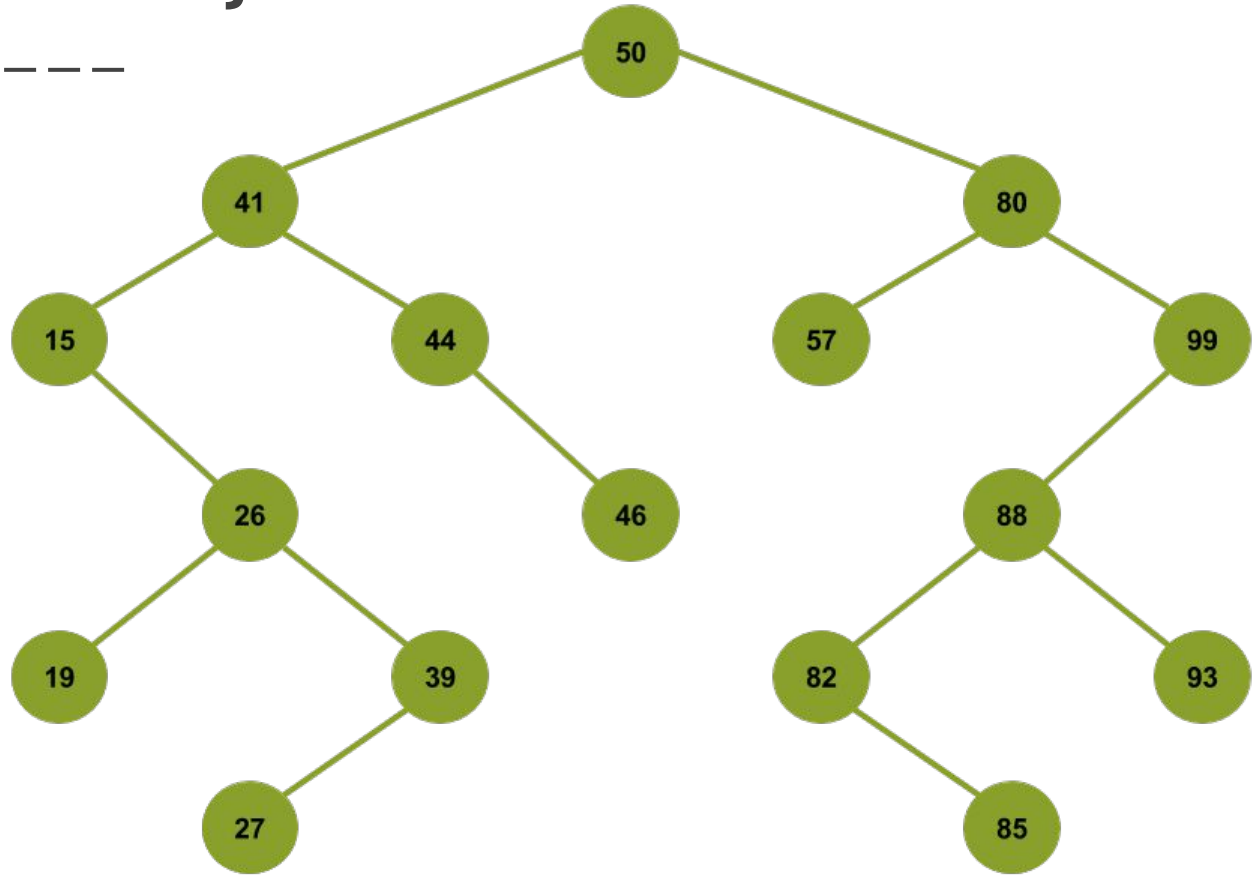


tree

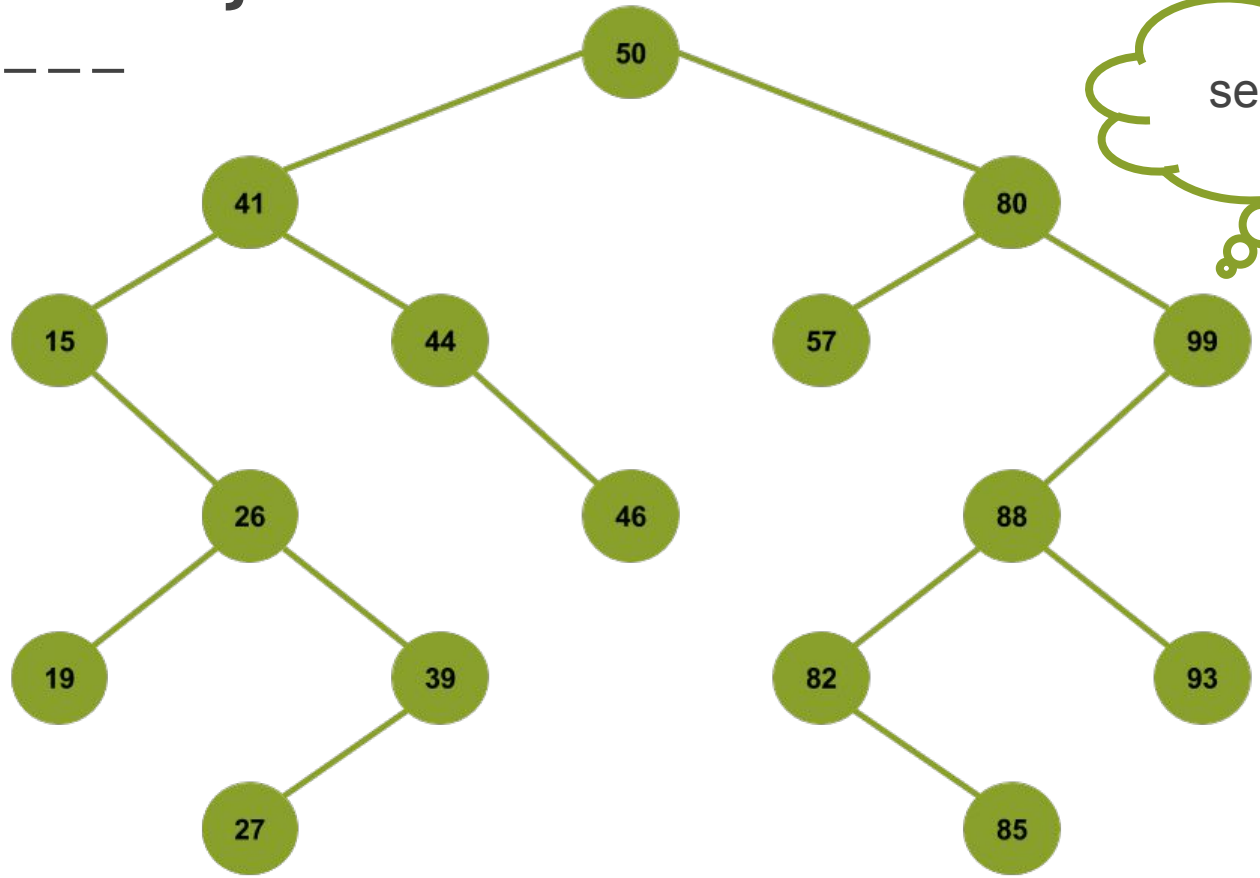
- an adt may be created to aid in fast search without the prohibitive time complexity of sorting
- let’s go and see an old friend
  - binary search tree



hello my old friend

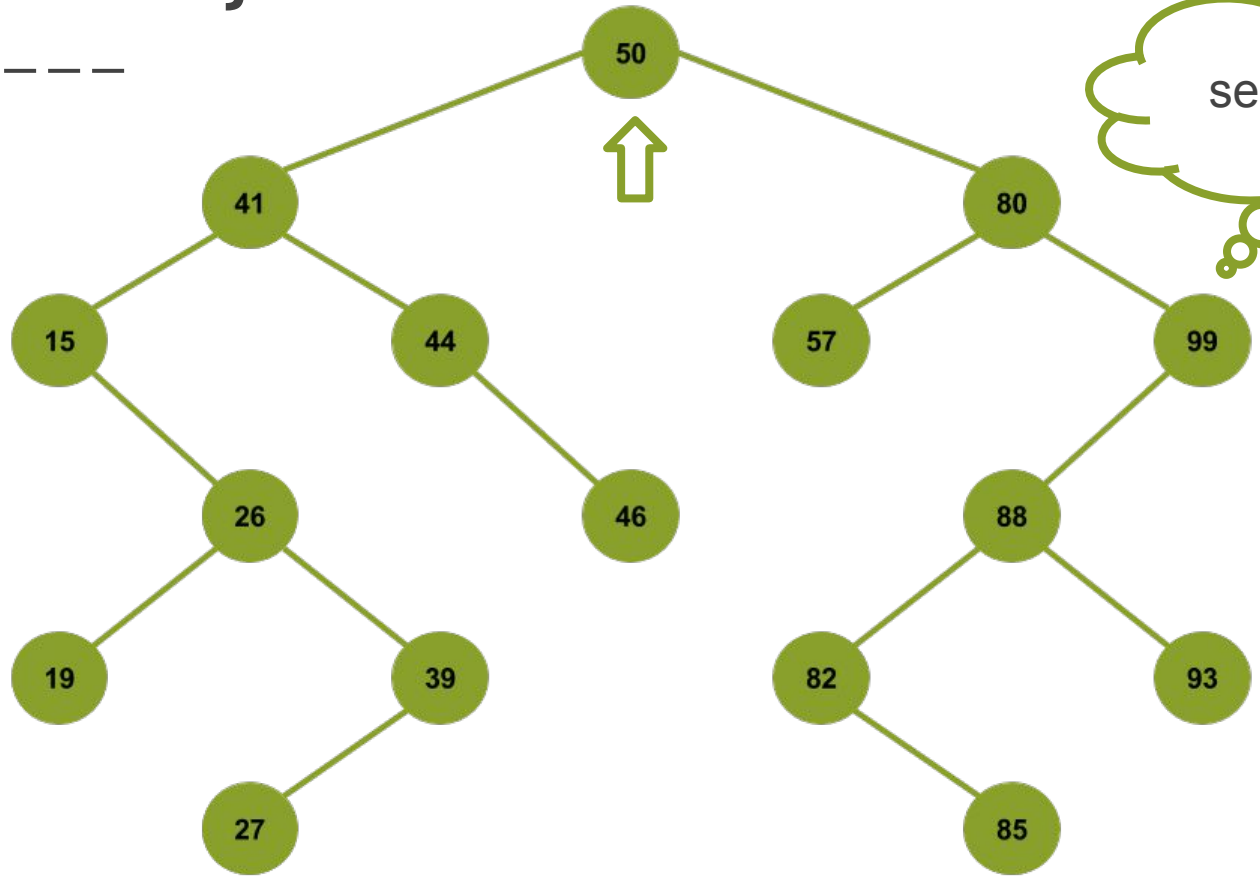


hello my old friend



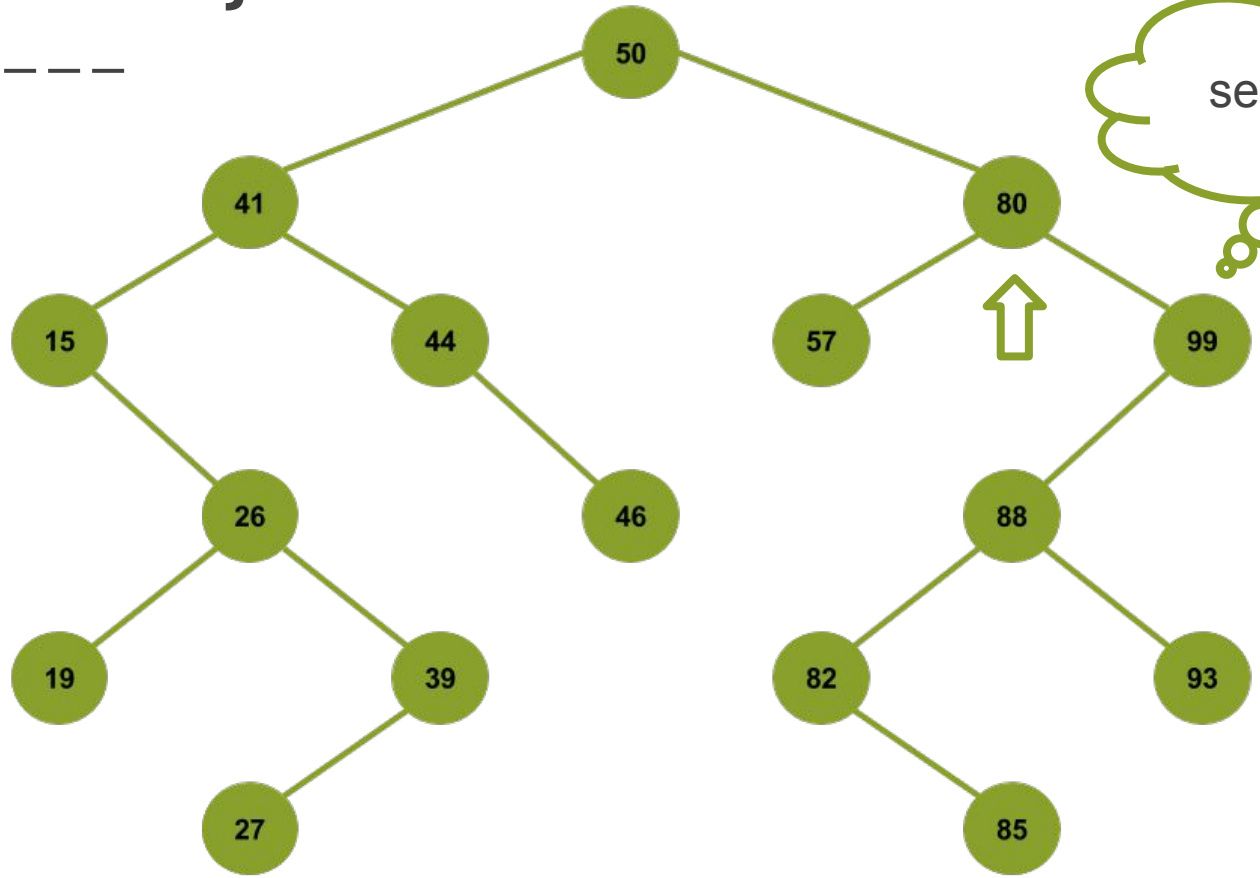
search for 52

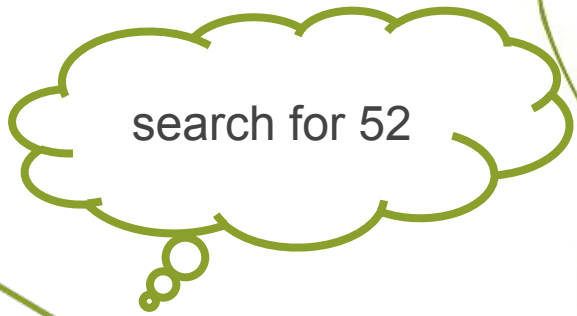
hello my old friend



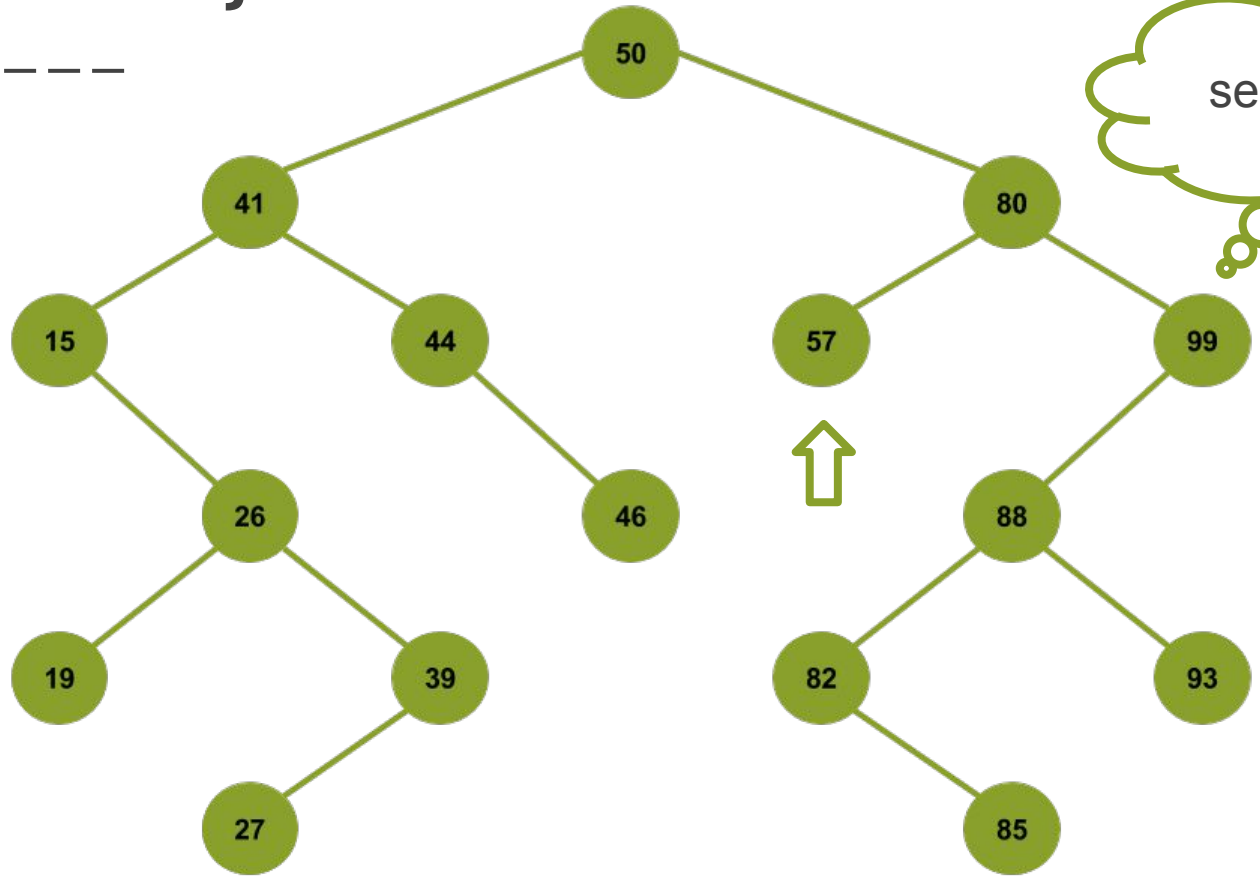
search for 52

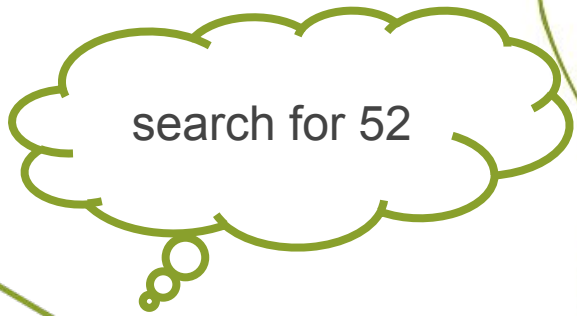
hello my old friend



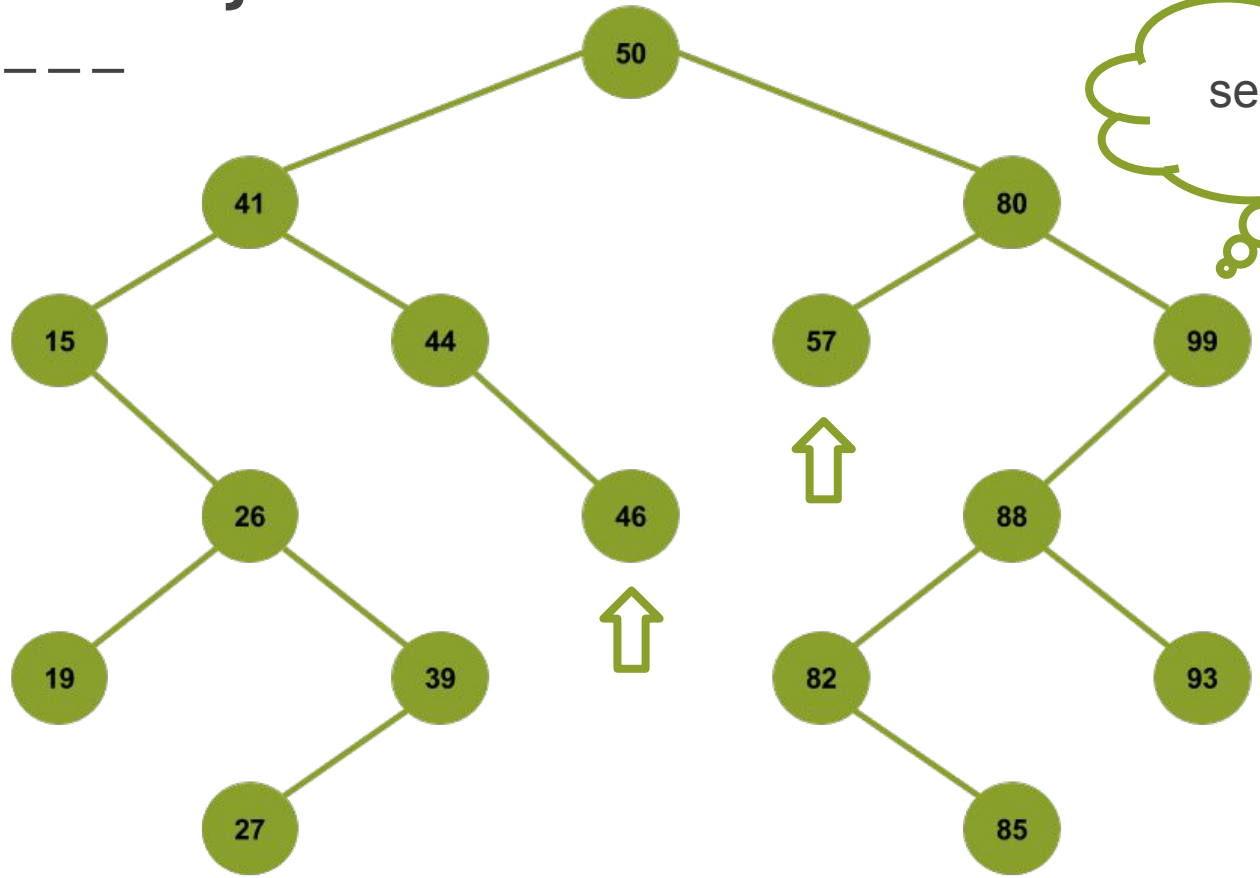


hello my old friend





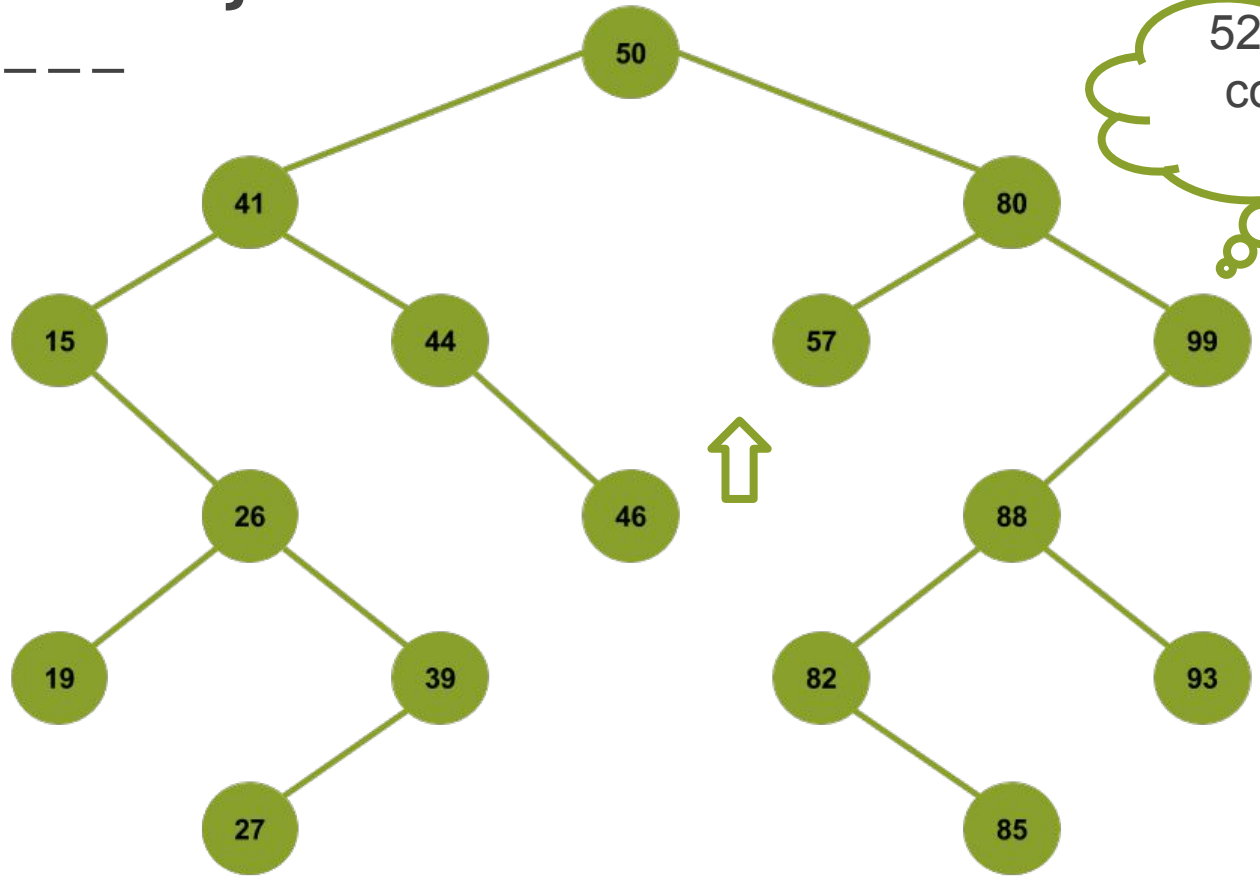
hello my old friend



search for 52

hello my old friend

– – –



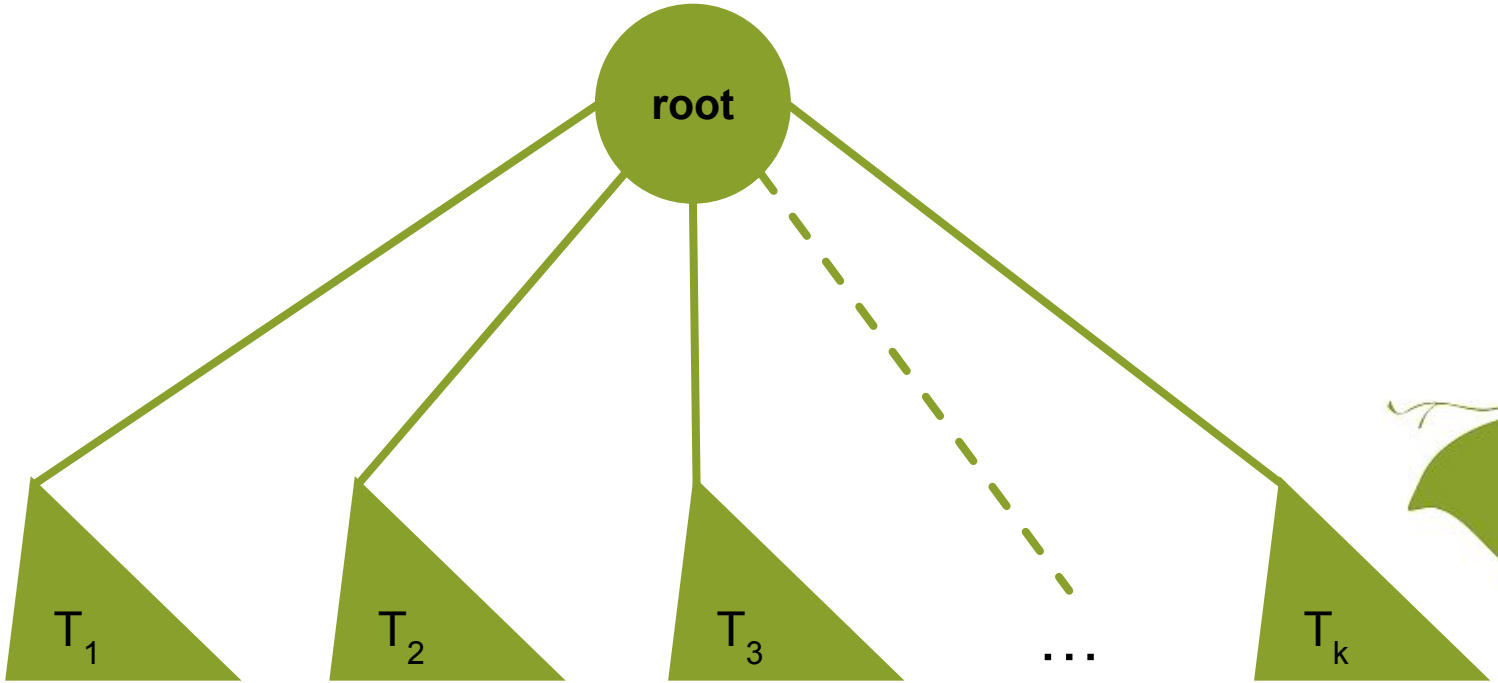
52 not found.
comparison
count?

tree

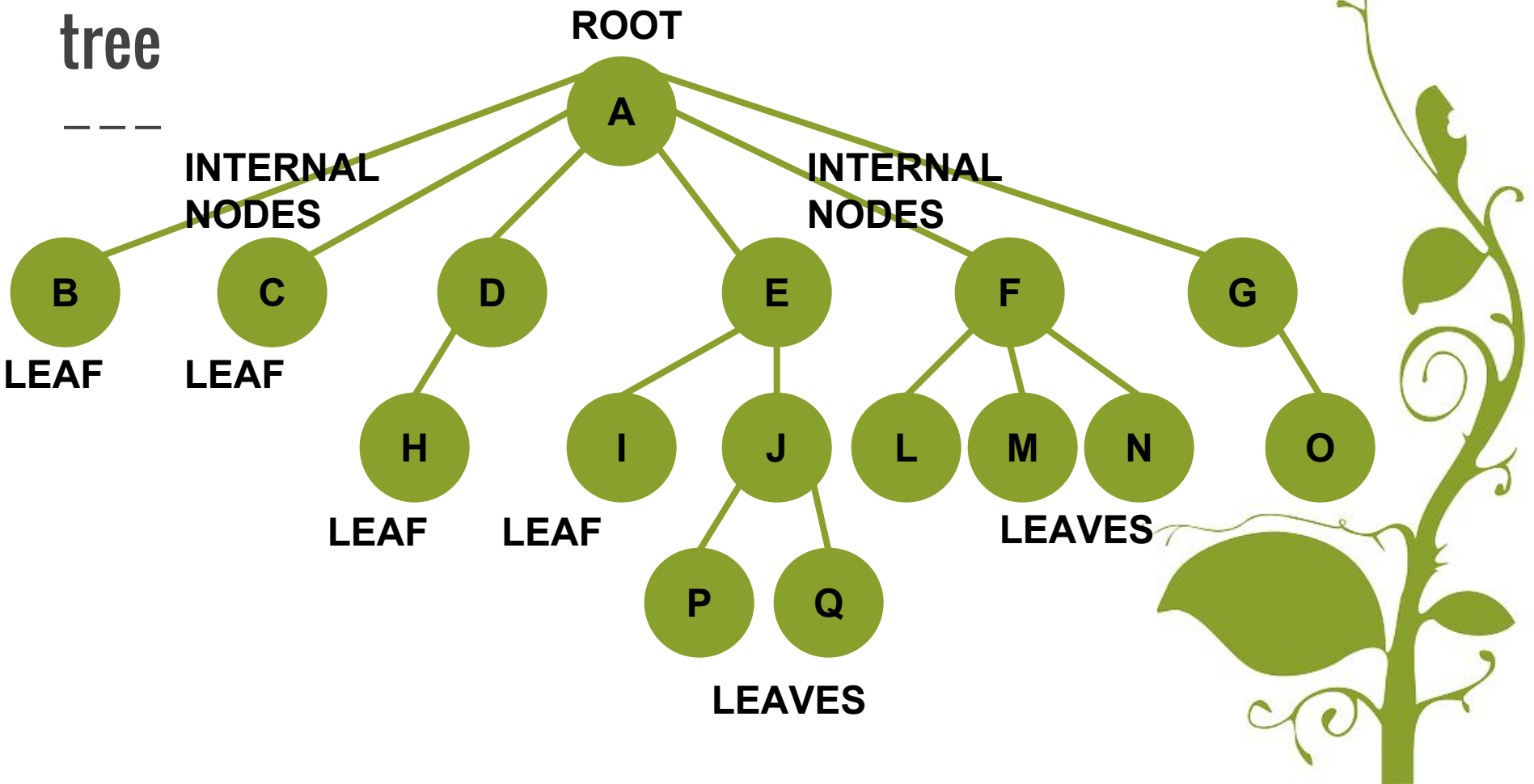
- collection of nodes
  - may be empty
  - otherwise, the tree consists of a distinguished node r, called the root, and zero or more (sub)trees T_1, T_2, T_3, …, T_k, each of whose roots are connected by a directed edge to r.
    - the root of each subtree is said to be a child of r, and r is the parent of each subtree root



tree

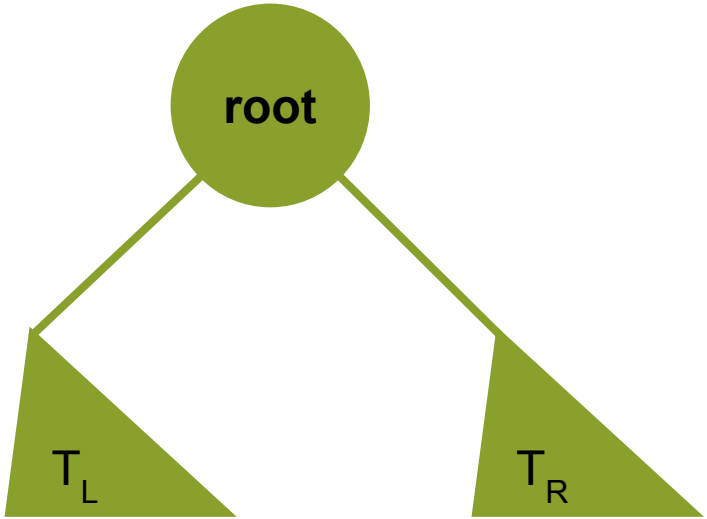


tree

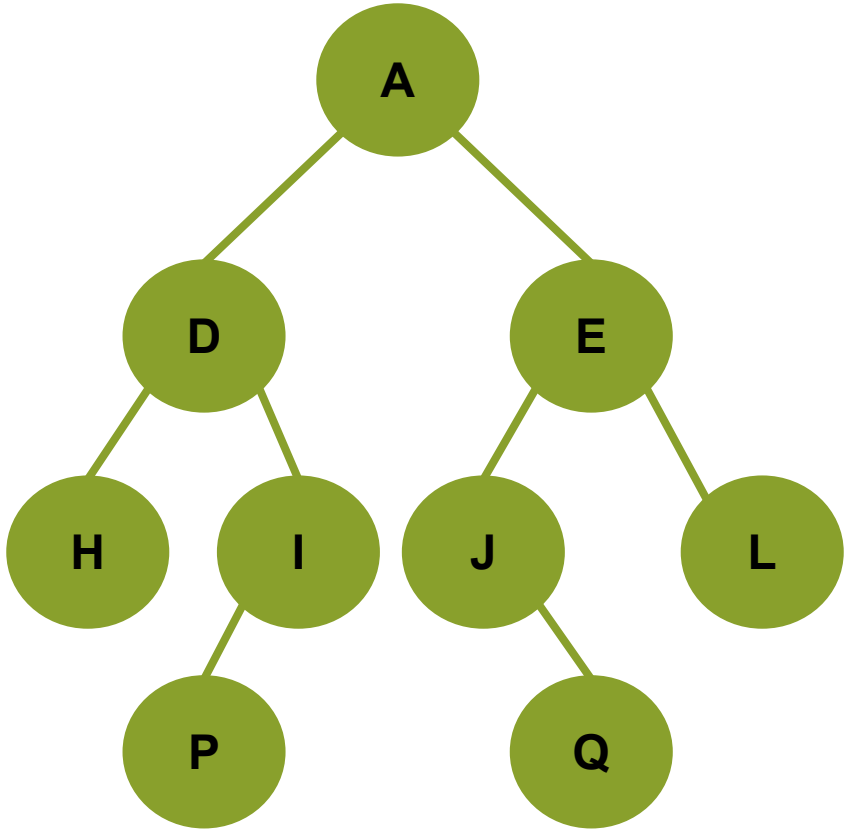

ROOT
A
INTERNAL NODES
INTERNAL NODES
B
C
D
E
F
G
LEAF
LEAF
H
I
J
L
M
N
O
LEAF
LEAF
LEAVES
P
Q
LEAVES

binary tree

- a tree in which no node can have more than two children



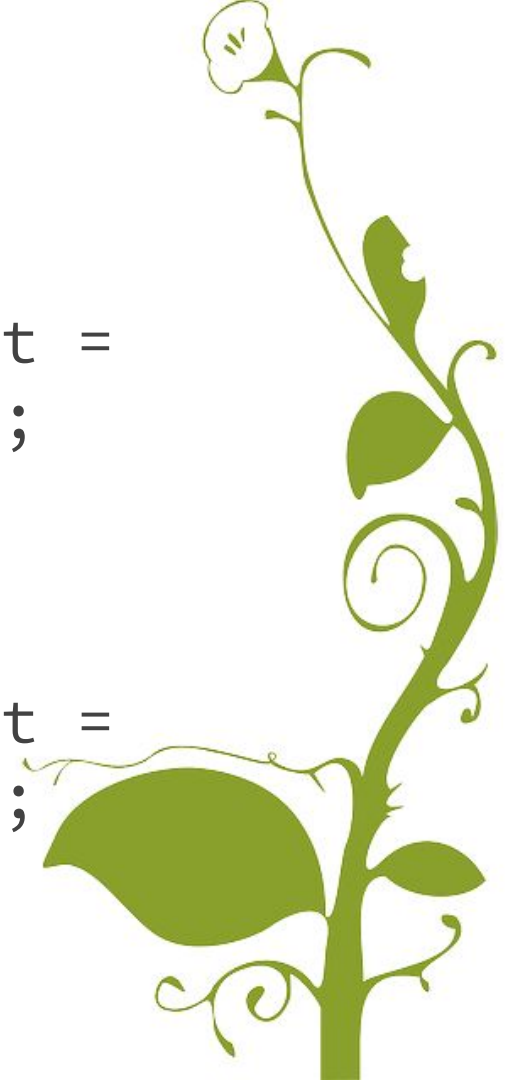
binary tree



binary tree

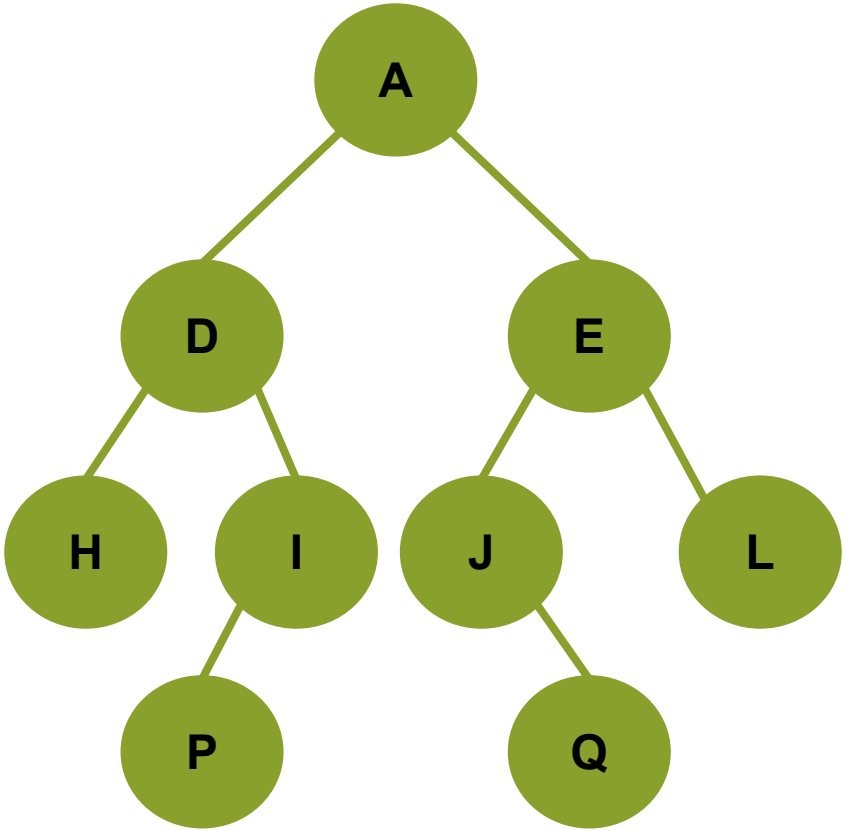
```
template <class T>
class node{
public:
   T item;
   node *left;
   node *right;
   node *prnt;
```

```
   node(){
      item = 0;
      left = right =
      prnt = NULL;
   }
   node(T x){
      item = x;
      left = right =
      prnt = NULL;
   }
};
```



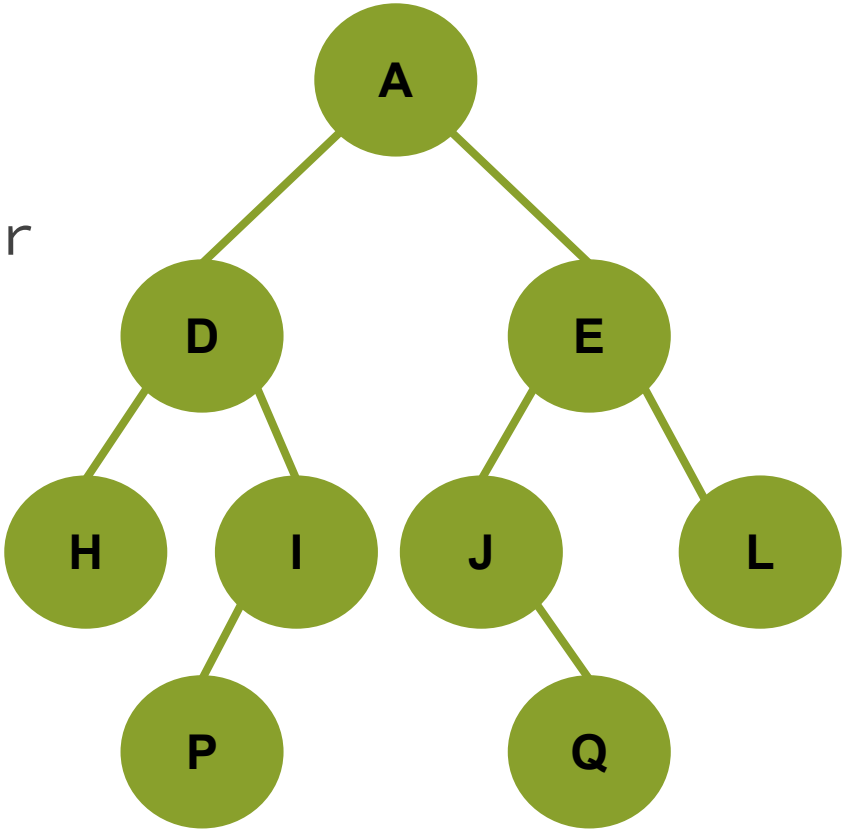
binary tree

- traversal



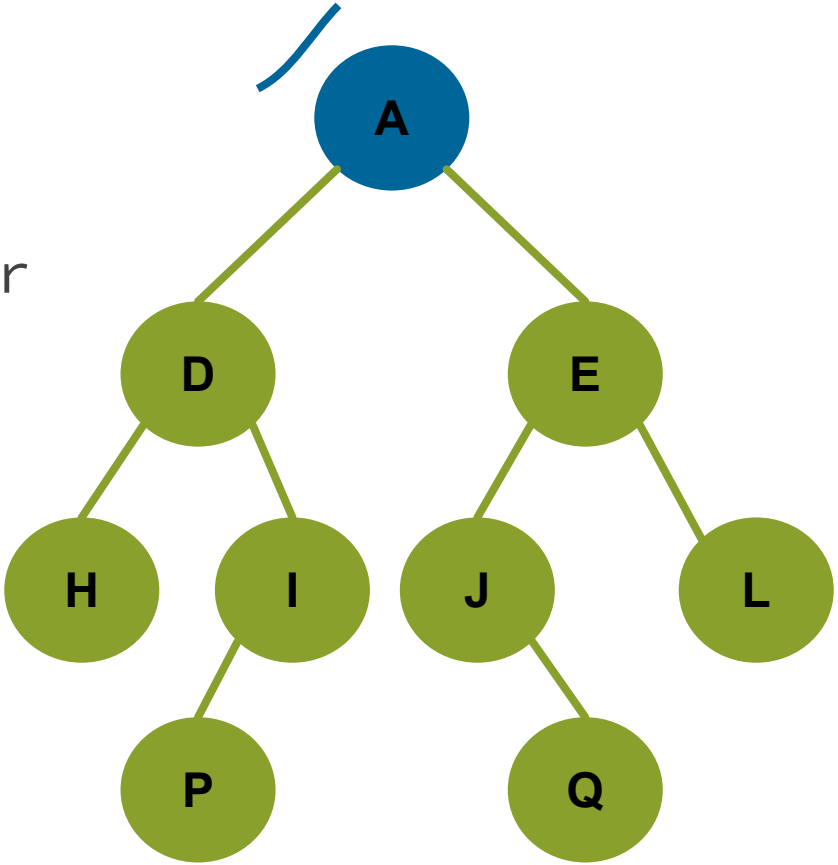
binary tree

- traversal
  - preorder



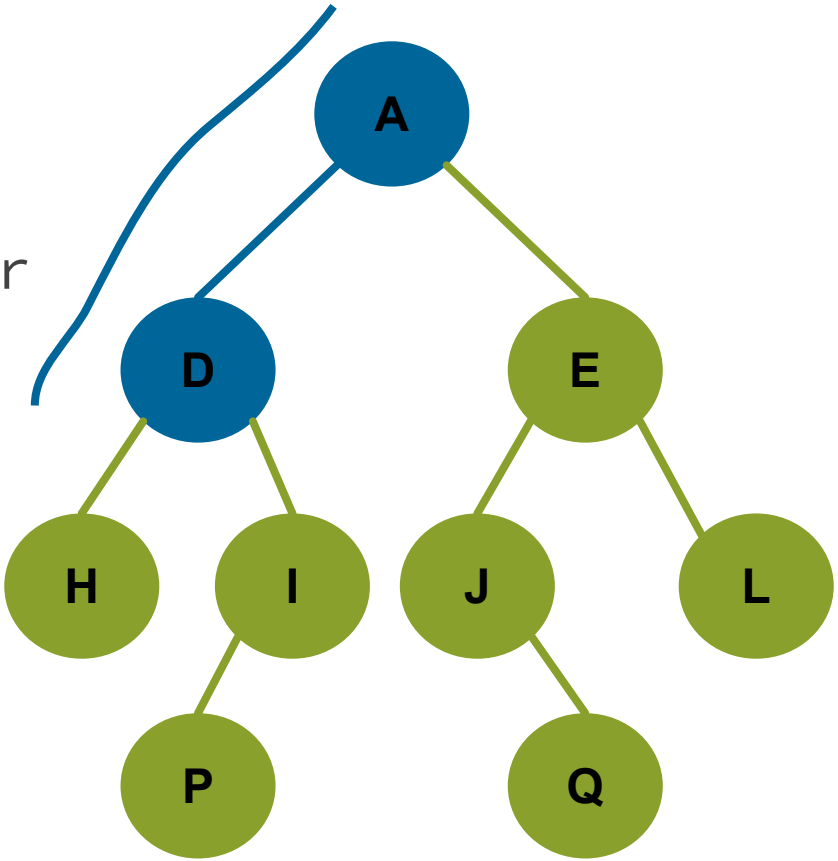
binary tree

- traversal
  - preorder



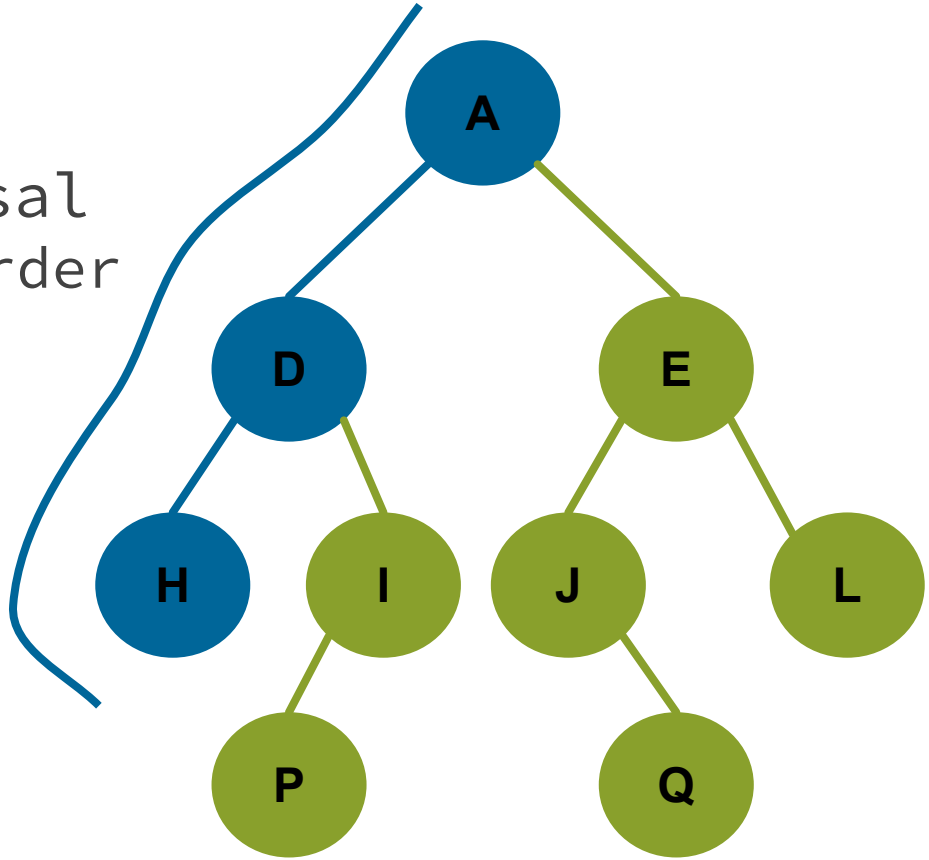
binary tree

- traversal
  - preorder



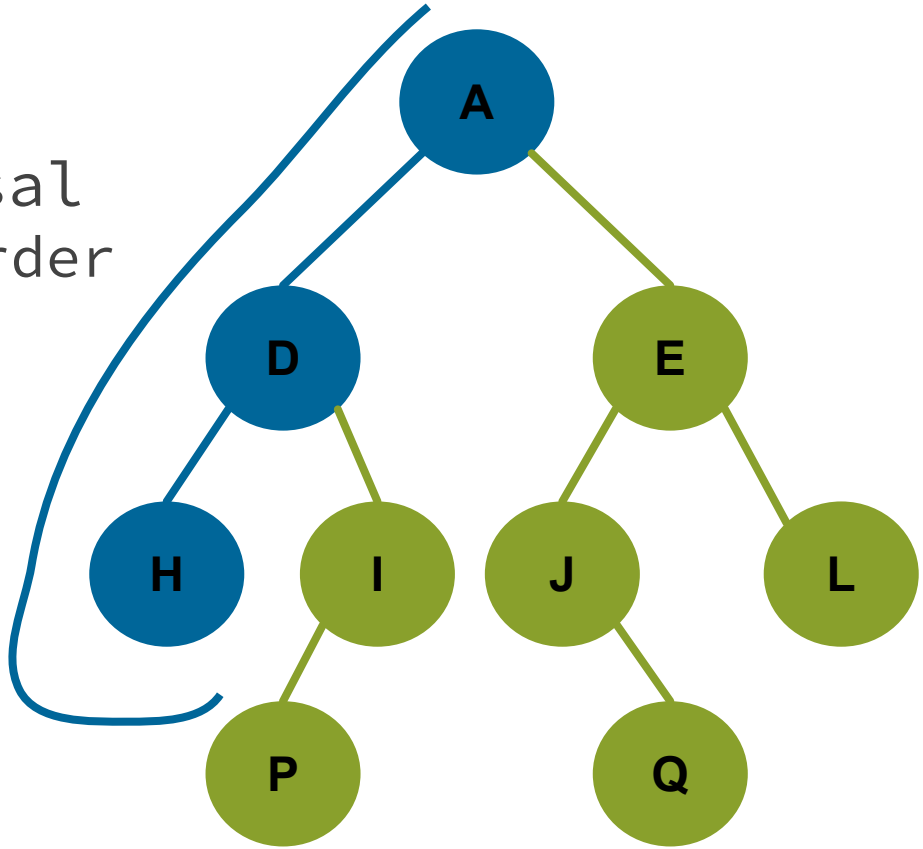
binary tree

- traversal
  - preorder



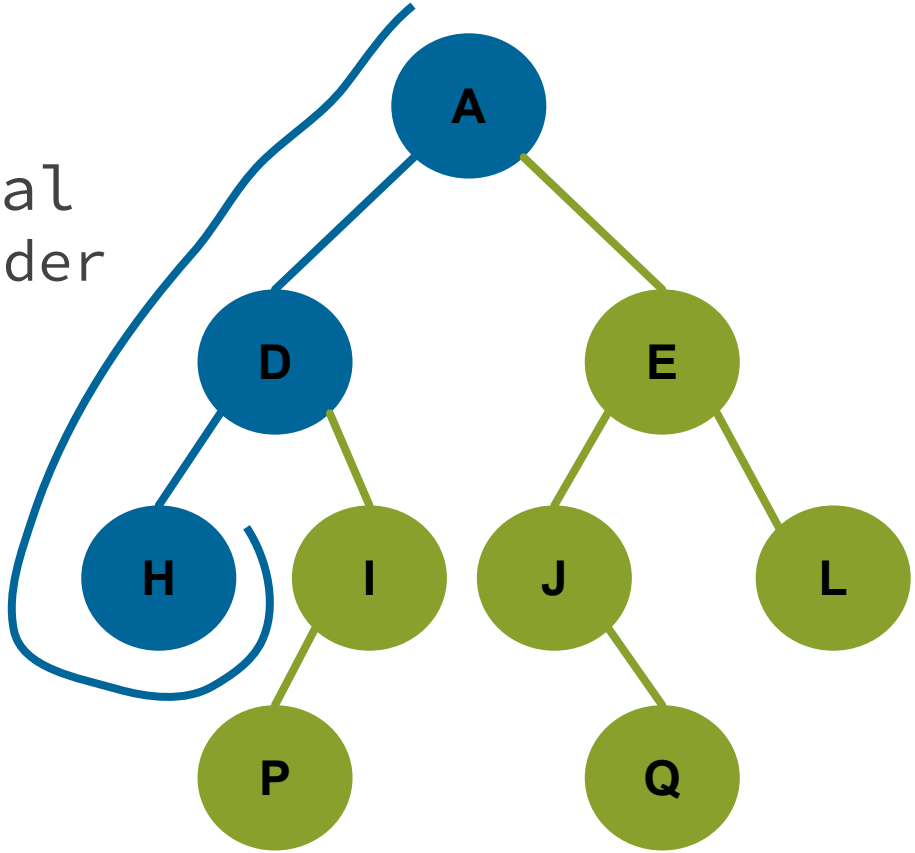
binary tree

- traversal
  - preorder



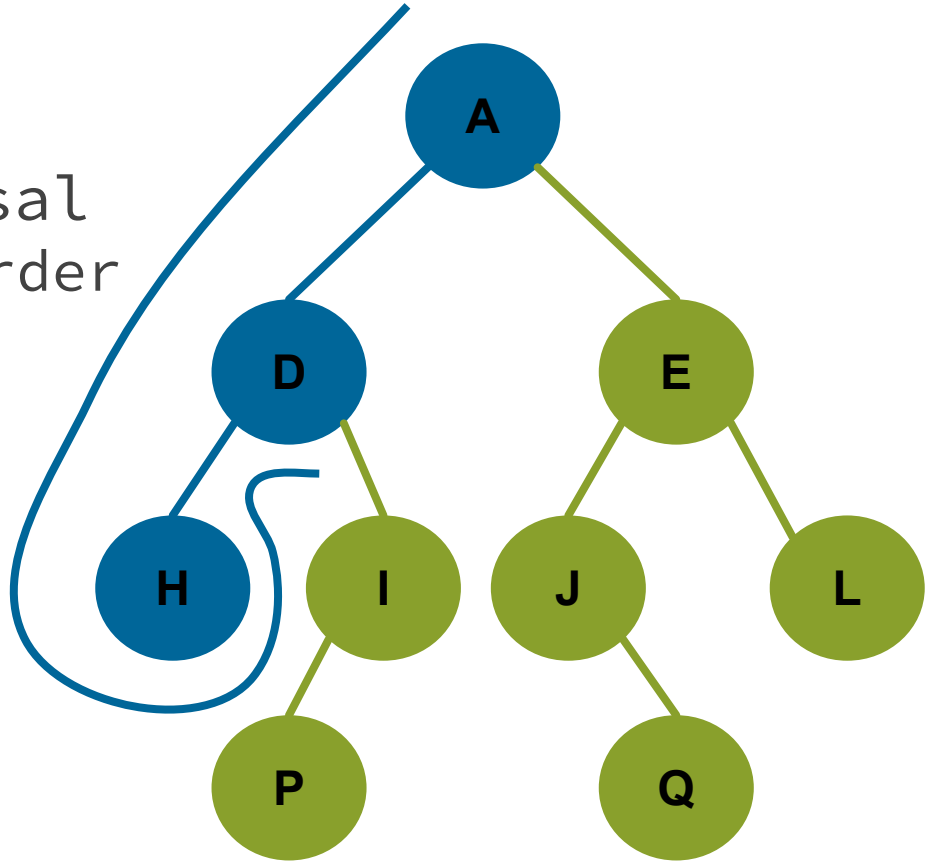
binary tree

- traversal
  - preorder



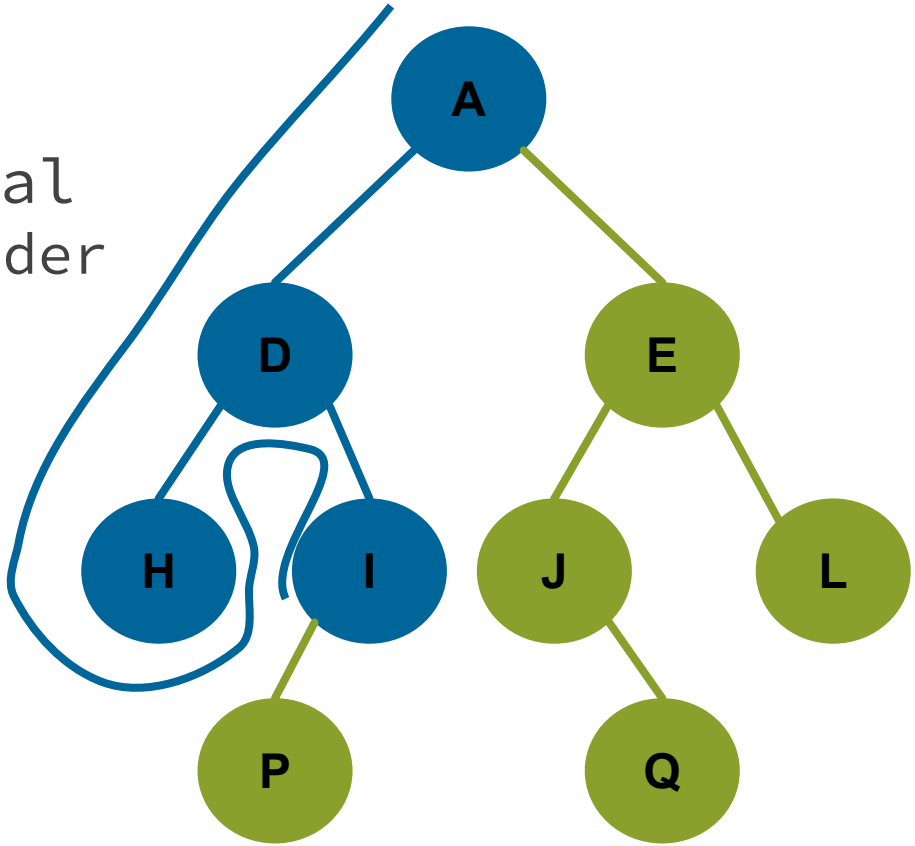
binary tree

- traversal
  - preorder



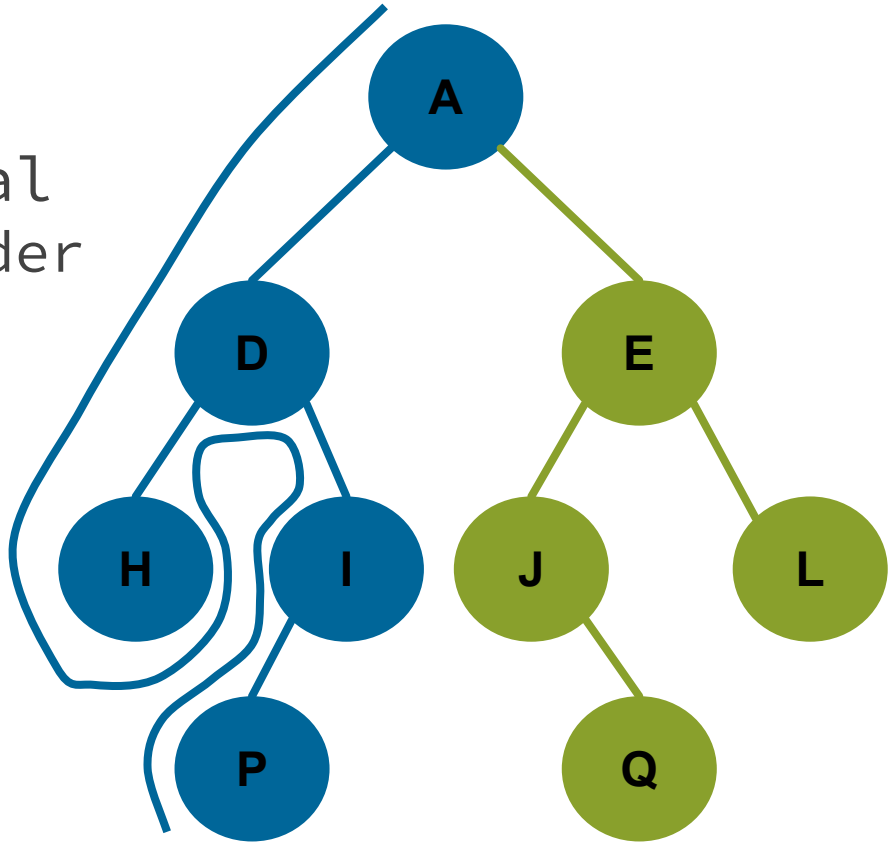
binary tree

- traversal
  - preorder



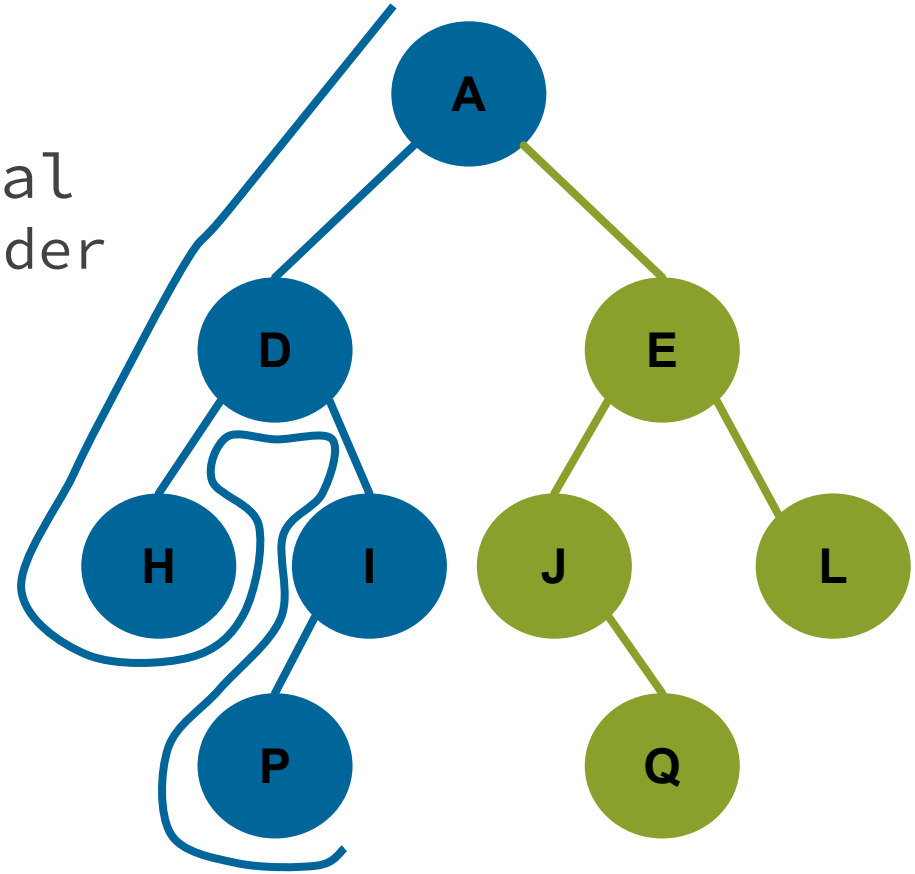
binary tree

- traversal
  - preorder



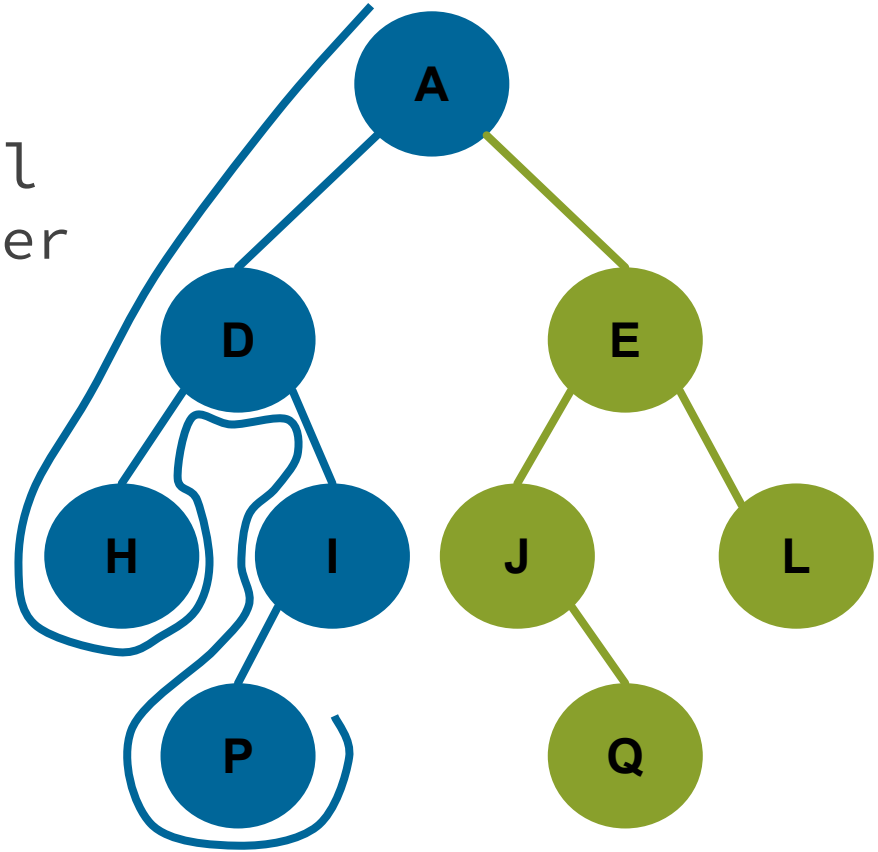
binary tree

- traversal
  - preorder



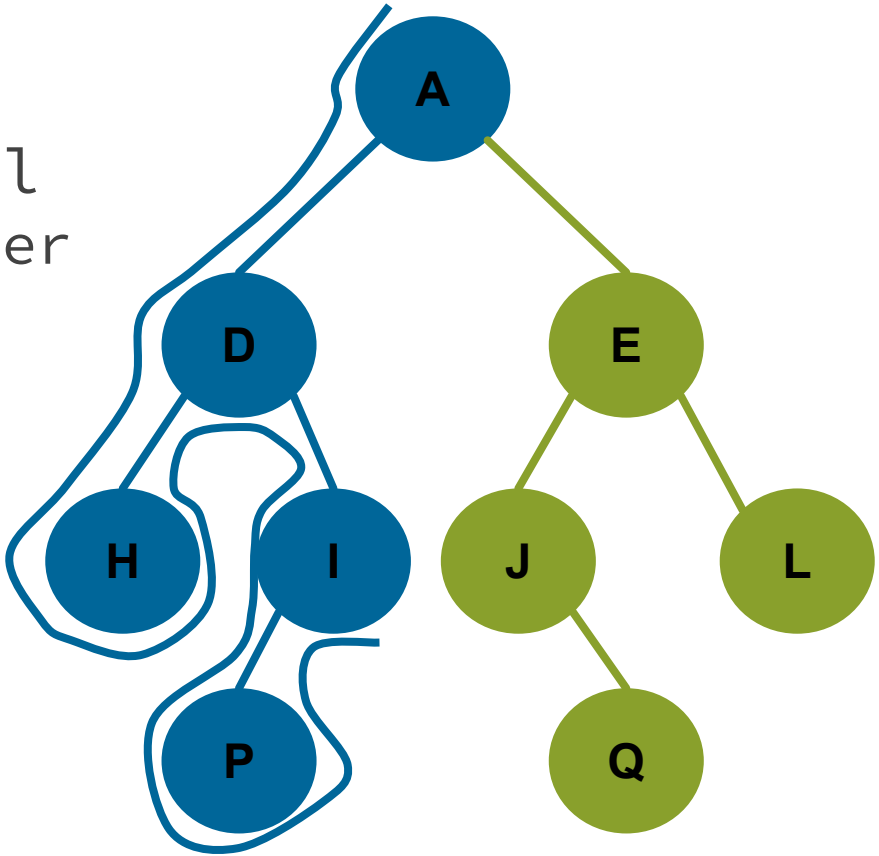
binary tree

- traversal
  - preorder



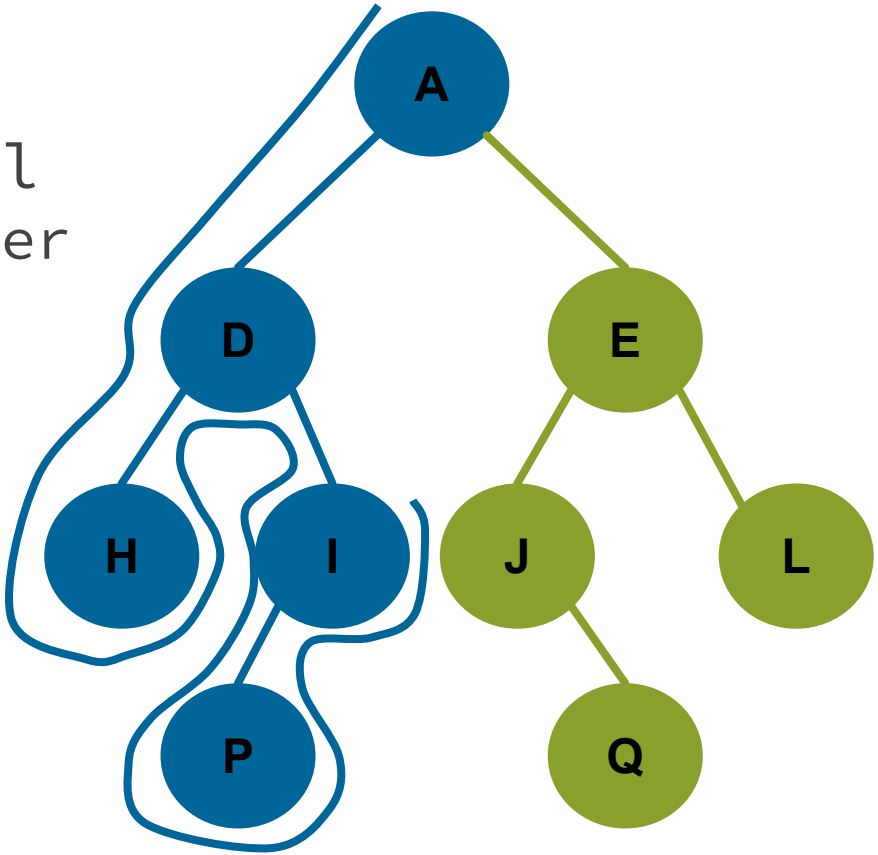
binary tree

- traversal
  - preorder



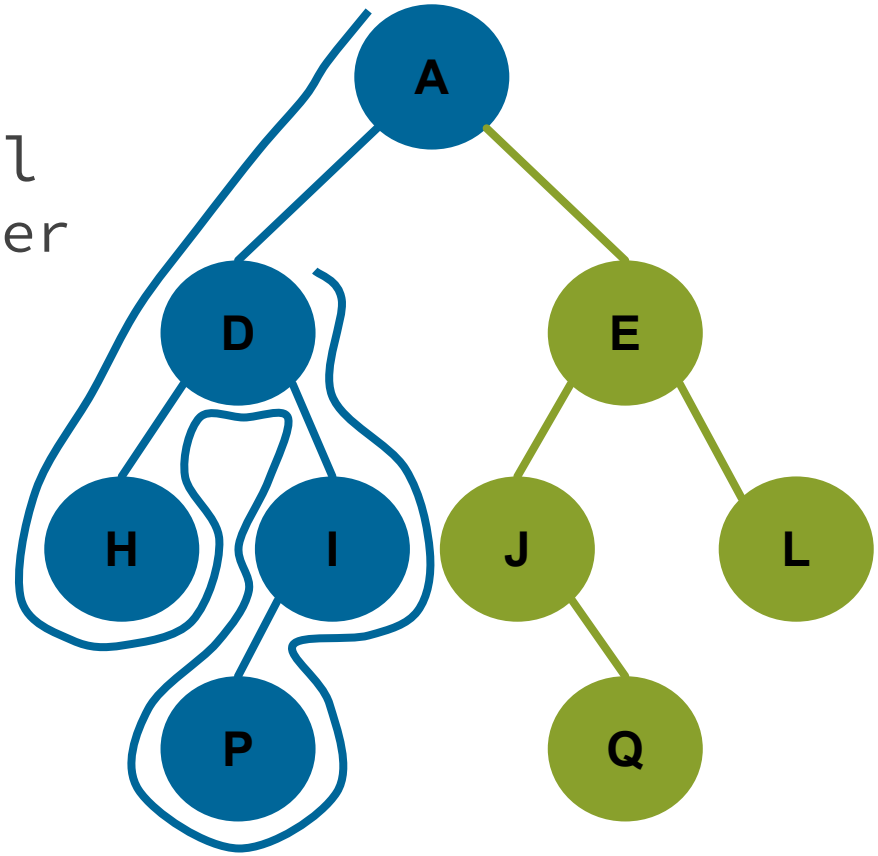
binary tree

- traversal
  - preorder



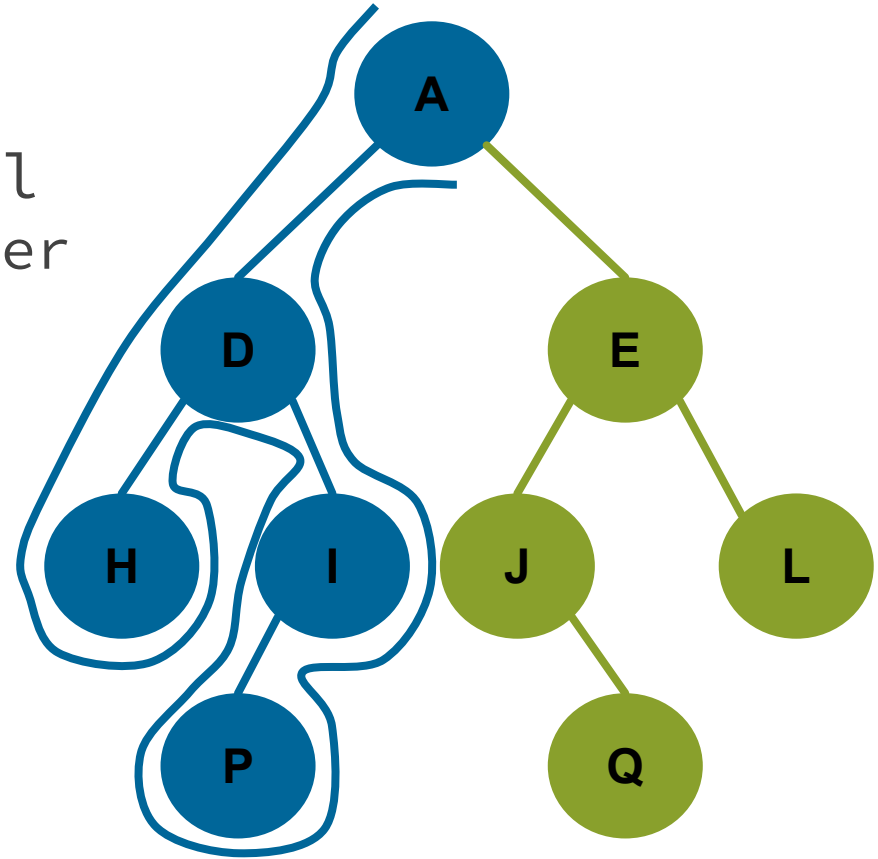
binary tree

- traversal
  - preorder



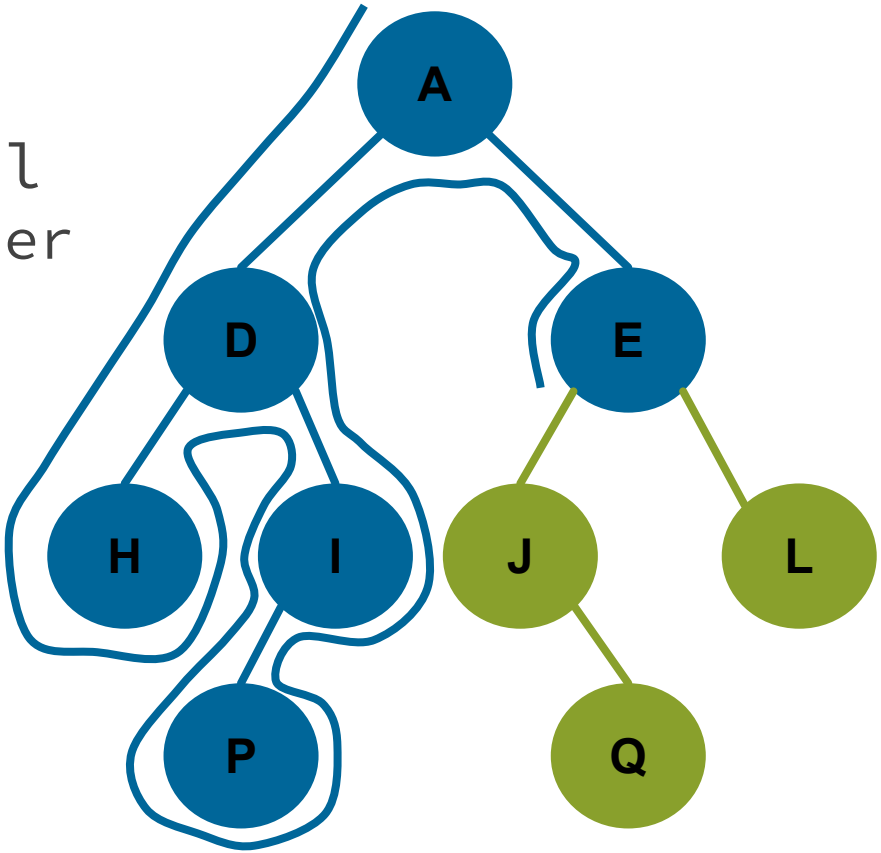
binary tree

- traversal
  - preorder



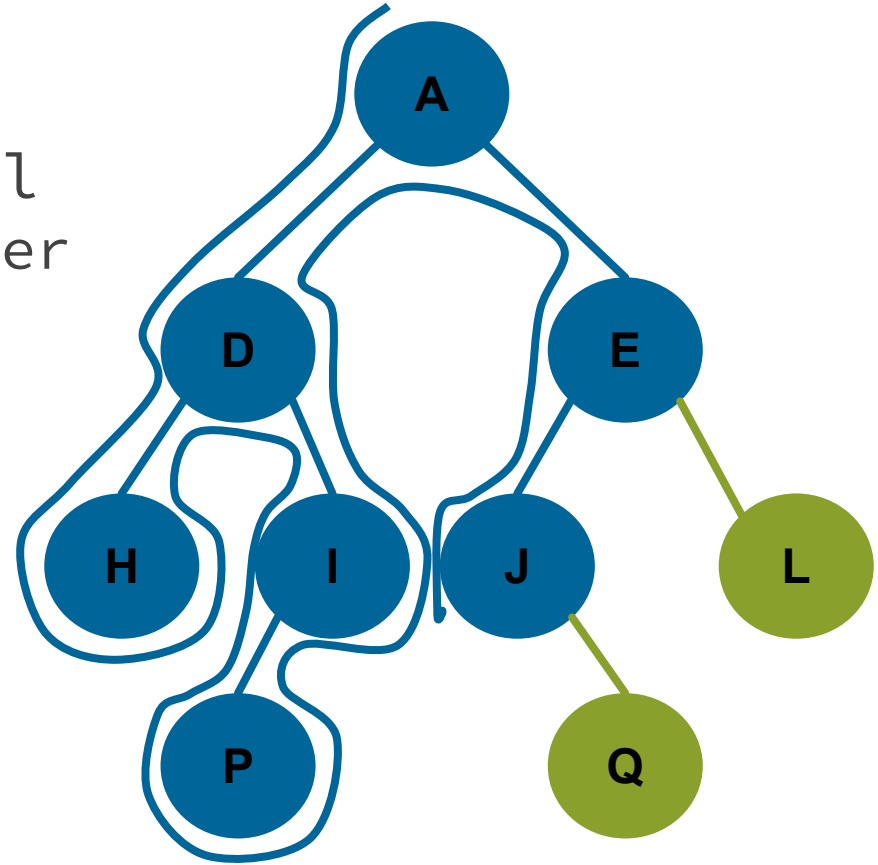
binary tree

- traversal
  - preorder



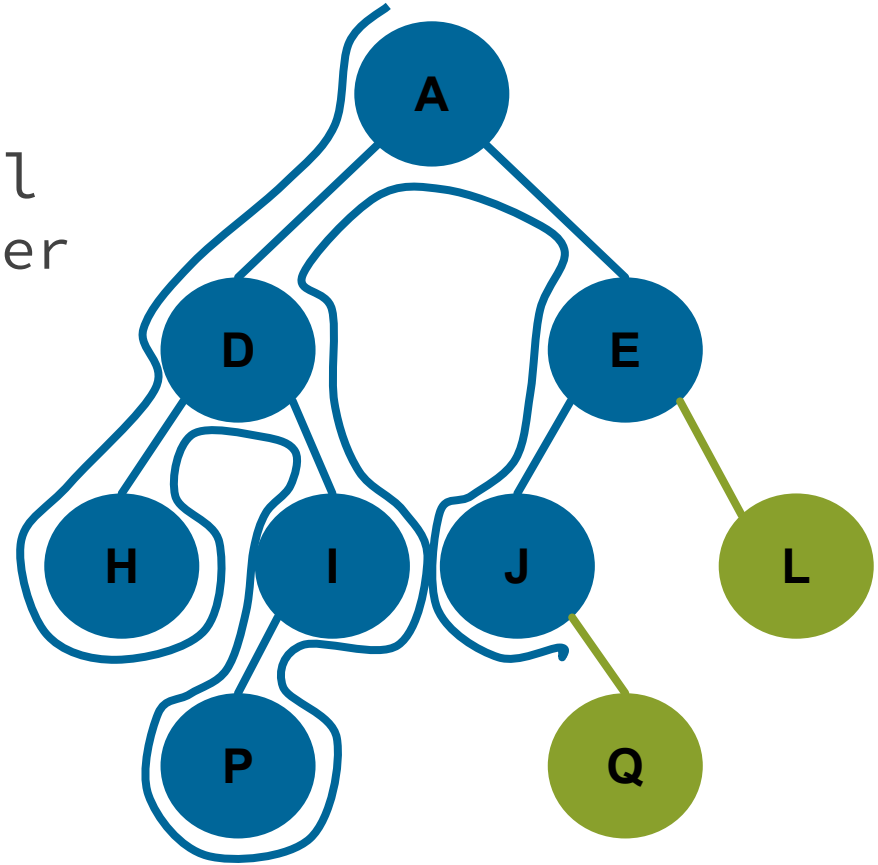
binary tree

- traversal
  - preorder



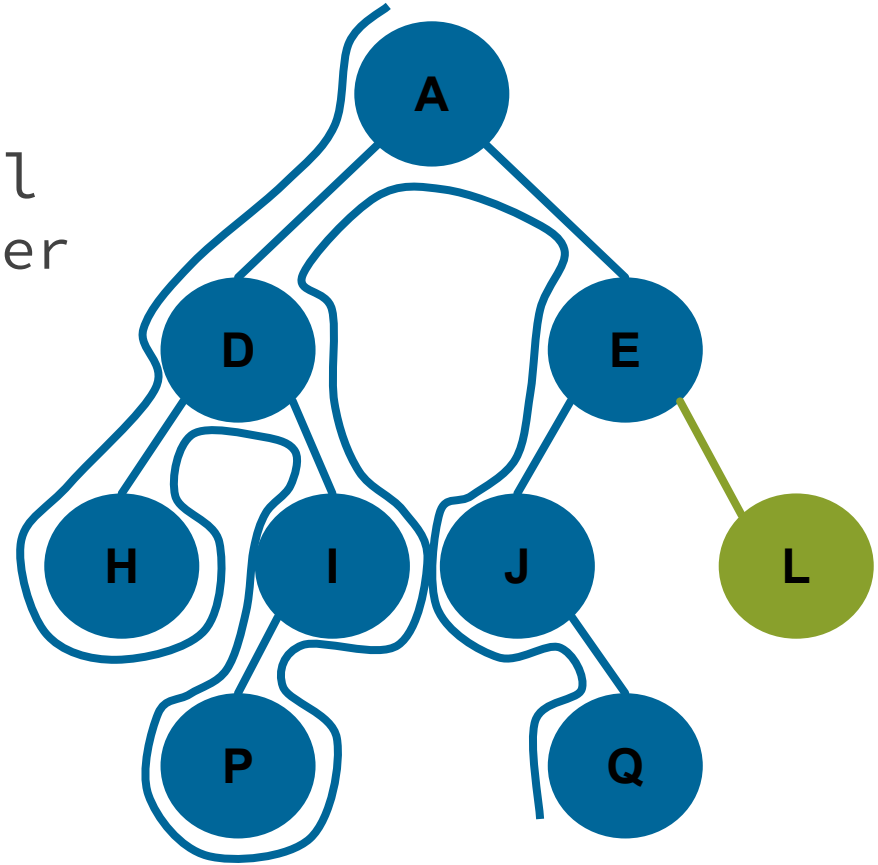
binary tree

- traversal
  - preorder



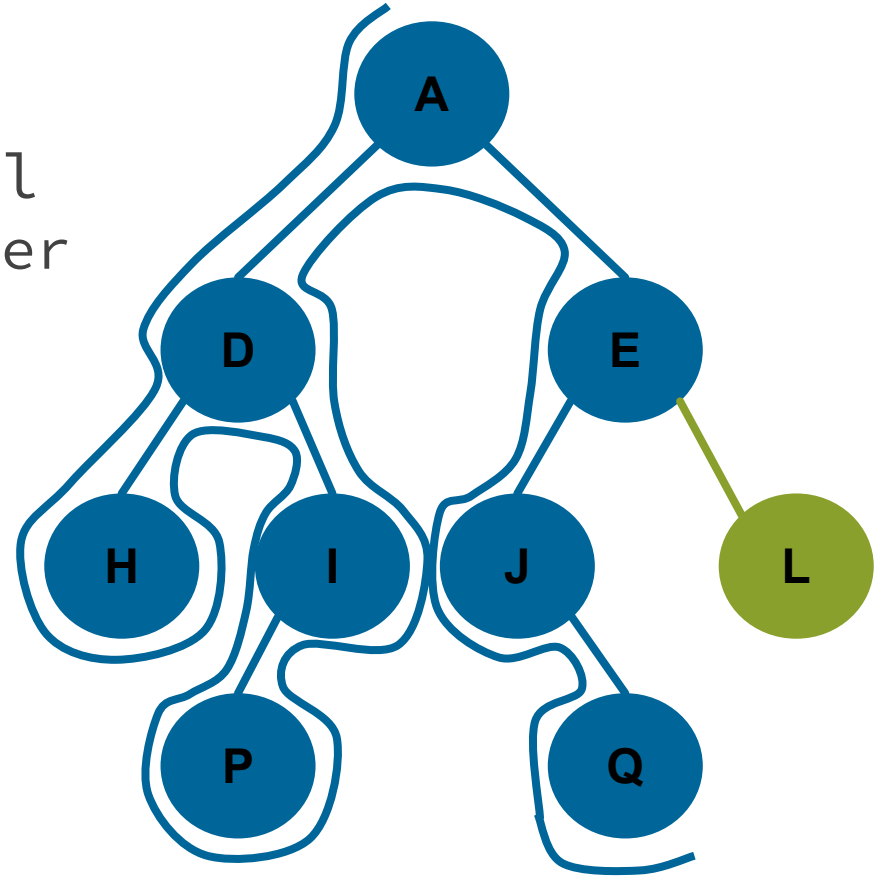
binary tree

- traversal
  - preorder



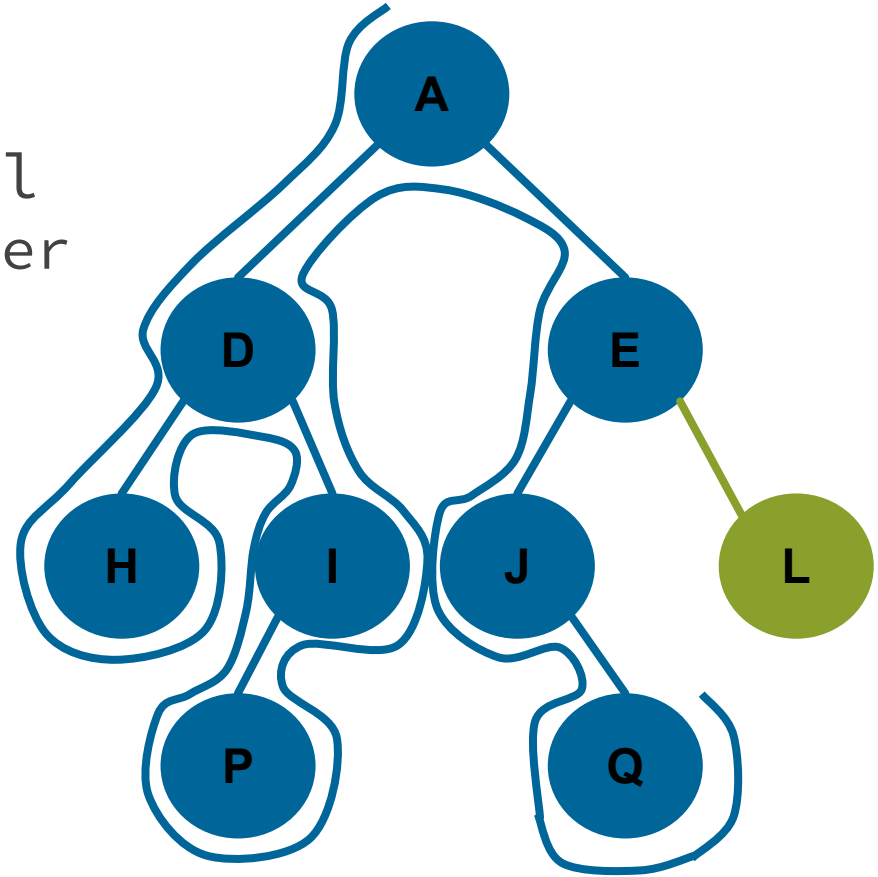
binary tree

- traversal
  - preorder



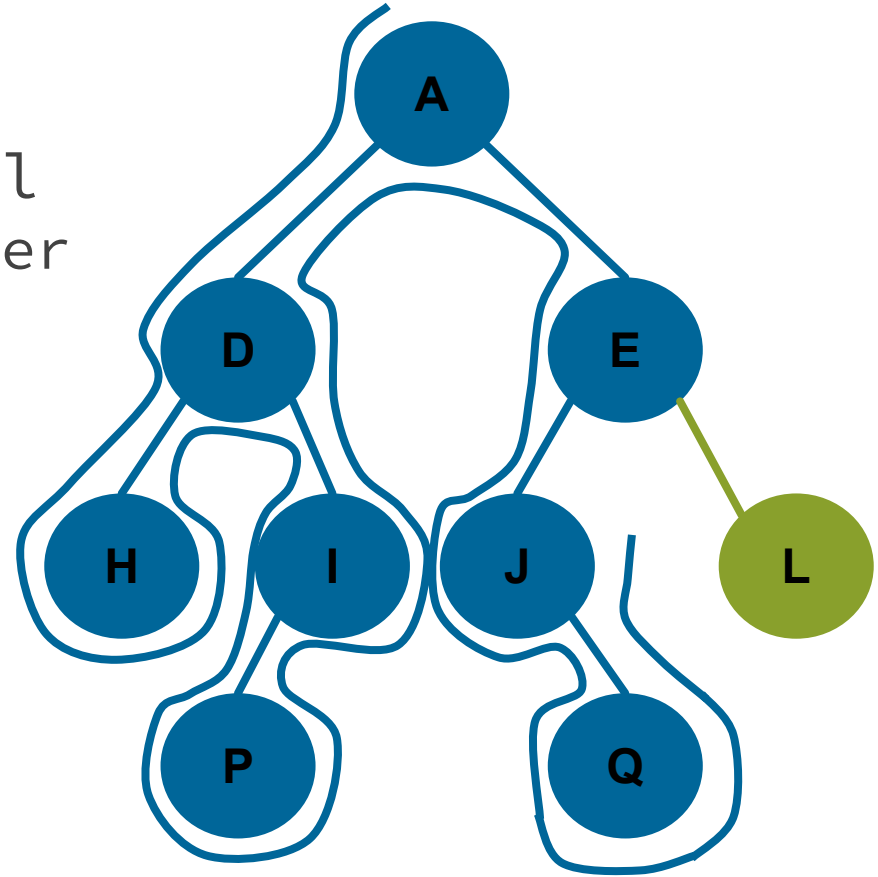
binary tree

- traversal
  - preorder



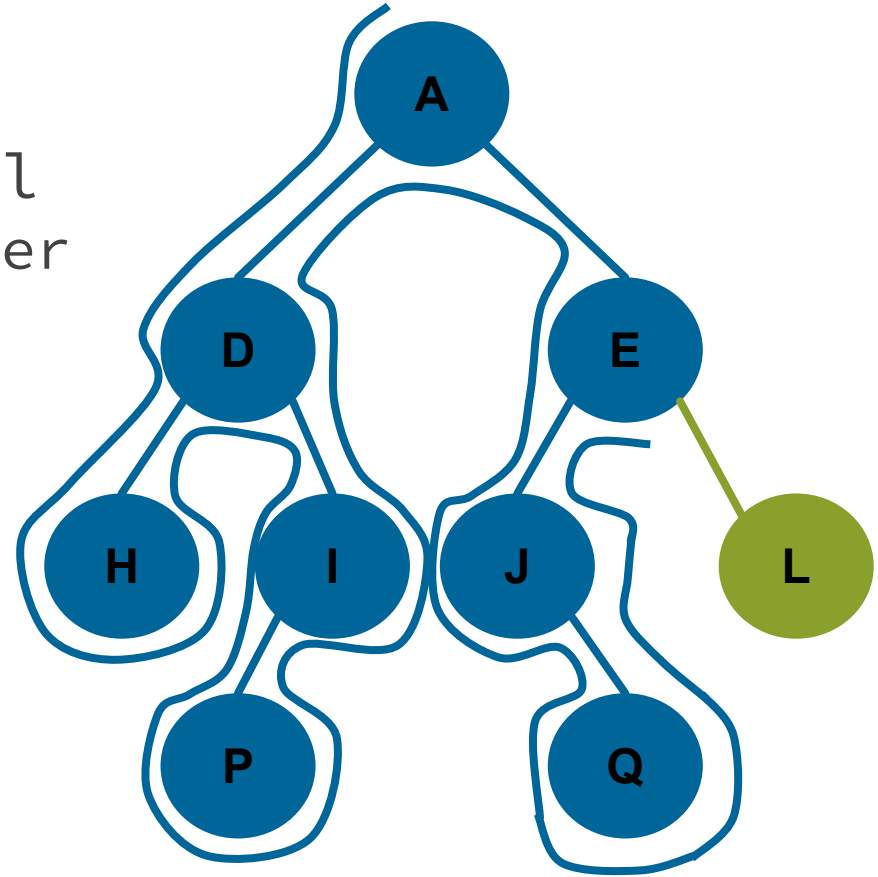
binary tree

- traversal
  - preorder



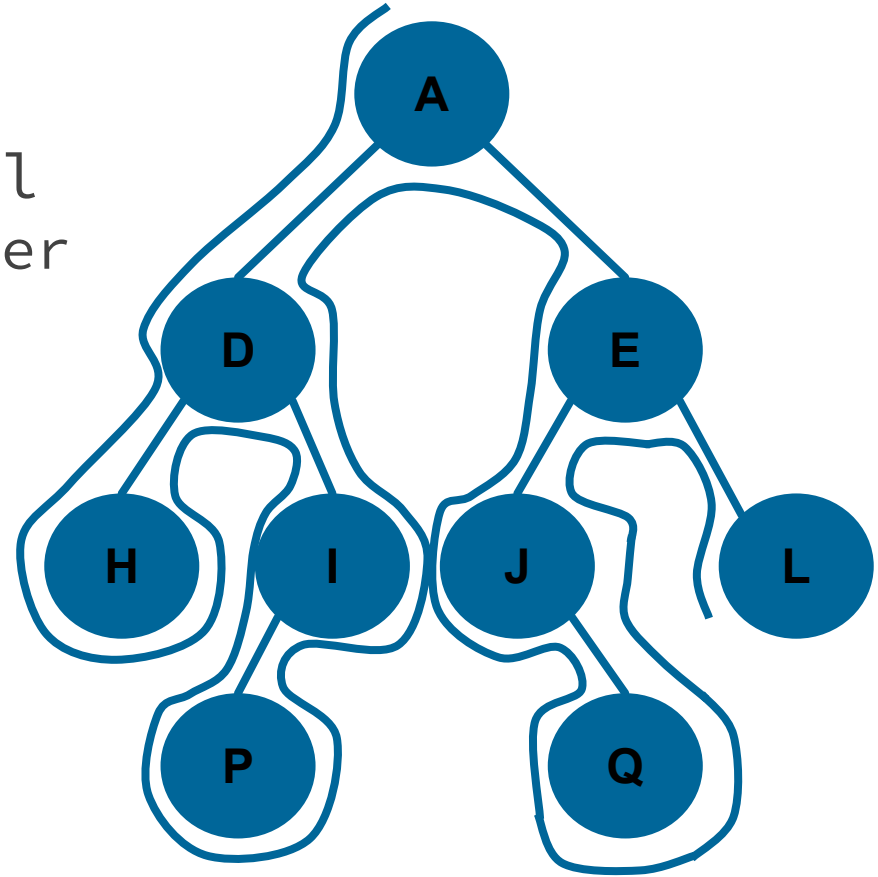
binary tree

- traversal
  - preorder



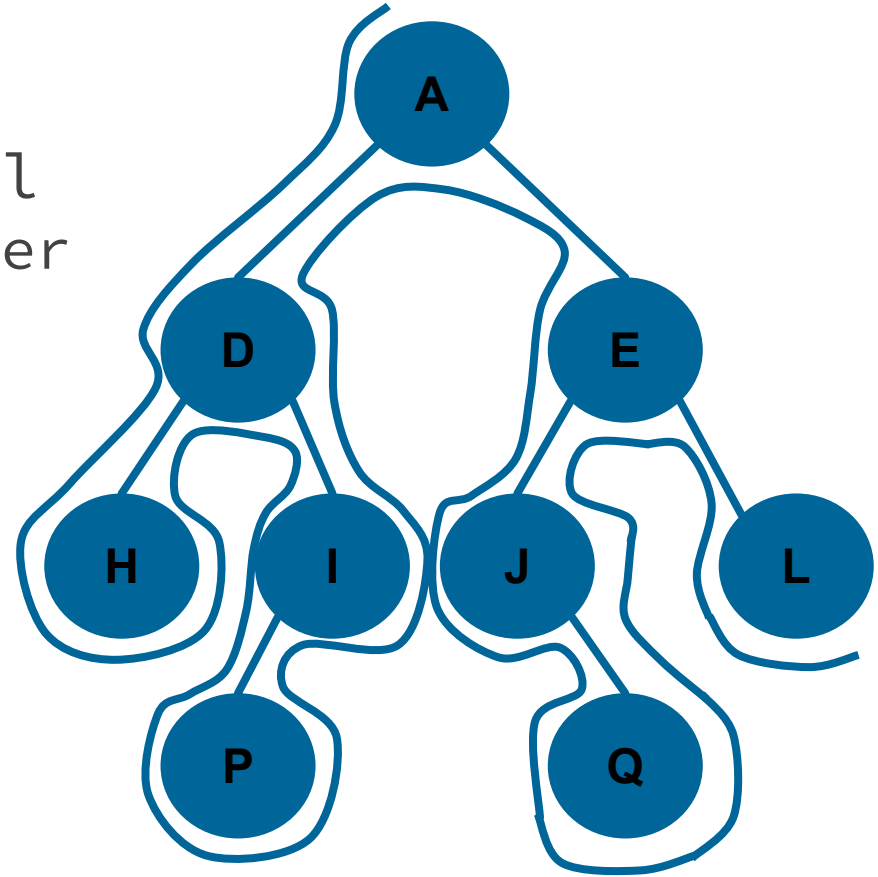
binary tree

- traversal
  - preorder



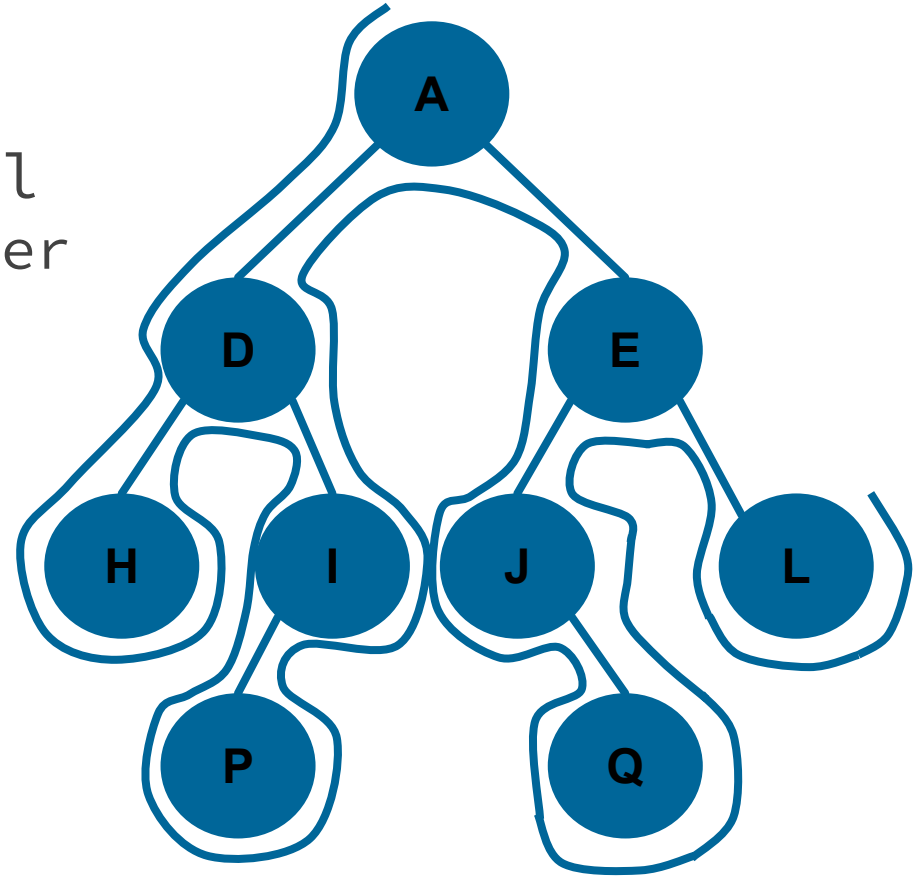
binary tree

- traversal
  - preorder



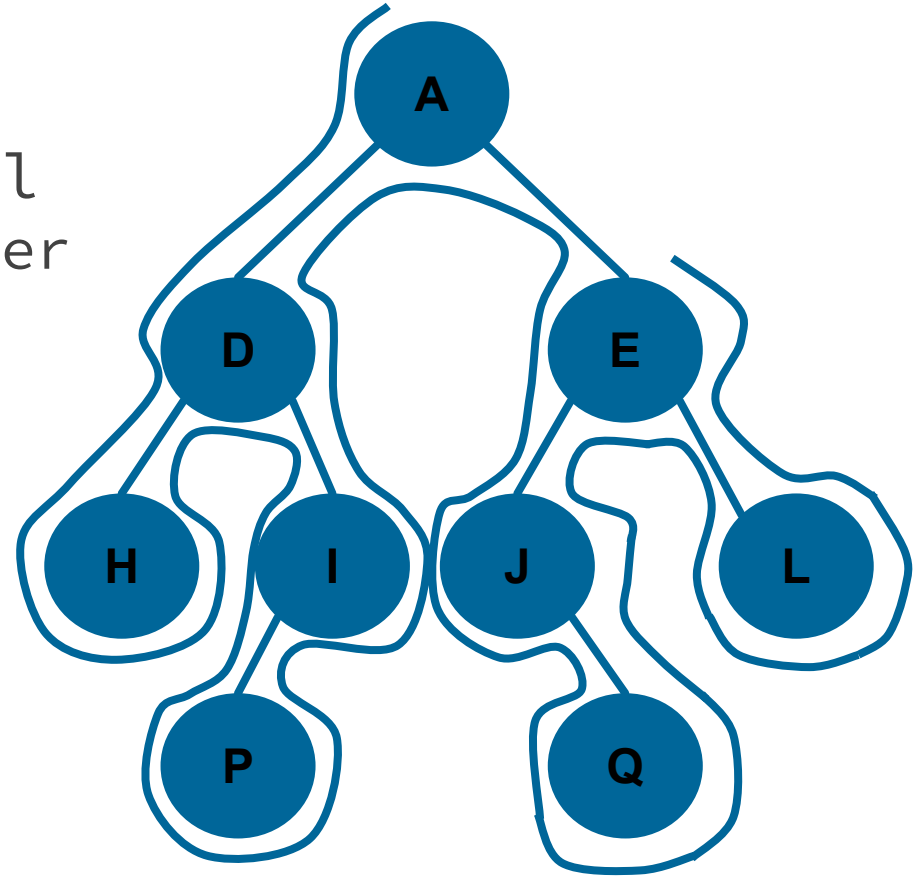
binary tree

- traversal
  - preorder



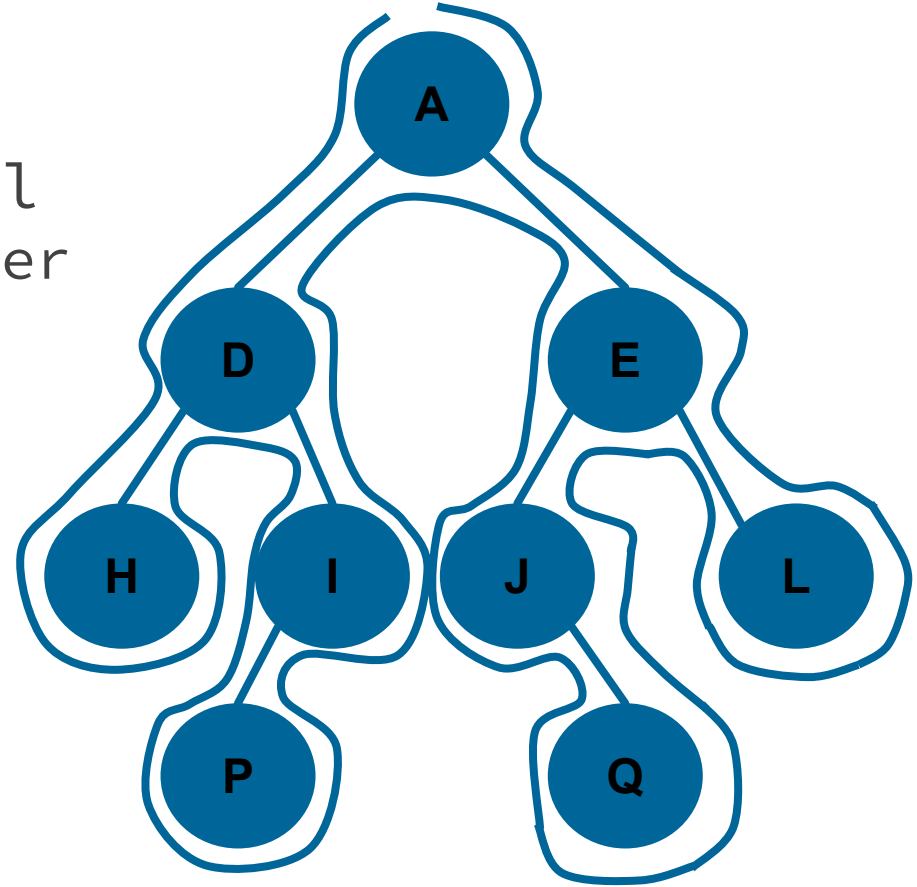
binary tree

- traversal
  - preorder



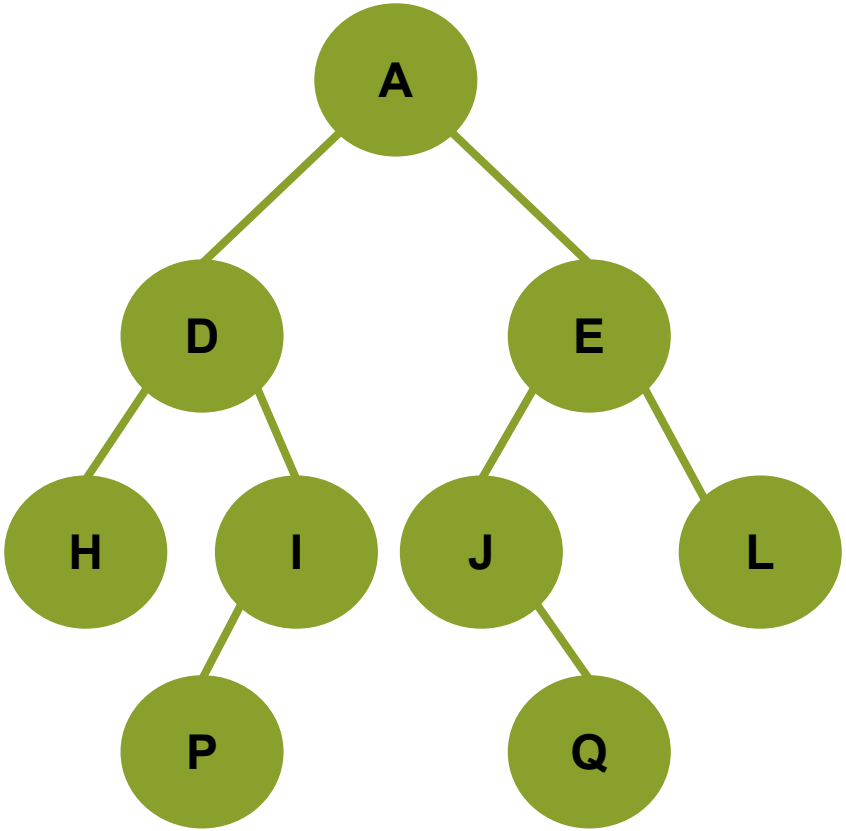
binary tree

- traversal
  - preorder



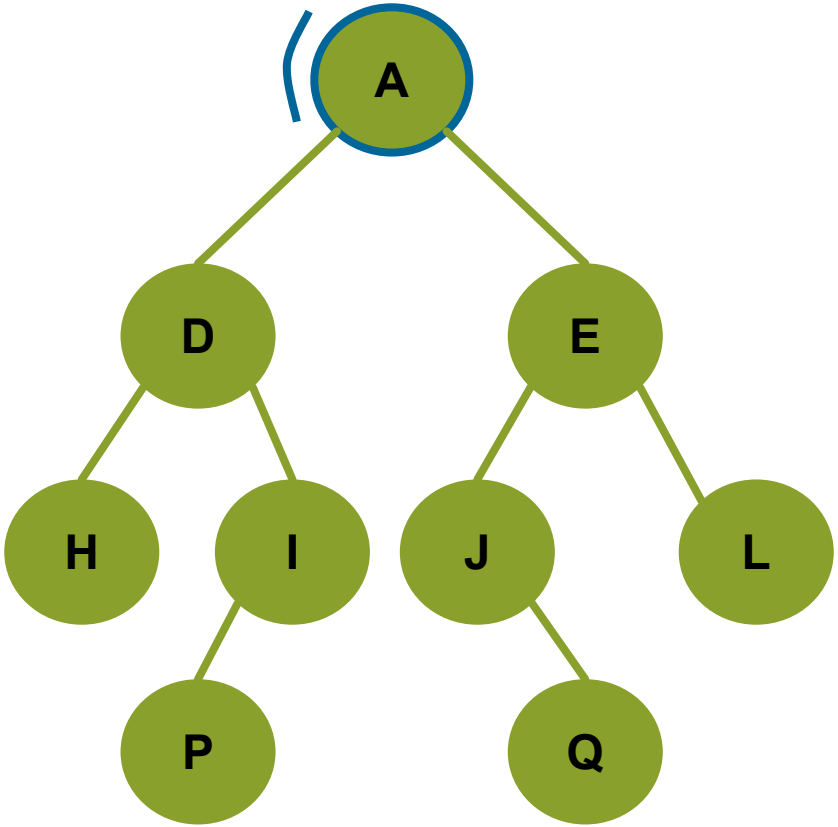
binary tree

- traversal
  - inorder



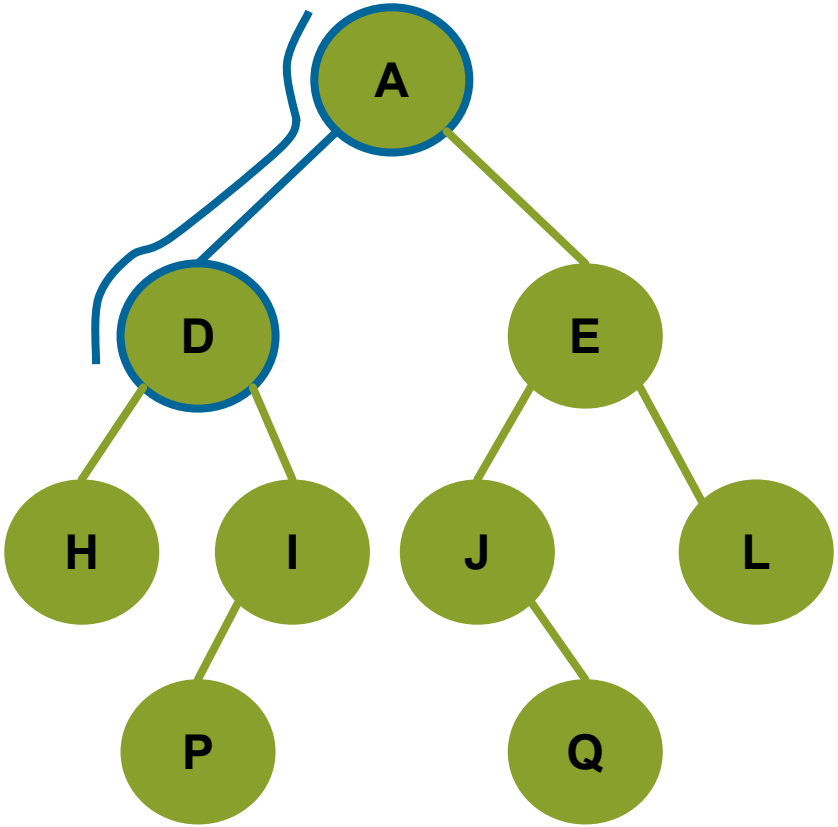
binary tree

- traversal
  - inorder



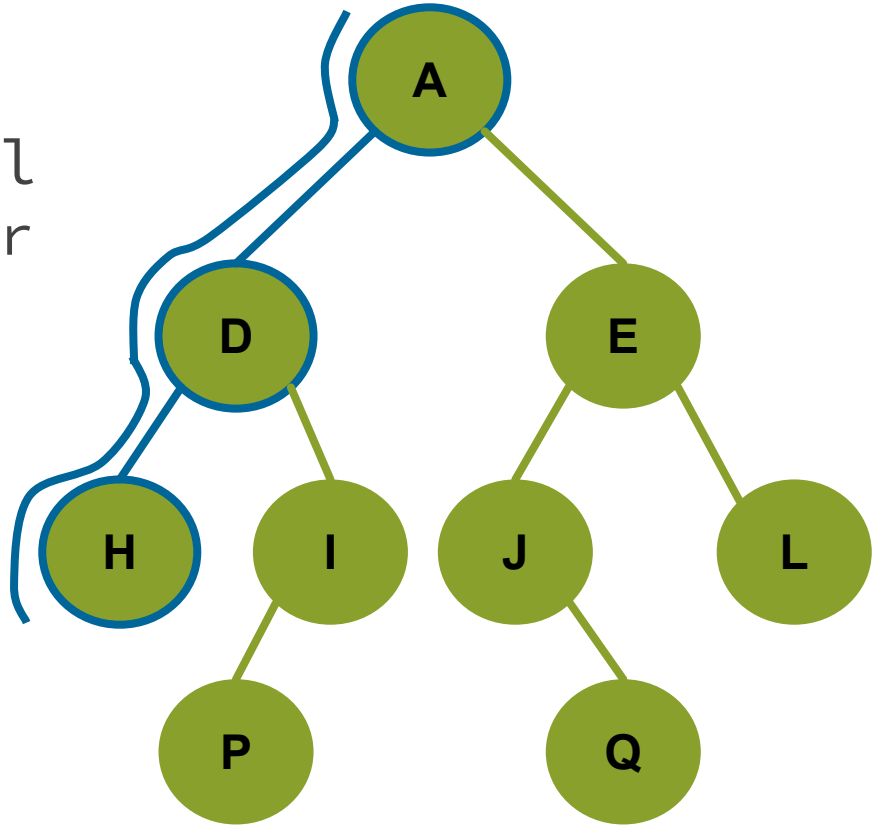
binary tree

- traversal
  - inorder



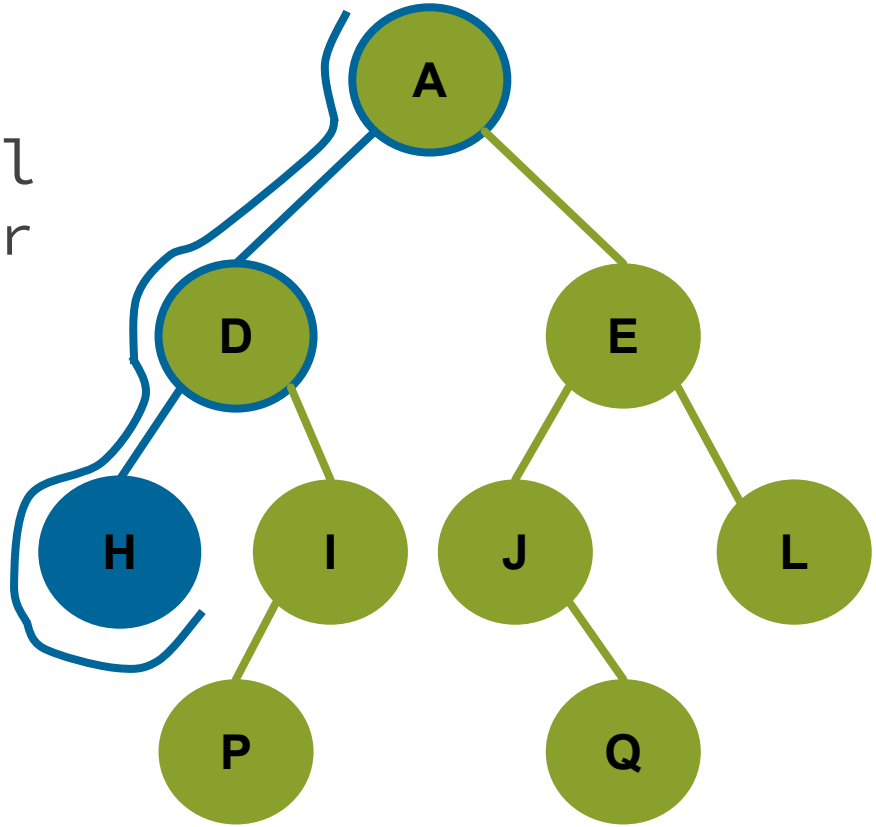
binary tree

- traversal
  - inorder



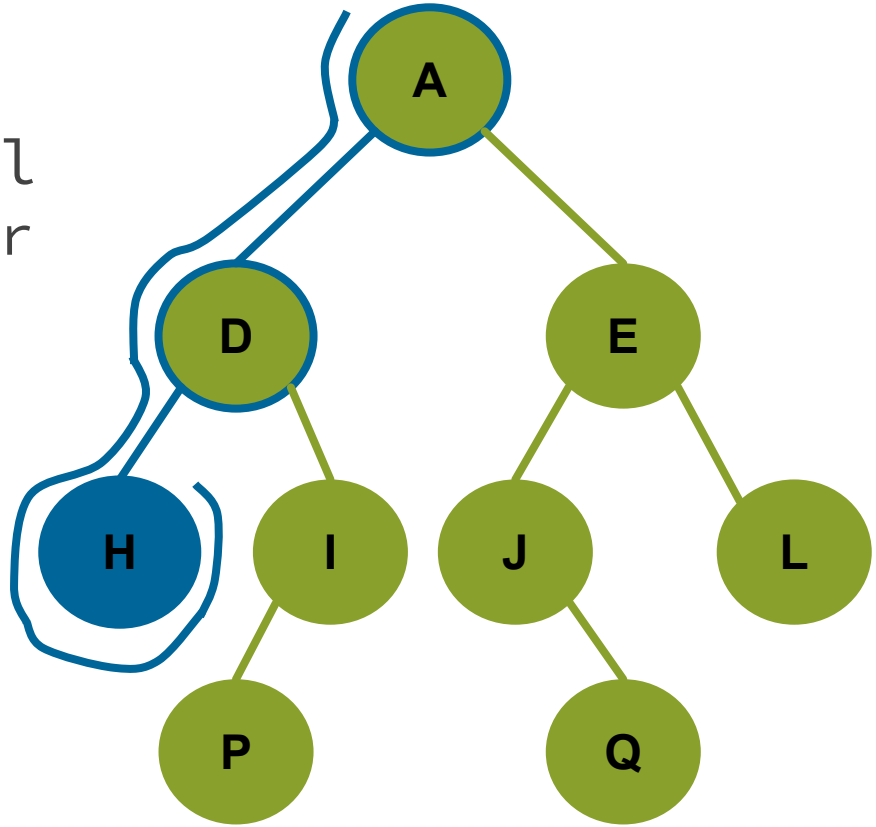
binary tree

- traversal
  - inorder



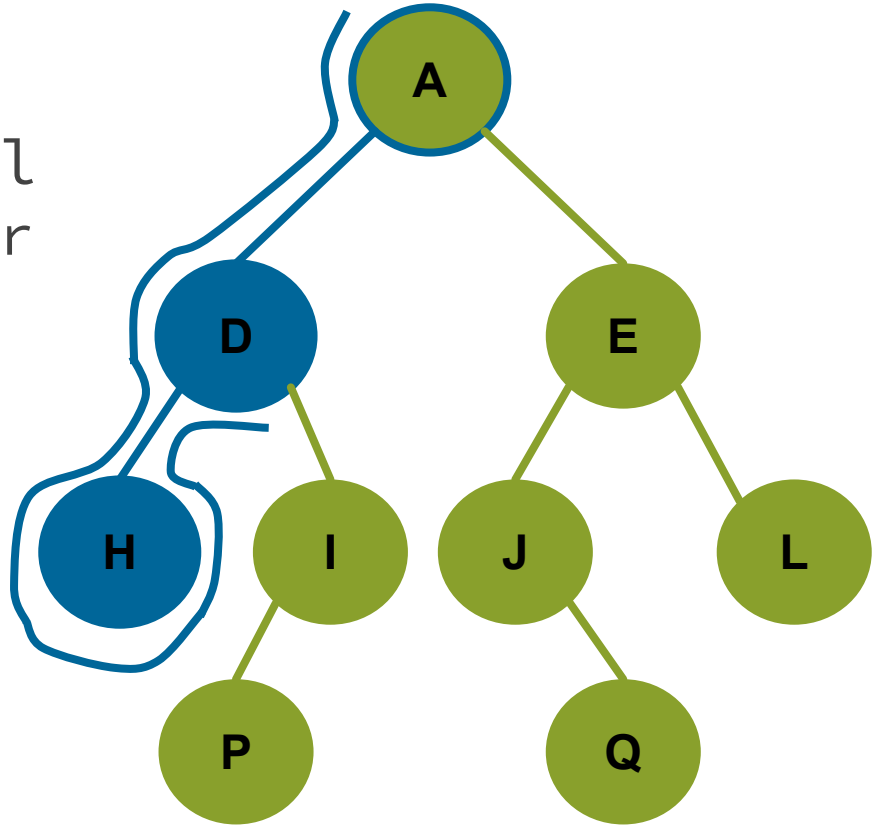
binary tree

- traversal
  - inorder



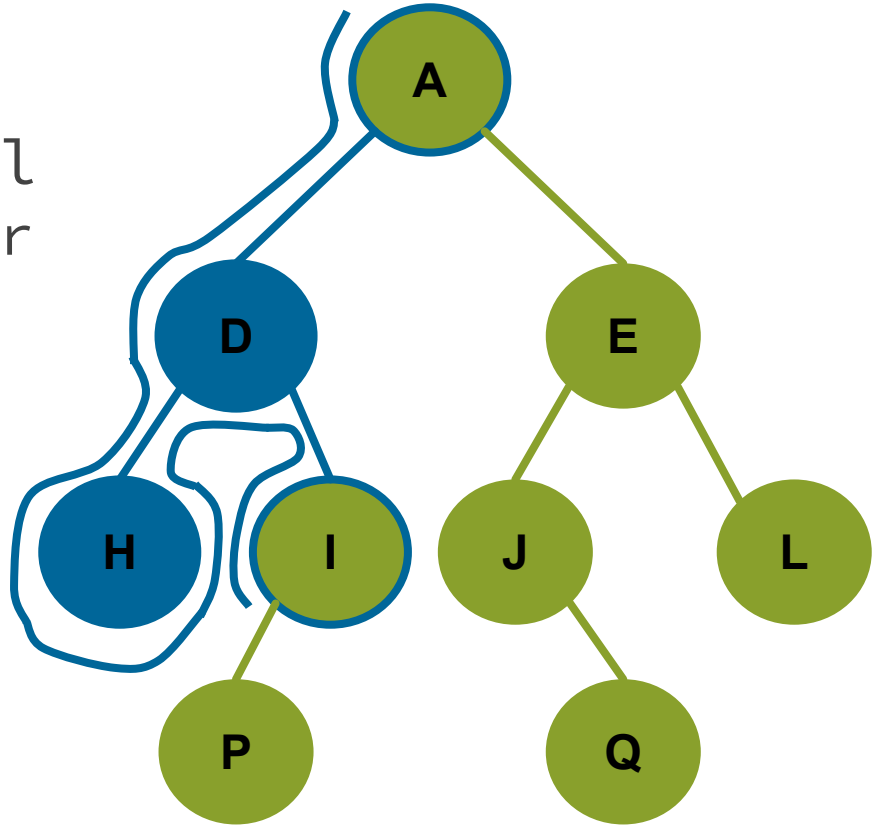
binary tree

- traversal
  - inorder



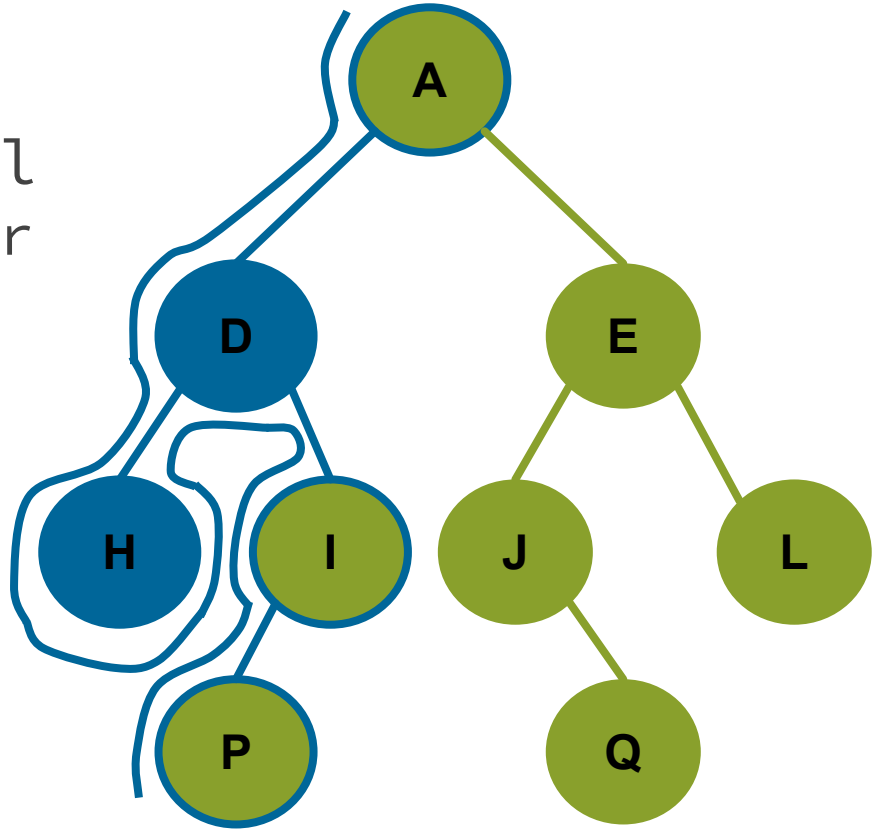
binary tree

- traversal
  - inorder



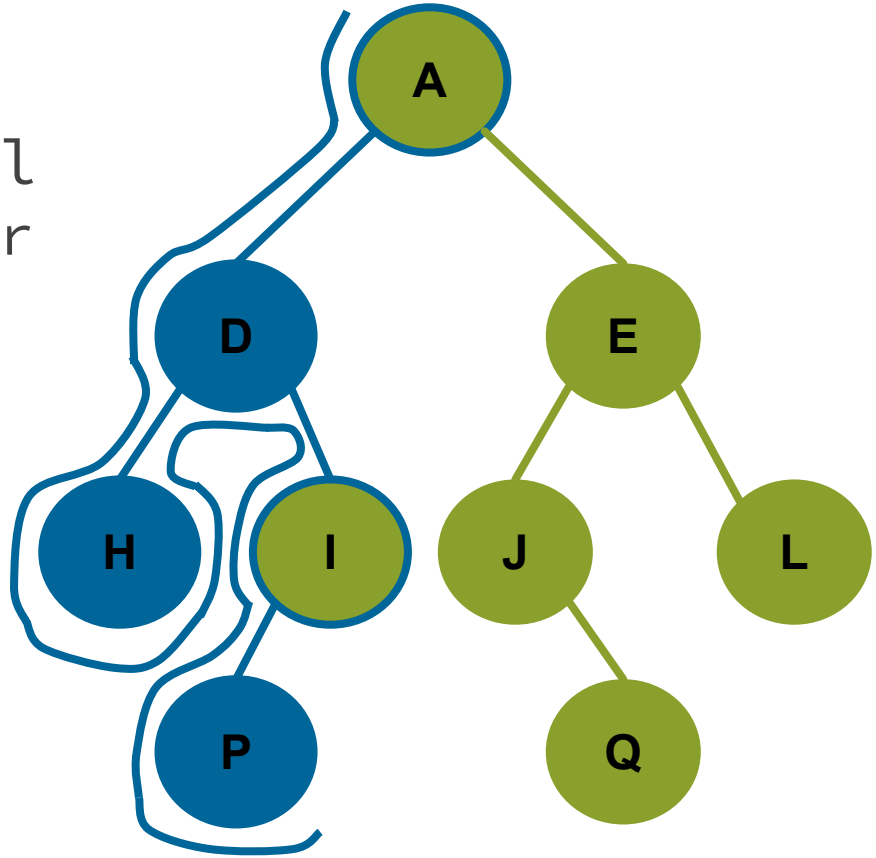
binary tree

- traversal
  - inorder



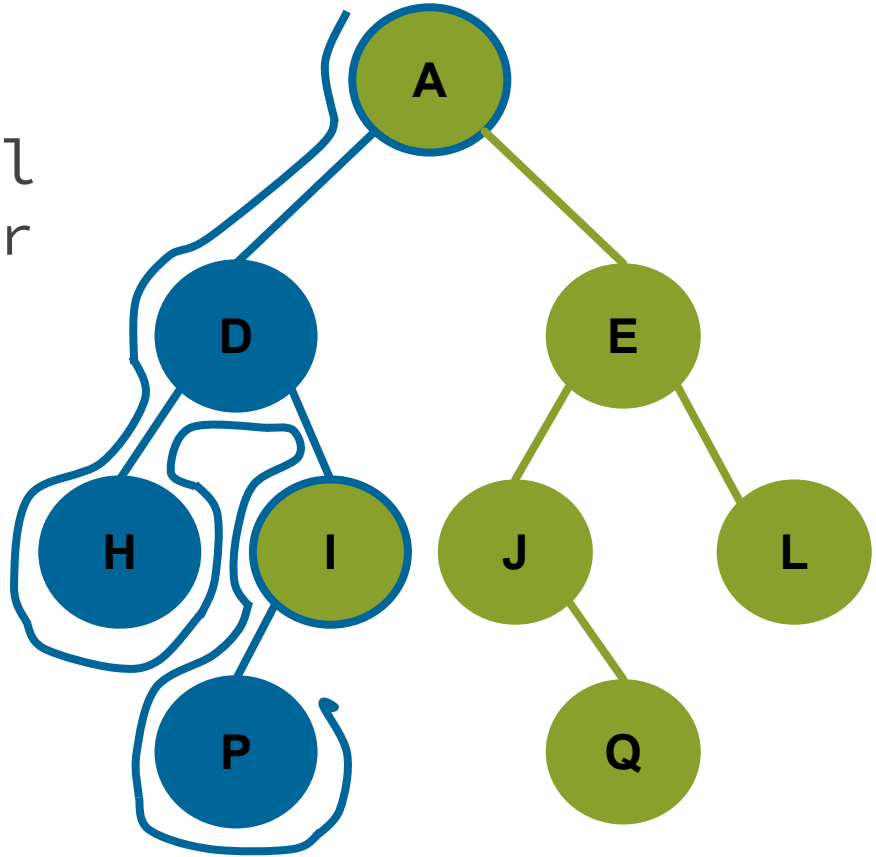
binary tree

- traversal
  - inorder



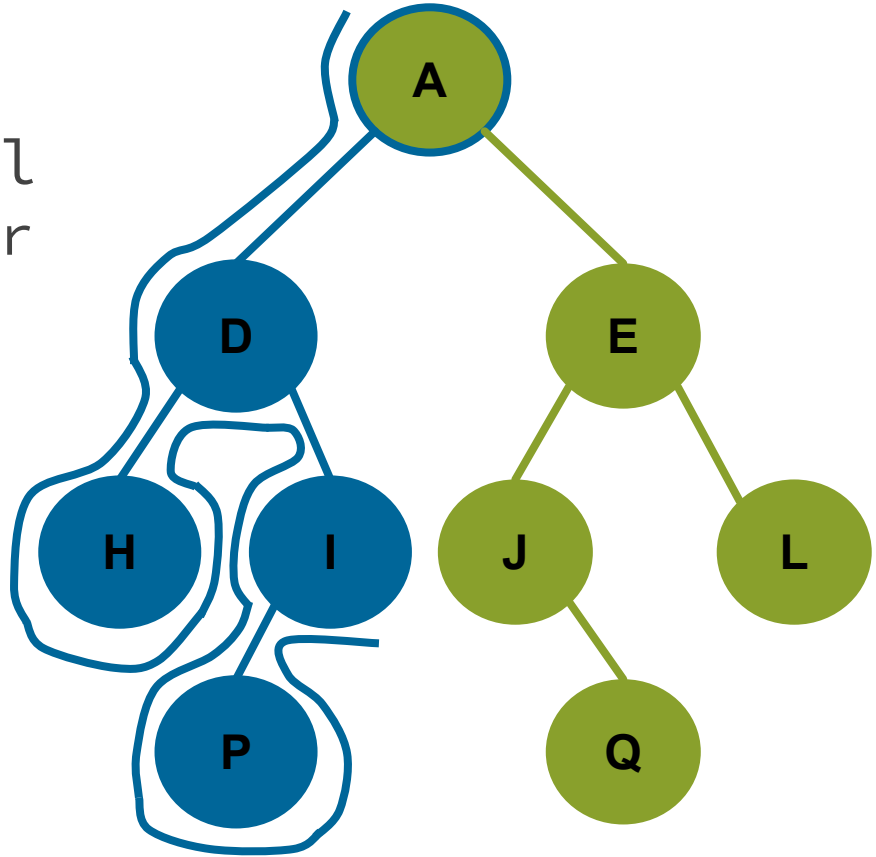
binary tree

- traversal
  - inorder



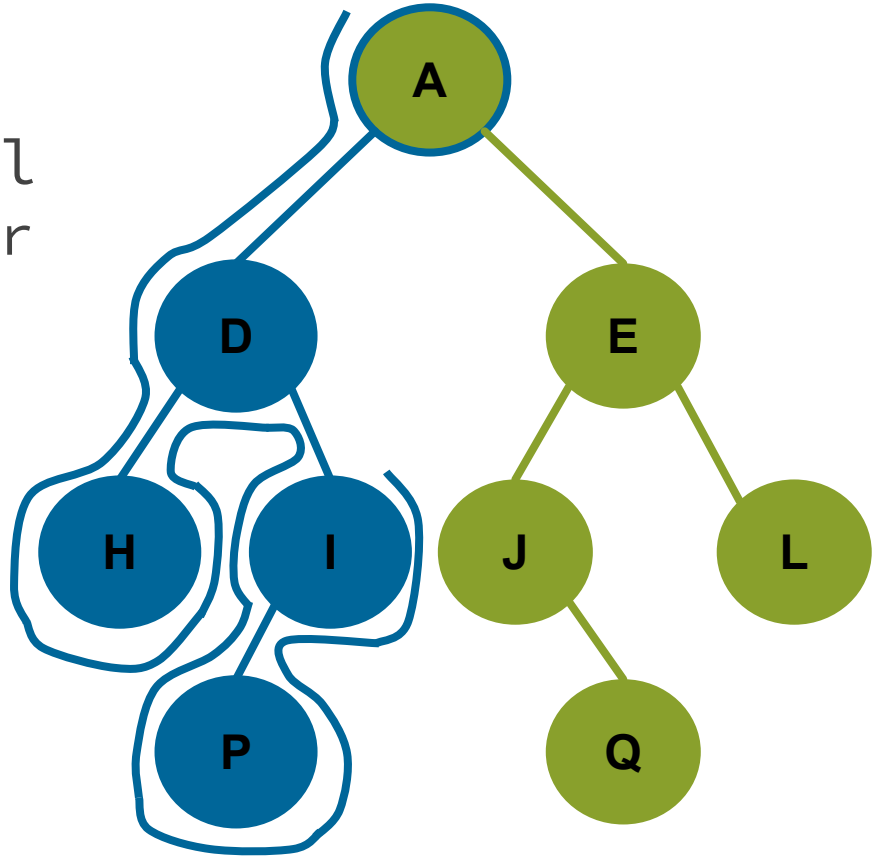
binary tree

- traversal
  - inorder



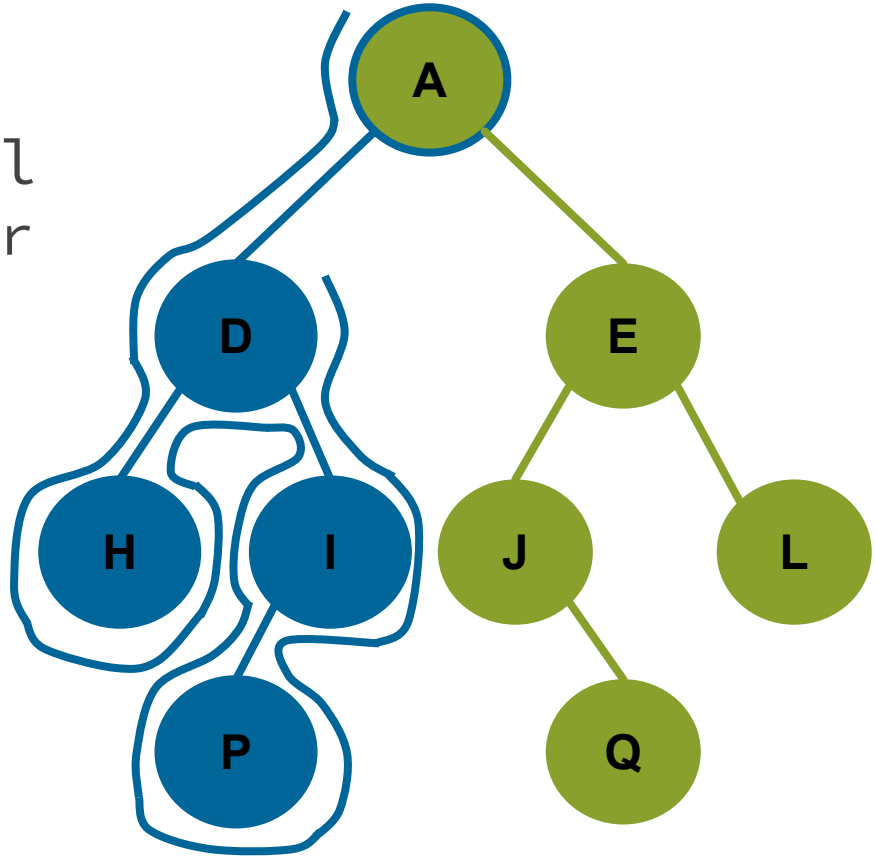
binary tree

- traversal
  - inorder



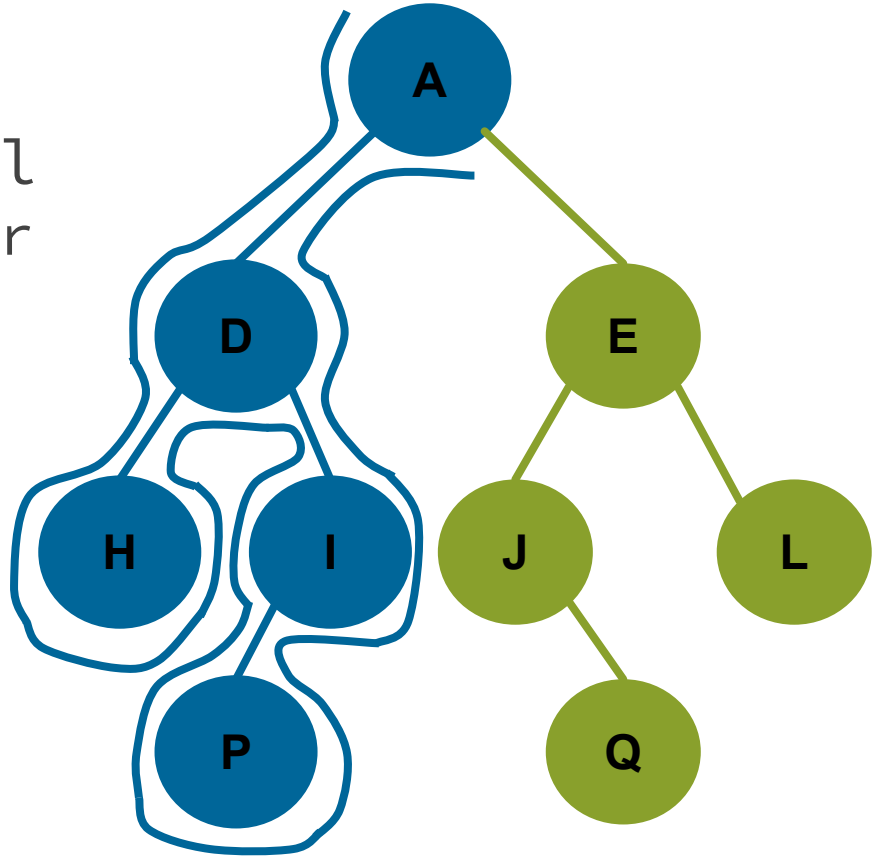
binary tree

- traversal
  - inorder



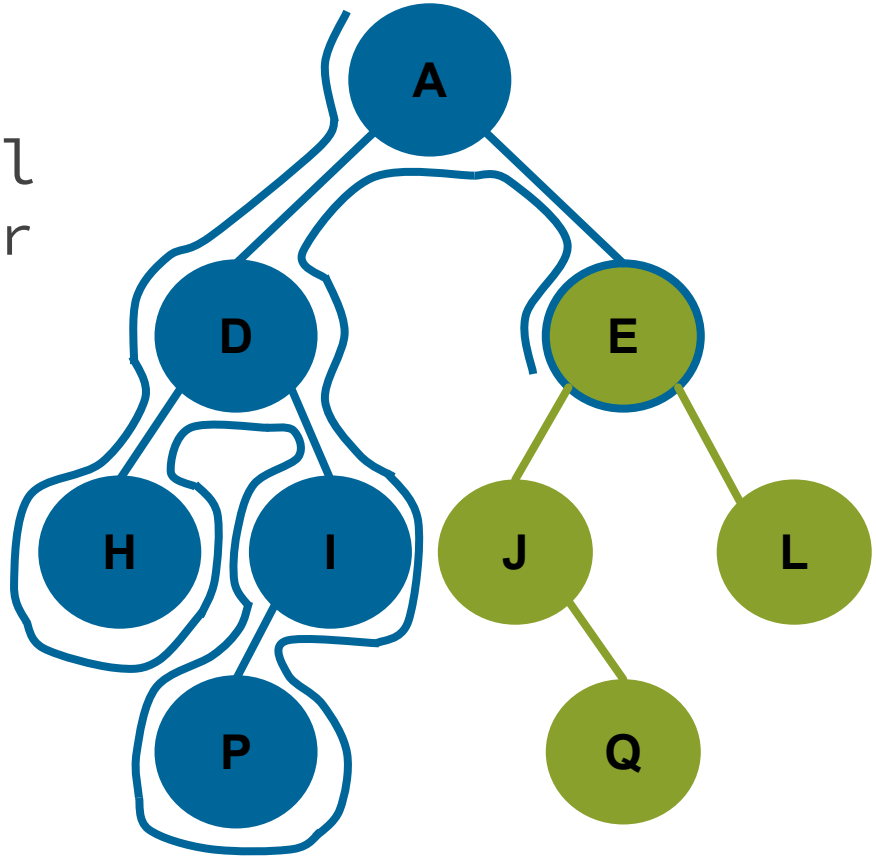
binary tree

- traversal
  - inorder



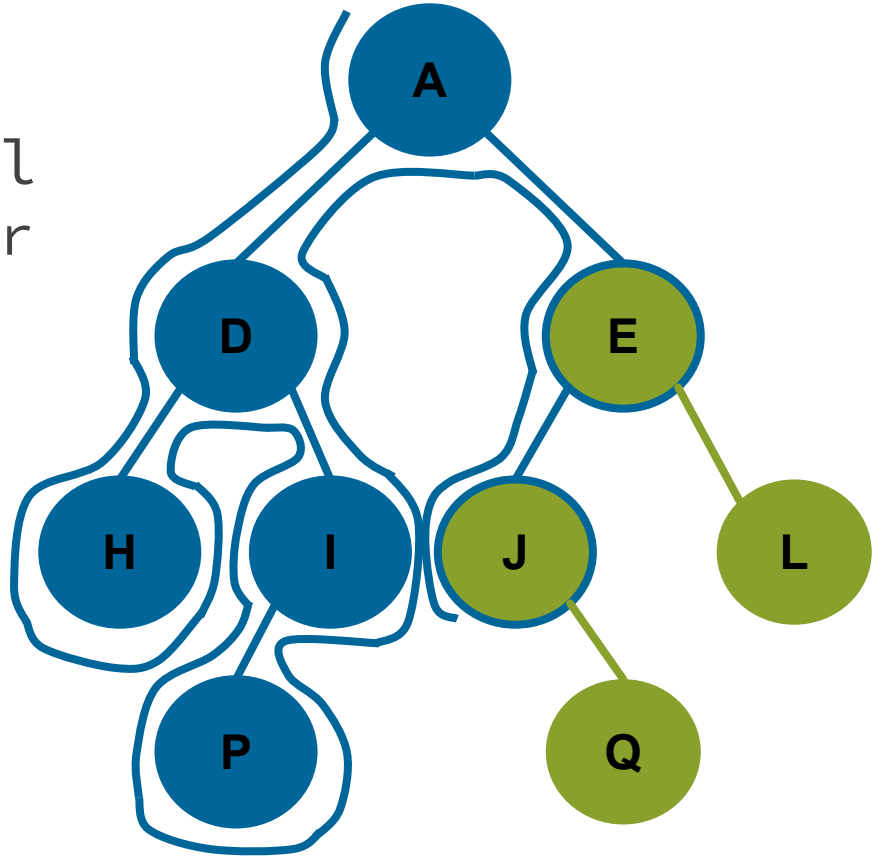
binary tree

- traversal
  - inorder



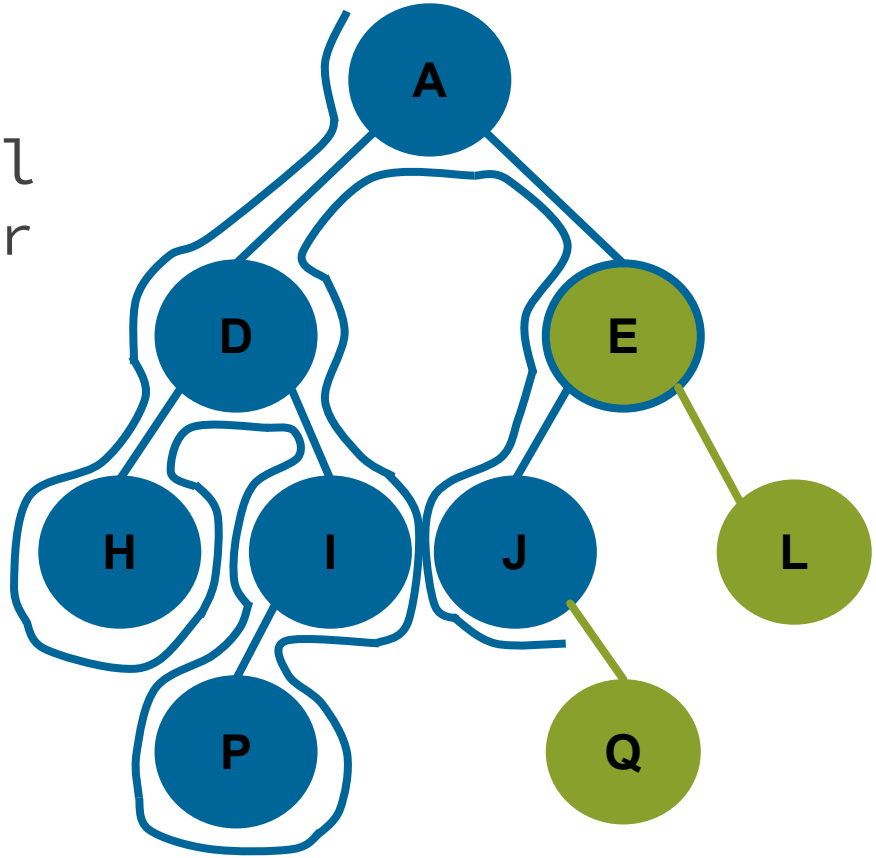
binary tree

- traversal
  - inorder



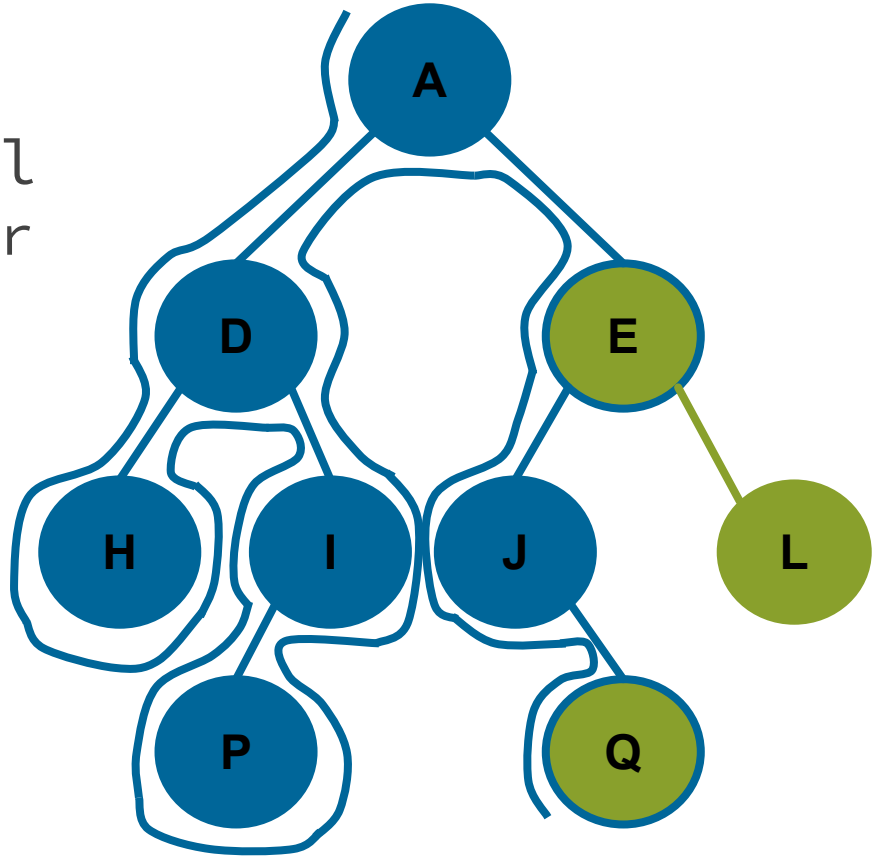
binary tree

- traversal
  - inorder



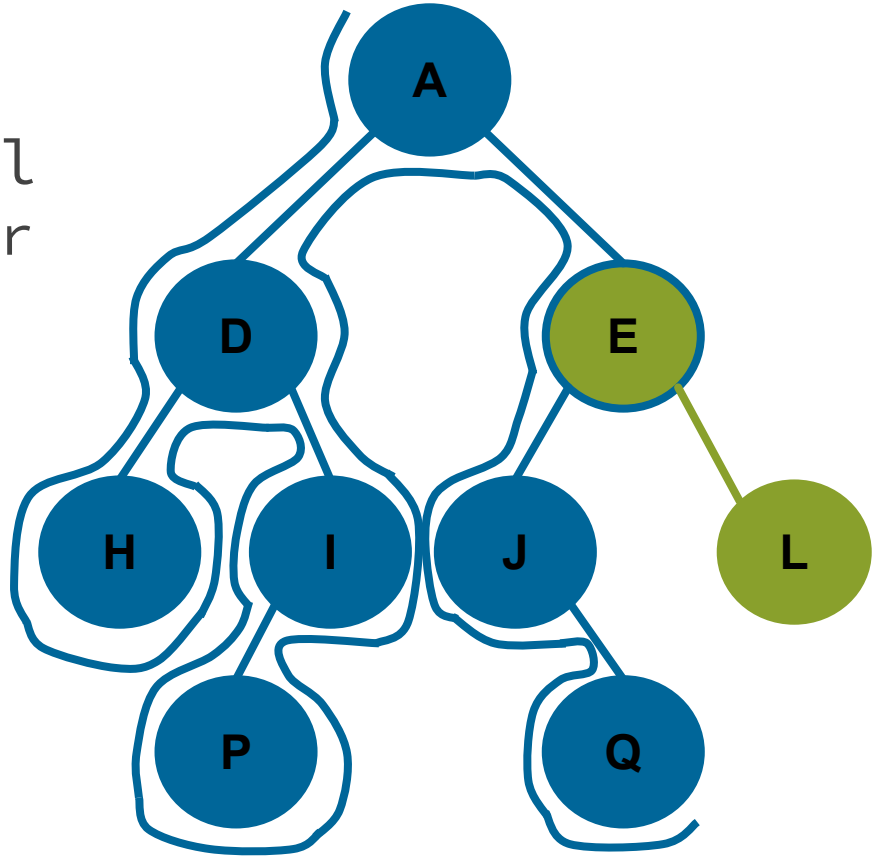
binary tree

- traversal
  - inorder



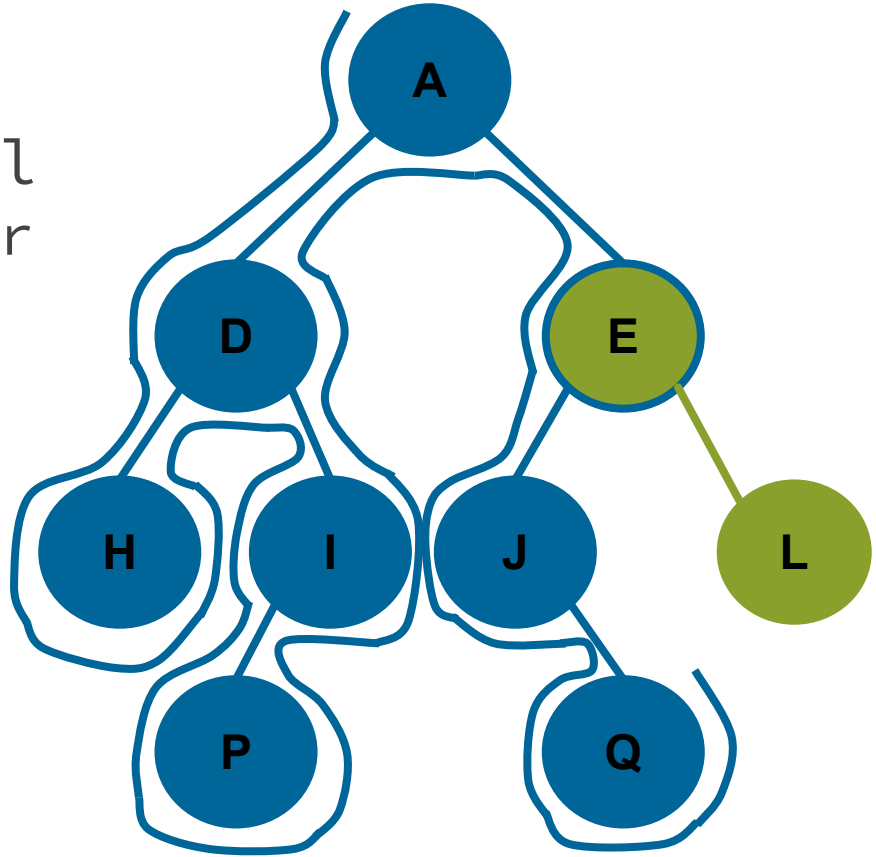
binary tree

- traversal
  - inorder



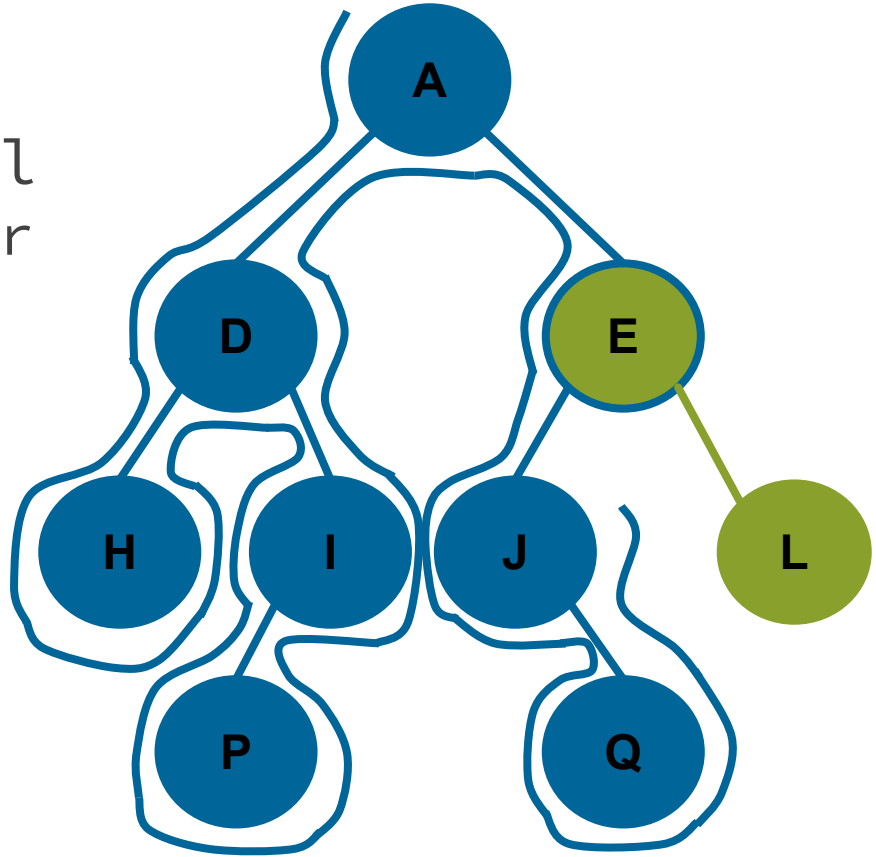
binary tree

- traversal
  - inorder



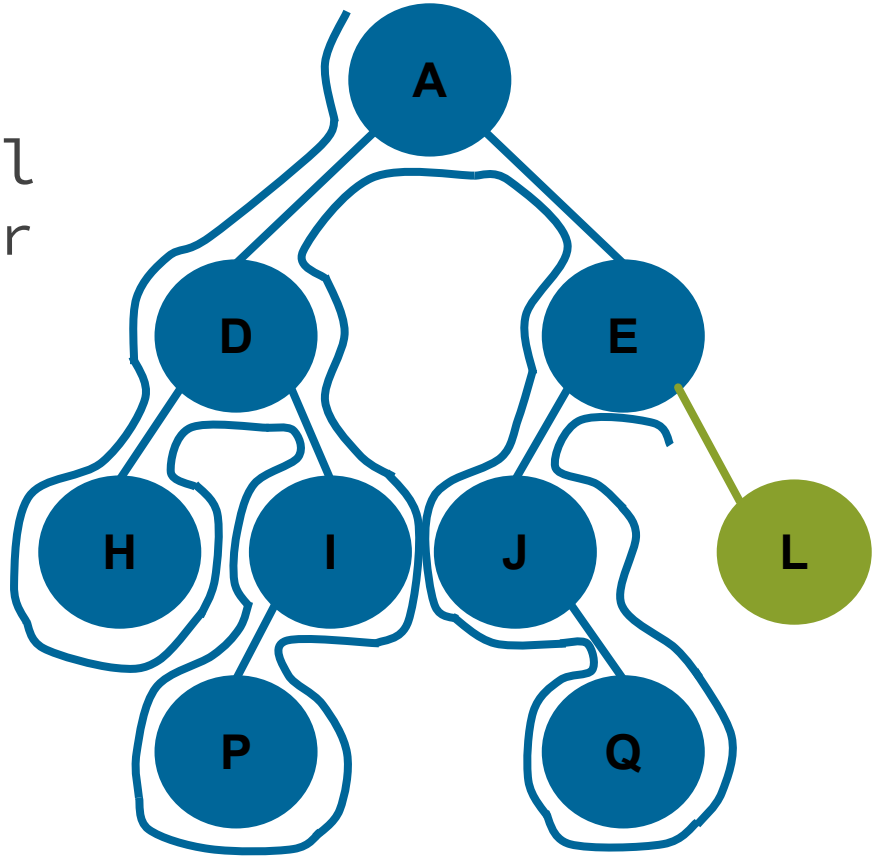
binary tree

- traversal
  - inorder



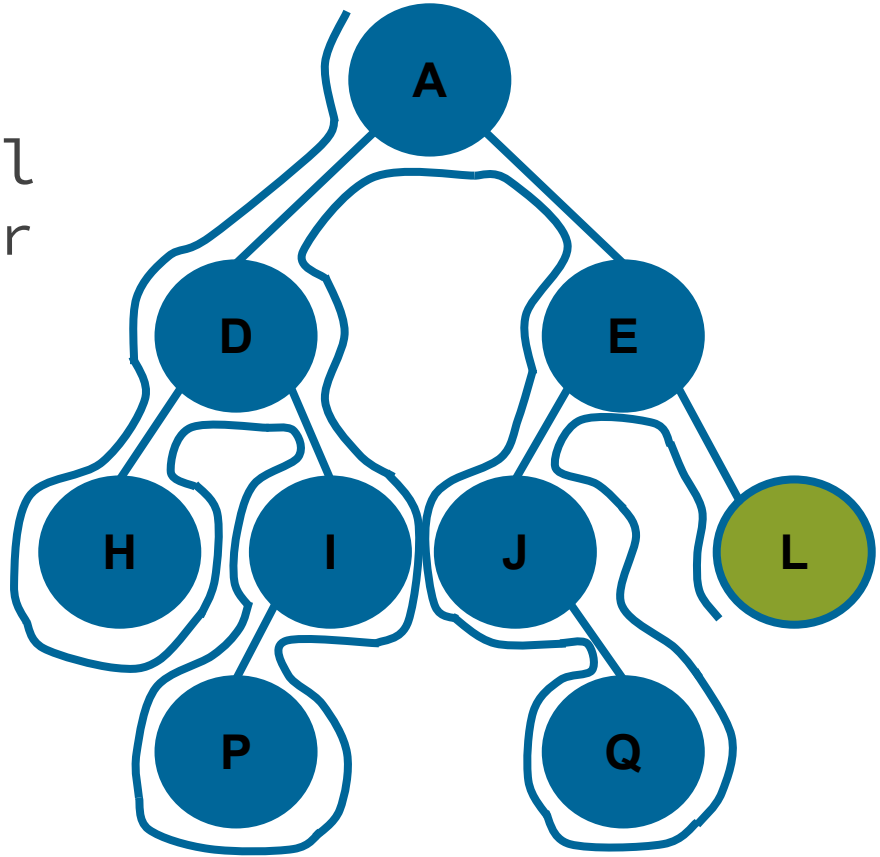
binary tree

- traversal
  - inorder



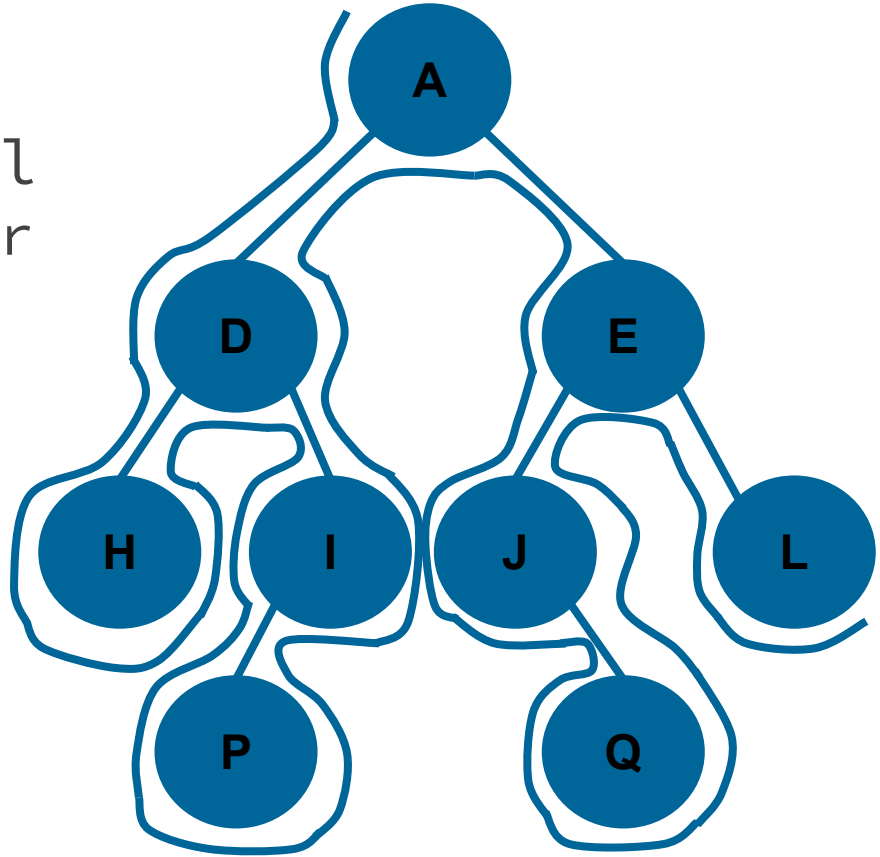
binary tree

- traversal
  - inorder



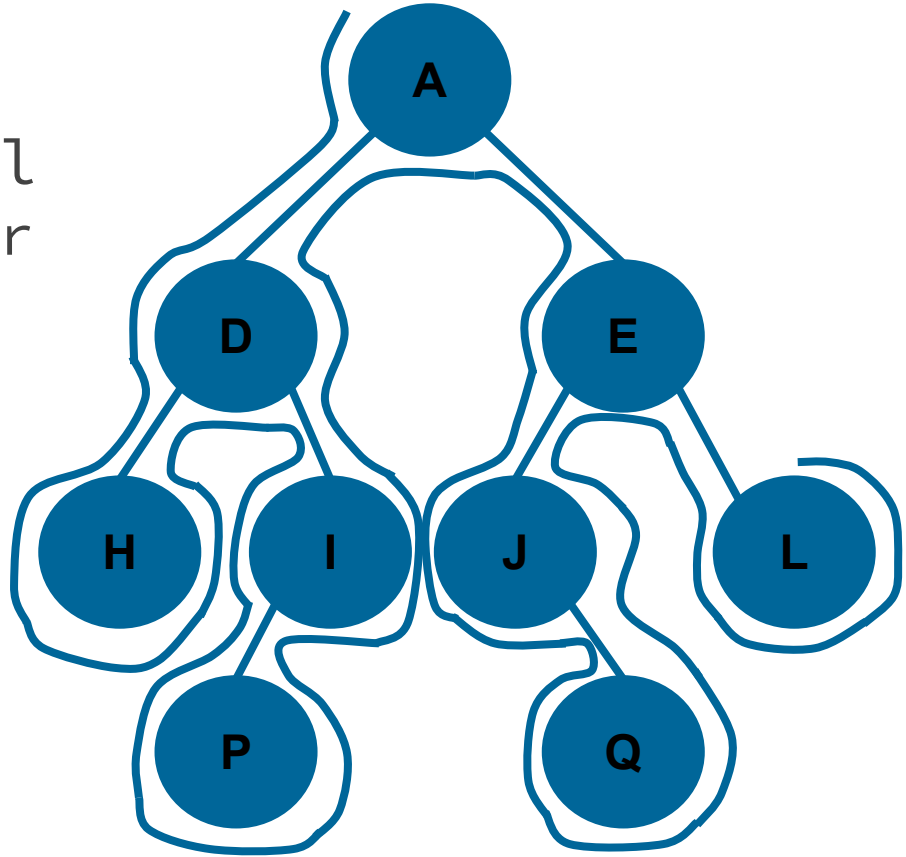
binary tree

- traversal
  - inorder



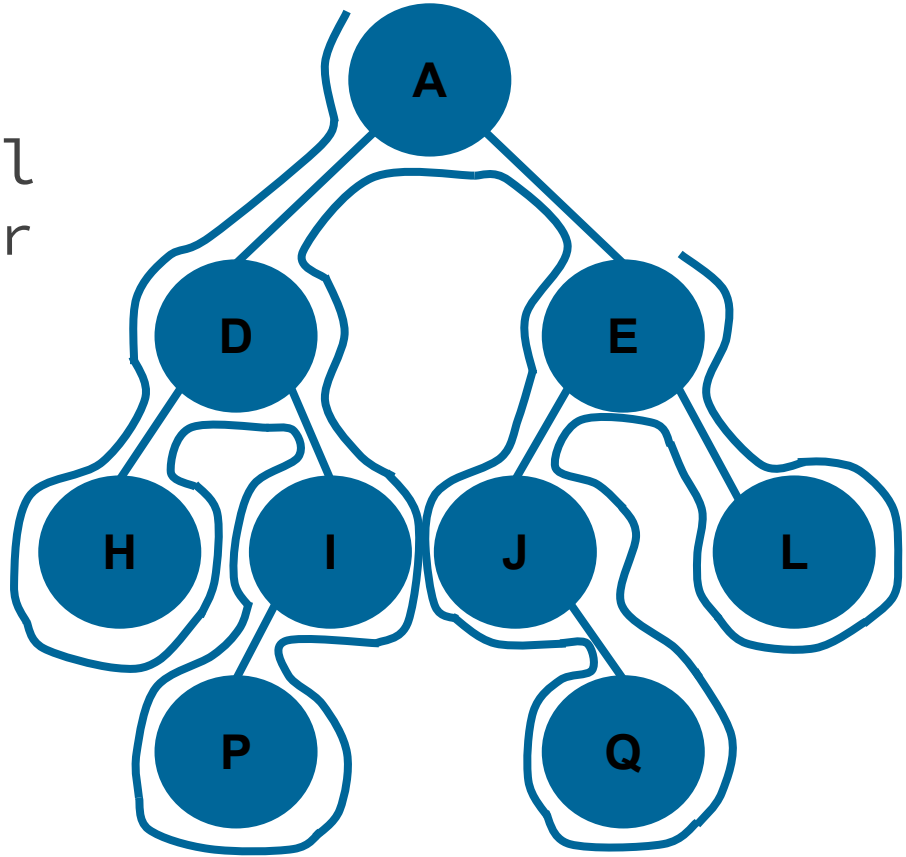
binary tree

- traversal
  - inorder



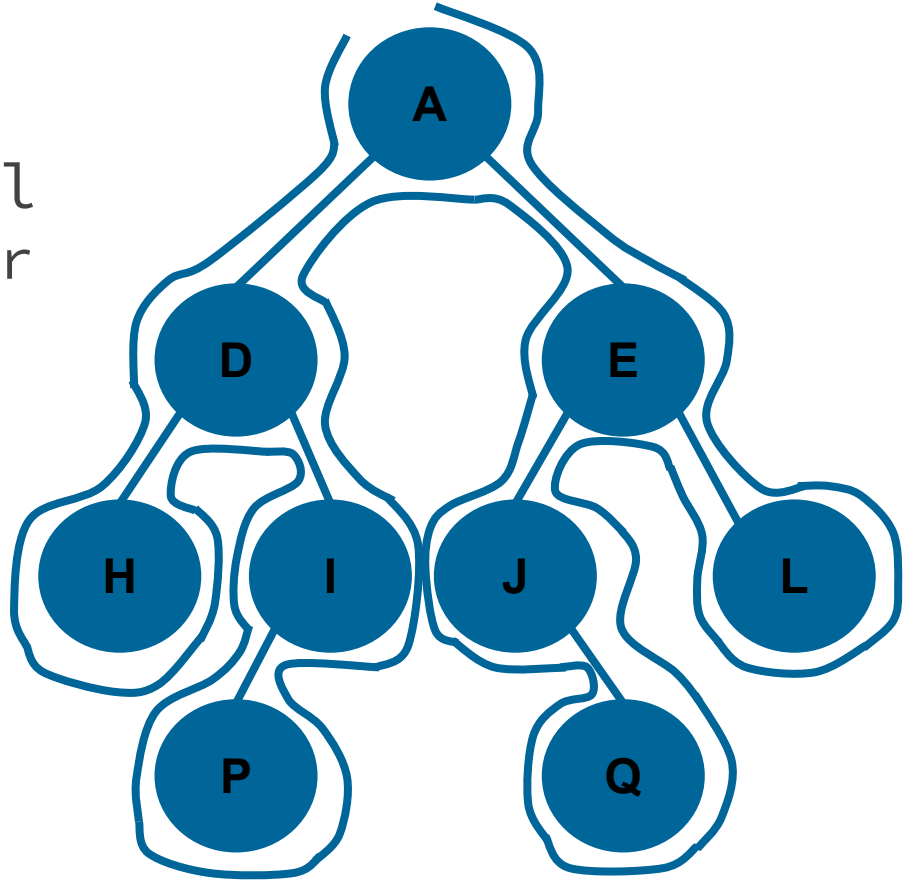
binary tree

- traversal
  - inorder



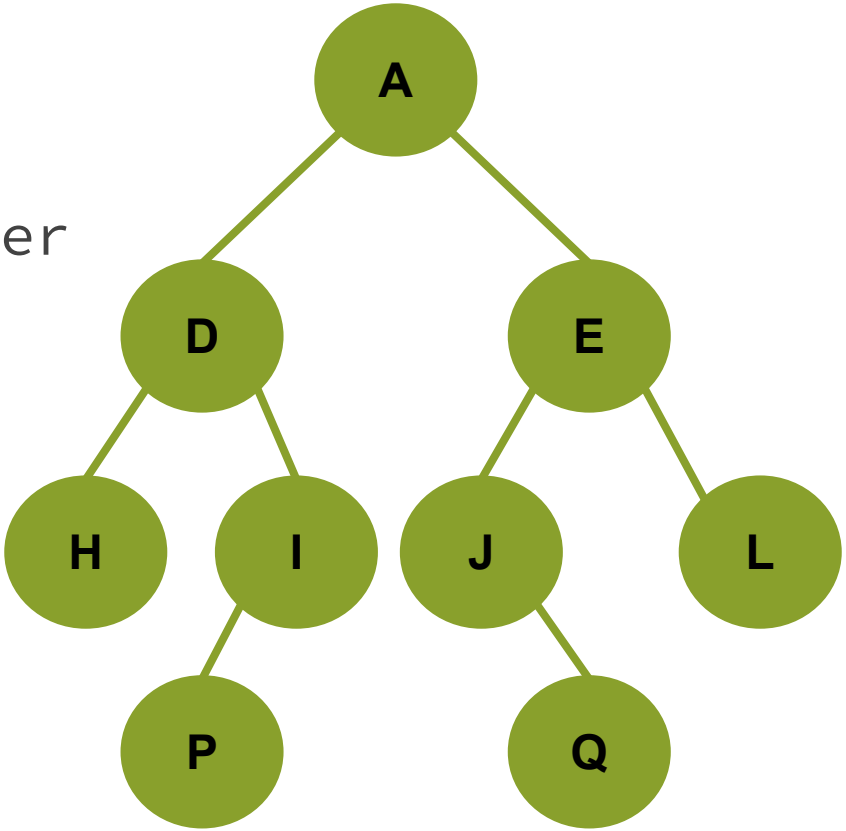
binary tree

- traversal
  - inorder



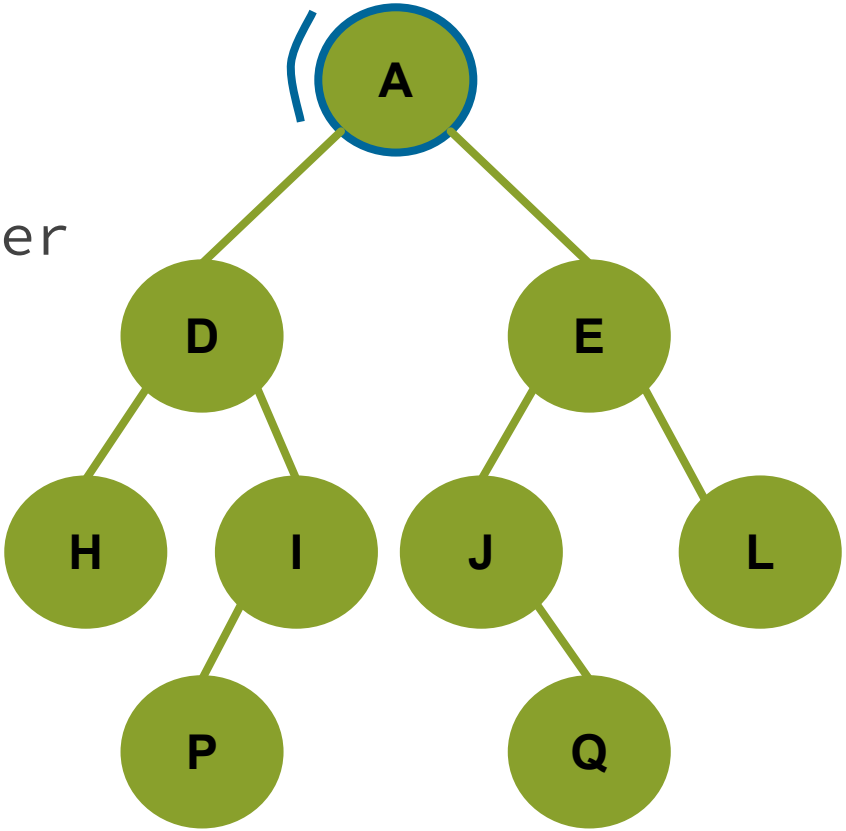
binary tree

- traversal
  - postorder



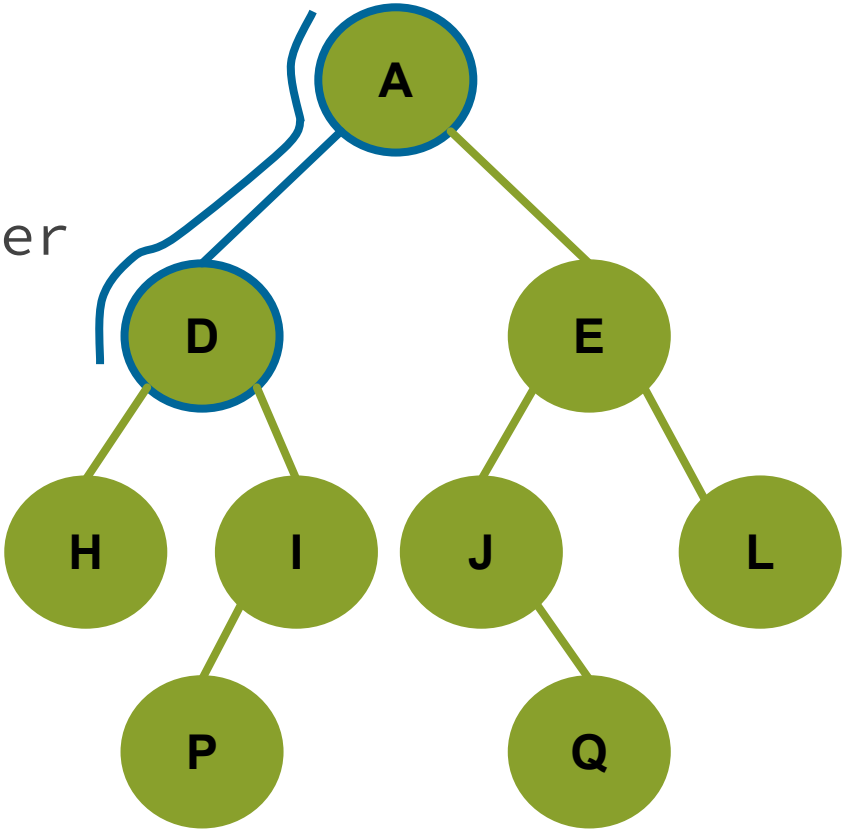
binary tree

- traversal
  - postorder



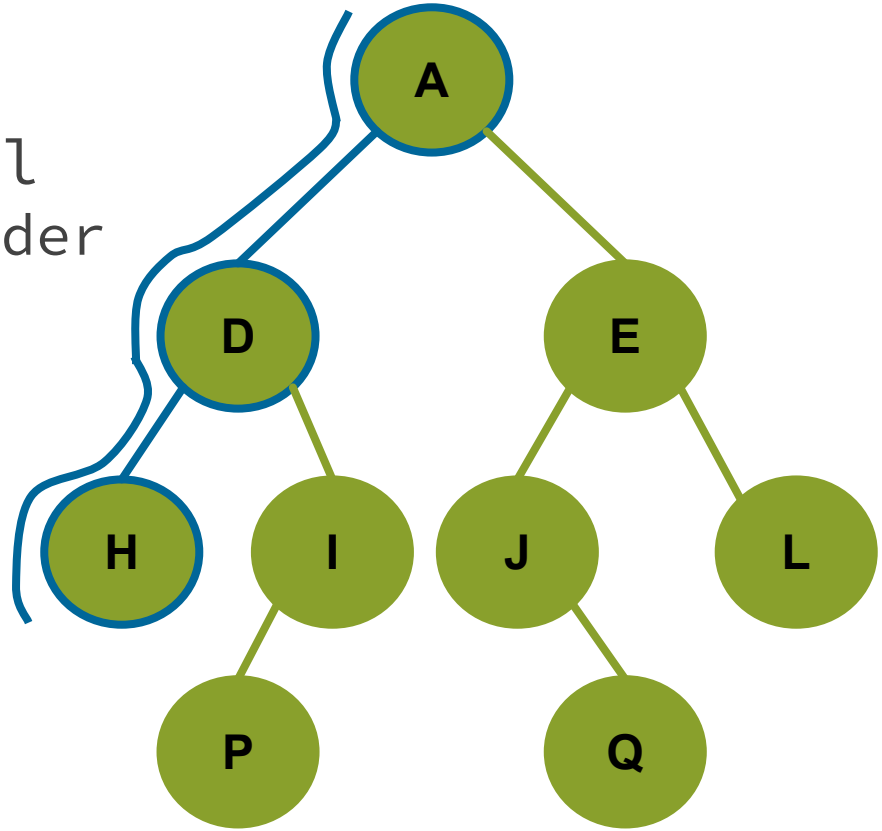
binary tree

- traversal
  - postorder



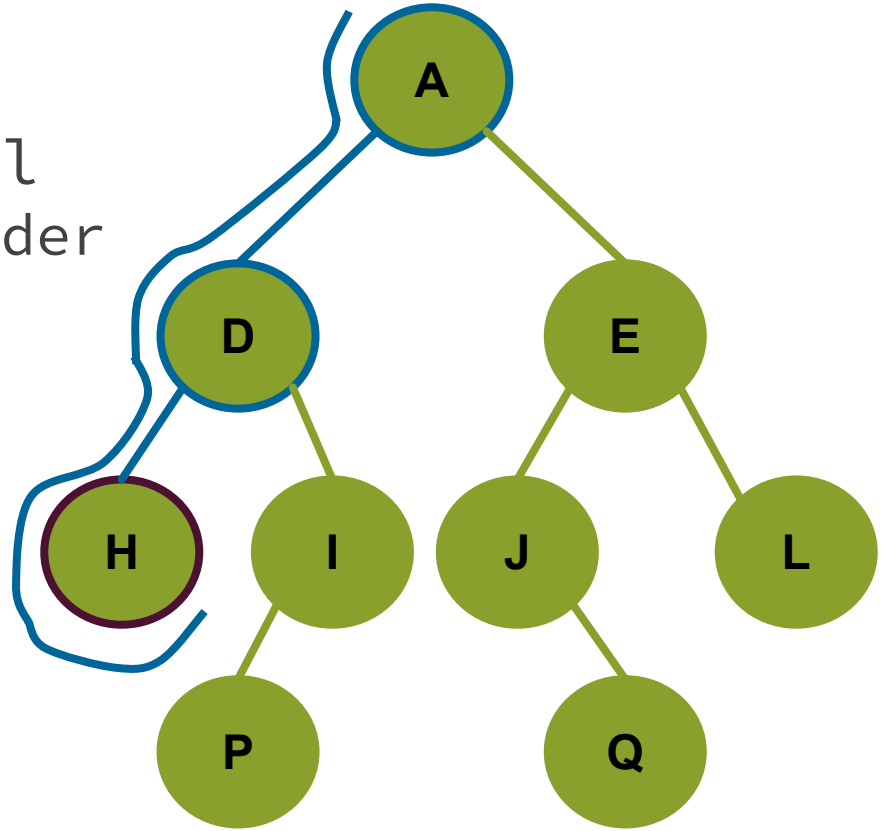
binary tree

- traversal
  - postorder



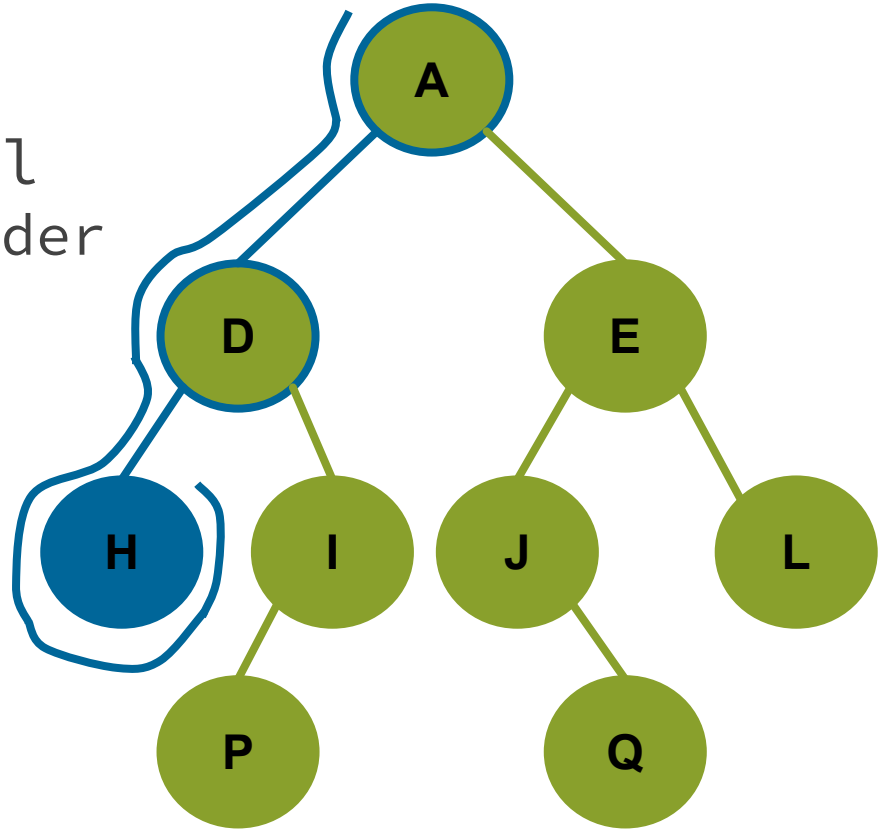
binary tree

- traversal
  - postorder



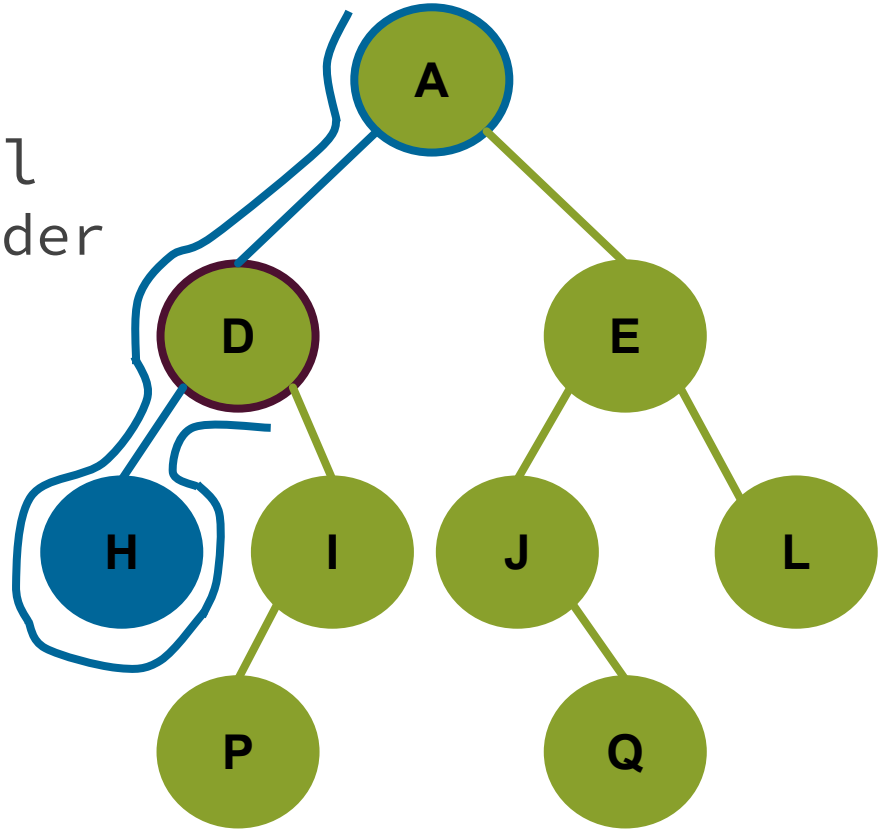
binary tree

- traversal
  - postorder



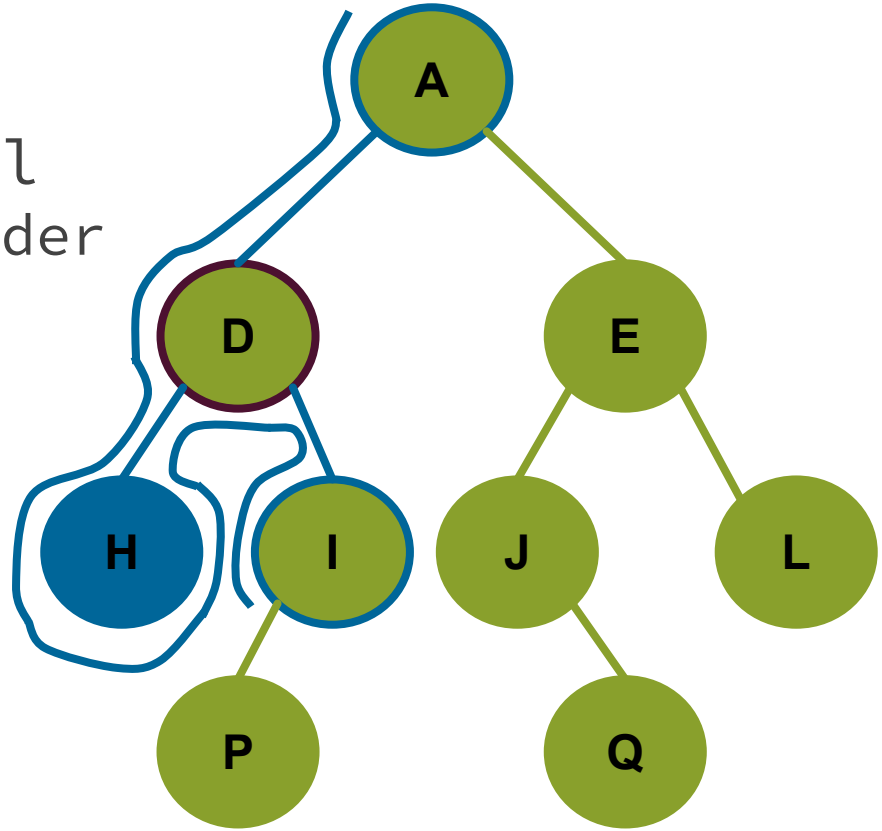
binary tree

- traversal
  - postorder



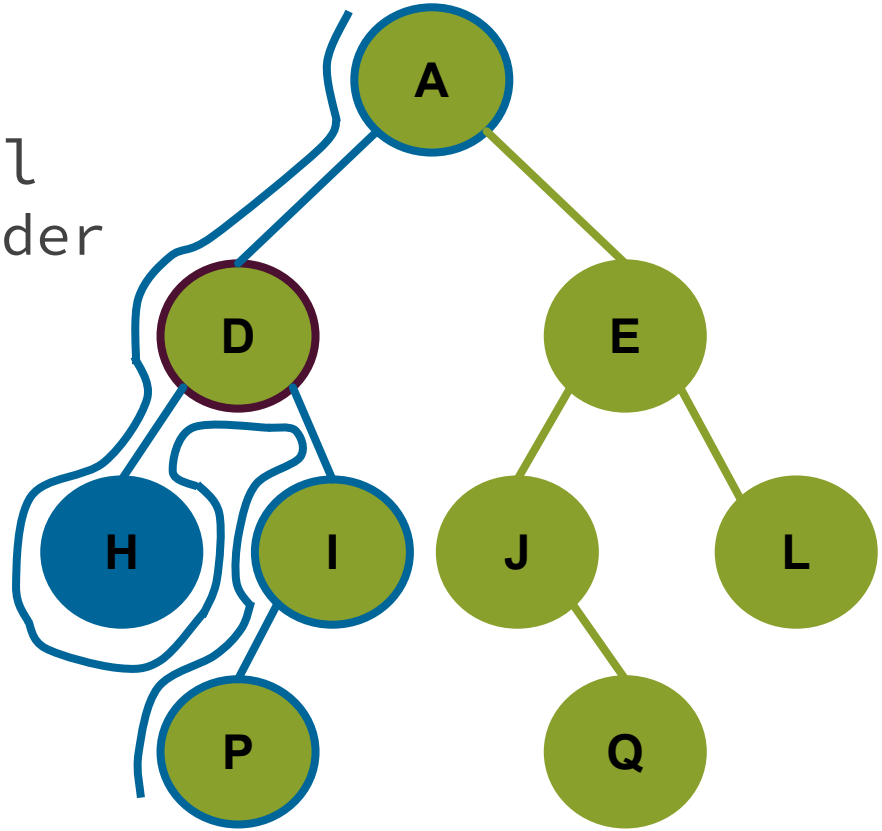
binary tree

- traversal
  - postorder



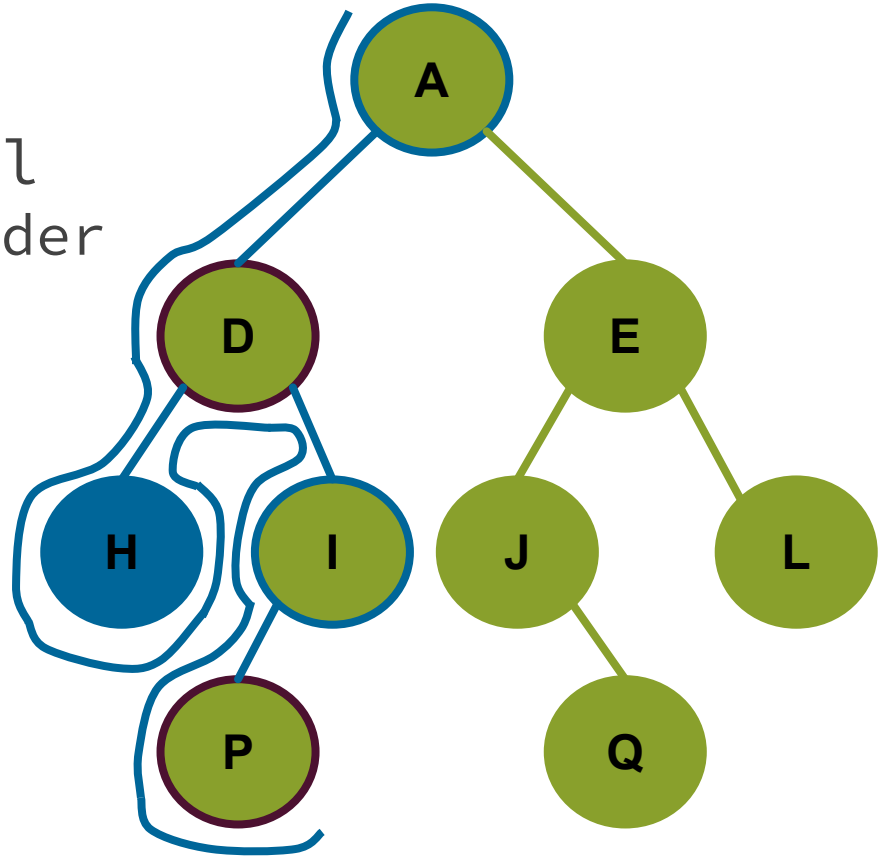
binary tree

- traversal
  - postorder



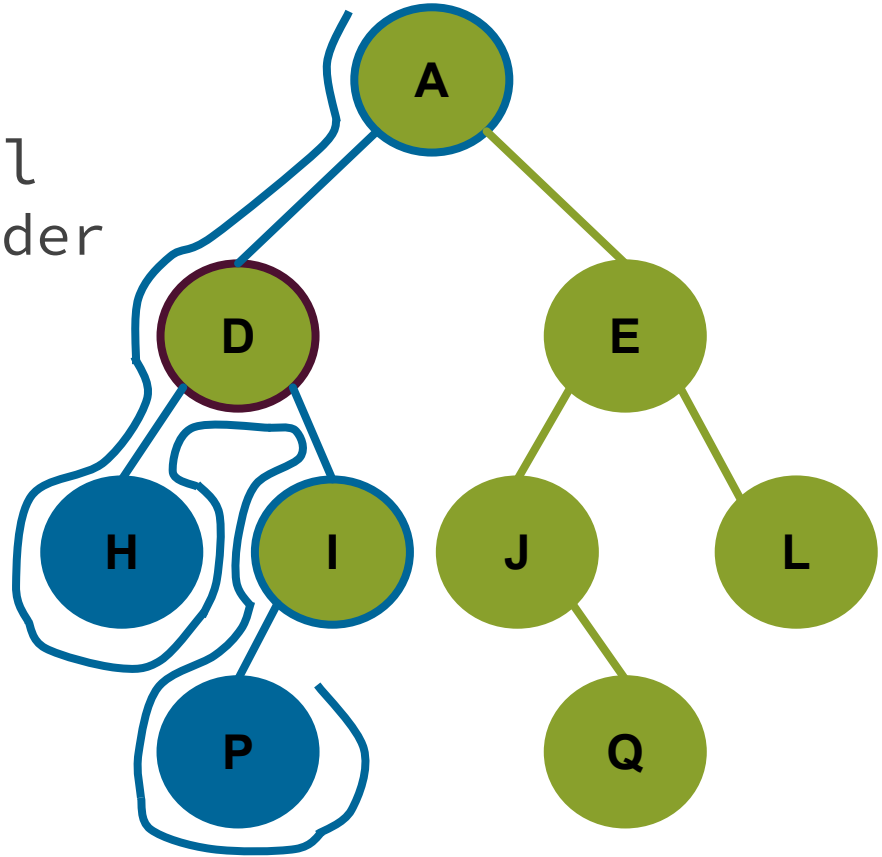
binary tree

- traversal
  - postorder



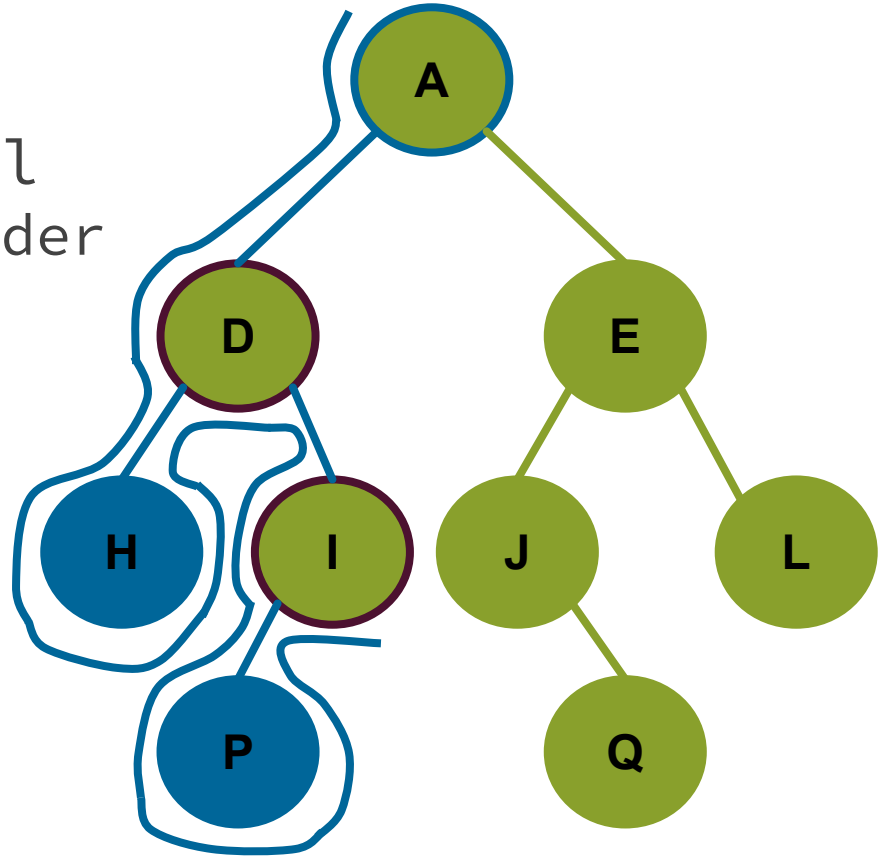
binary tree

- traversal
  - postorder



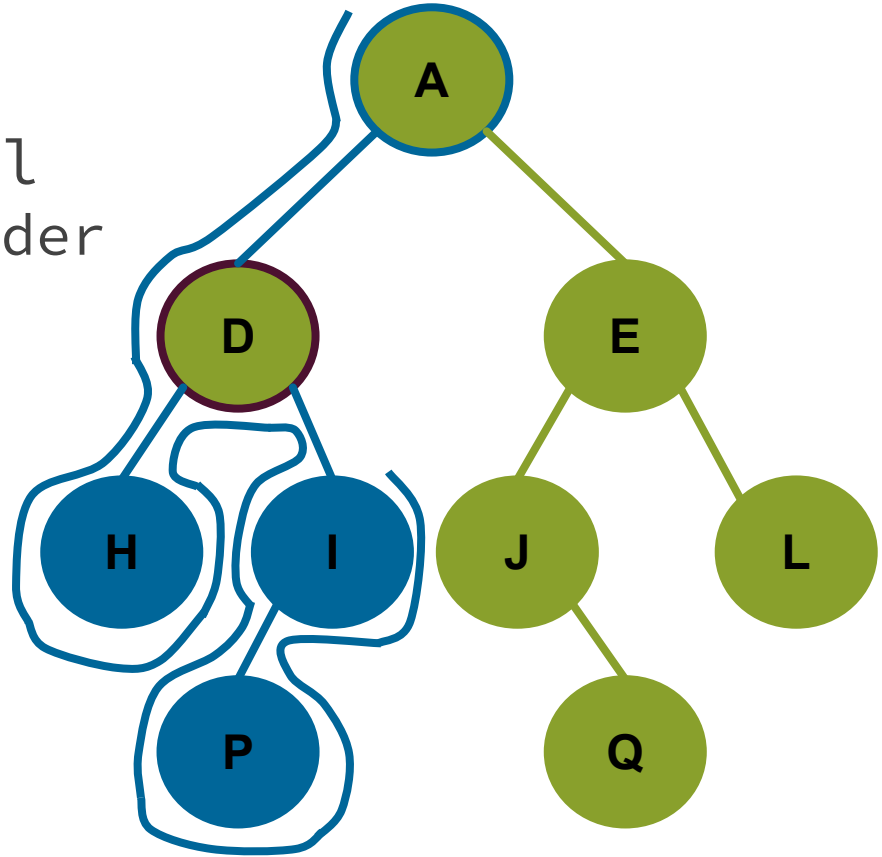
binary tree

- traversal
  - postorder



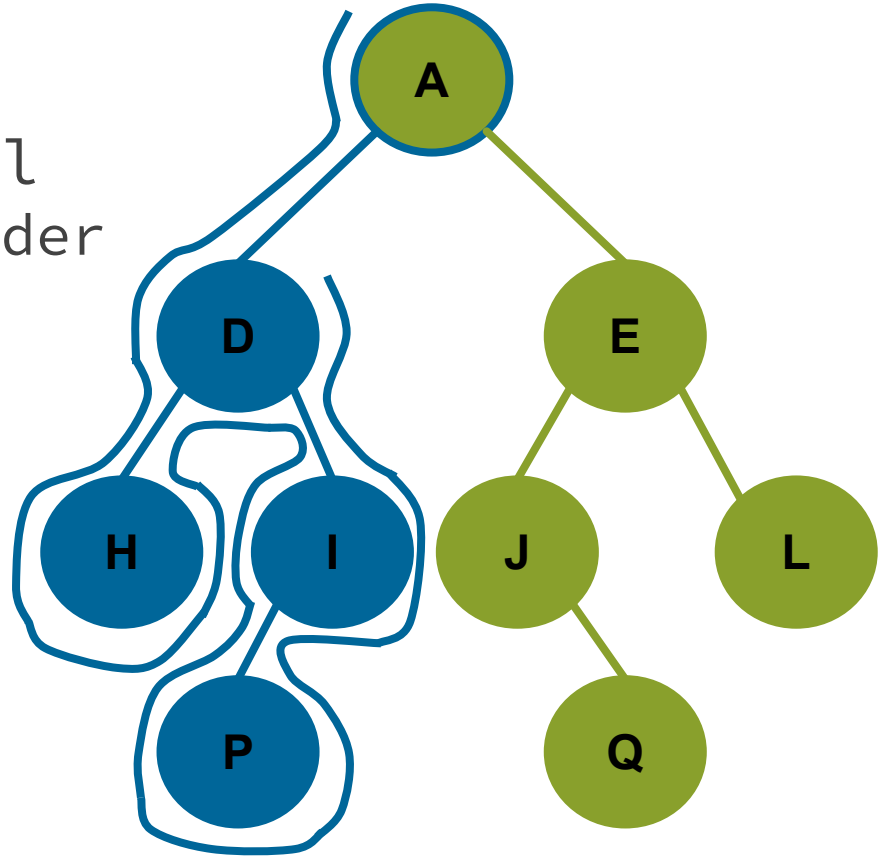
binary tree

- traversal
  - postorder



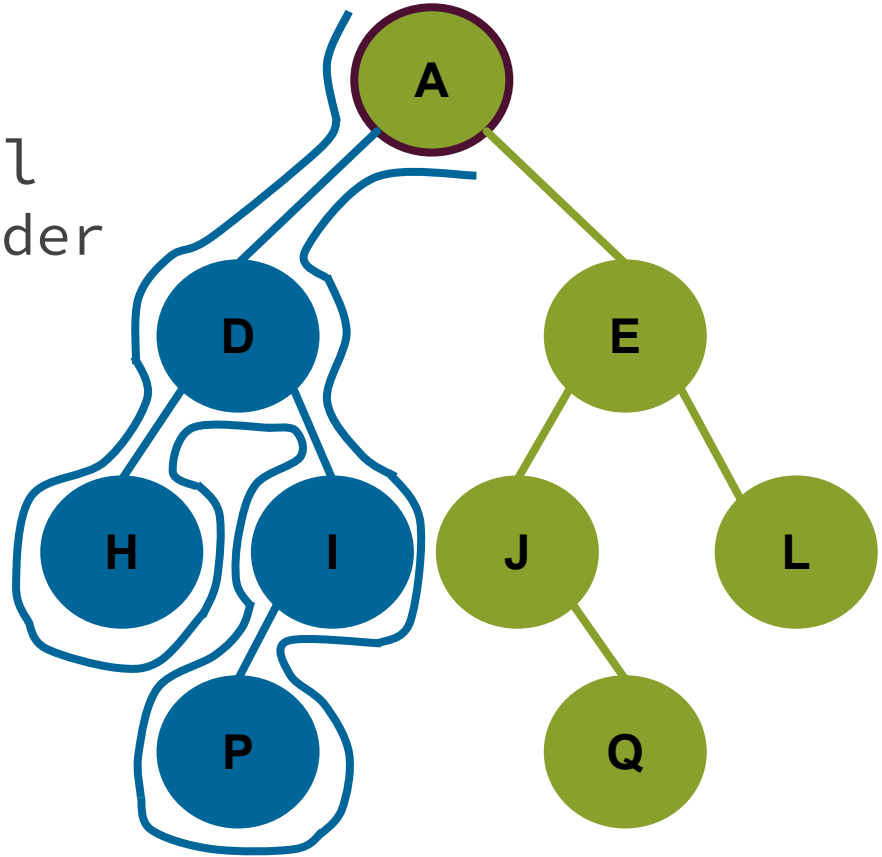
binary tree

- traversal
  - postorder



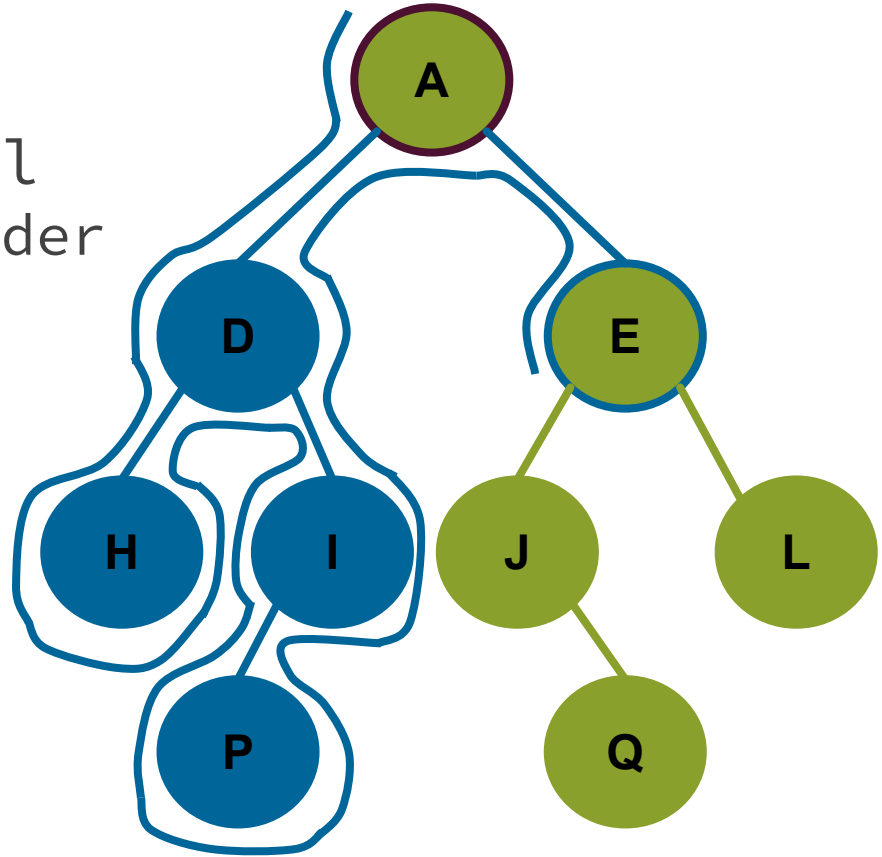
binary tree

- traversal
  - postorder



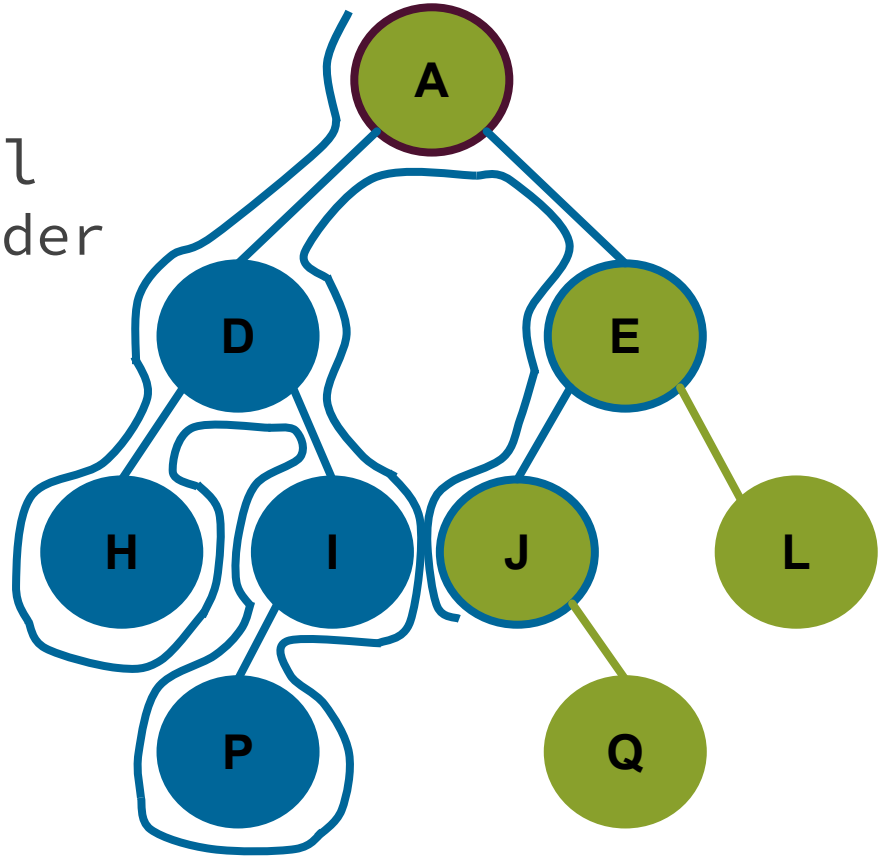
binary tree

- traversal
  - postorder



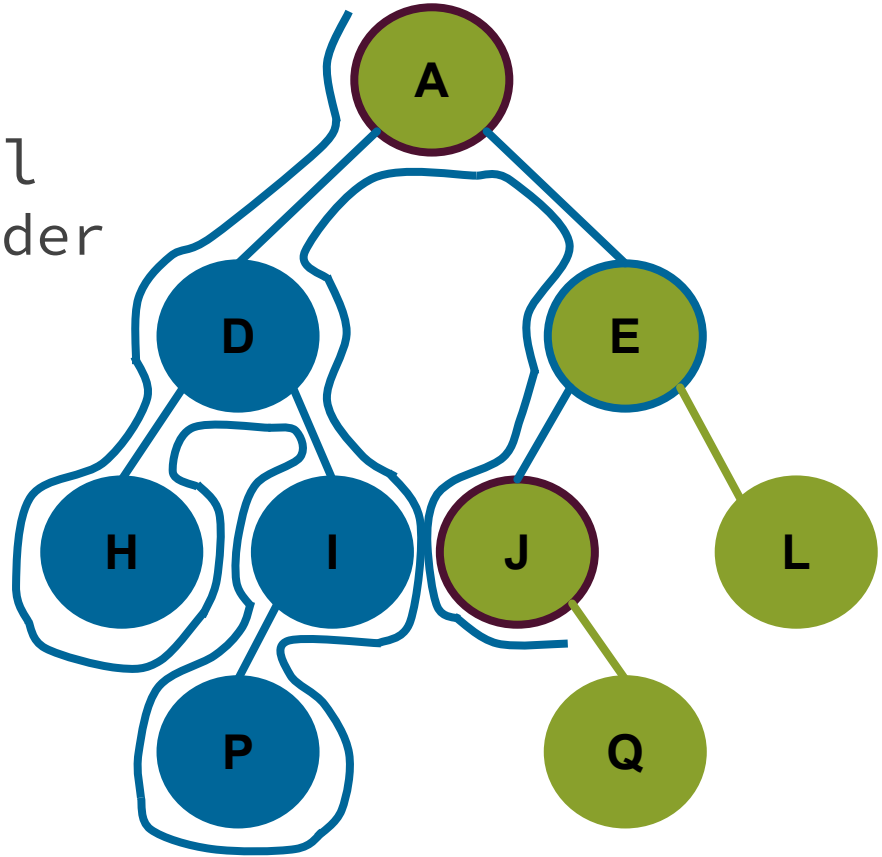
binary tree

- traversal
  - postorder



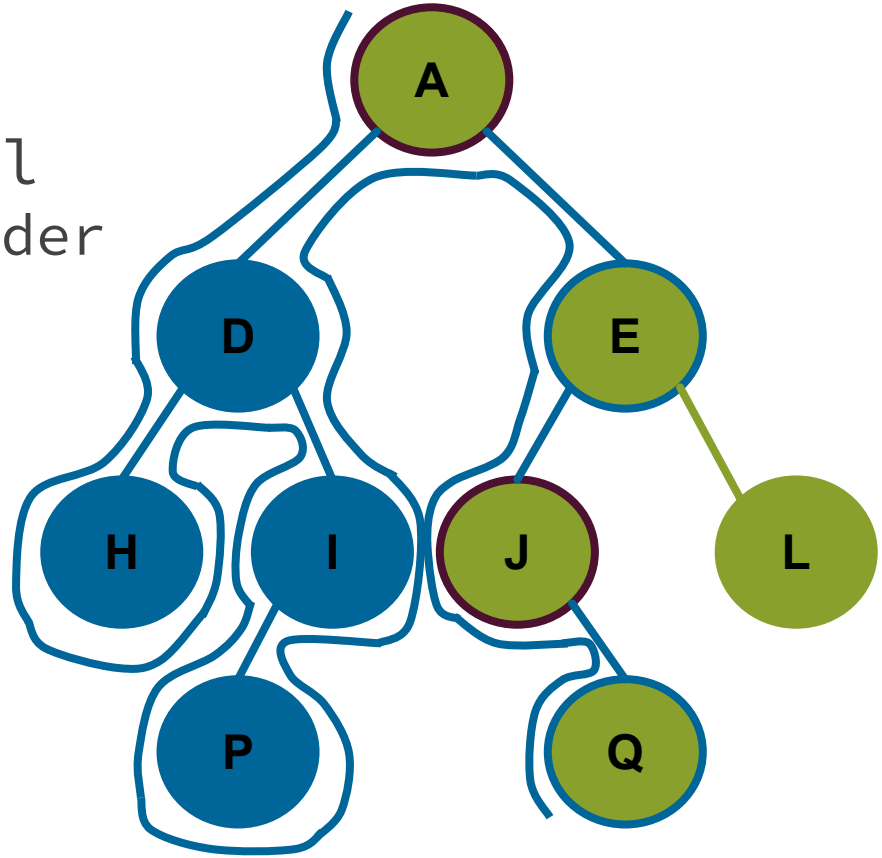
binary tree

- traversal
  - postorder



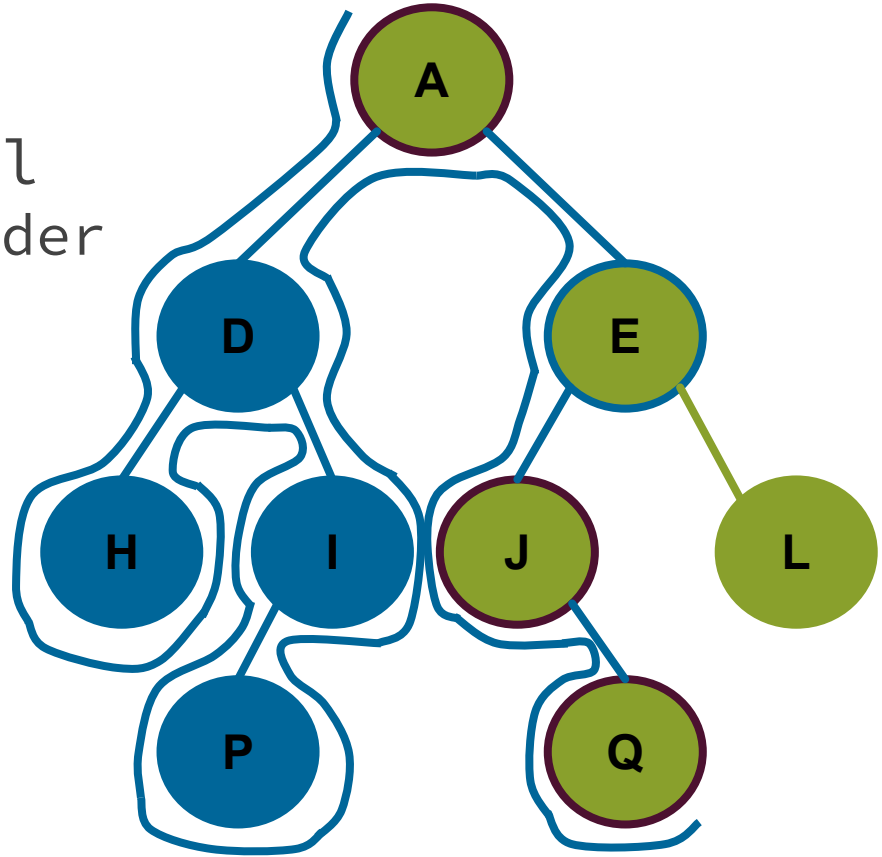
binary tree

- traversal
  - postorder



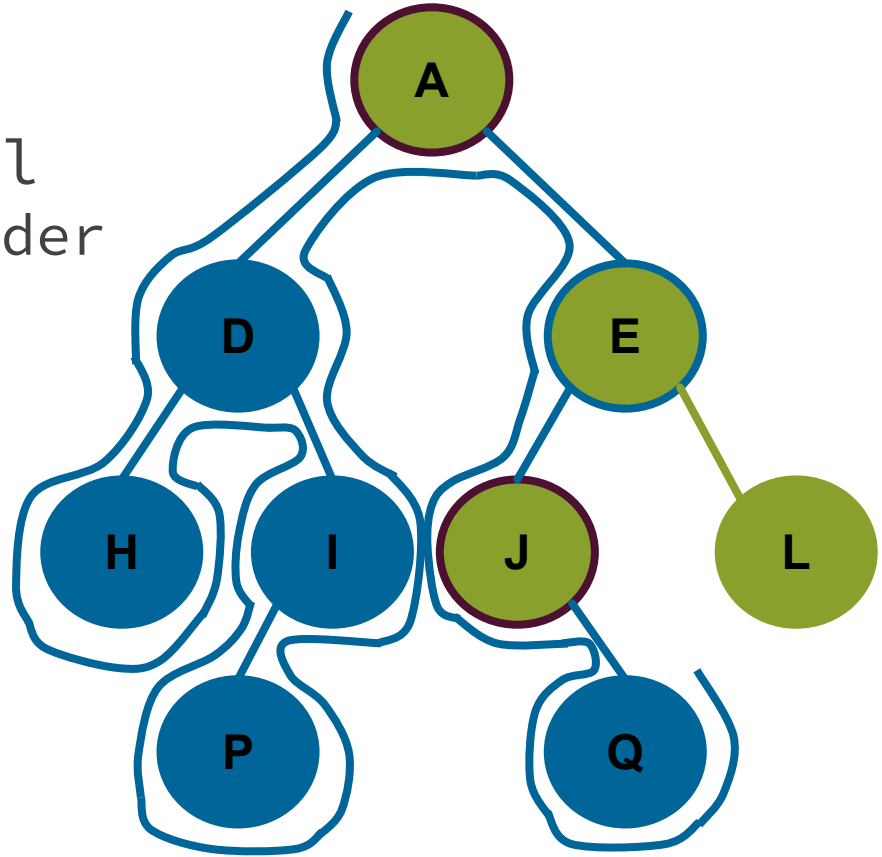
binary tree

- traversal
  - postorder



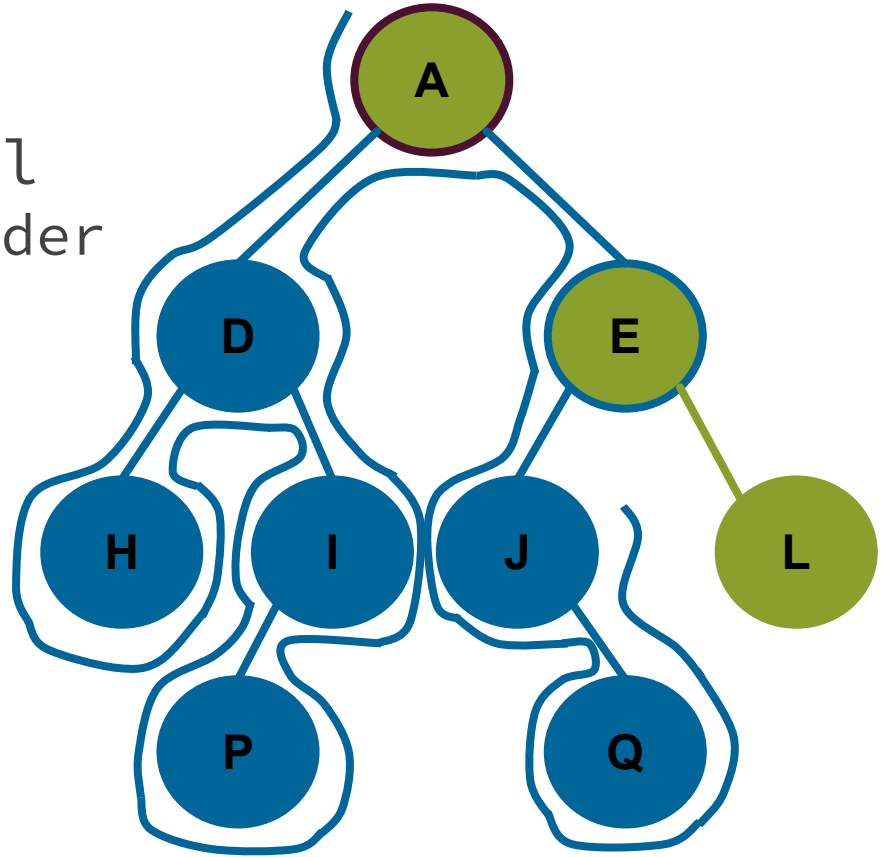
binary tree

- traversal
  - postorder



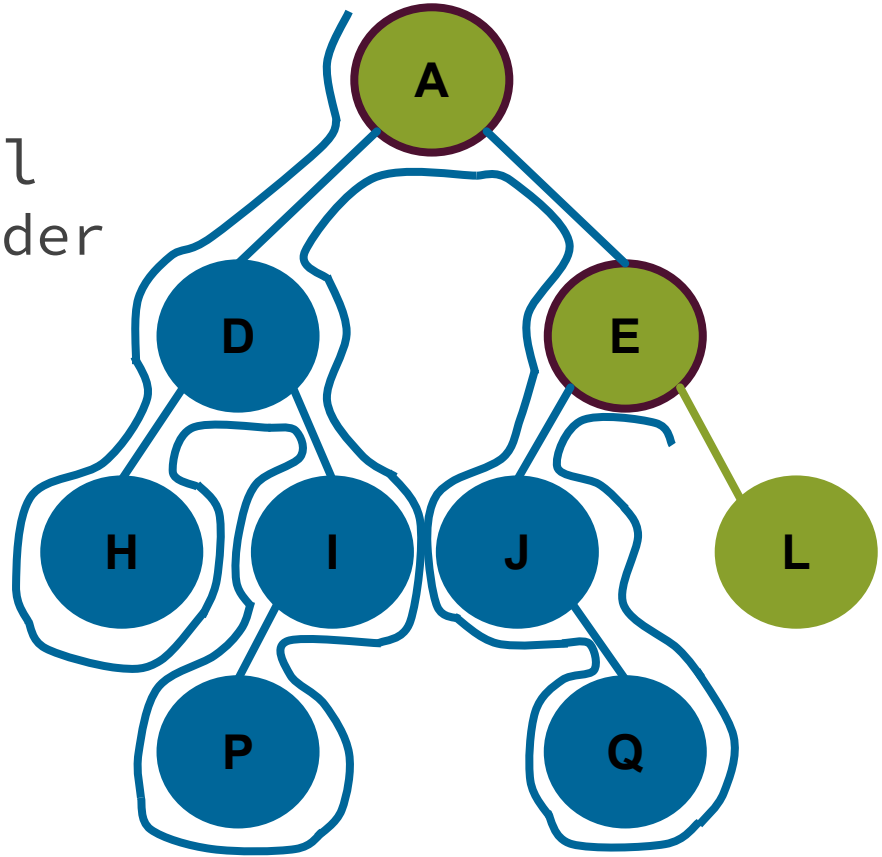
binary tree

- traversal
  - postorder



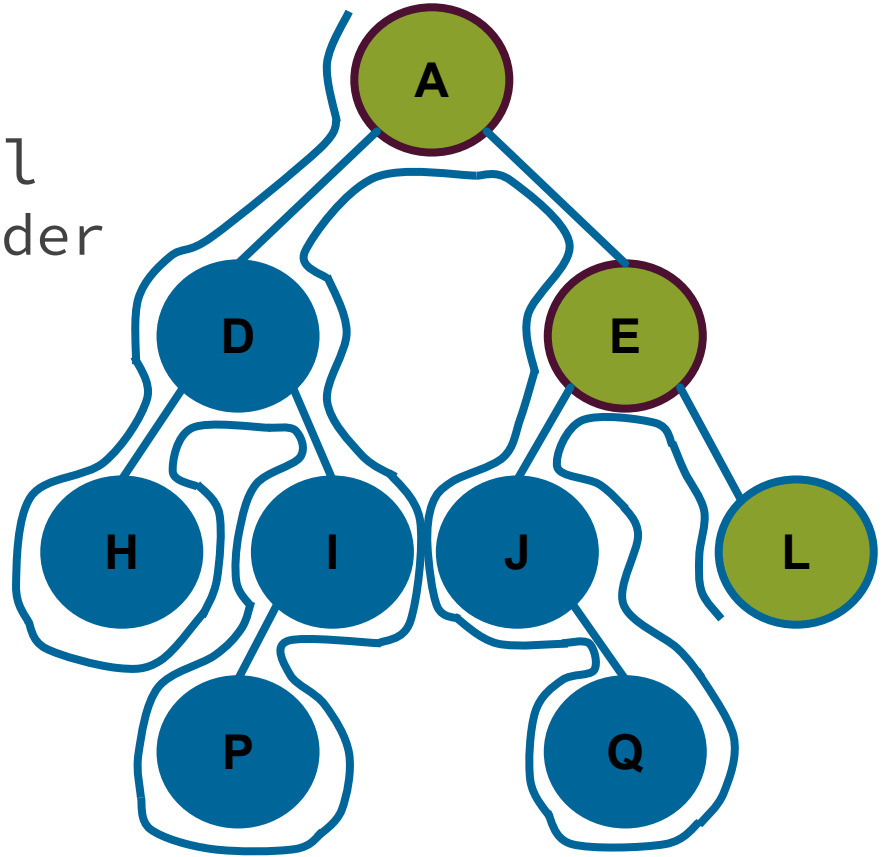
binary tree

- traversal
  - postorder



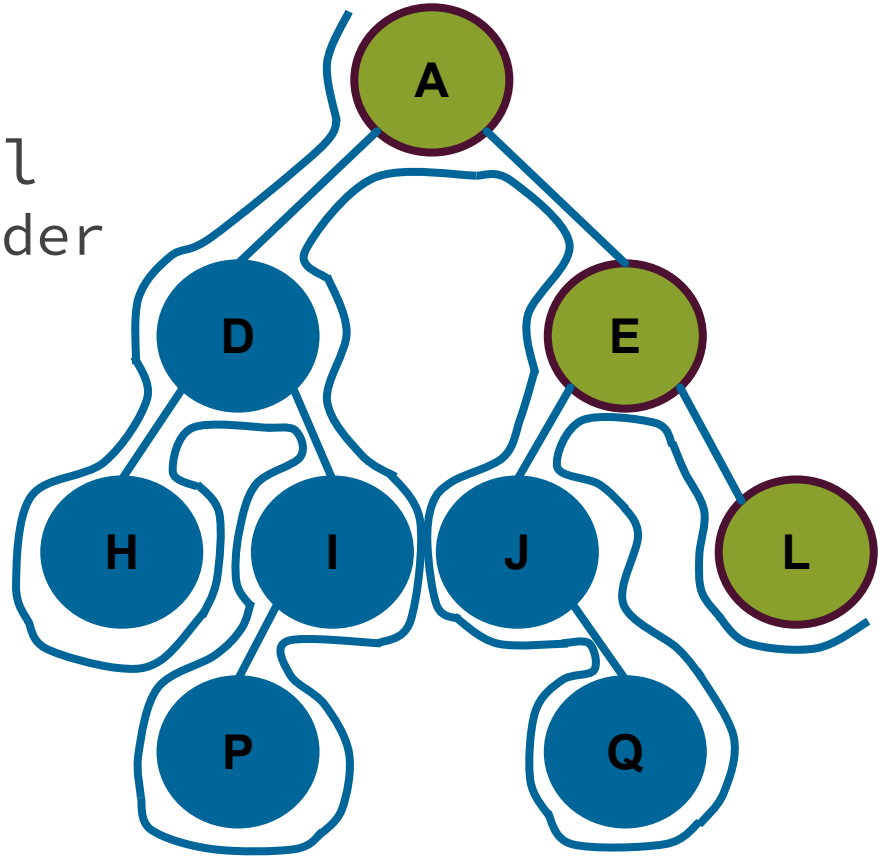
binary tree

- traversal
  - postorder



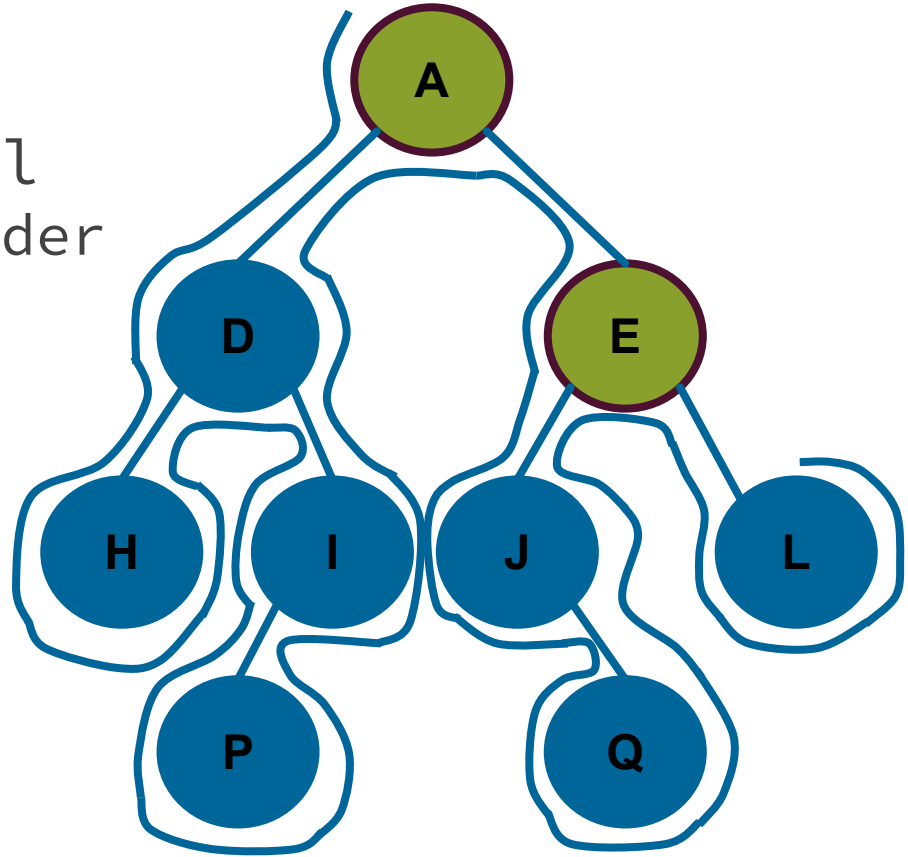
binary tree

- traversal
  - postorder



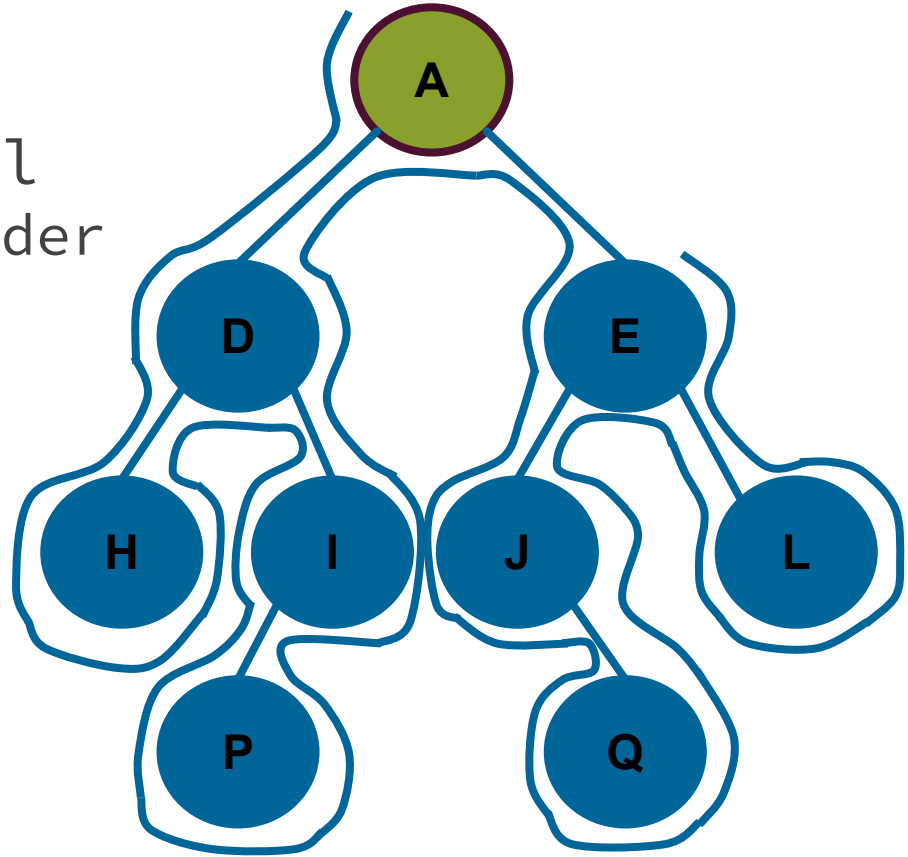
binary tree

- traversal
  - postorder



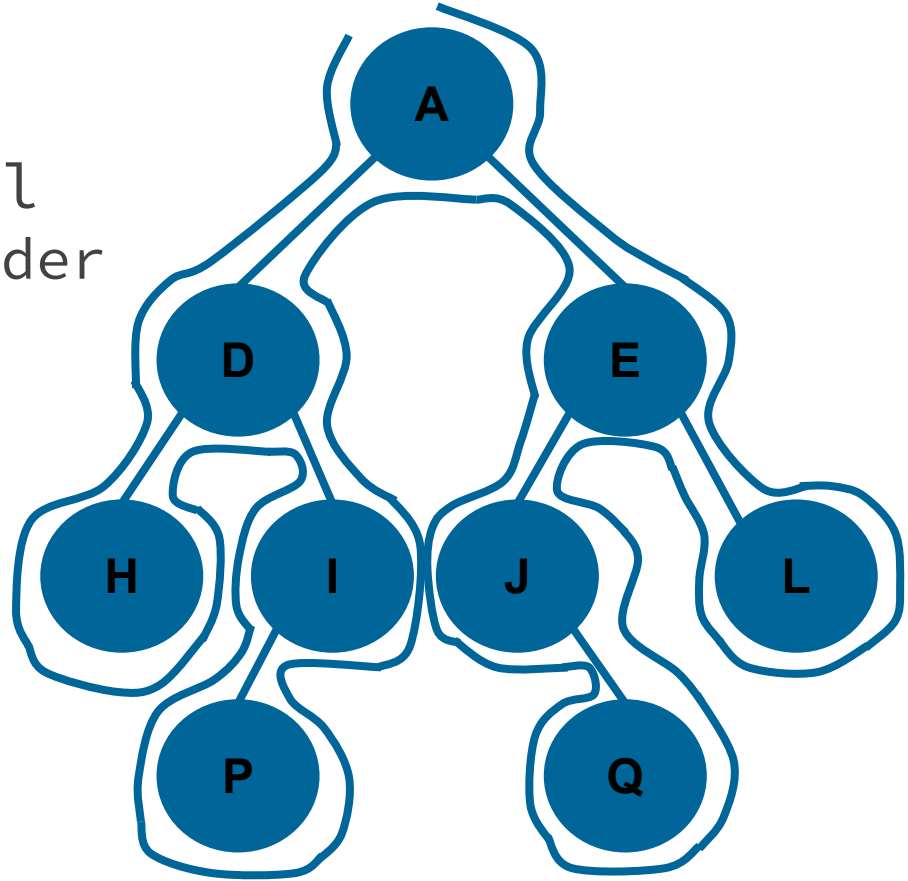
binary tree

- traversal
  - postorder



binary tree

- traversal
  - postorder



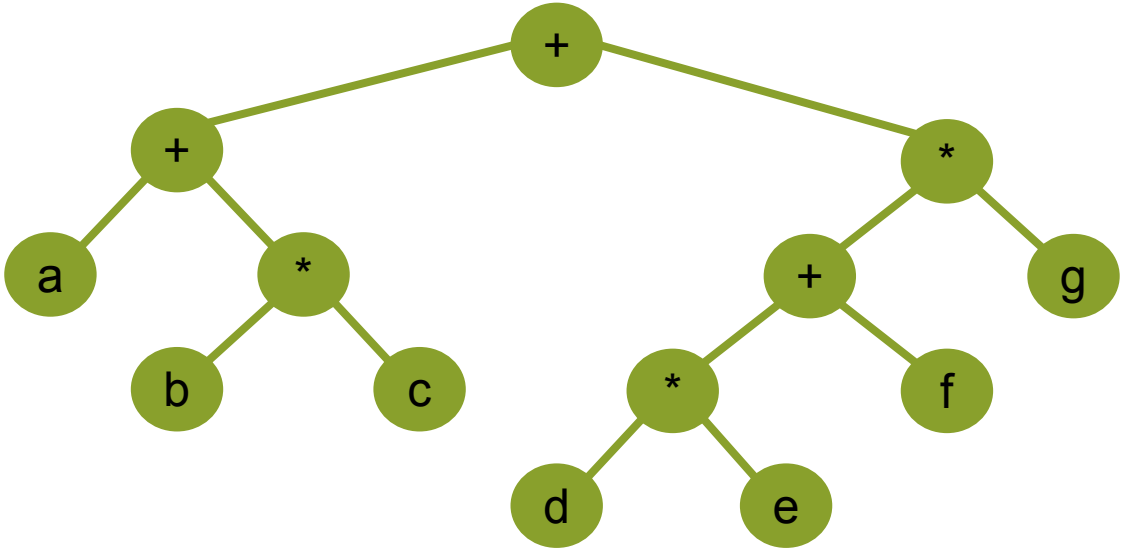
binary tree

- application
  - recall converting an infix expression to its postfix equivalent
    - use stack (reading assignment)
    - use of expression tree



binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)



binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)



binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

- expression tree
  - **(a+b*c)**+((d*e+f)*g)





binary tree

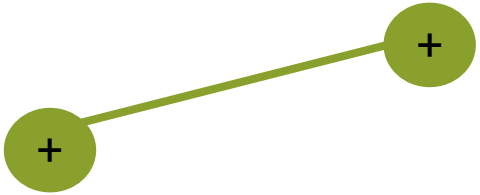
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

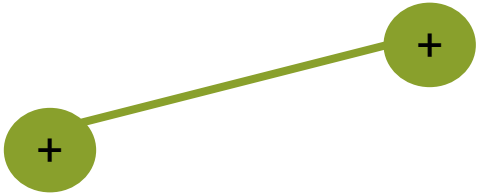
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

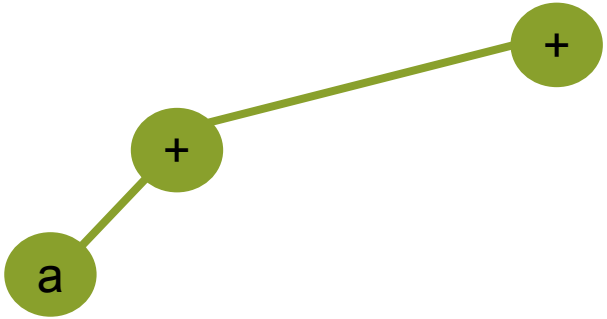
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

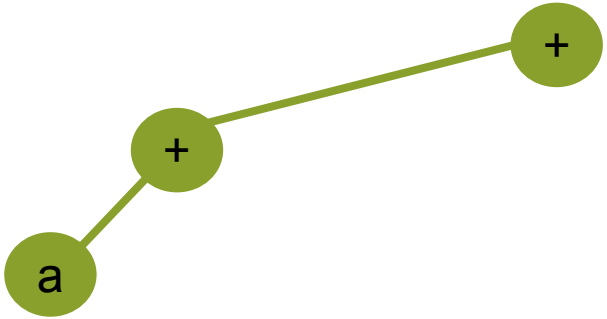
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

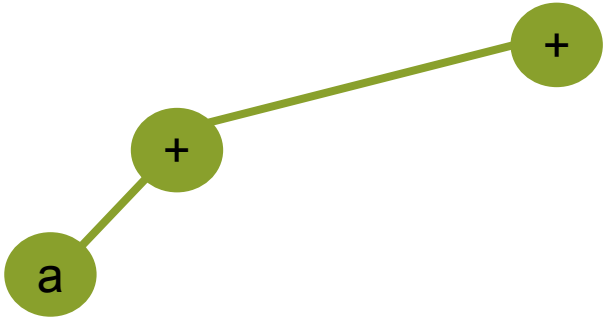
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

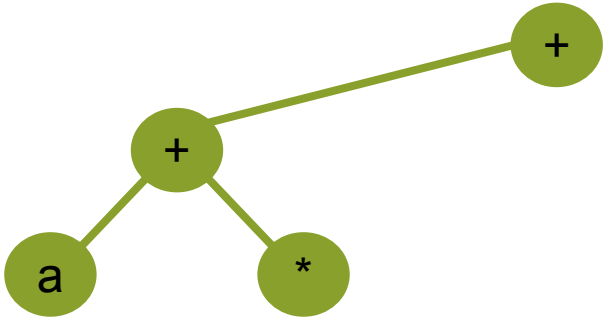
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

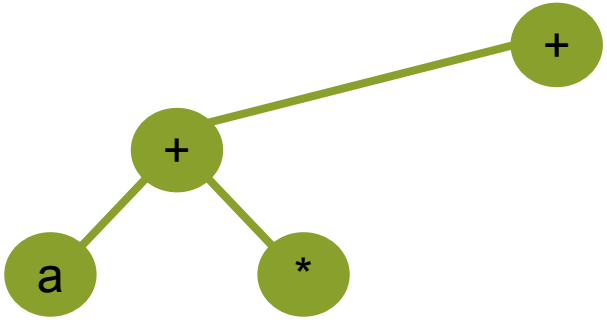
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

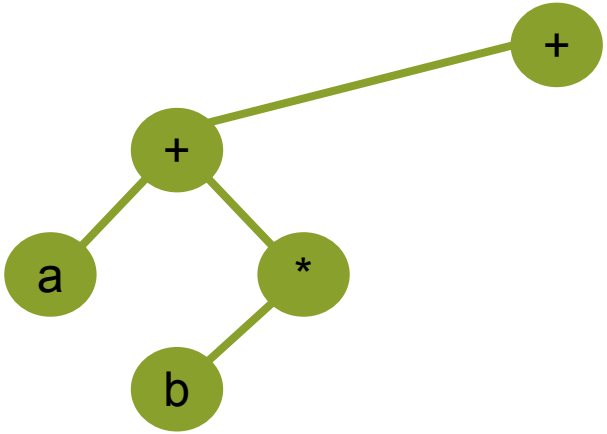
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

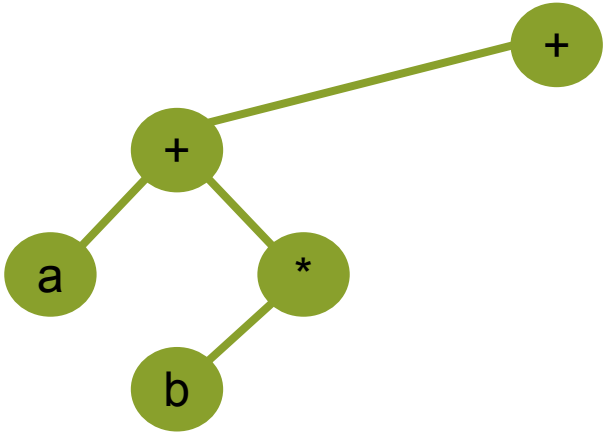
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

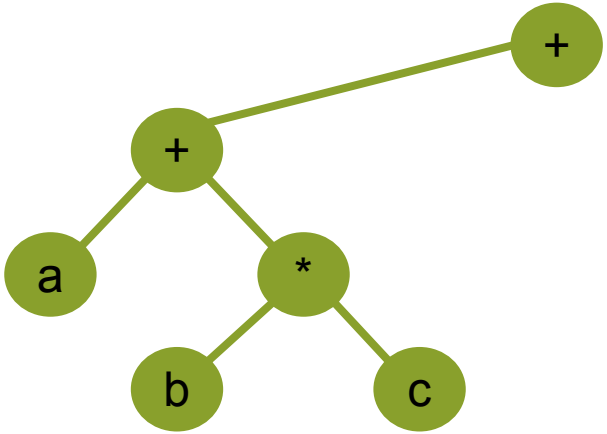
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

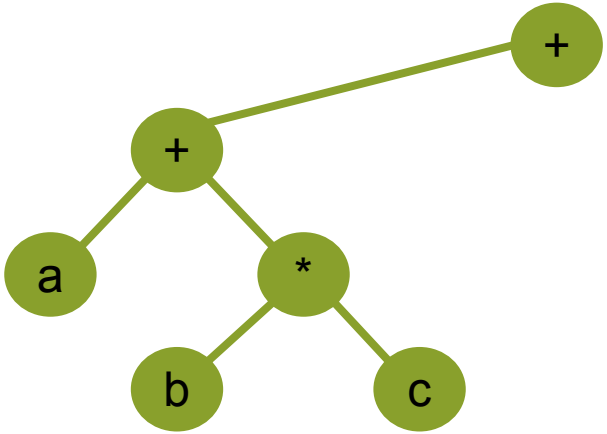
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

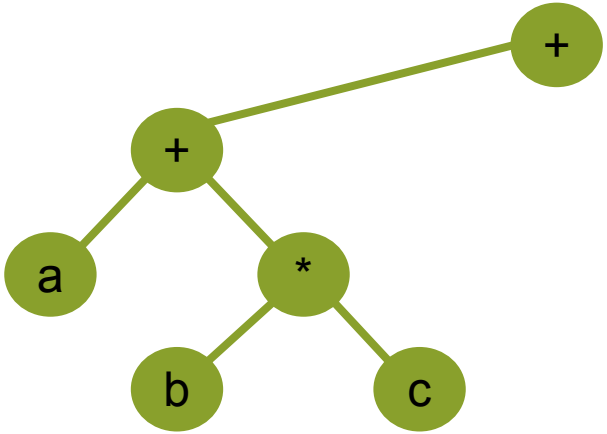
- expression tree
  - **(a+b*c)**+((d*e+f)*g)





binary tree

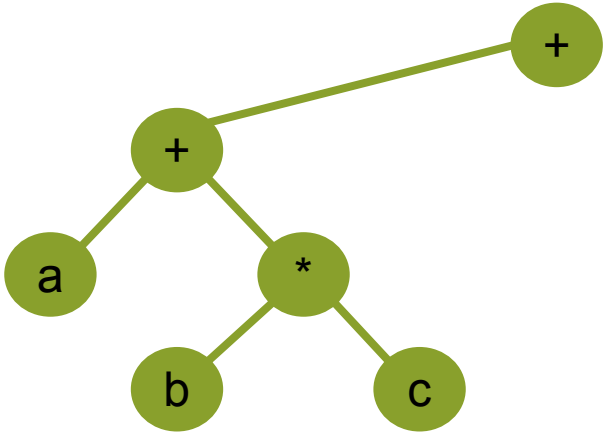
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

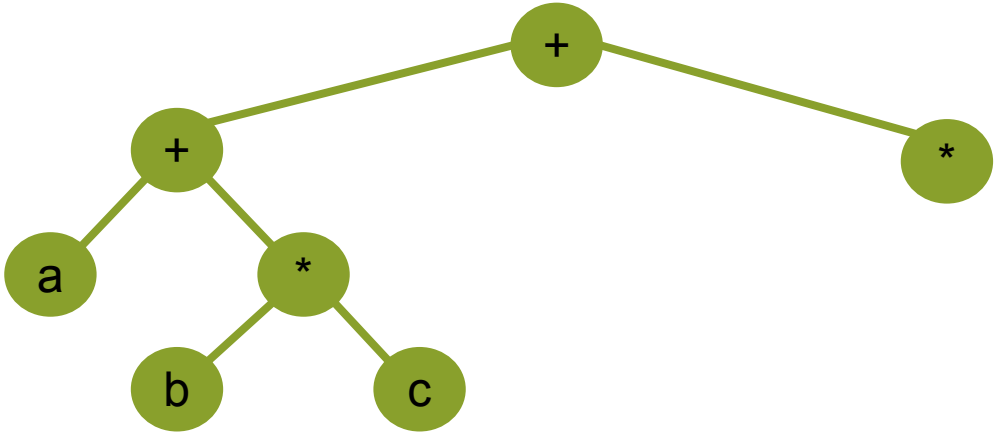
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

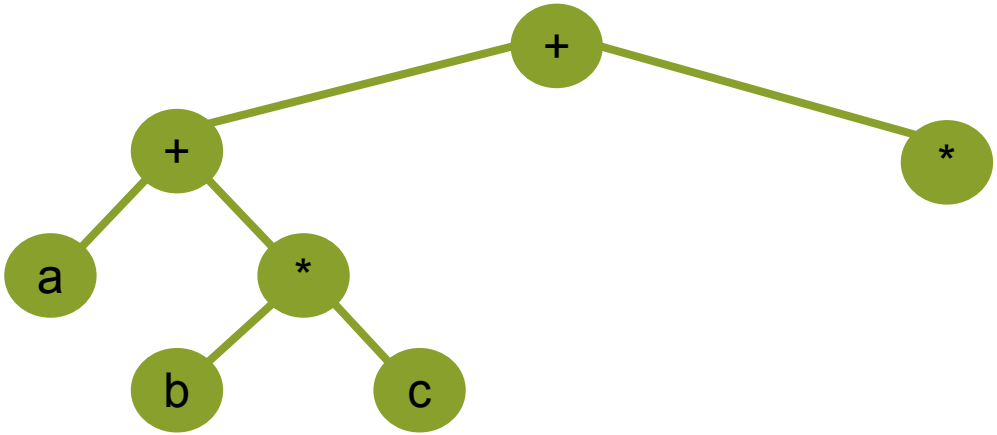
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

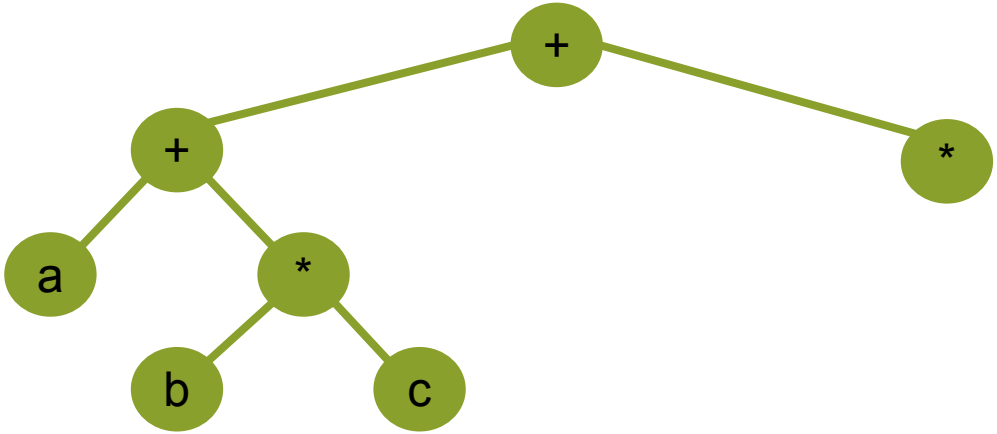
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

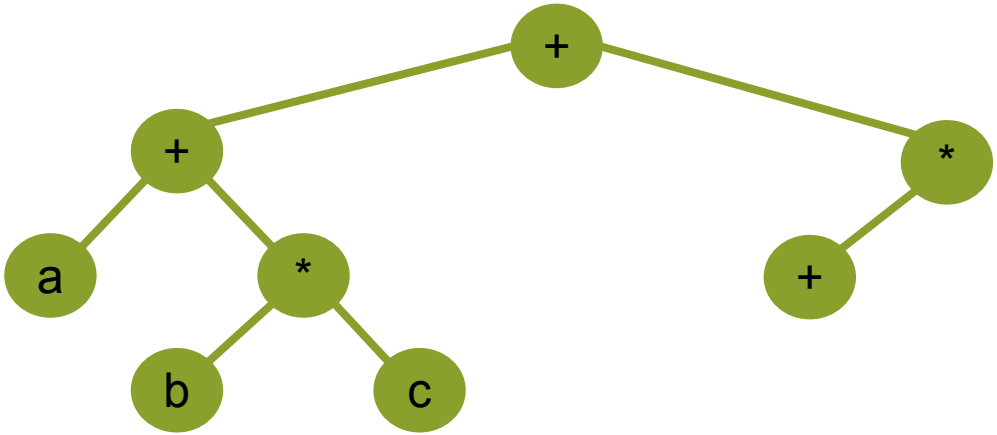
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

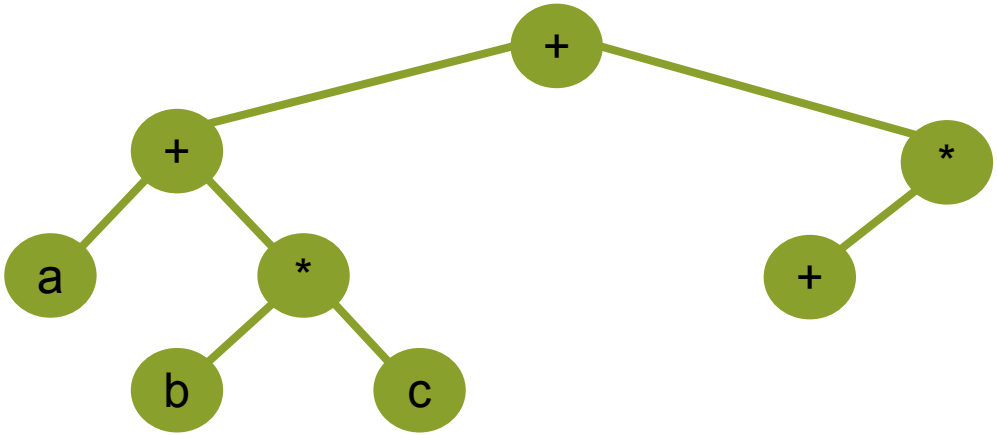
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

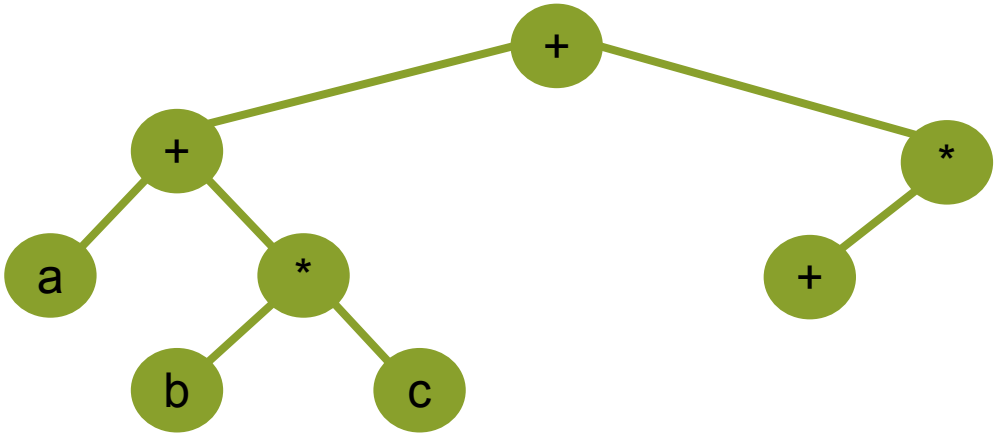
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

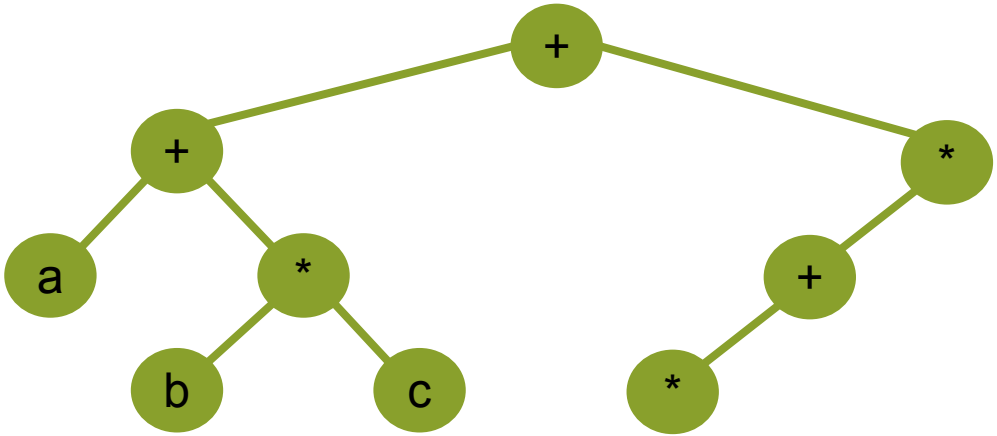
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

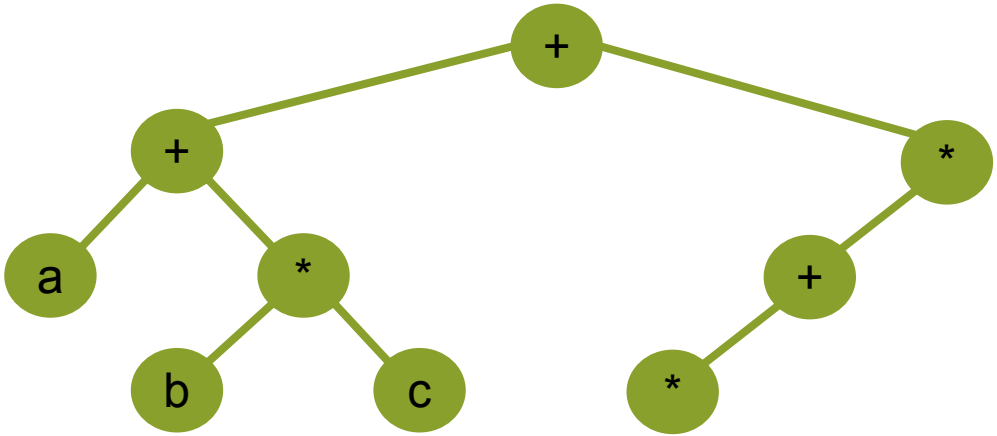
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

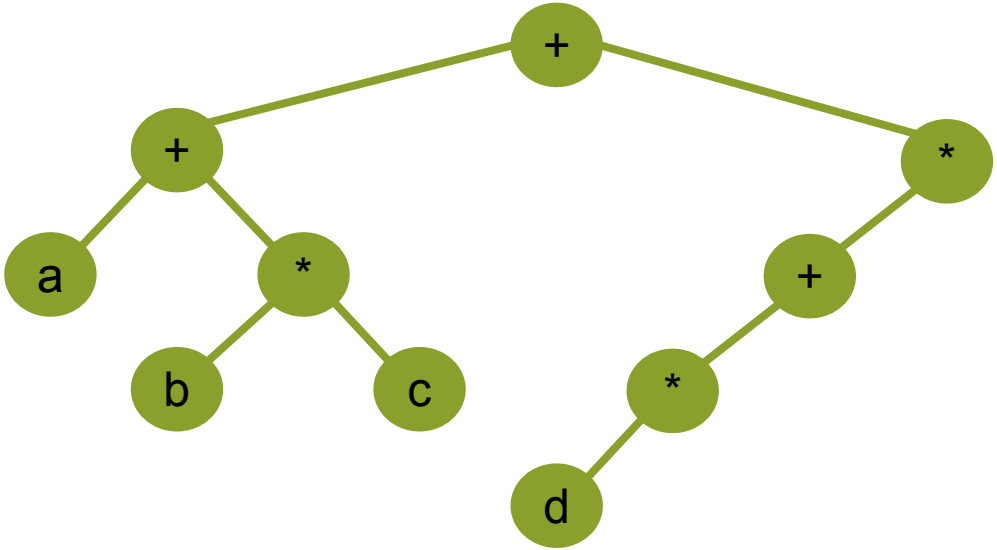
- expression tree
  - (a+b*c)+((d*e+f)*g)





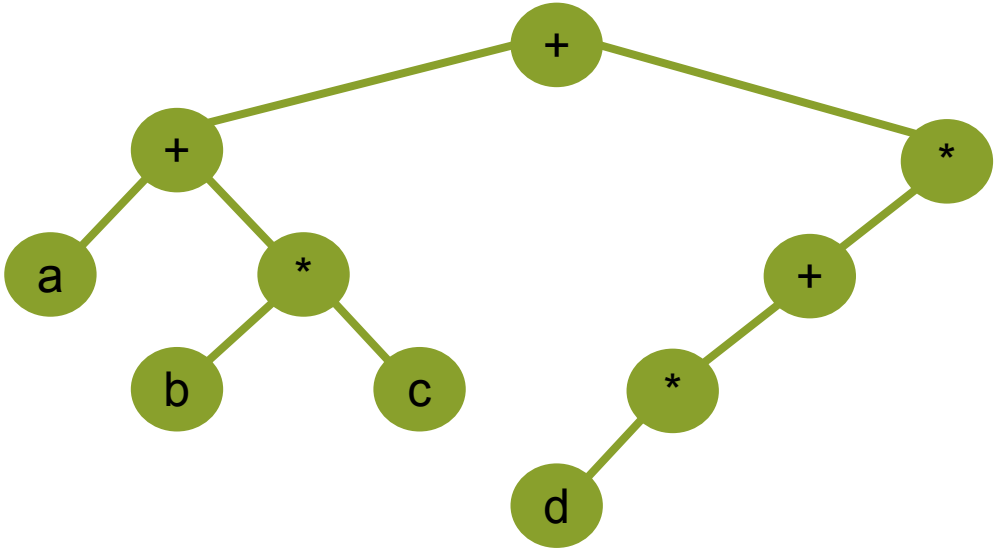
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)



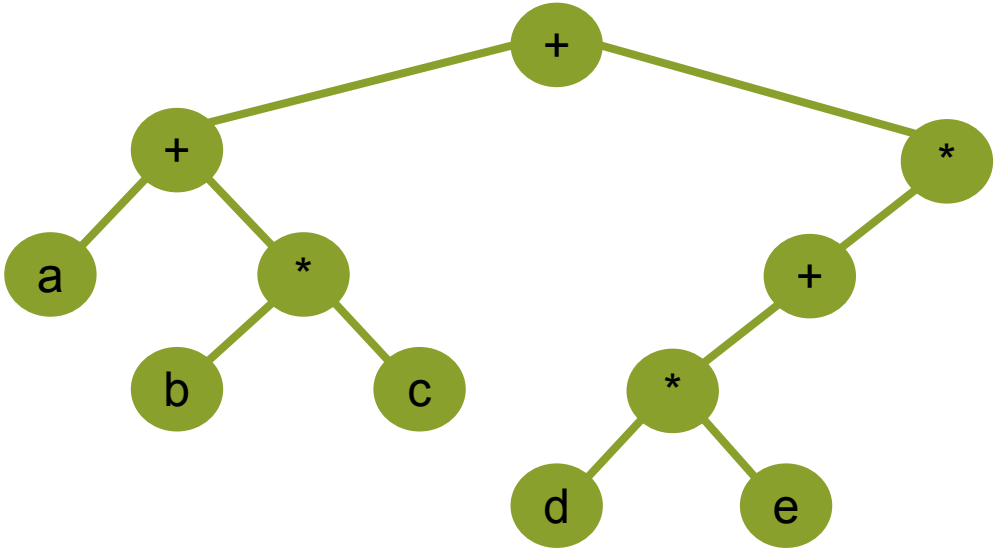
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)



binary tree

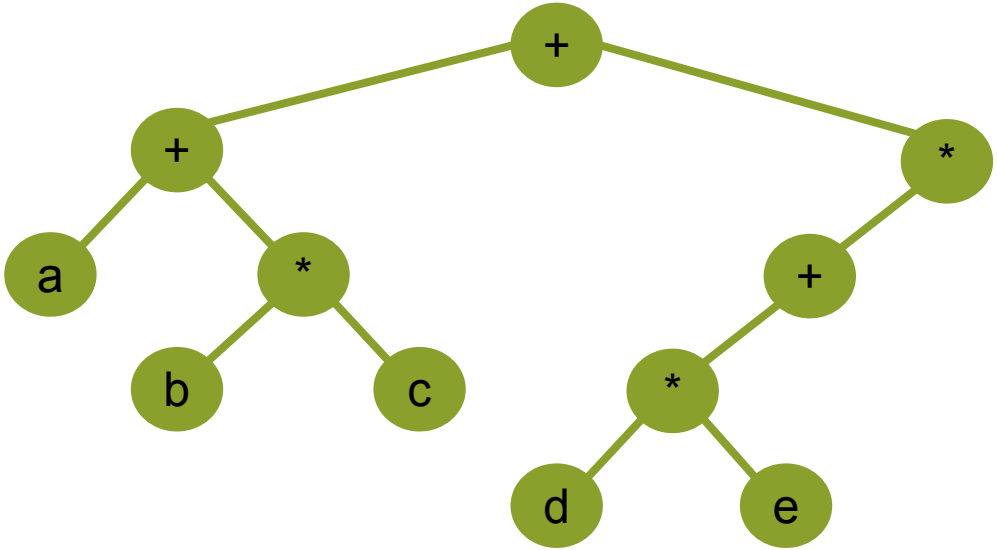
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

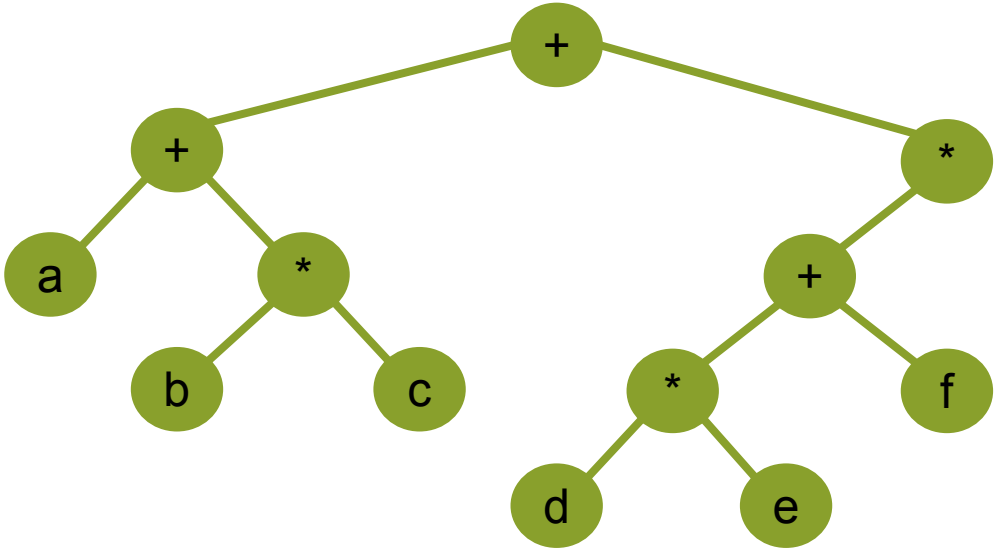
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

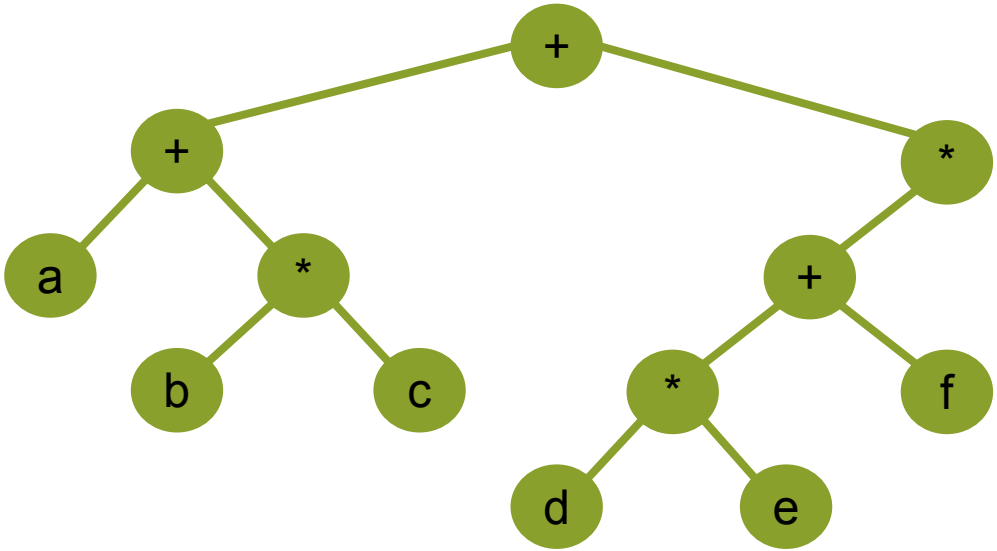
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

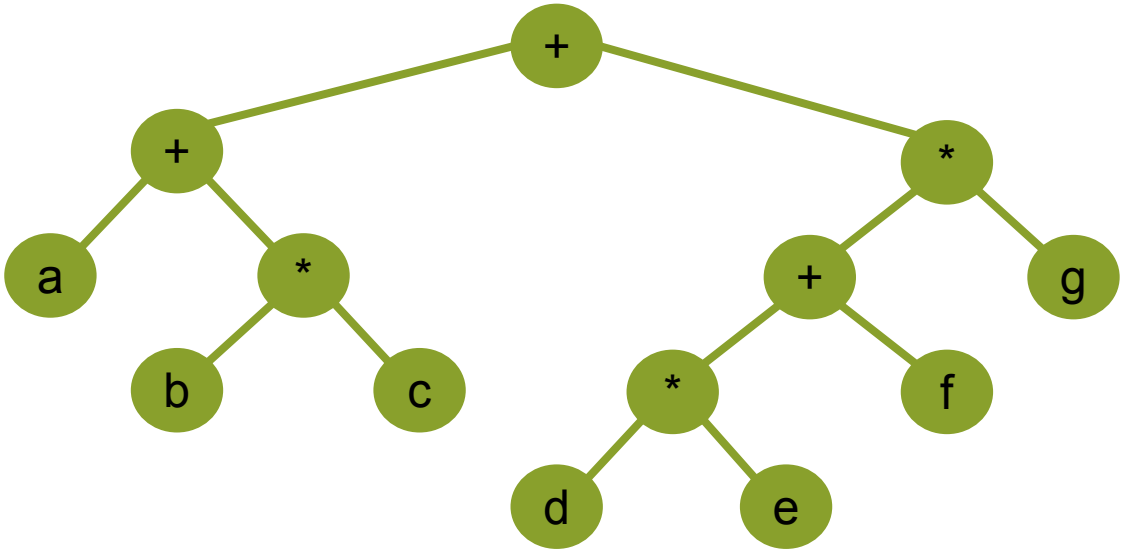
- expression tree
  - (a+b*c)+((d*e+f)*g)





binary tree

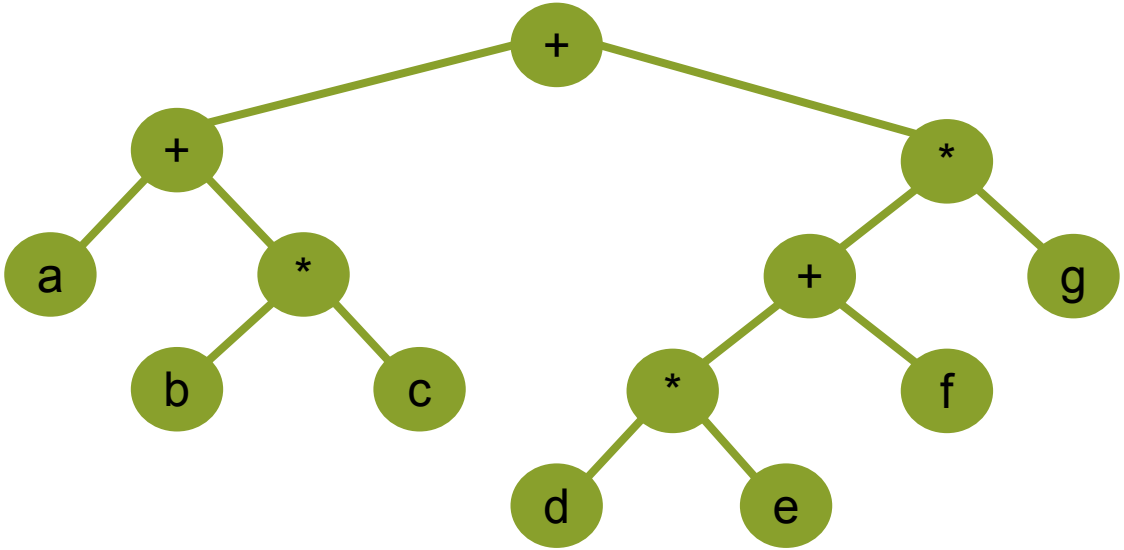
- expression tree
  - (a+b*c)+((d*e+f)*g)





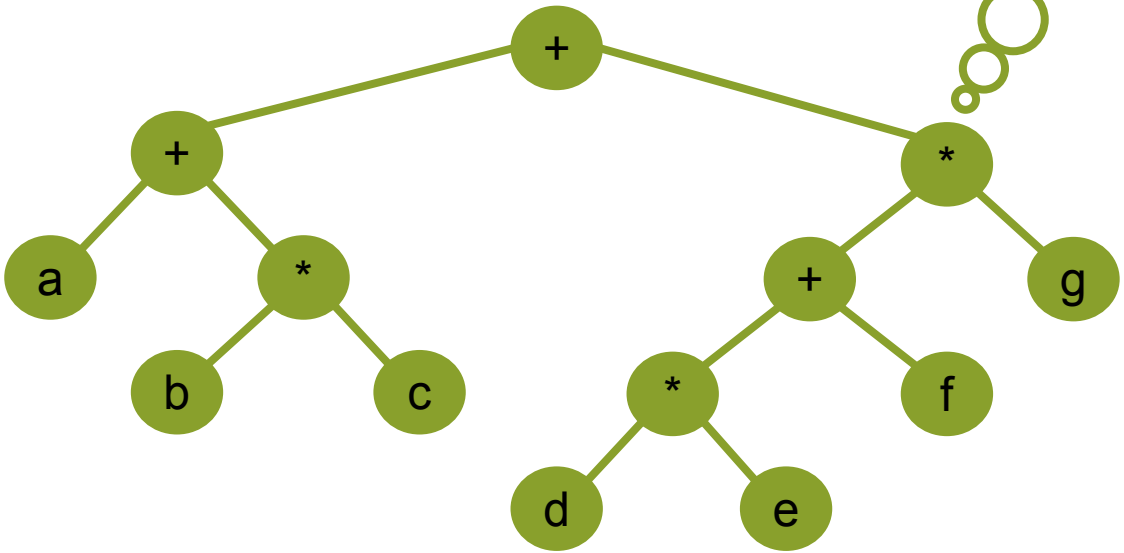
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)



binary tree

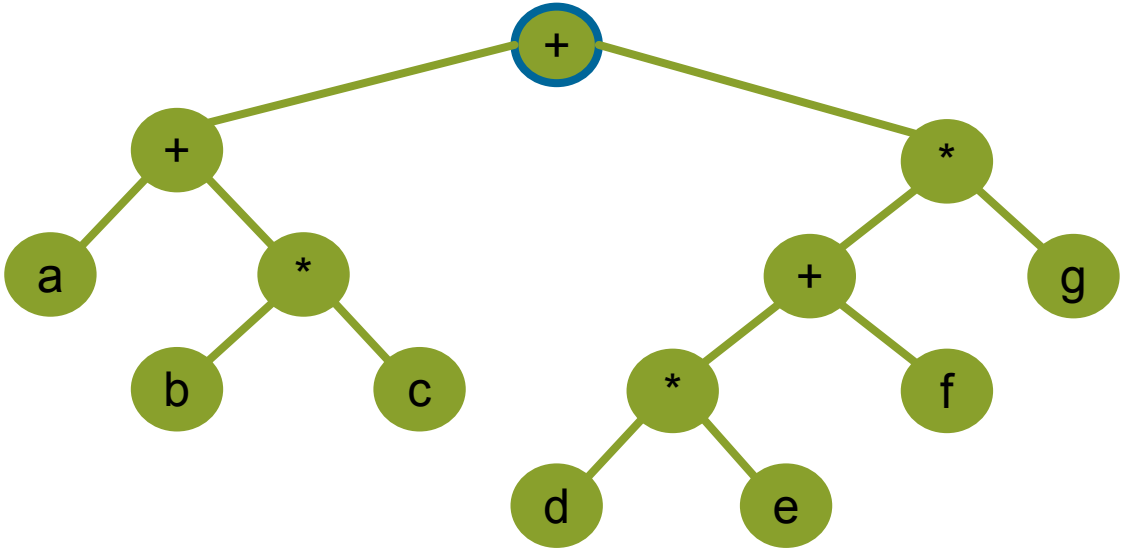
- expression tree
  - (a+b*c)+((d*e+f)*g)



When the postfix expression is desired, what traversal should be executed?

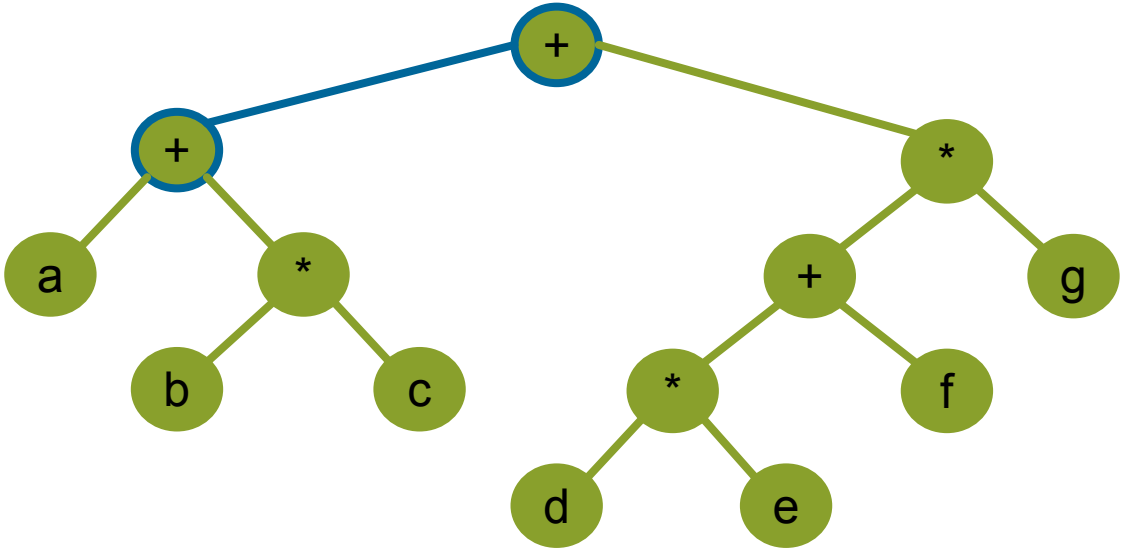
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  -



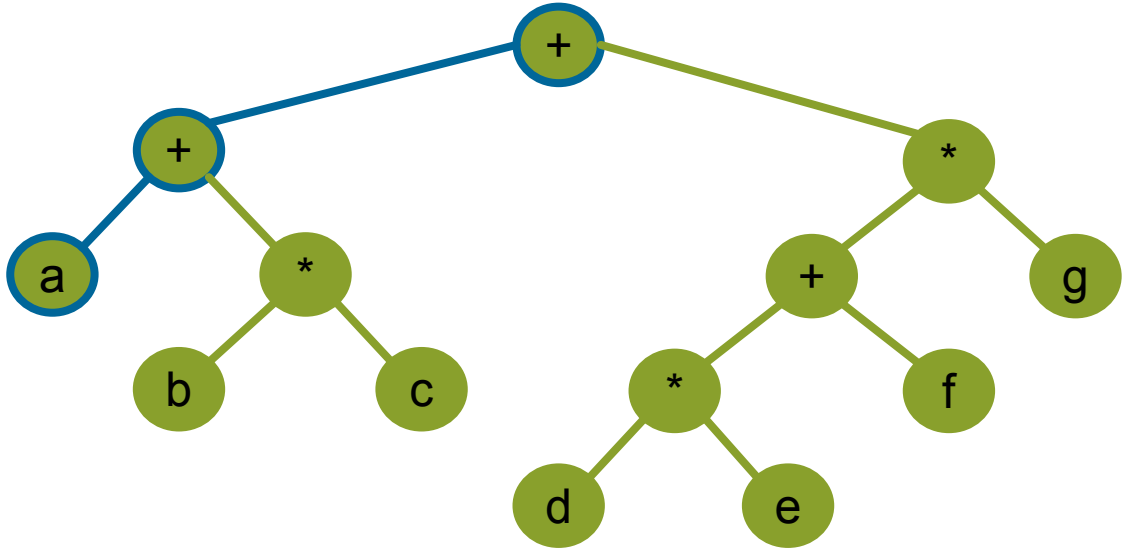
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  -



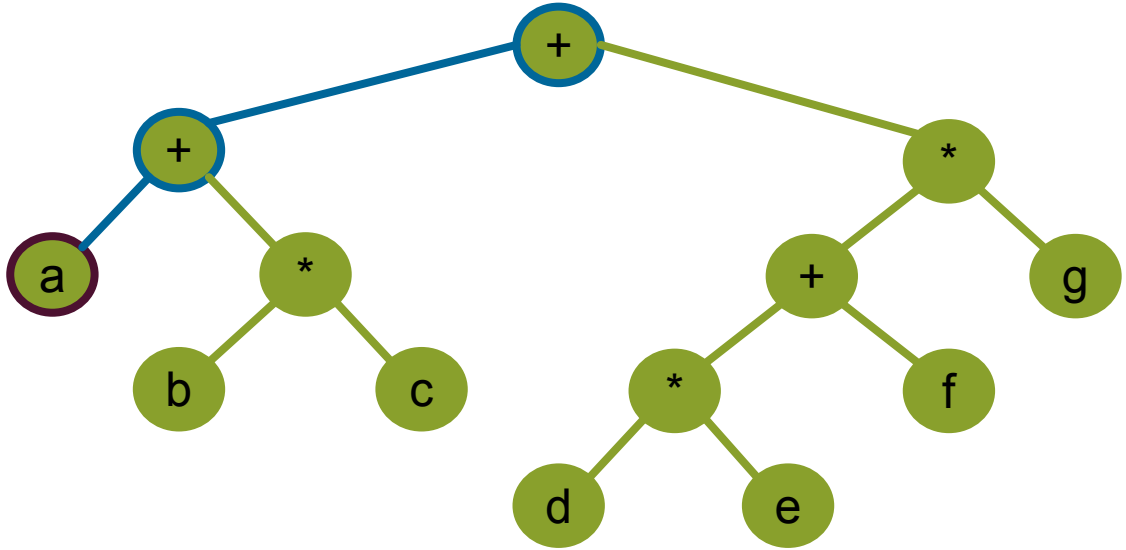
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  -



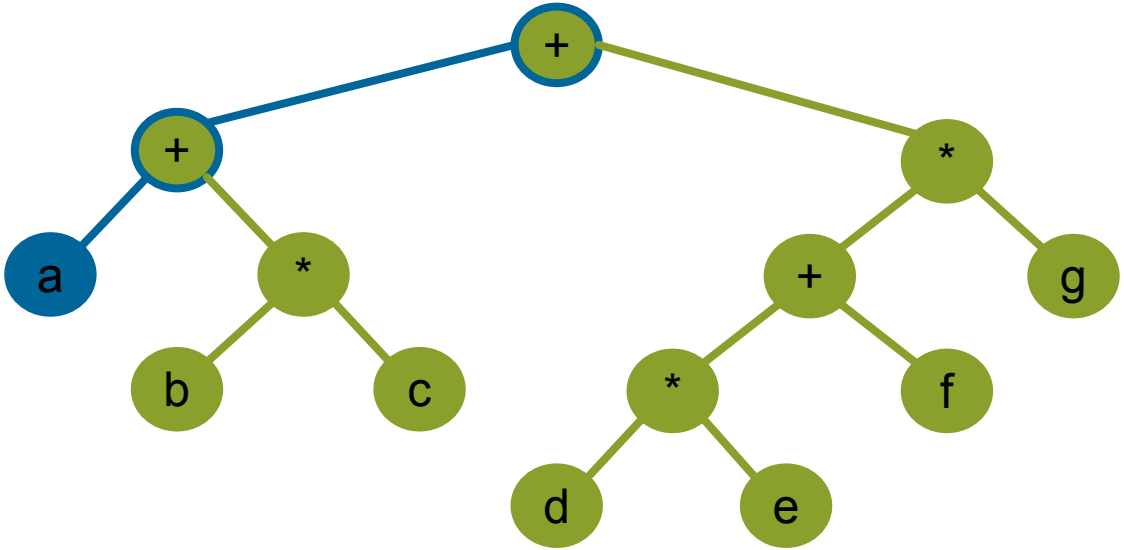
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  -



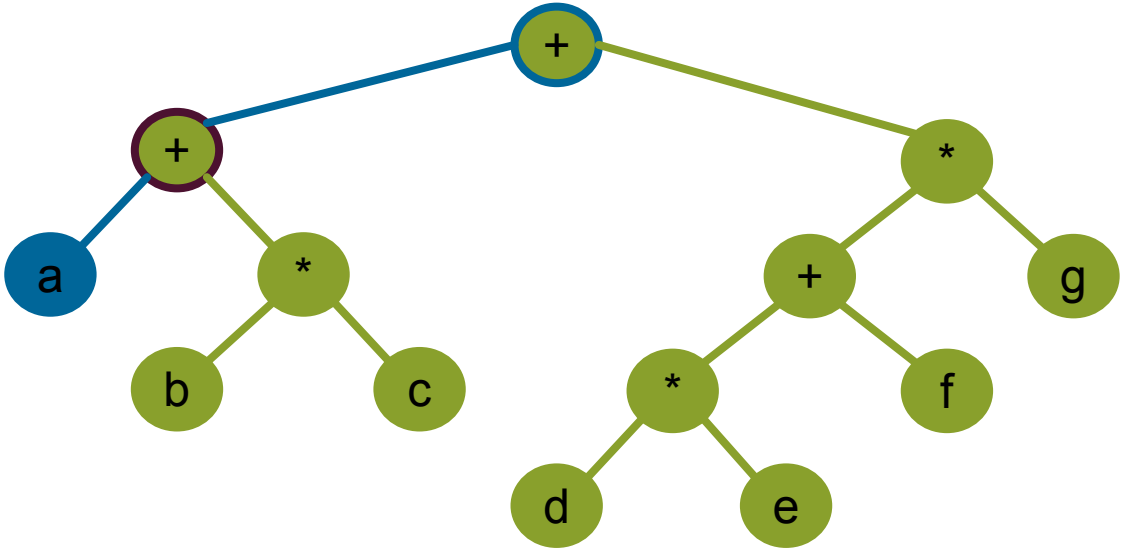
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a



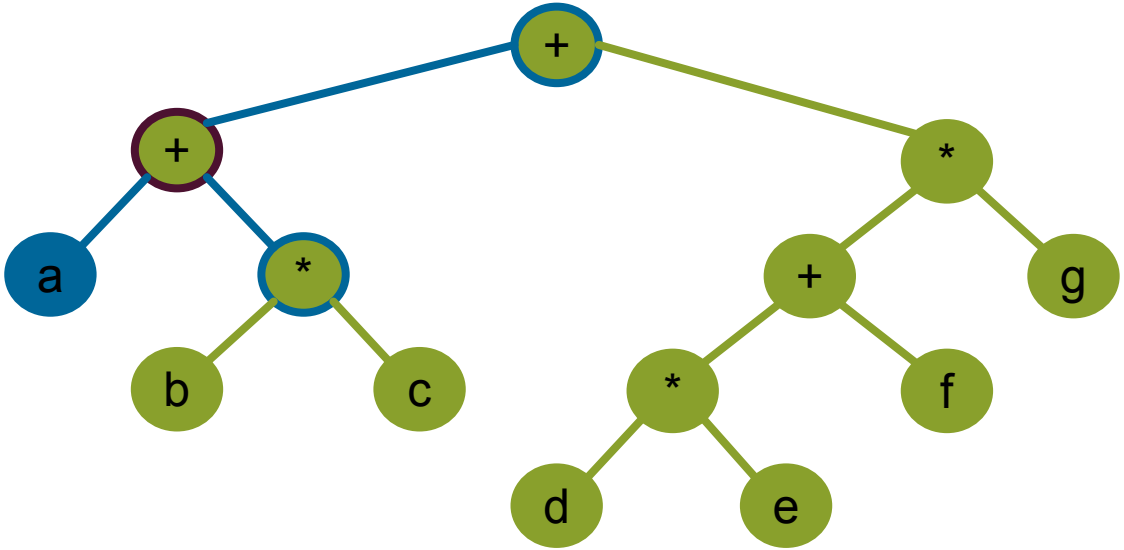
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a



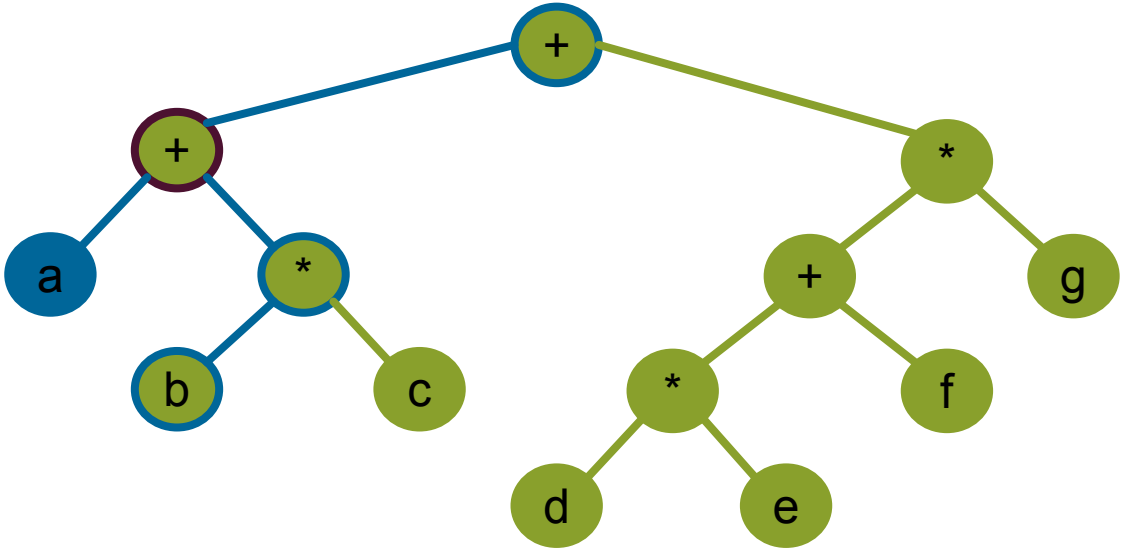
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a



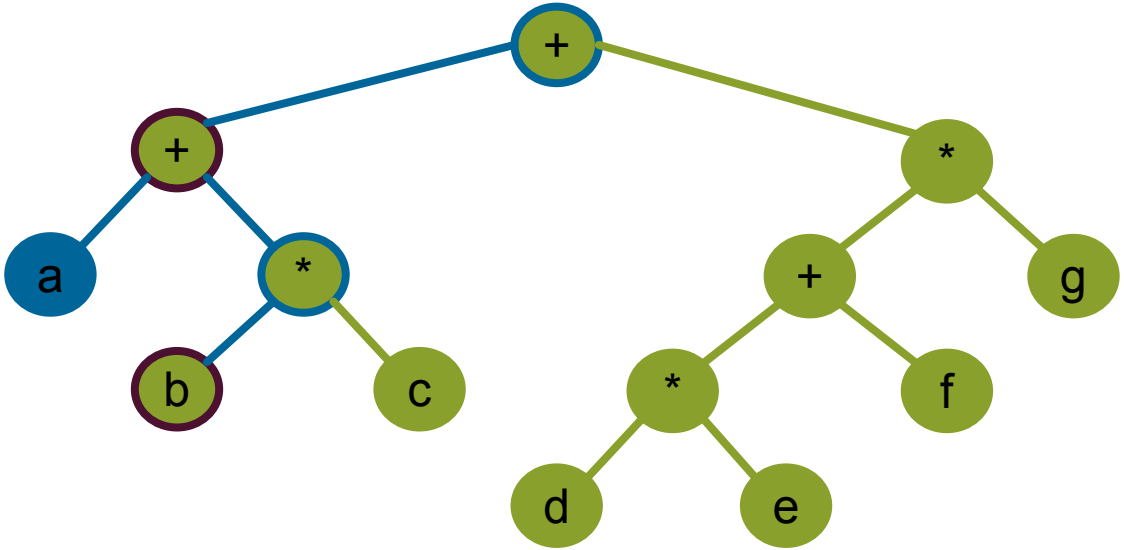
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a



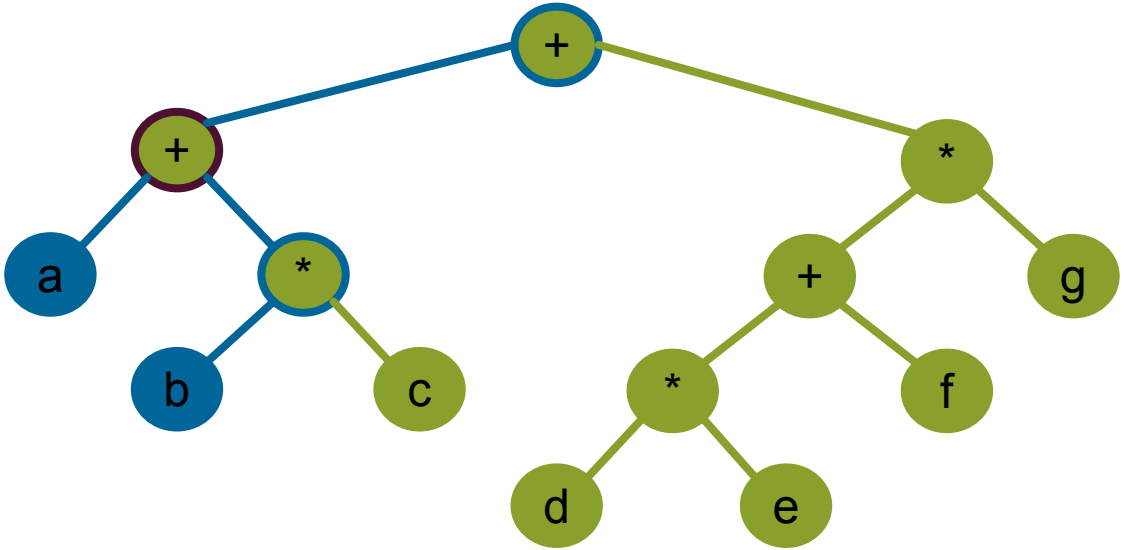
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a



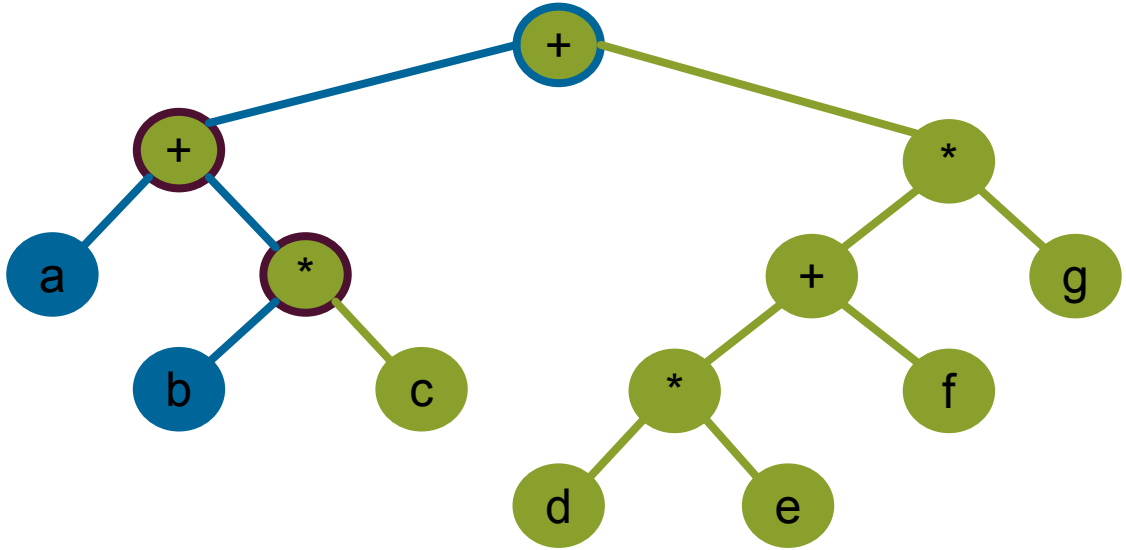
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b



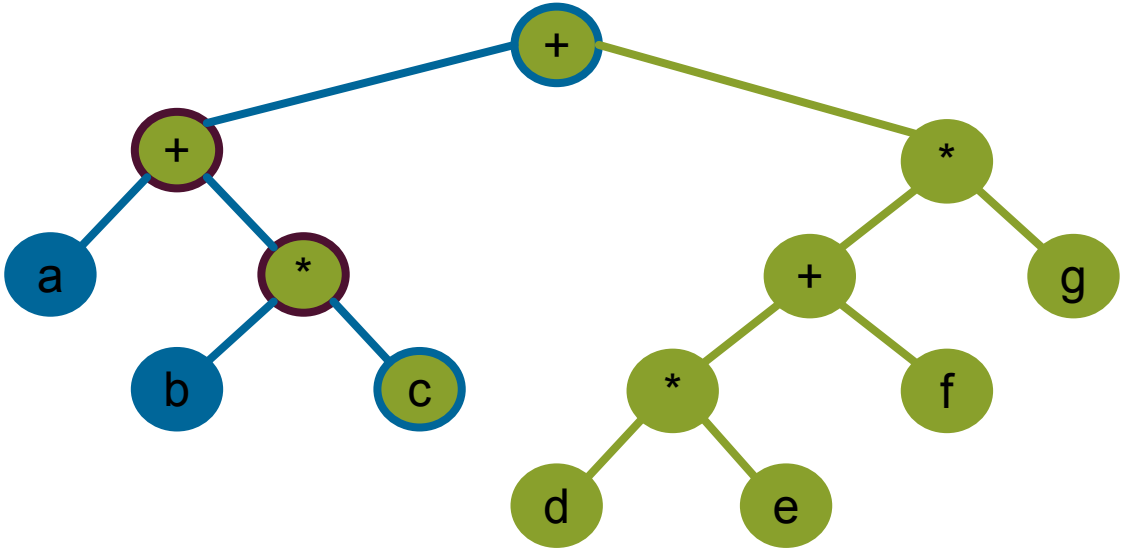
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b



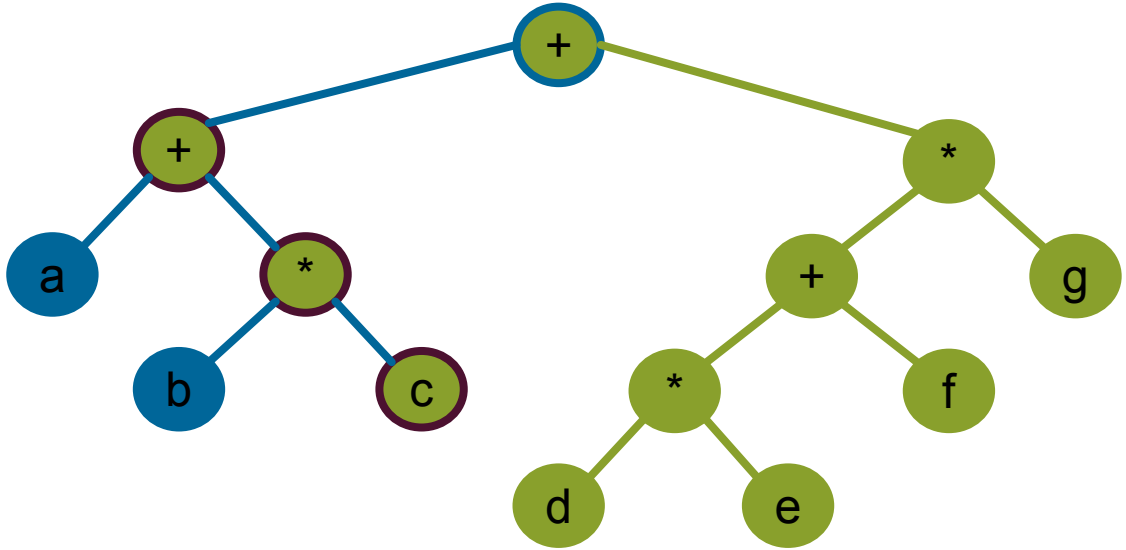
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b



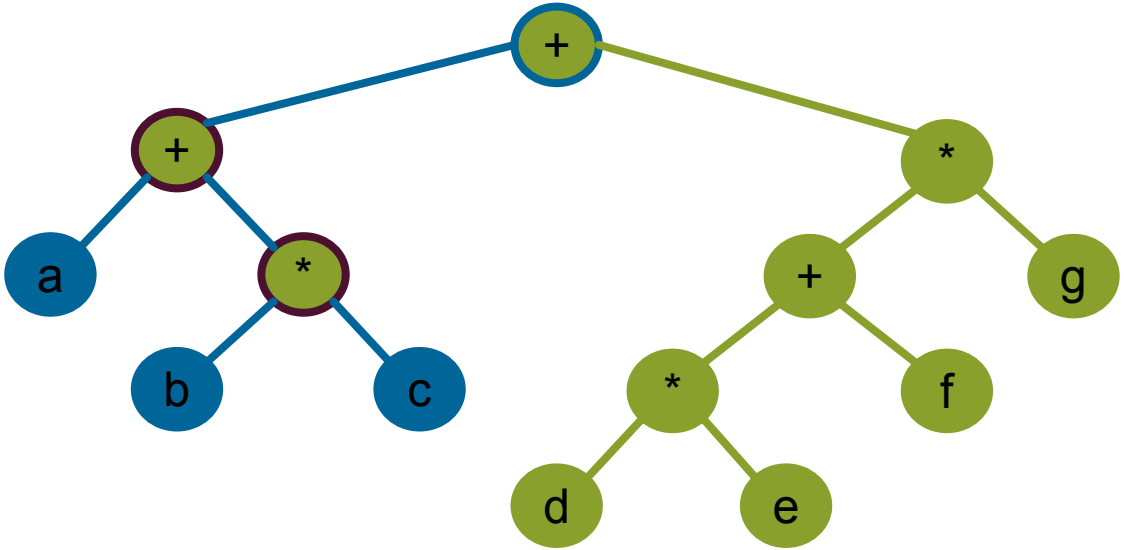
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b



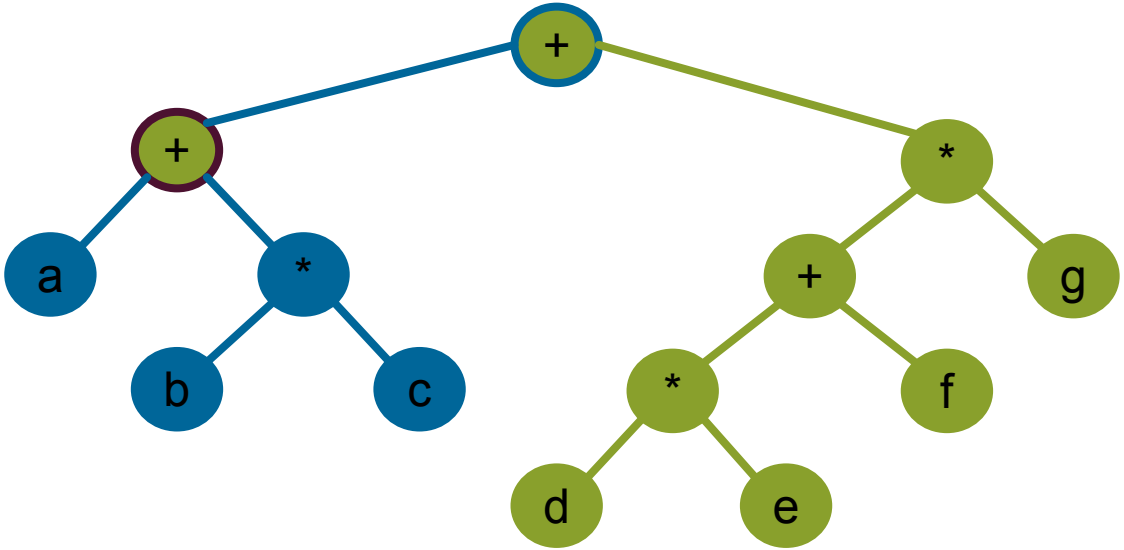
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c



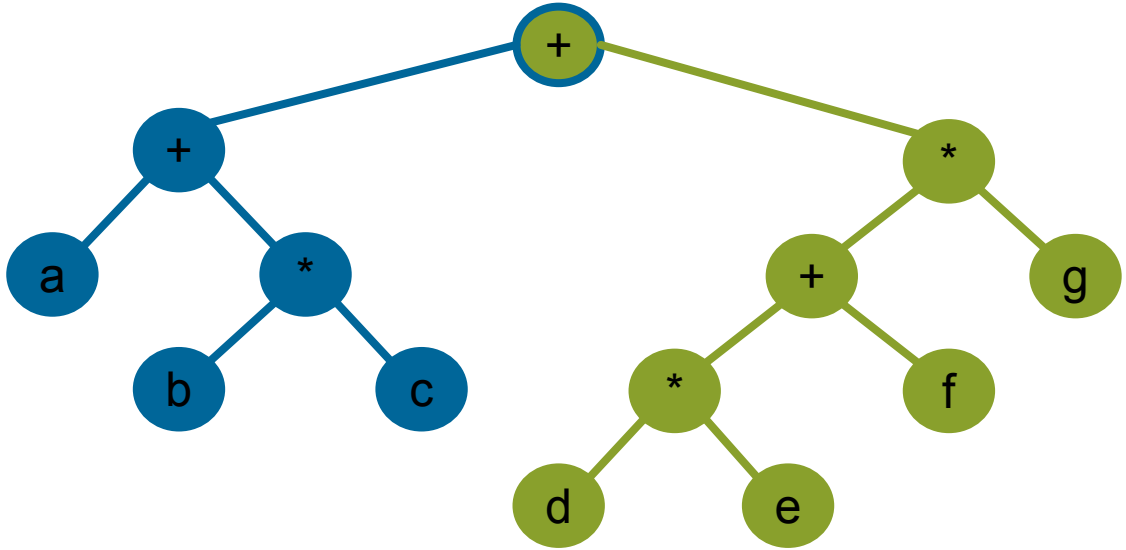
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c *



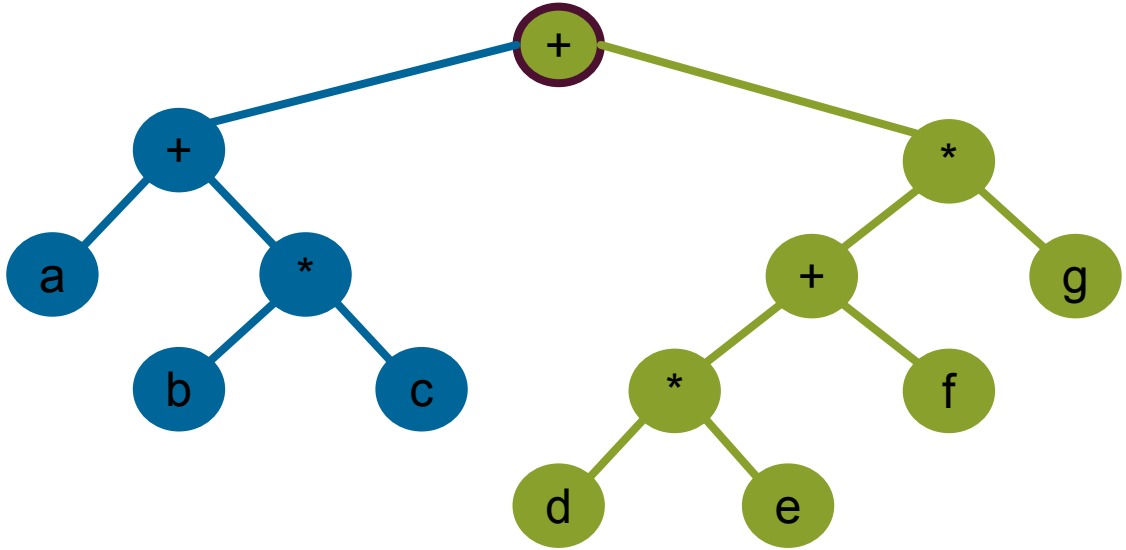
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



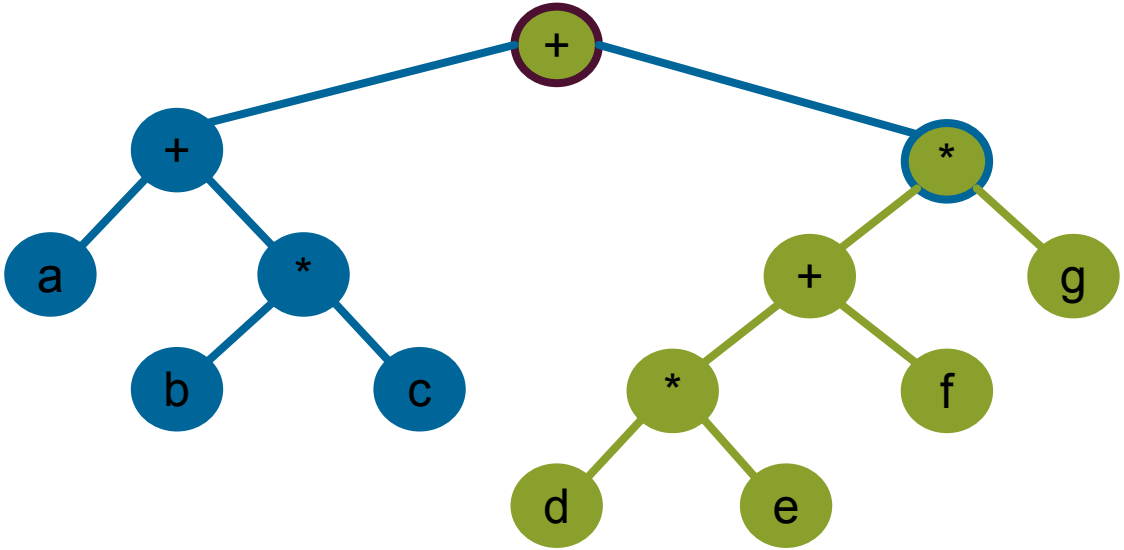
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



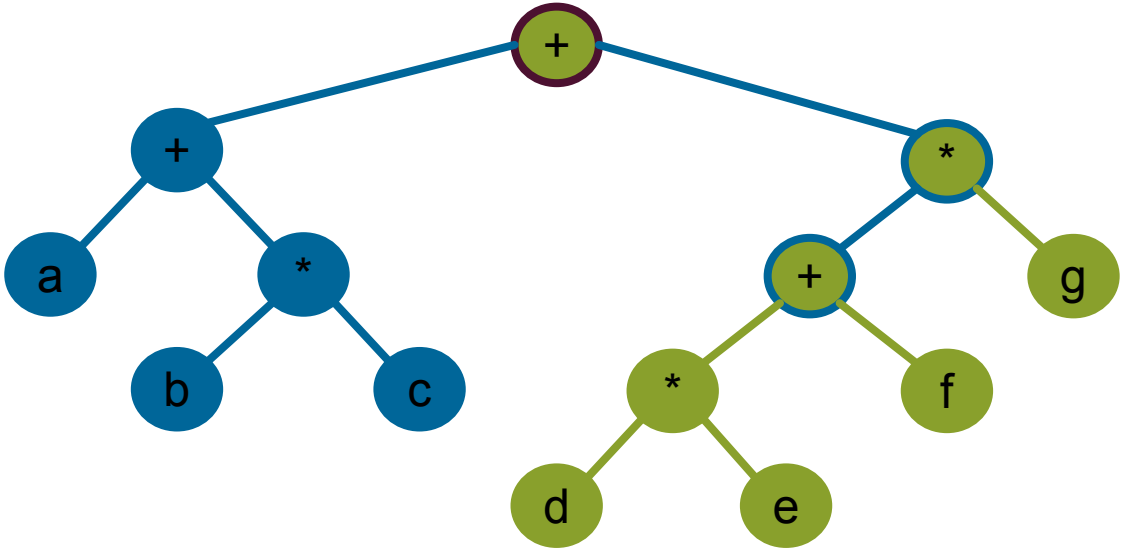
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



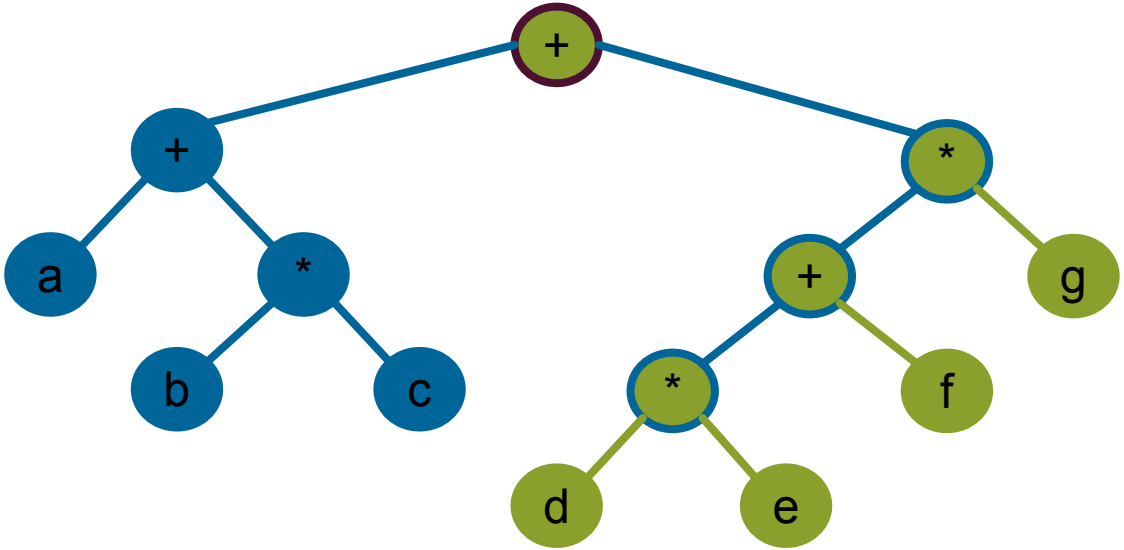
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



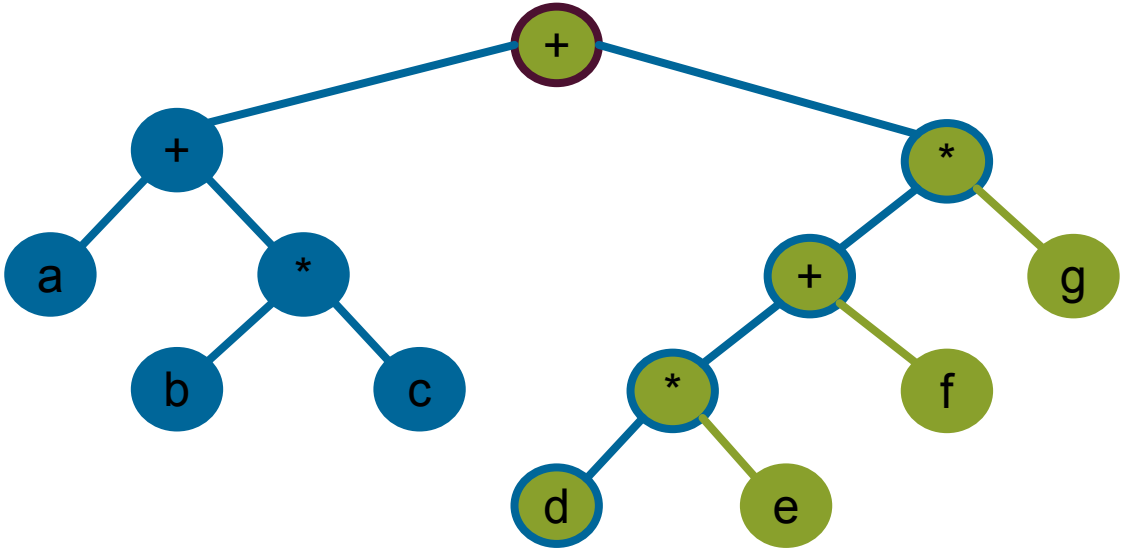
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



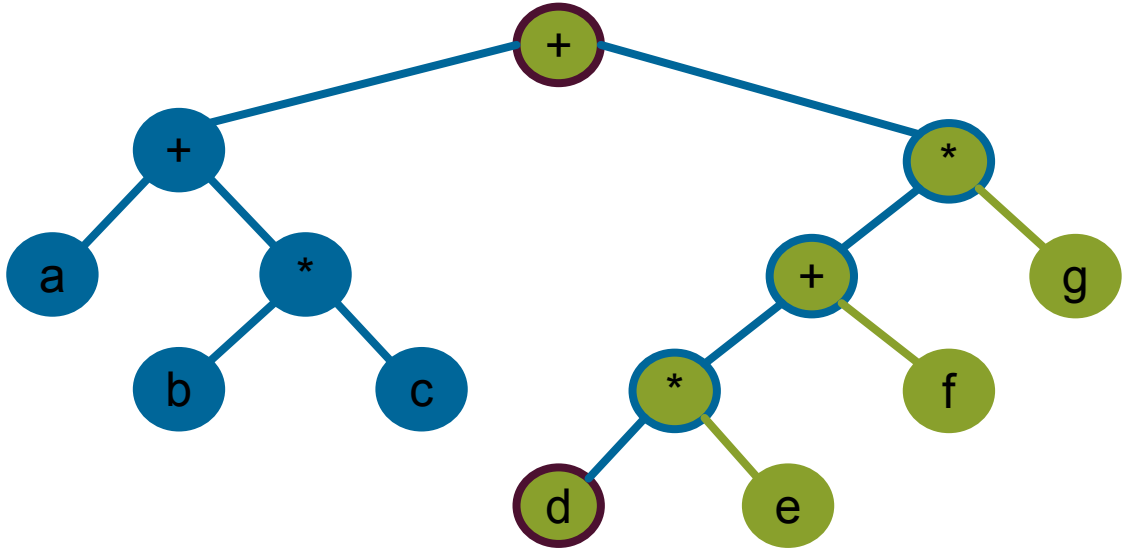
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



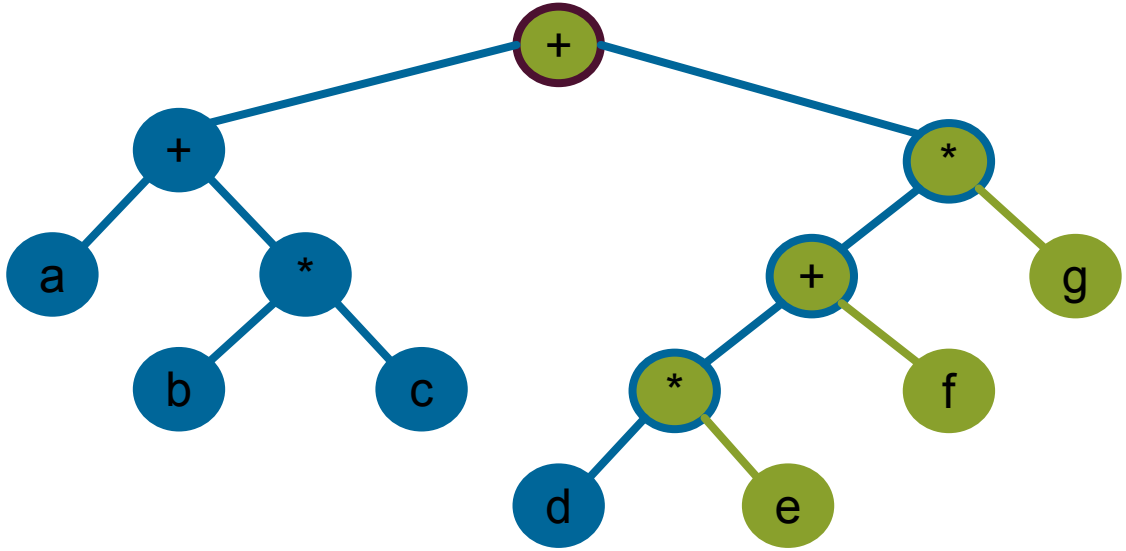
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * +



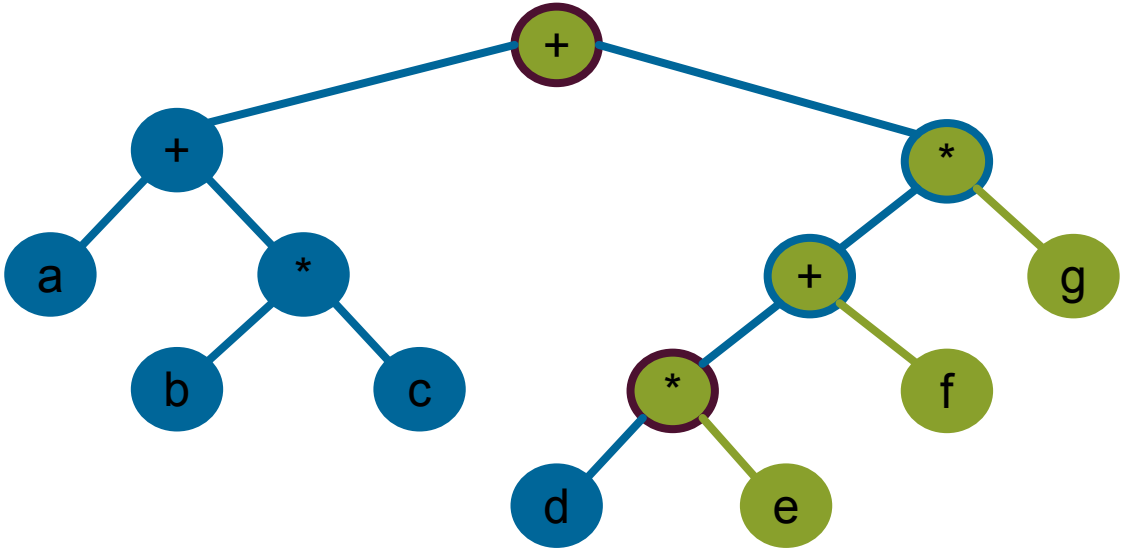
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d



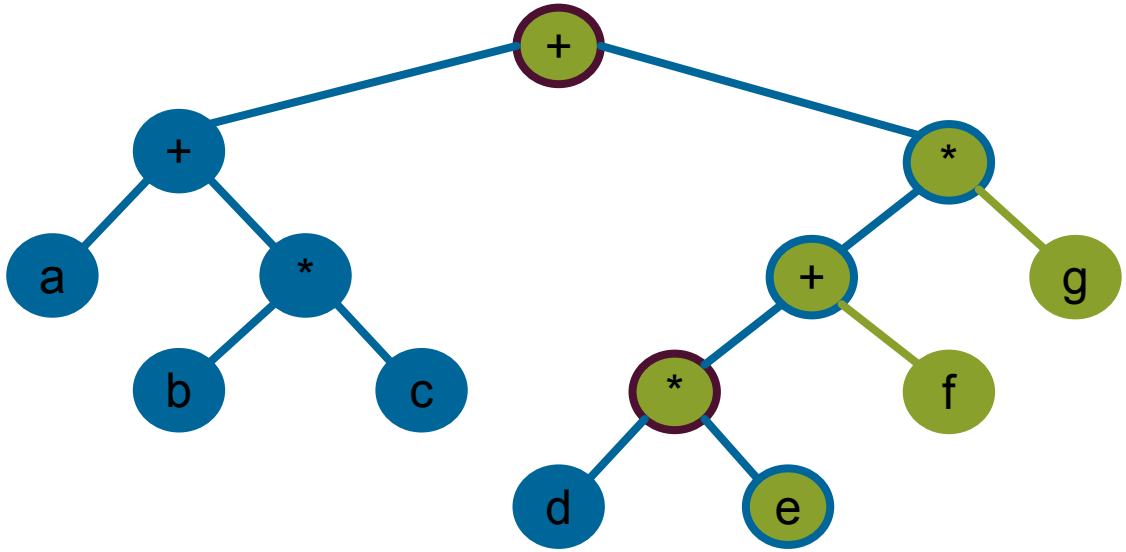
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d



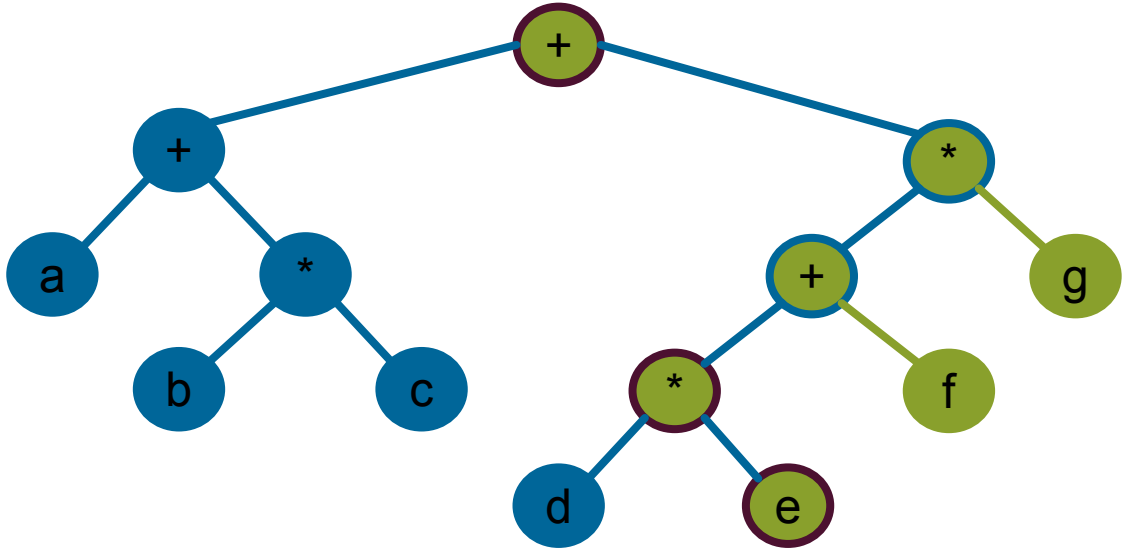
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d



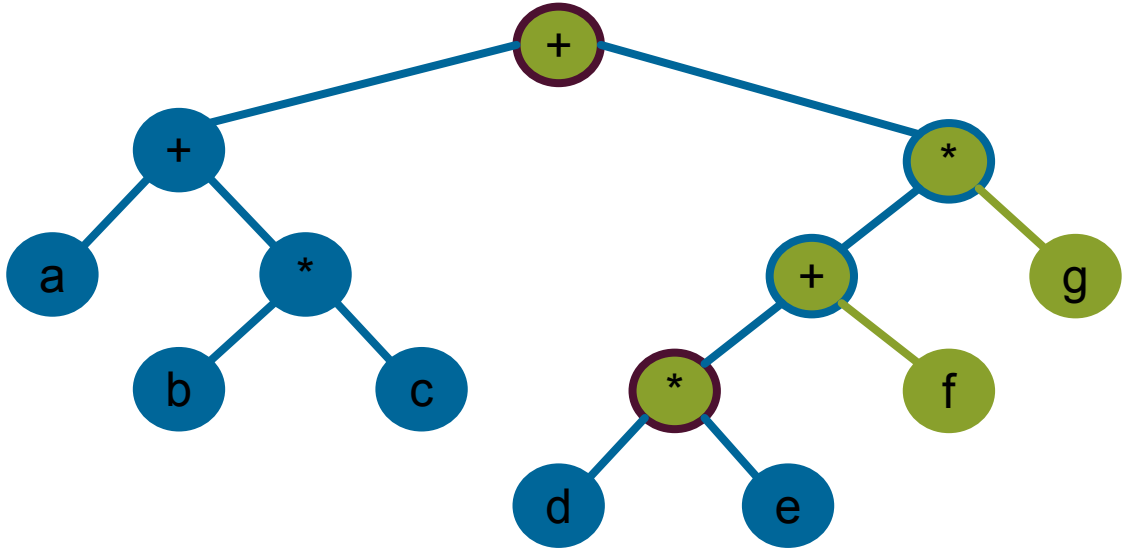
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d



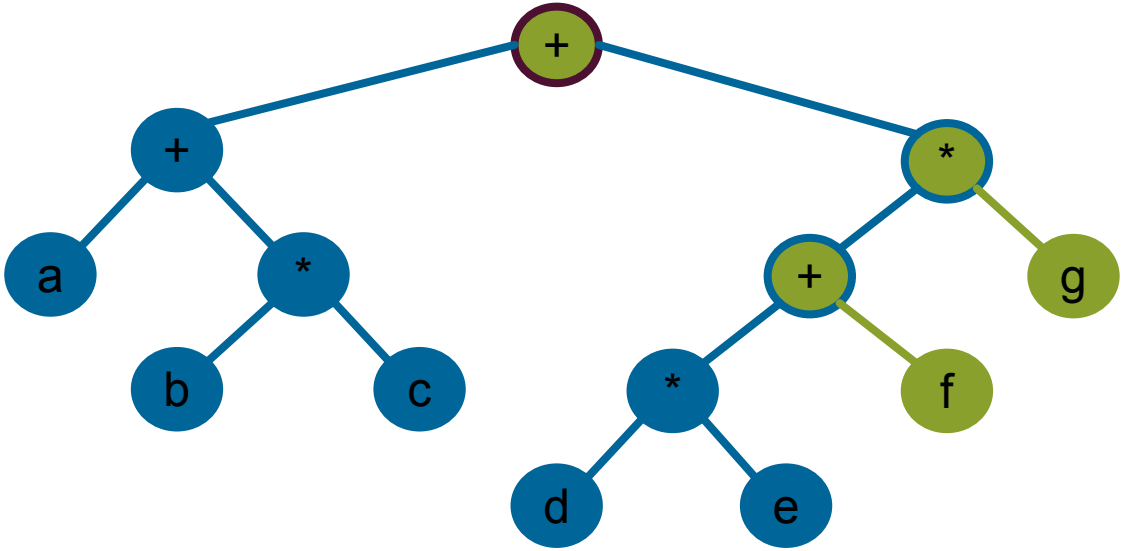
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e



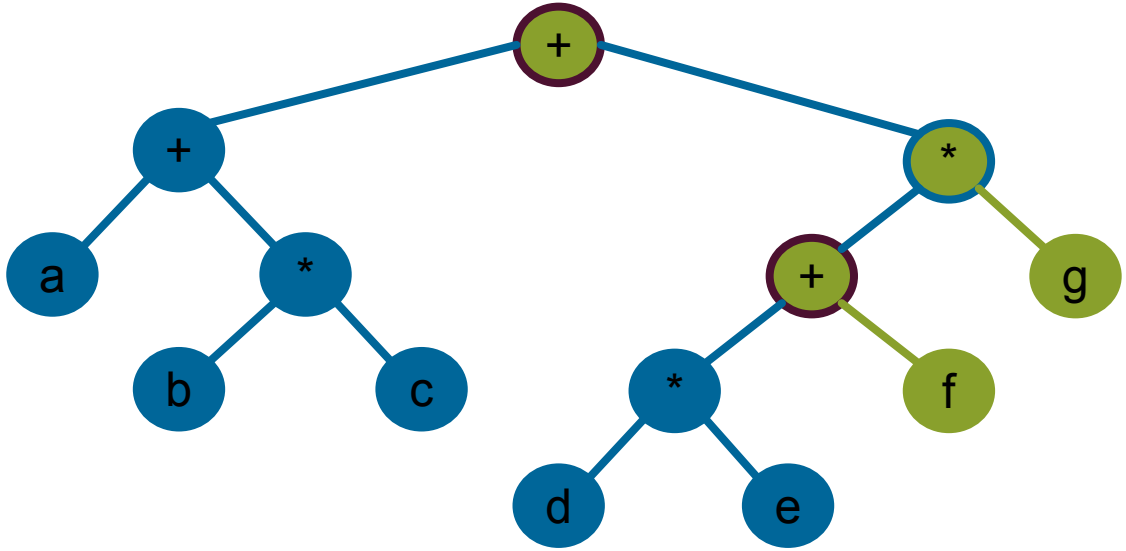
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * *



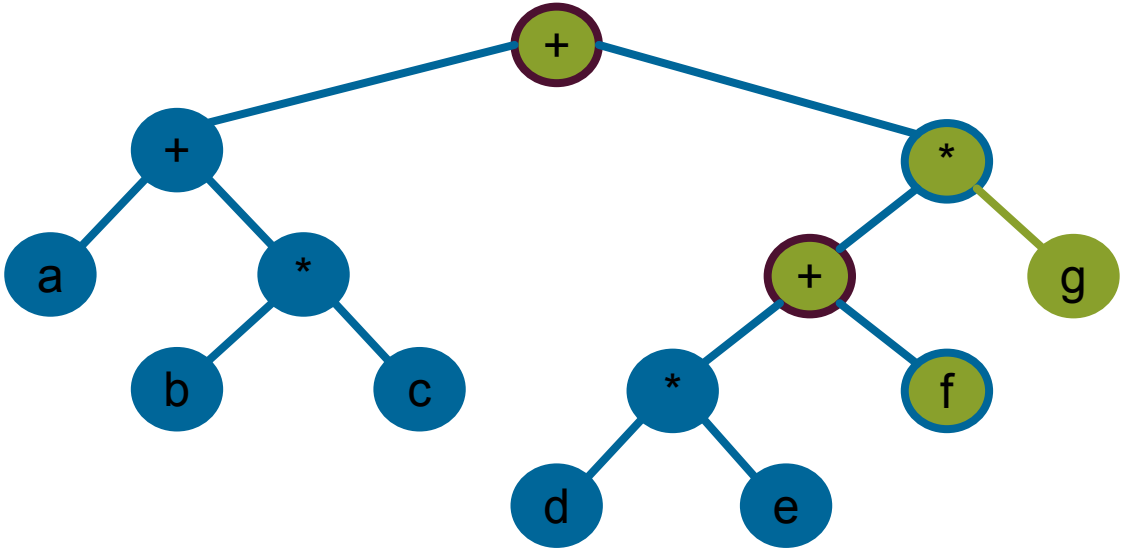
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * *



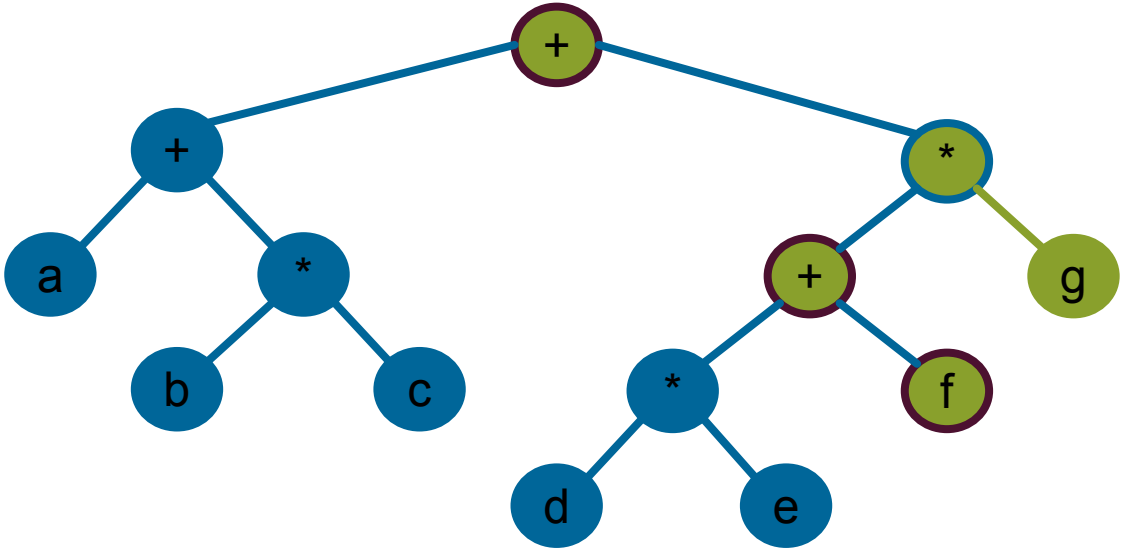
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * *



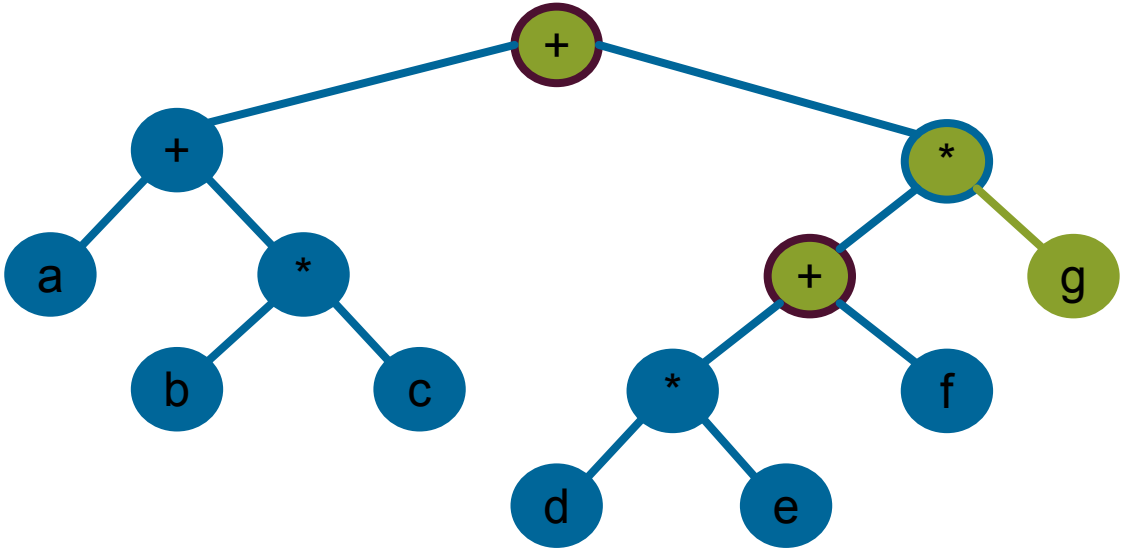
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * *



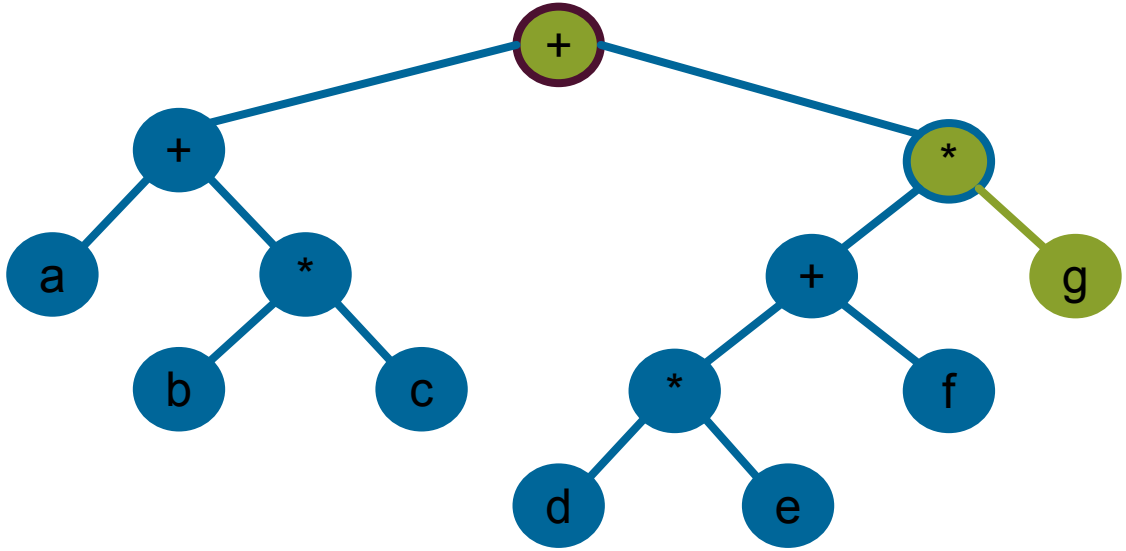
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f



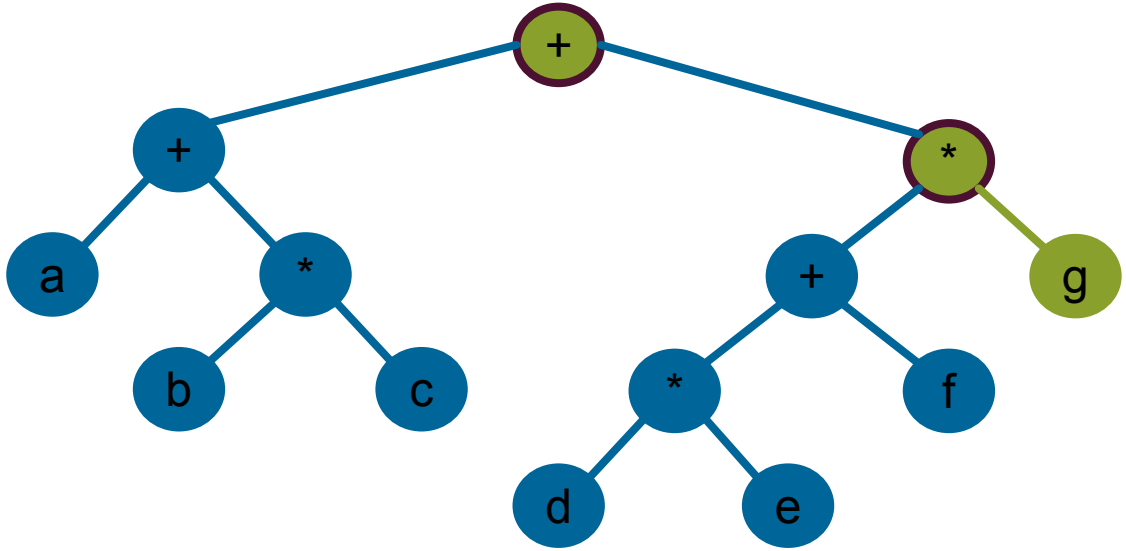
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f +



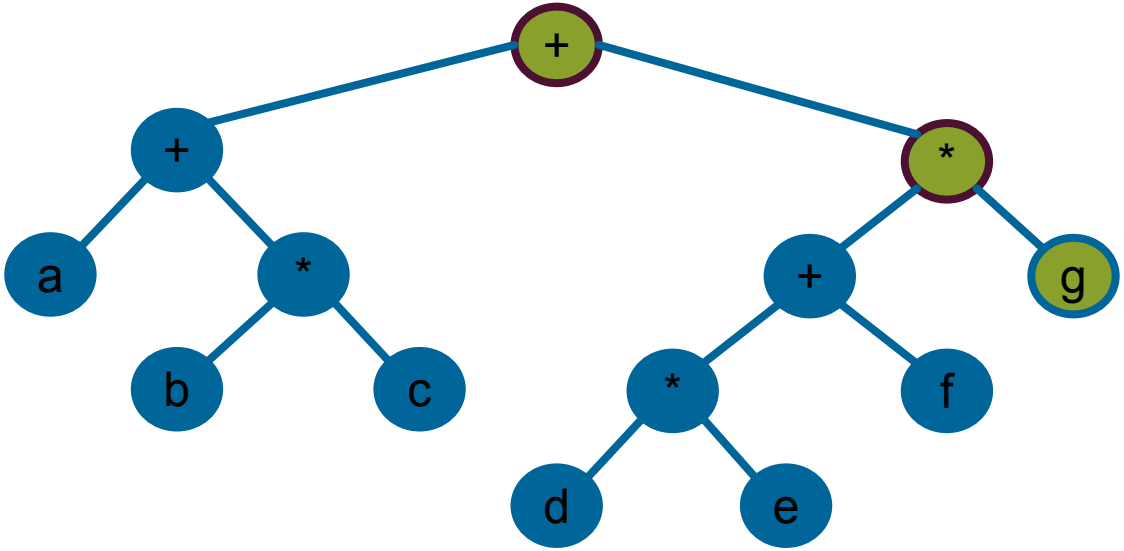
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f +



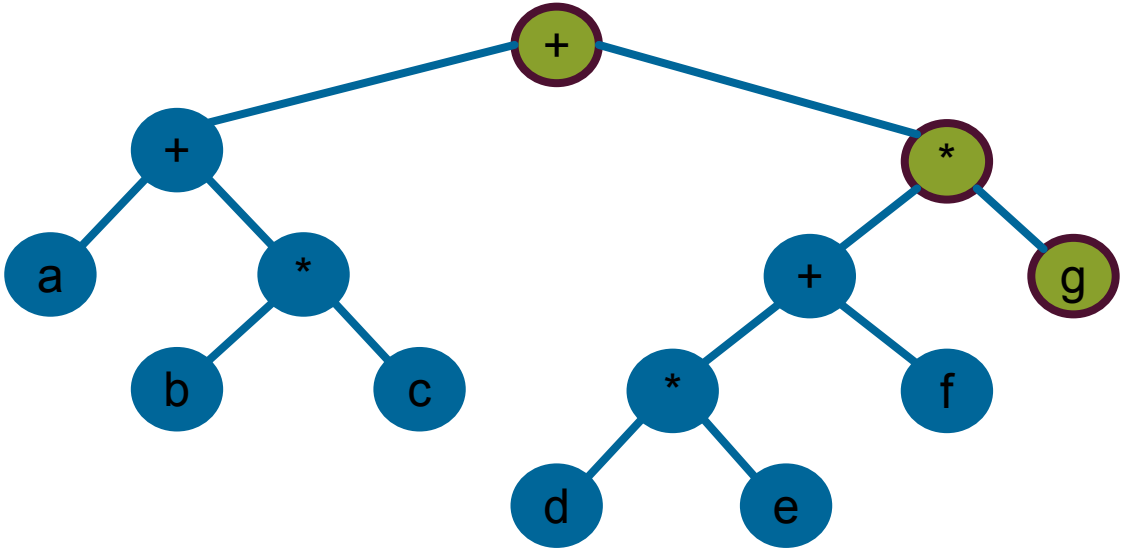
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f +



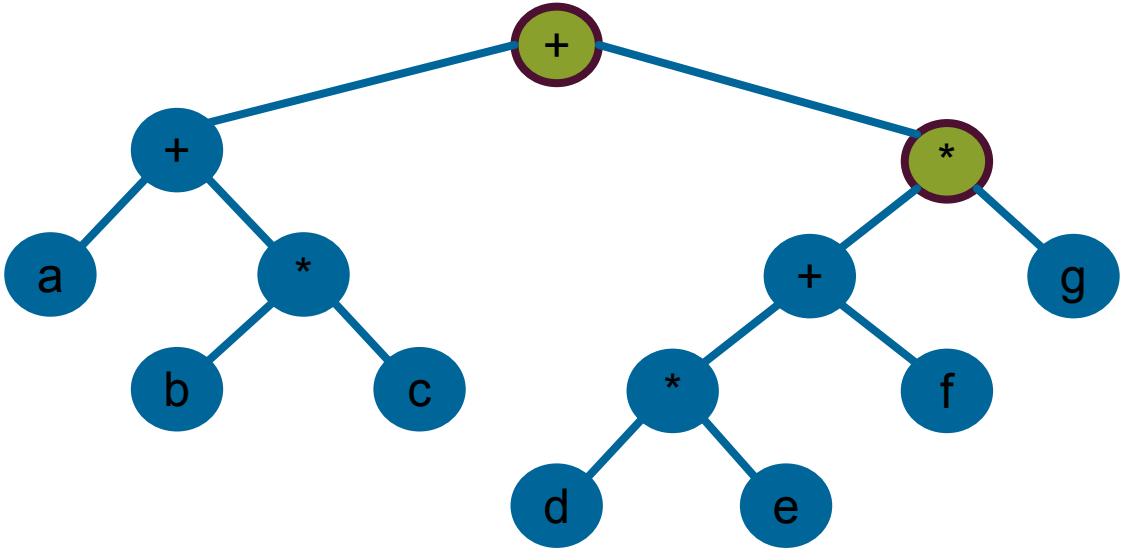
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f +



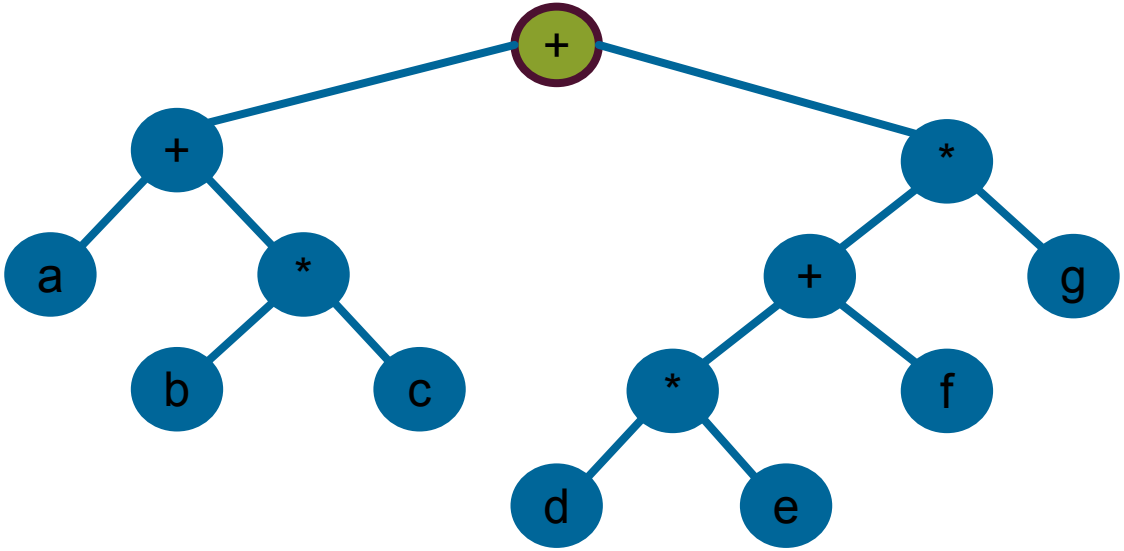
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f + g



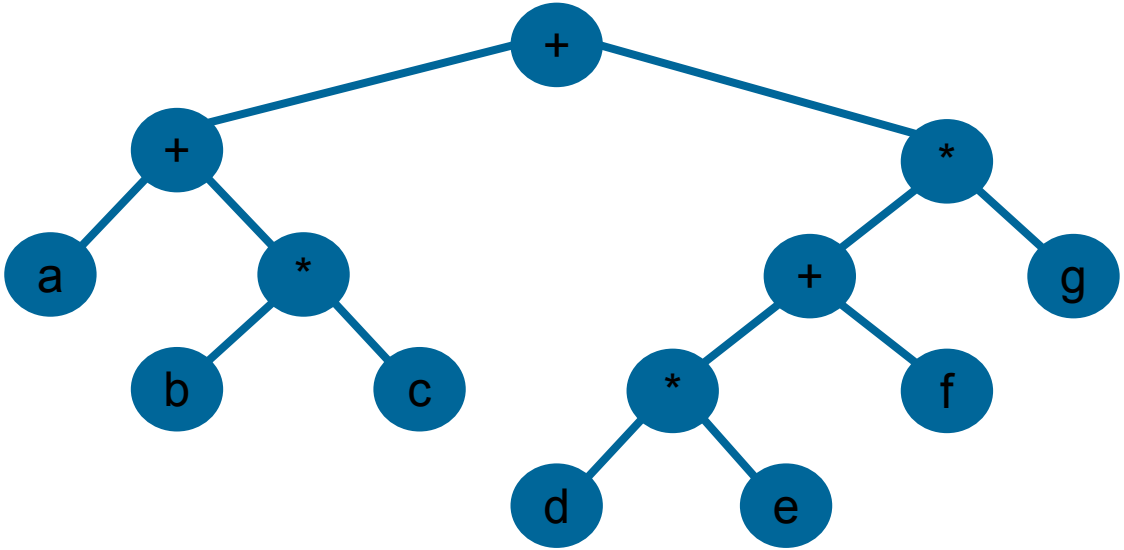
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f + g *



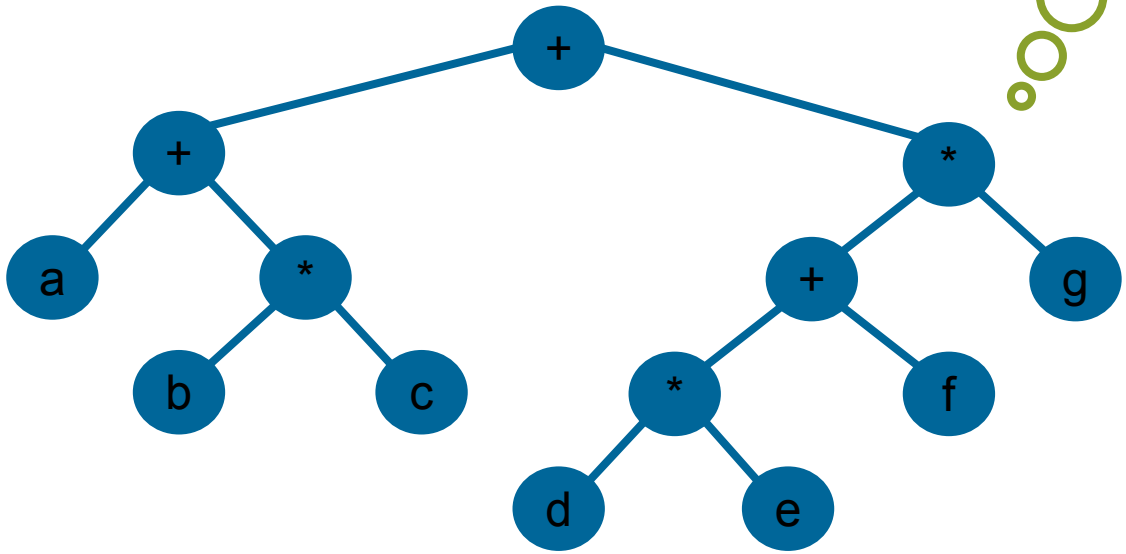
binary tree

- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f + g * +



binary tree

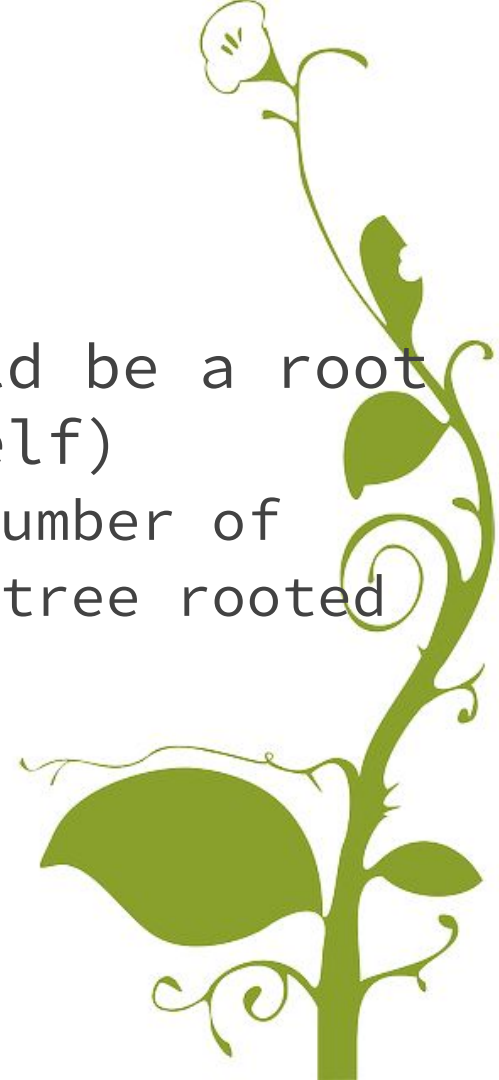
- expression tree
  - (a+b*c)+((d*e+f)*g)
  - a b c * + d e * f + g * +



What to do to evaluate the expression?

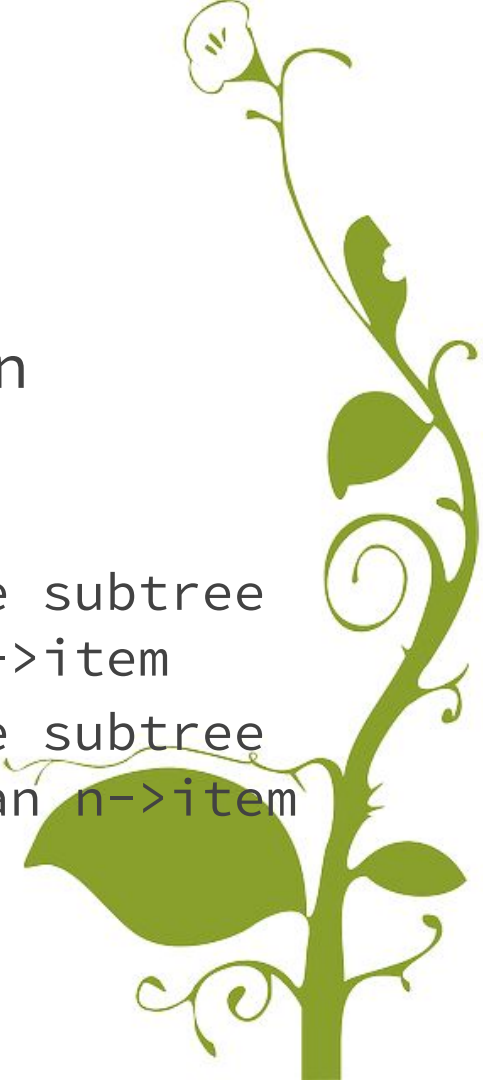
quiz (¼ sheet of paper - 5 minutes)

- given a tree node* n (which could be a root of some subtree or the tree itself)
  - make function that returns the number of nodes present in the tree or subtree rooted at n.

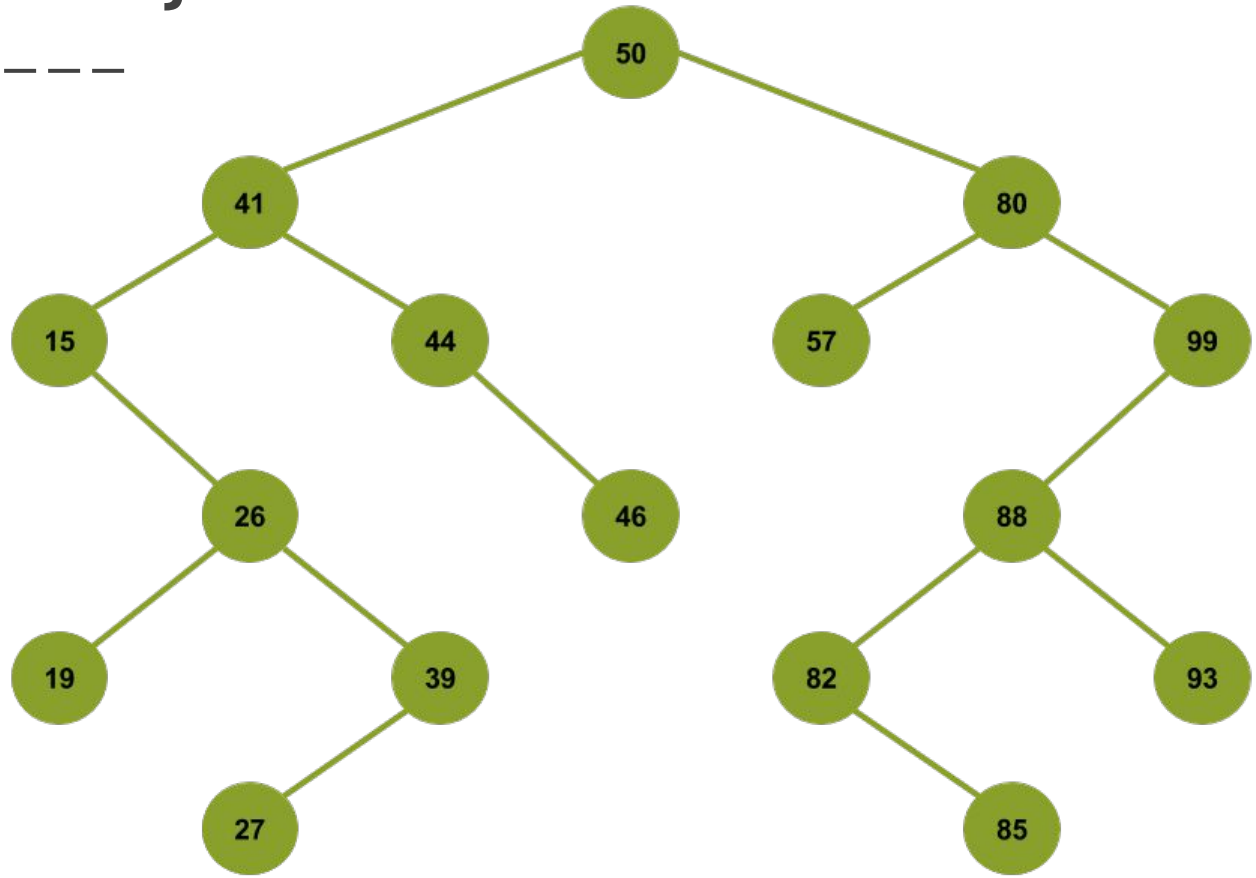


binary tree

- some binary trees observe certain properties
  - for any node n
    - the data value/item stored in the subtree rooted at n->left is less than n->item
    - the data value/item stored in the subtree rooted at n->right is greater than n->item
  - binary search tree property



binary search tree

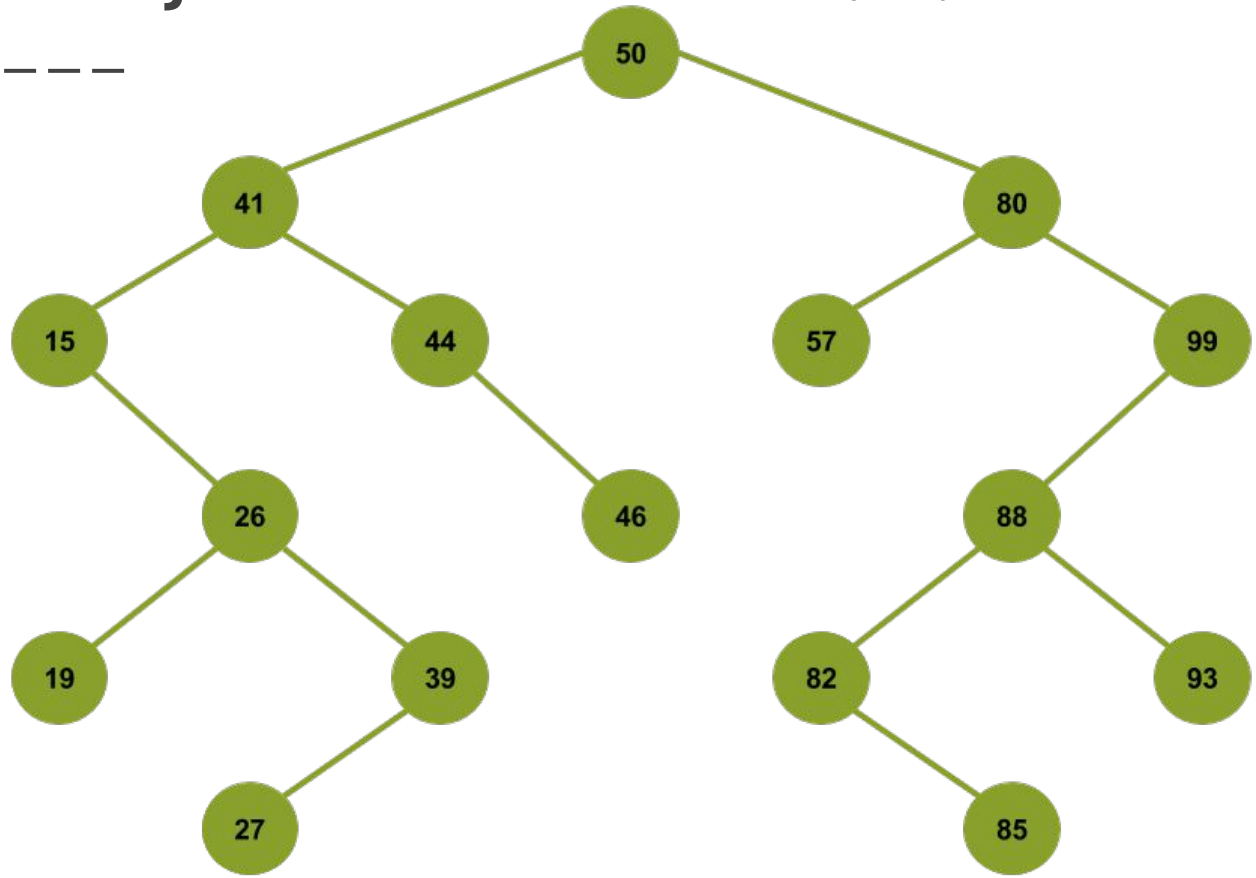


binary search tree - operations

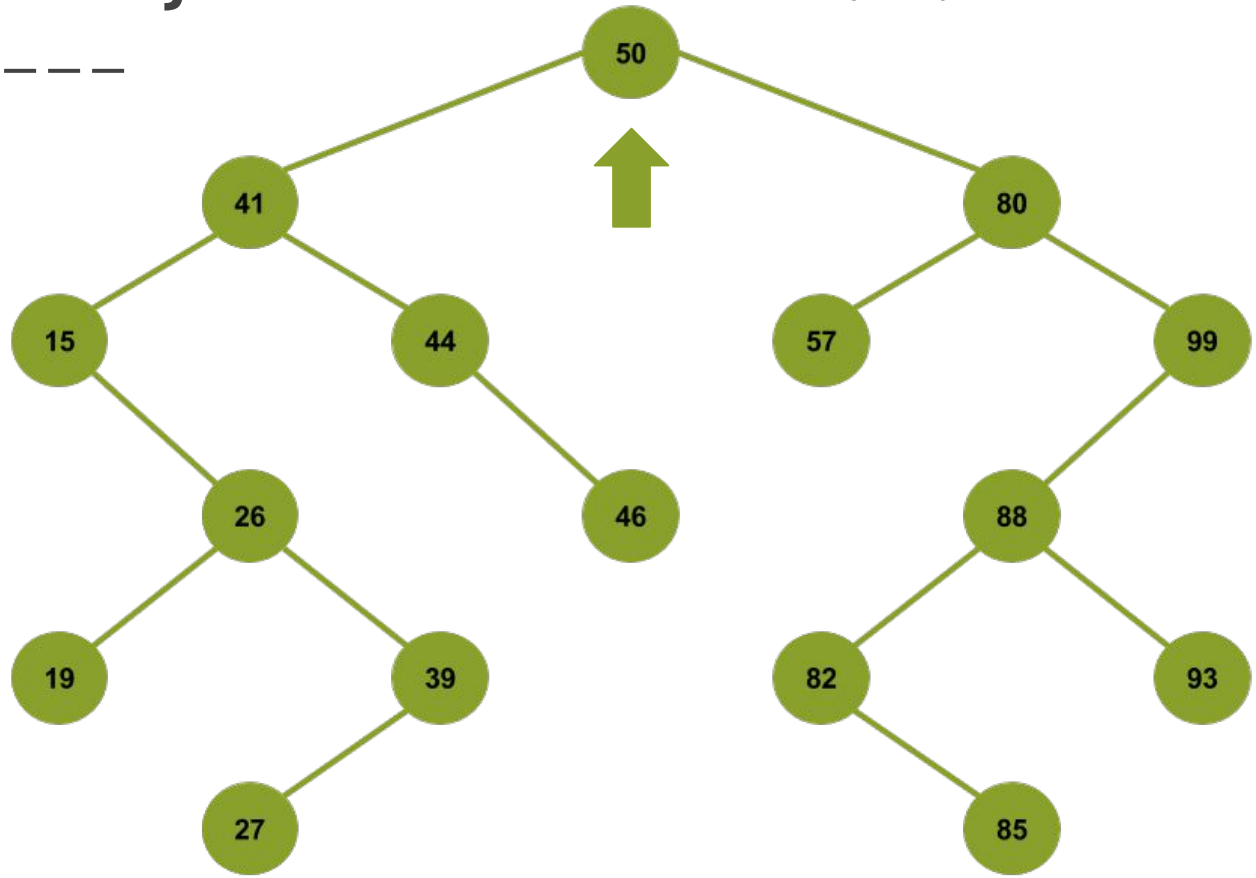
- search
- minimum
- maximum
- insert
- remove



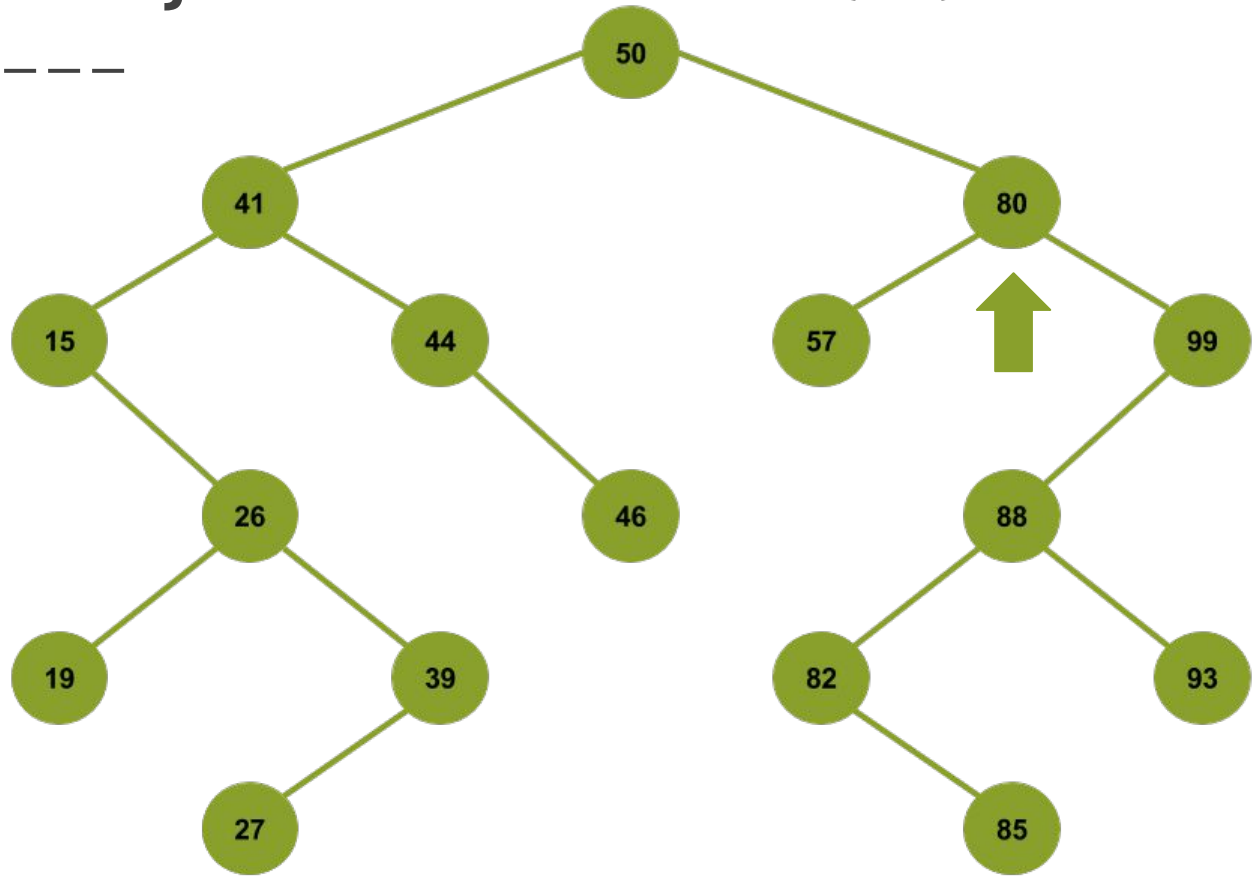
binary search tree - search (85)



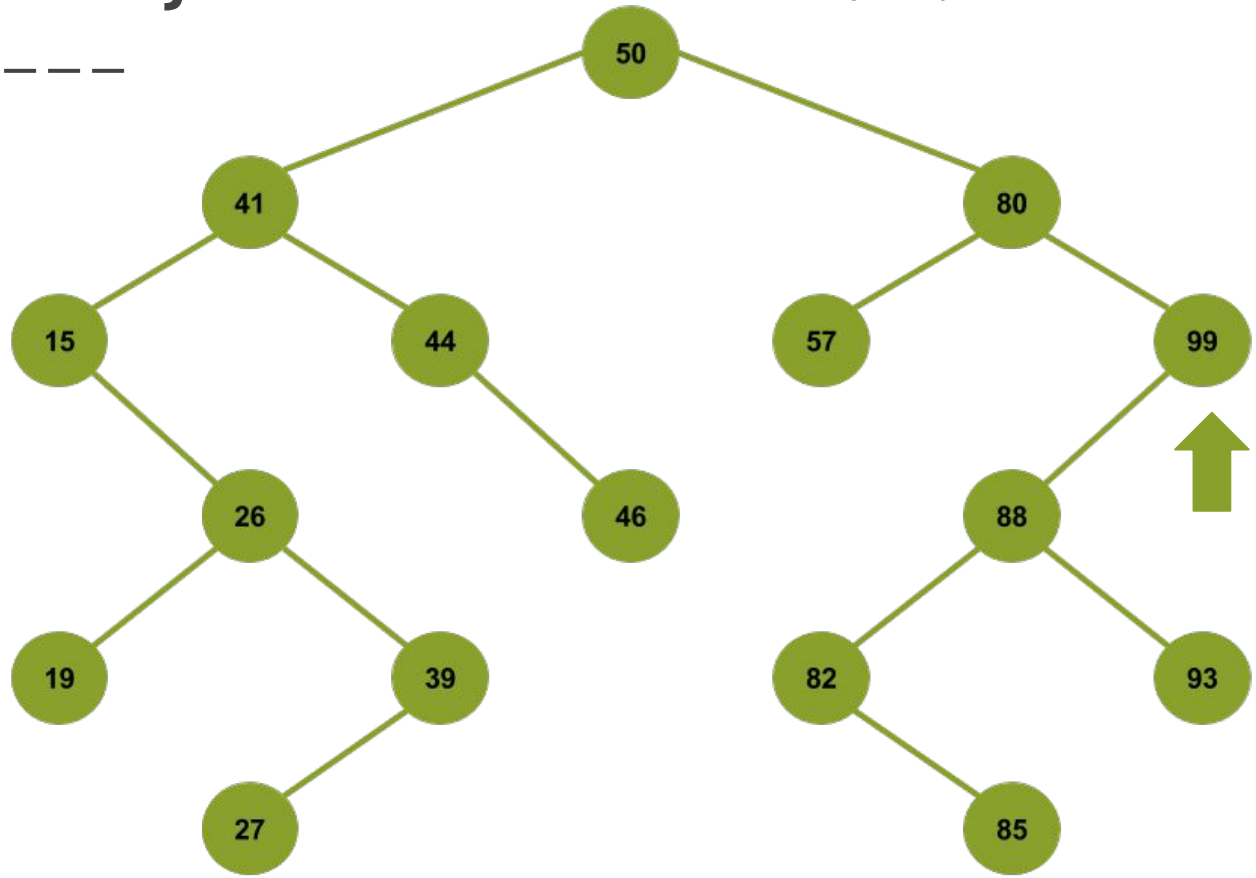
binary search tree - search (85)



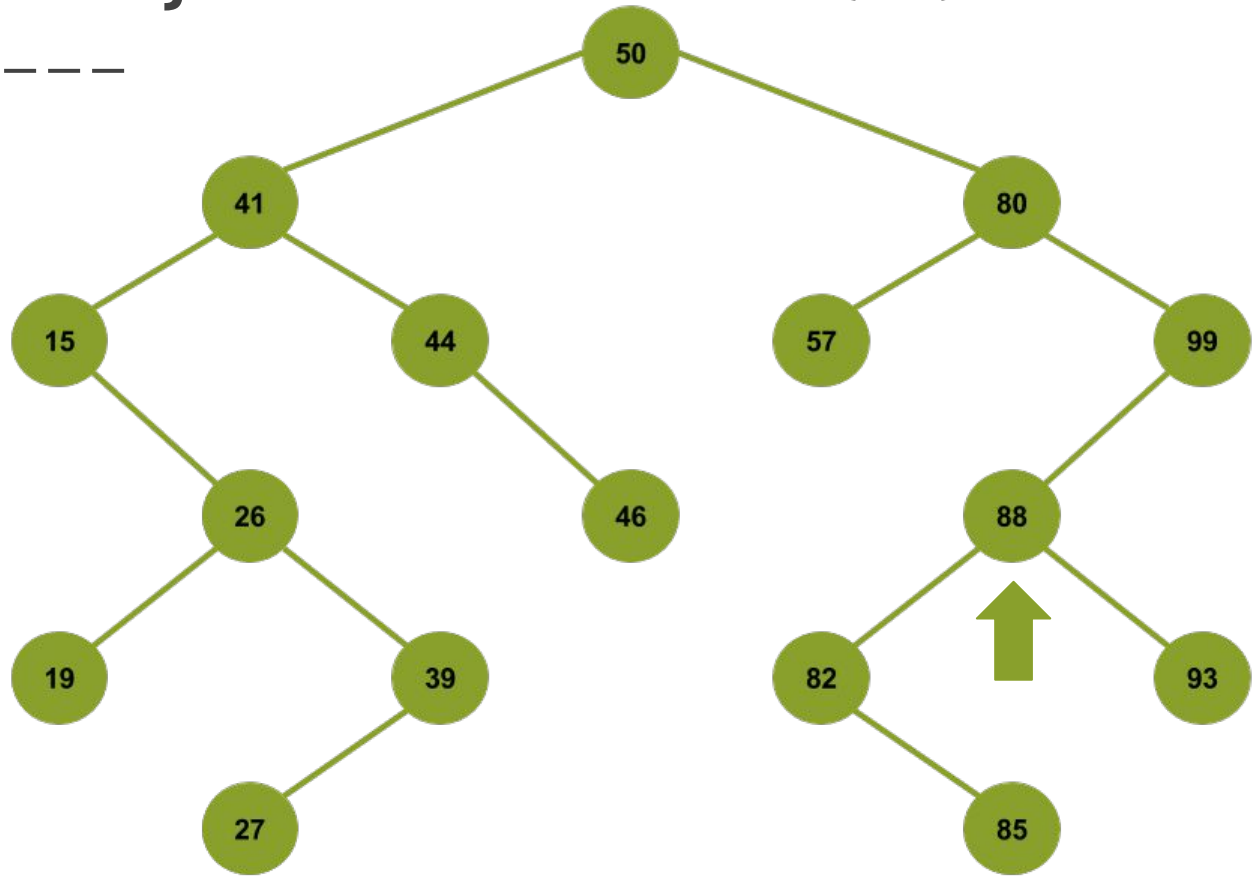
binary search tree - search (85)



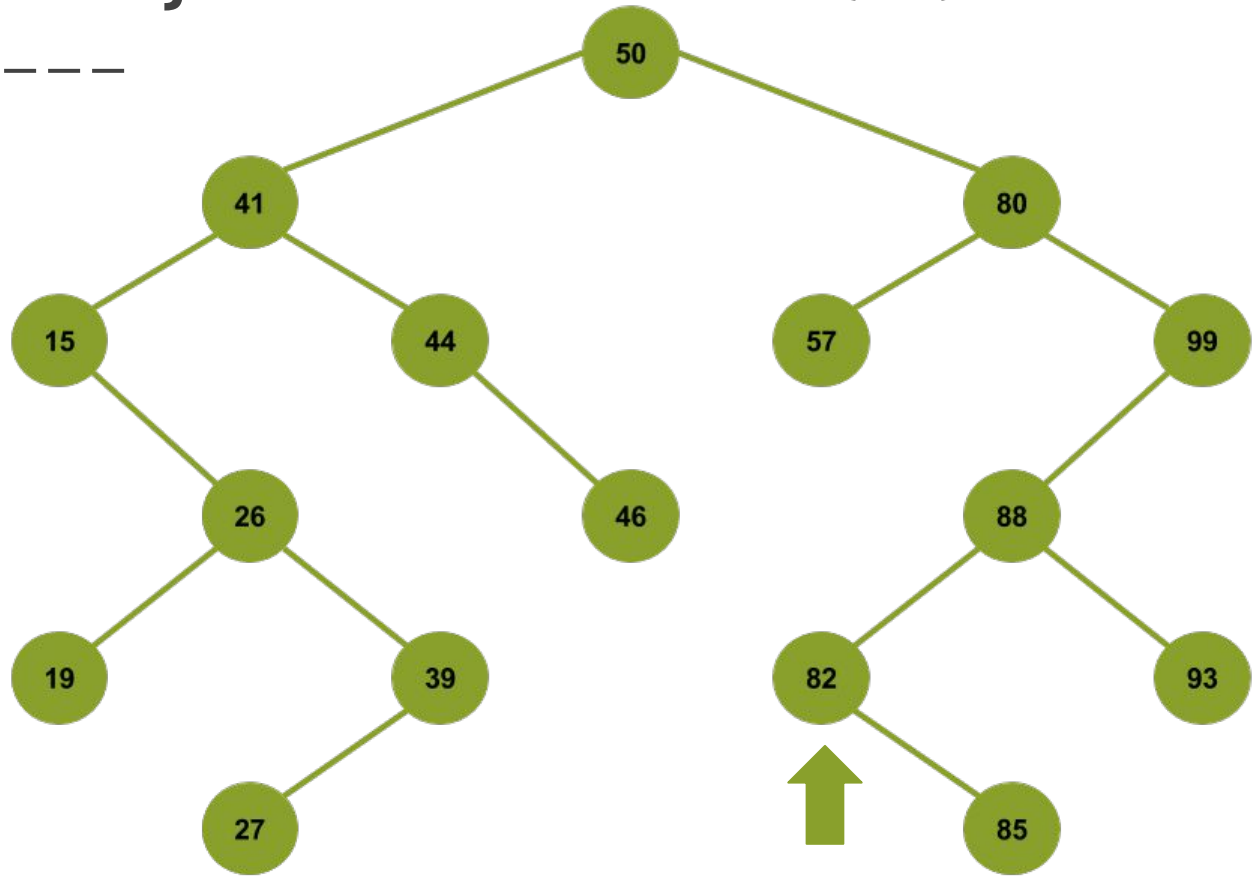
binary search tree - search (85)



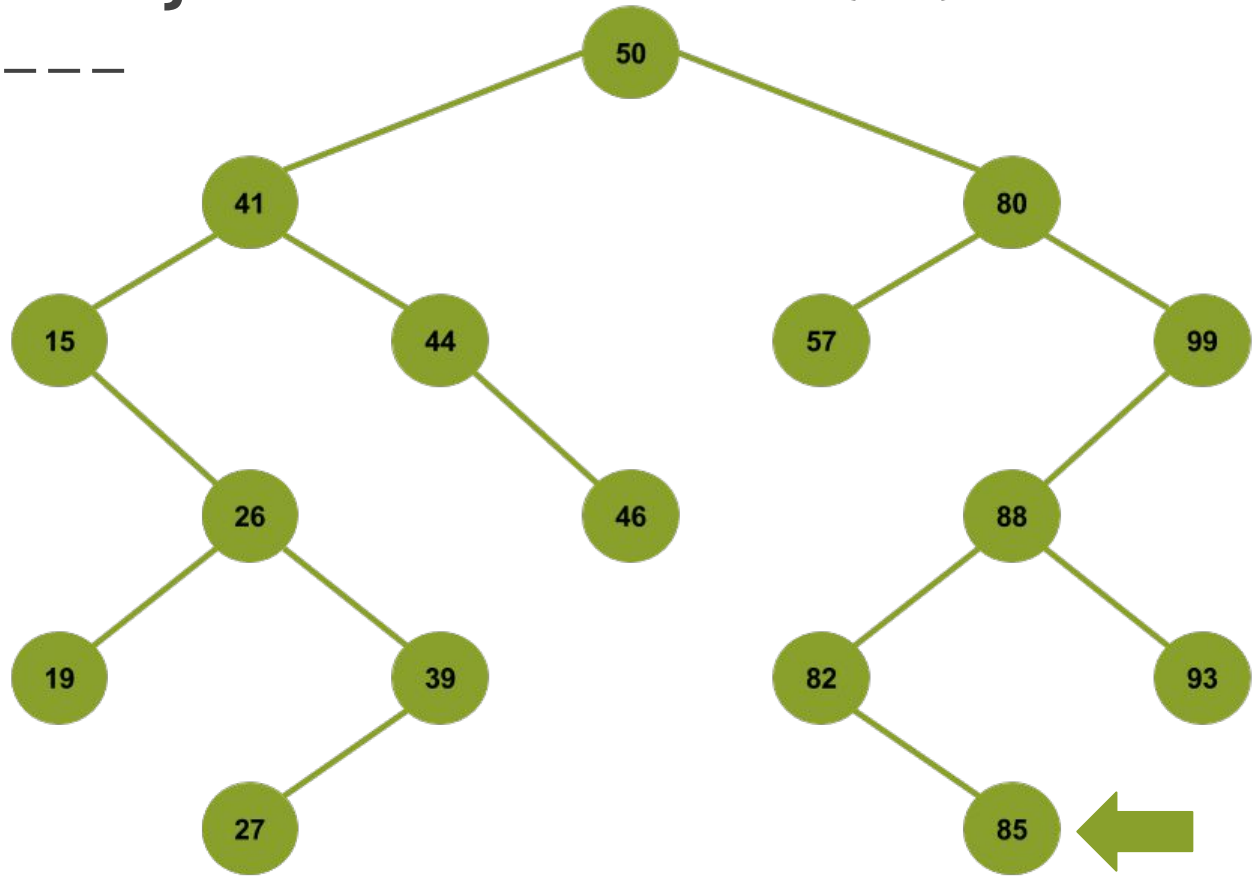
binary search tree - search (85)



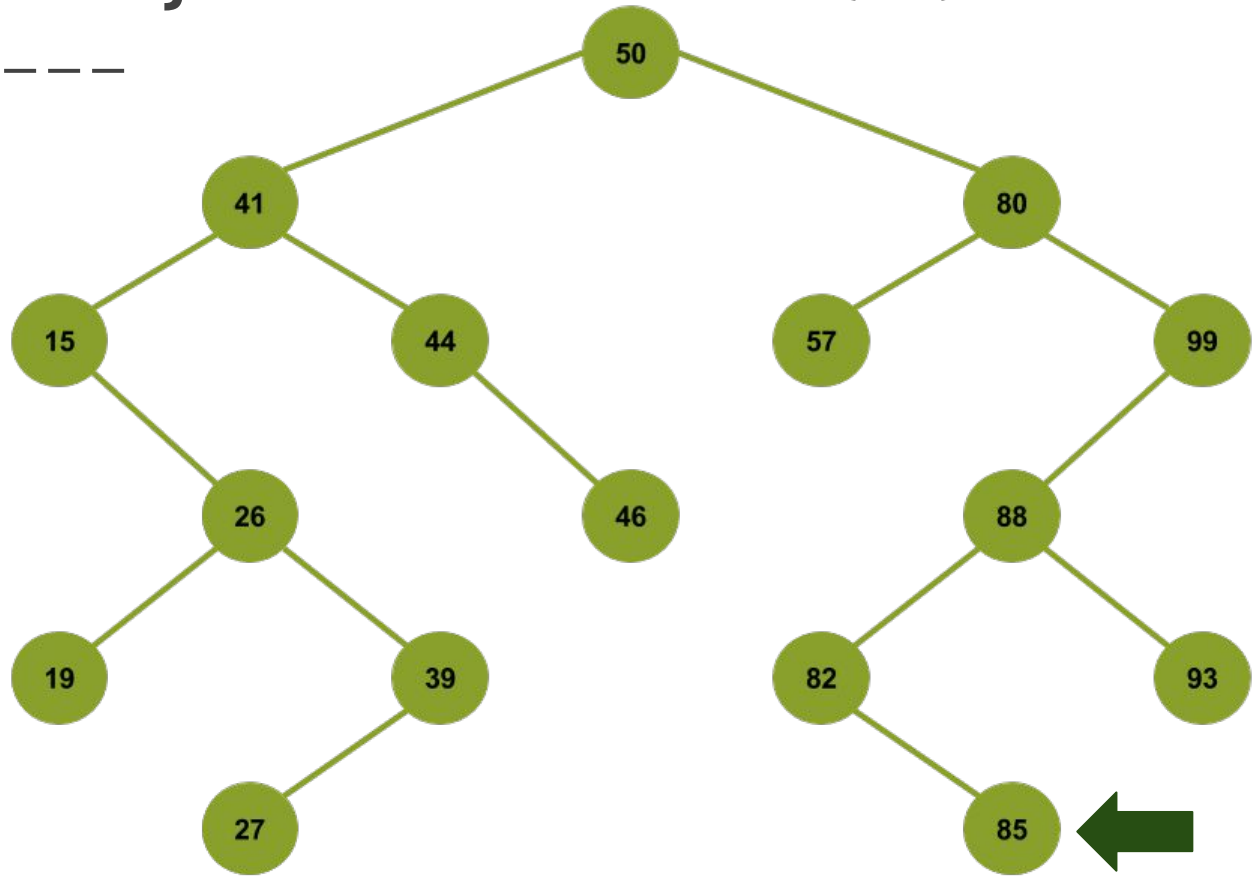
binary search tree - search (85)



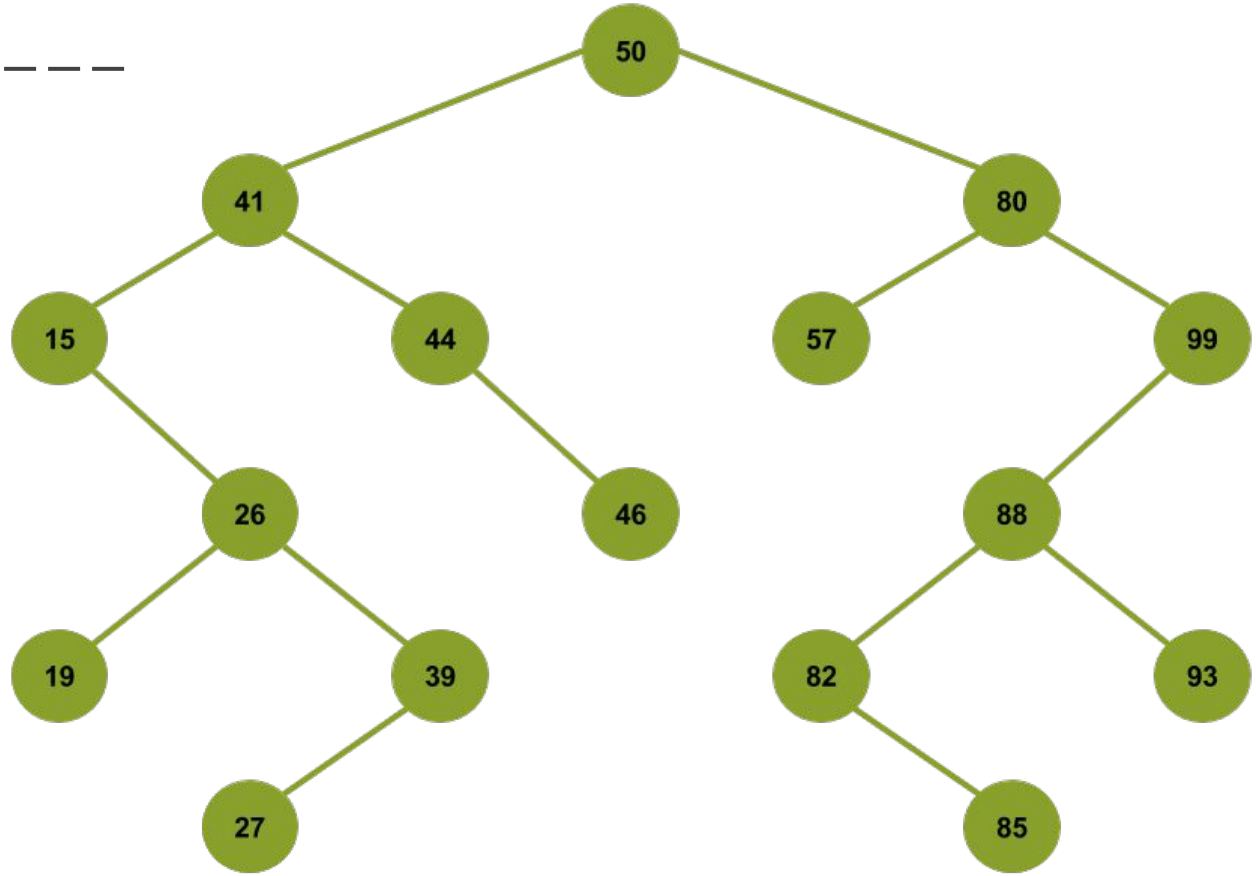
binary search tree - search (85)



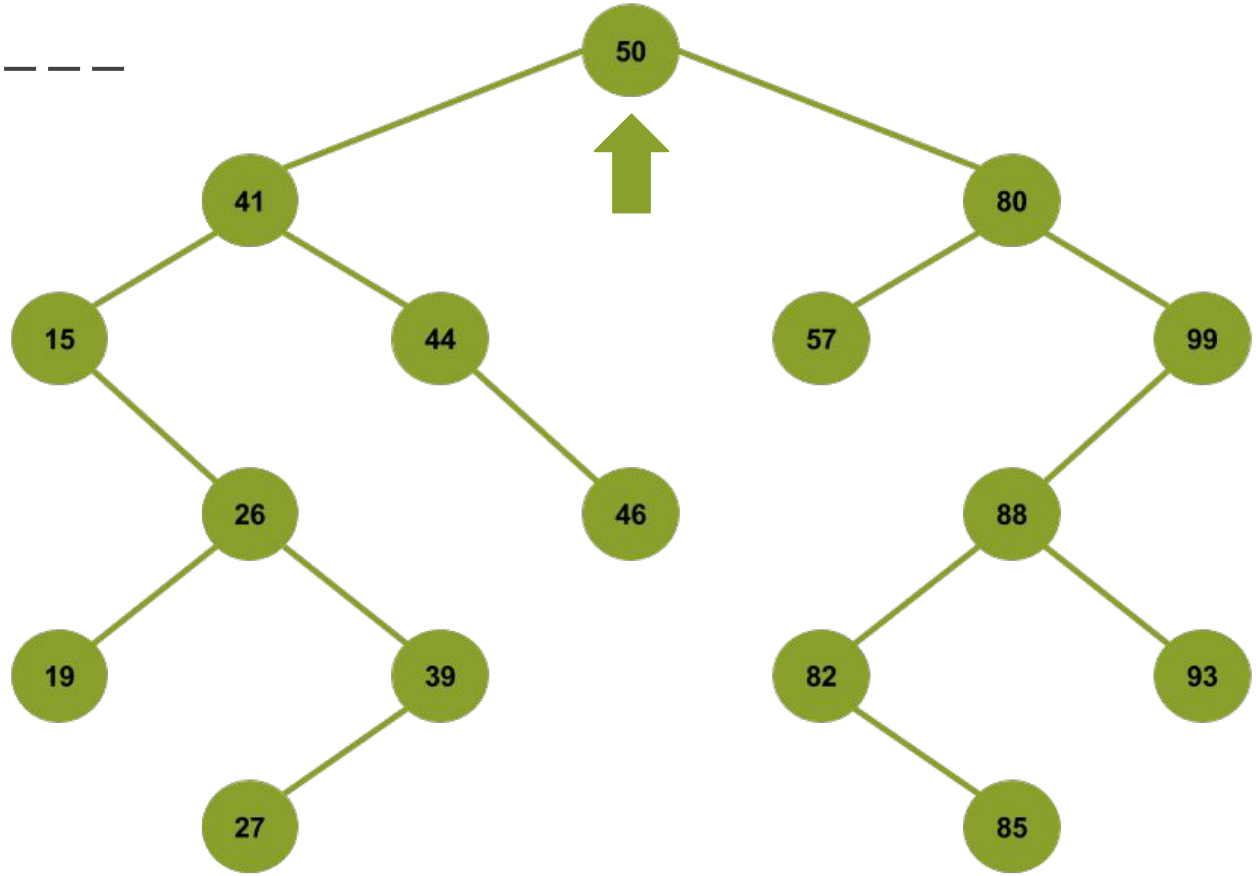
binary search tree - search (85)



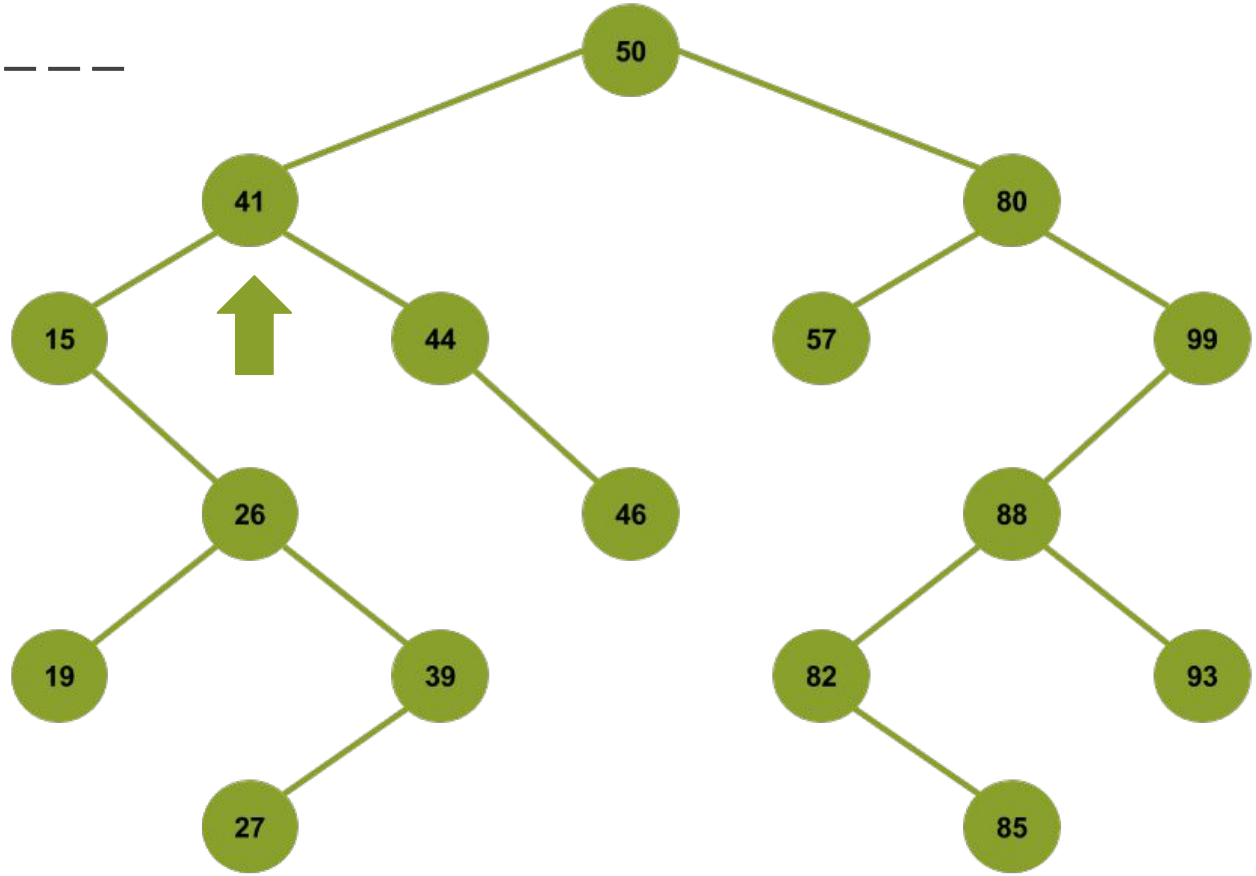
binary search tree - minimum



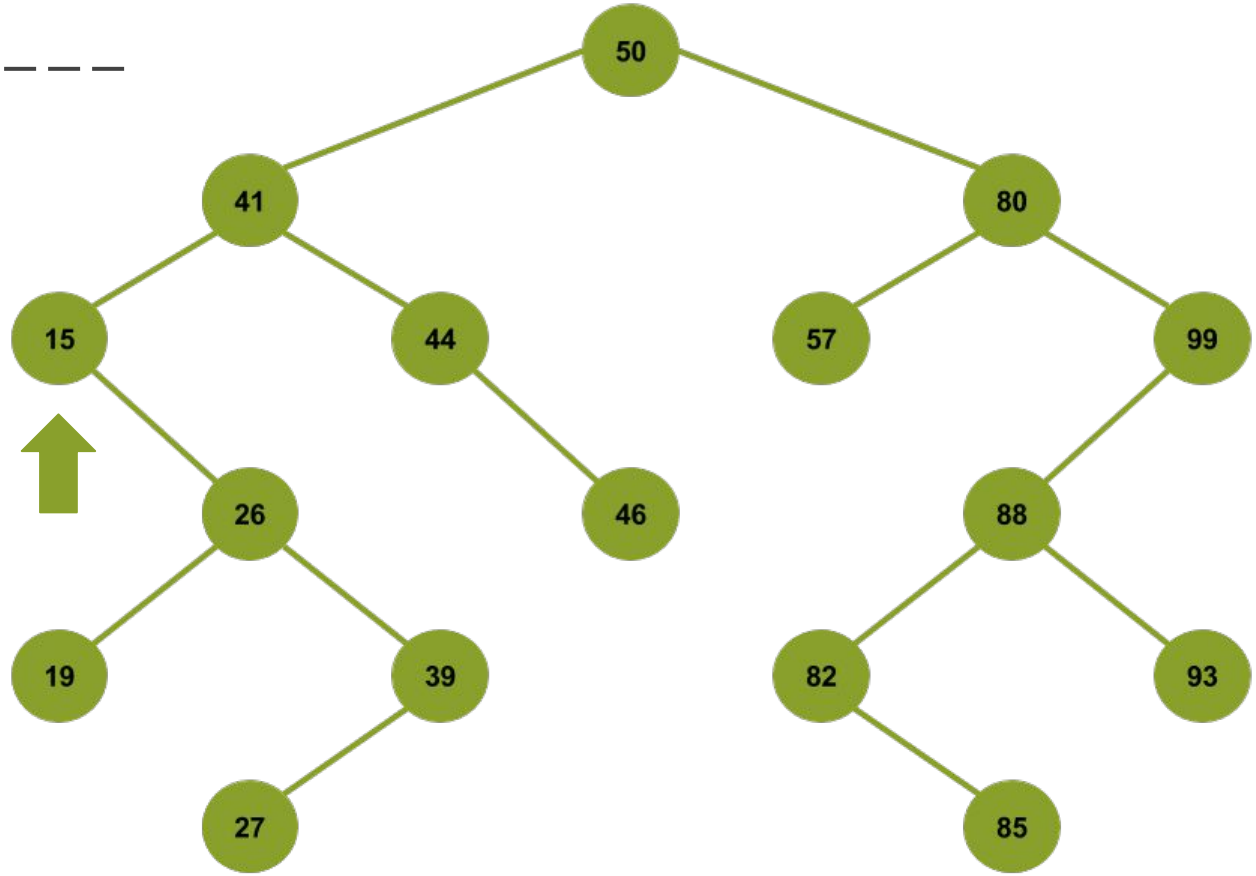
binary search tree - minimum



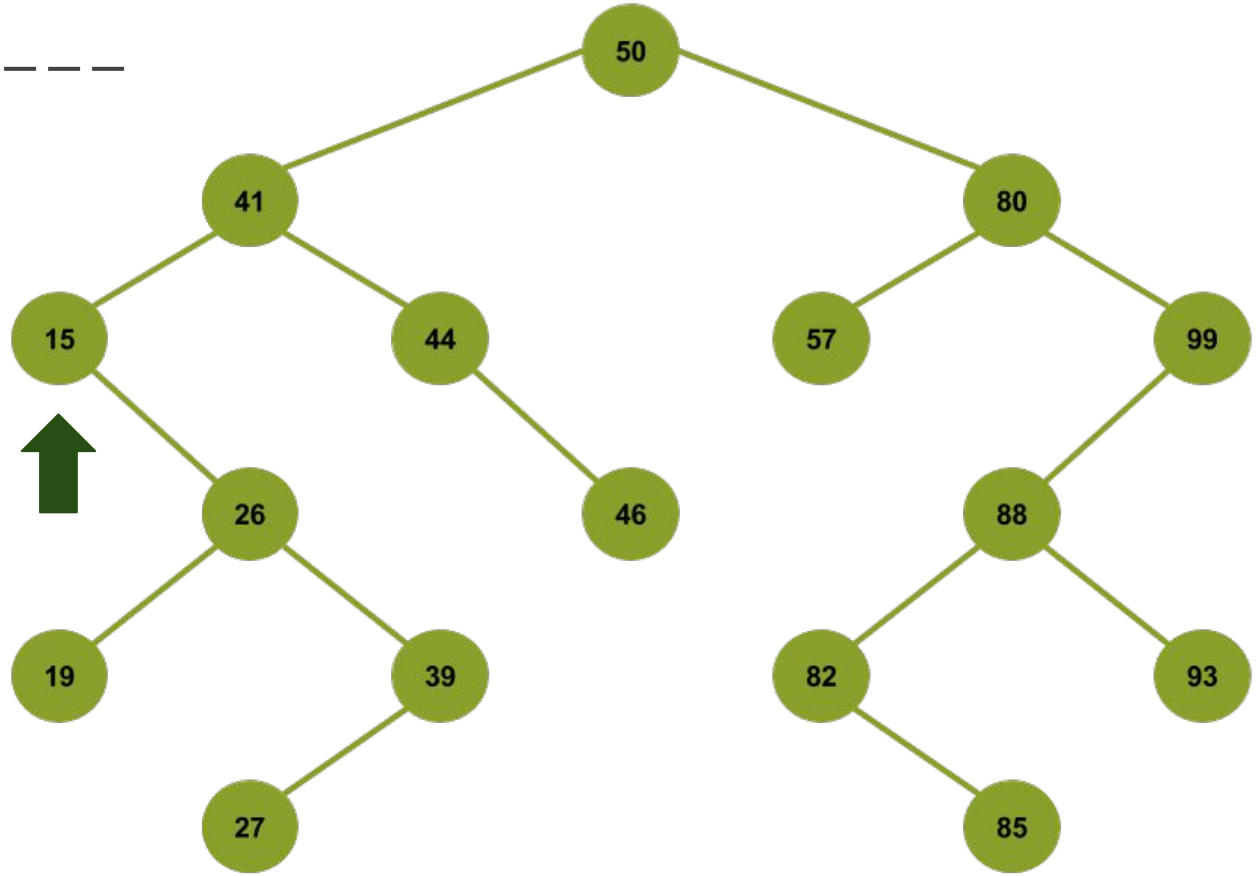
binary search tree - minimum



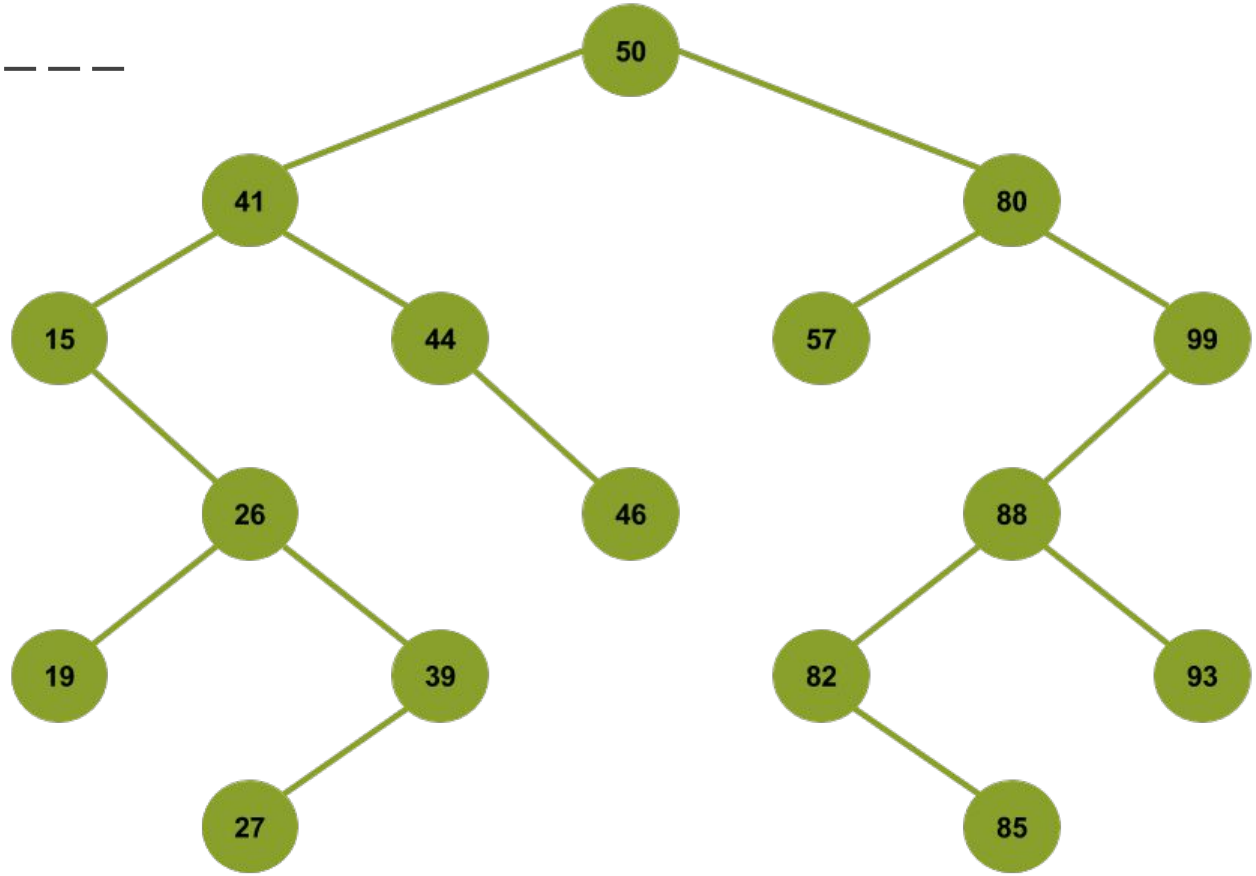
binary search tree - minimum



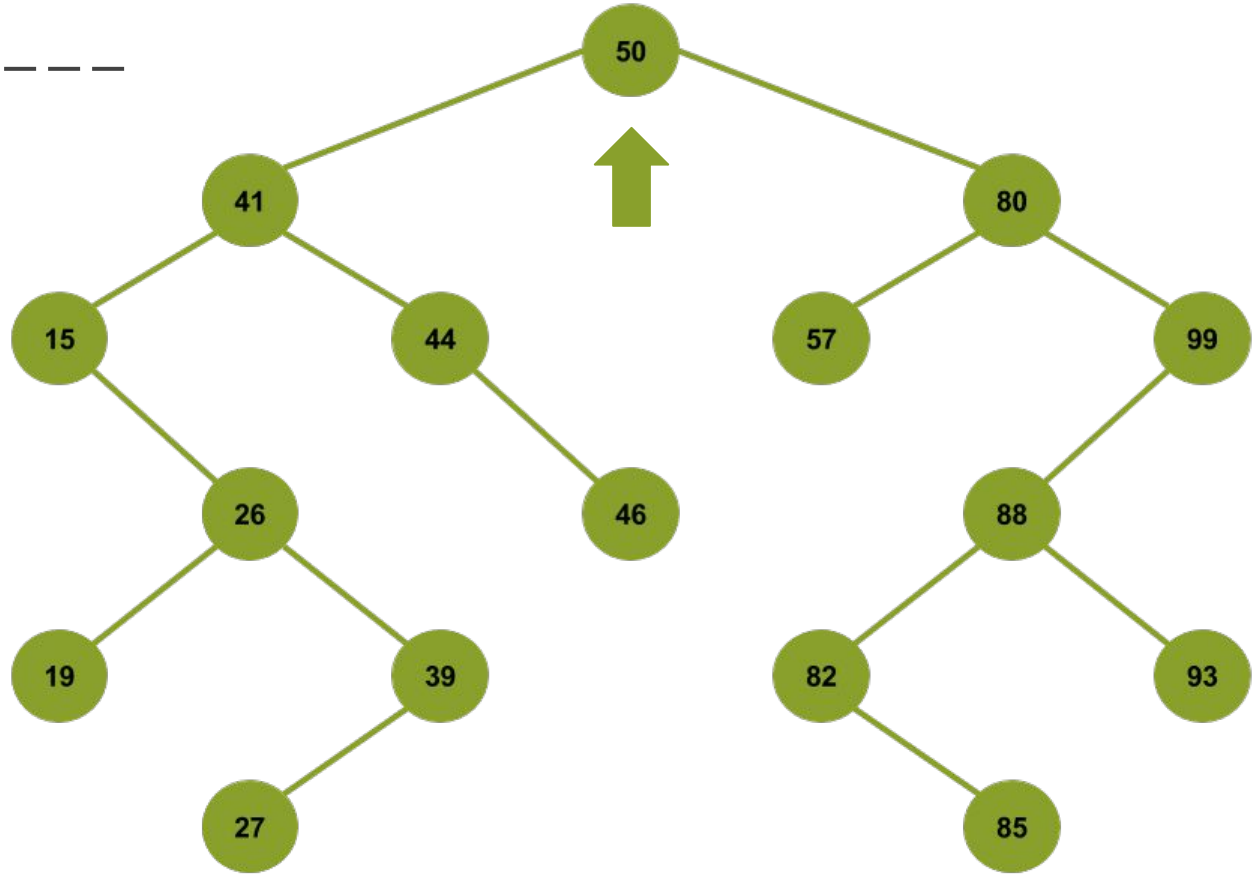
binary search tree - minimum



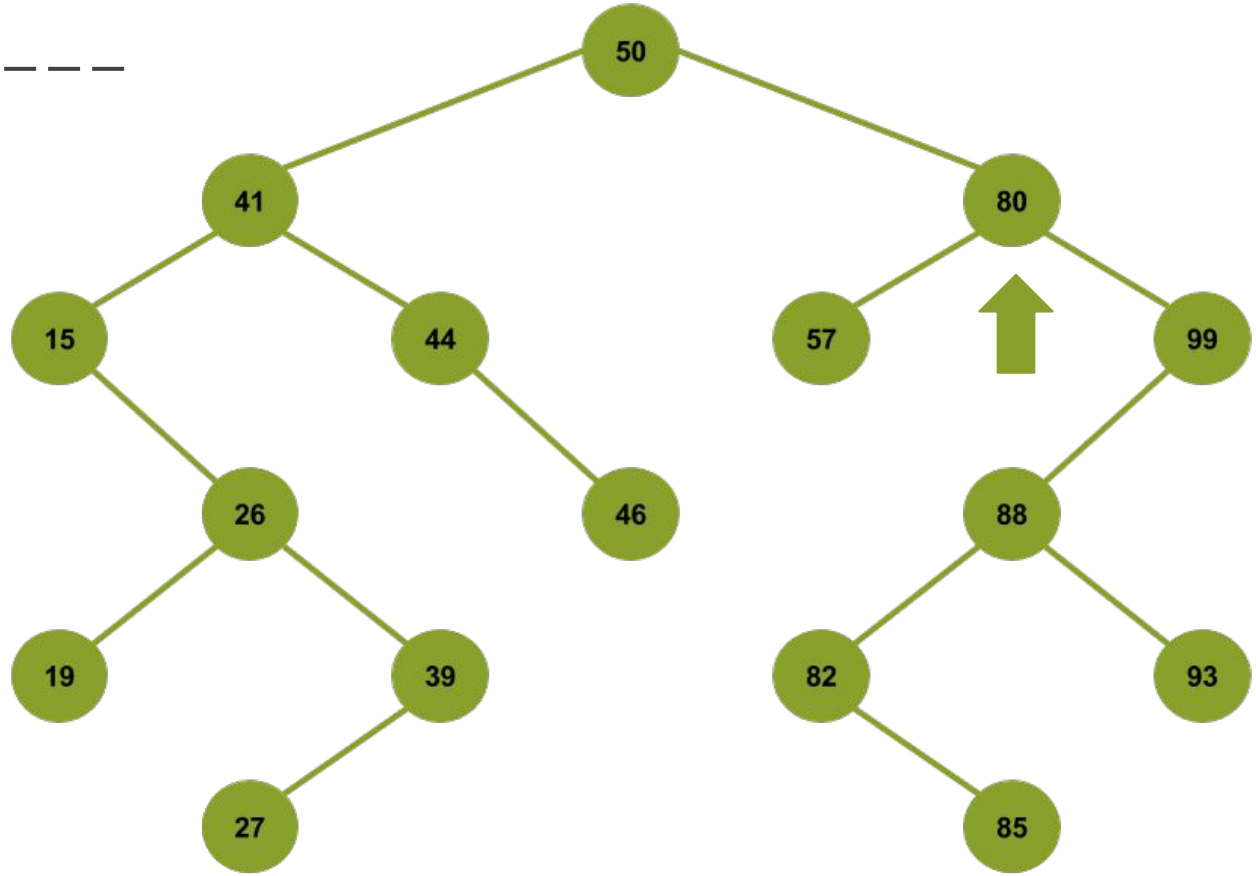
binary search tree - maximum



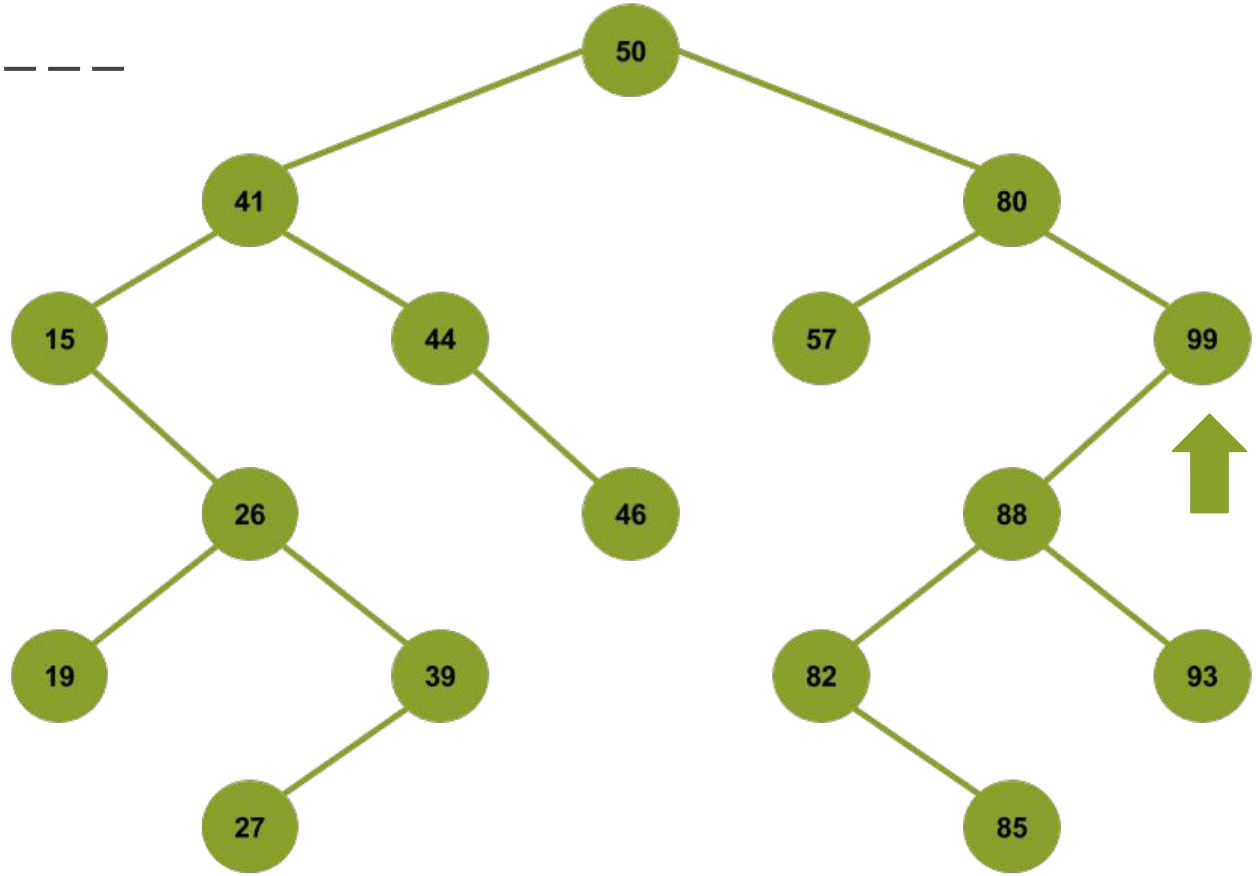
binary search tree - maximum



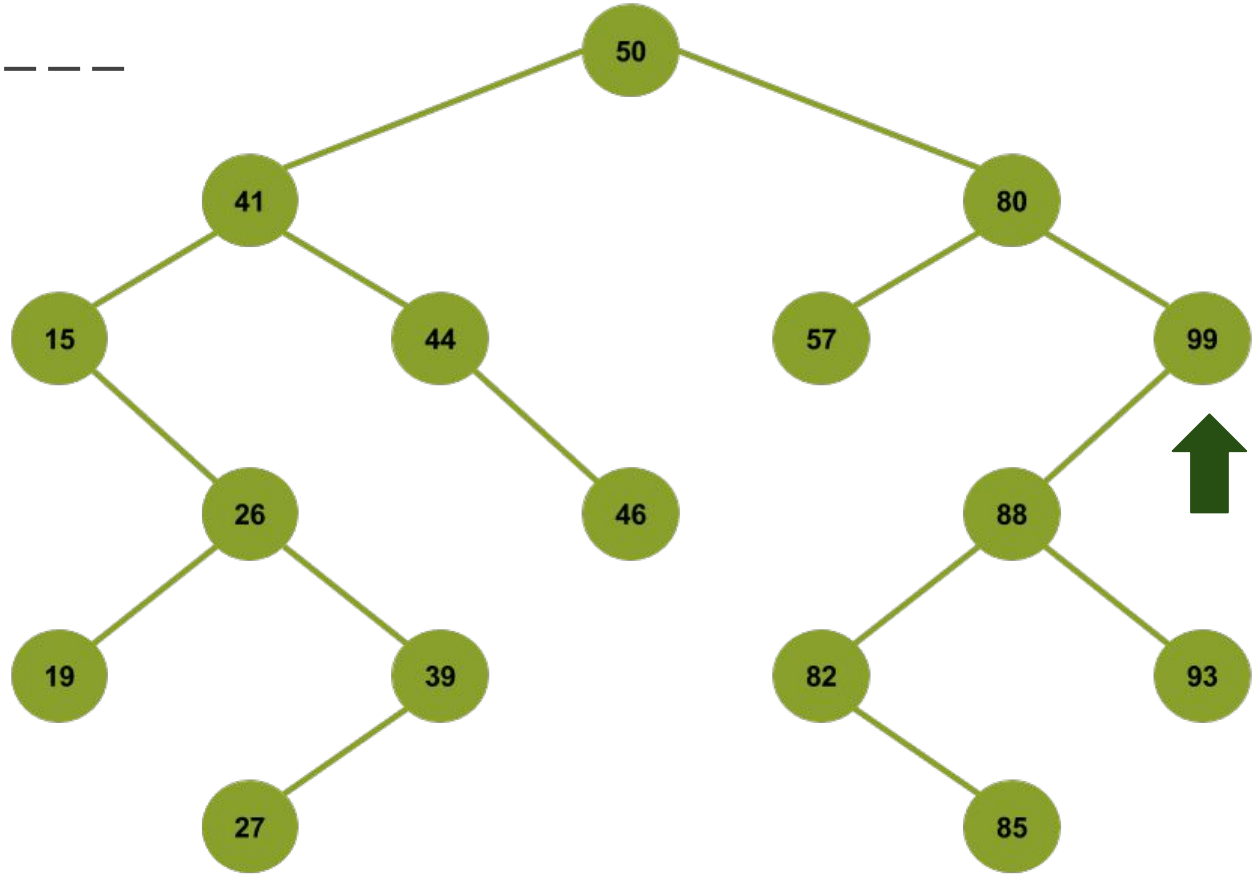
binary search tree - maximum



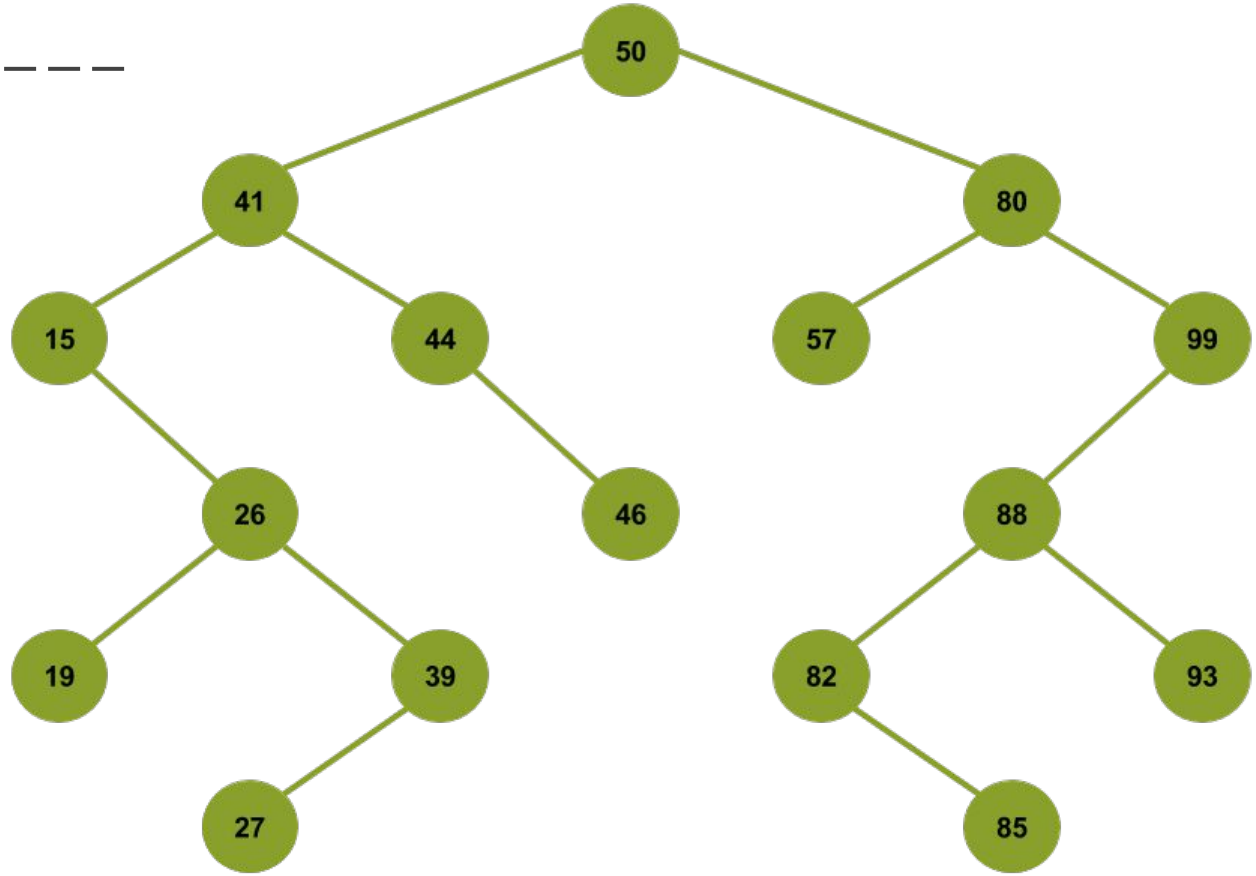
binary search tree - maximum



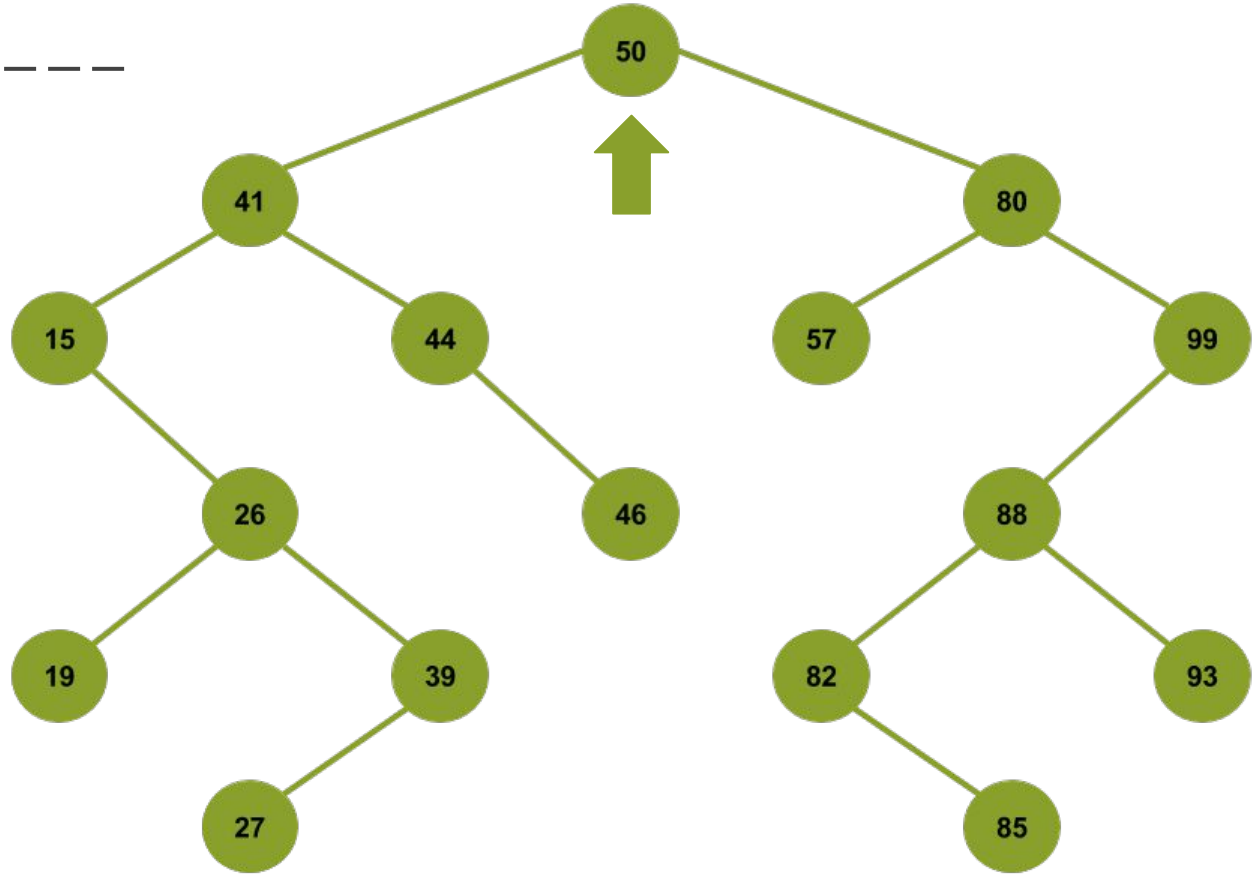
binary search tree - maximum



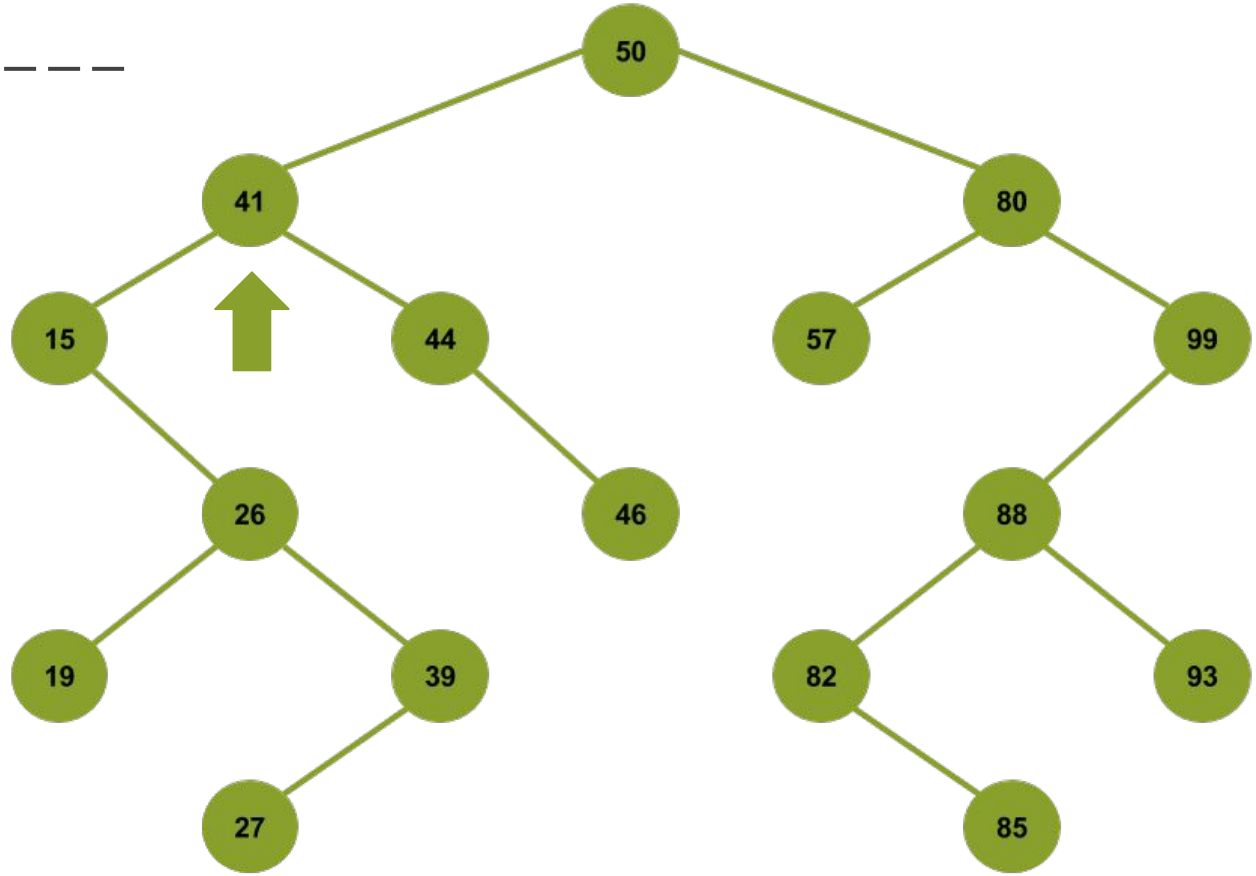
binary search tree - insert



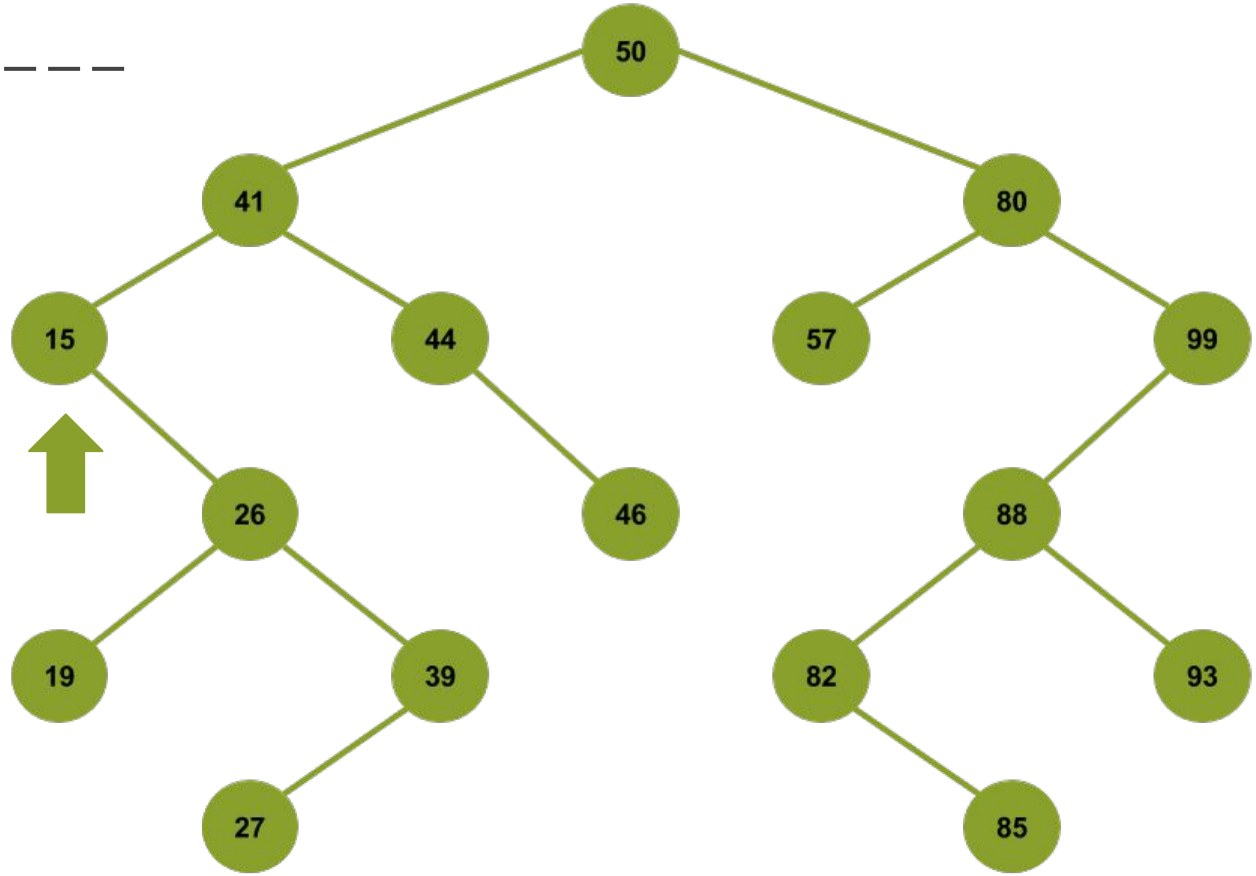
binary search tree - insert (40)



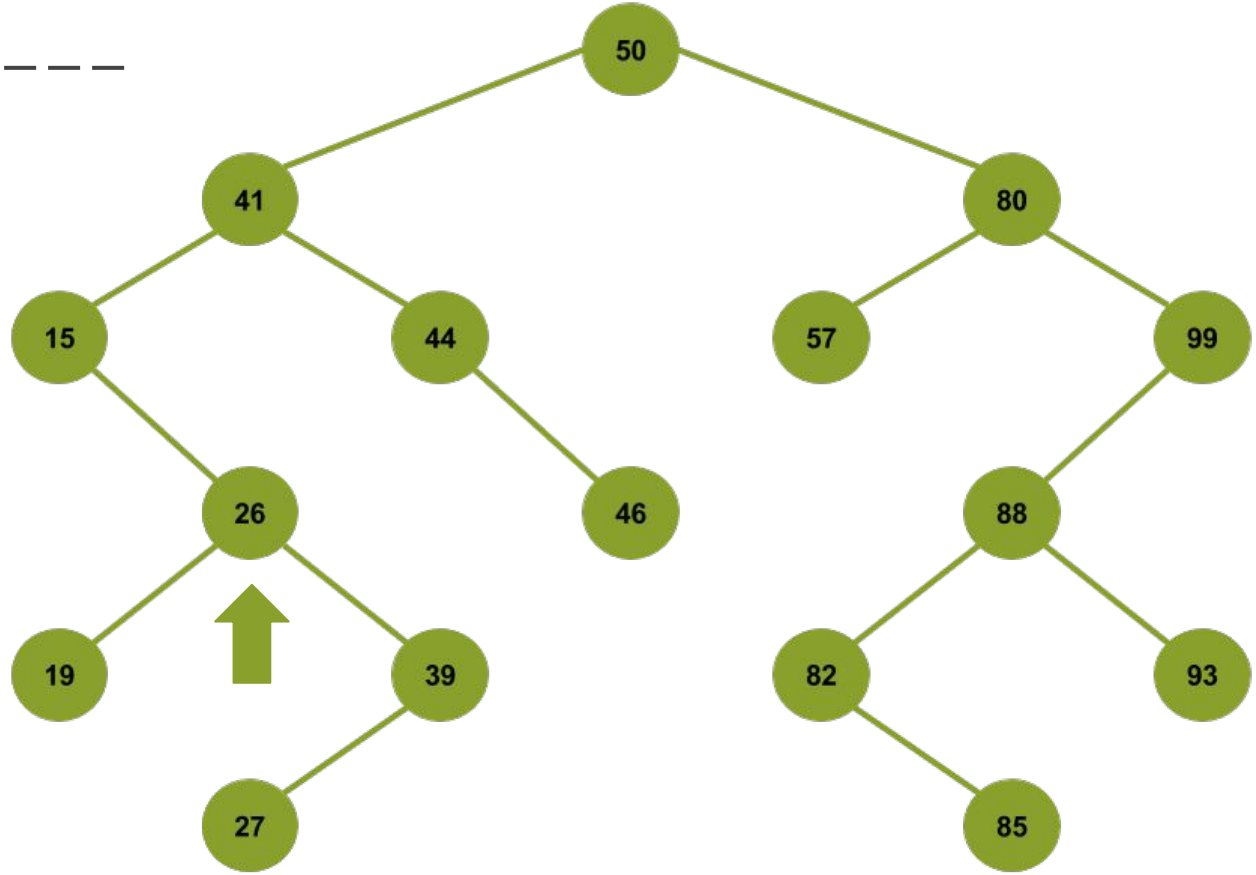
binary search tree - insert (40)



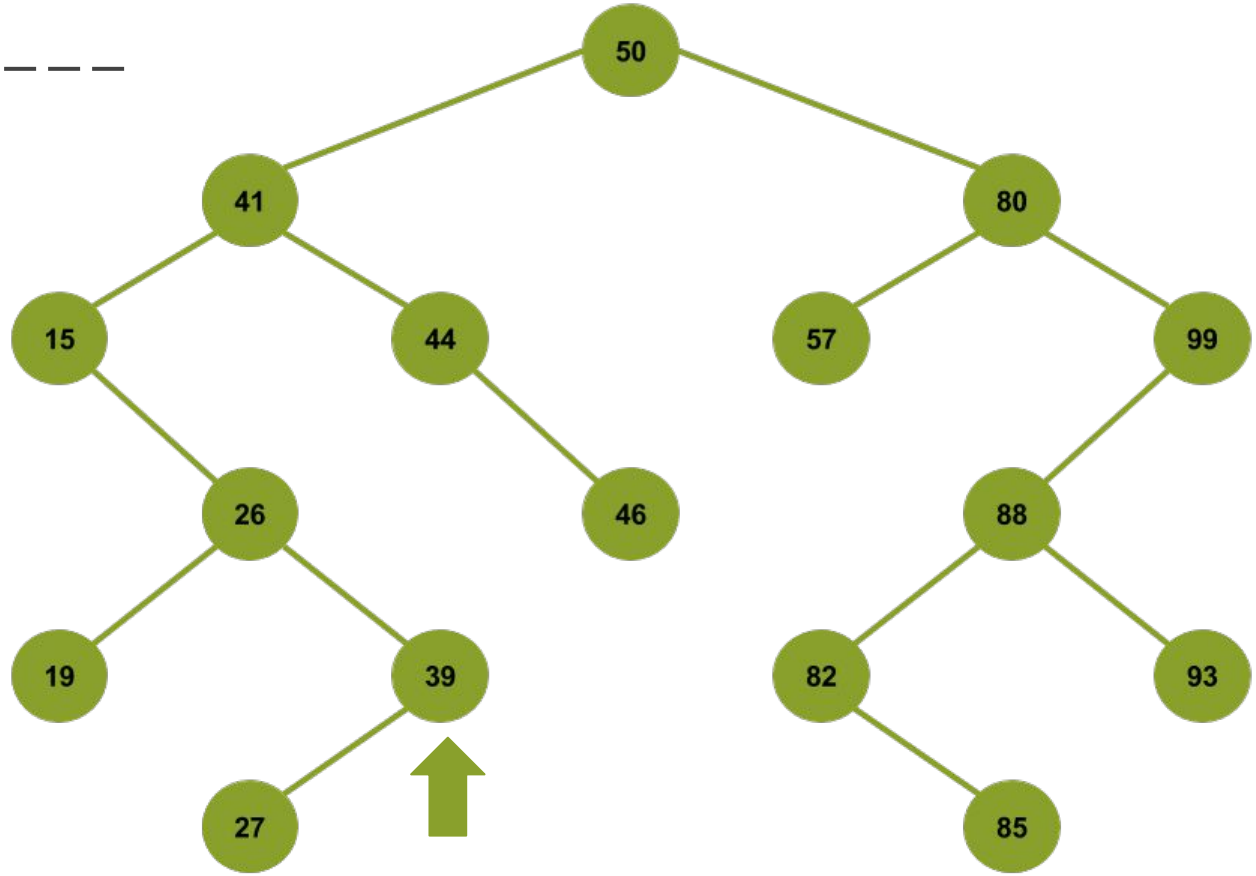
binary search tree - insert (40)



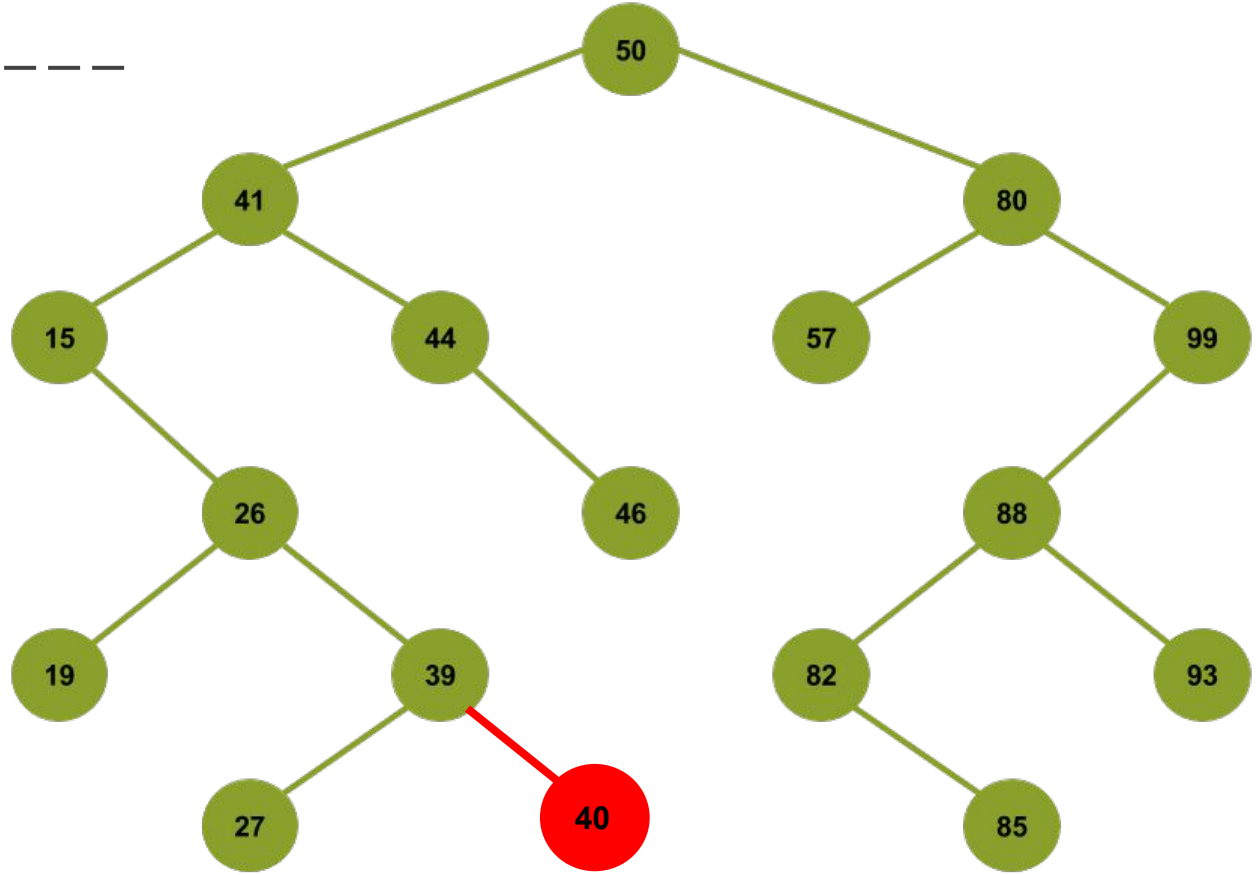
binary search tree - insert (40)



binary search tree - insert (40)



binary search tree - insert (40)



binary search tree - insert (50)

_ _ _



binary search tree - insert (50)

_ _ _

50



binary search tree - insert (25)

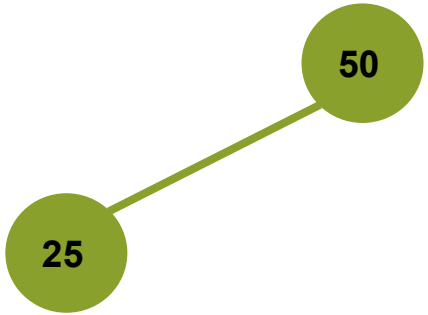
_ _ _





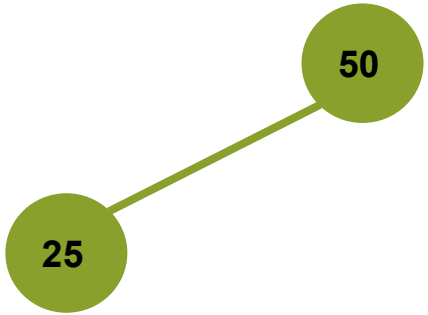
binary search tree - insert (25)

_ _ _



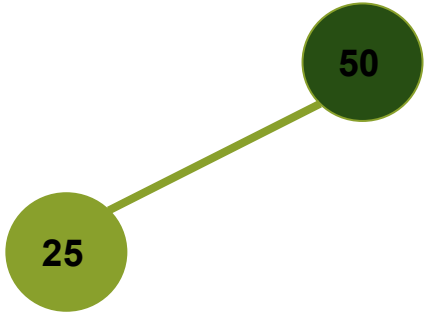


binary search tree - insert (36)

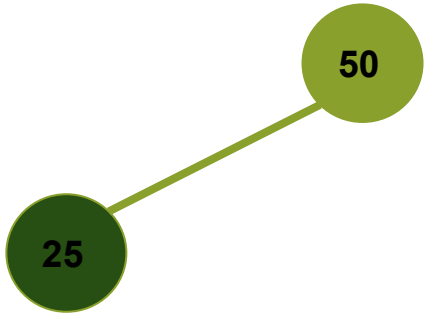


binary search tree - insert (36)

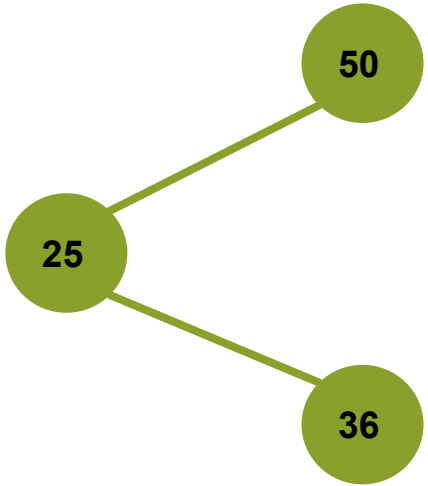
_ _ _



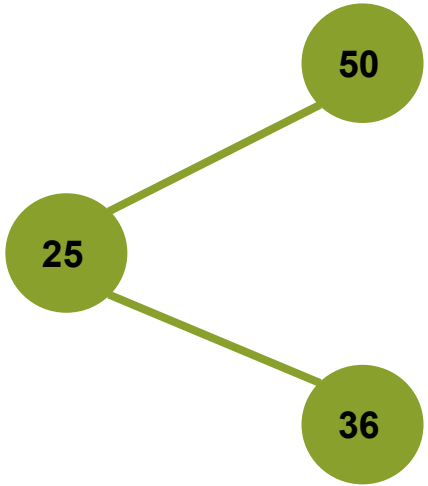
binary search tree - insert (36)



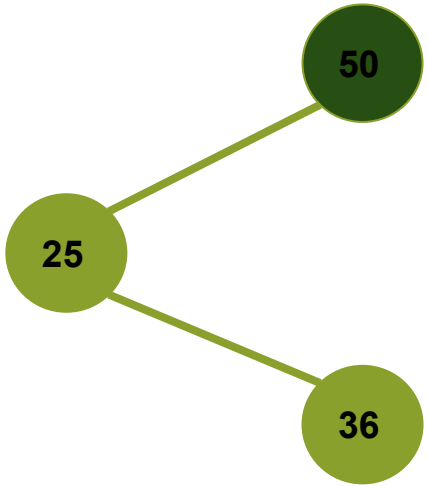
binary search tree - insert (36)



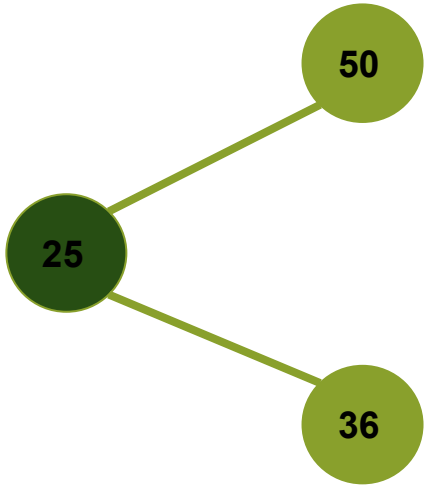
binary search tree - insert (10)



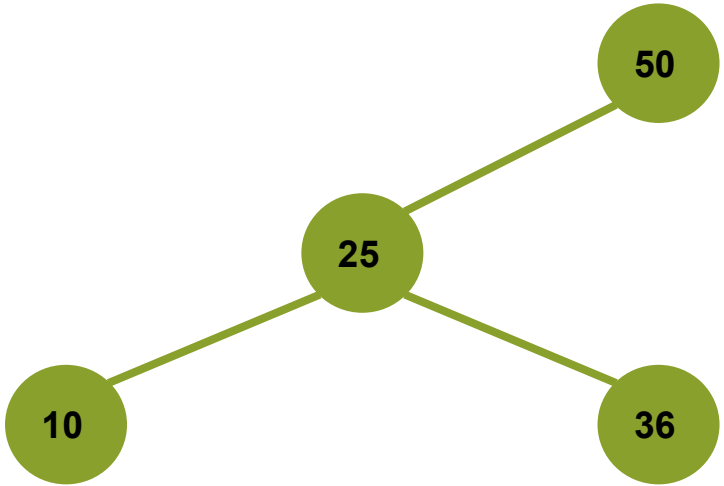
binary search tree - insert (10)



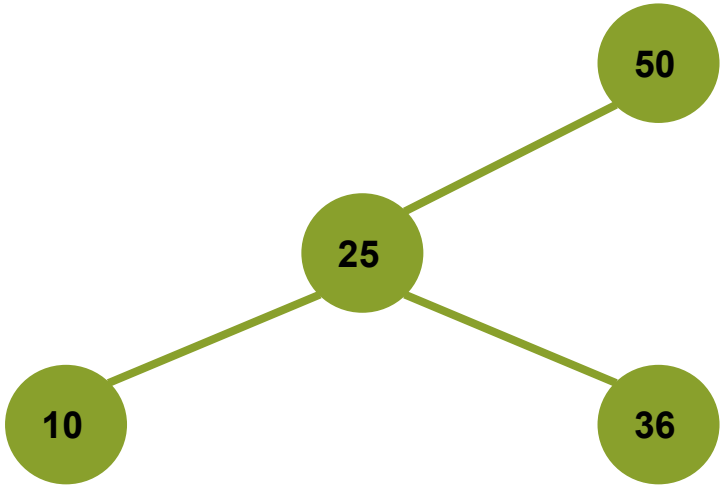
binary search tree - insert (10)



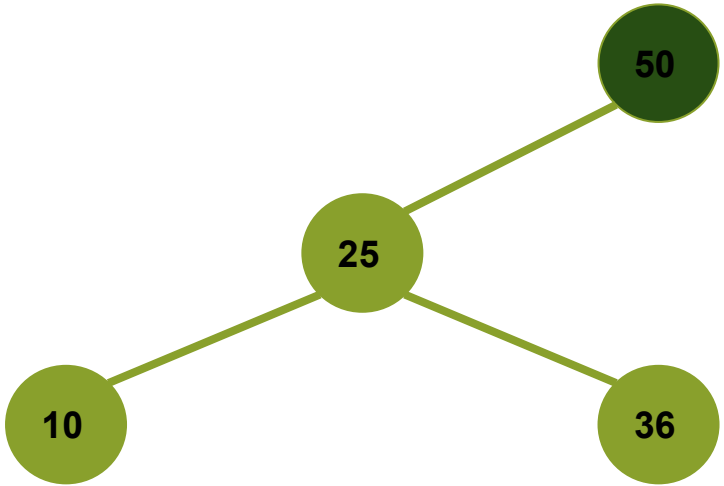
binary search tree - insert (10)



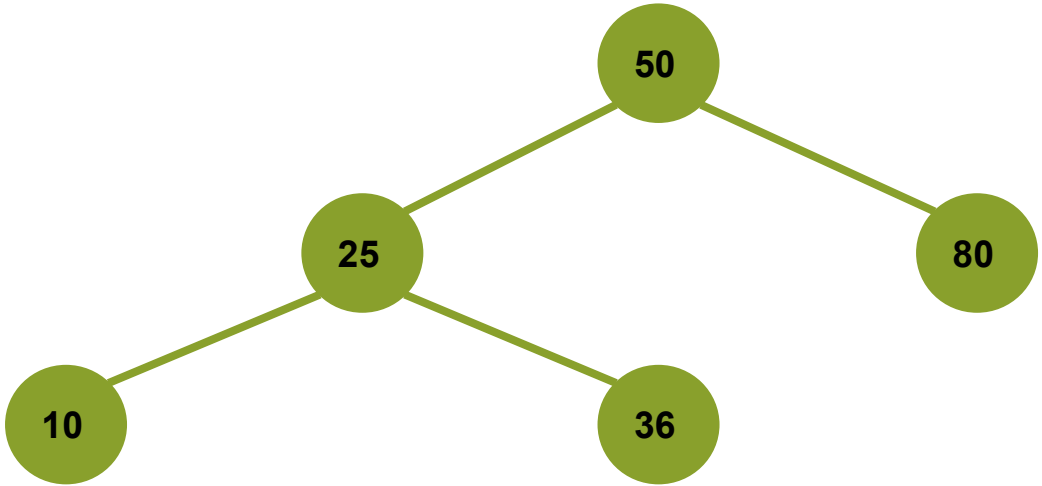
binary search tree - insert (80)



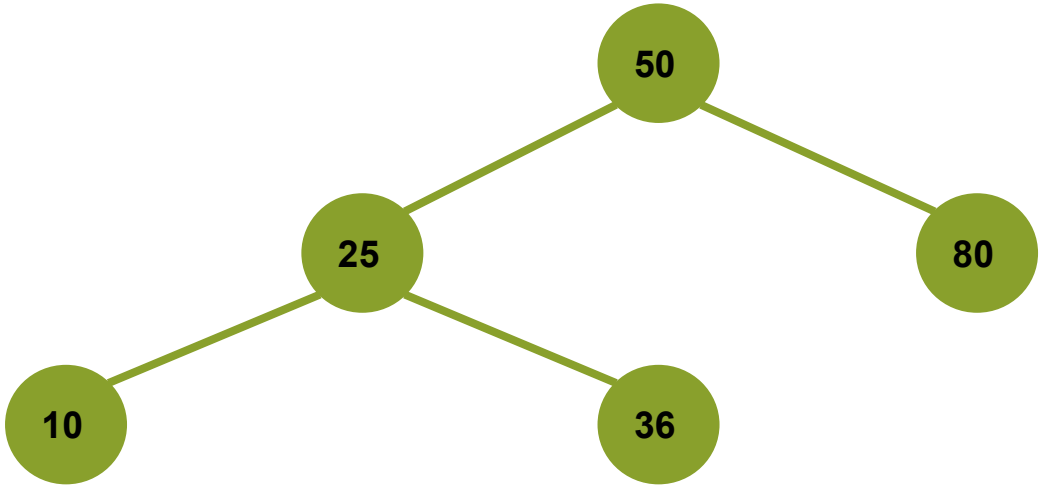
binary search tree - insert (80)



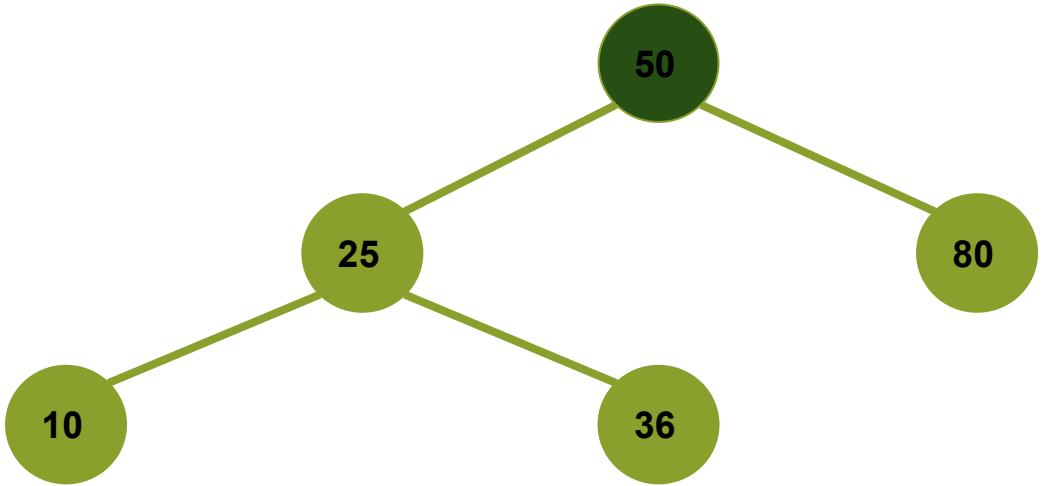
binary search tree - insert (80)



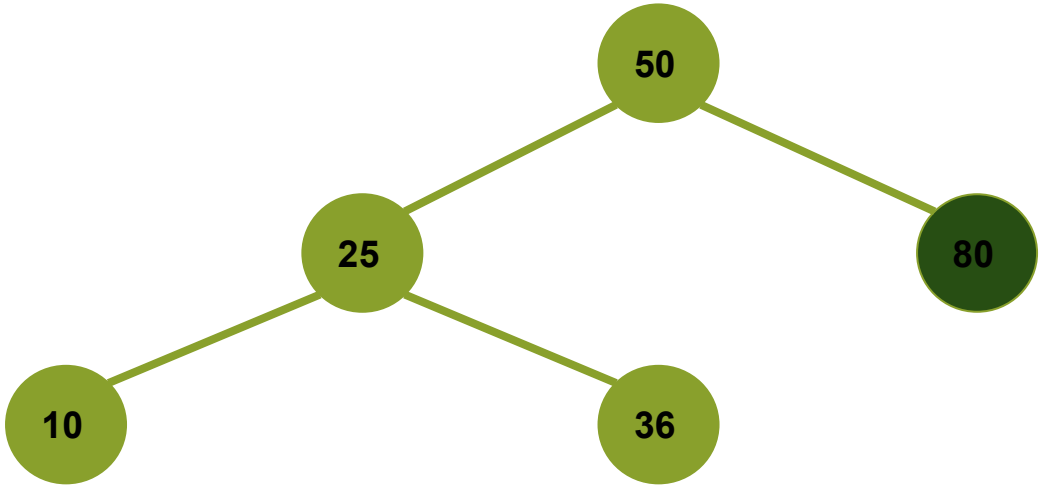
binary search tree - insert (91)



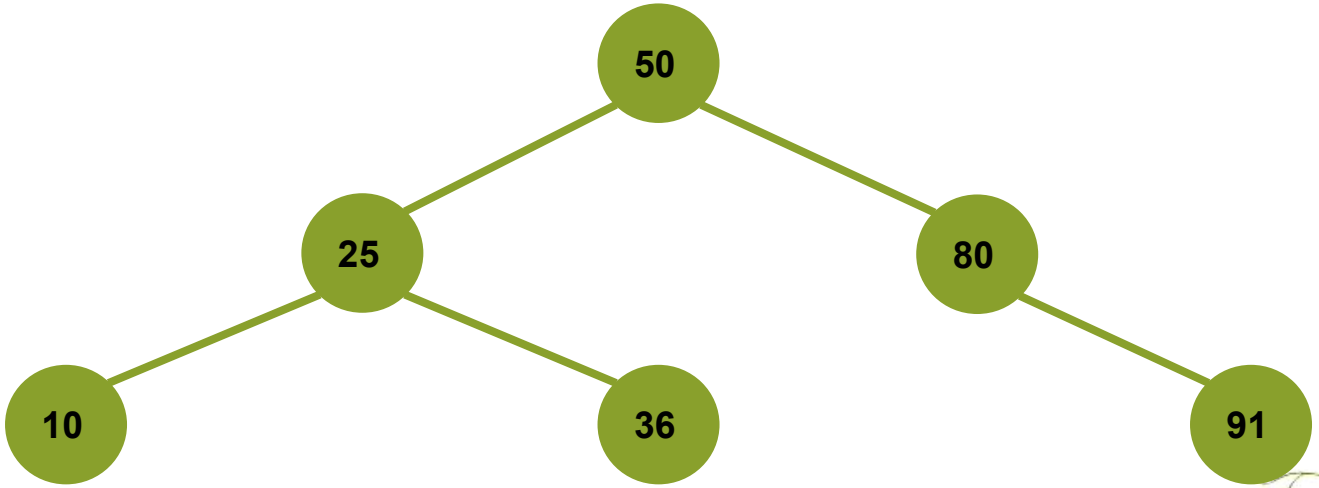
binary search tree - insert (91)



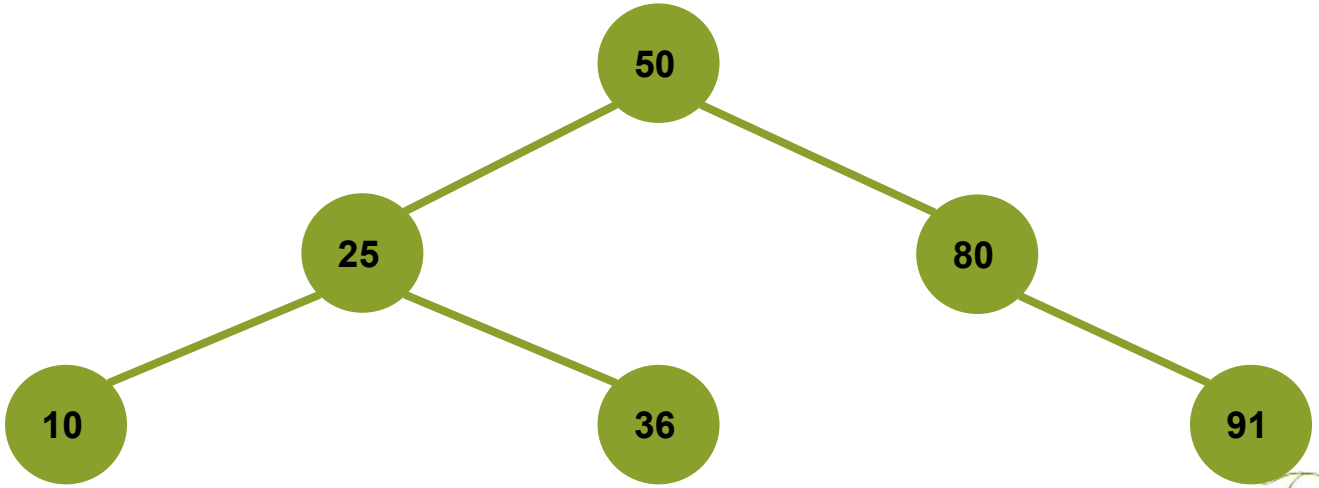
binary search tree - insert (91)



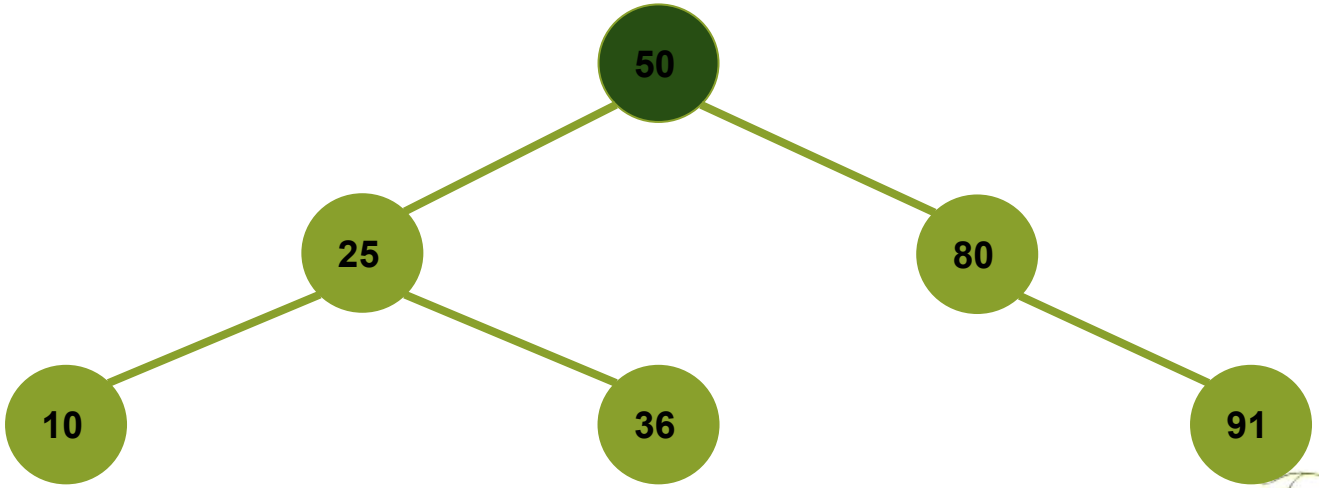
binary search tree - insert (91)



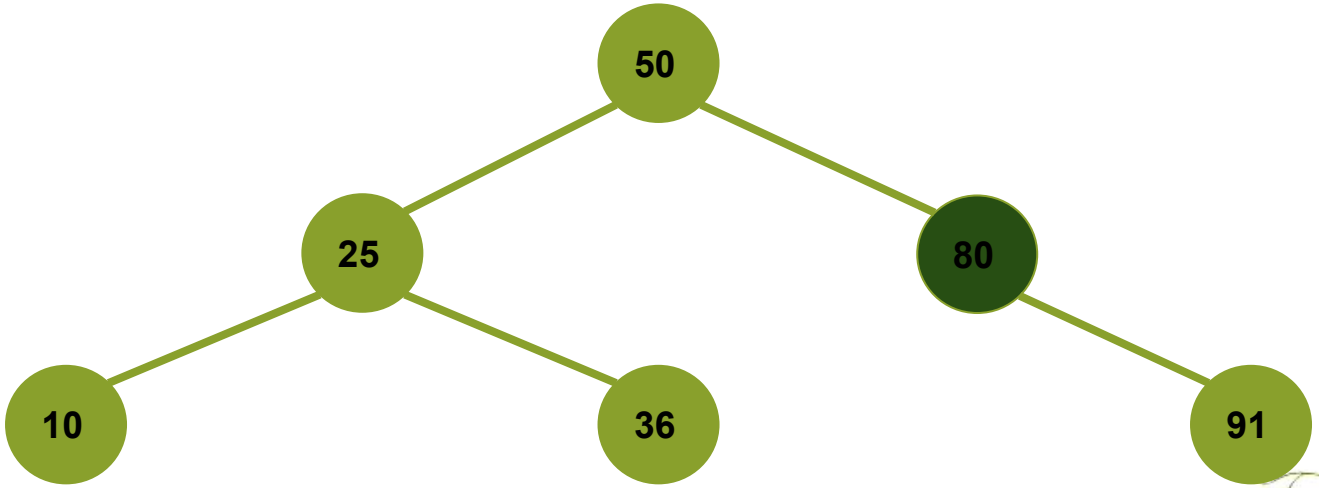
binary search tree - insert (84)



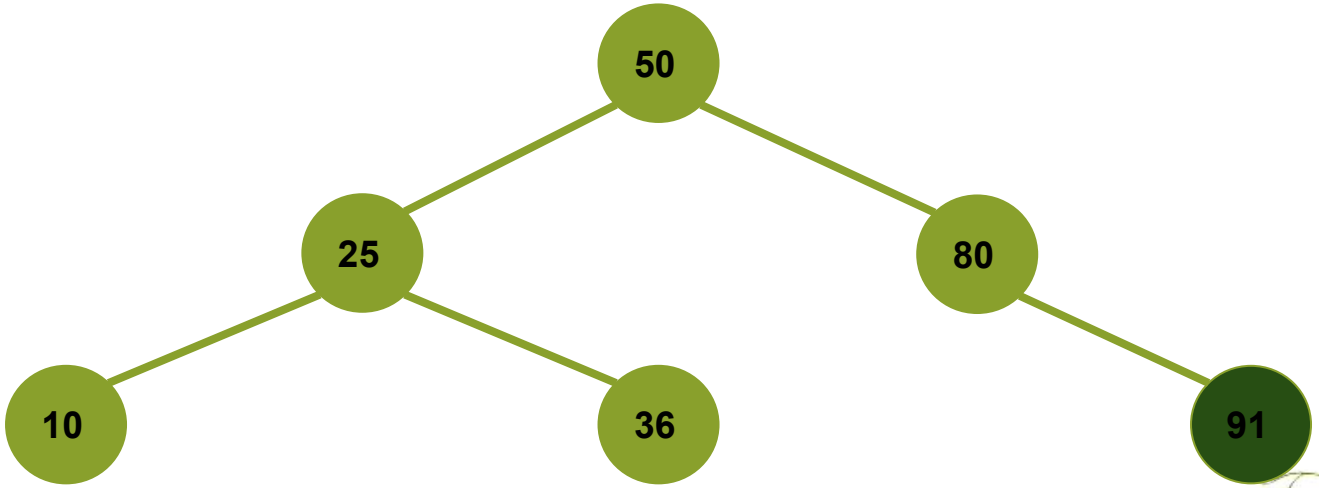
binary search tree - insert (84)



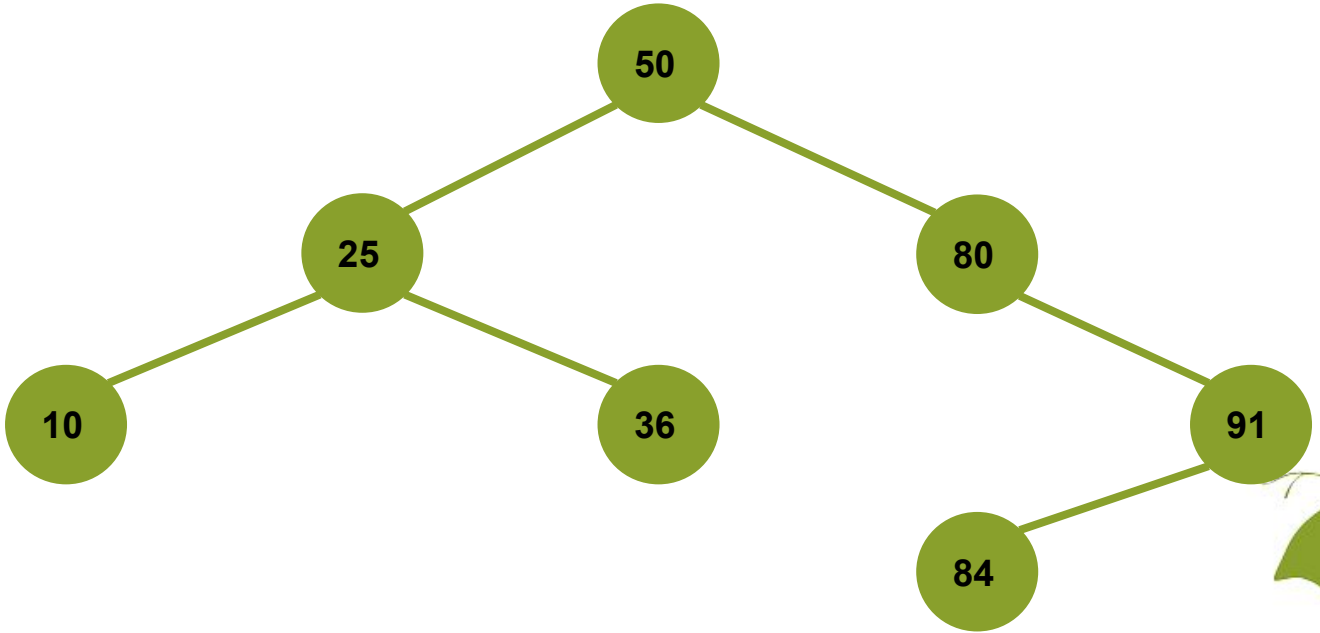
binary search tree - insert (84)



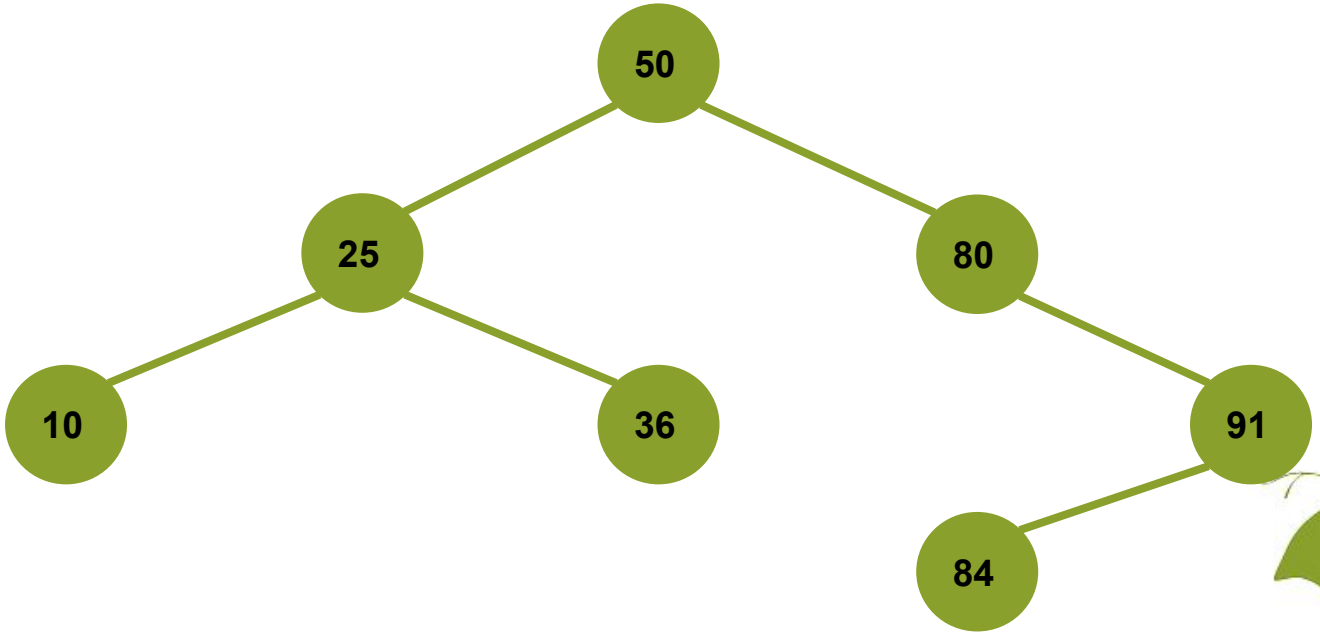
binary search tree - insert (84)



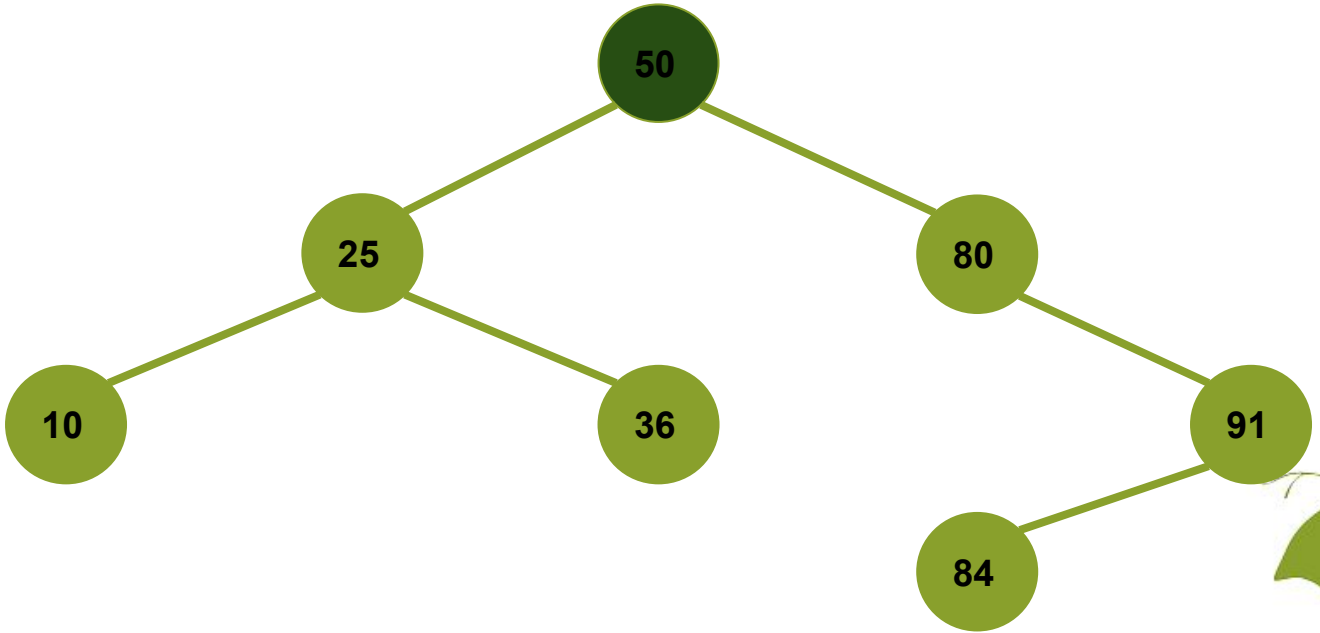
binary search tree - insert (84)



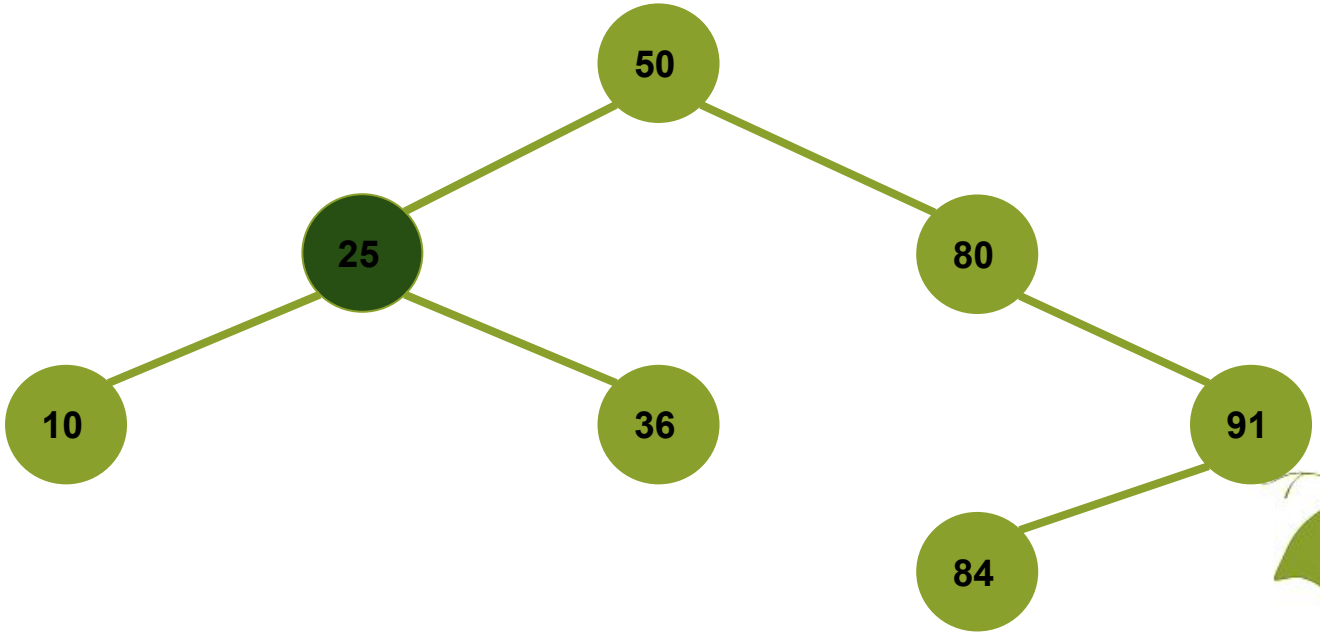
binary search tree - insert (29)



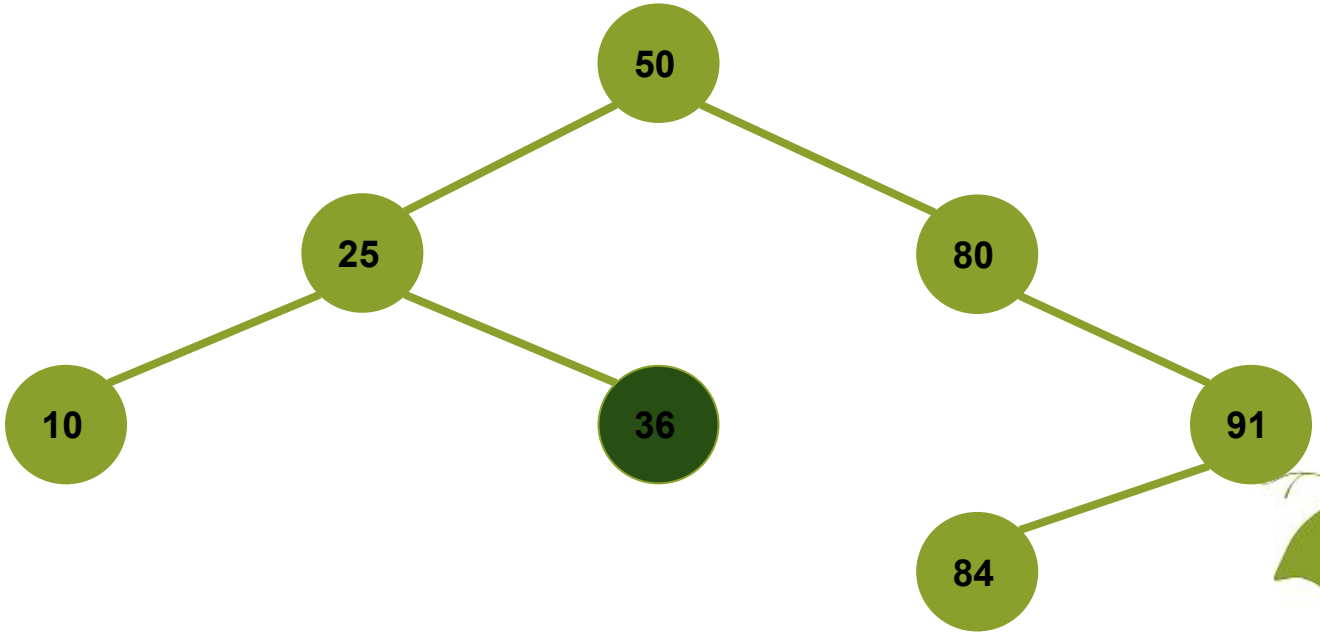
binary search tree - insert (29)



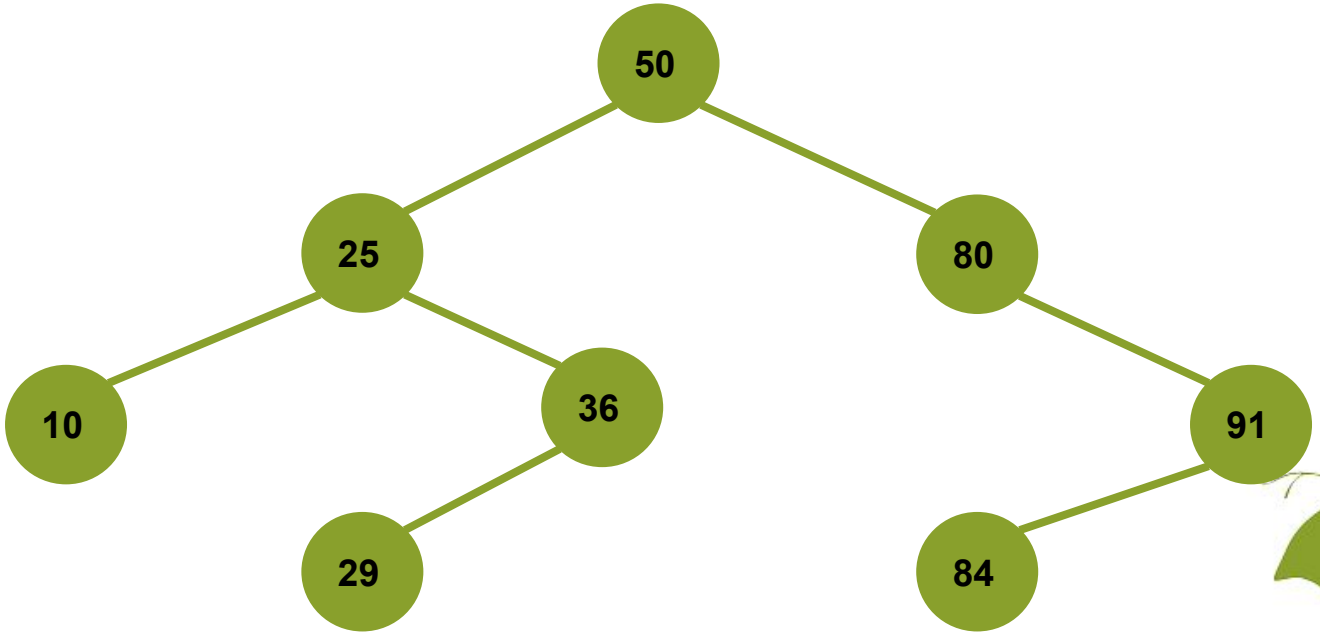
binary search tree - insert (29)



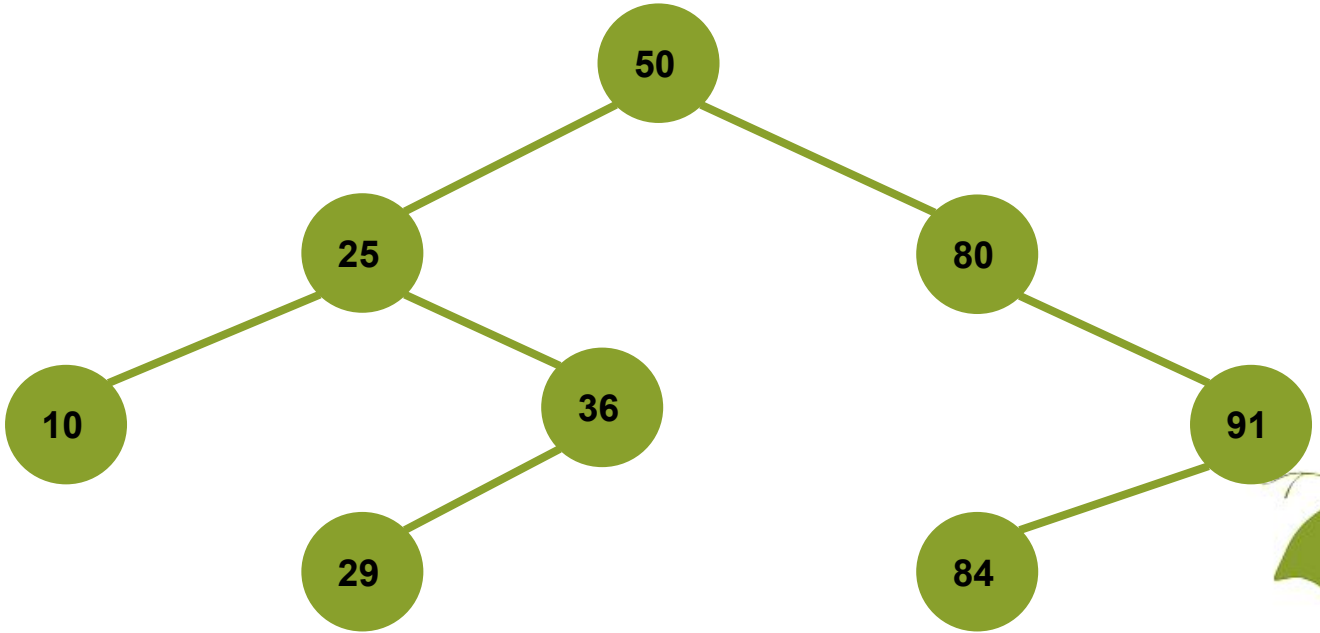
binary search tree - insert (29)



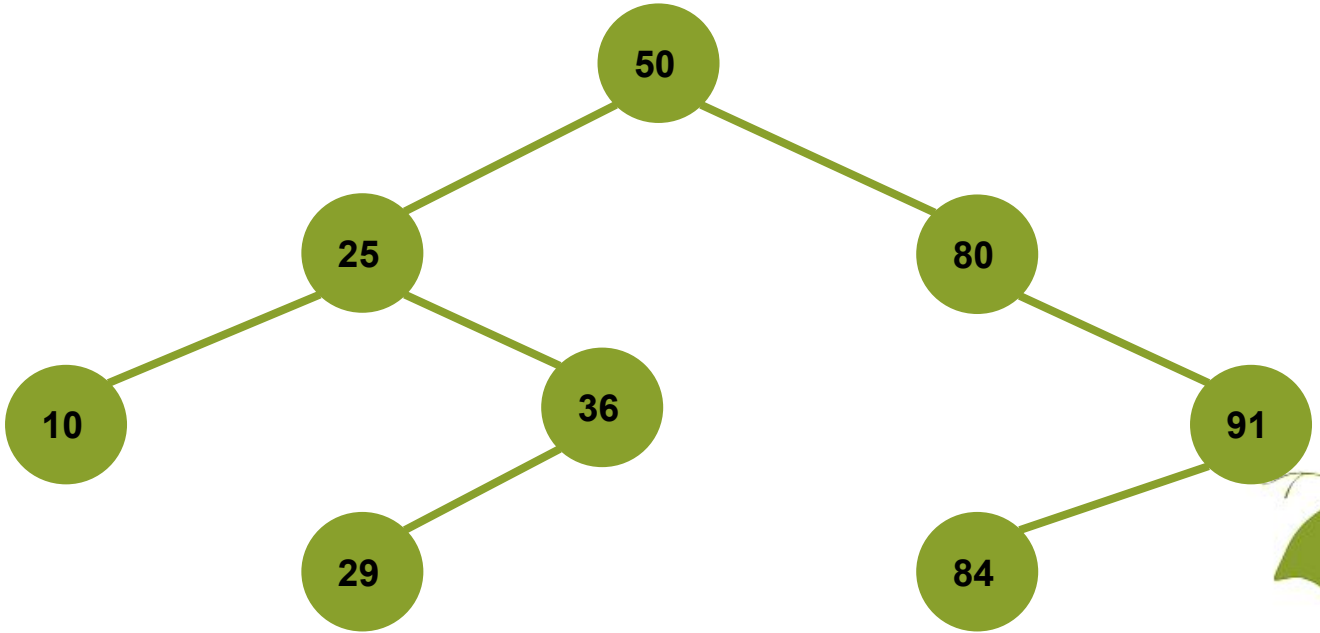
binary search tree - insert (29)



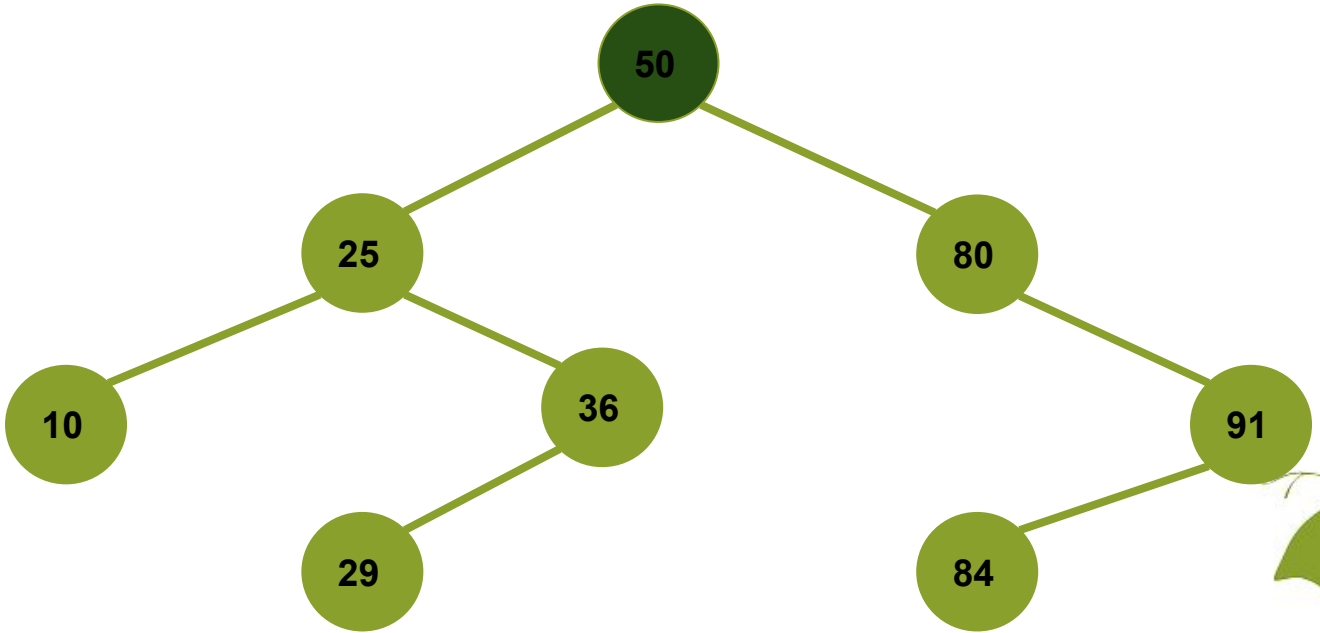
binary search tree - remove



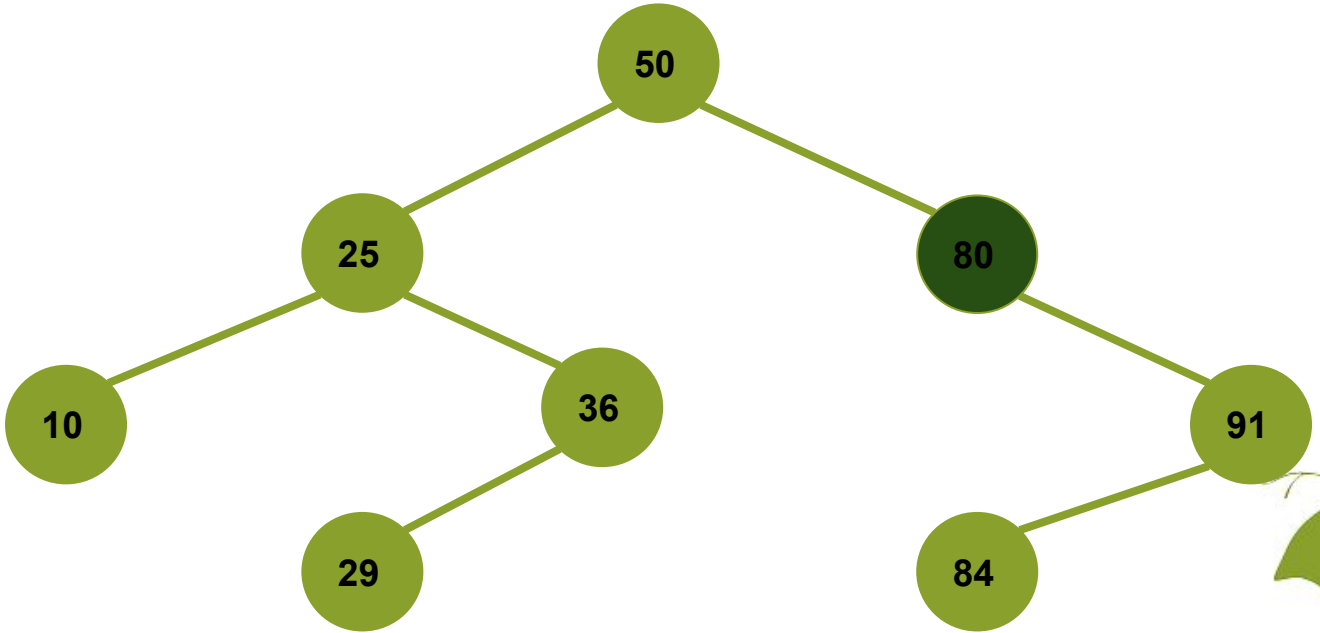
binary search tree - remove (84 - a leaf)



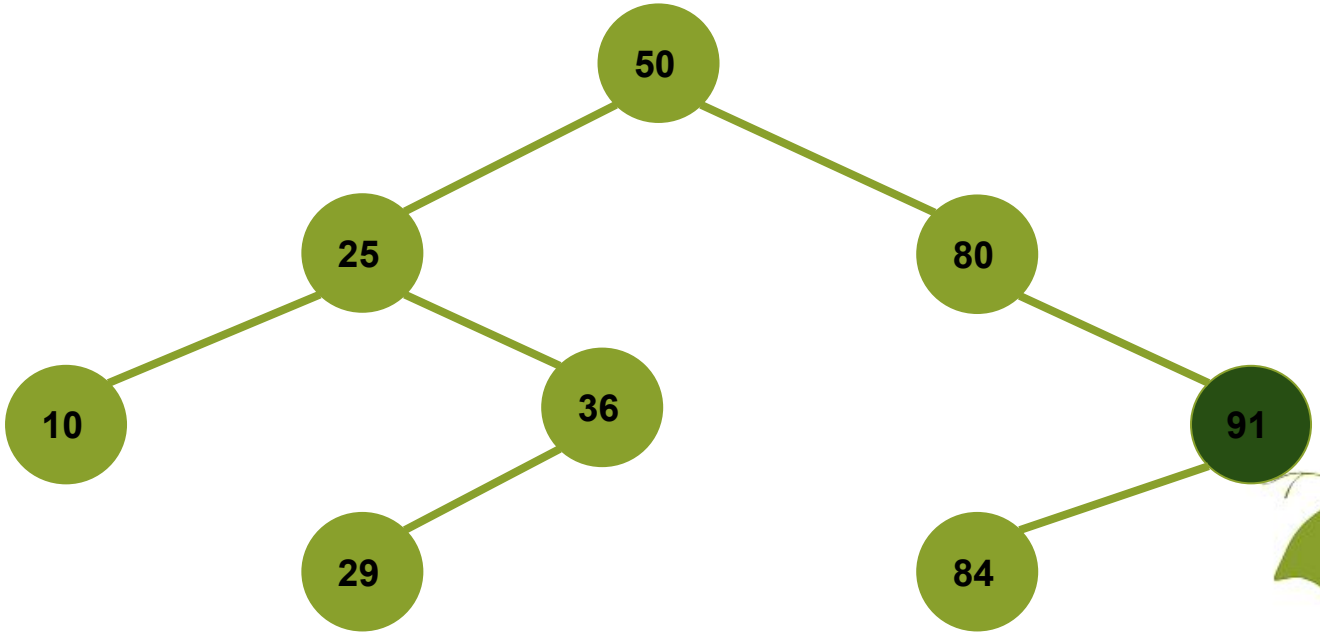
binary search tree - remove (84 - a leaf)



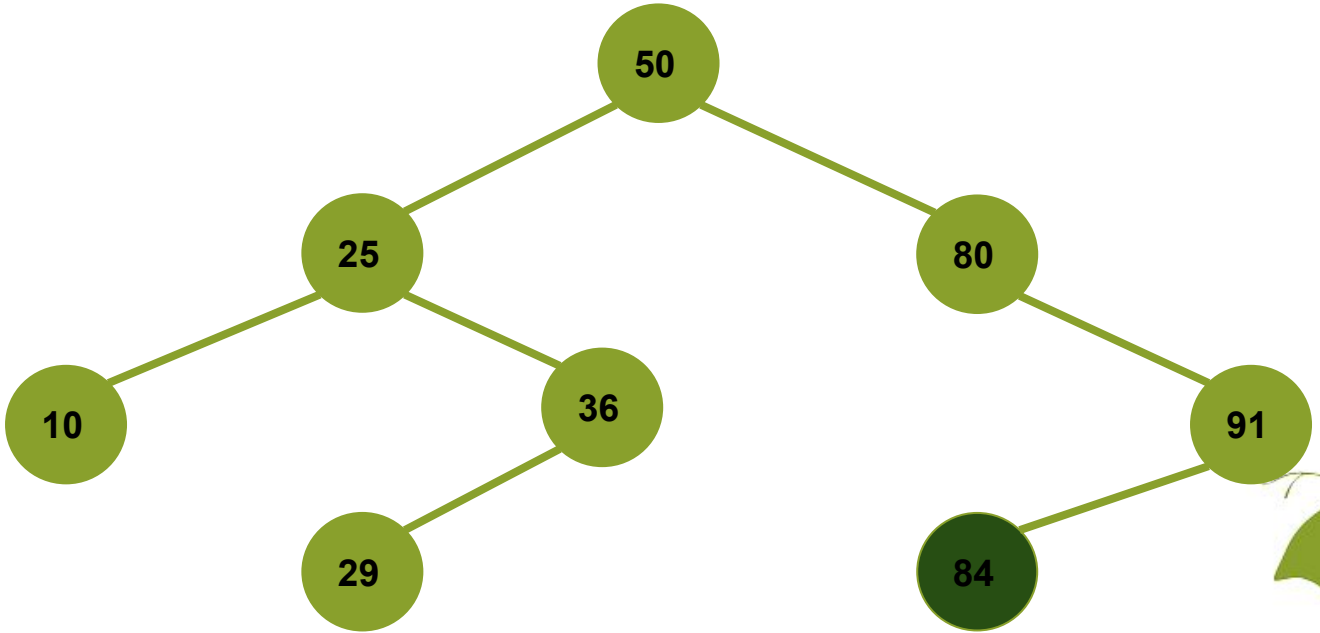
binary search tree - remove (84 - a leaf)



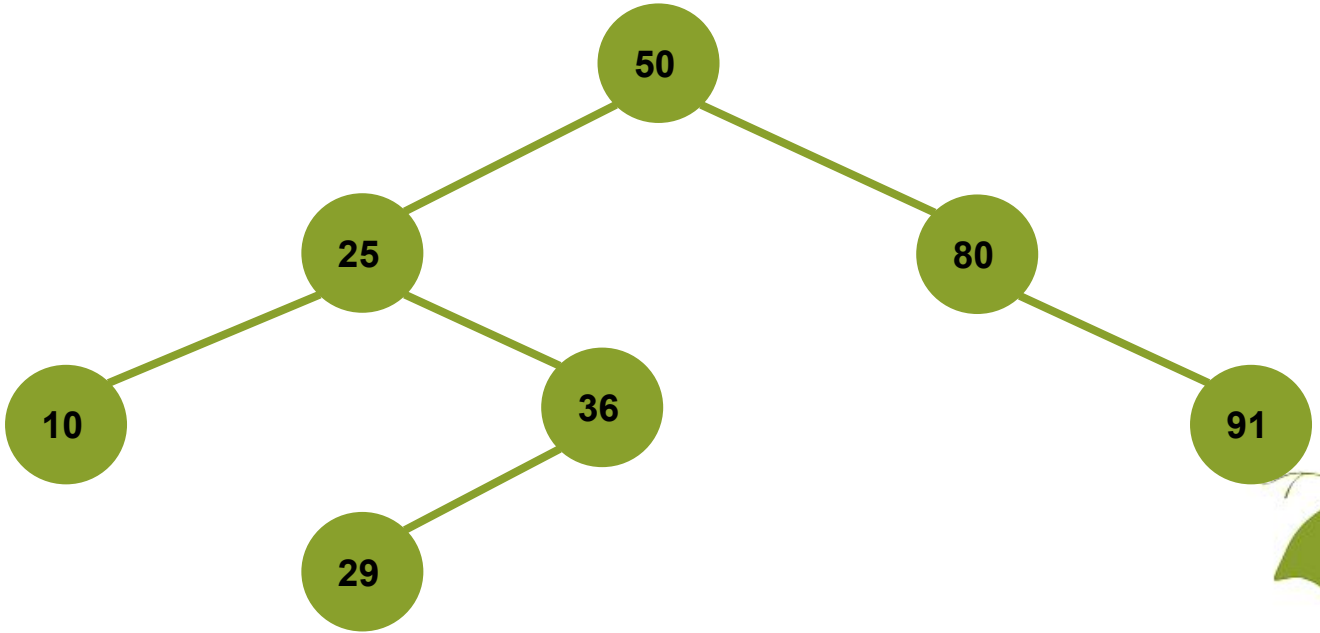
binary search tree - remove (84 - a leaf)



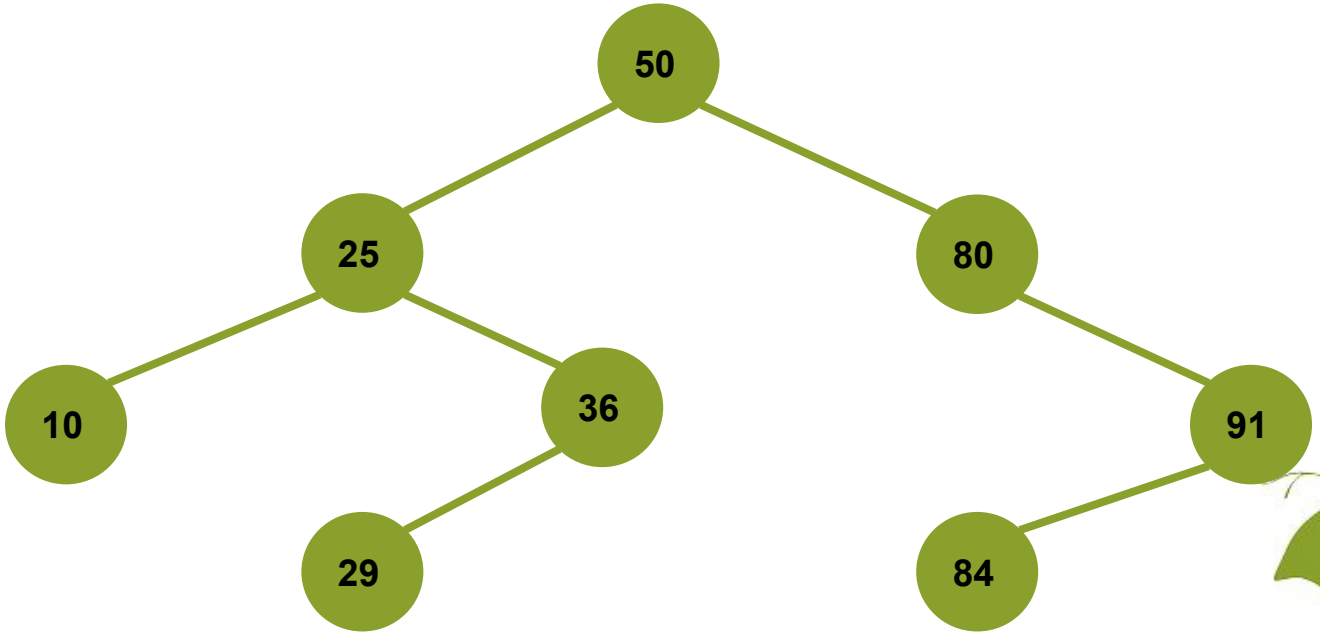
binary search tree - remove (84 - a leaf)



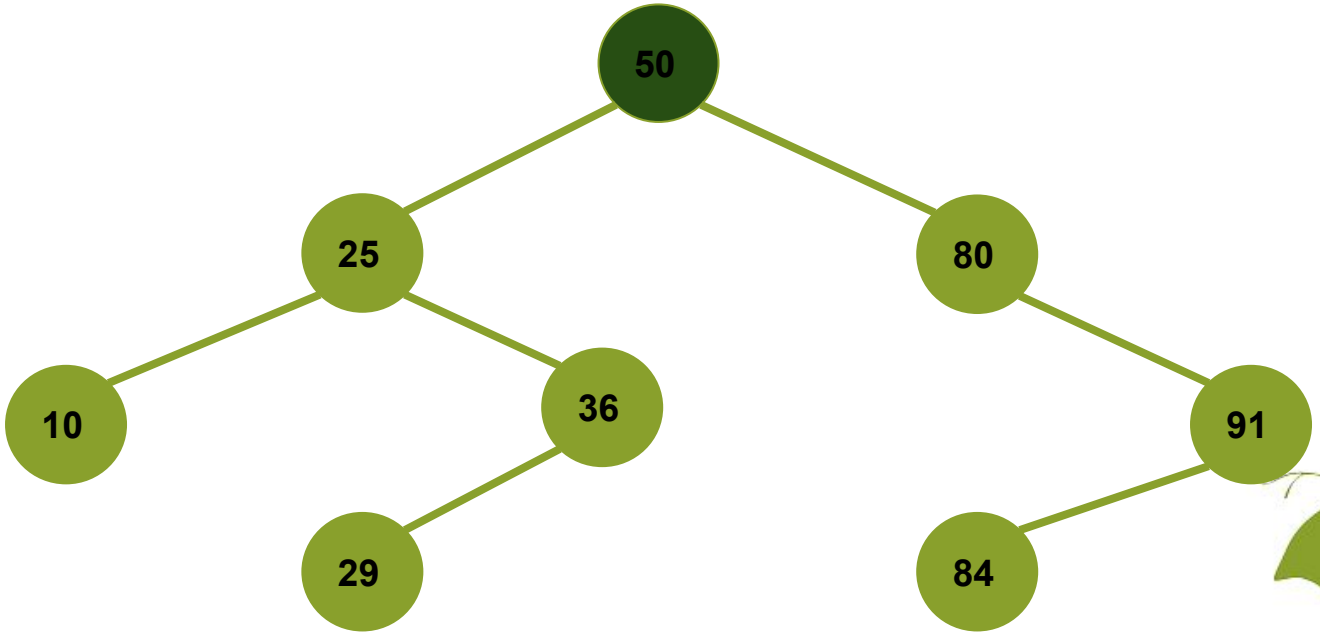
binary search tree - remove (84 - a leaf)



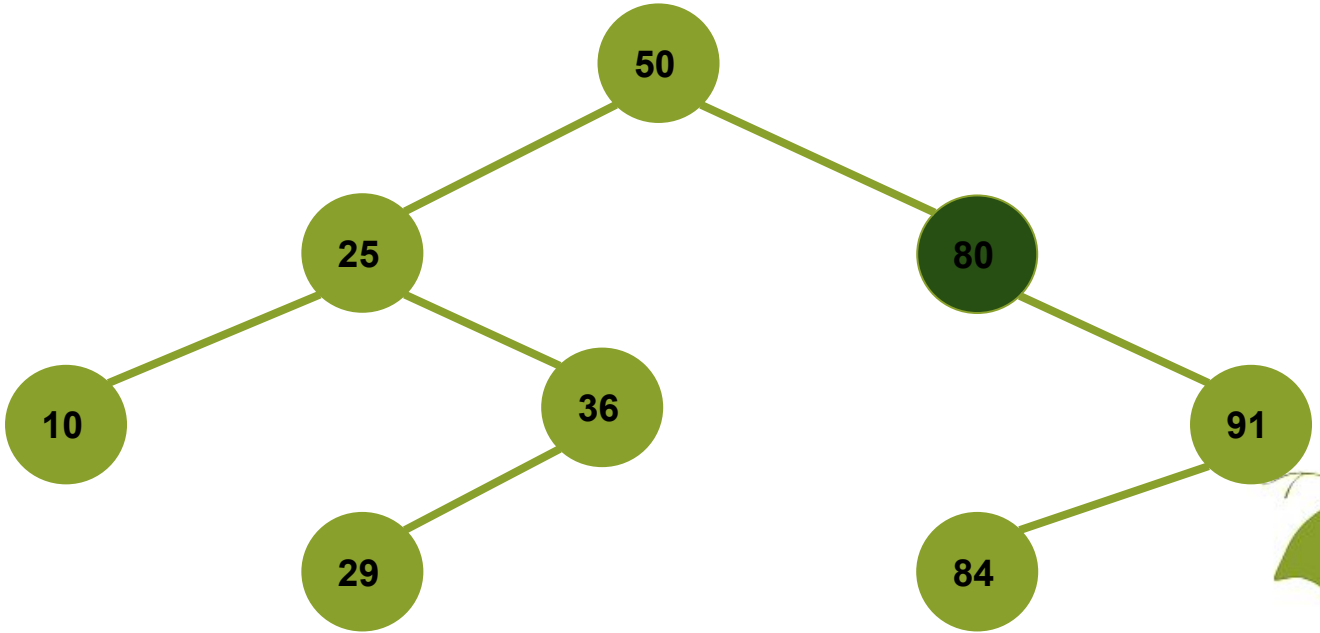
binary search tree - remove (80 - has a child)



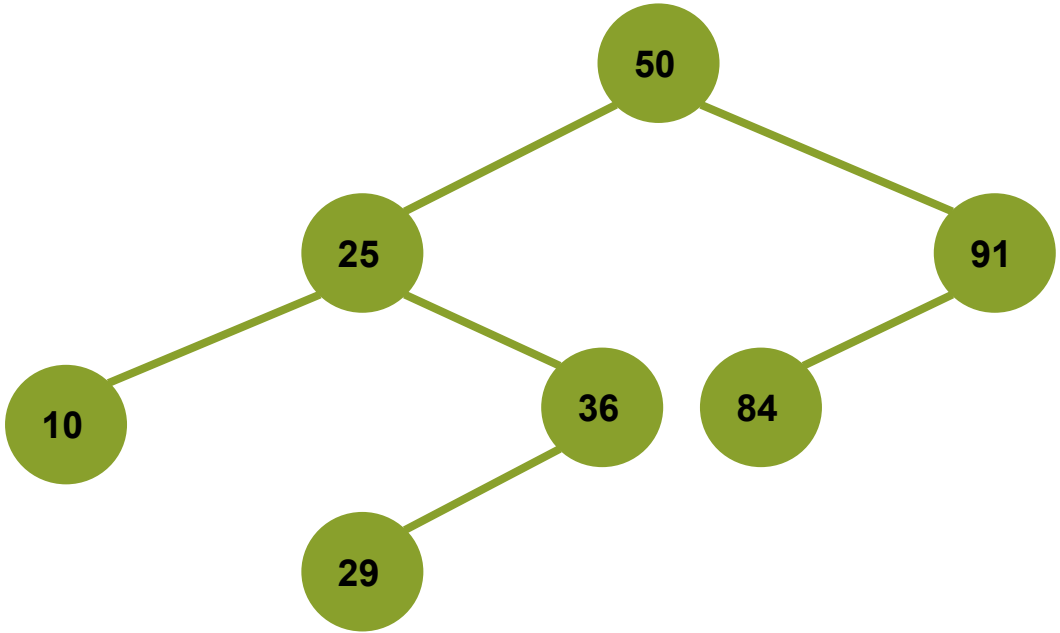
binary search tree - remove (80 - has a child)



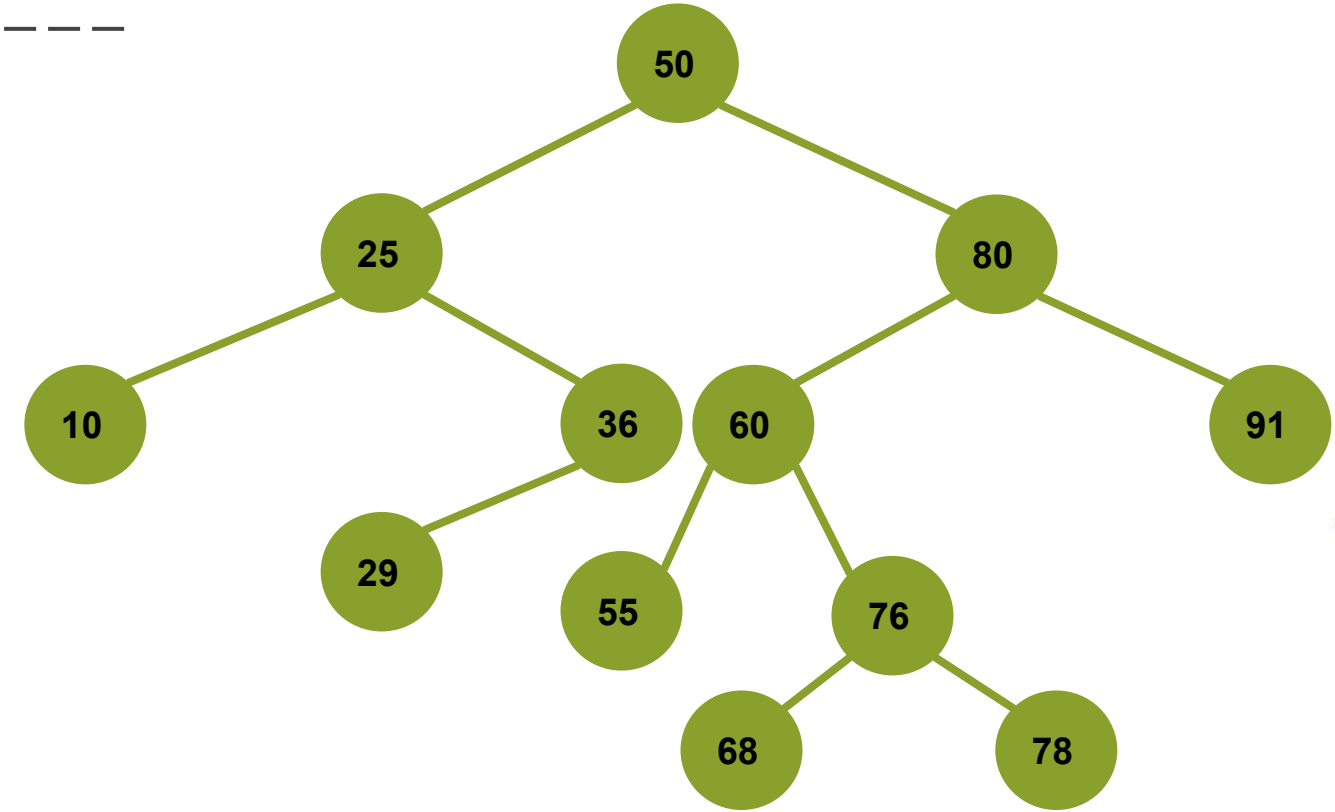
binary search tree - remove (80 - has a child)



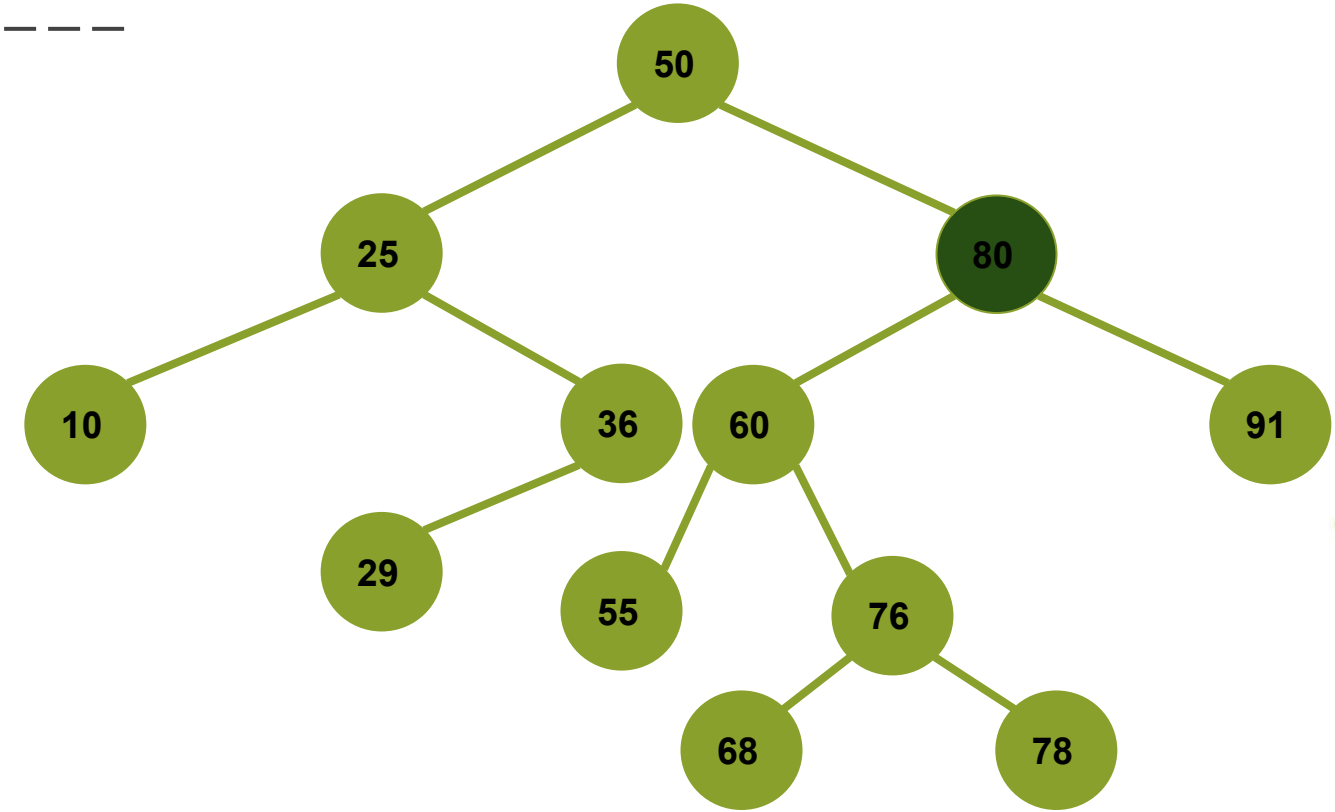
binary search tree - remove (80 - has a child)



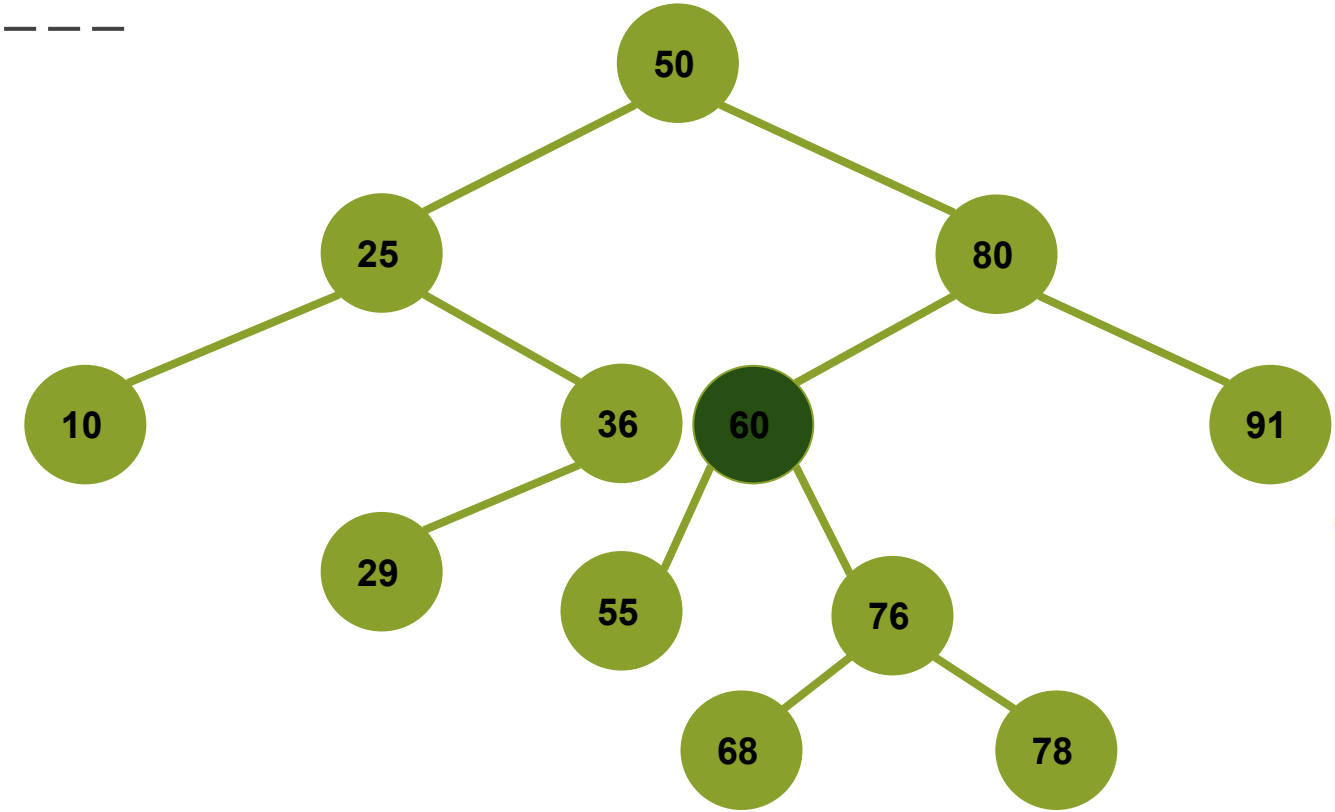
binary search tree - remove (50 - has two children)



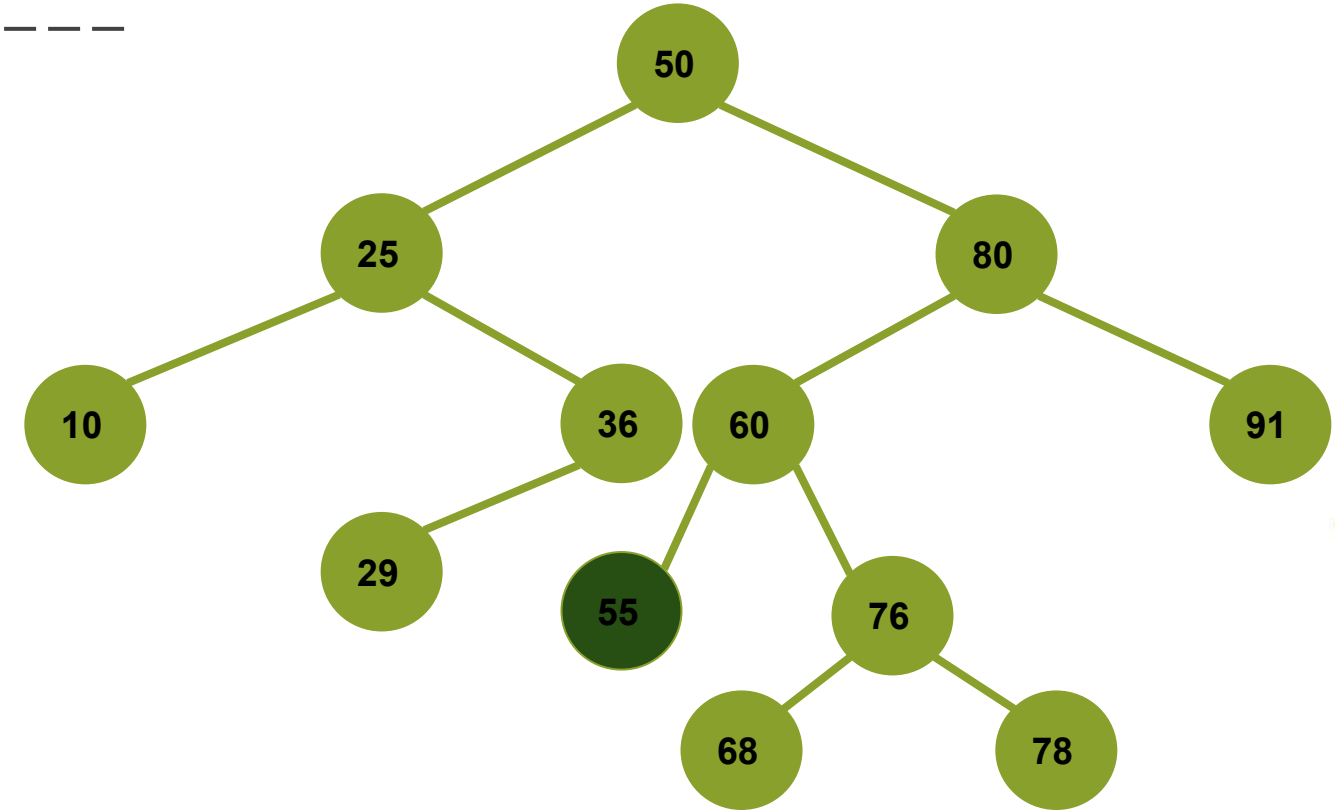
binary search tree - remove (50 - has two children)



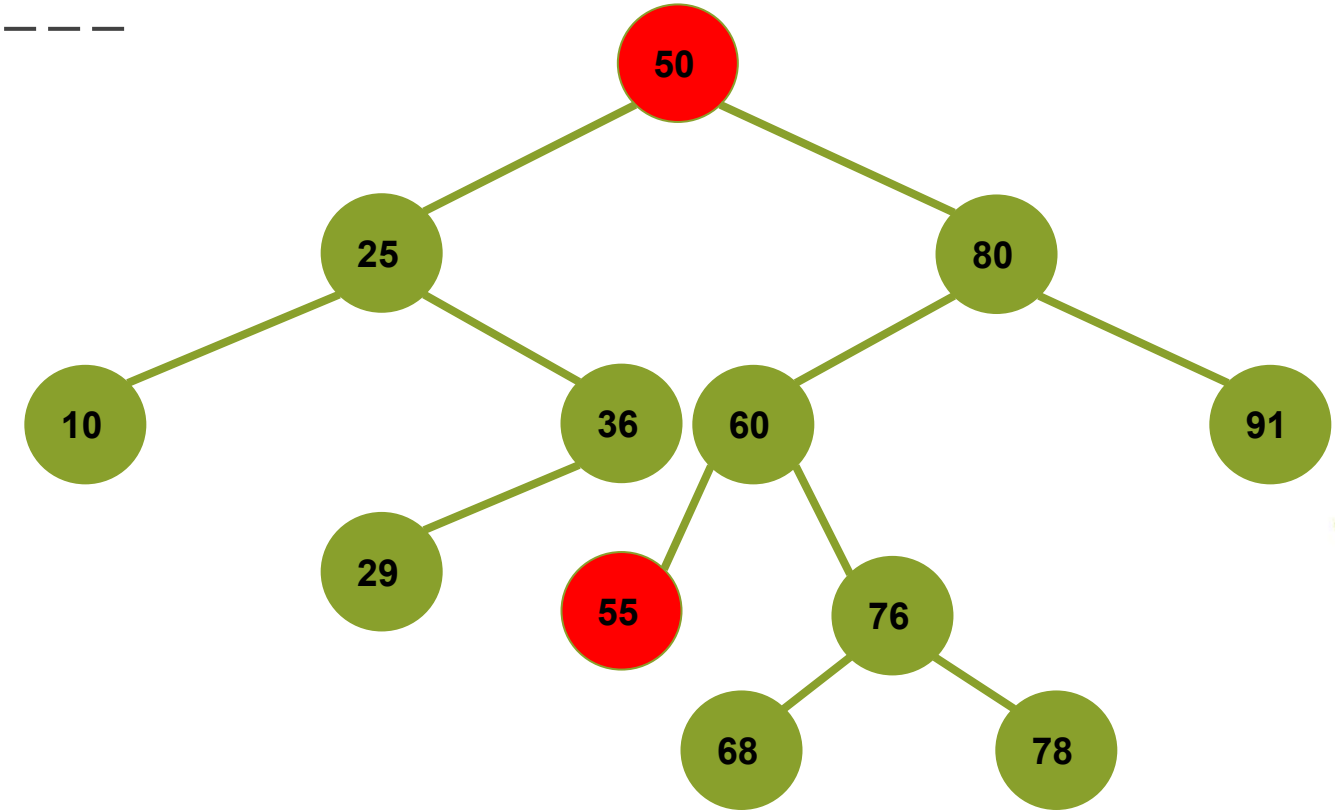
binary search tree - remove (50 - has two children)



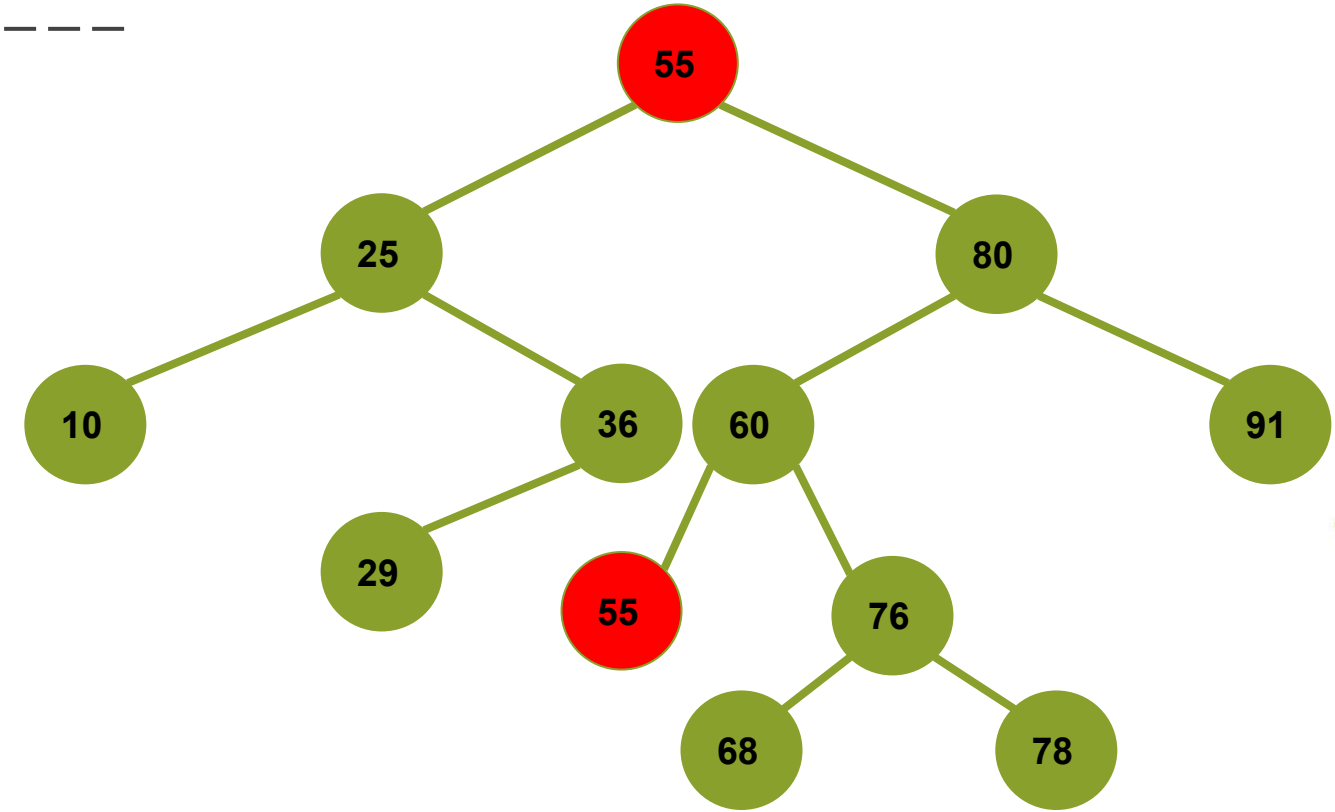
binary search tree - remove (50 - has two children)



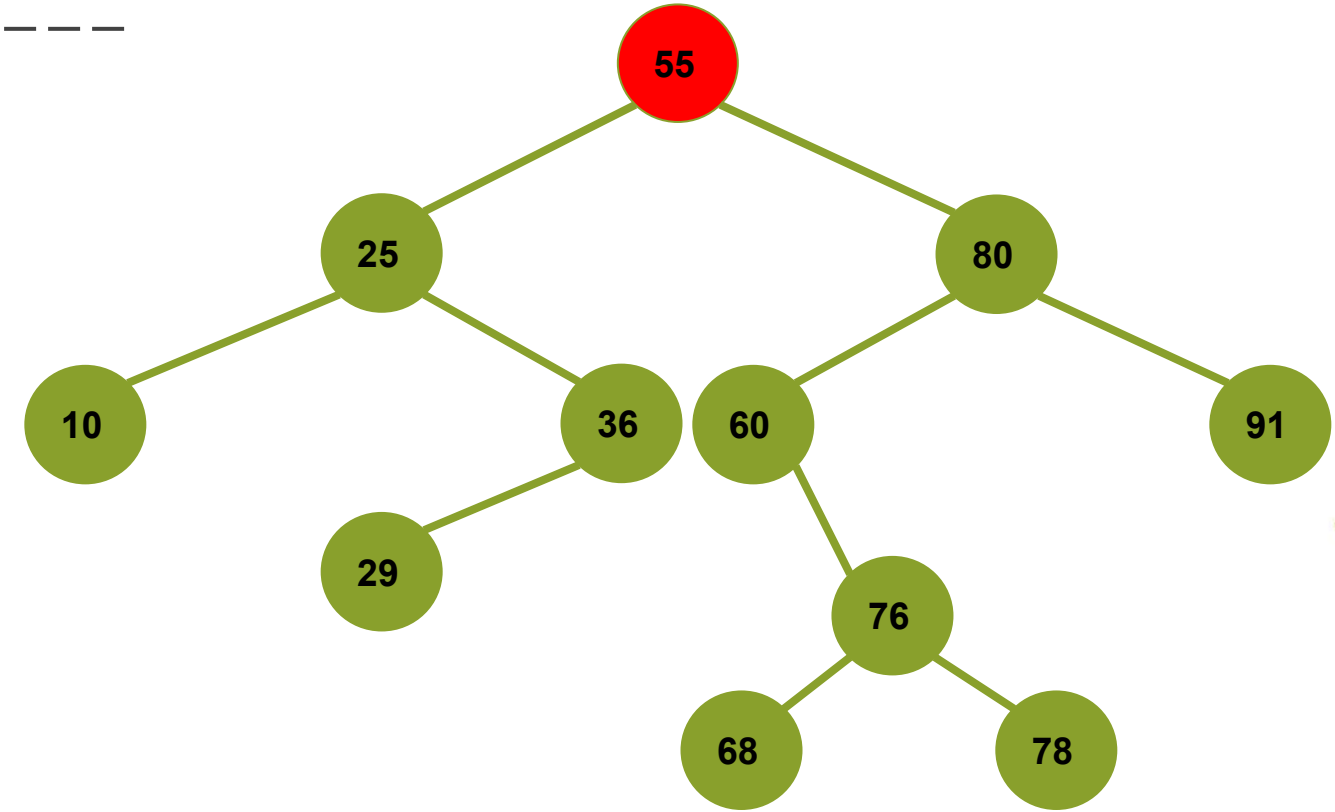
binary search tree - remove (50 - has two children)



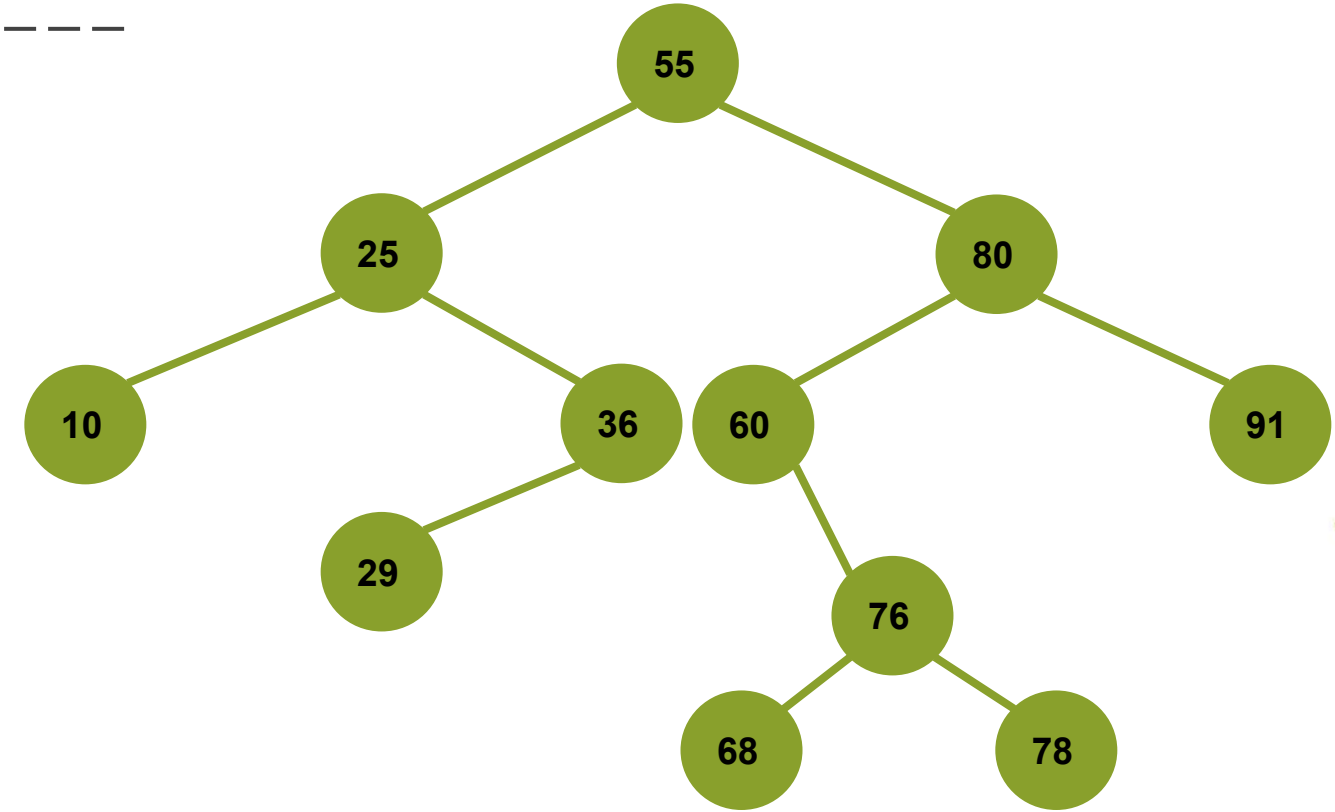
binary search tree - remove (50 - has two children)



binary search tree - remove (50 - has two children)



binary search tree - remove (50 - has two children)



binary search tree - operations' time complexity

operation	time complexity
search	?
minimum	?
maximum	?
insert	?
delete	?

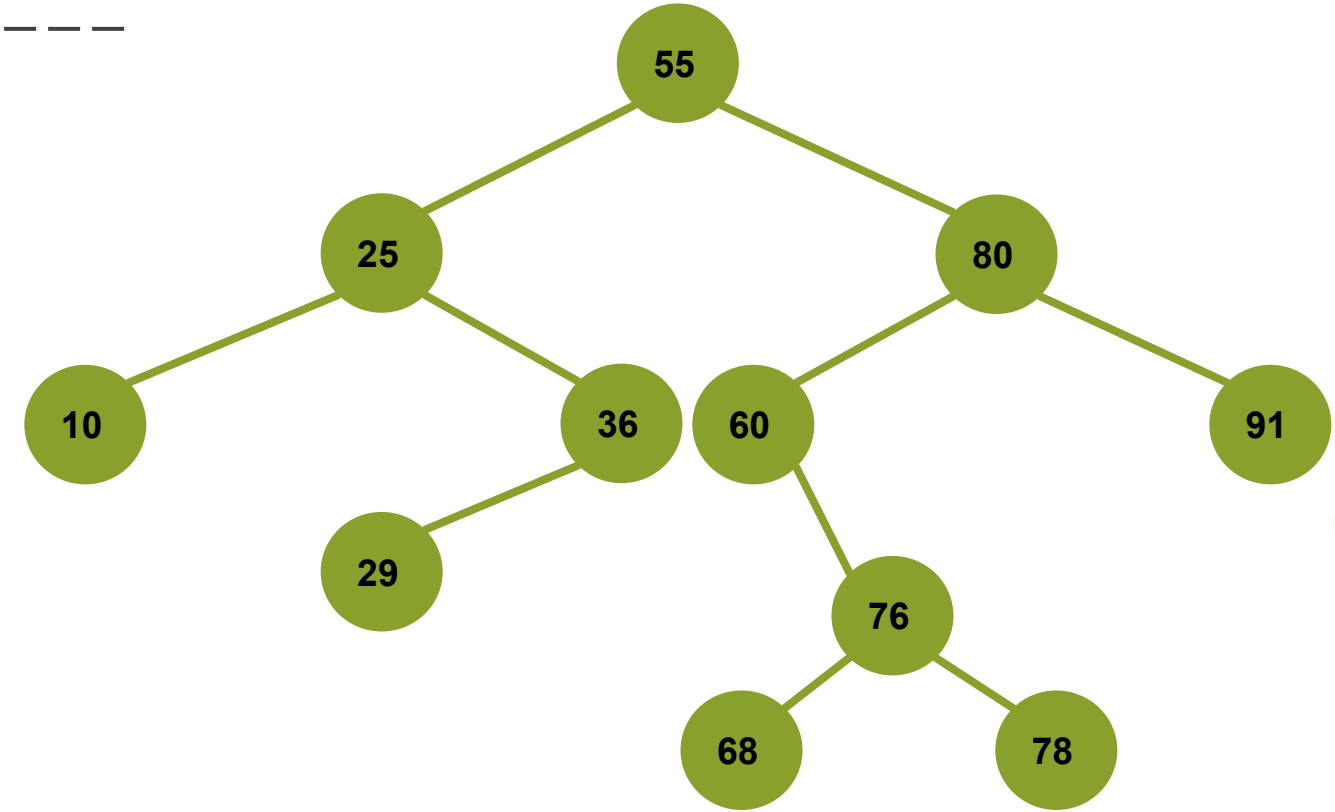


binary search tree

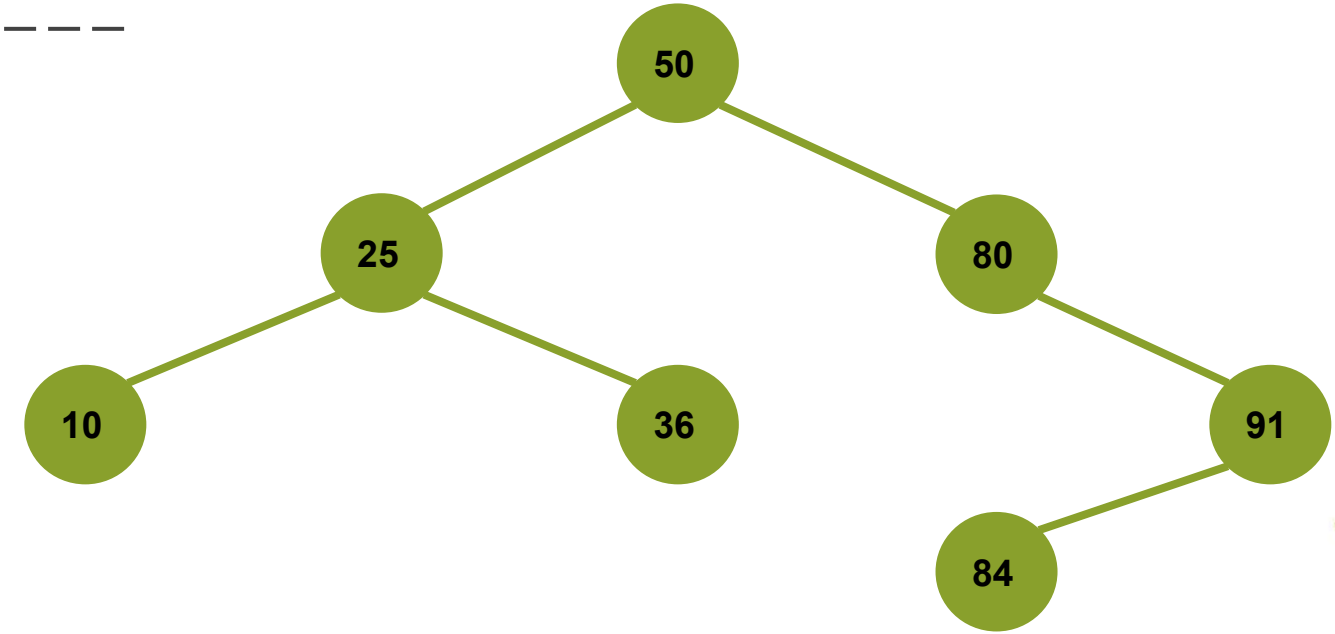
- time complexity of operations
  - depends on the bst is configured
  - height of the tree
    - the number of edges on the longest path from the root to a leaf



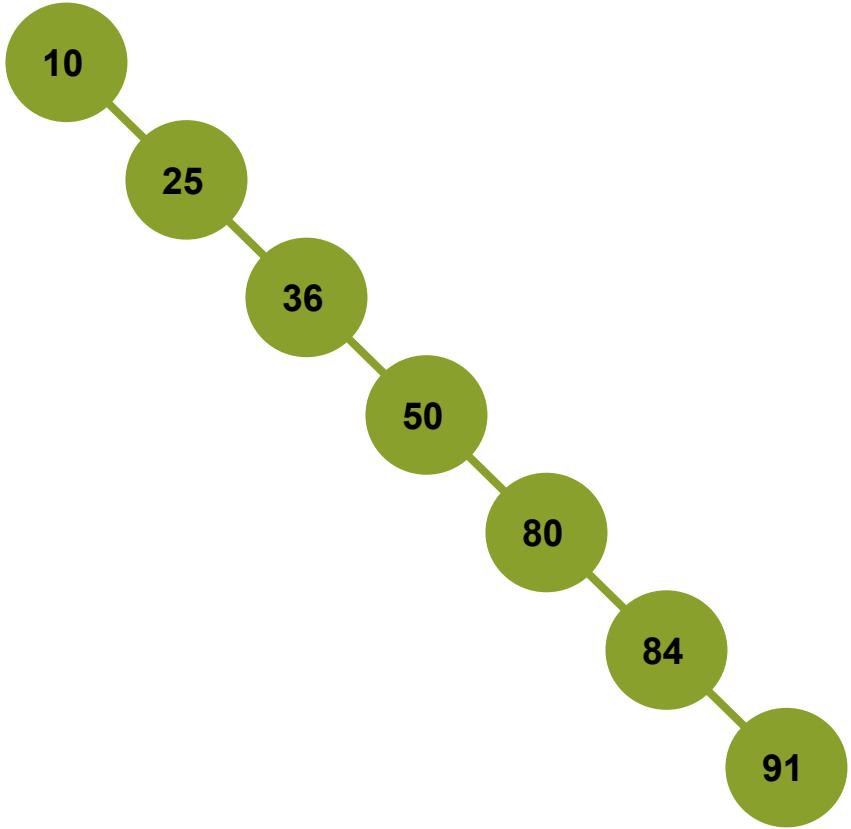
binary search tree - height



binary search tree - height



binary search tree - height (worst case)

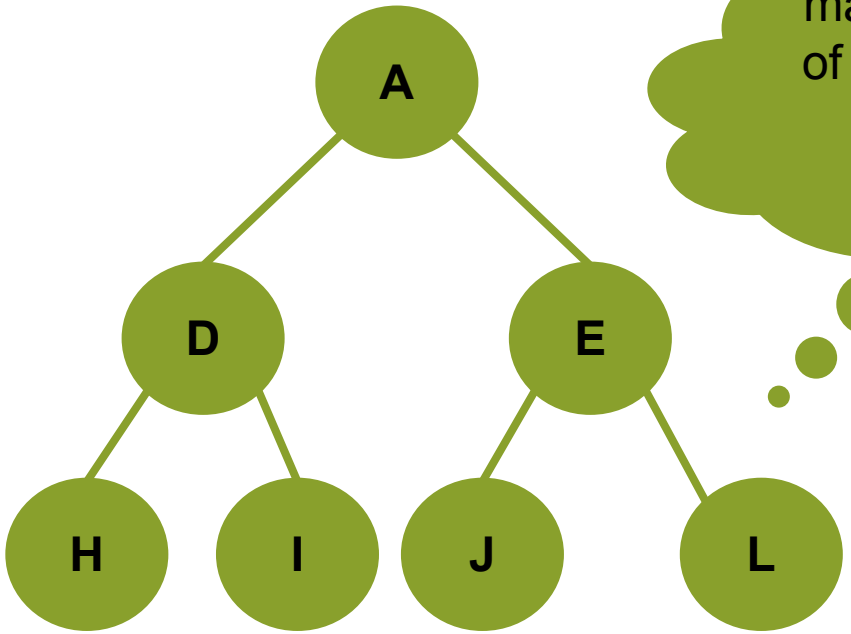


binary search tree - operations' time complexity

operation	time complexity
search	linear
minimum	linear
maximum	linear
insert	linear
delete	linear



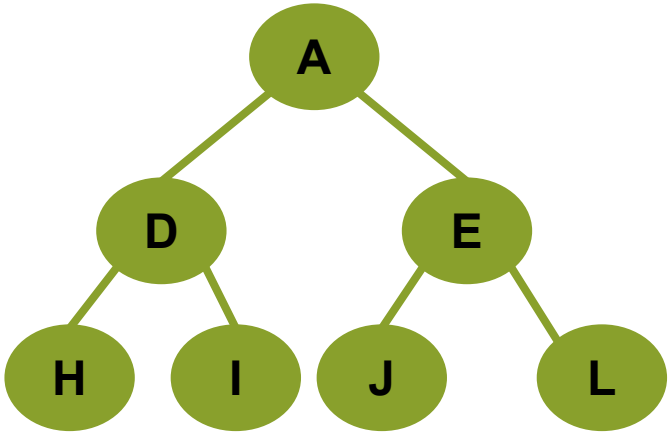
binary search tree - operations' time complexity

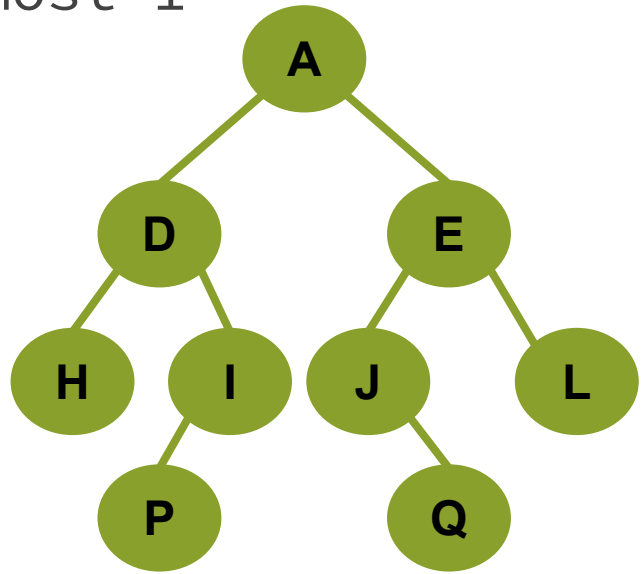


make sure that height of the left subtree and right subtree is balanced

balanced binary search tree

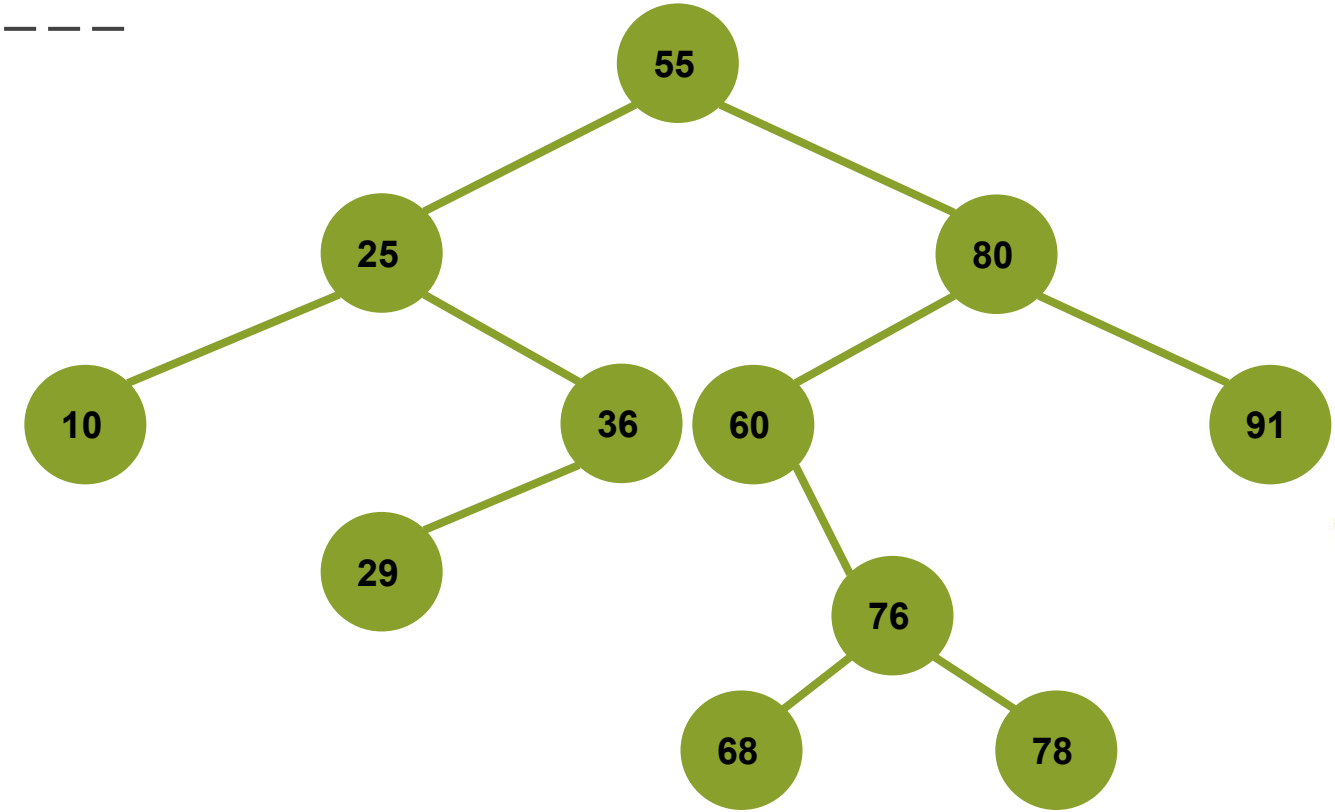
- balance may be defined as
  - heights of the left and right subtrees differ by at most 1



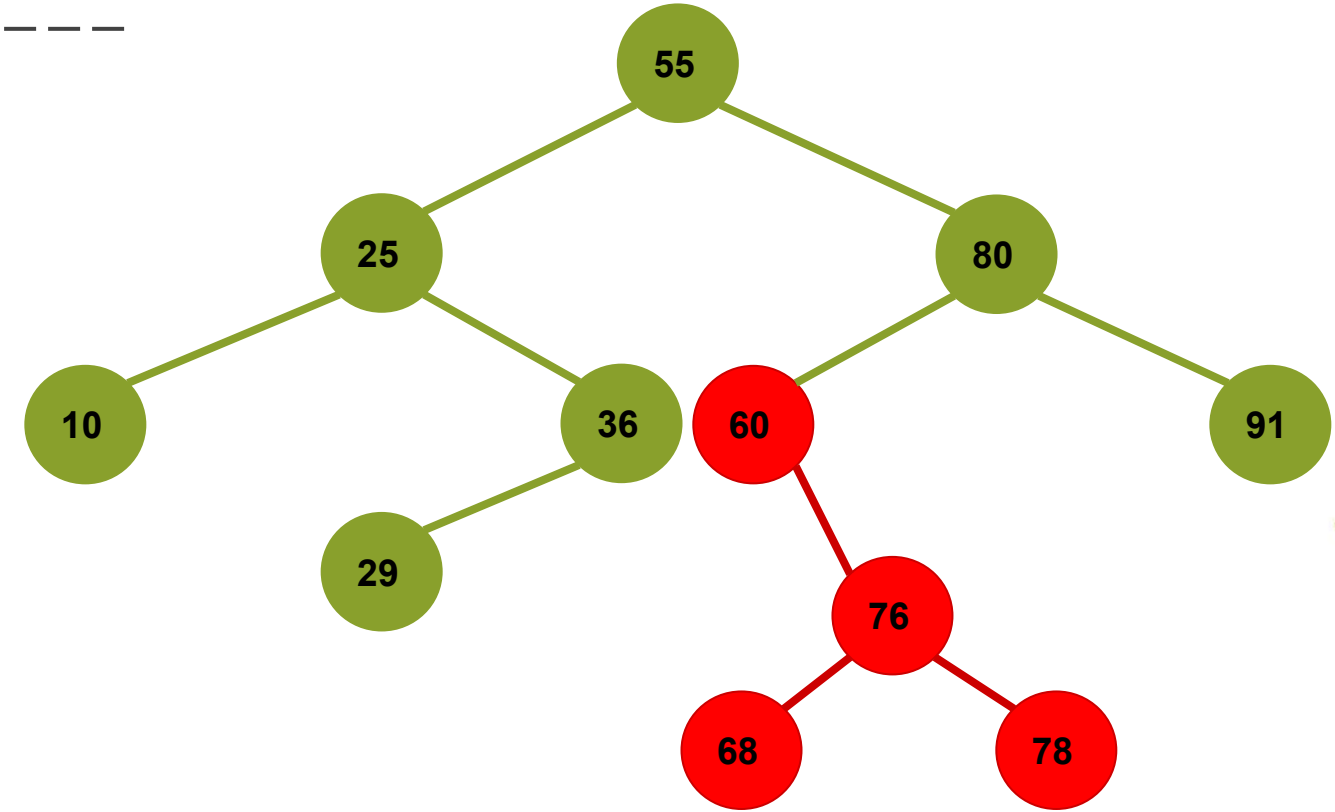




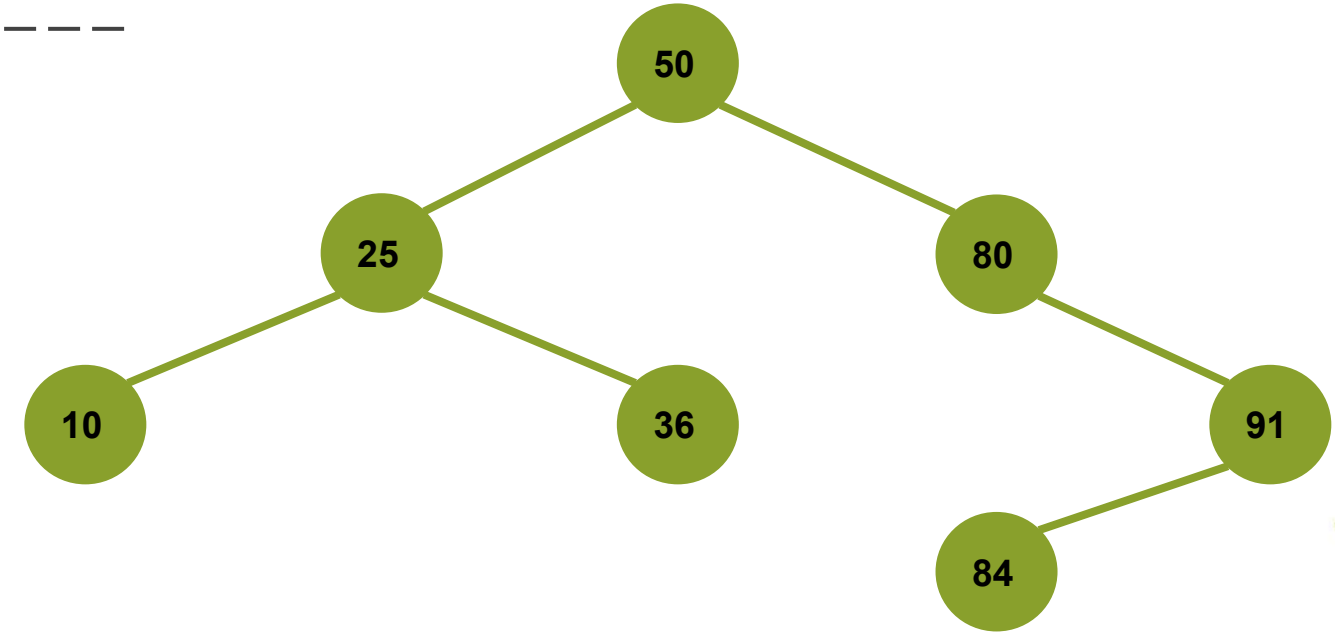
balanced binary search tree



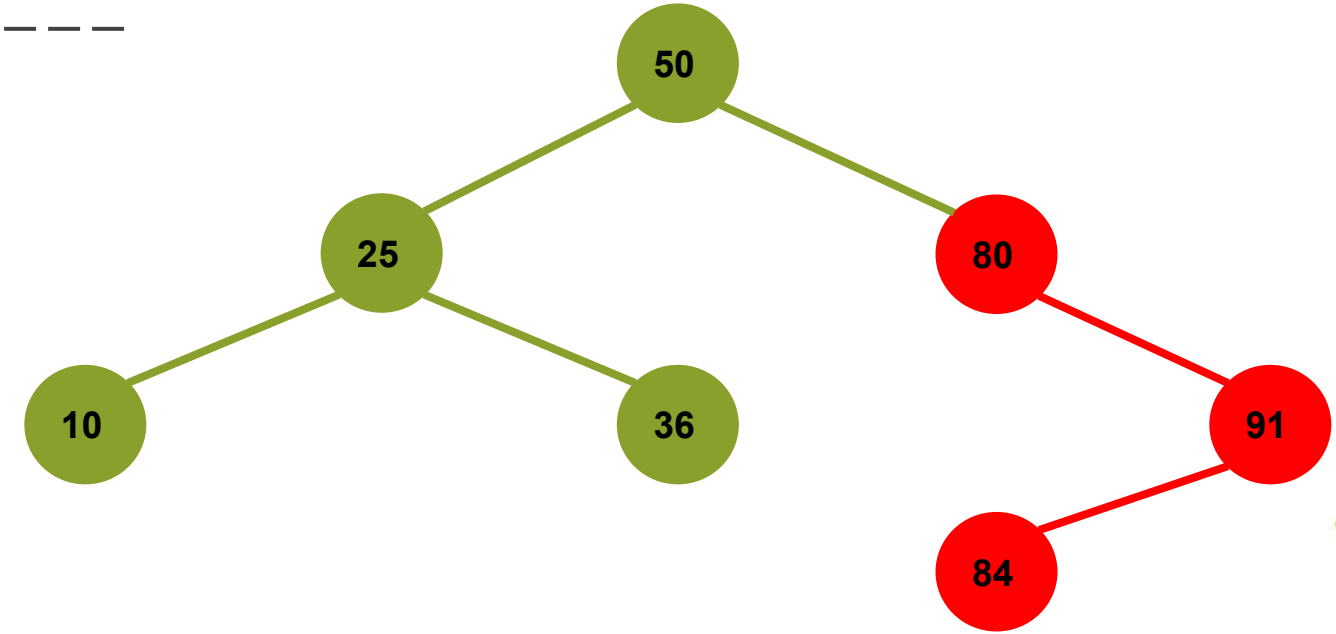
balanced binary search tree



balanced binary search tree



balanced binary search tree

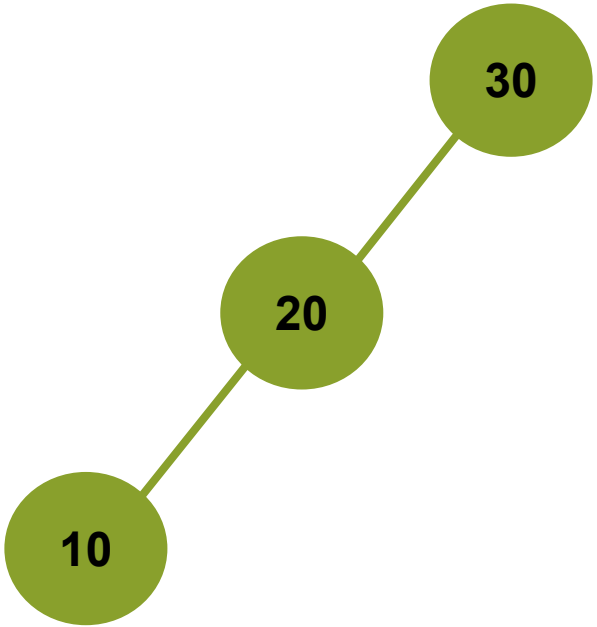


balanced binary search tree

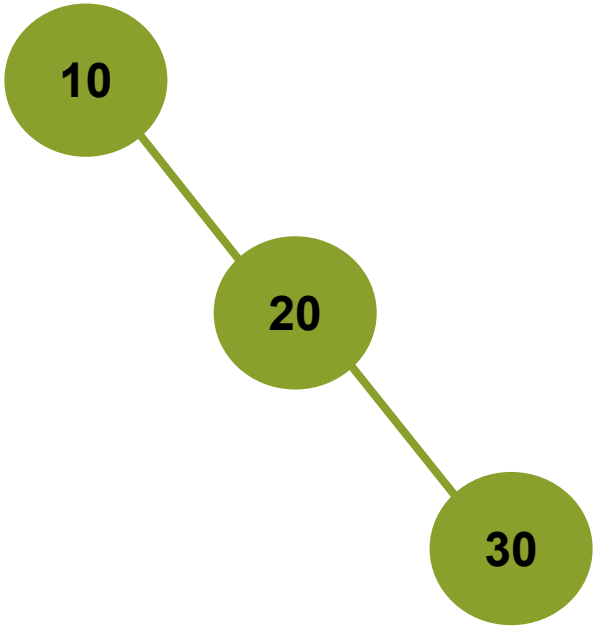
- violations to watch out for
  - left leaning
  - right leaning
  - left-right leaning
  - right-left leaning



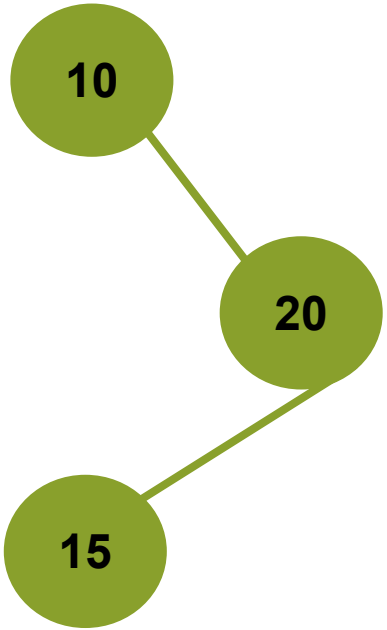
balanced binary search tree - left leaning



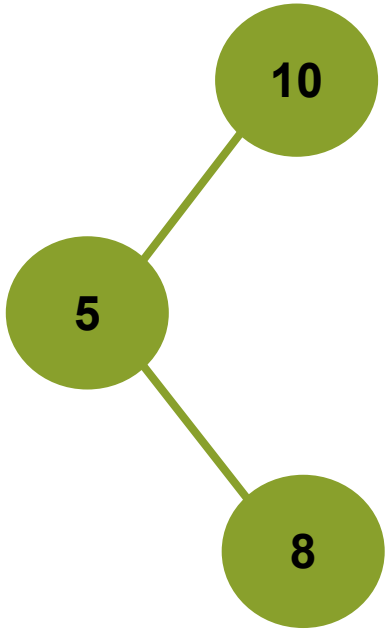
balanced binary search tree - right leaning



balanced binary search tree - left-right leaning



balanced binary search tree - right-left leaning

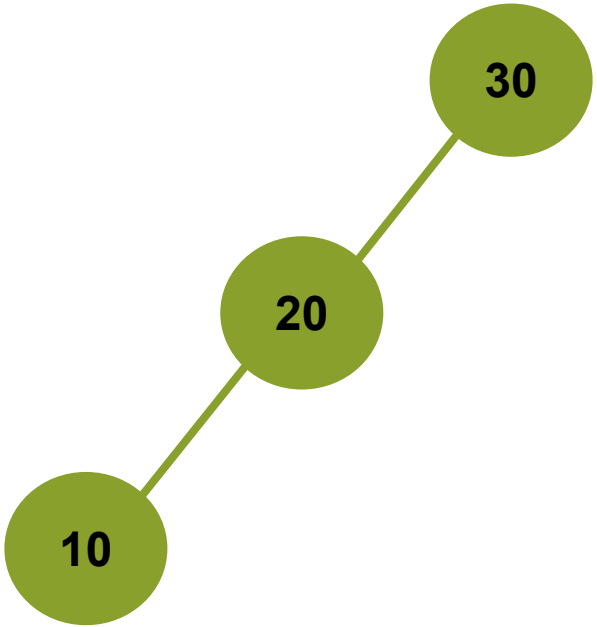


balanced binary search tree

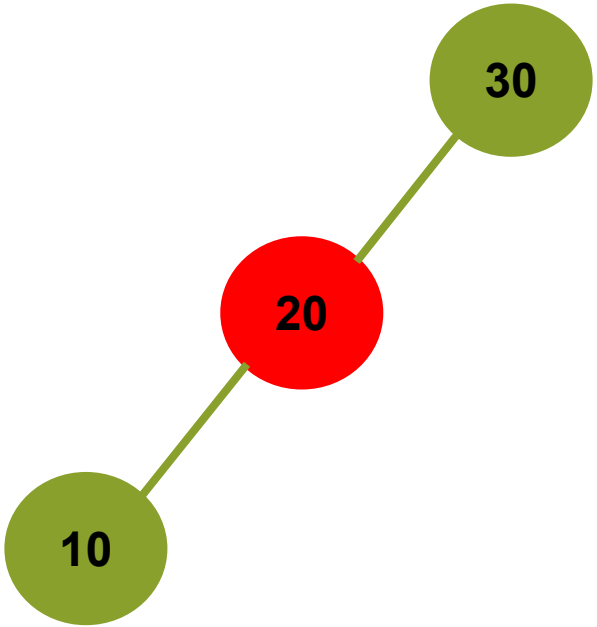
- apply rotations to fix the violations
  - left rotate
  - right rotate
  - left-right rotate
  - right-left rotate



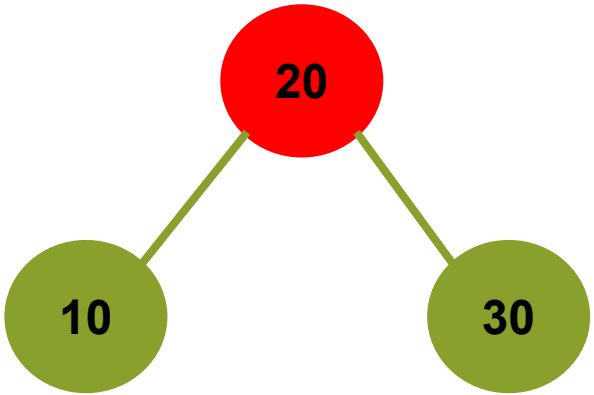
balanced binary search tree - right rotate



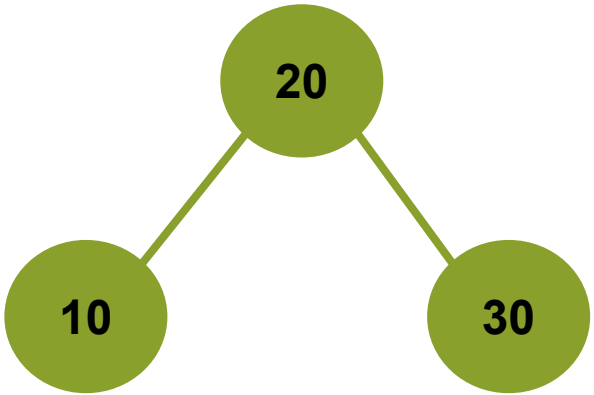
balanced binary search tree - right rotate



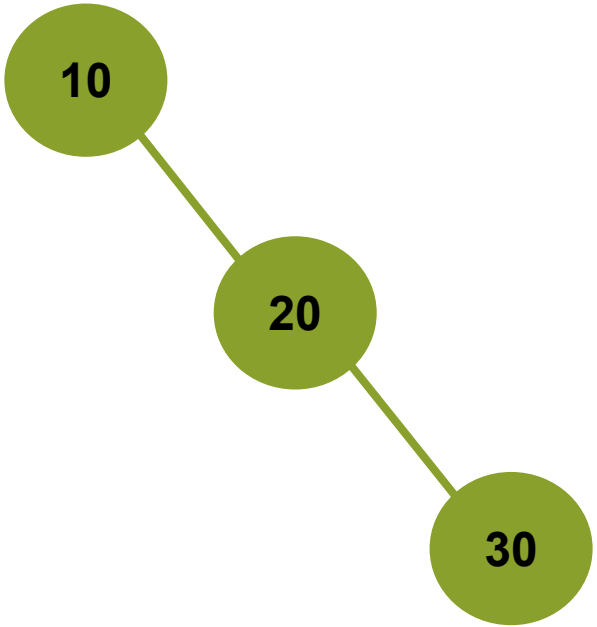
balanced binary search tree - right rotate



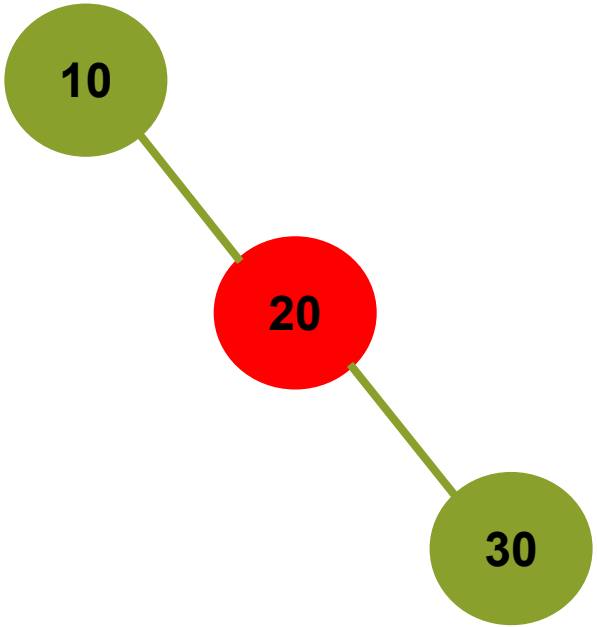
balanced binary search tree - right rotate



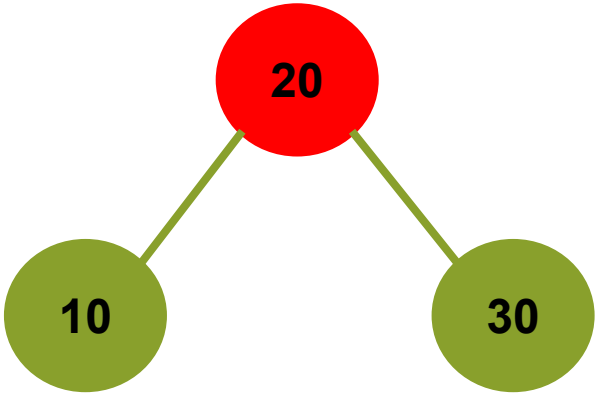
balanced binary search tree - left rotate



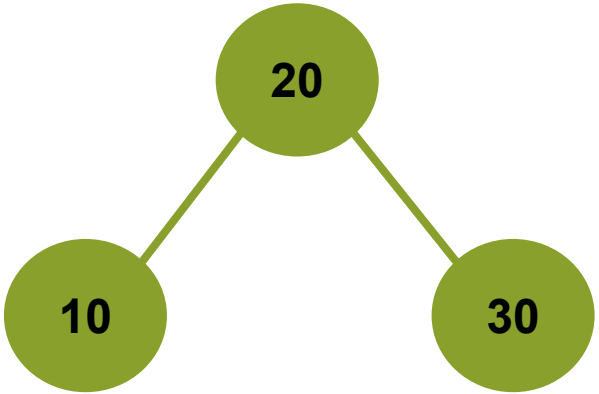
balanced binary search tree - left rotate



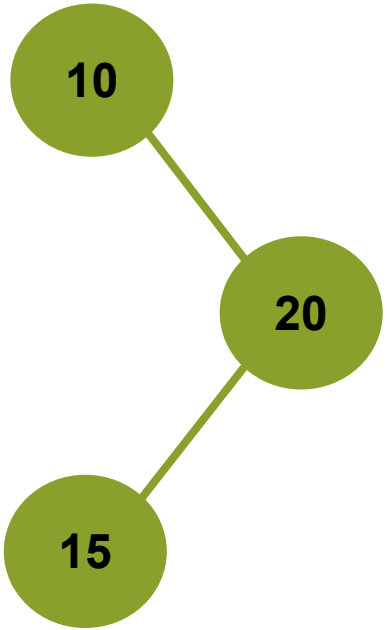
balanced binary search tree - left rotate



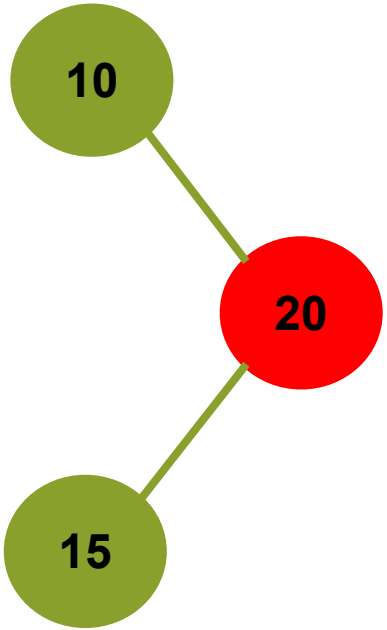
balanced binary search tree - left rotate



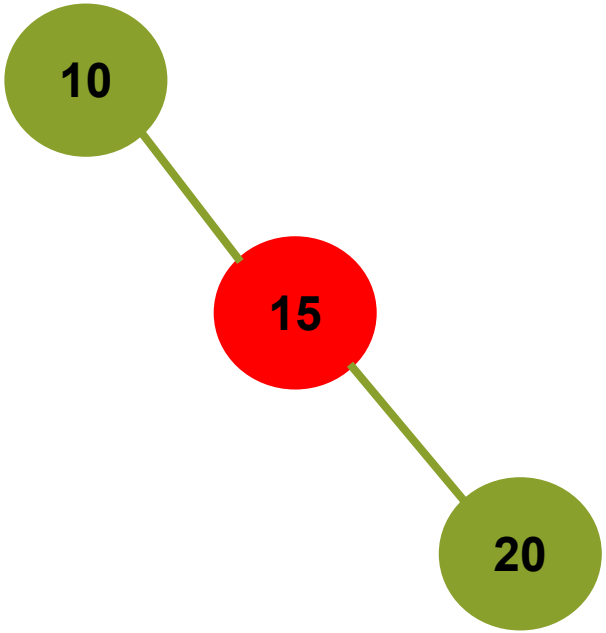
balanced binary search tree - right-left rotate



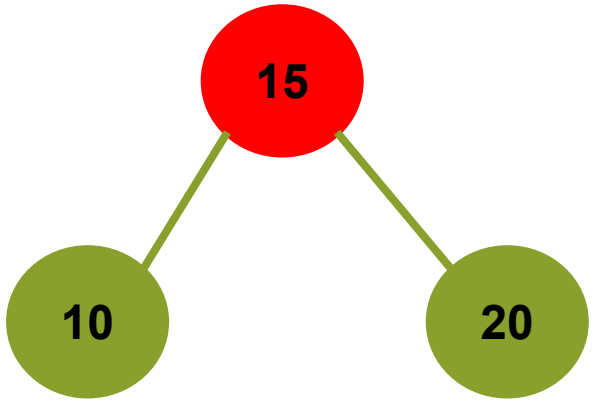
balanced binary search tree - right-left rotate



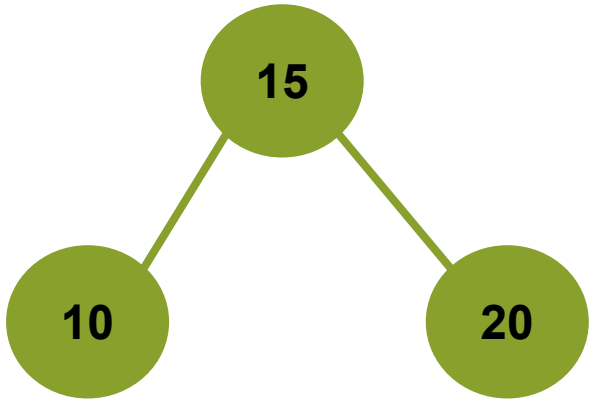
balanced binary search tree - right-left rotate



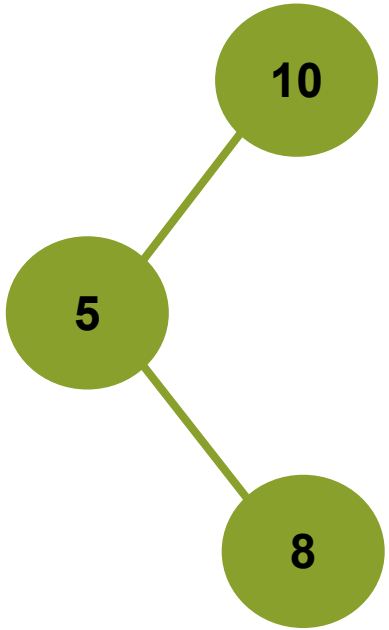
balanced binary search tree - right-left rotate



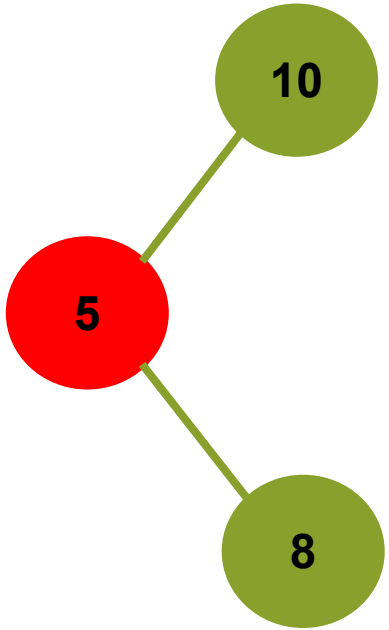
balanced binary search tree - right-left rotate



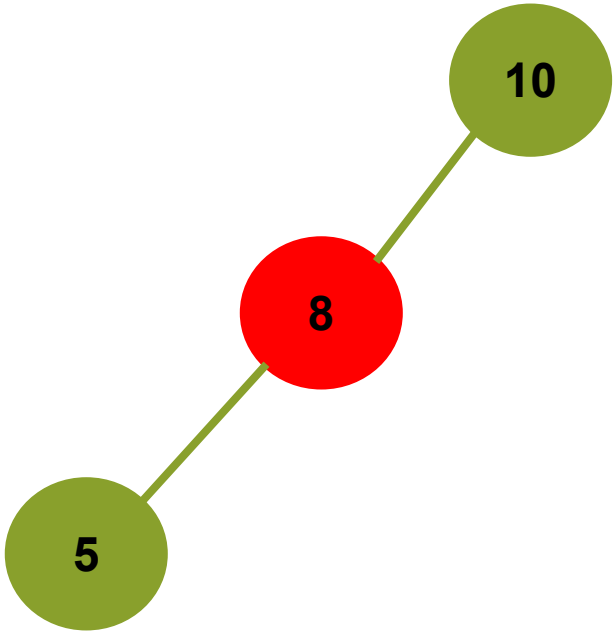
balanced binary search tree - left-right rotate



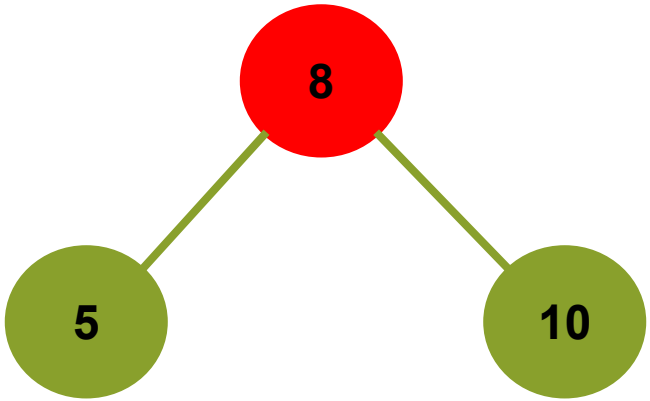
balanced binary search tree - left-right rotate



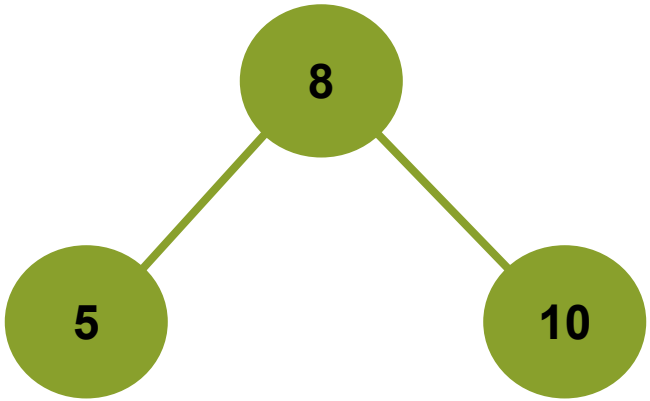
balanced binary search tree - left-right rotate



balanced binary search tree - left-right rotate



balanced binary search tree - left-right rotate



binary search tree - operations' time complexity

operation	time complexity
search	O(log n)
minimum	O(log n)
maximum	O(log n)
insert	O(log n)
delete	O(log n)



avl tree

- adelson-velski and landis
- balanced bst described in the earlier slides



avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11





avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



avl tree

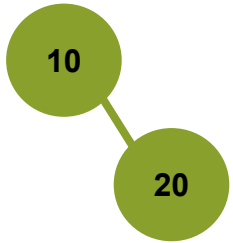
10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11





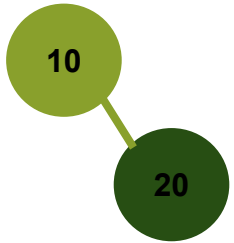
avl tree

10, **20**, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



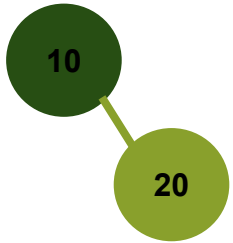
avl tree

10, **20**, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



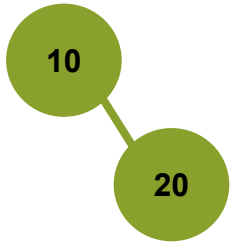
avl tree

10, **20**, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



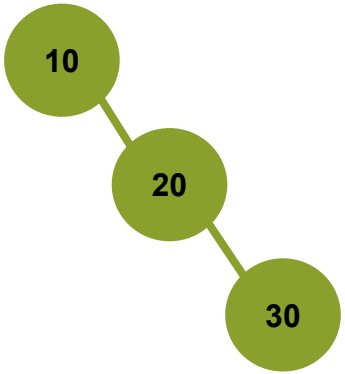
avl tree

10, **20**, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



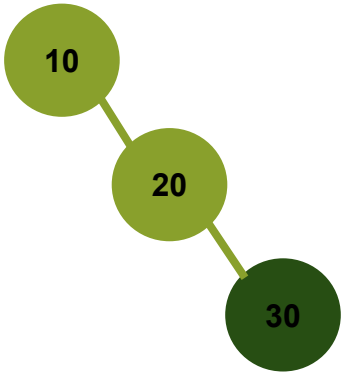
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



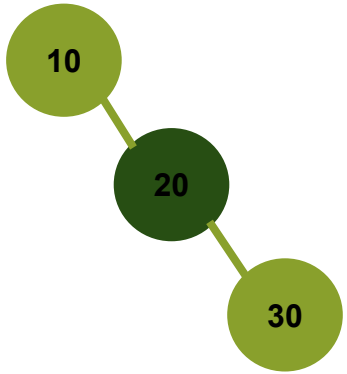
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



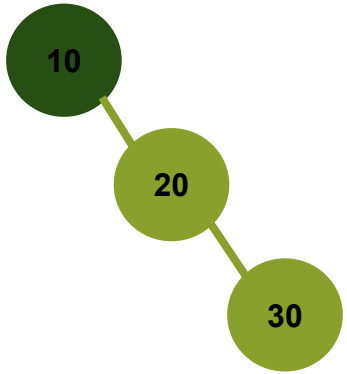
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



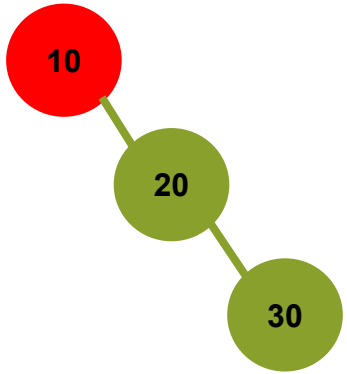
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



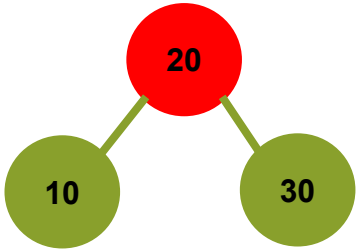
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



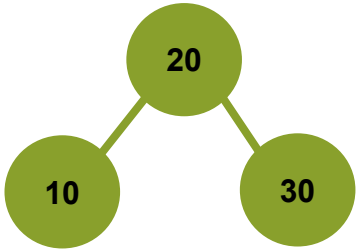
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



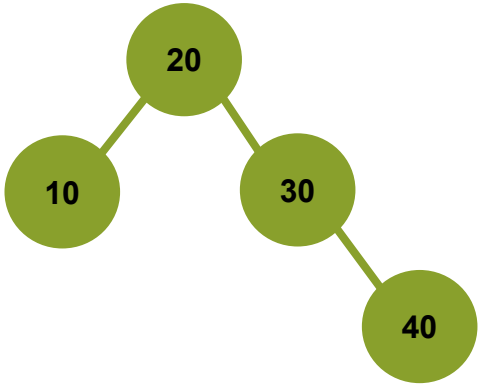
avl tree

10, 20, **30**, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



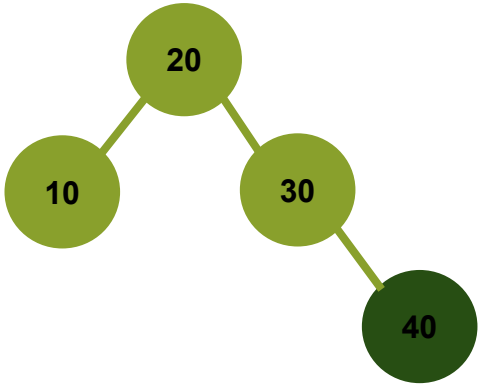
avl tree

10, 20, 30, **40**, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



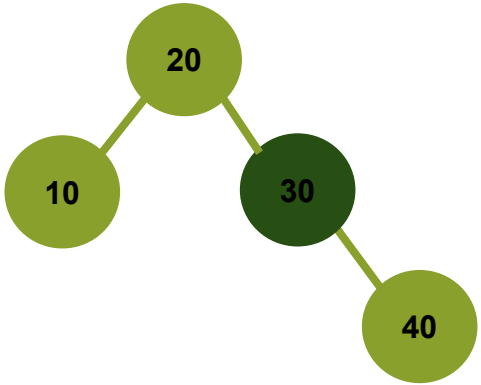
avl tree

10, 20, 30, **40**, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



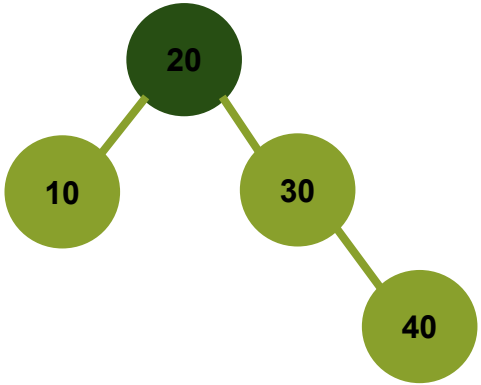
avl tree

10, 20, 30, **40**, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



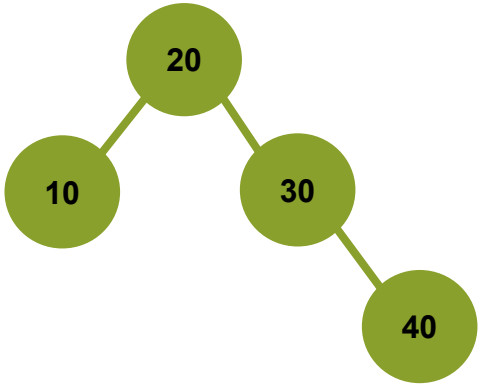
avl tree

10, 20, 30, **40**, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



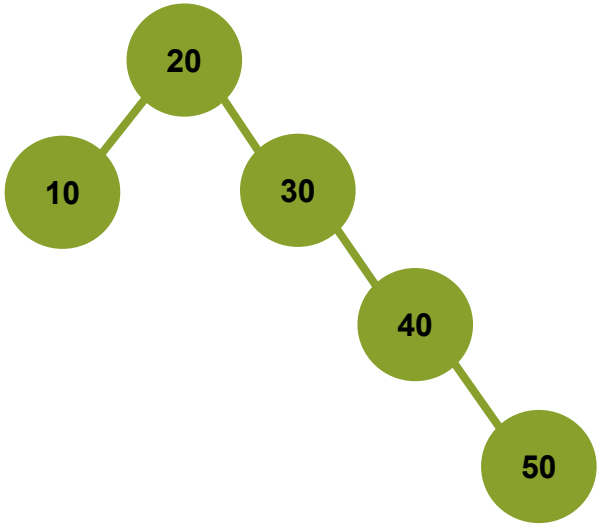
avl tree

10, 20, 30, **40**, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



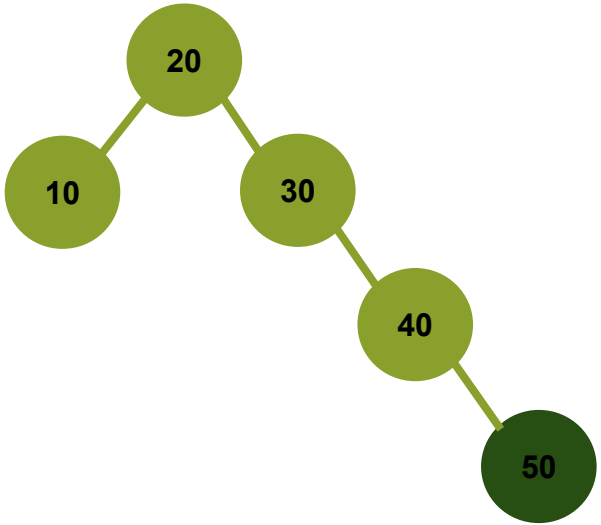
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



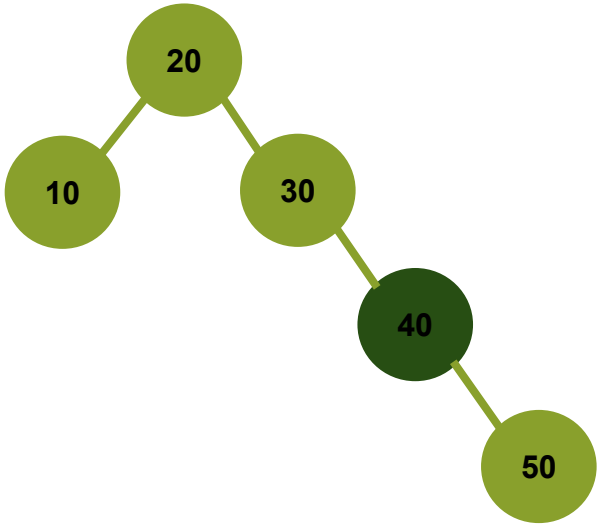
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



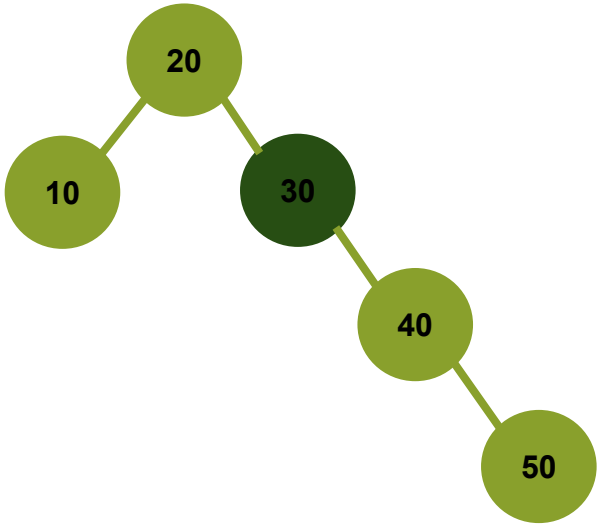
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



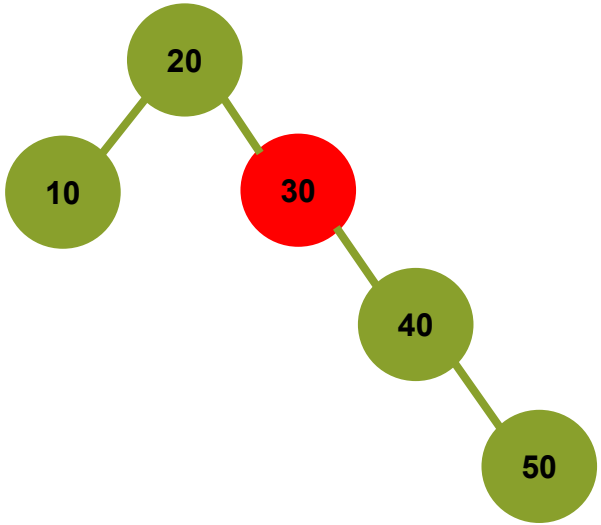
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



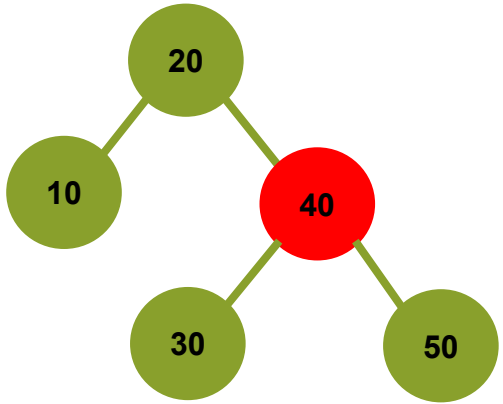
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



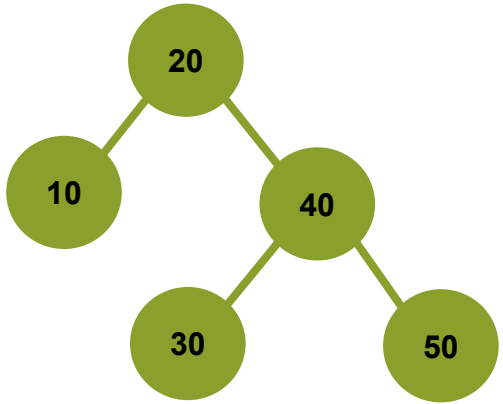
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



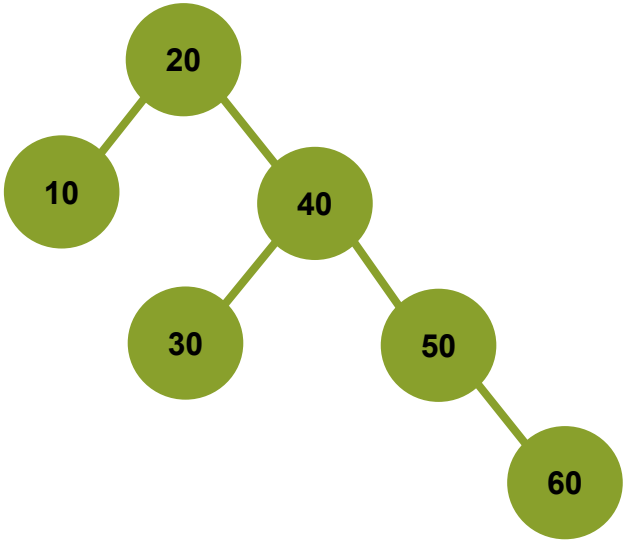
avl tree

10, 20, 30, 40, **50**, 60, 70, 80, 71, 61, 51, 41, 31, 21, 11



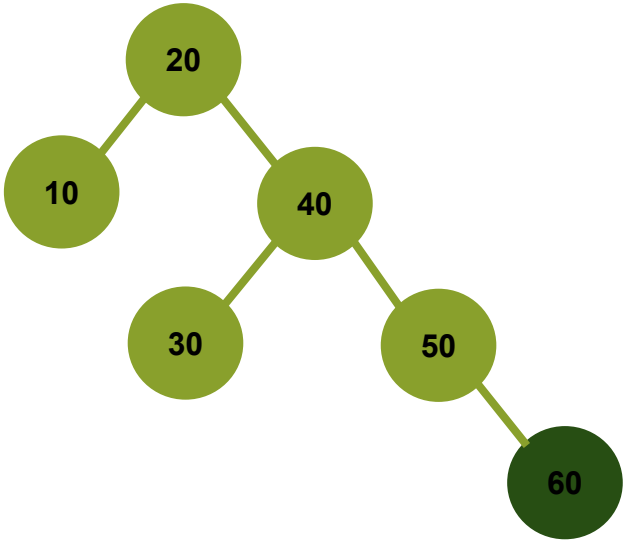
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



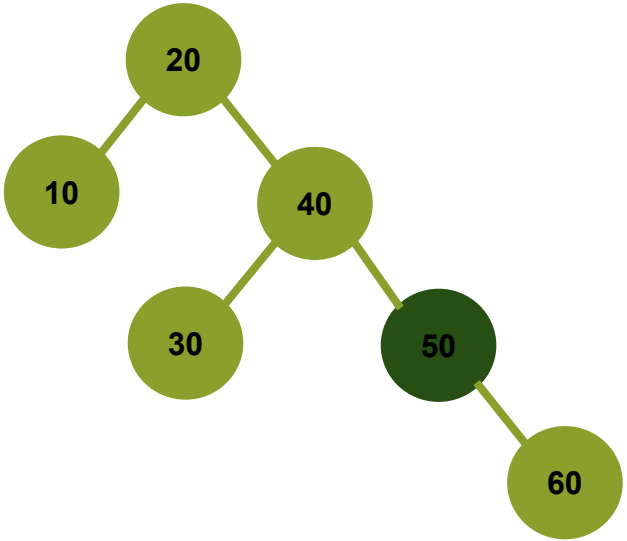
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



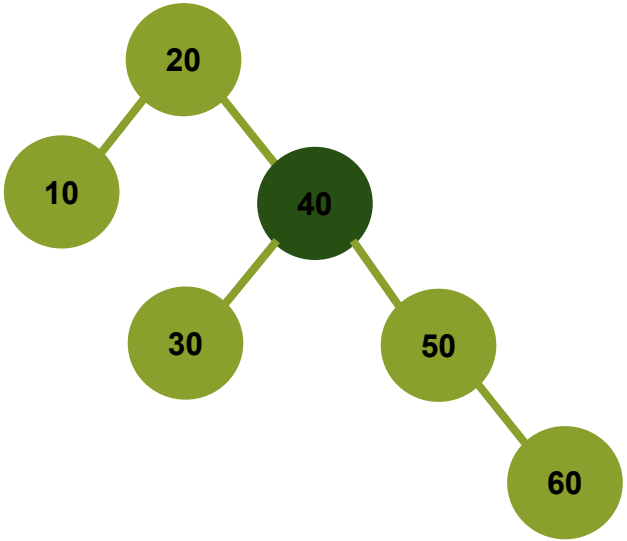
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



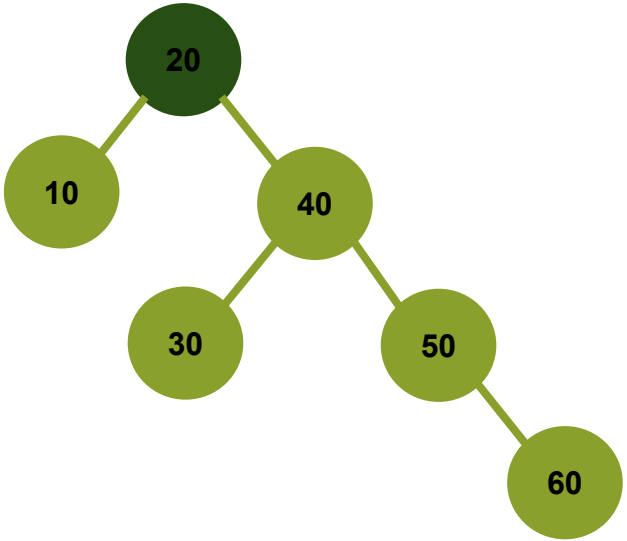
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



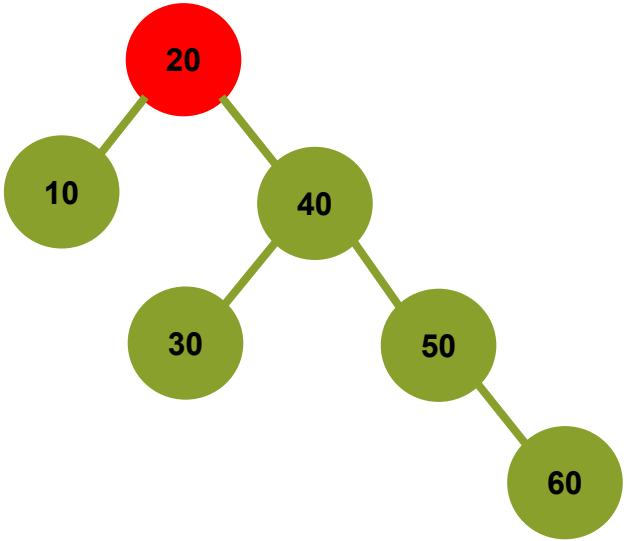
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



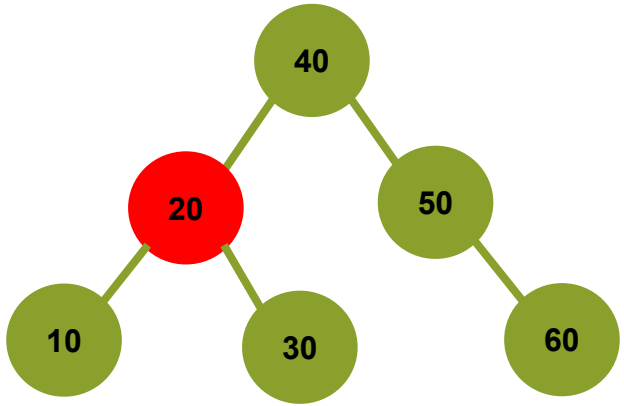
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



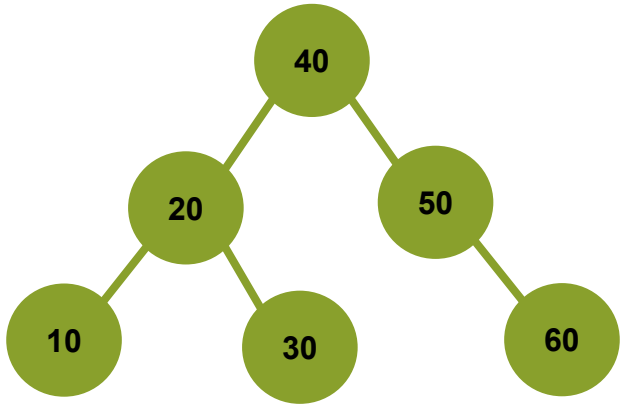
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



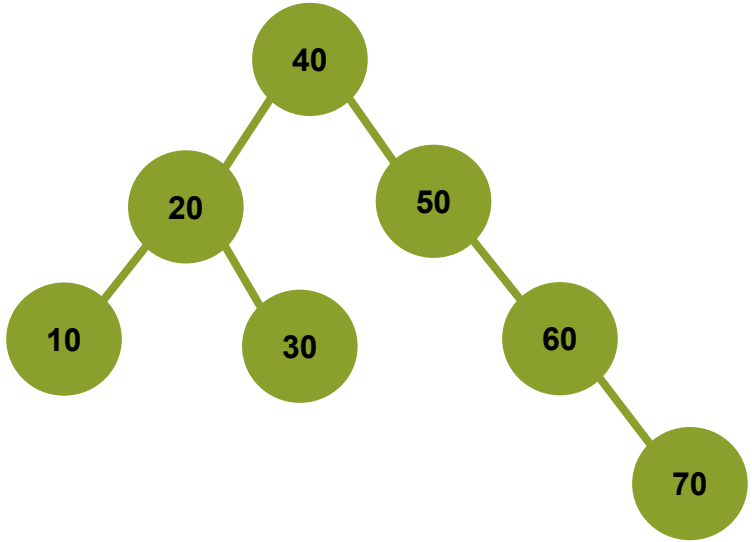
avl tree

10, 20, 30, 40, 50, **60**, 70, 80, 71, 61, 51, 41, 31, 21, 11



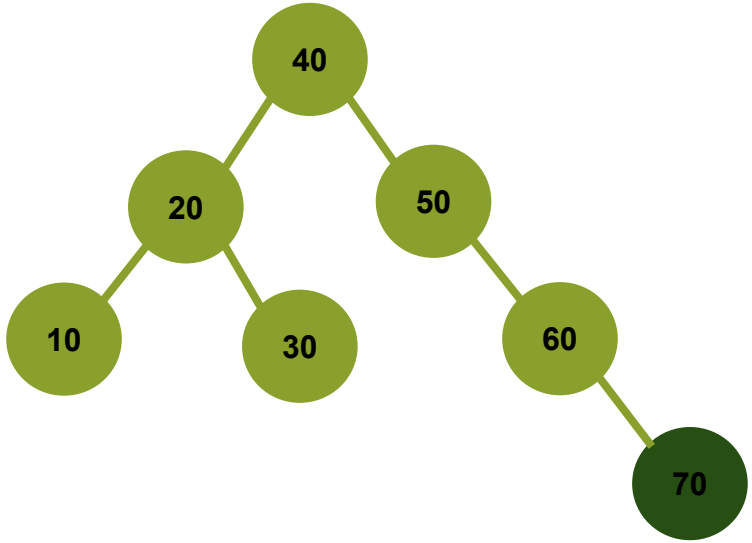
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



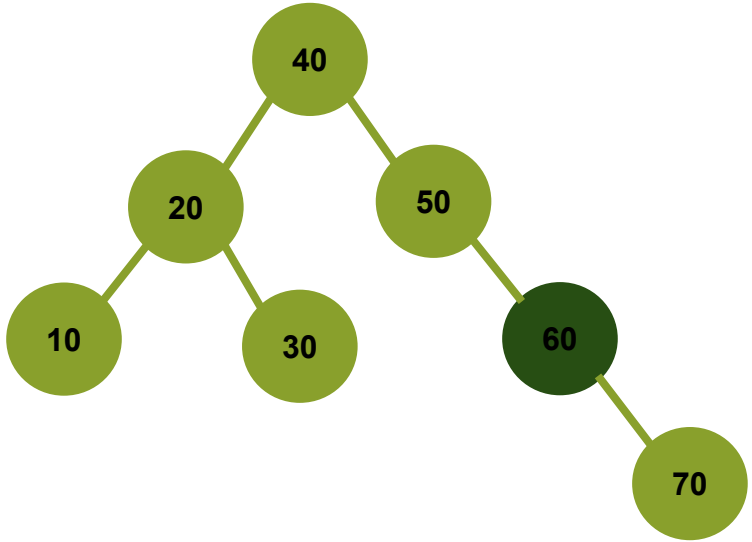
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



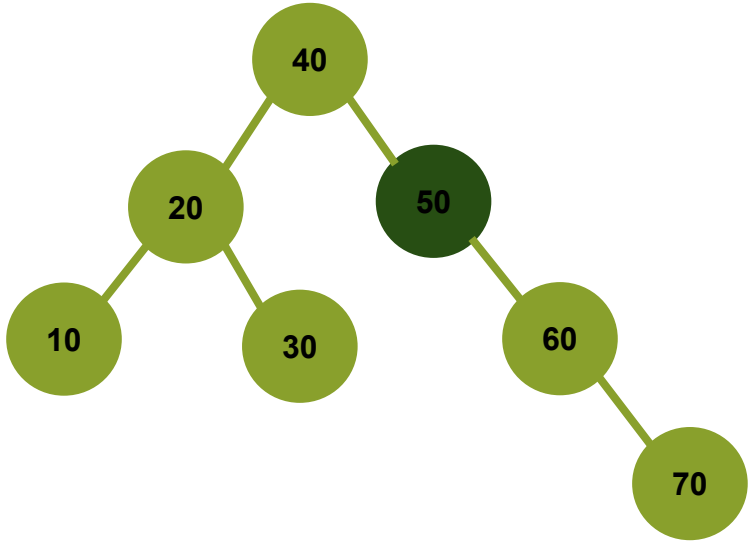
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



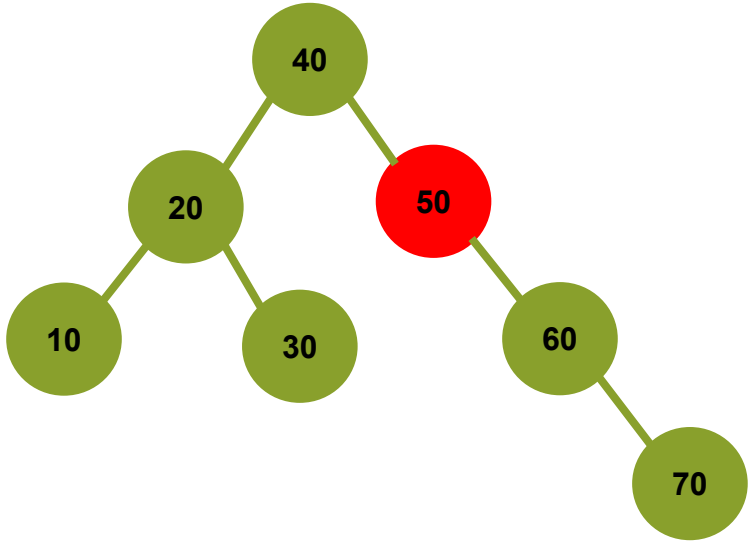
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



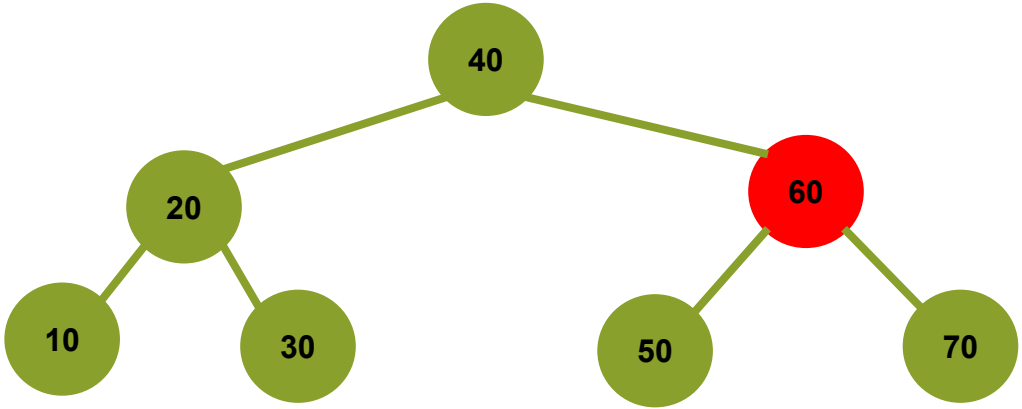
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



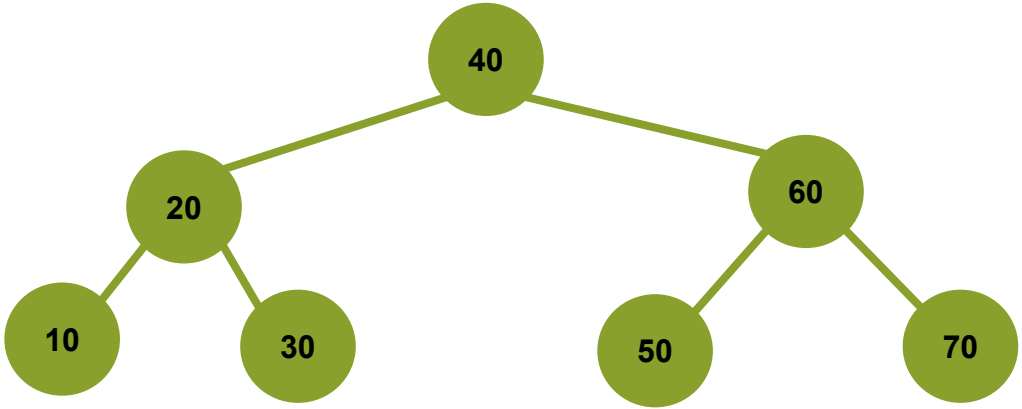
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



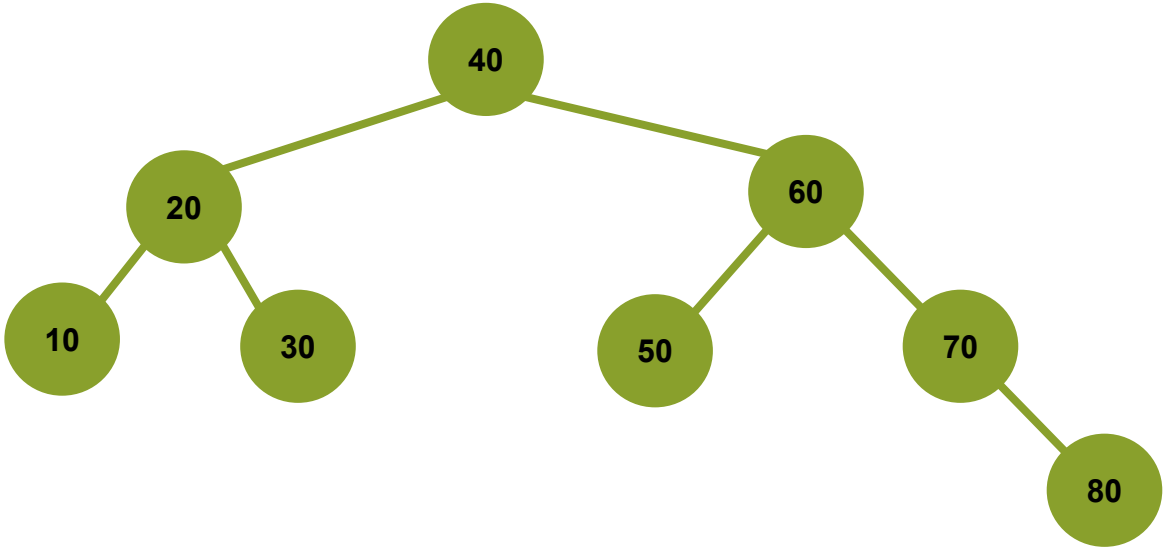
avl tree

10, 20, 30, 40, 50, 60, **70**, 80, 71, 61, 51, 41, 31, 21, 11



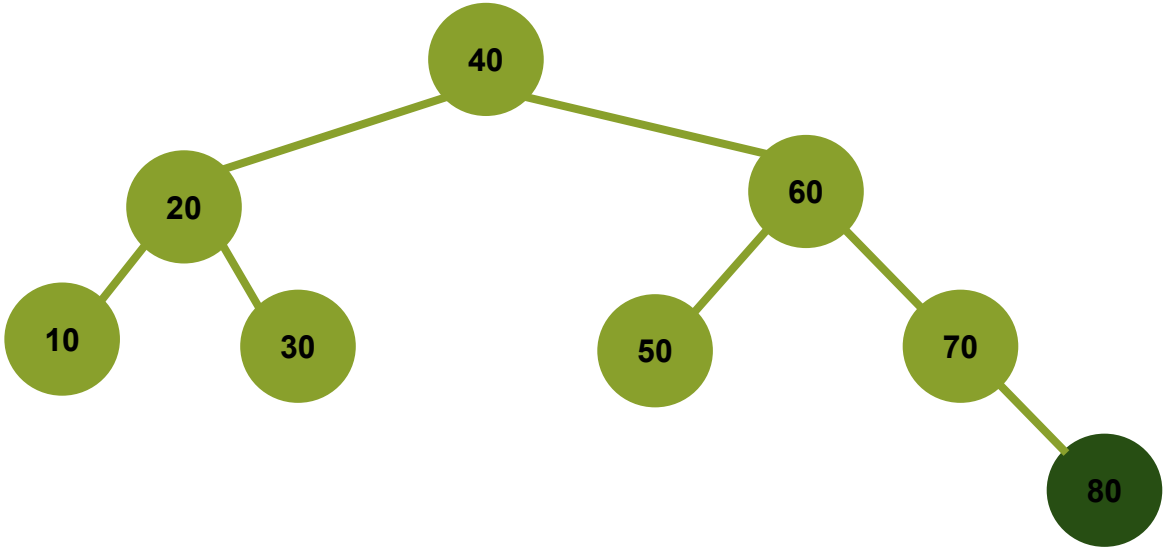
avl tree

10, 20, 30, 40, 50, 60, 70, **80**, 71, 61, 51, 41, 31, 21, 11



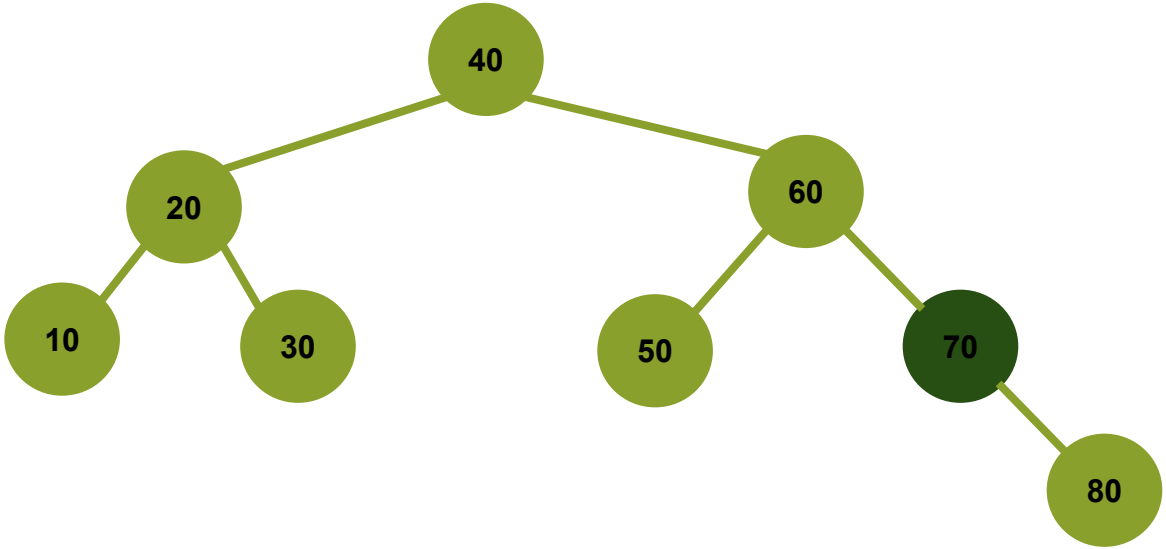
avl tree

10, 20, 30, 40, 50, 60, 70, **80**, 71, 61, 51, 41, 31, 21, 11



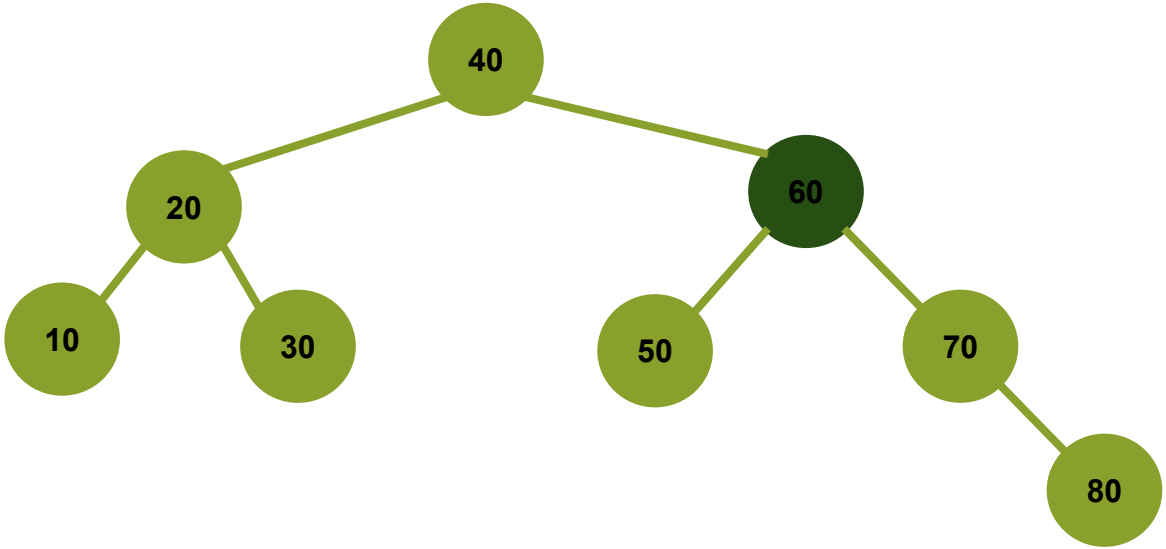
avl tree

10, 20, 30, 40, 50, 60, 70, **80**, 71, 61, 51, 41, 31, 21, 11



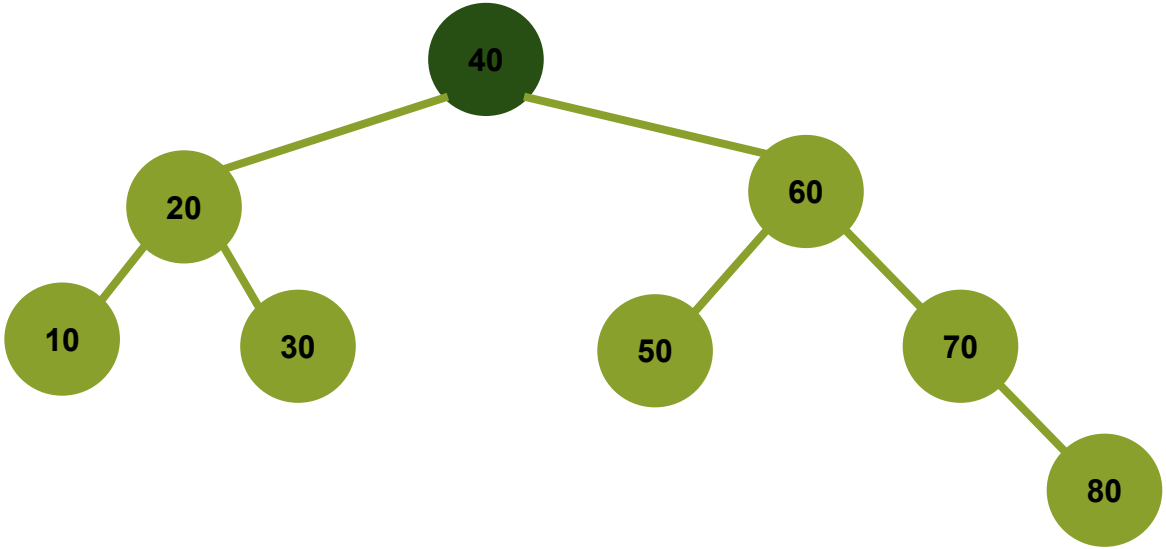
avl tree

10, 20, 30, 40, 50, 60, 70, **80**, 71, 61, 51, 41, 31, 21, 11



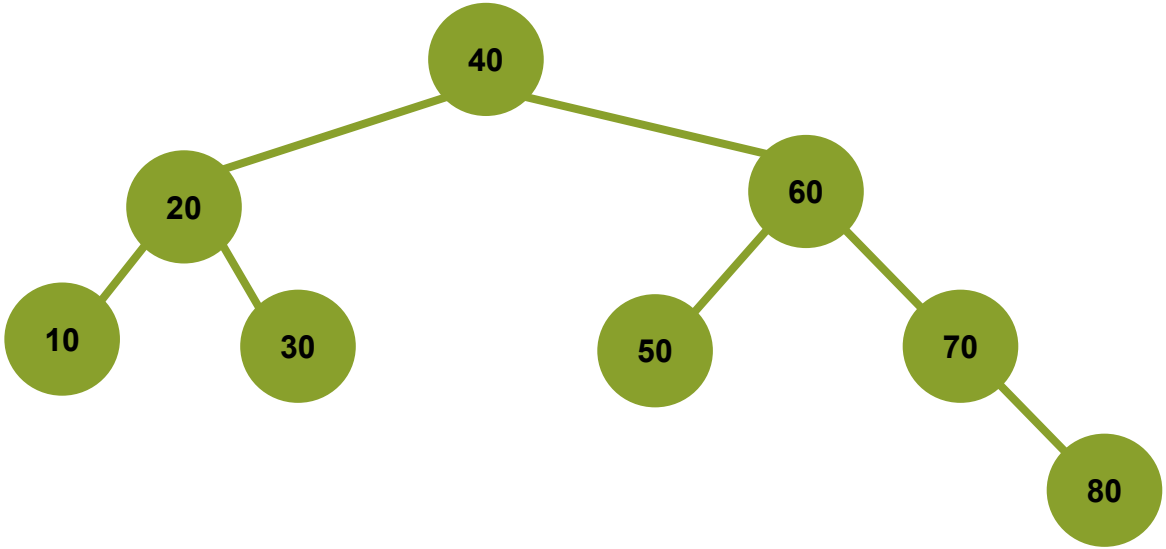
avl tree

10, 20, 30, 40, 50, 60, 70, **80**, 71, 61, 51, 41, 31, 21, 11



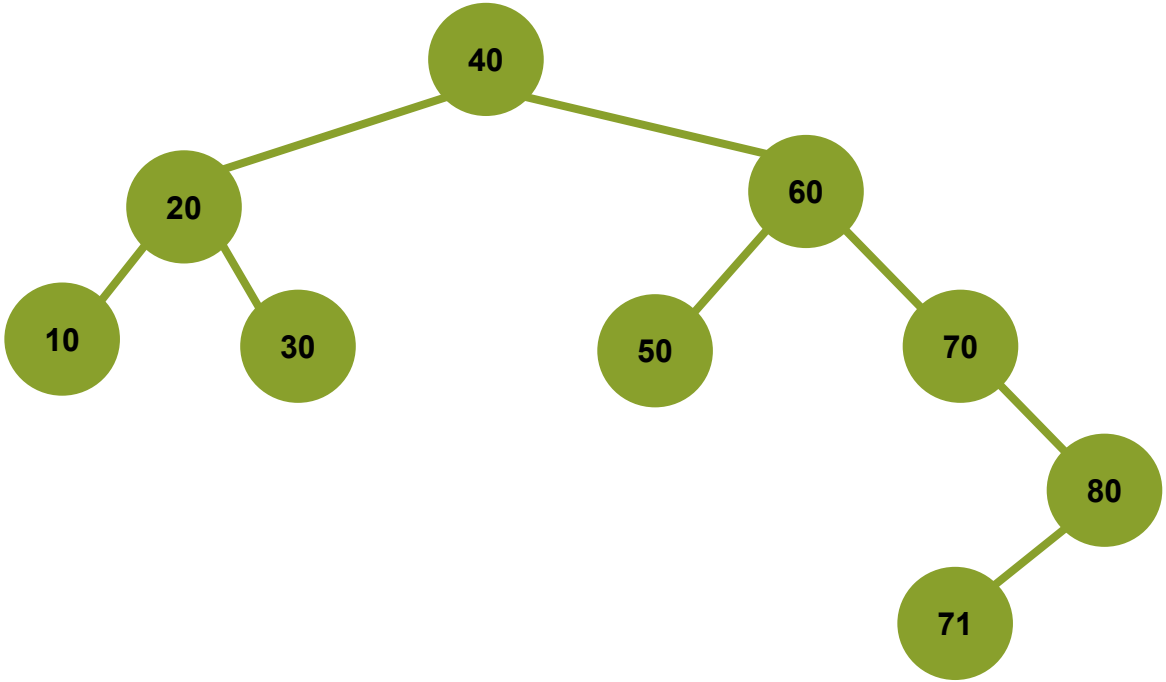
avl tree

10, 20, 30, 40, 50, 60, 70, **80**, 71, 61, 51, 41, 31, 21, 11



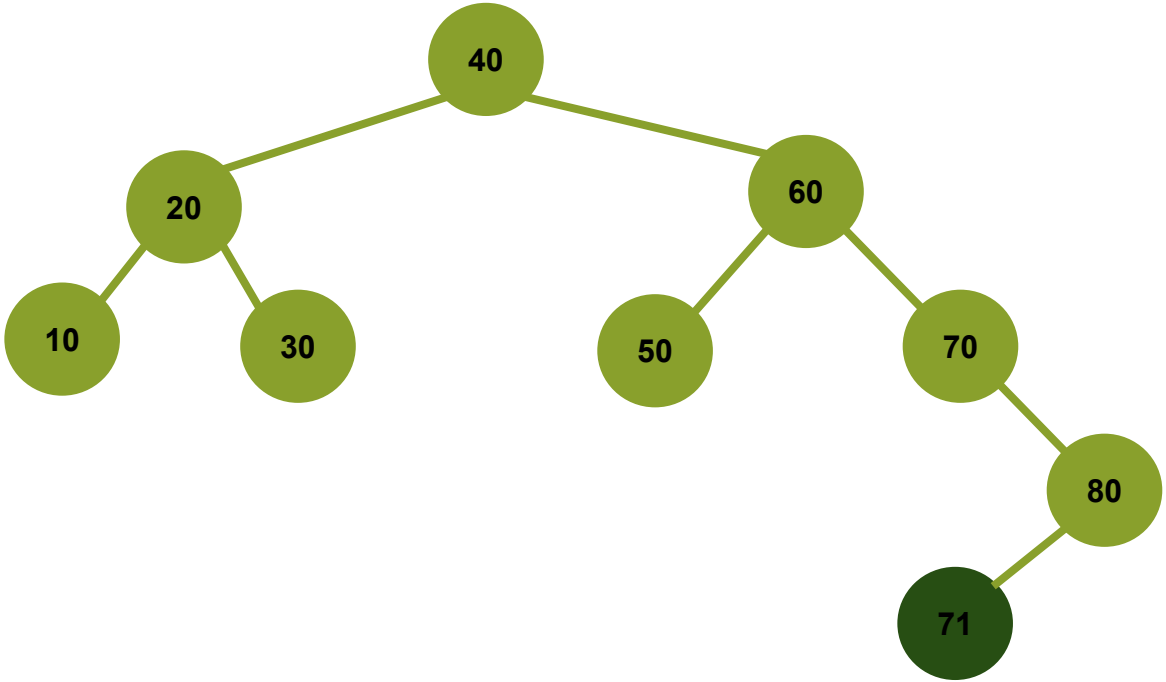
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



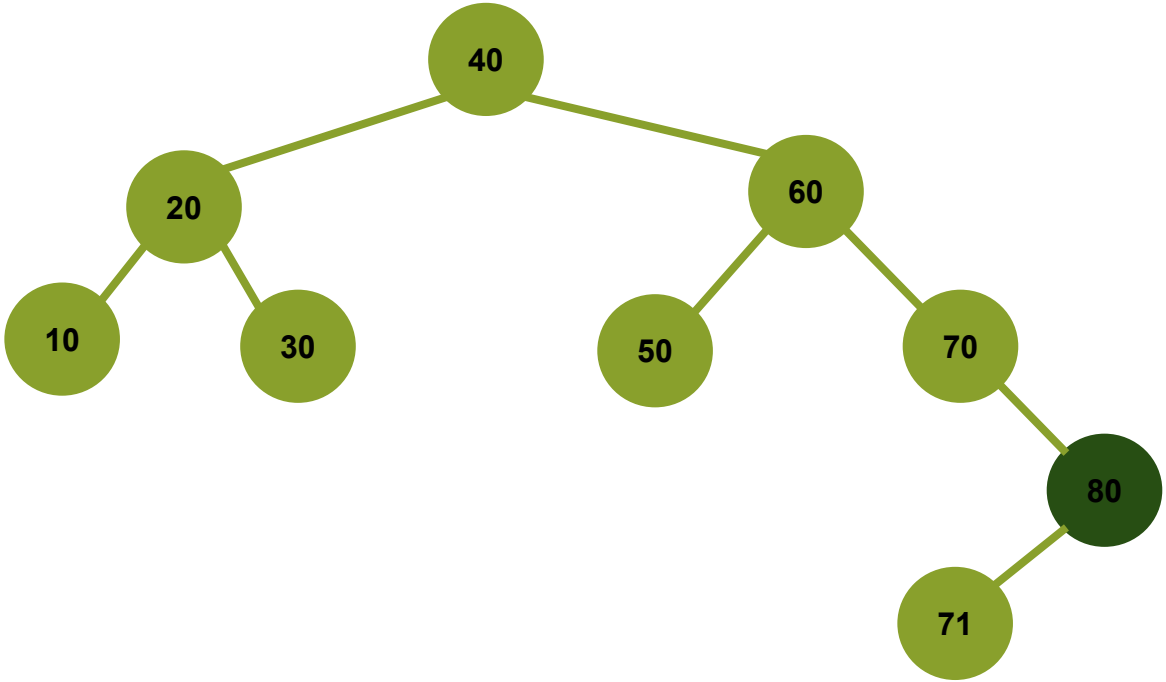
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



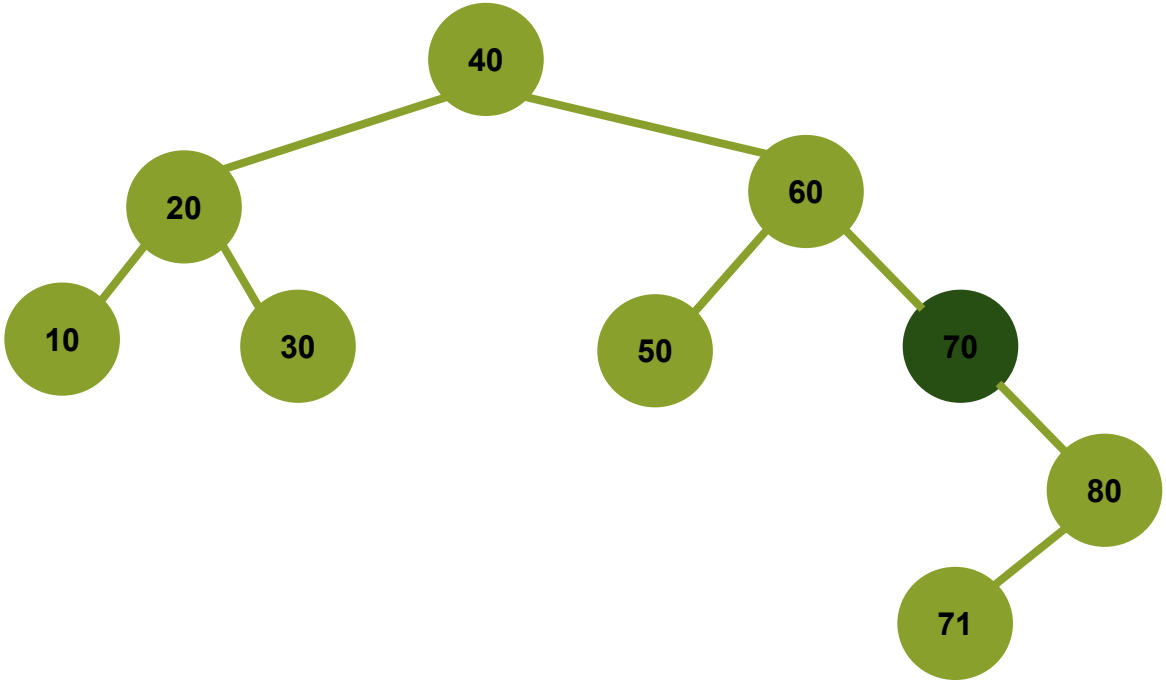
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



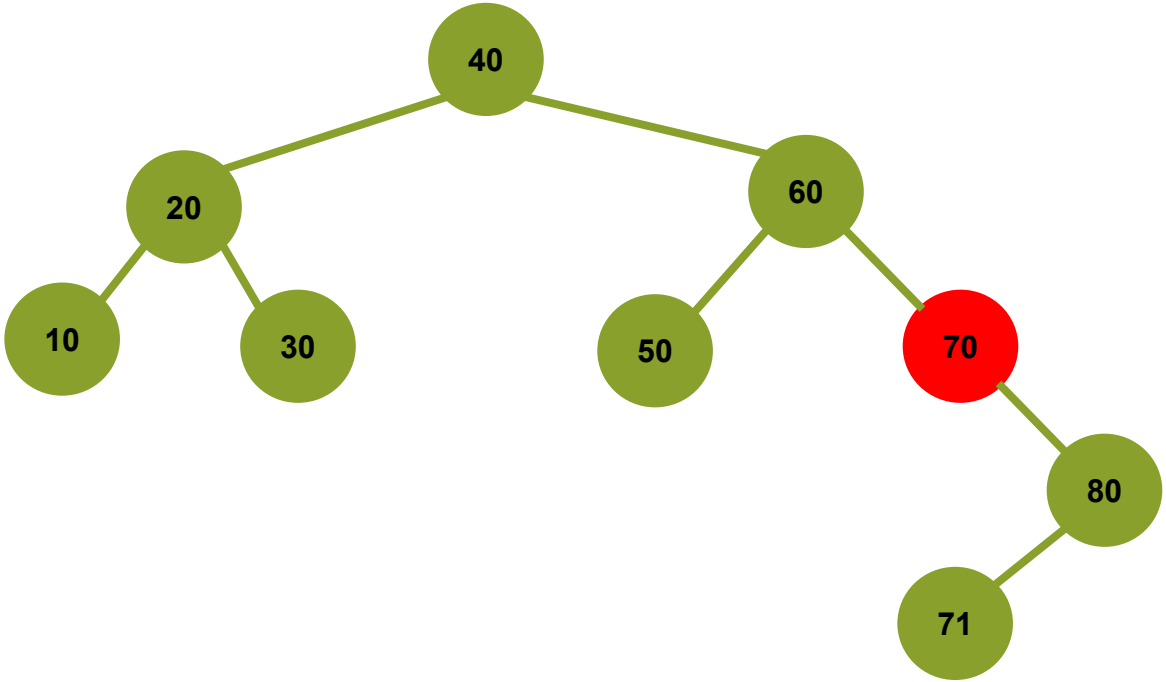
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



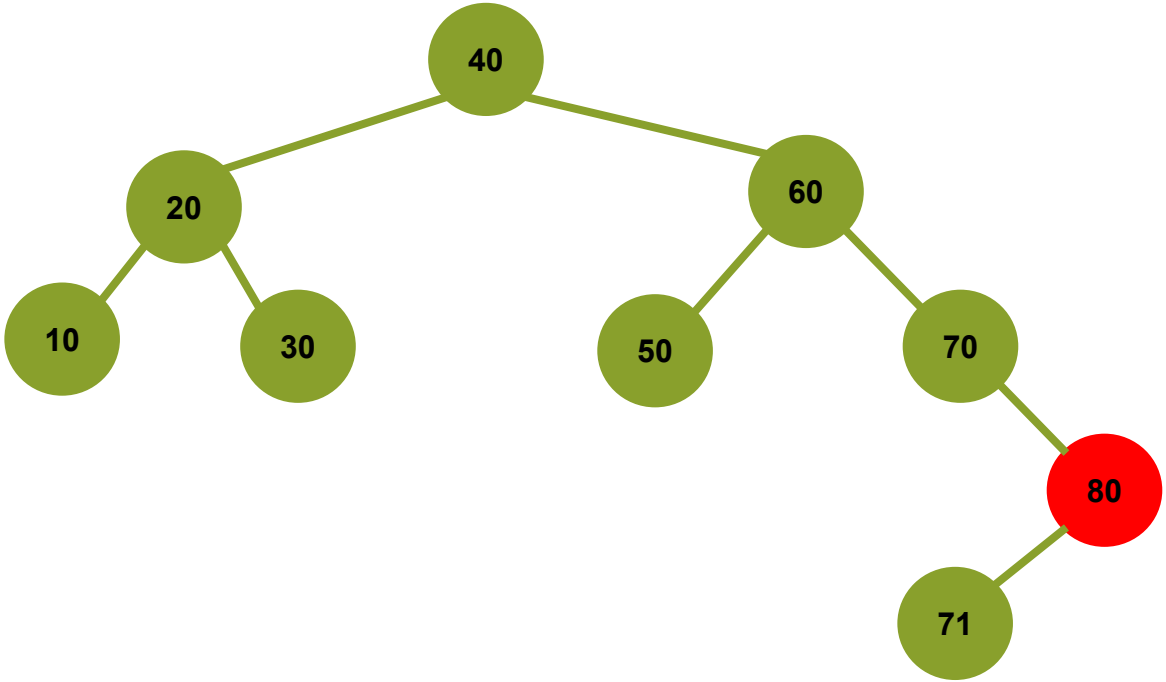
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



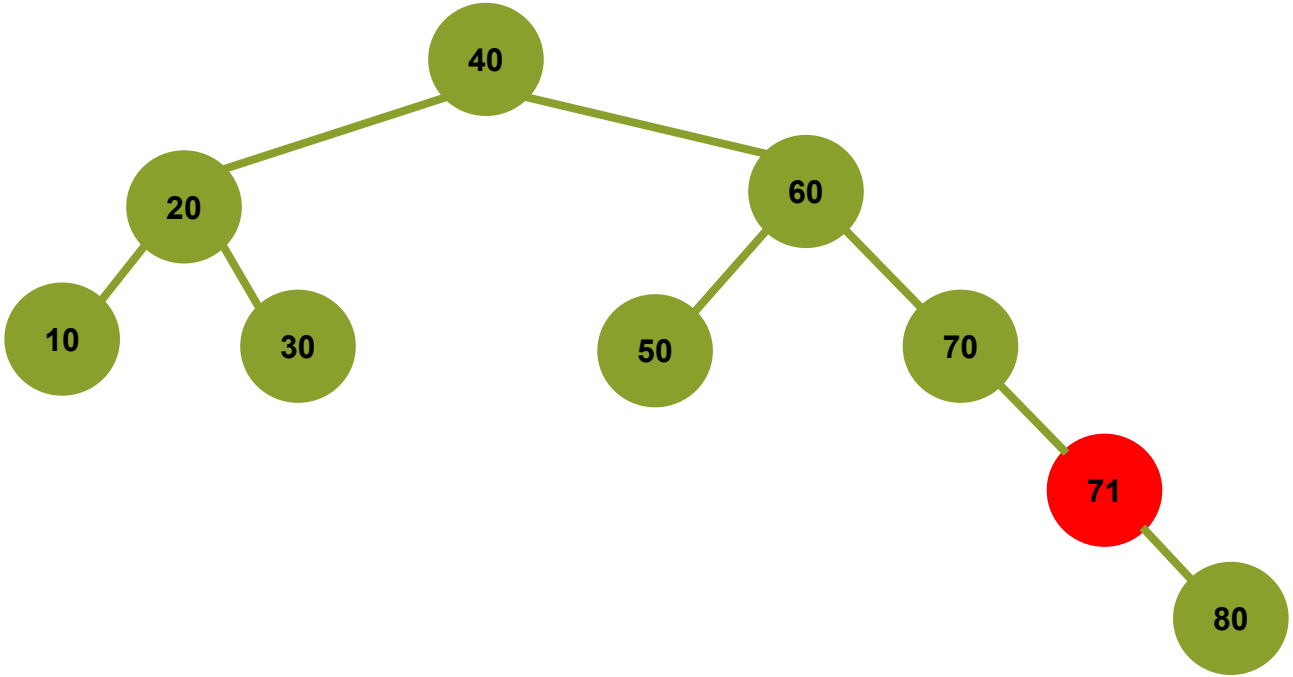
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



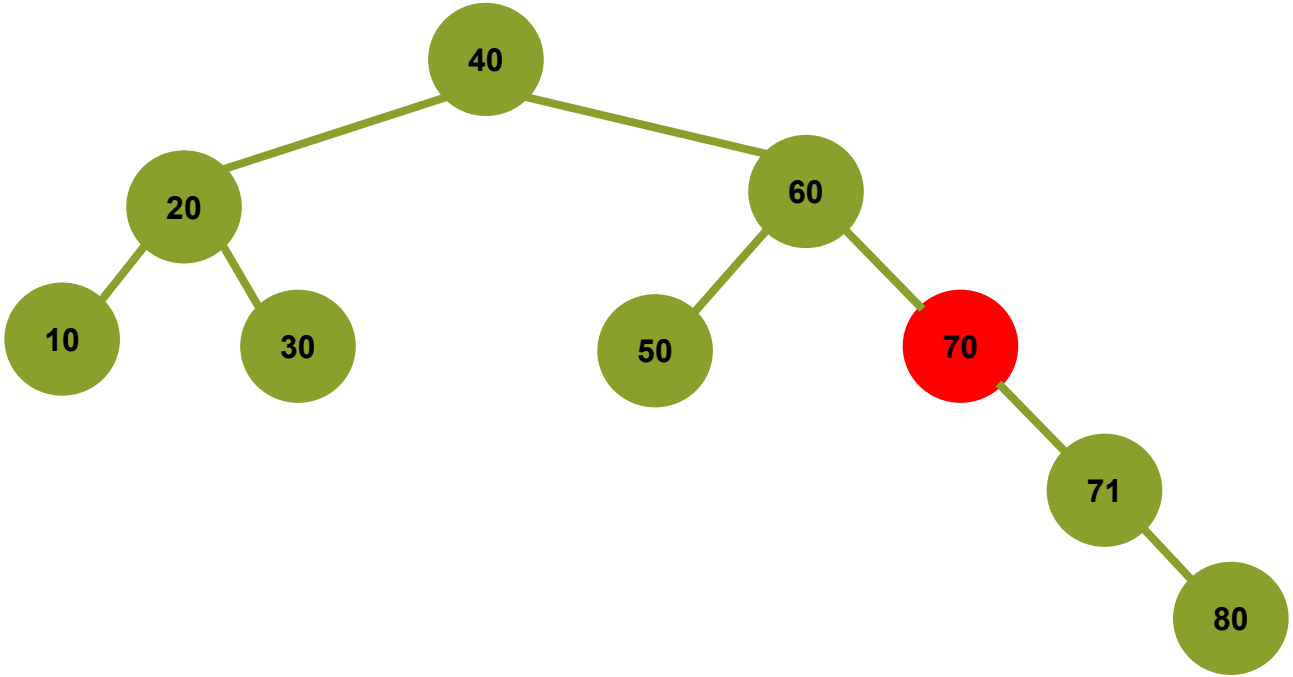
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



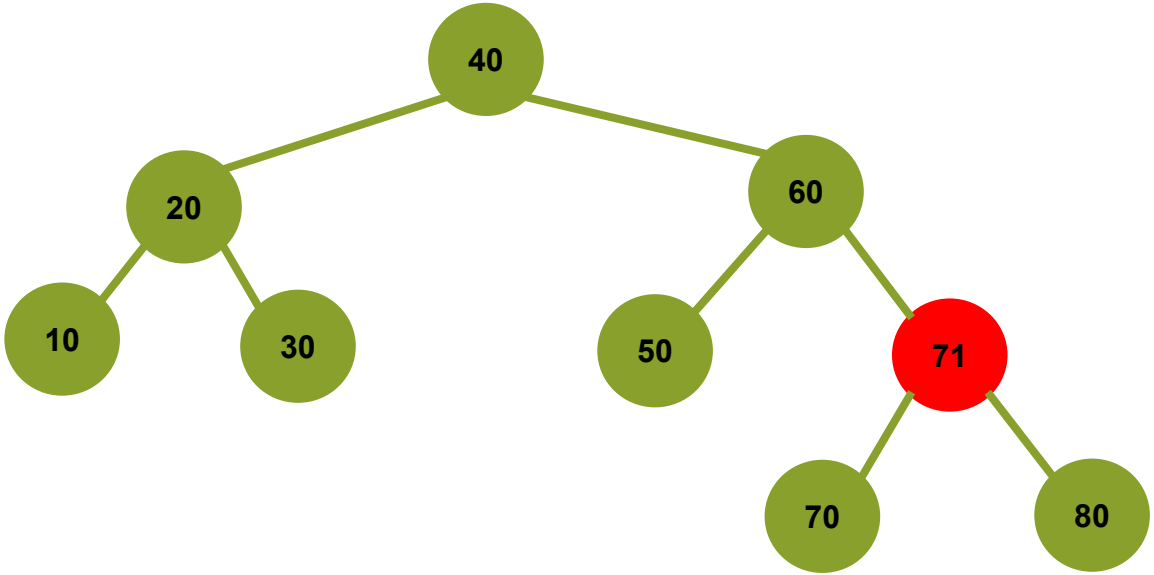
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



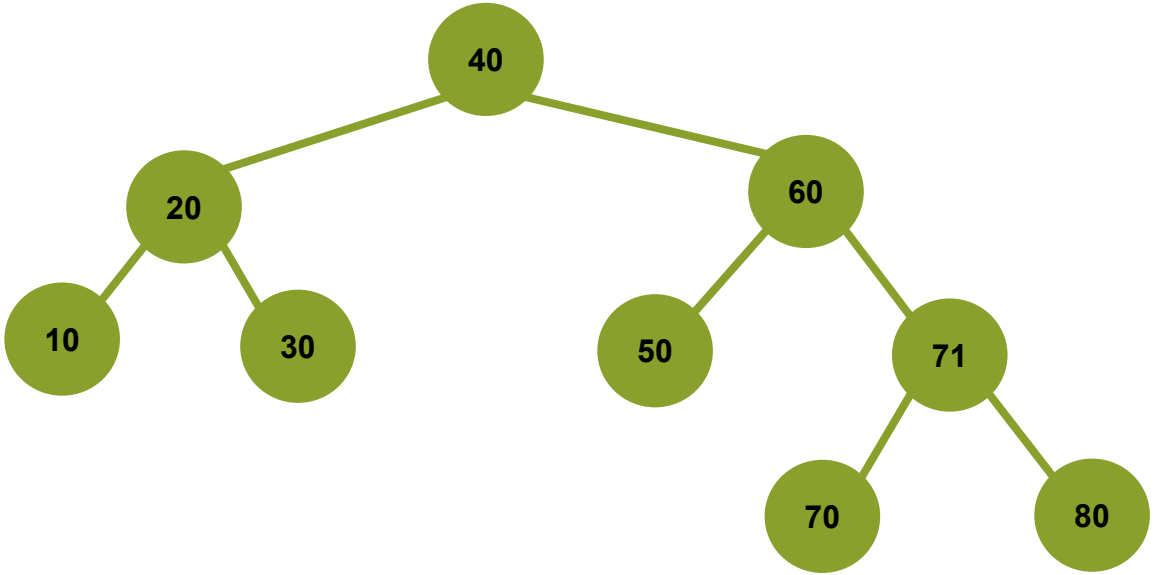
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



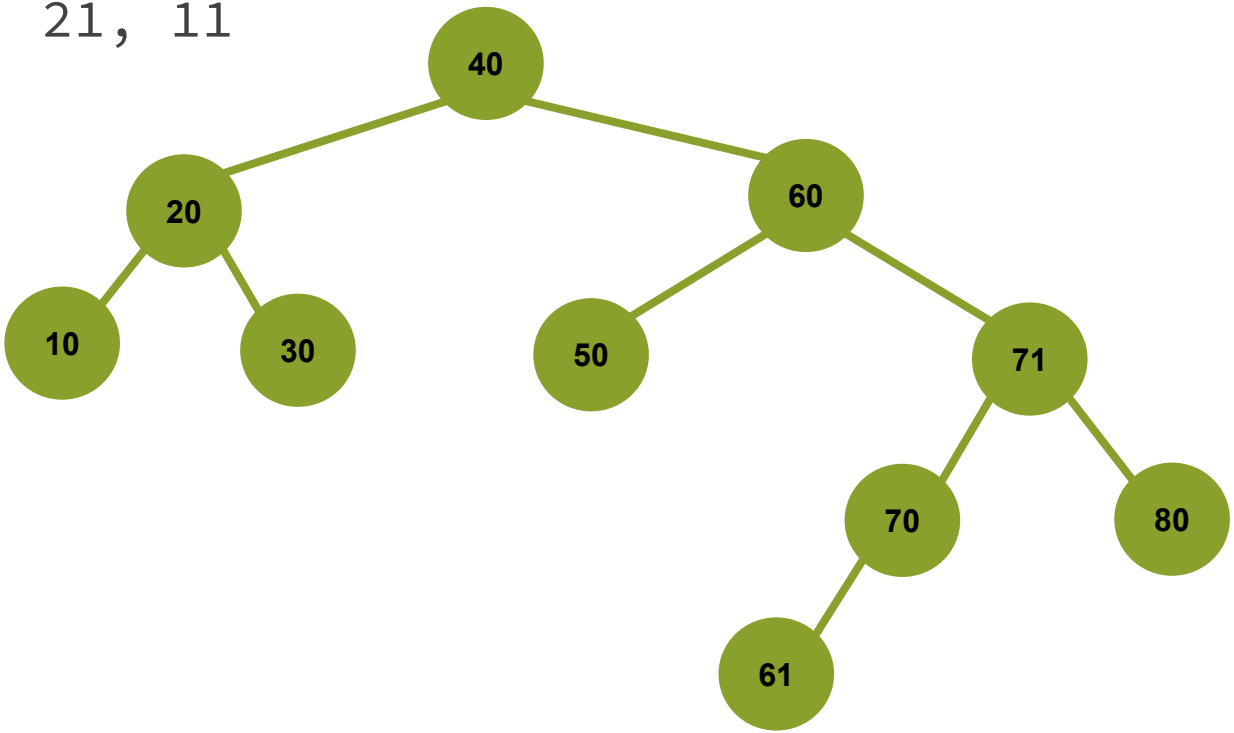
avl tree

10, 20, 30, 40, 50, 60, 70, 80, **71**, 61, 51, 41, 31, 21, 11



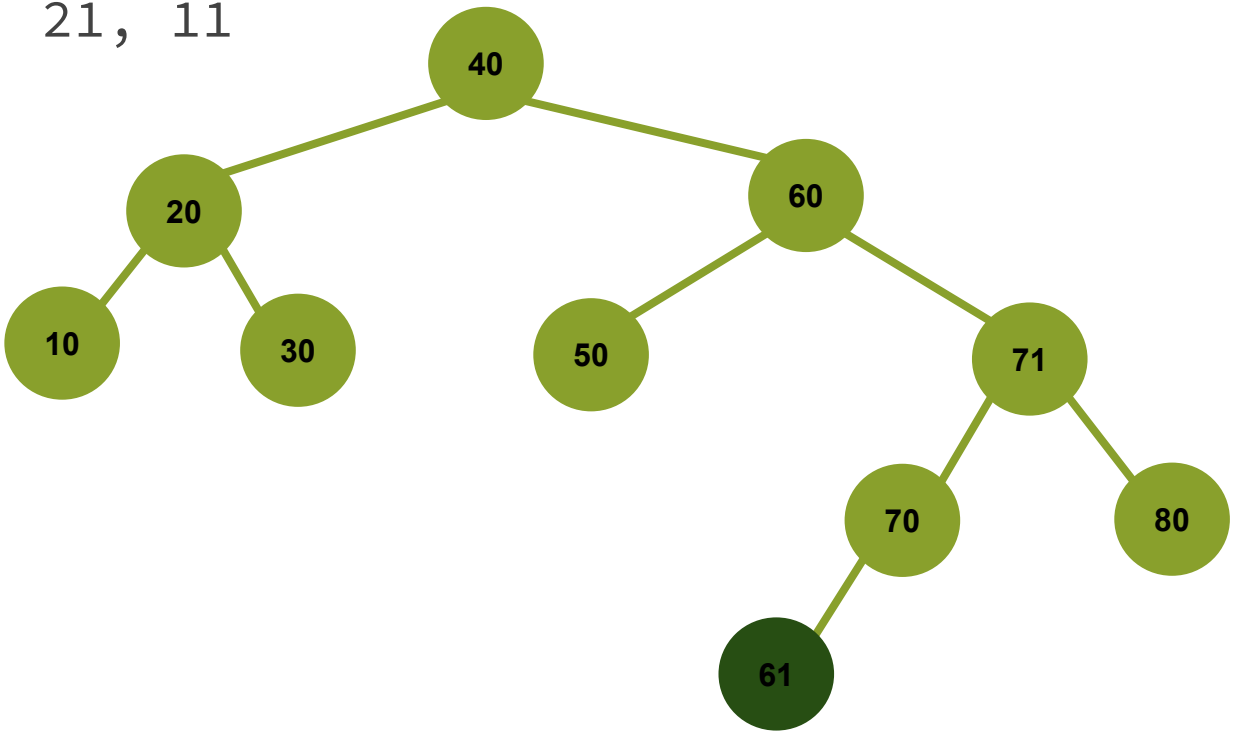
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



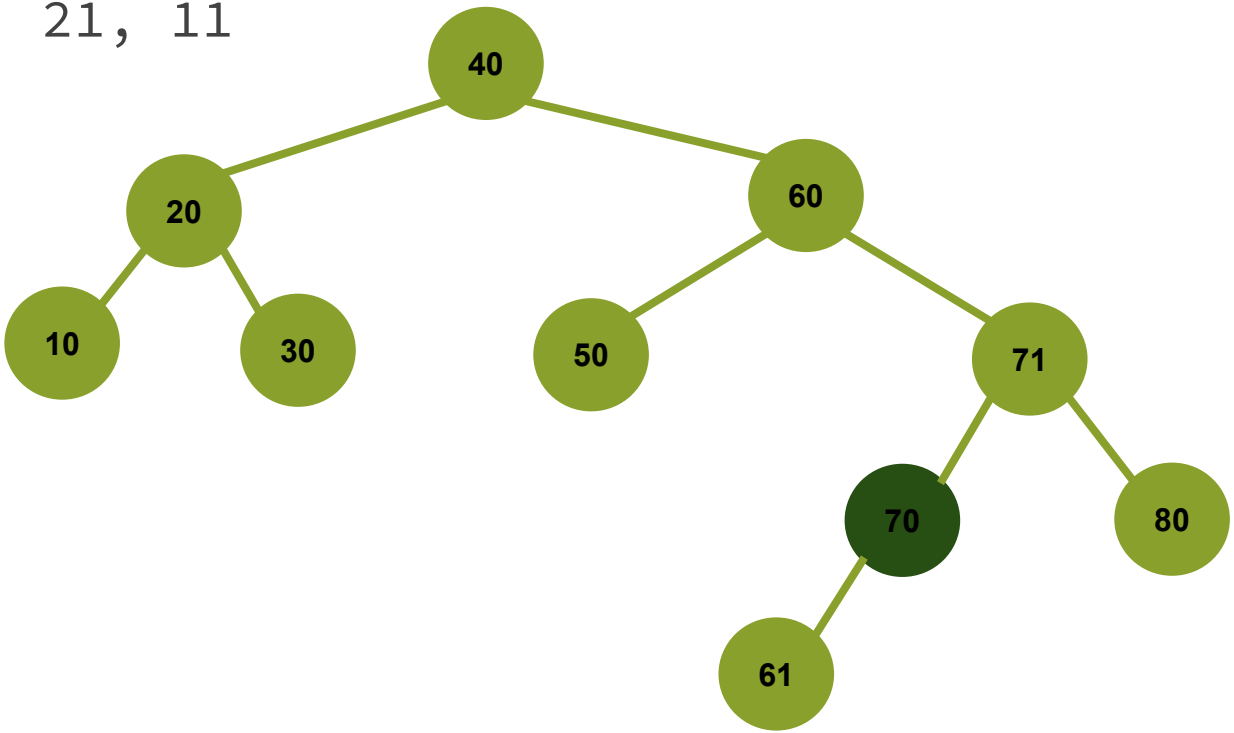
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



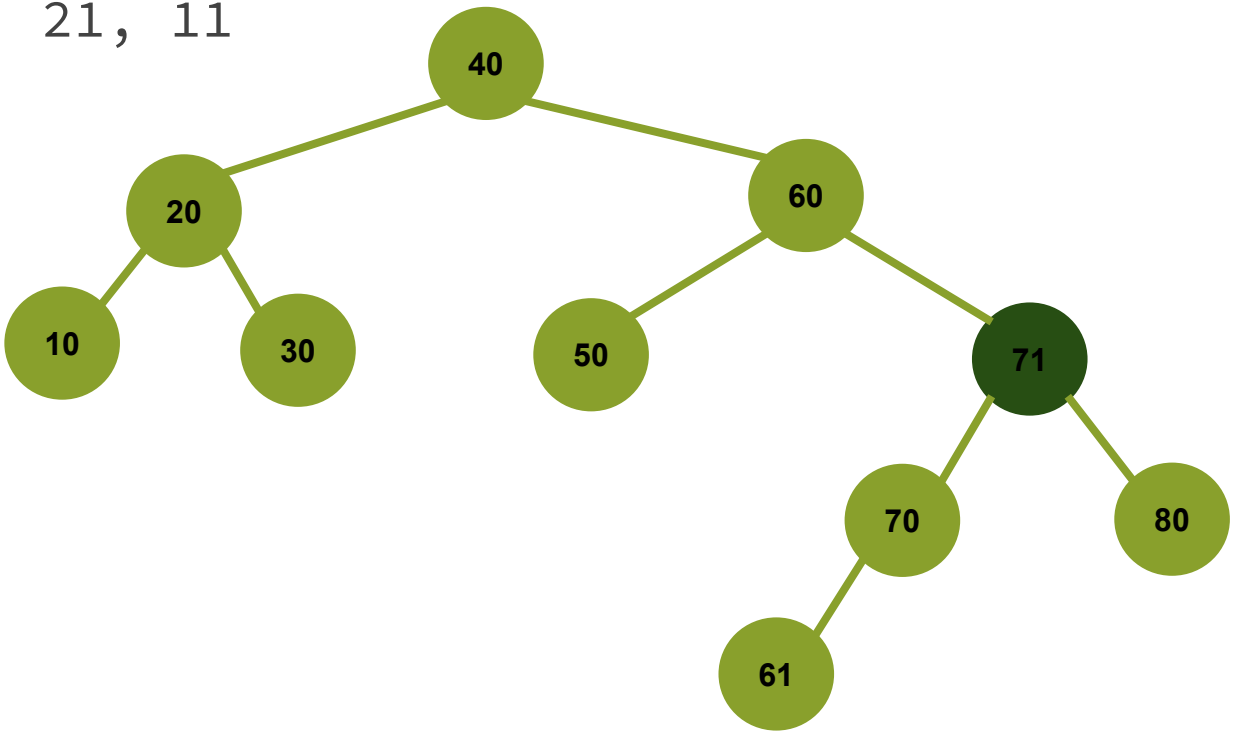
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



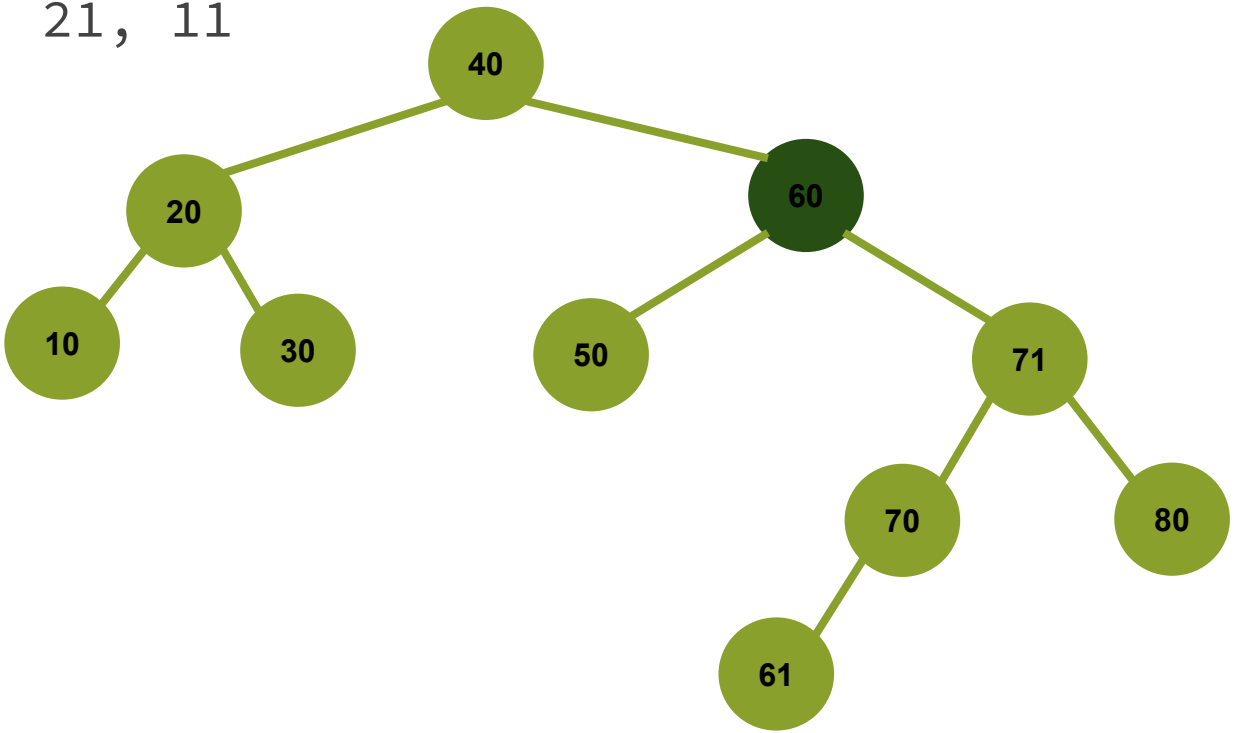
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



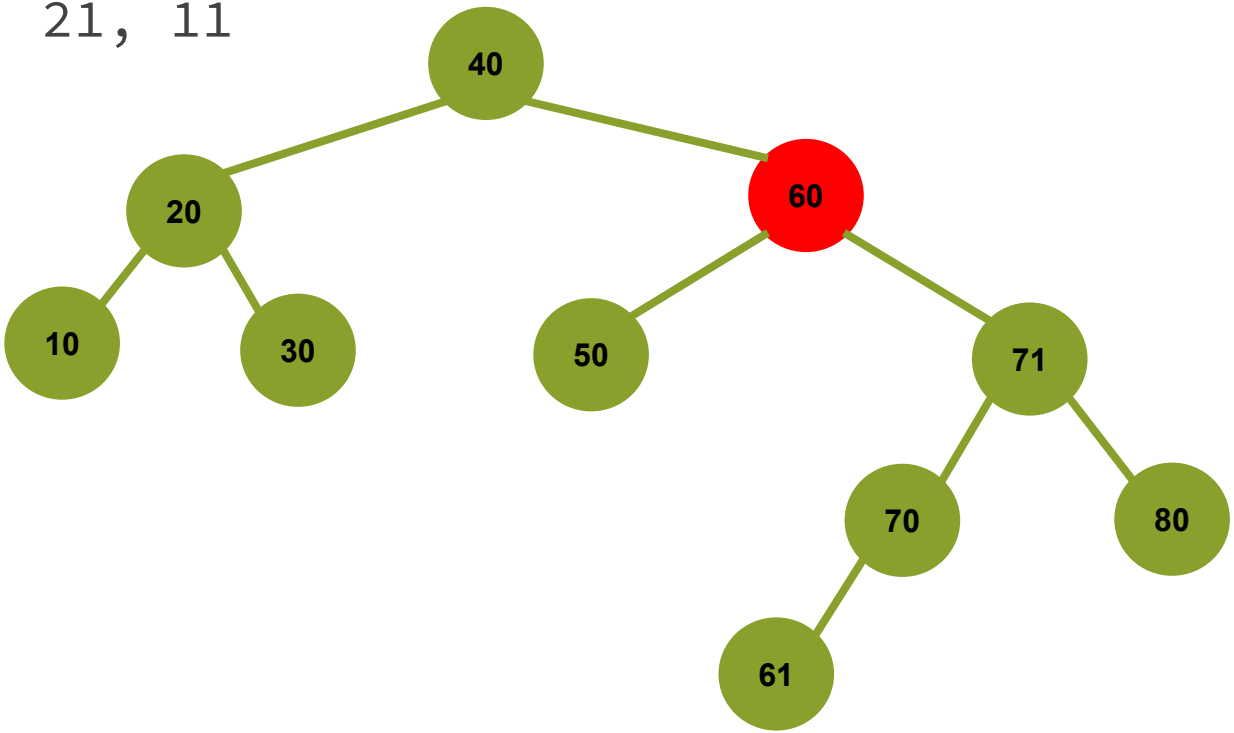
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



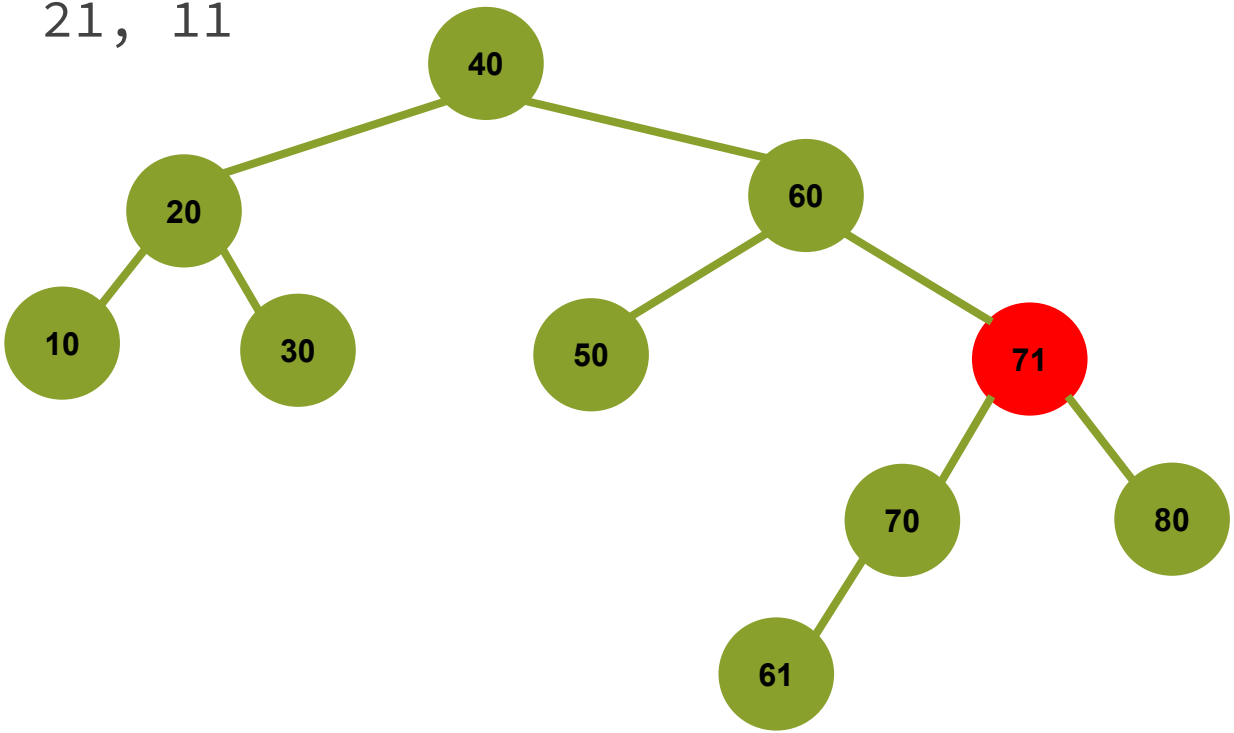
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



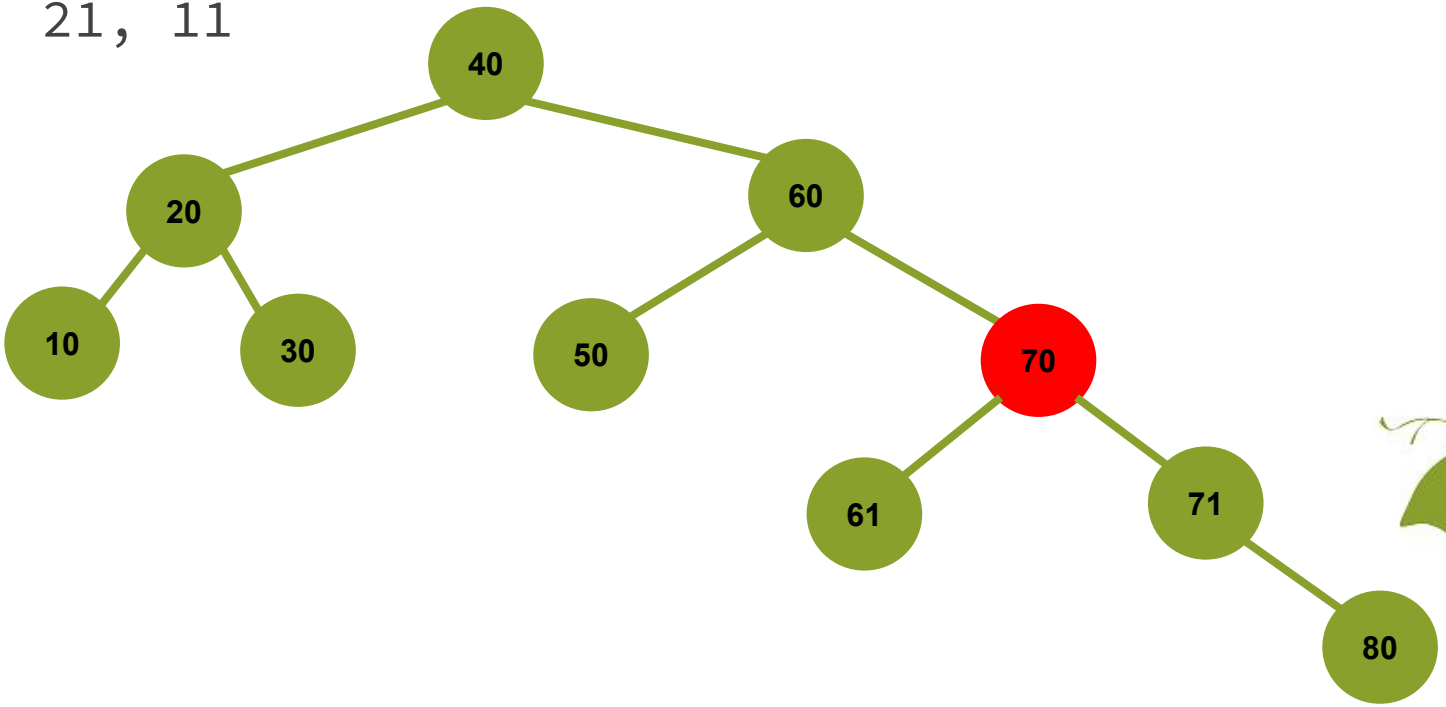
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



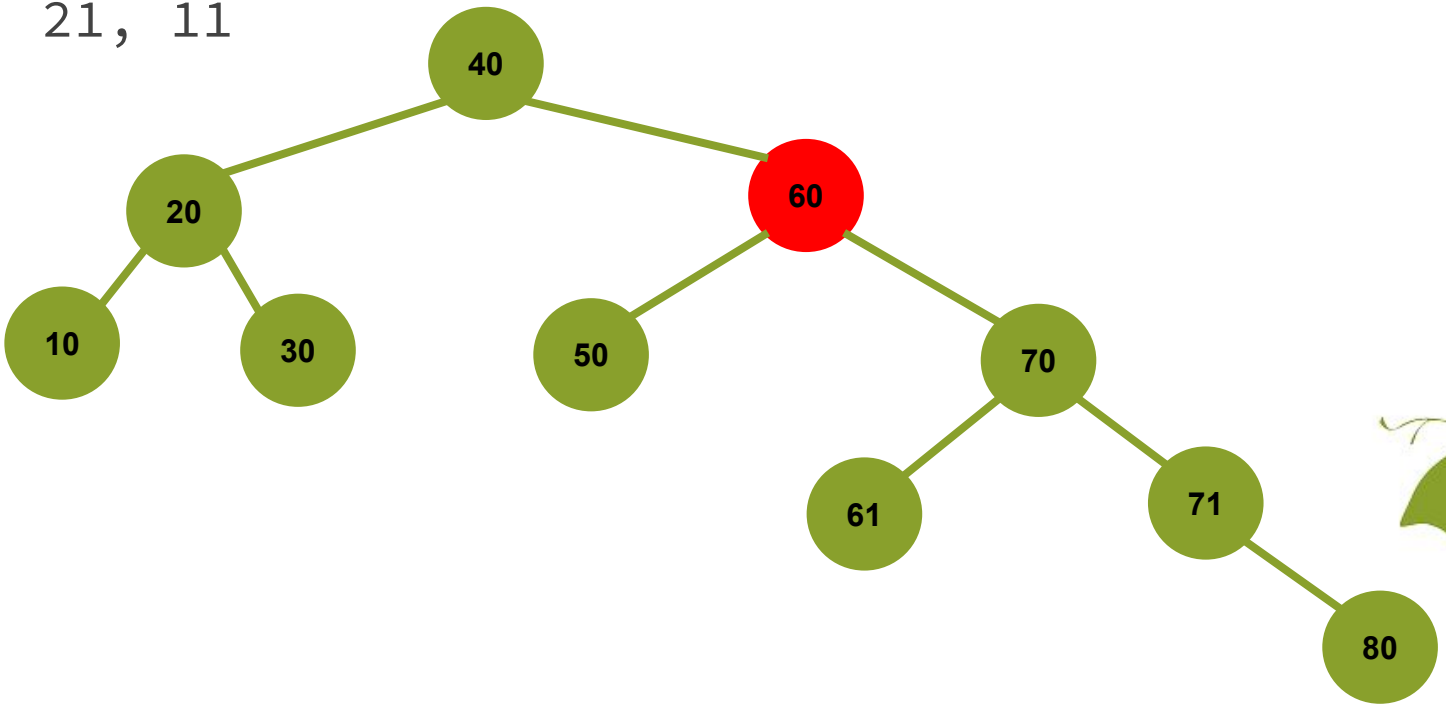
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



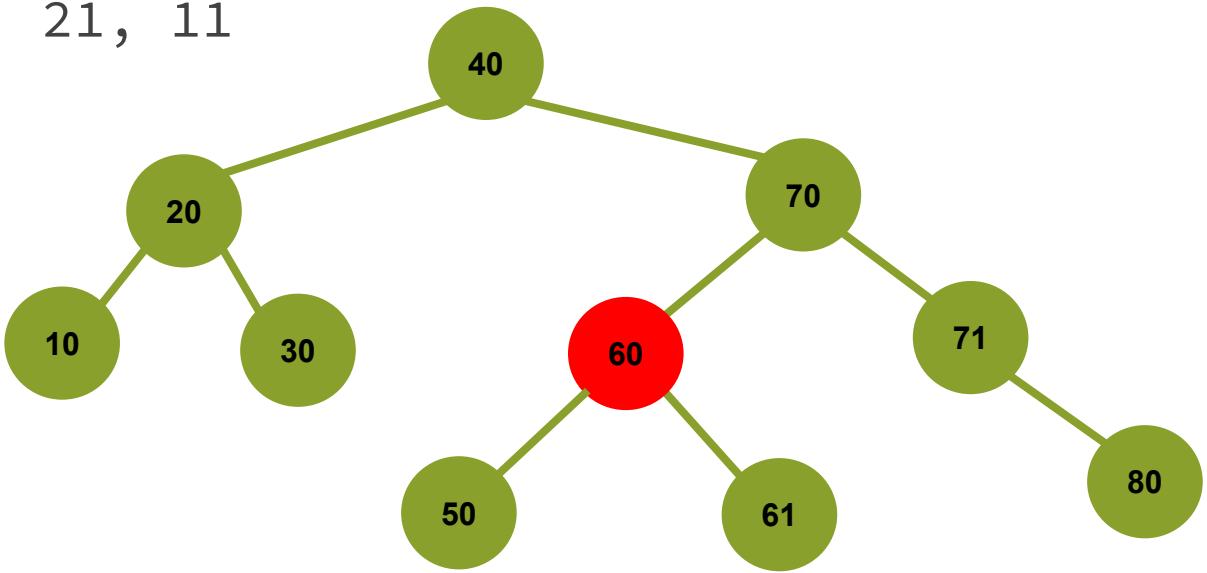
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



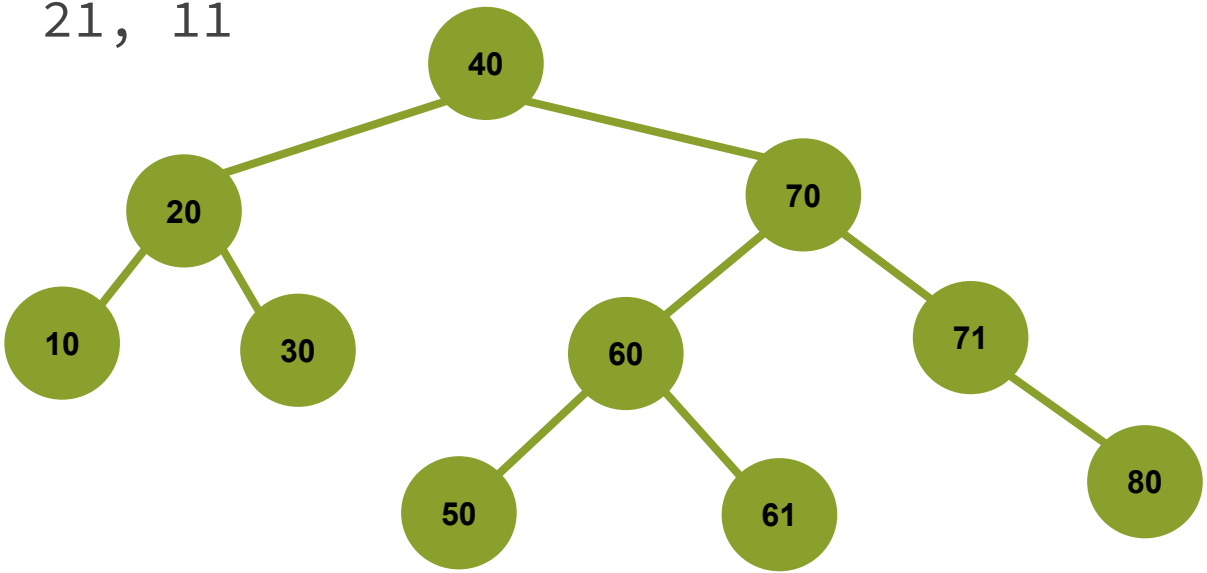
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



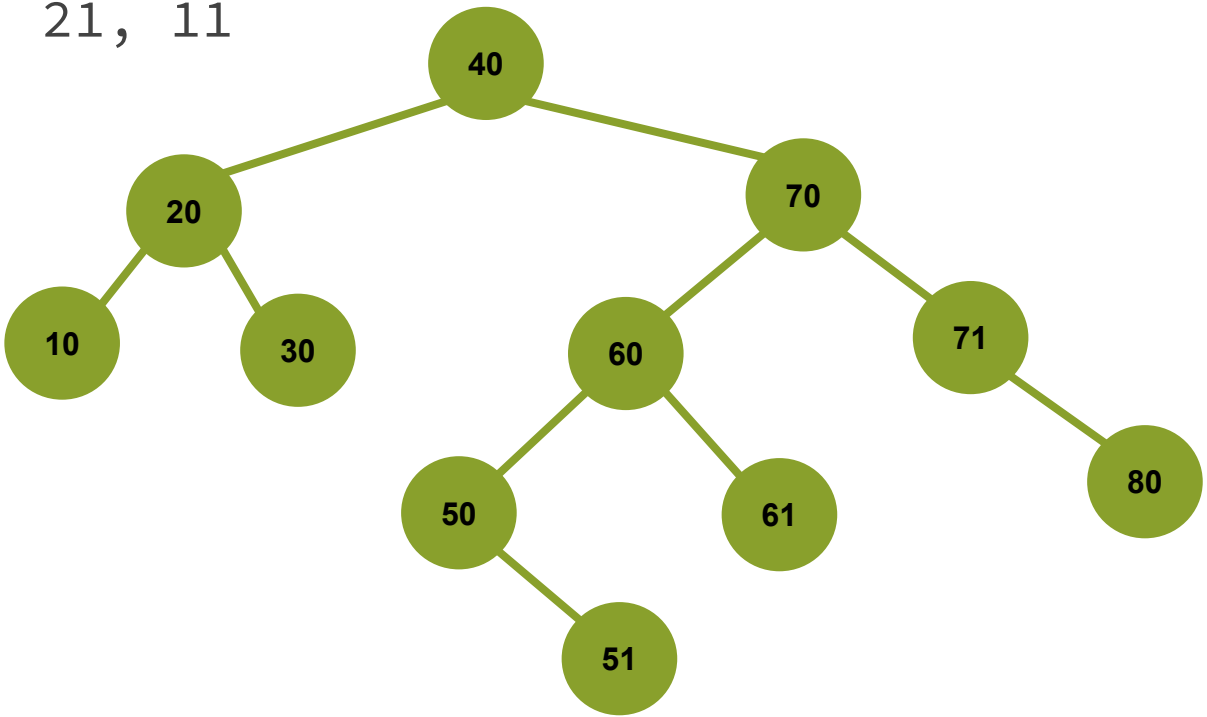
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, **61**, 51, 41, 31, 21, 11



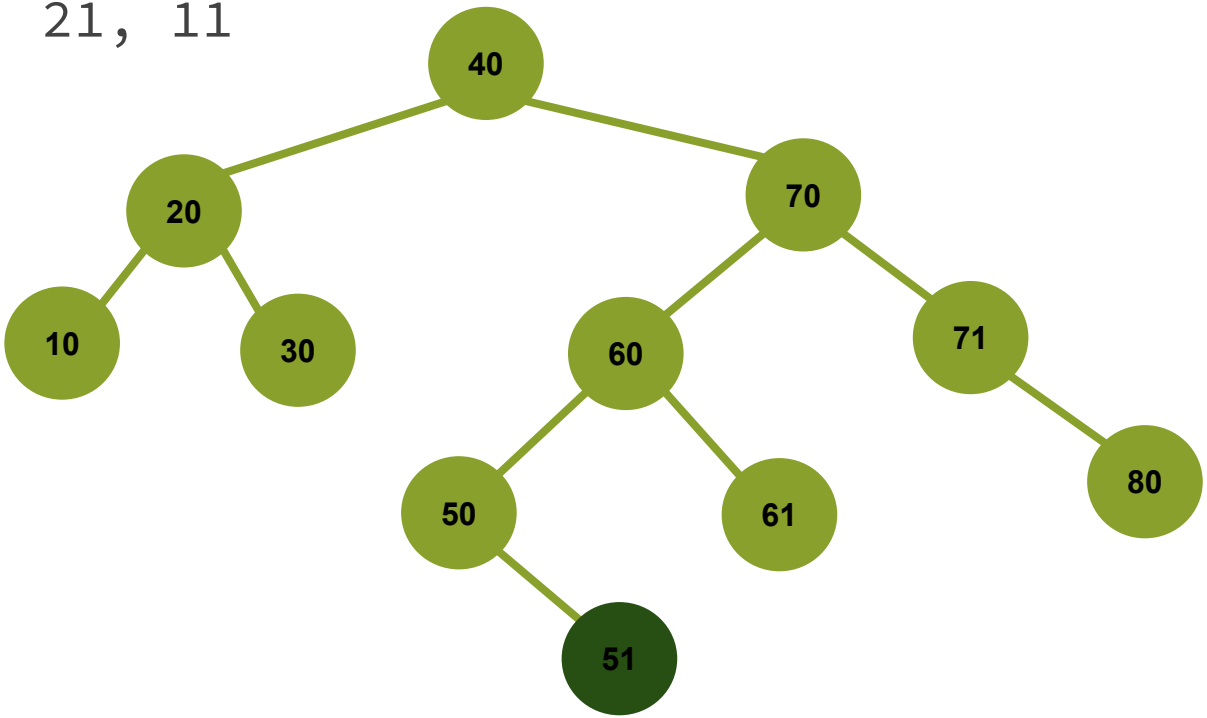
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



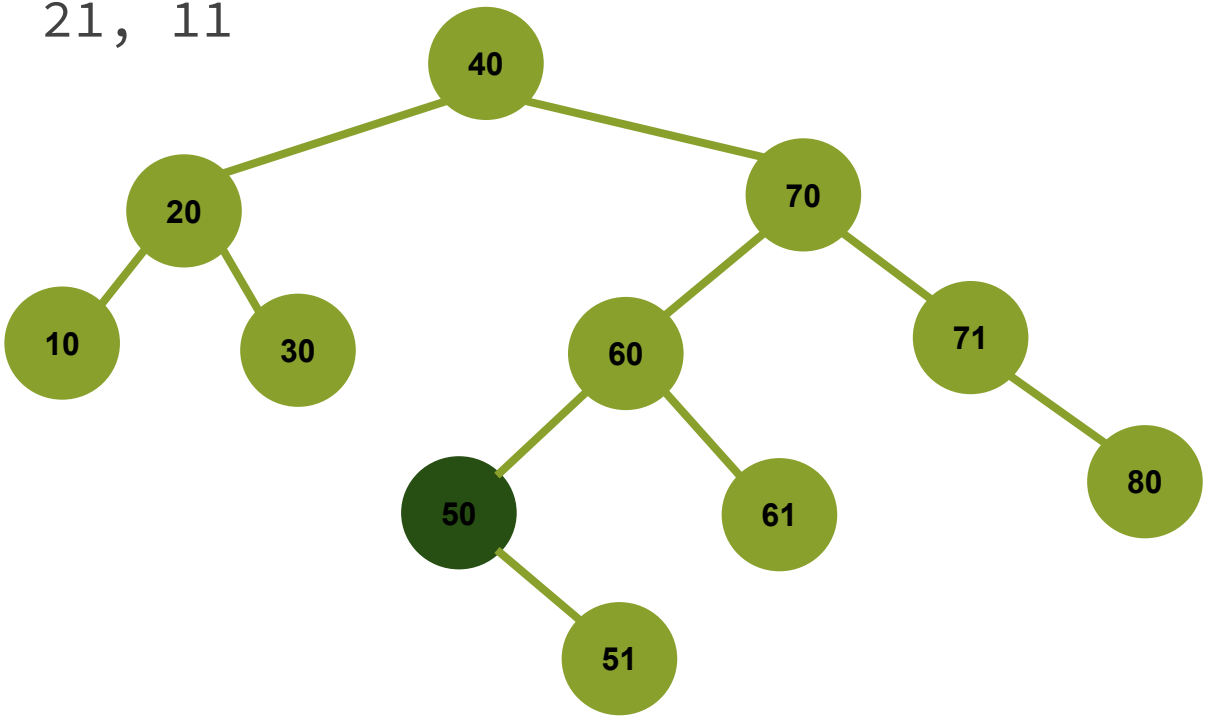
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



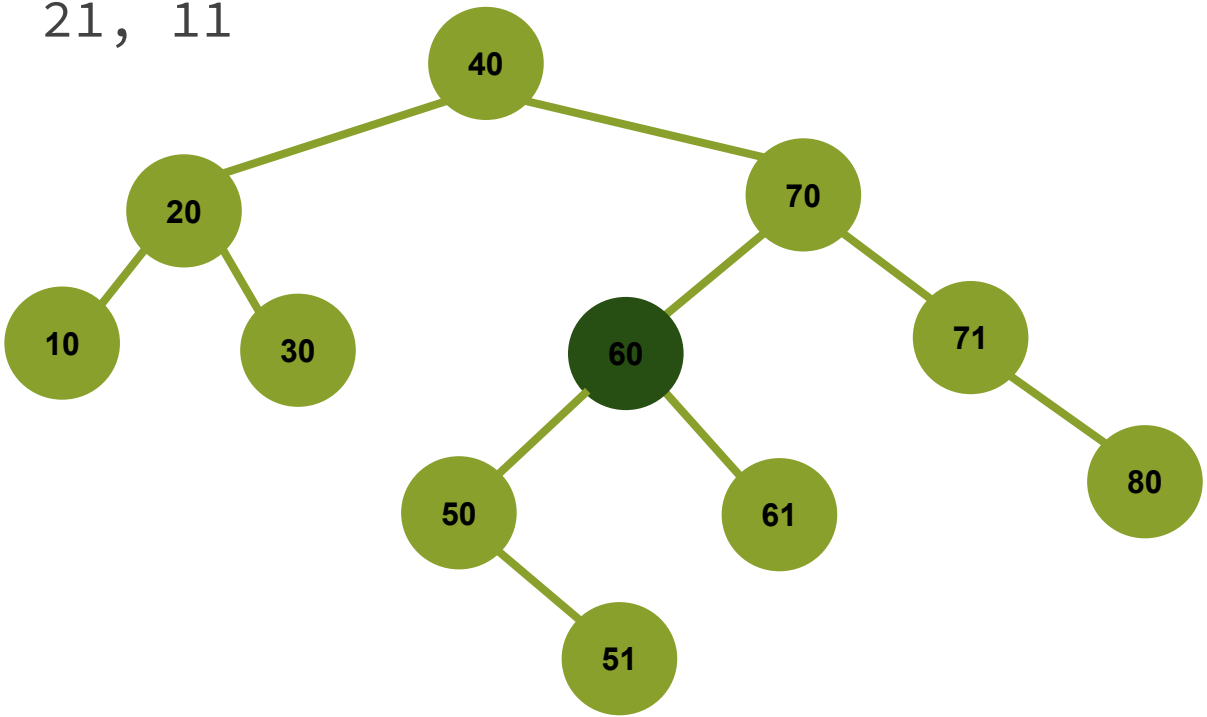
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



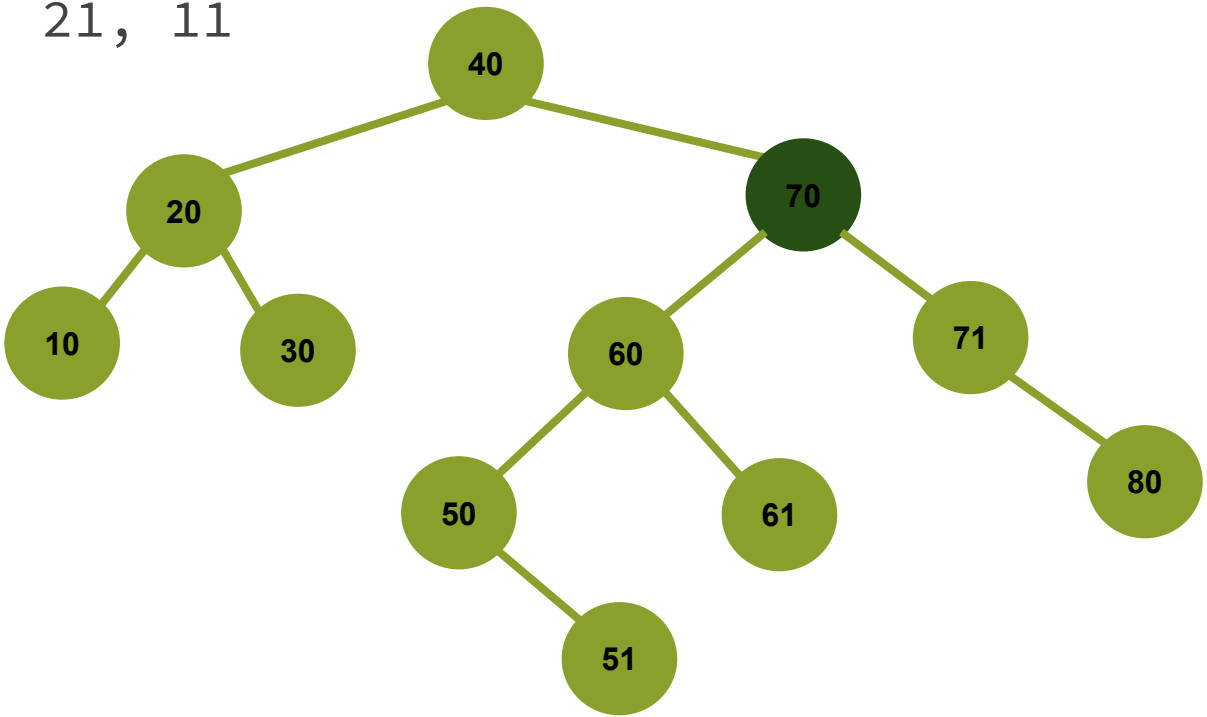
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



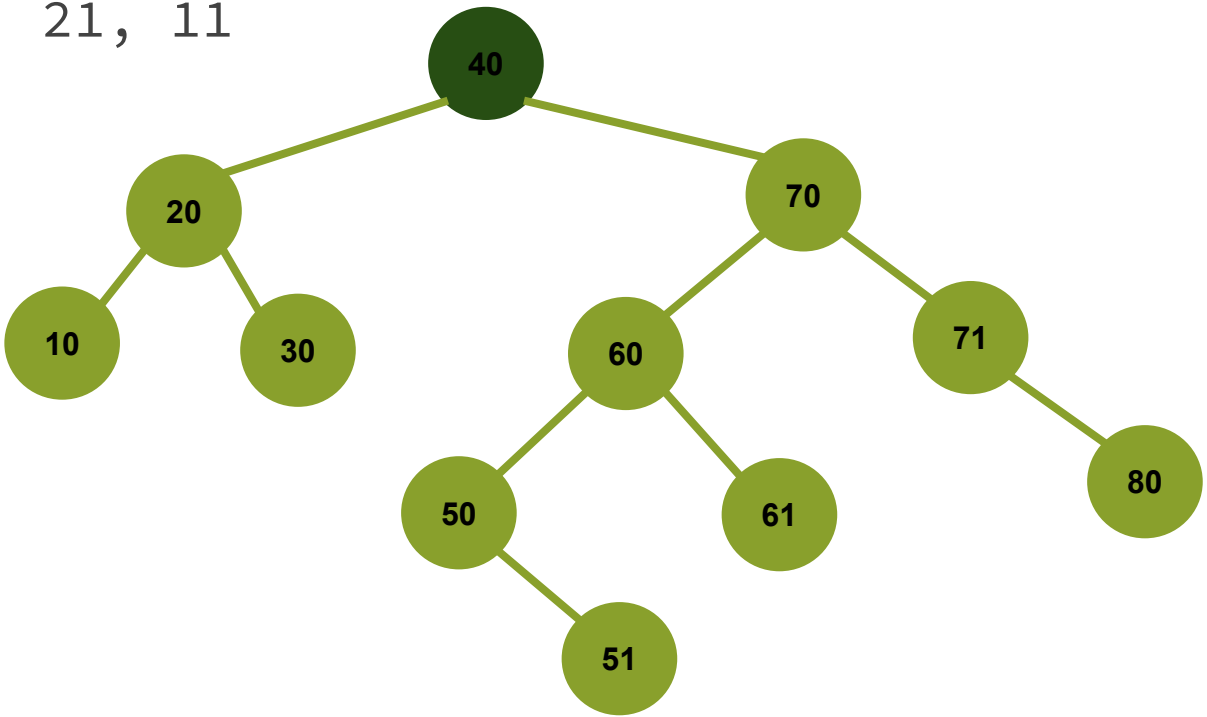
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



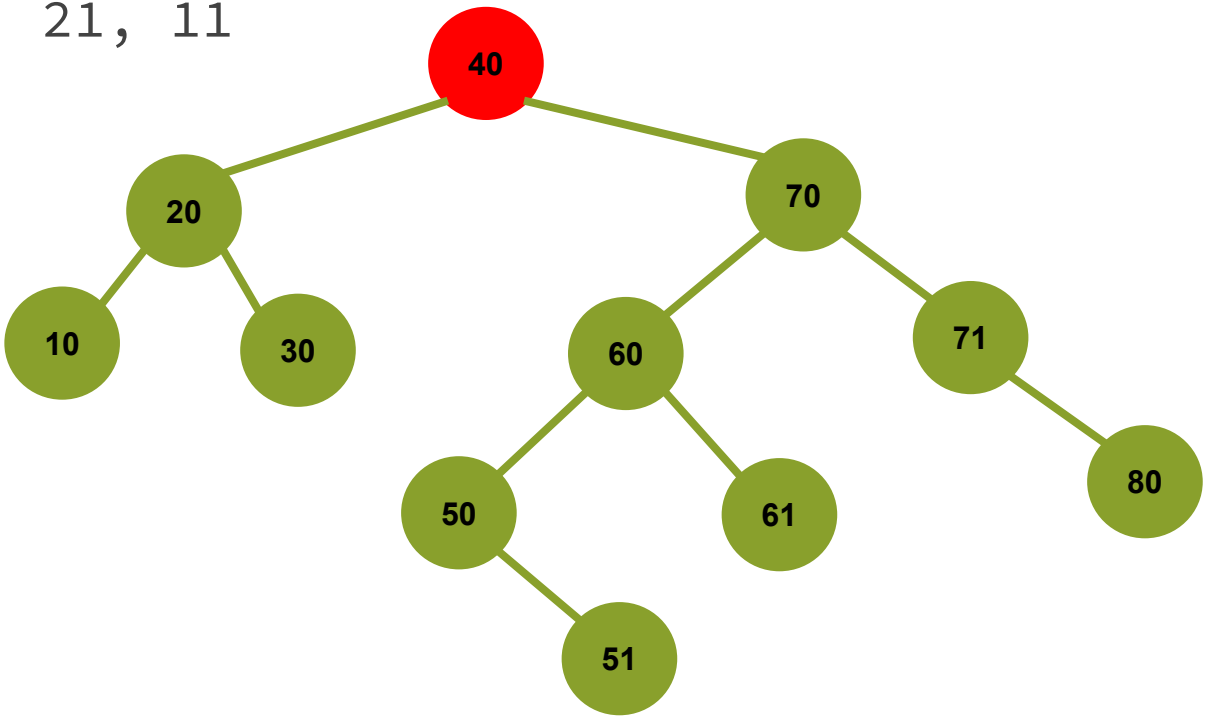
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



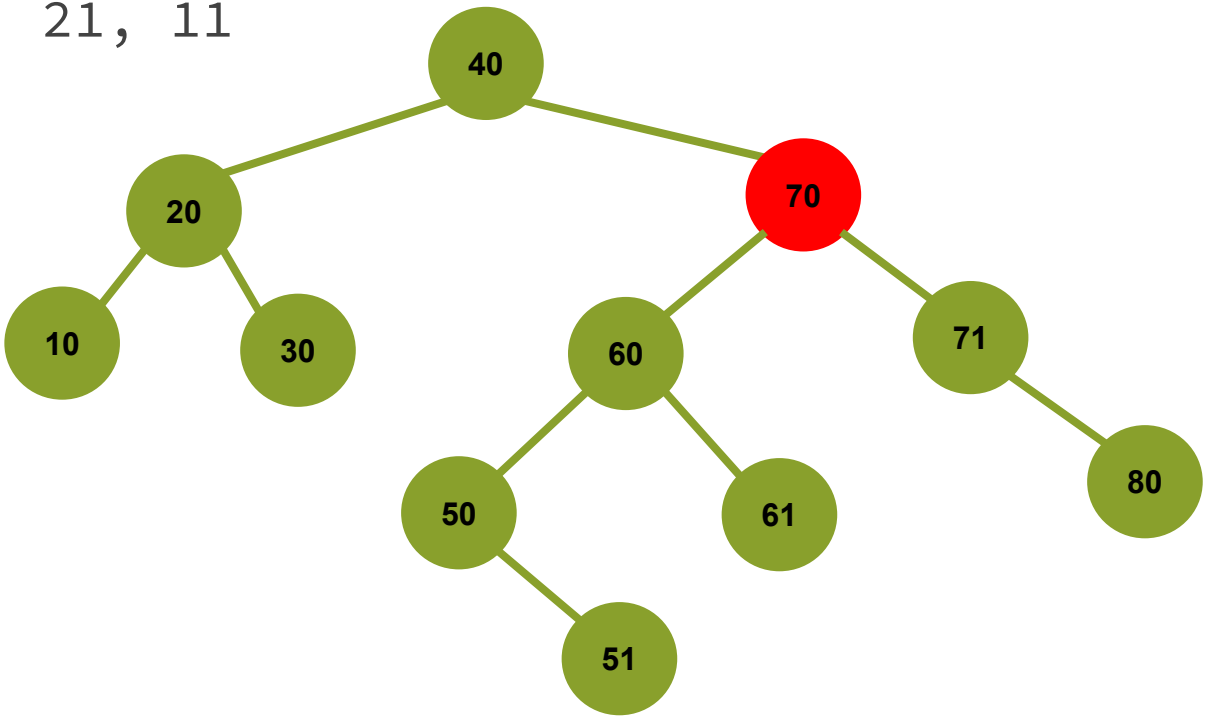
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



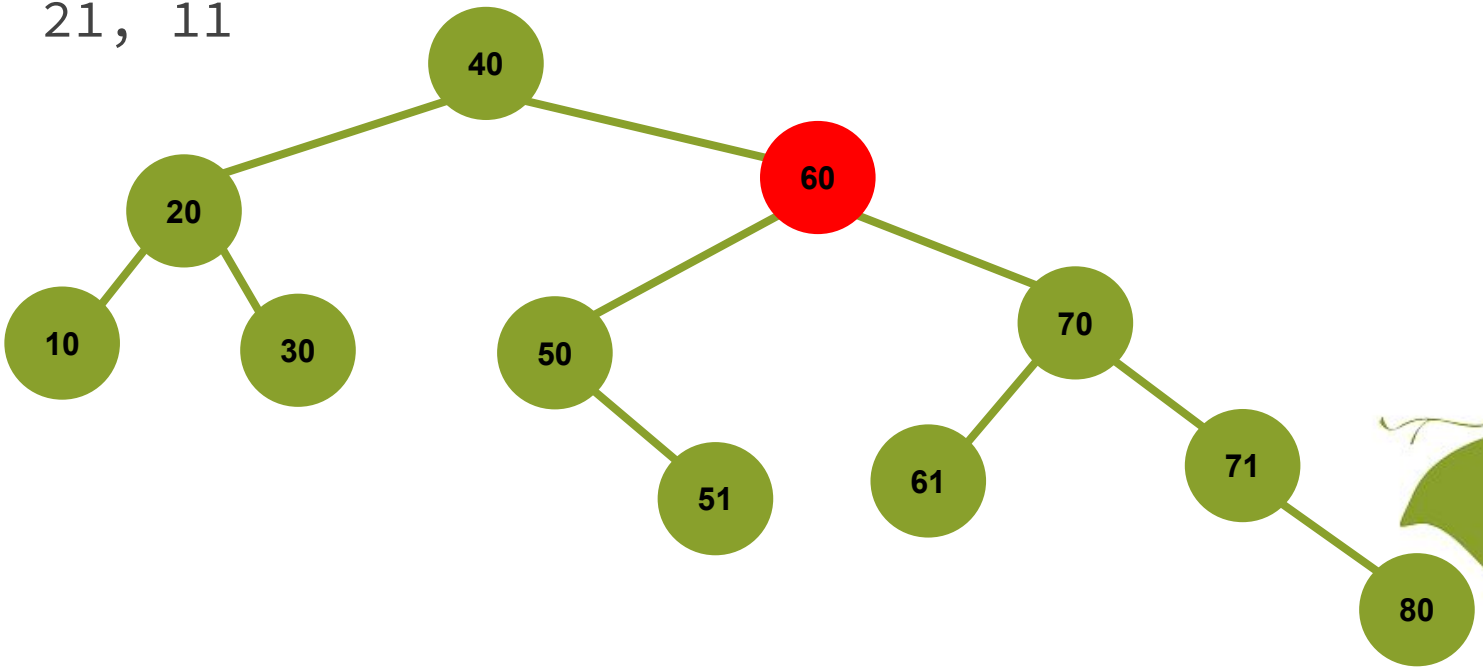
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



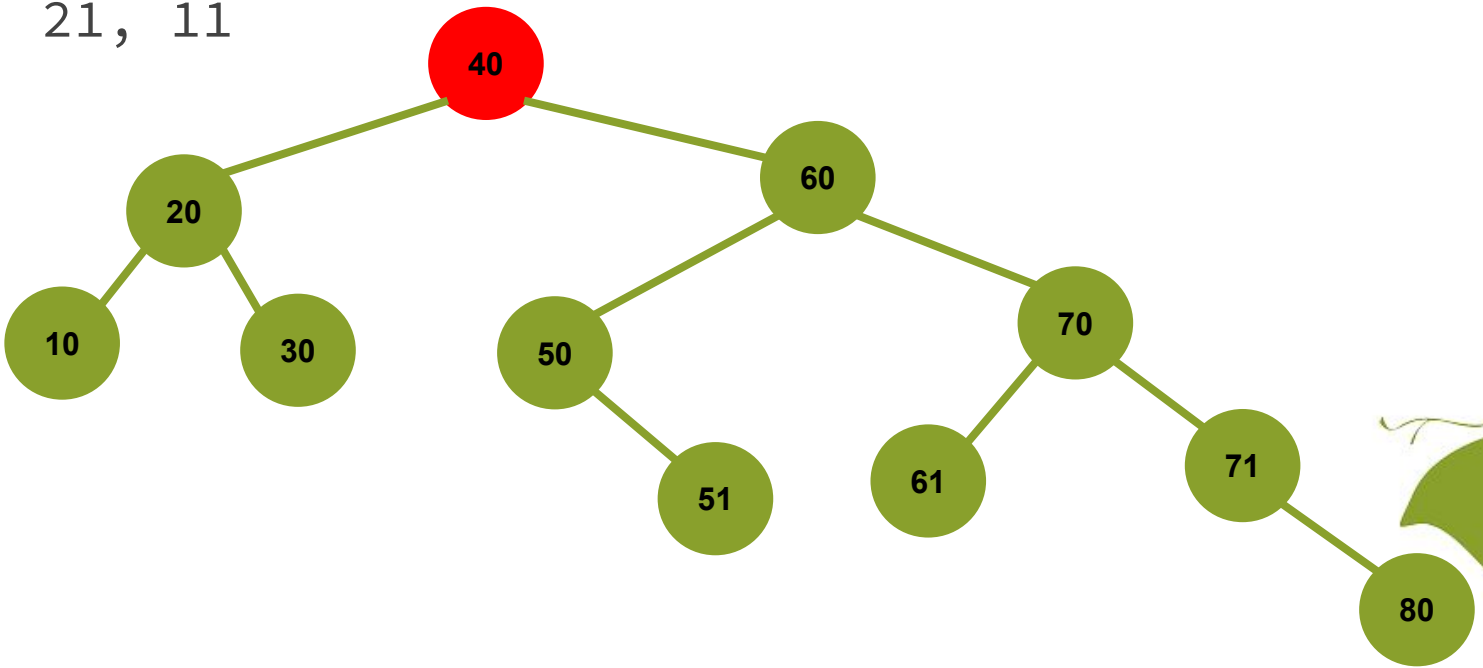
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



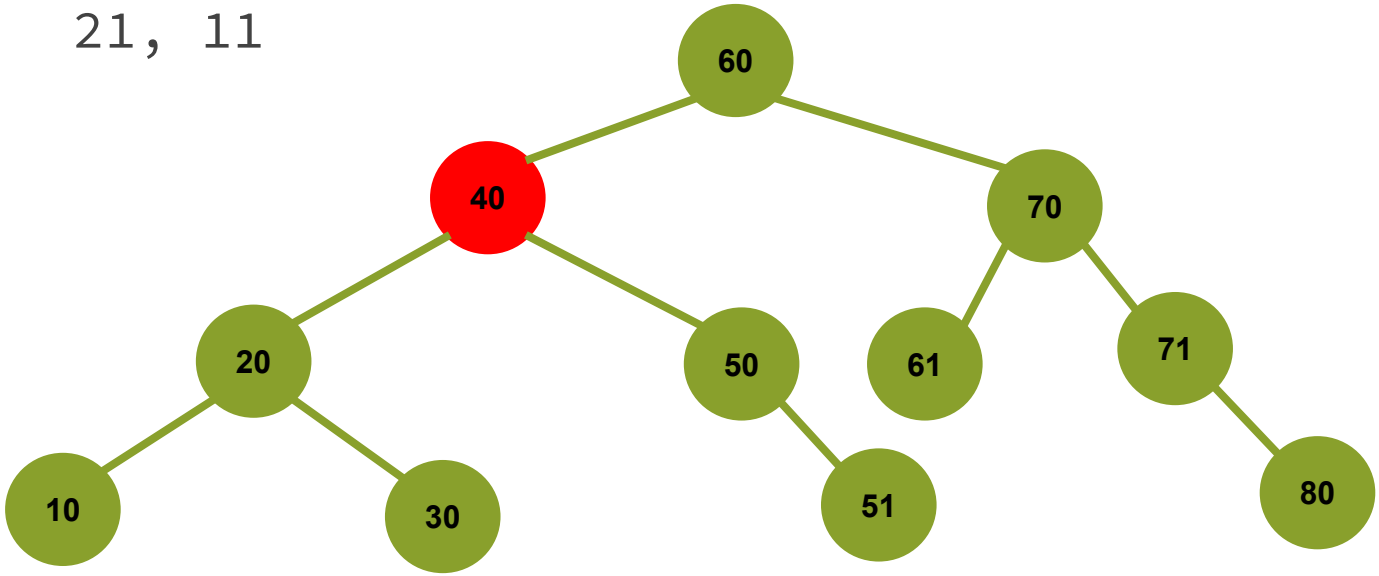
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



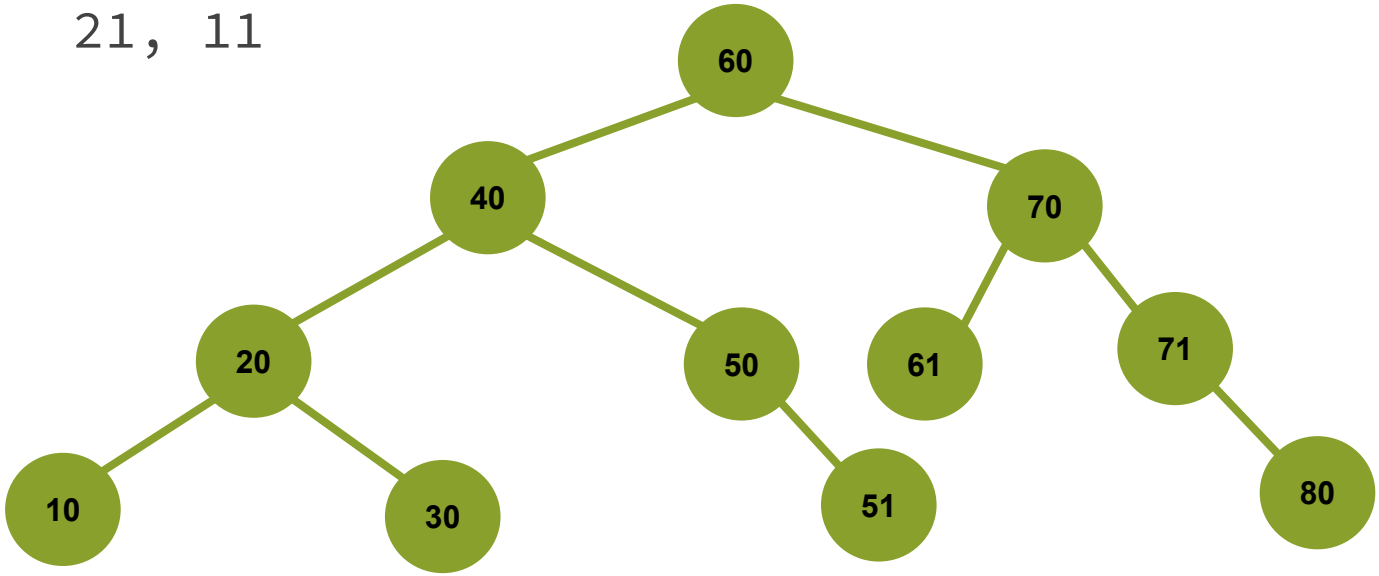
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



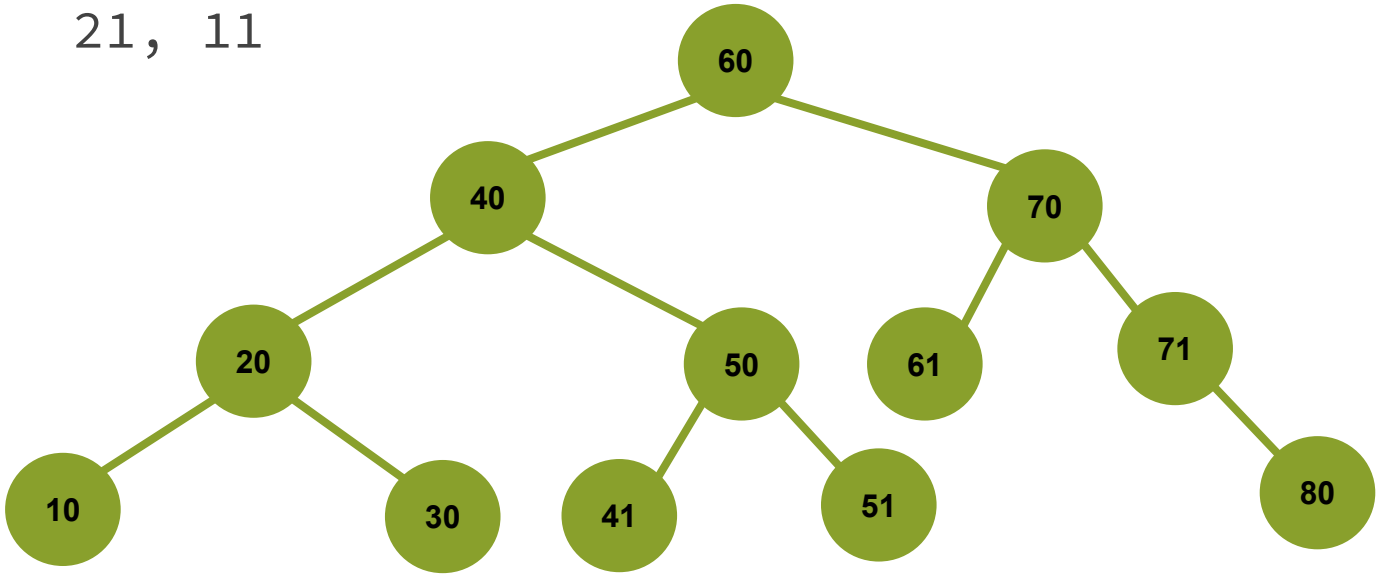
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, **51**, 41, 31, 21, 11



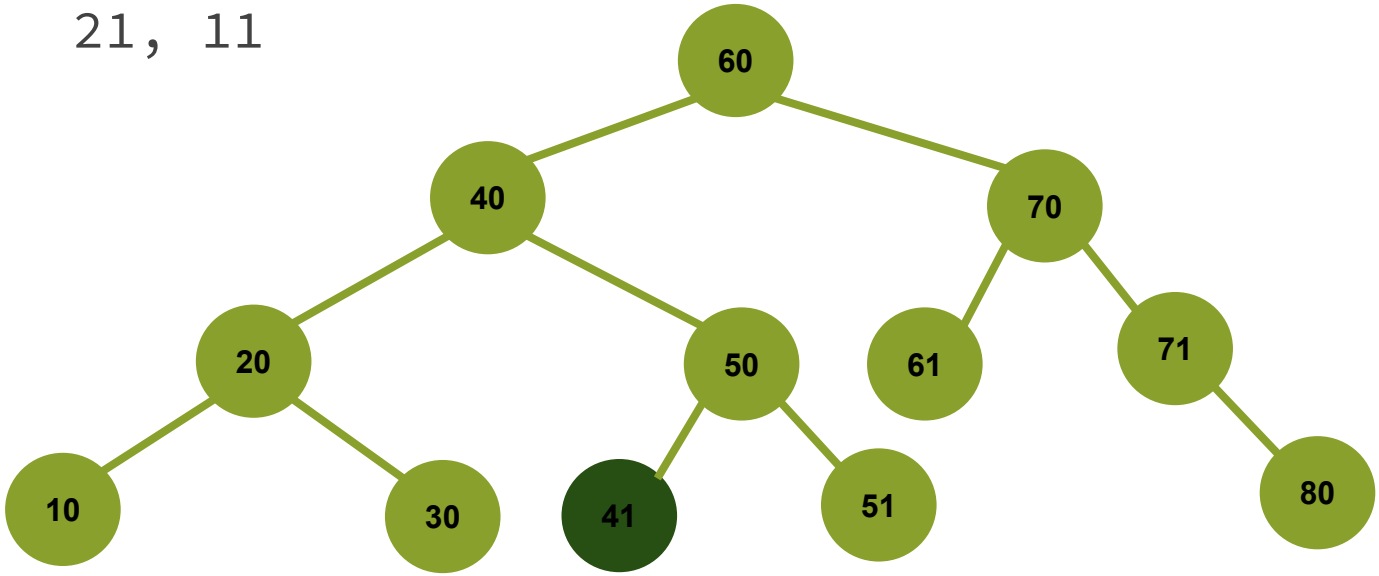
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, **41**, 31, 21, 11



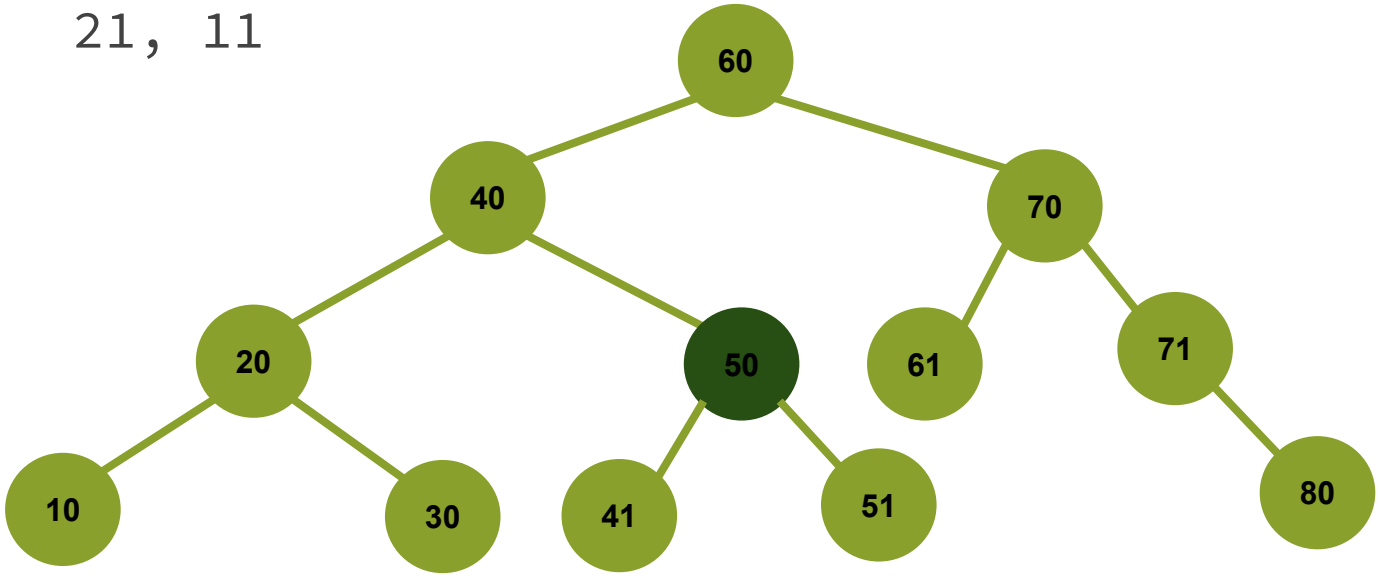
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, **41**, 31, 21, 11



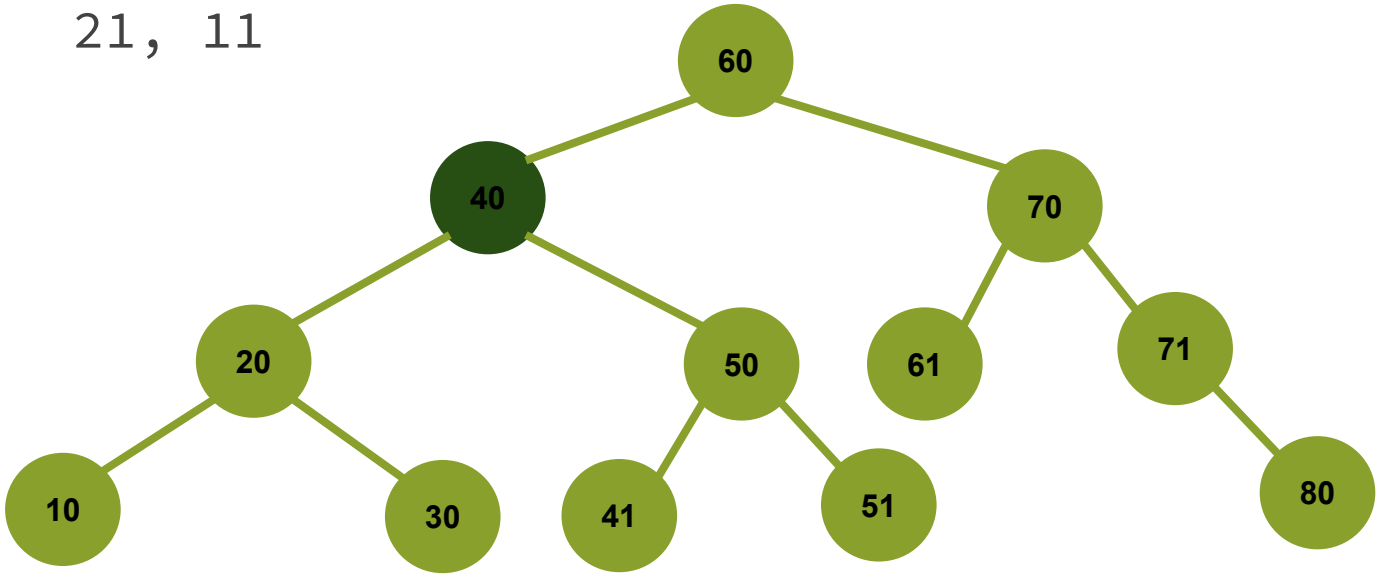
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, **41**, 31, 21, 11



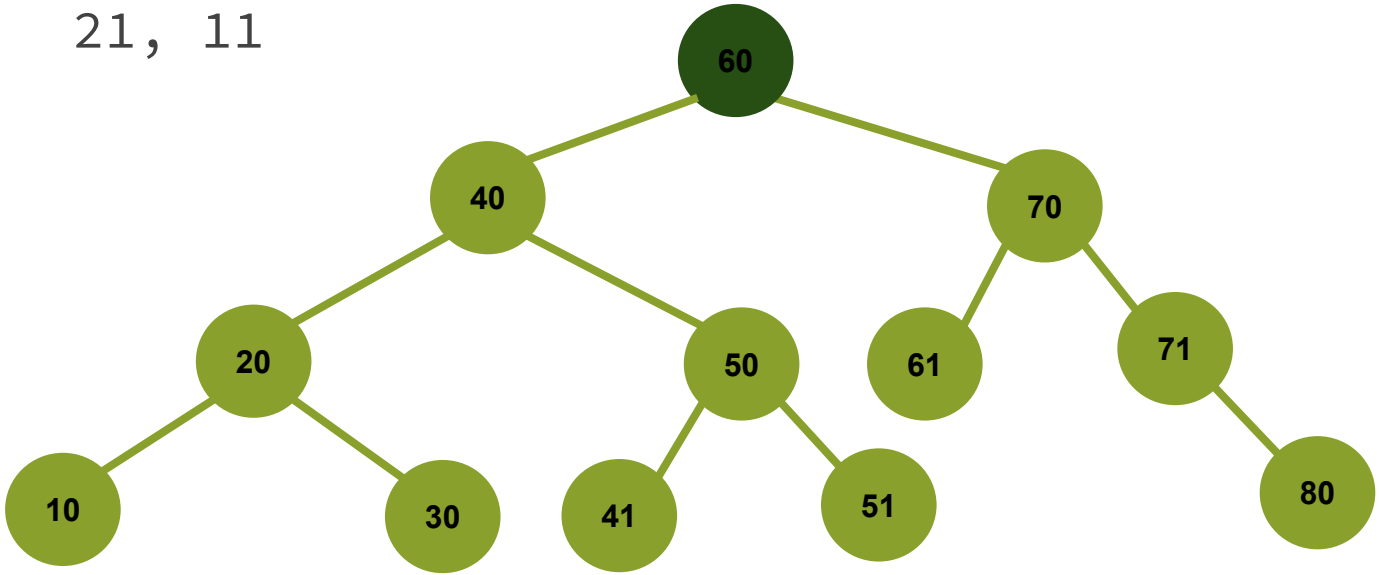
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, **41**, 31, 21, 11



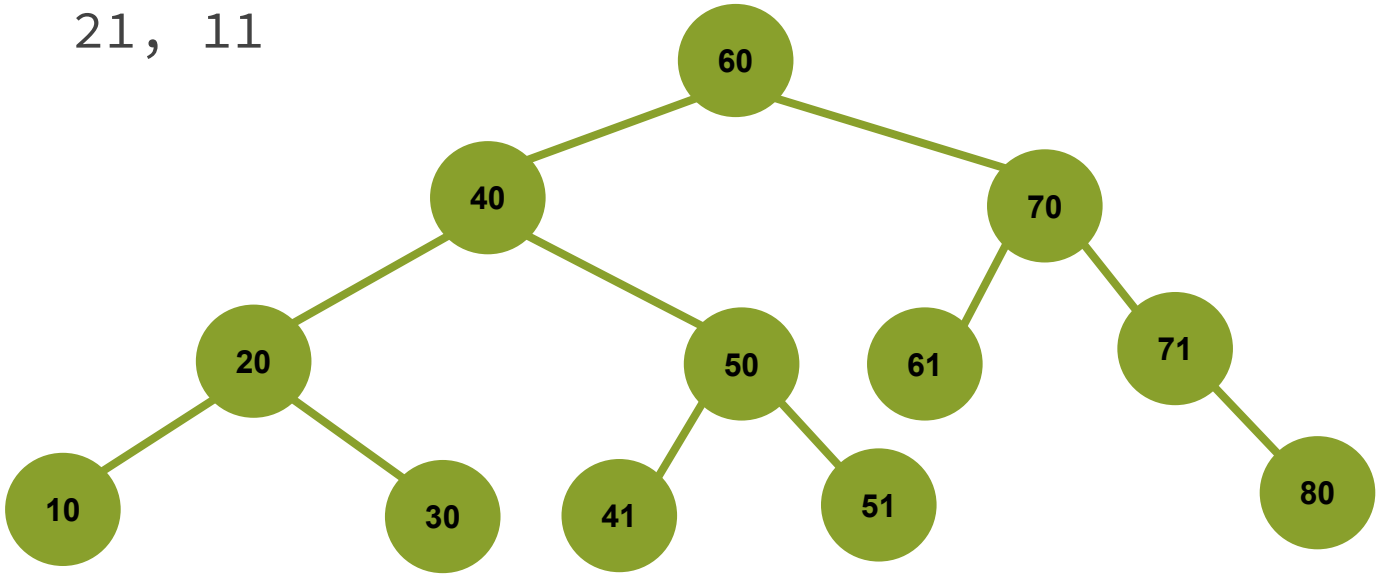
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, **41**, 31, 21, 11



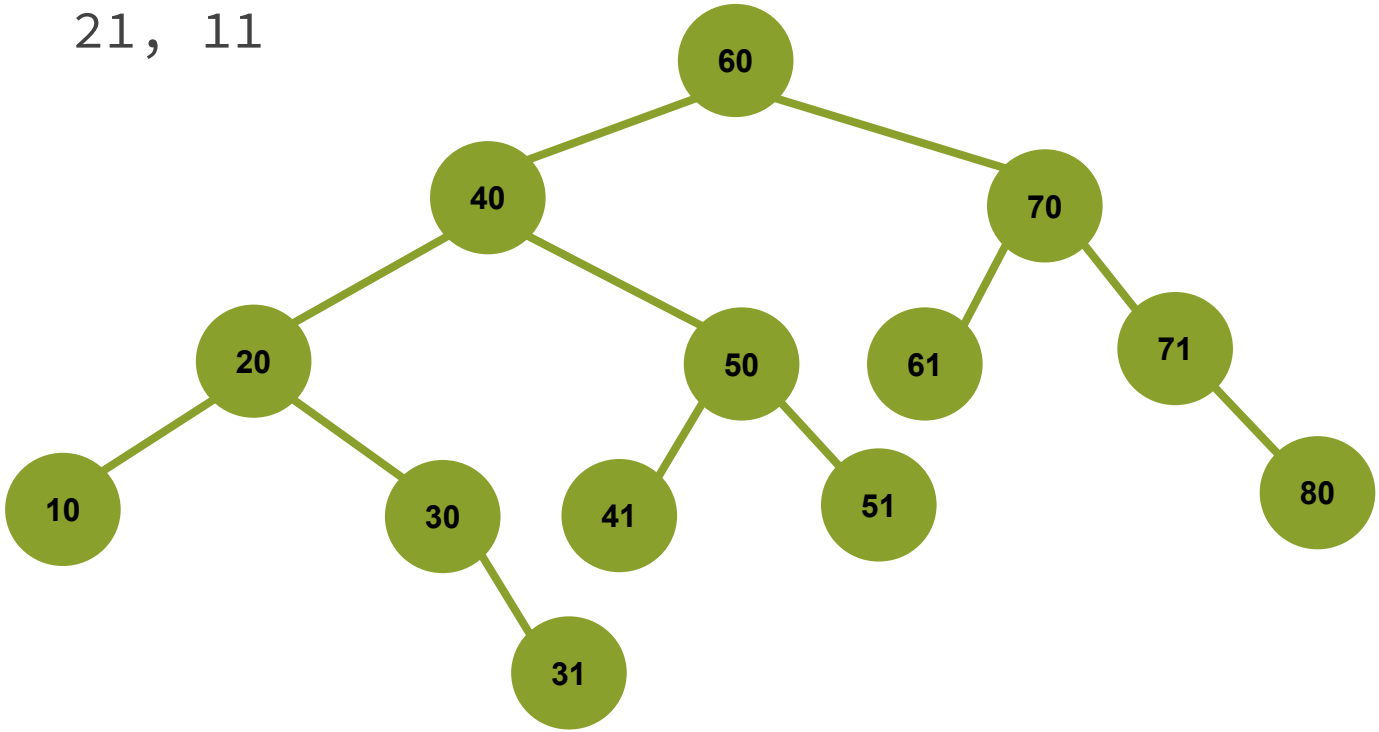
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, **41**, 31, 21, 11



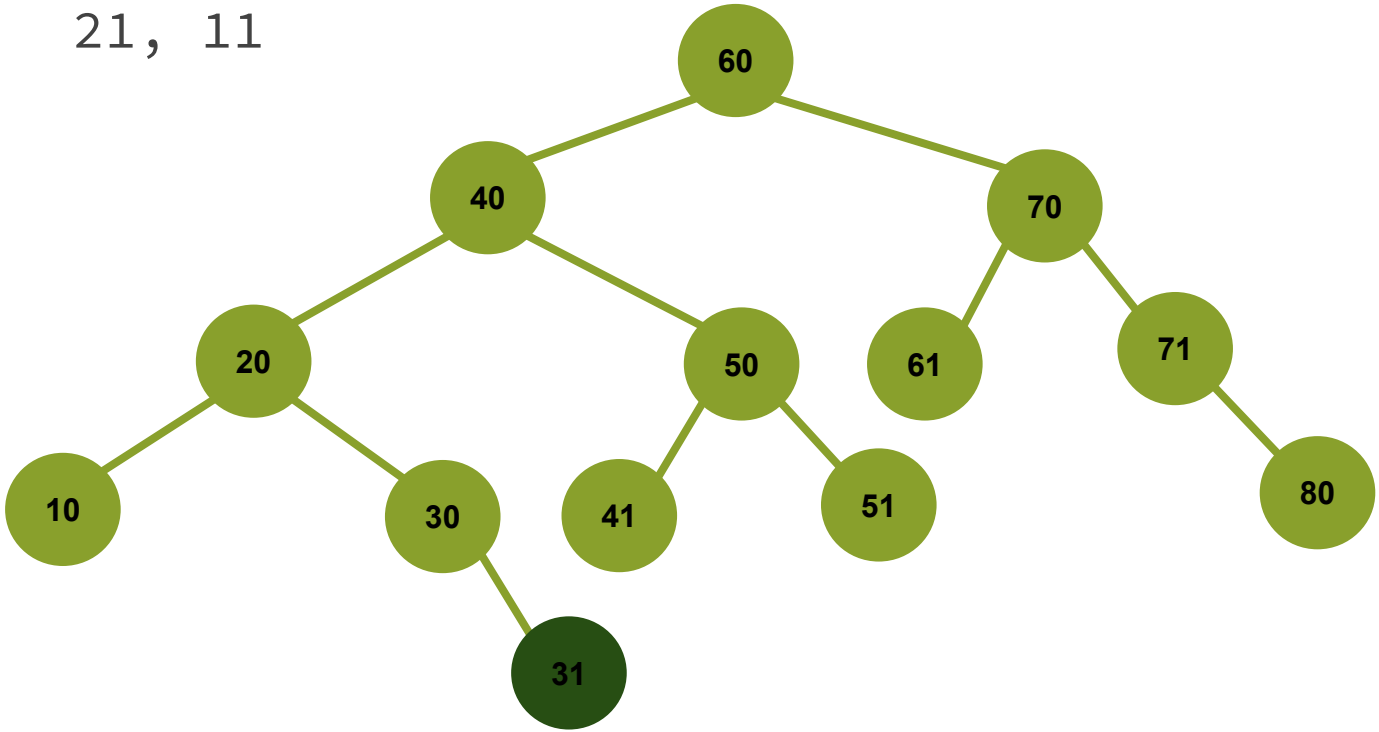
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



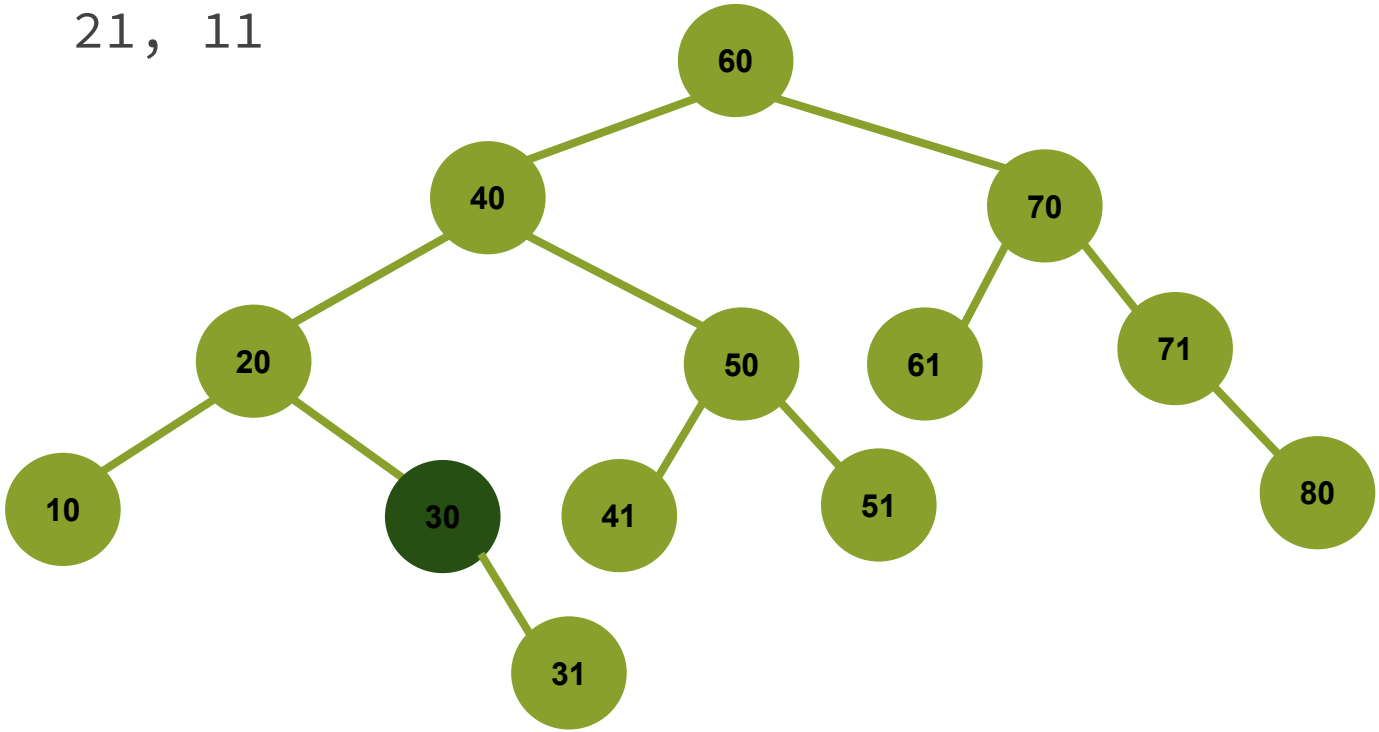
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



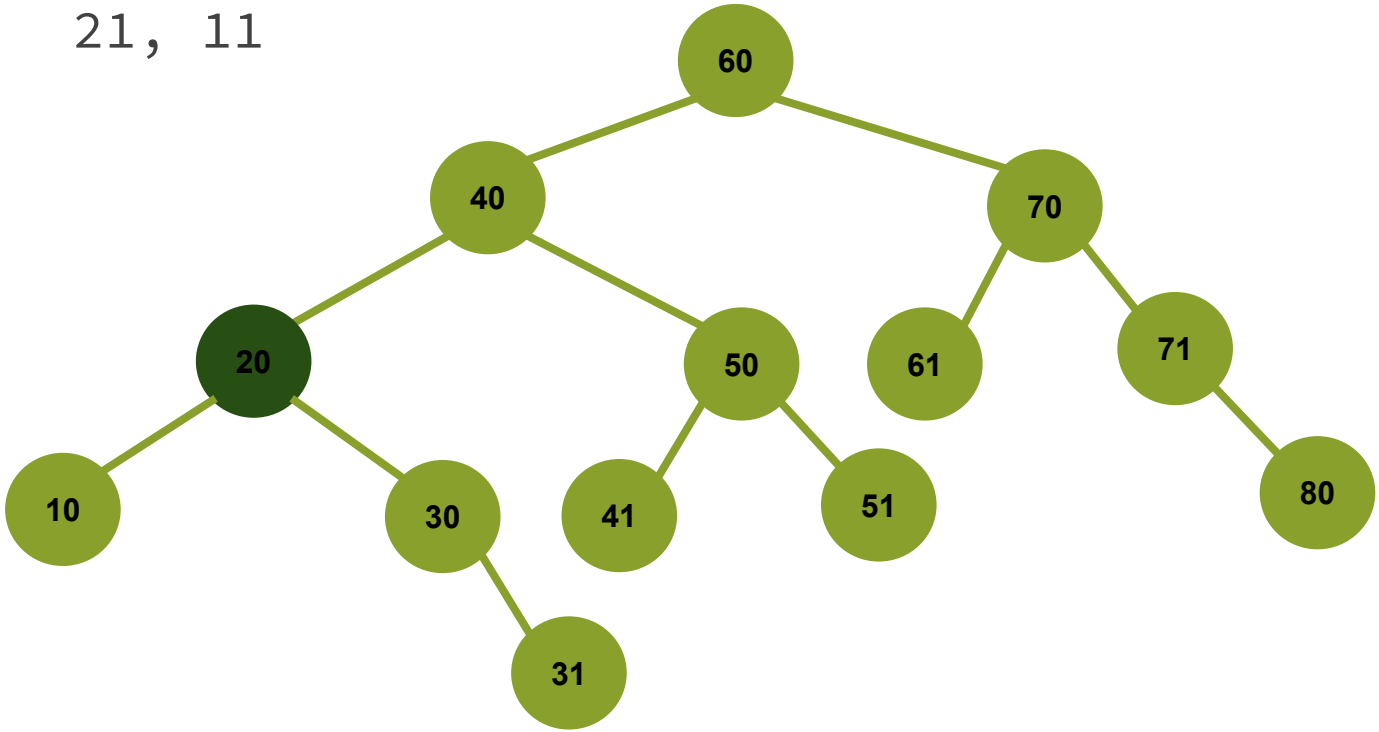
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



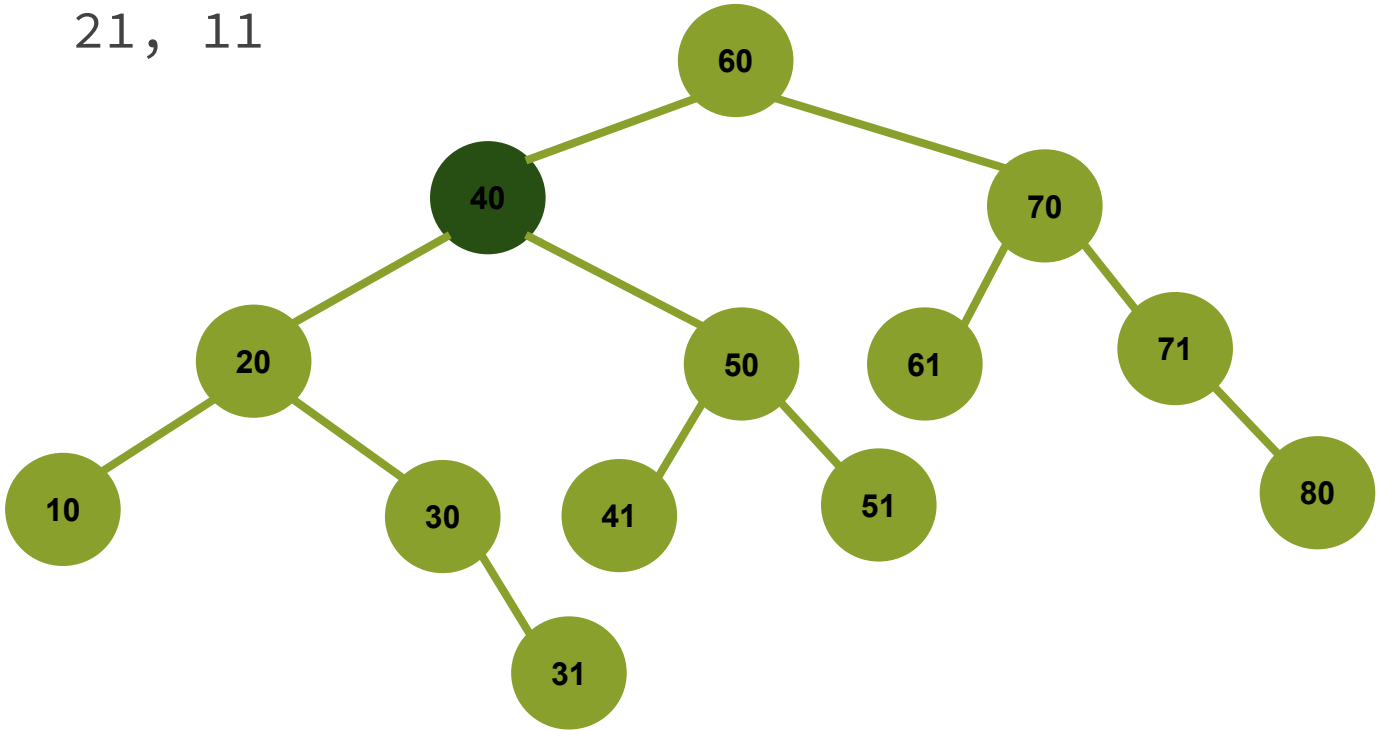
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



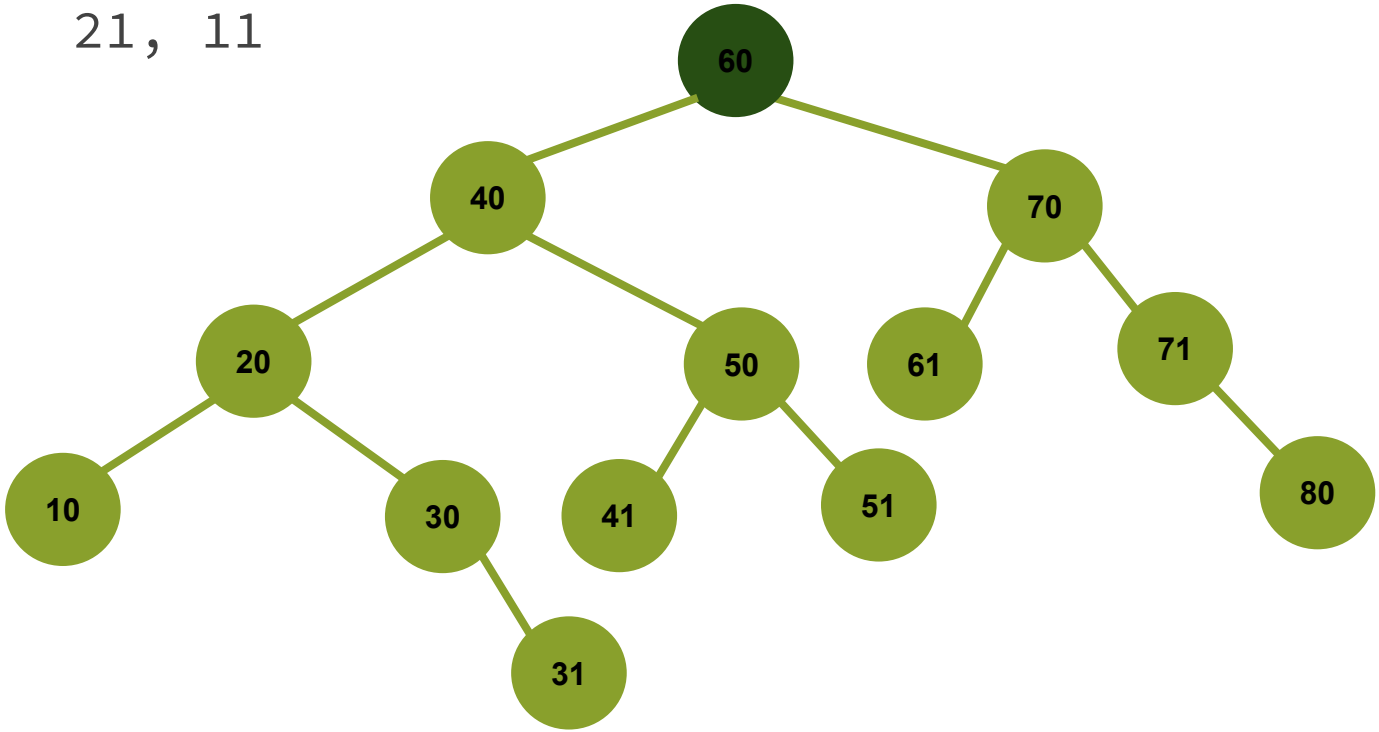
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



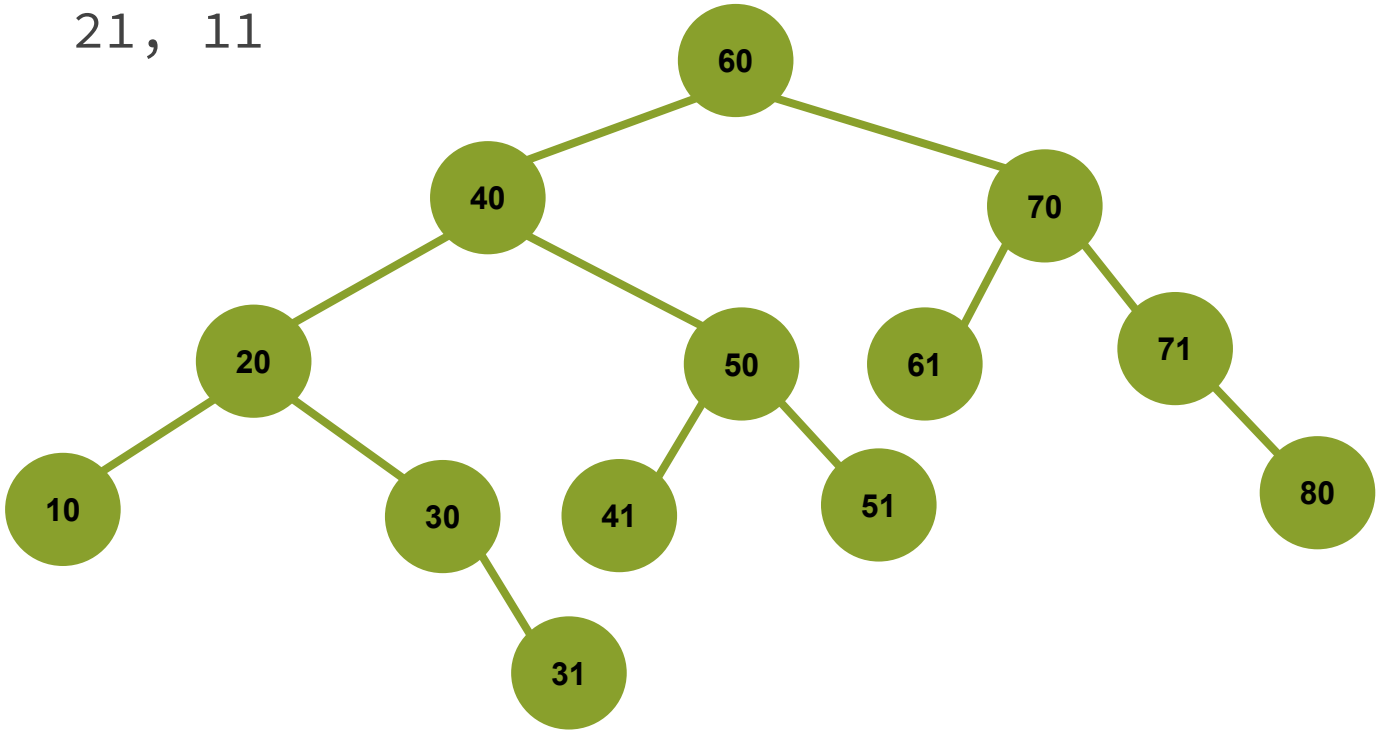
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



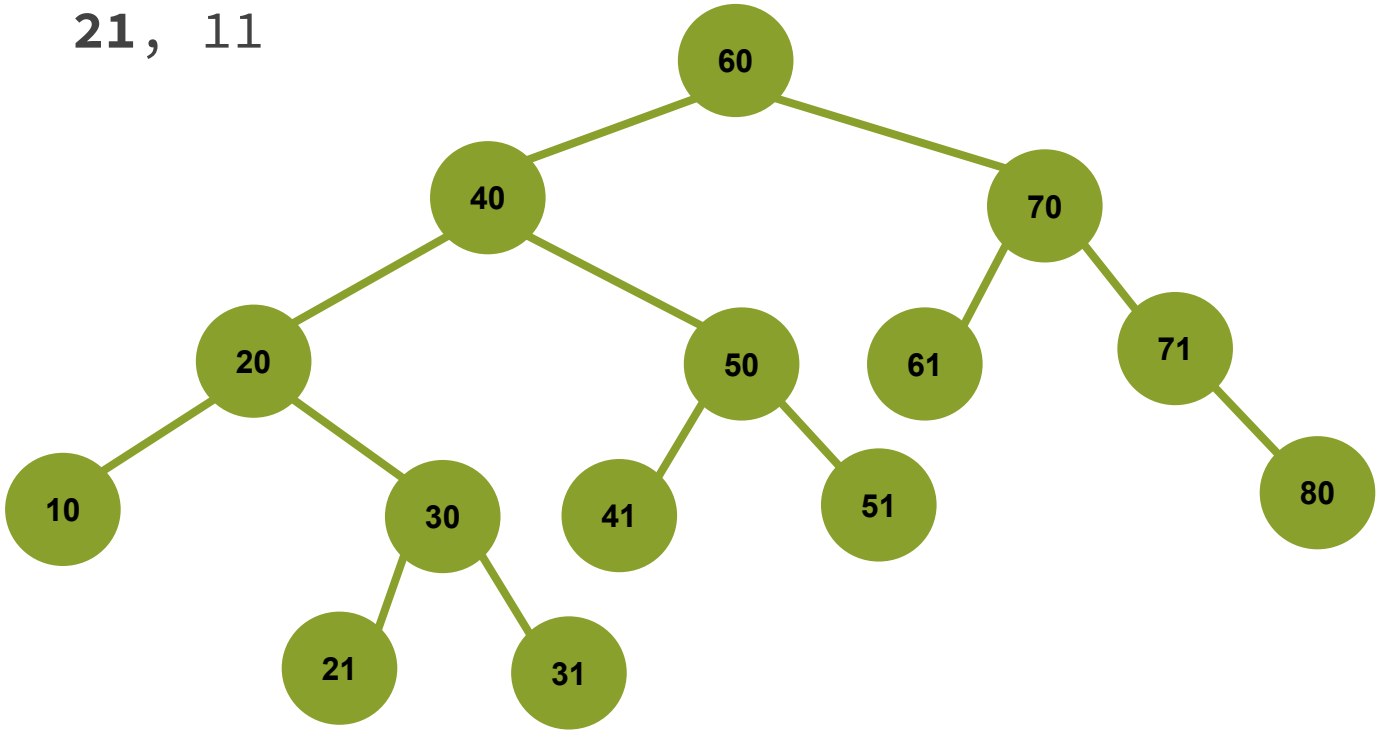
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, **31**, 21, 11



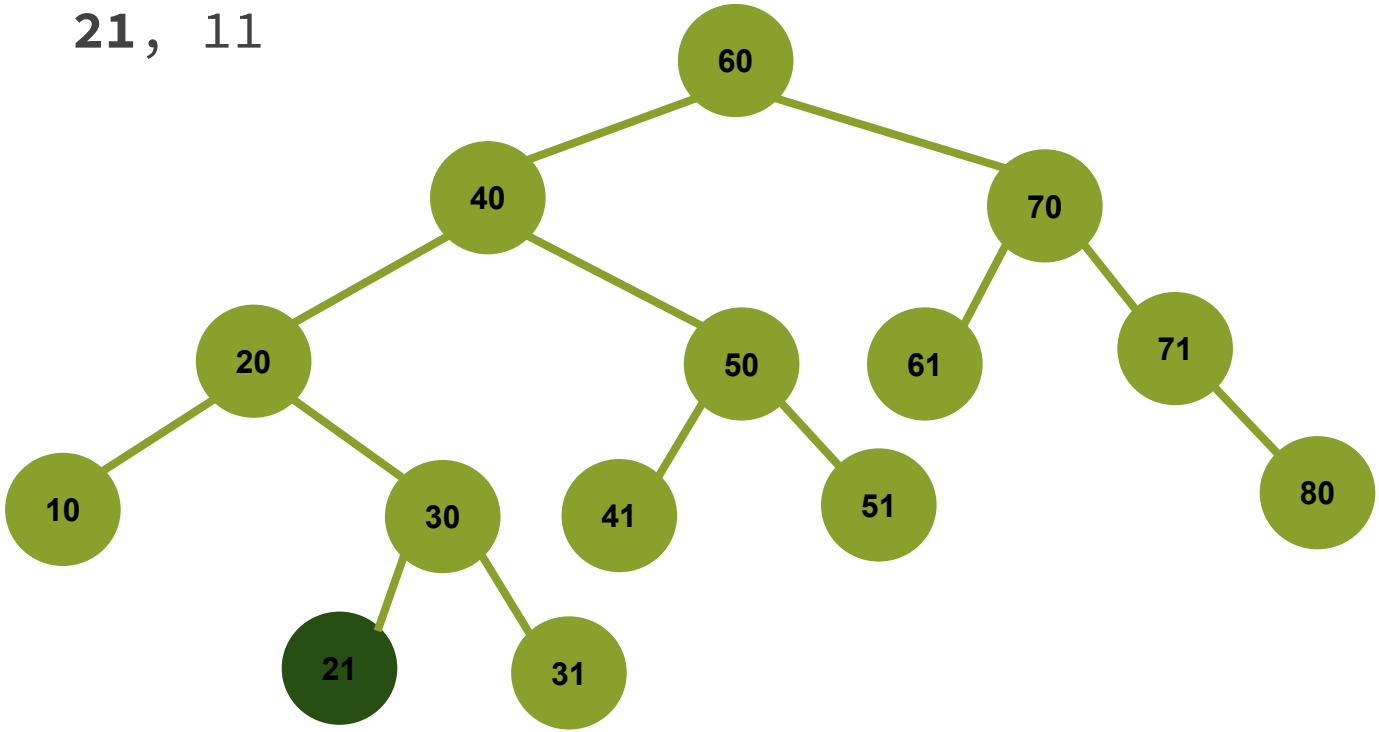
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



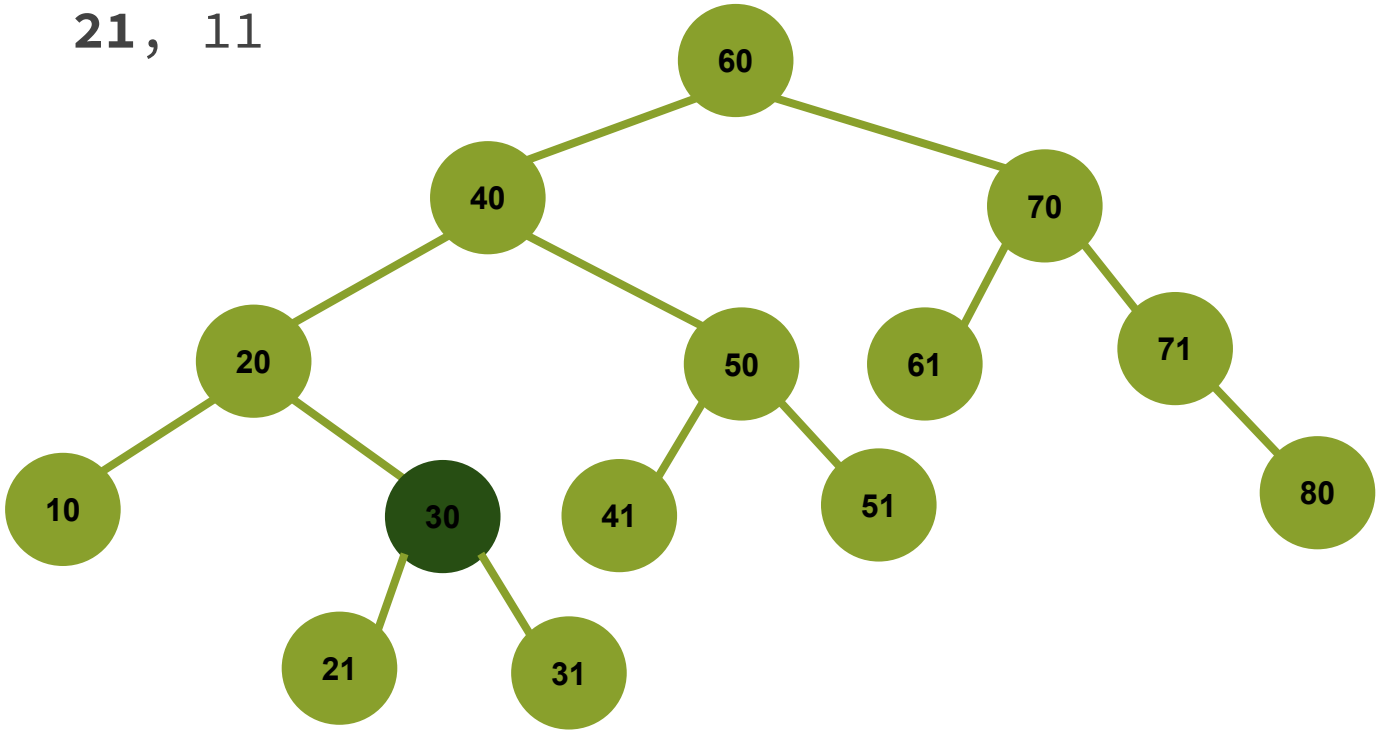
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



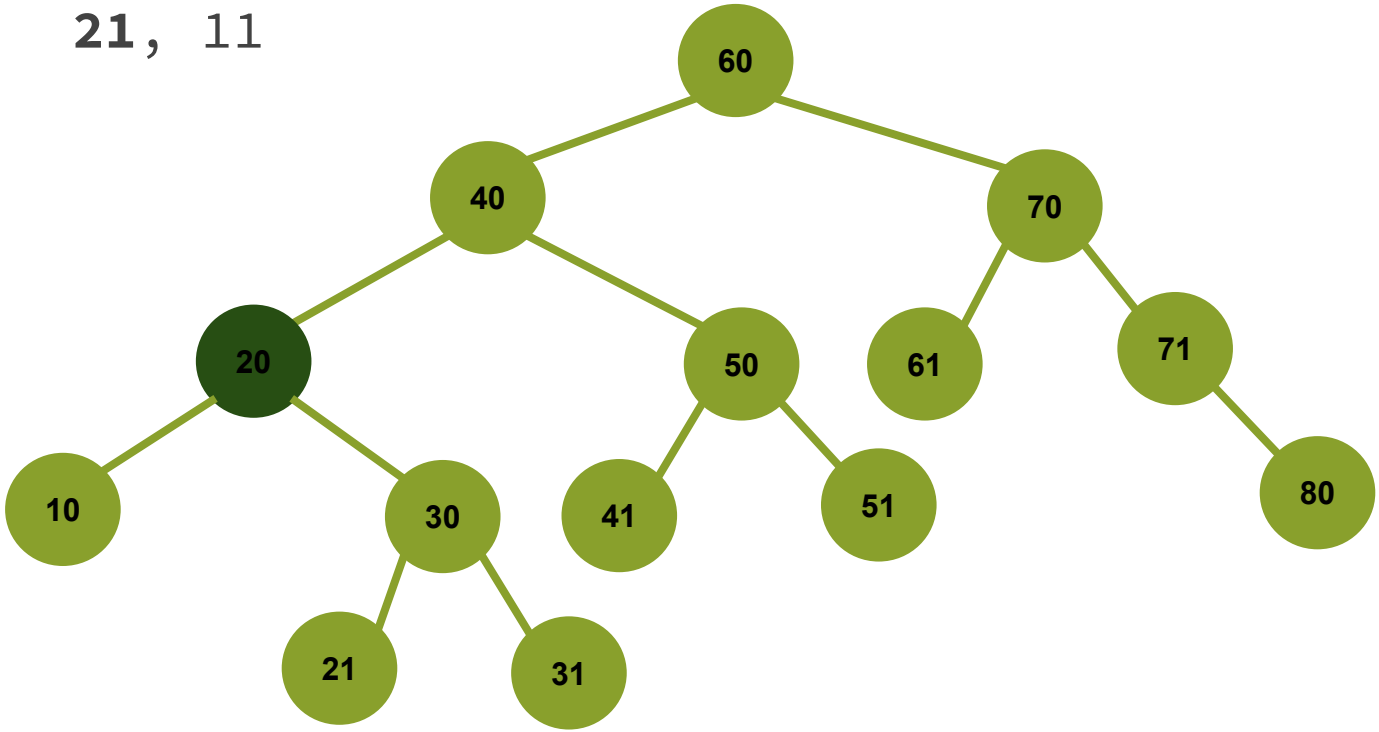
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



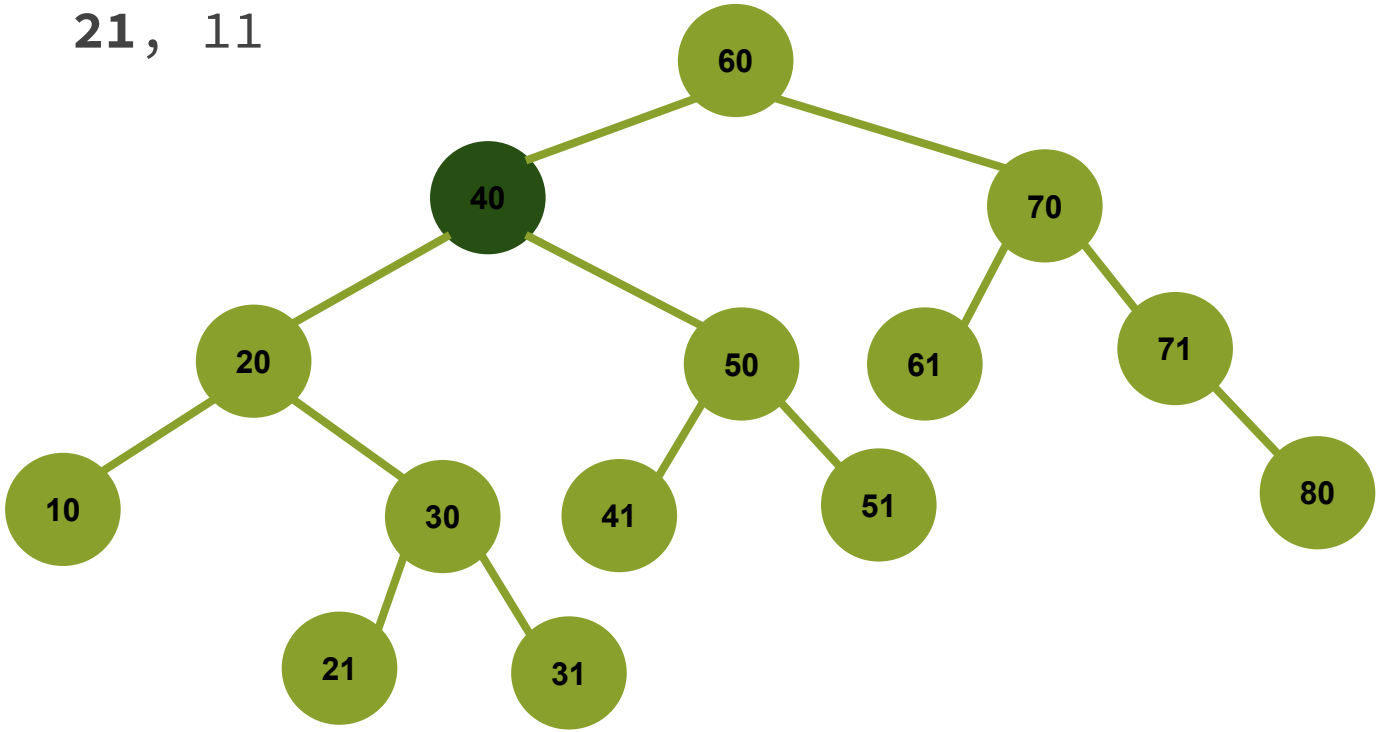
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



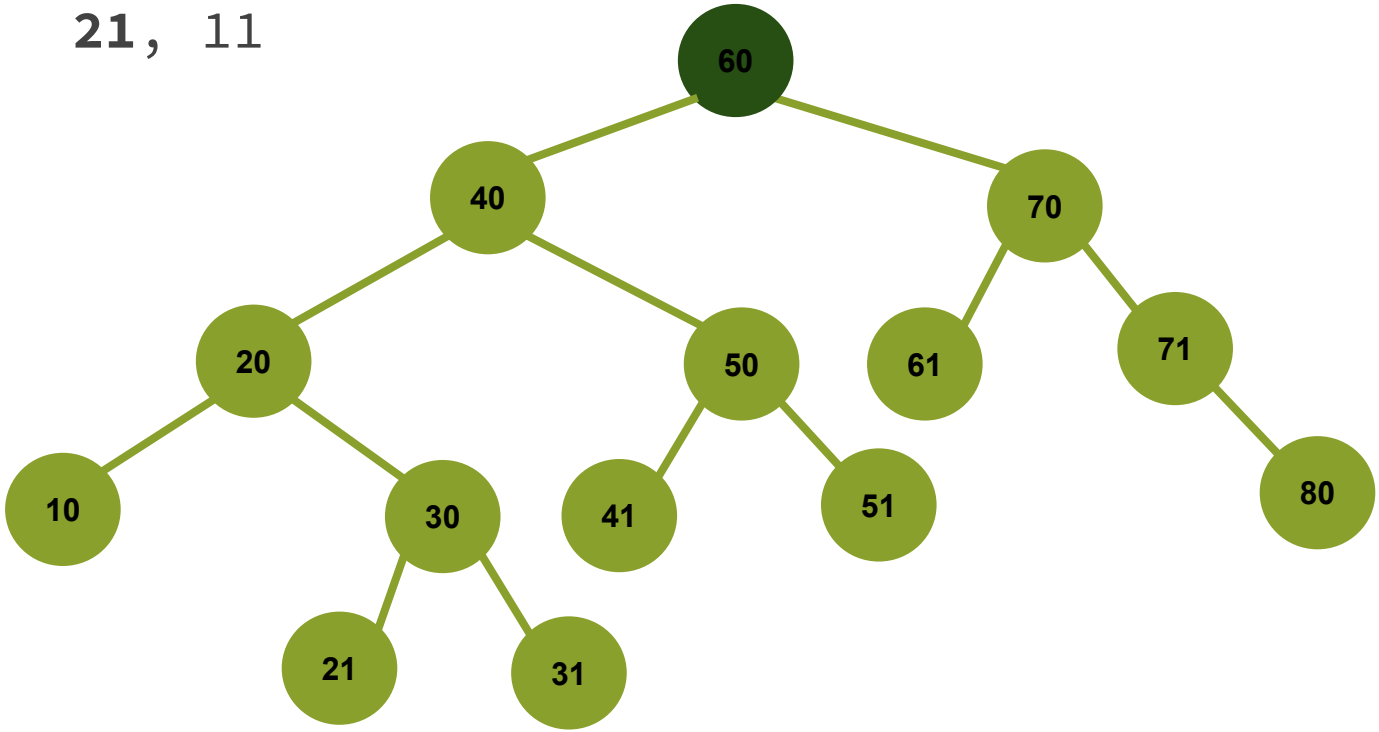
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



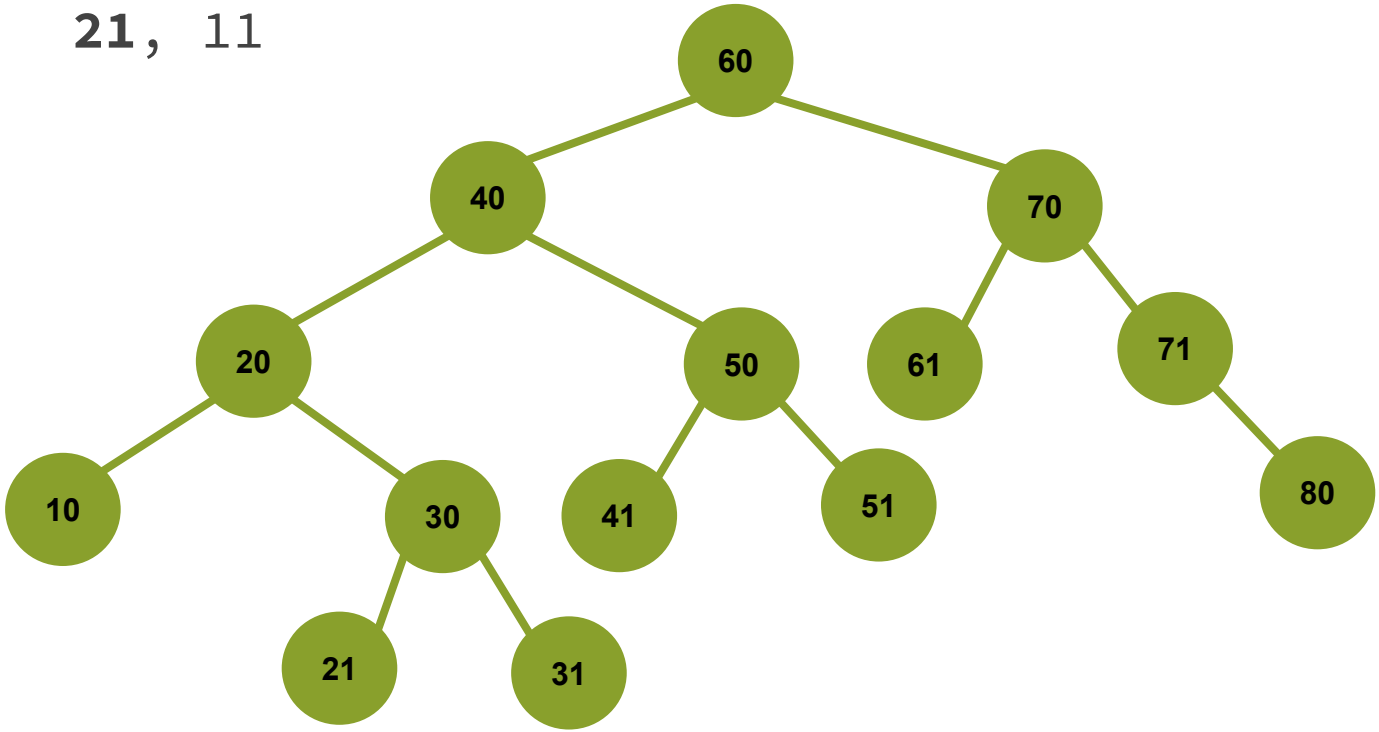
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



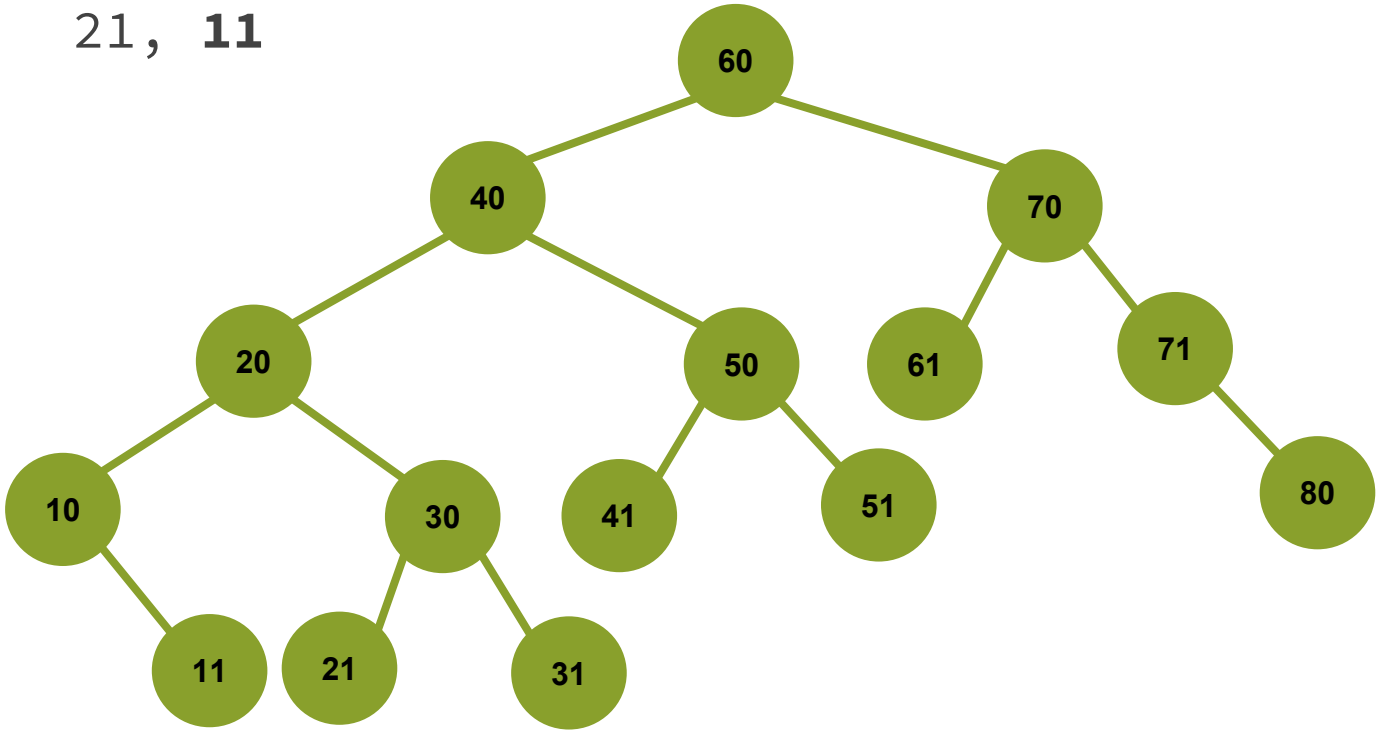
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, **21**, 11



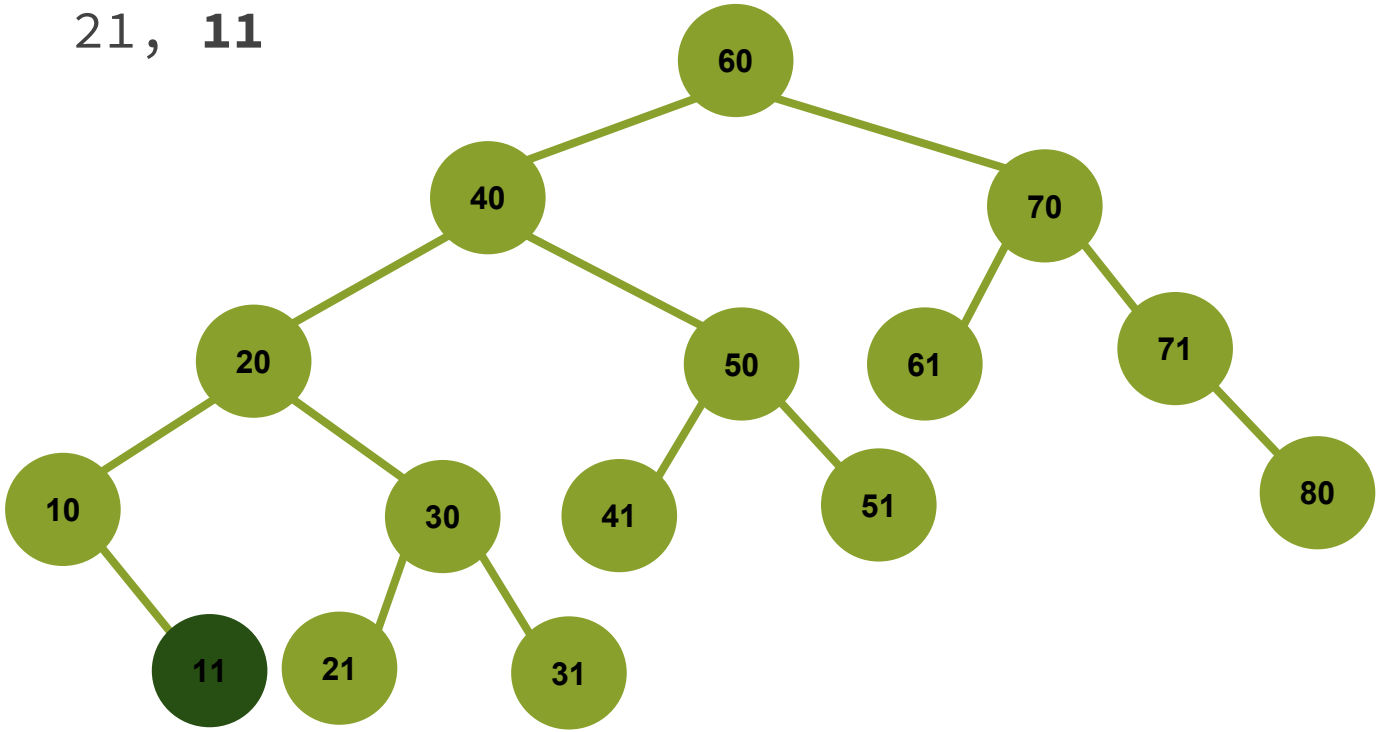
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



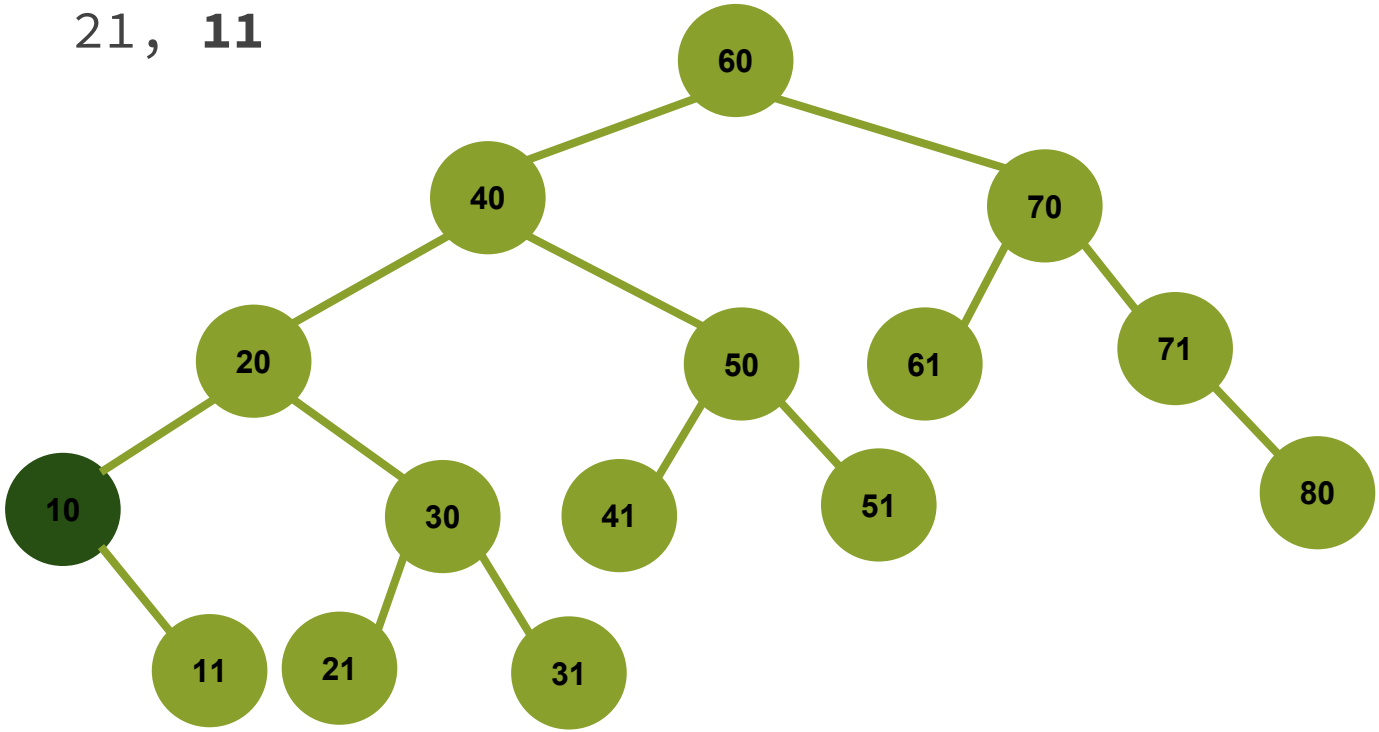
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



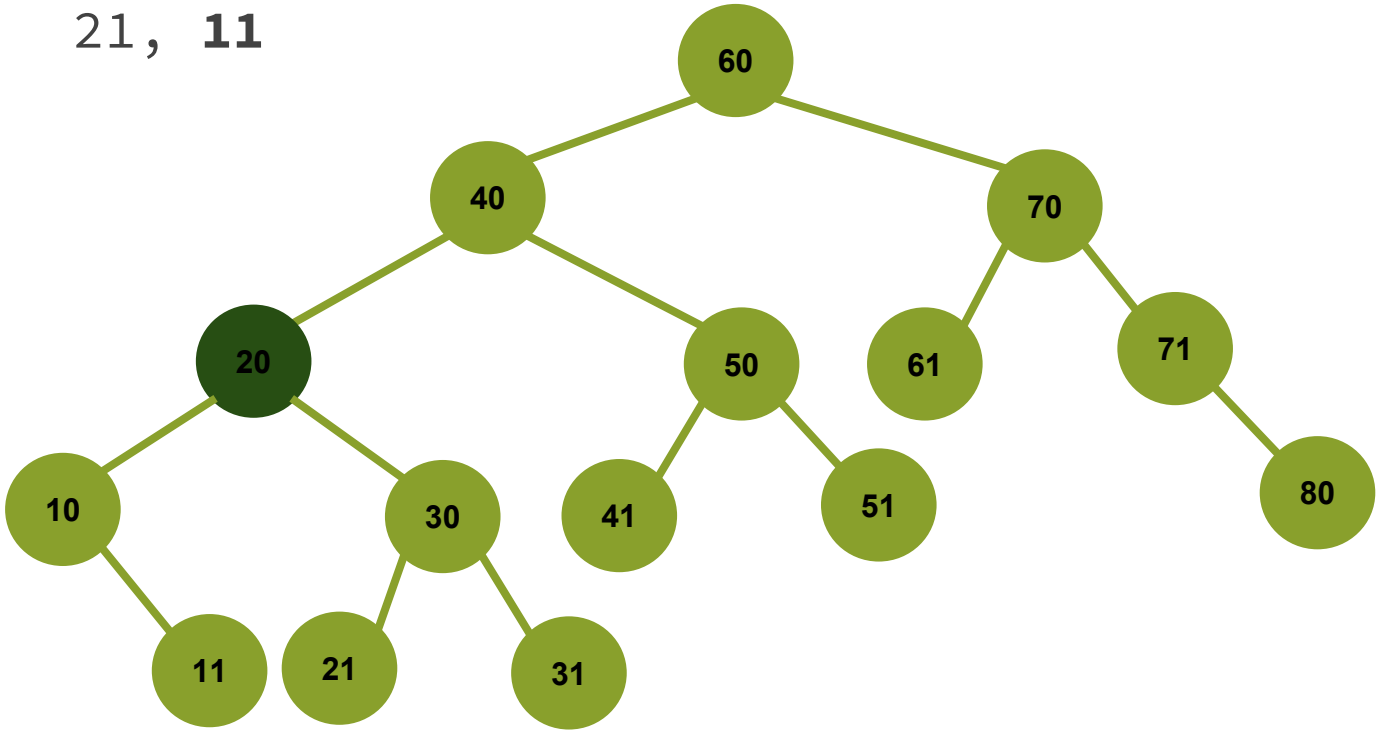
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



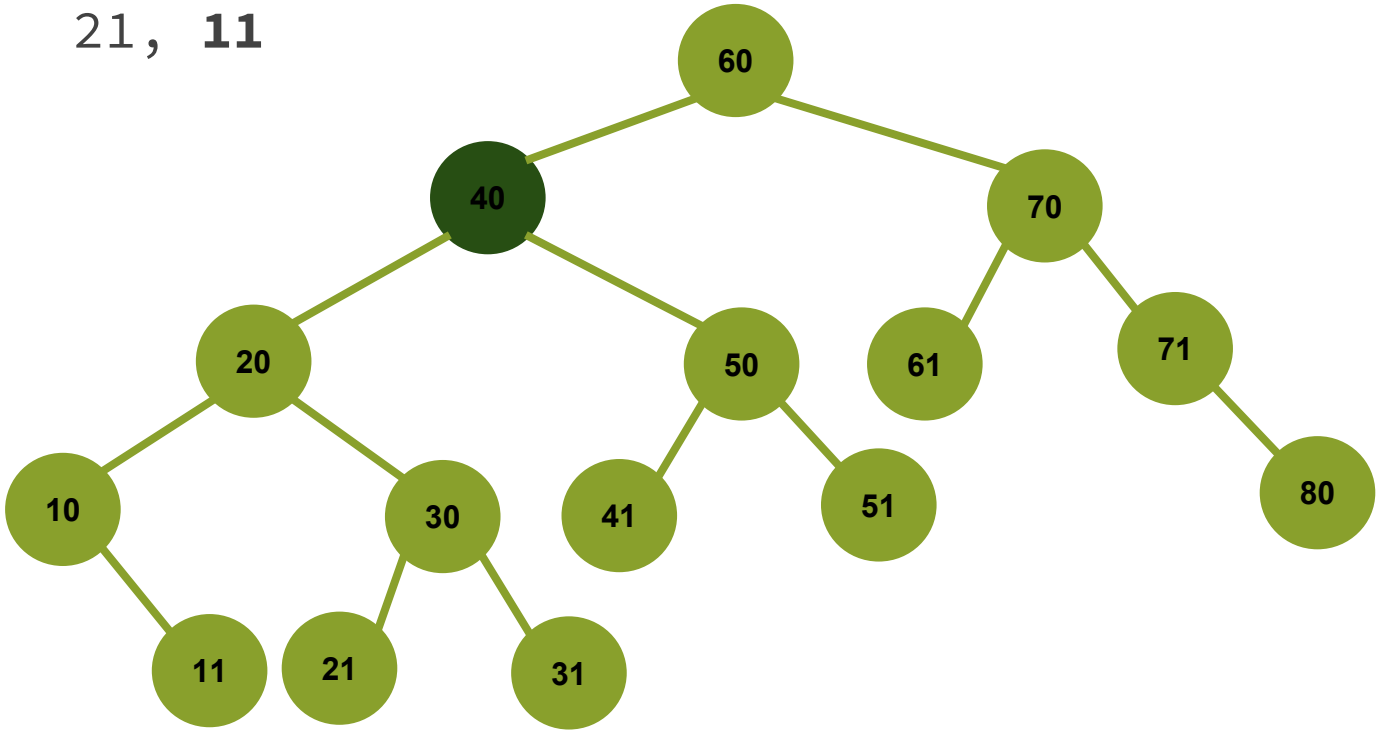
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



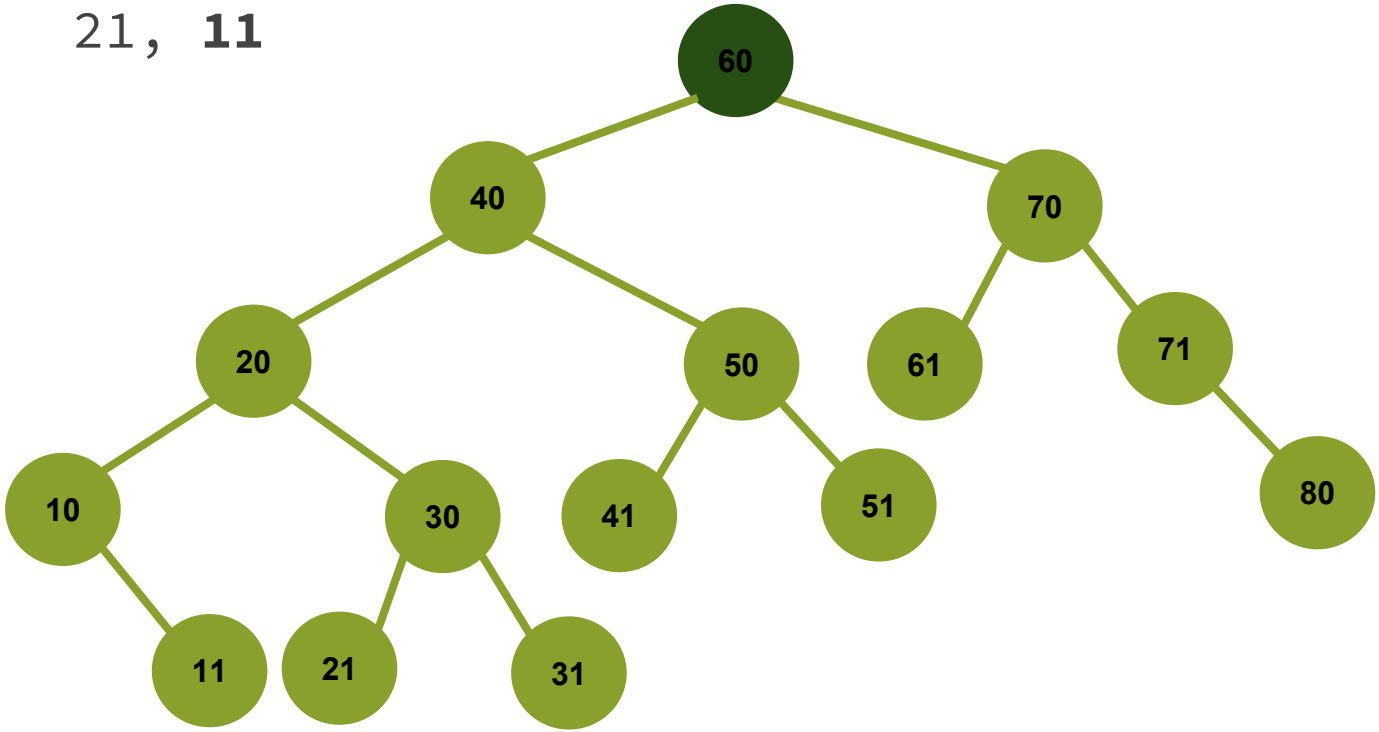
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



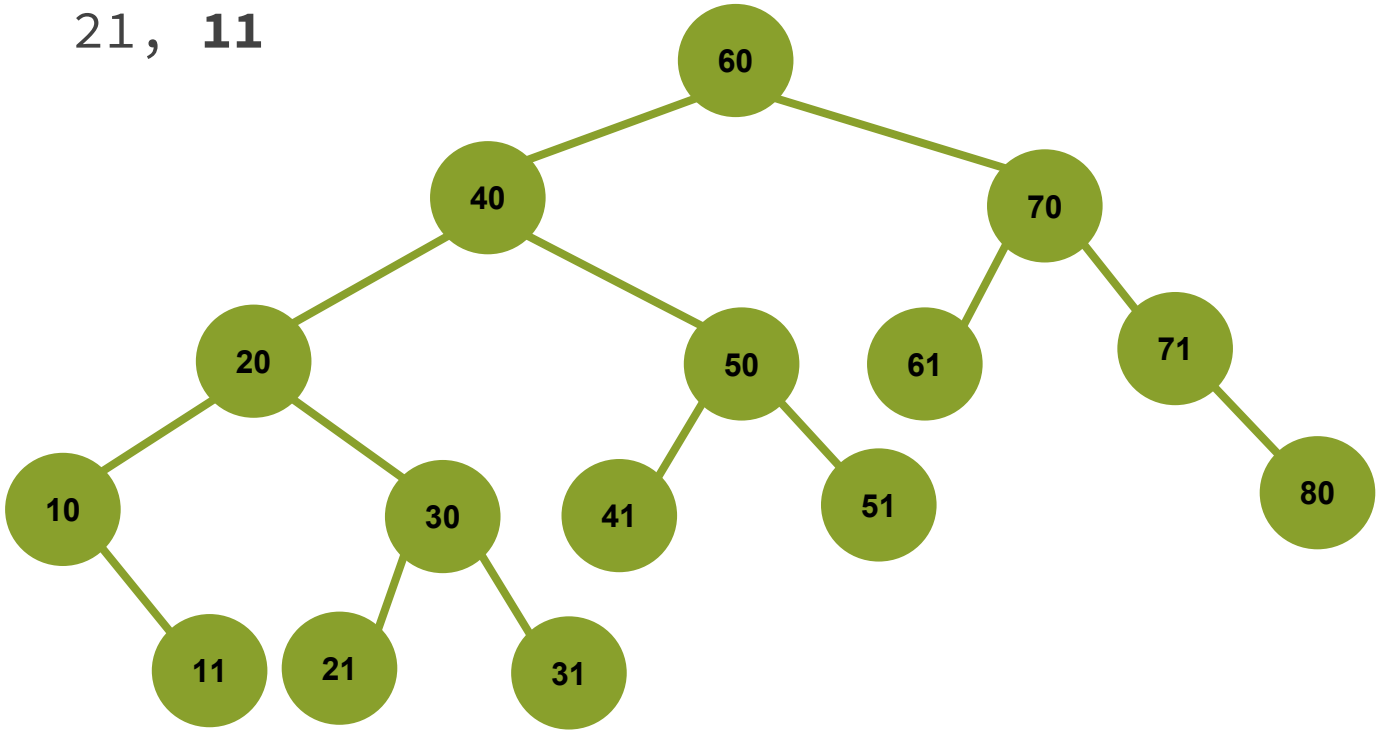
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



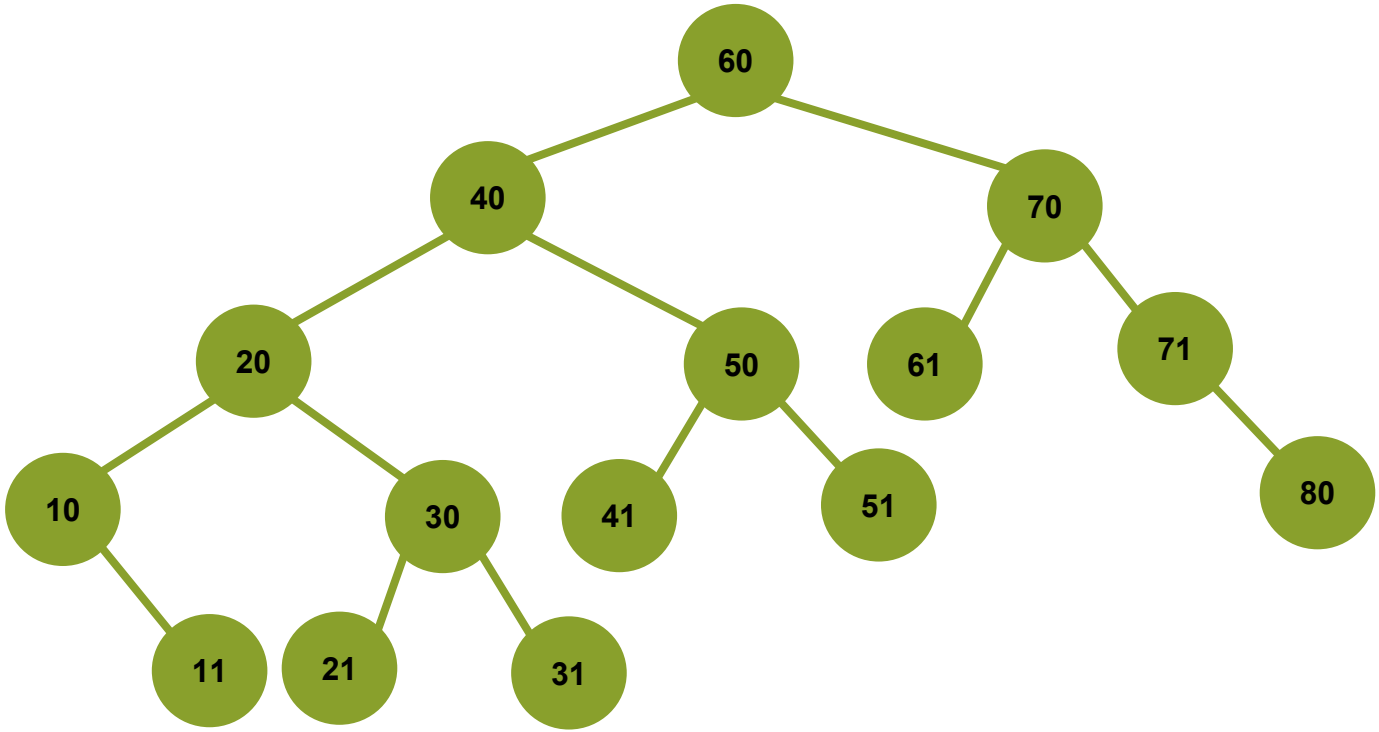
avl tree

10, 20, 30, 40, 50, 60, 70, 80, 71, 61, 51, 41, 31, 21, **11**



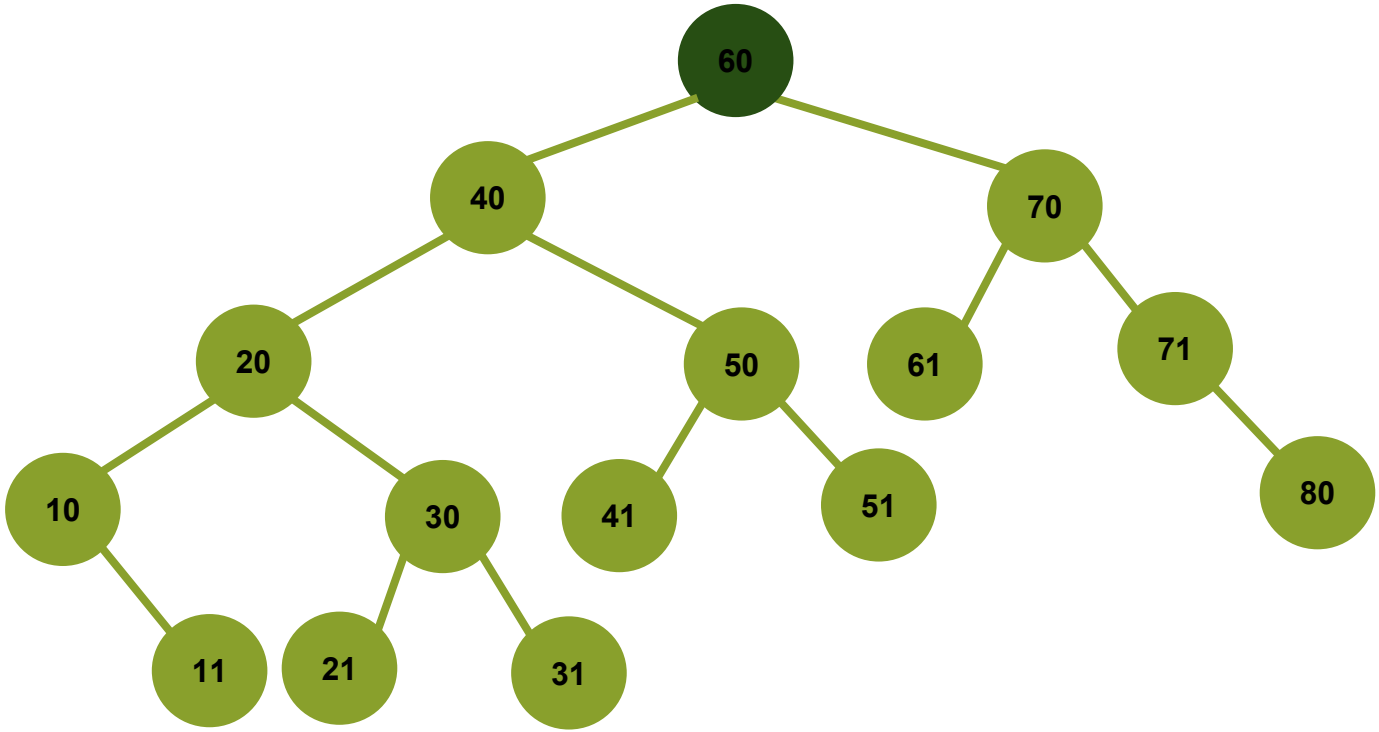
avl tree

delete 80



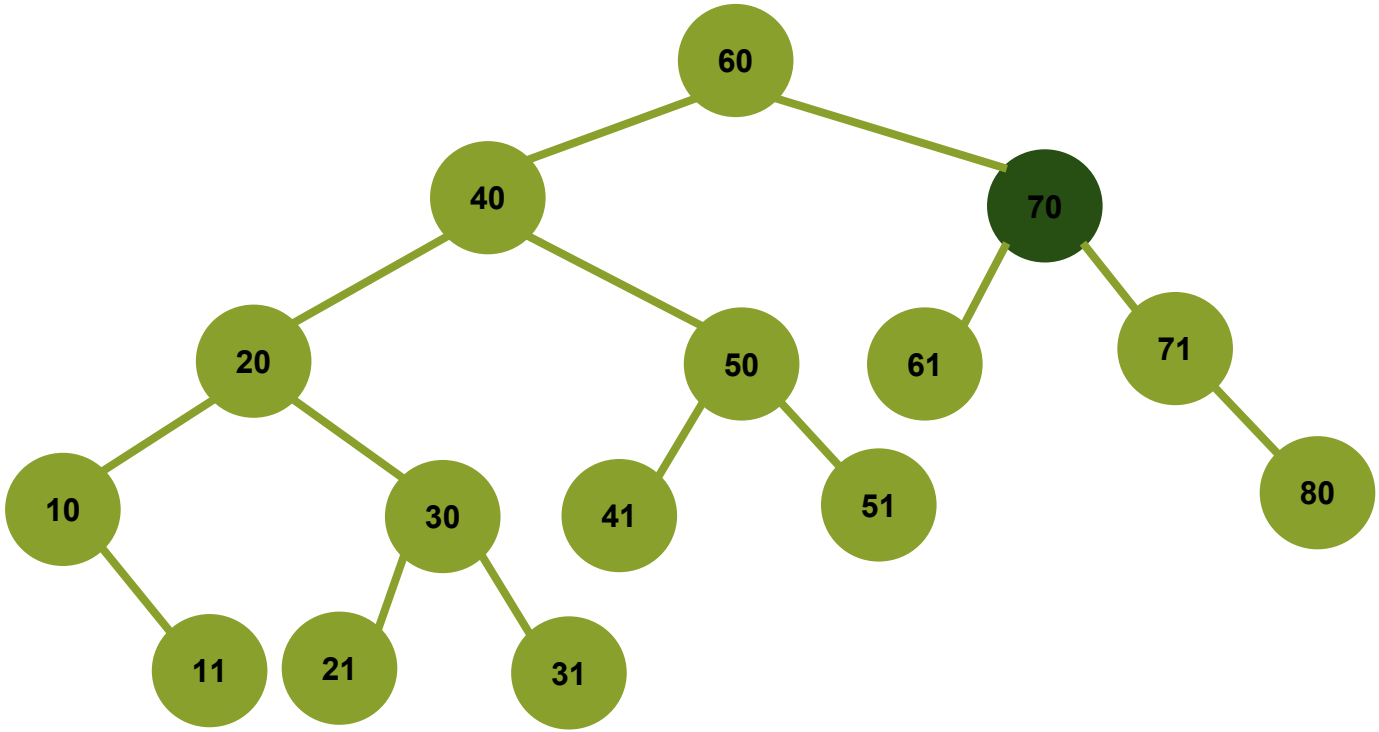
avl tree

delete 80



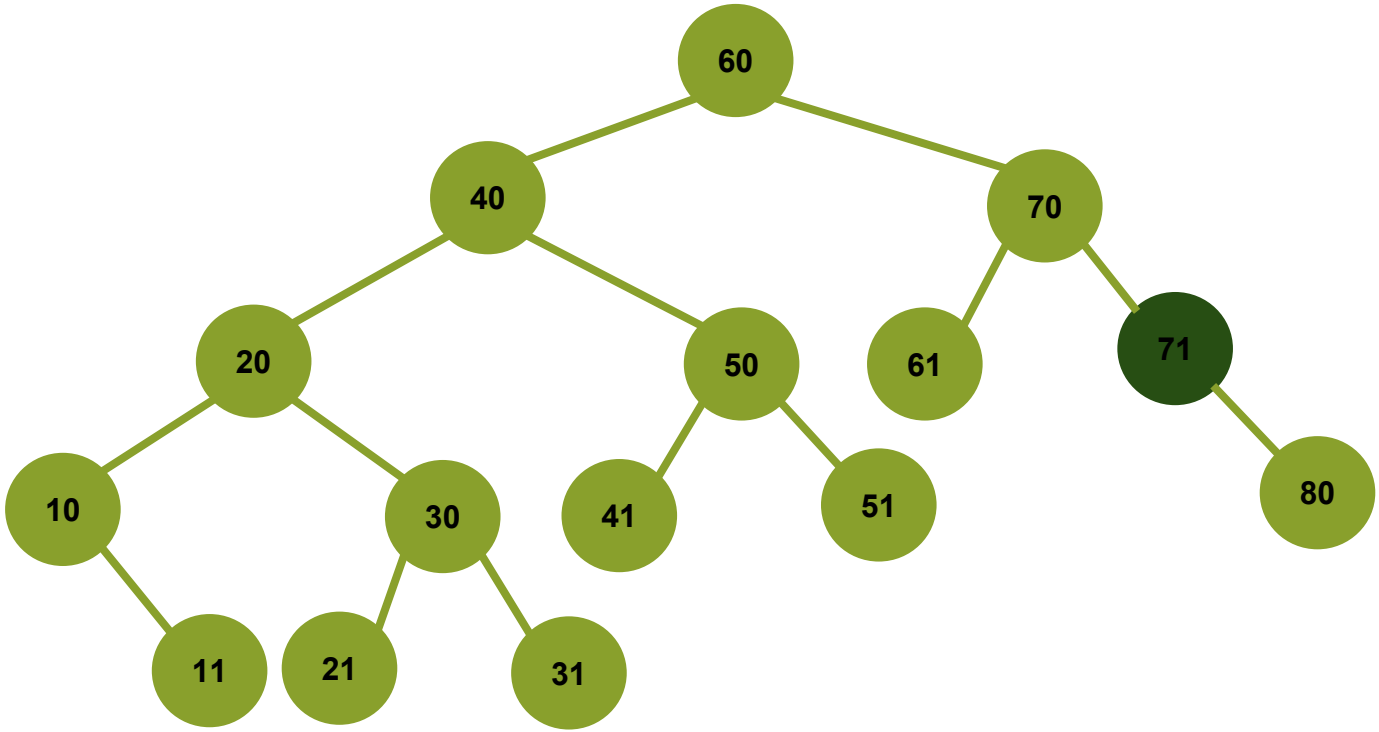
avl tree

delete 80



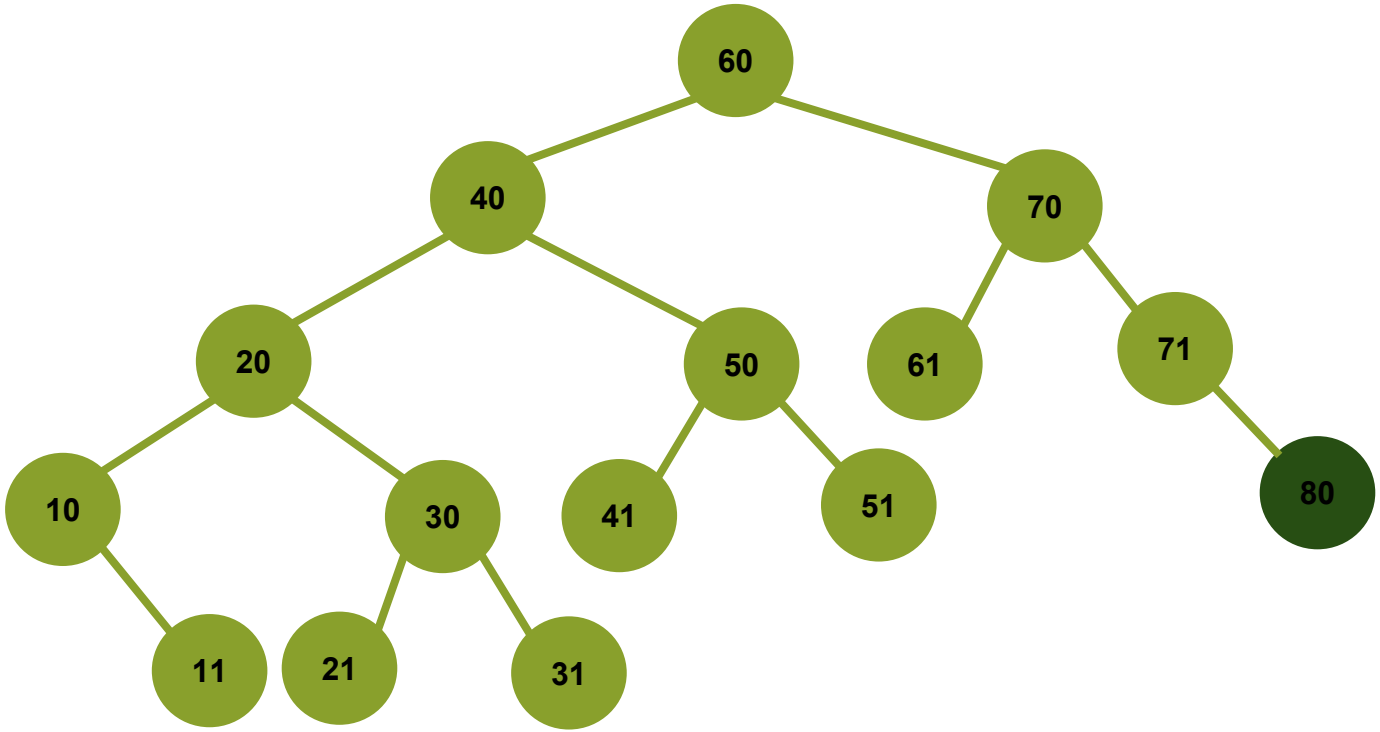
avl tree

delete 80



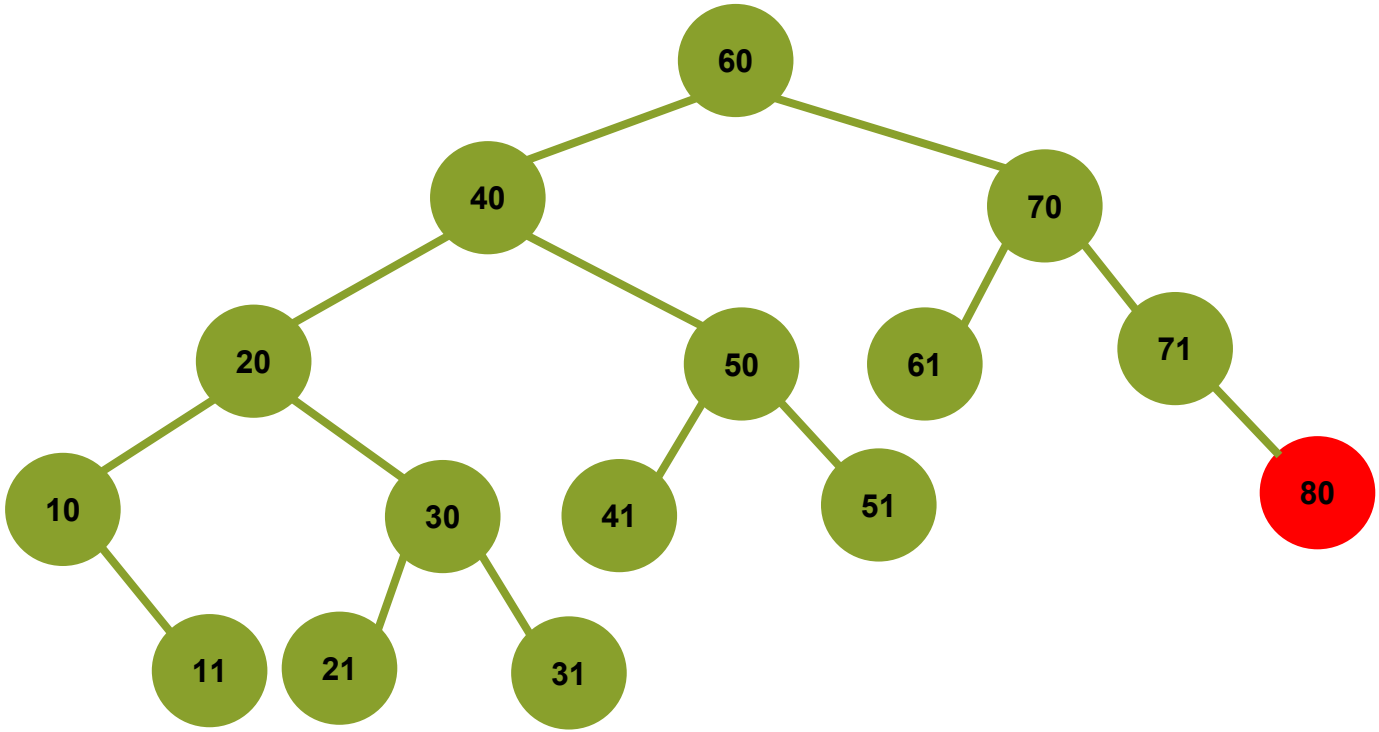
avl tree

delete 80



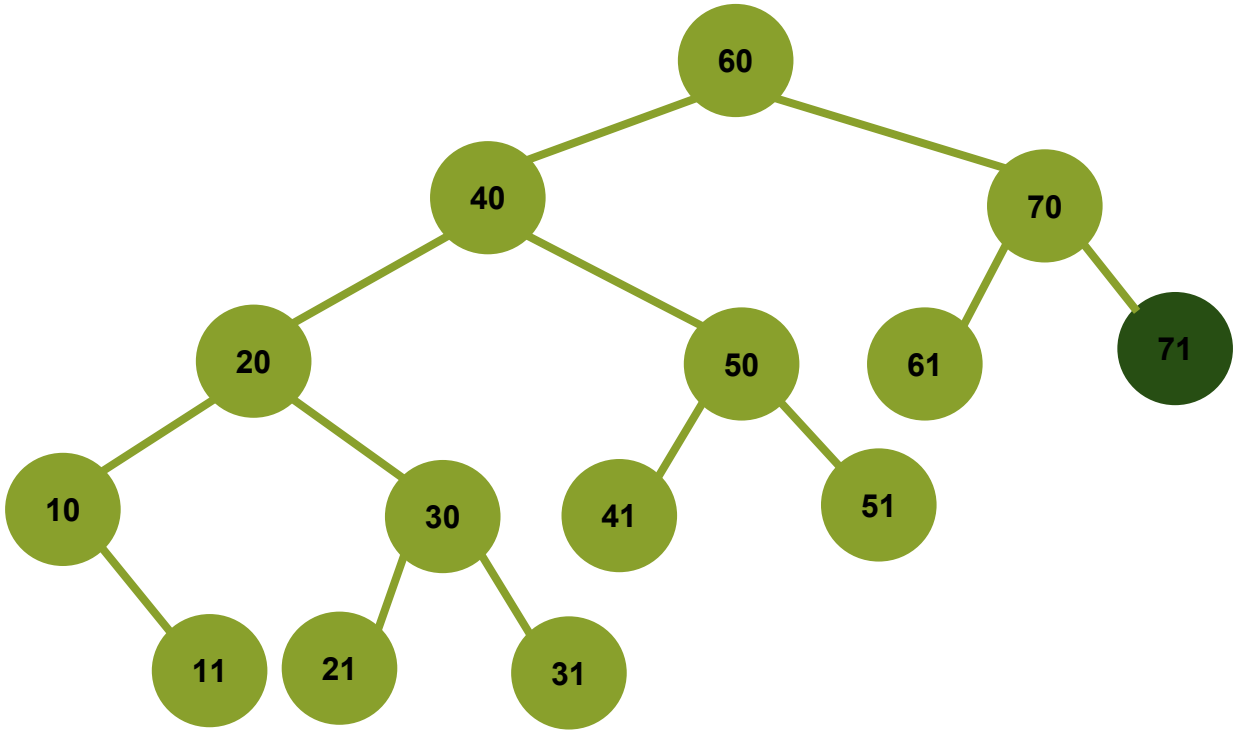
avl tree

delete 80



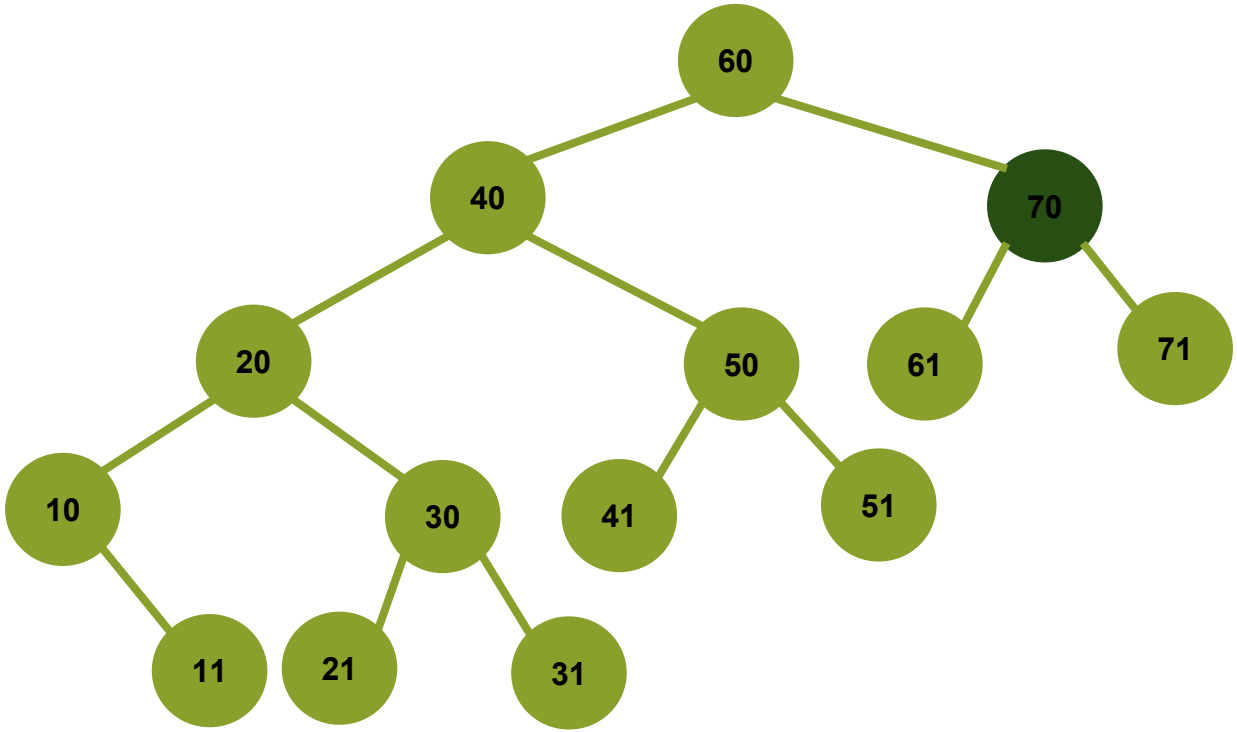
avl tree

delete 80



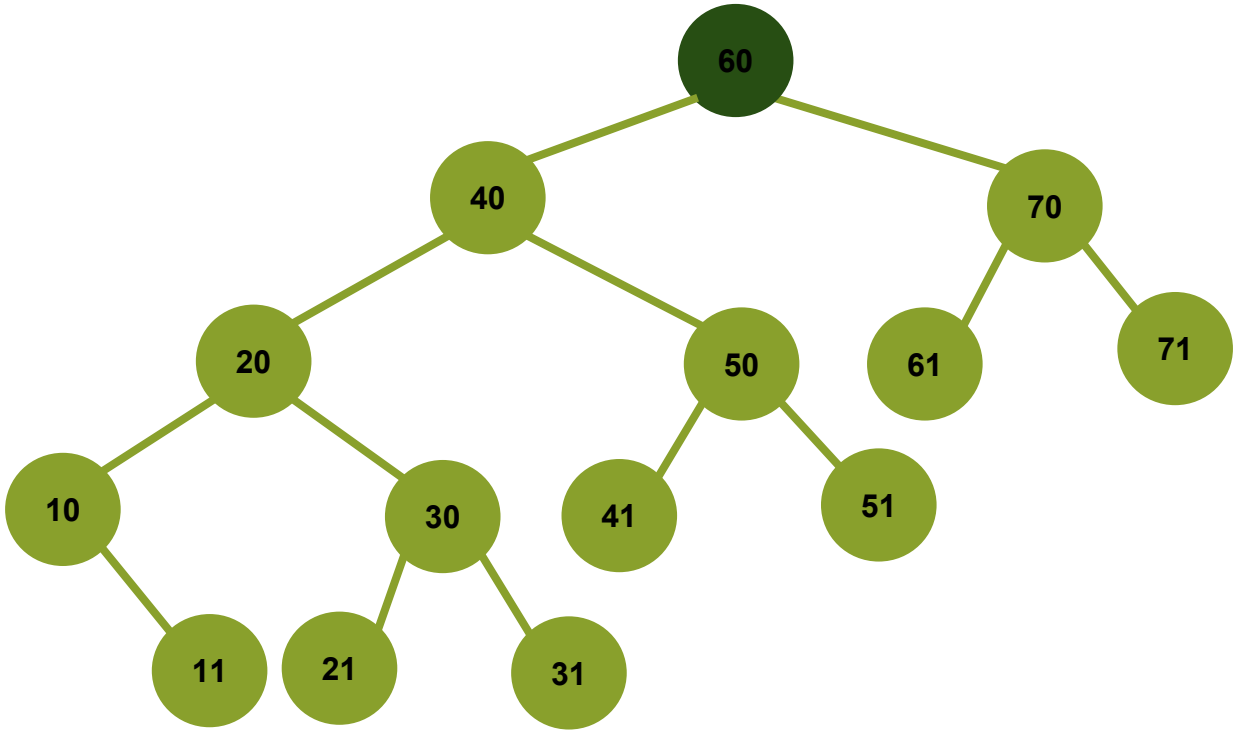
avl tree

delete 80



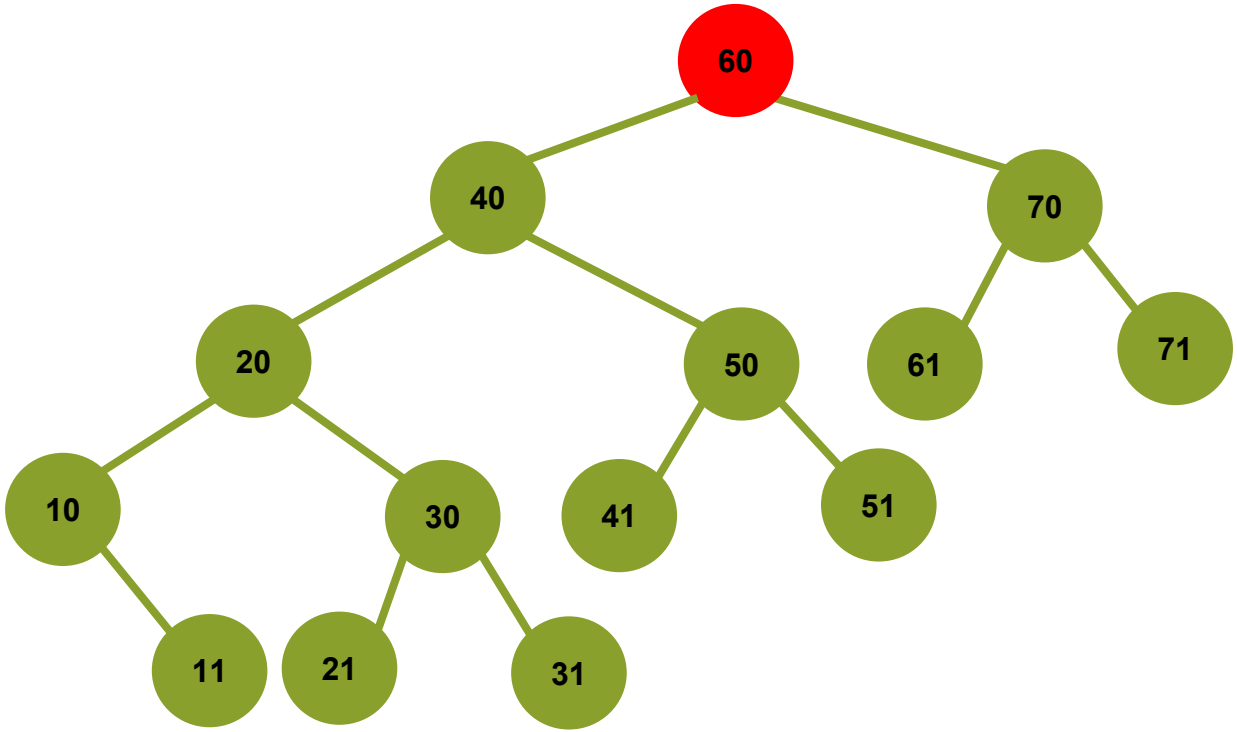
avl tree

delete 80



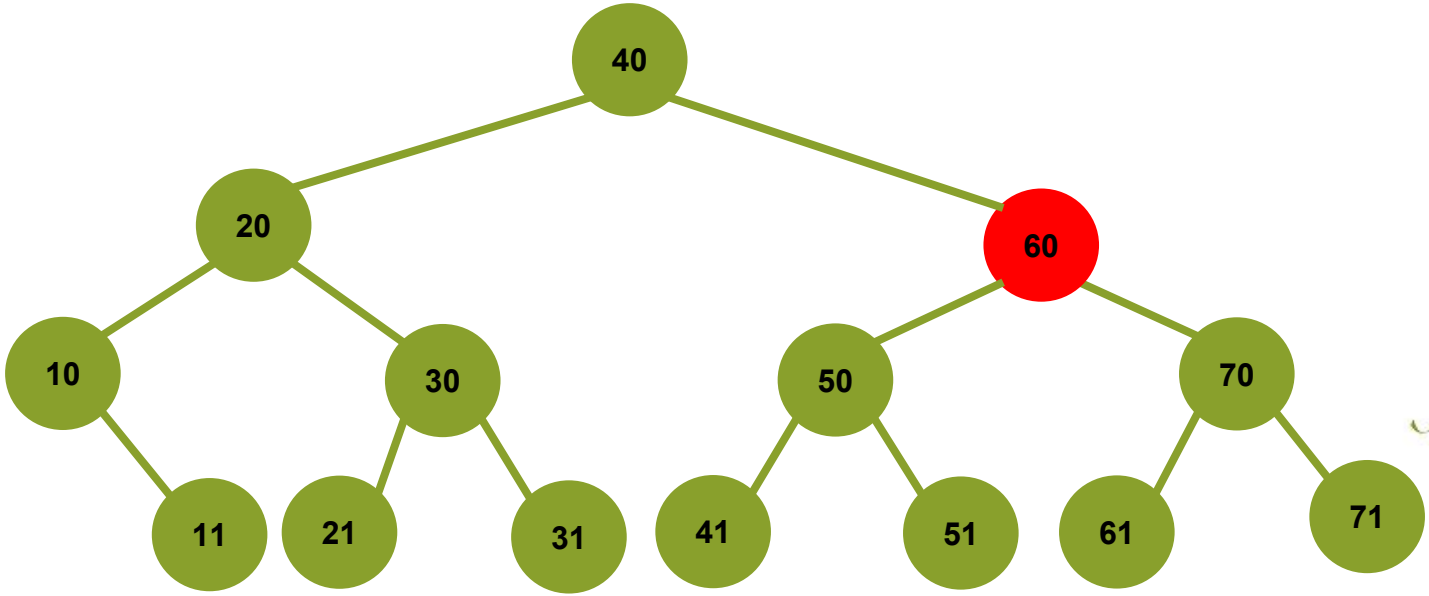
avl tree

delete 80



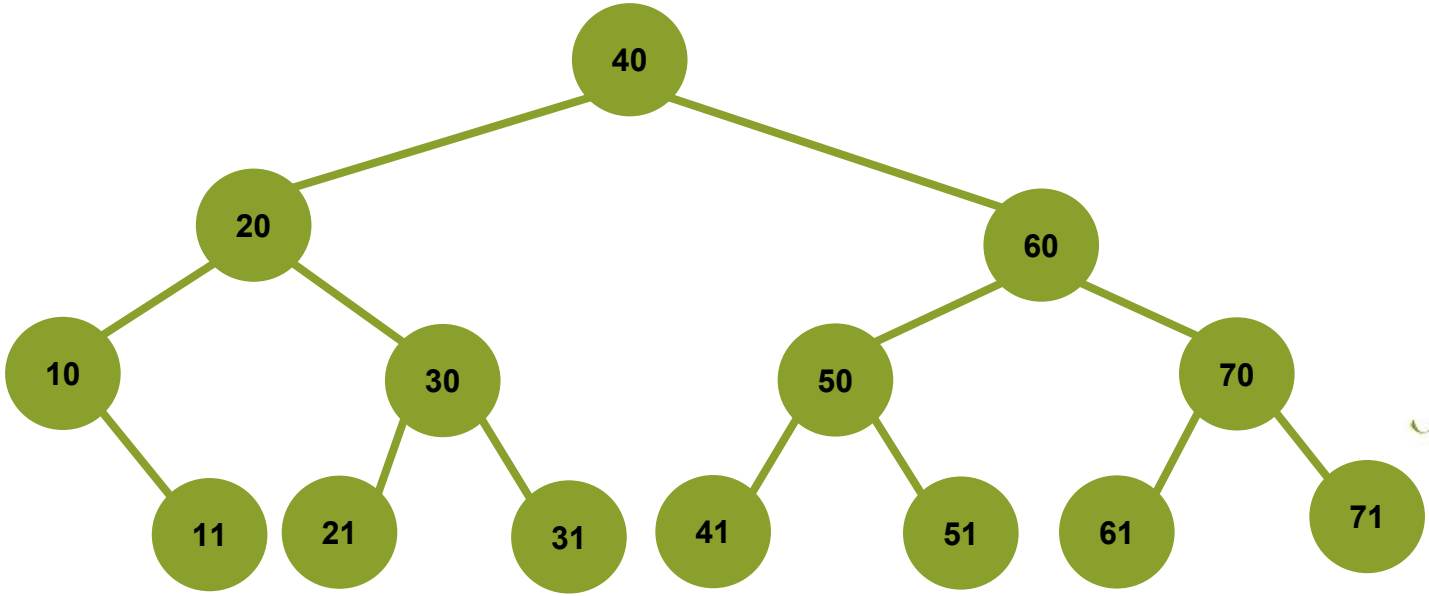
avl tree

delete 80



avl tree

delete 80

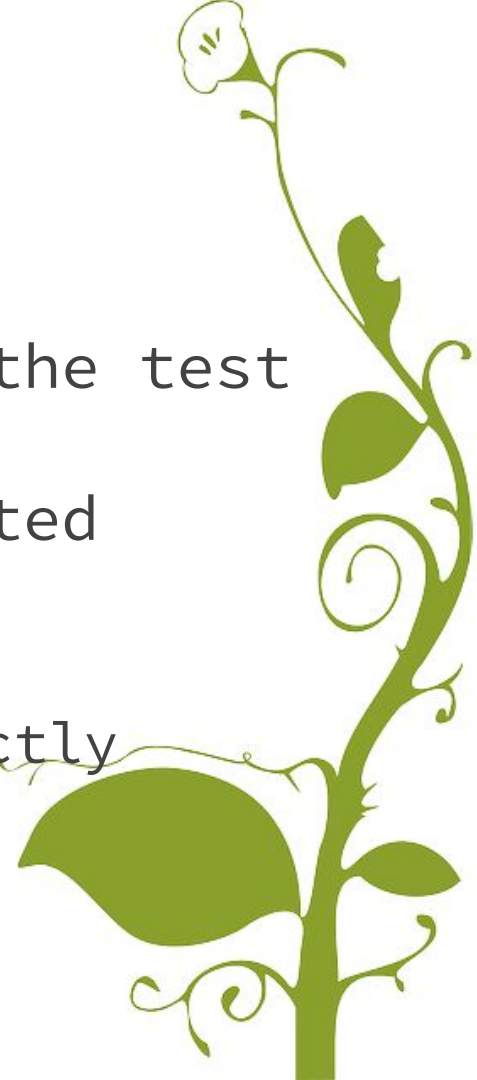


quiz - ¼ sheet of paper (5 minutes)



Notes on the remove MP of bst

- the remove operation worked for the test cases
- when the destructor was implemented suddenly segmentation faults are encountered
  - remove was not implemented correctly



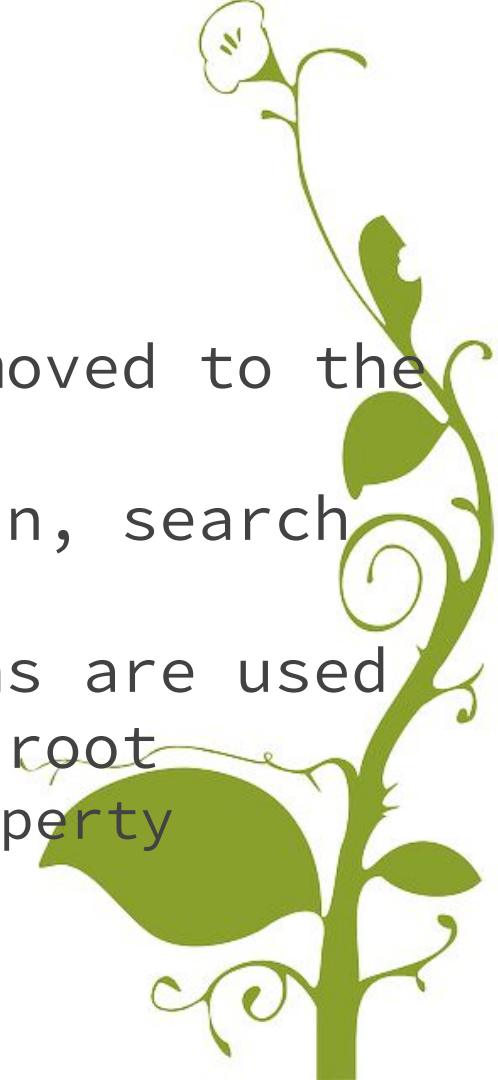
splay tree

- when a search is made on a bst
  - most of the time the element is in the tree
  - sometimes, only a few of the keys are accessed more frequently than other keys
  - it is best that the most frequently accessed keys are placed near the top of the tree
  - splay tree
    - self-adjusting binary search tree



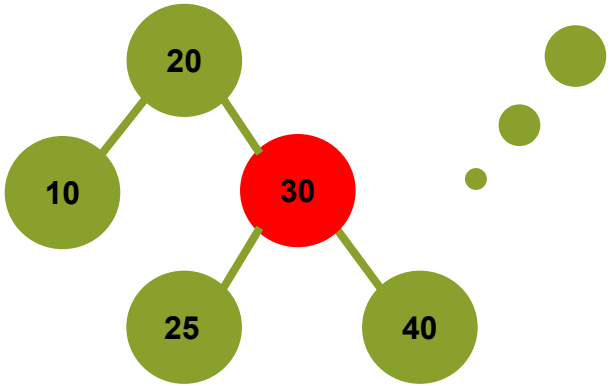
splay tree - search

- when a node is accessed, it is moved to the top/root of the tree
- so that when it is accessed again, search is kept minimal
- very much like the AVL, rotations are used to move the desired node to the root
  - rotations keep the bst order property
  - this is called splaying

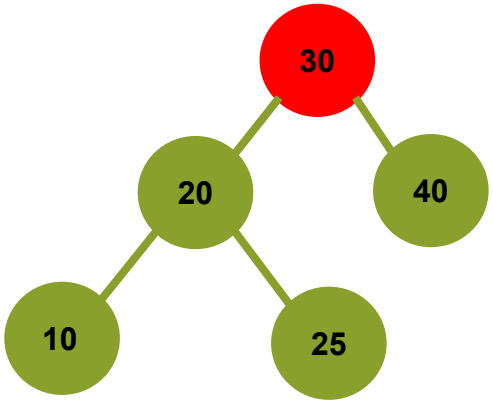


splay tree - zig

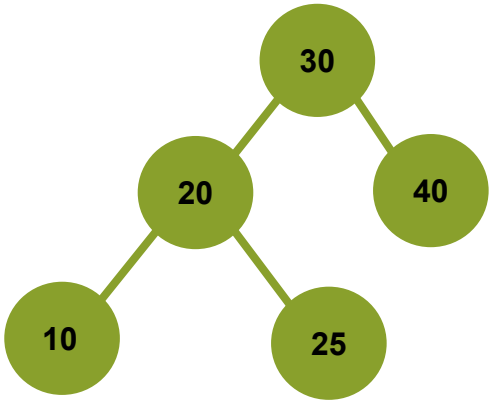
parent of desired node is the root



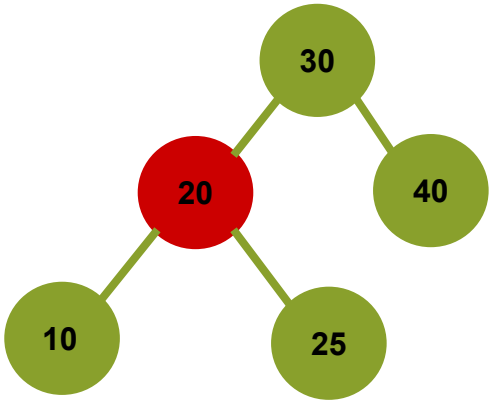
splay tree - zig



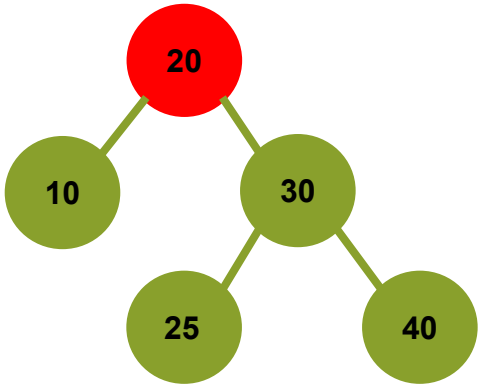
splay tree - zig



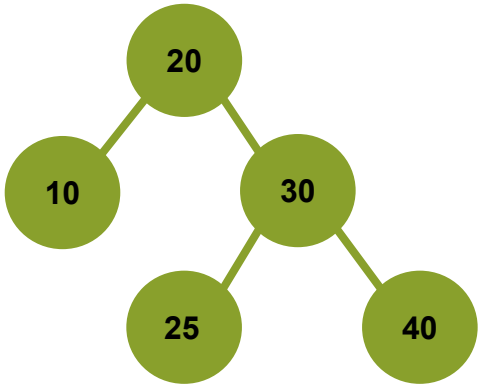
splay tree - zig



splay tree - zig

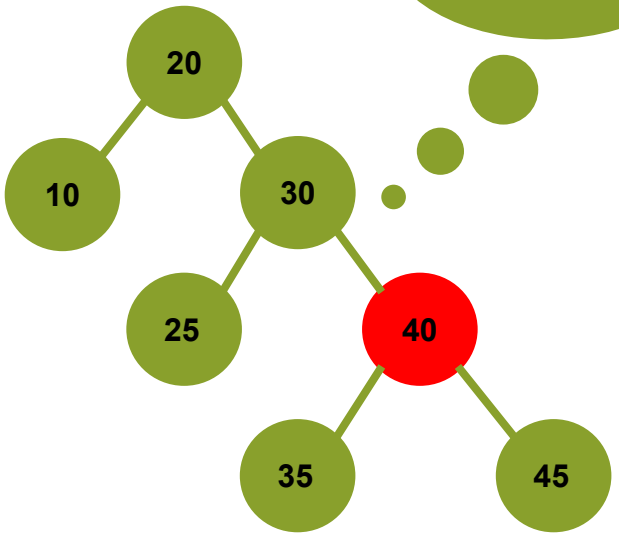


splay tree - zig (single rotation)

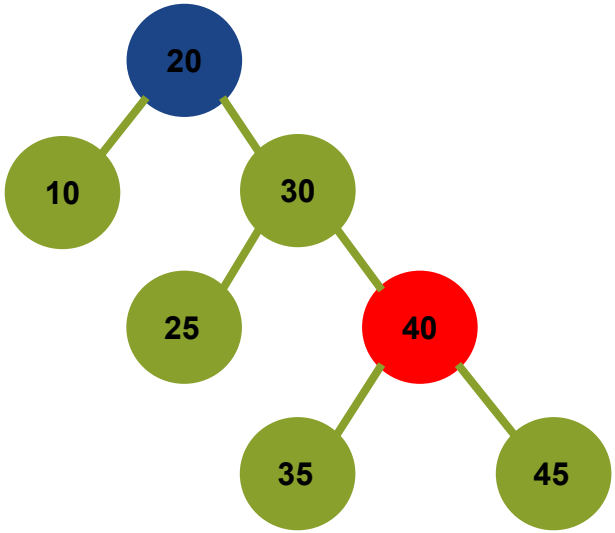


splay tree - zig-zig

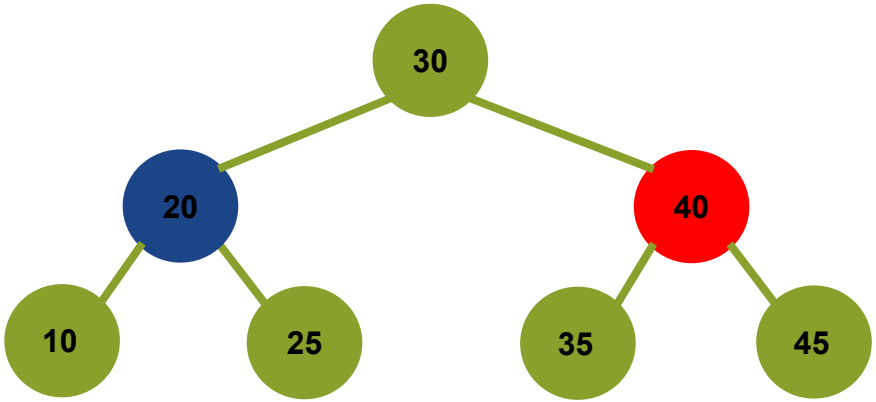
parent of desired node is not the root and parent and grandparent are both right (or left) children



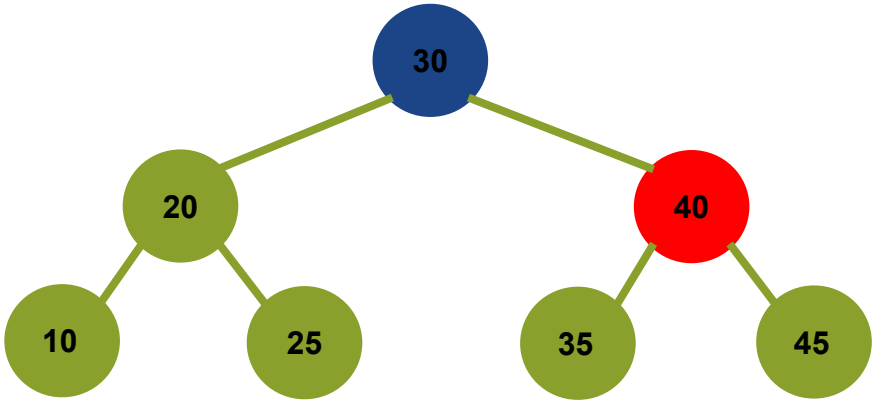
splay tree - zig-zig



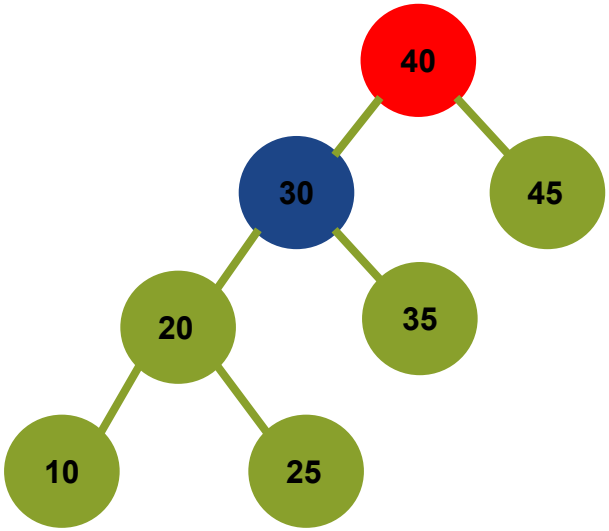
splay tree - zig-zig



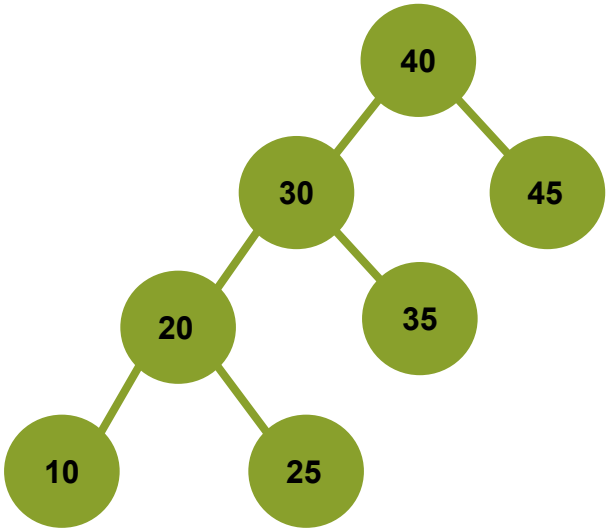
splay tree - zig-zig



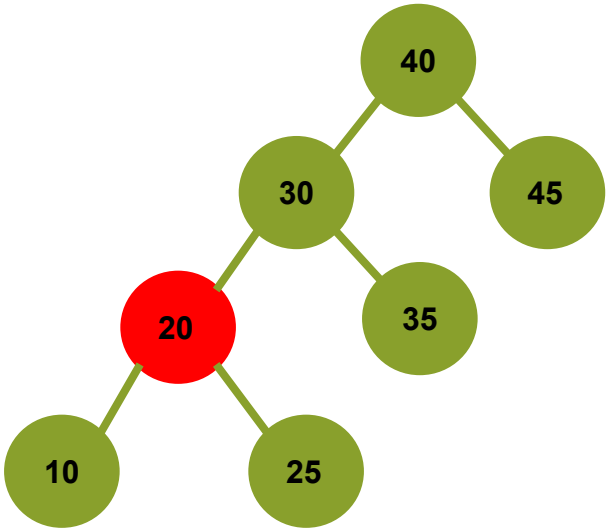
splay tree - zig-zig



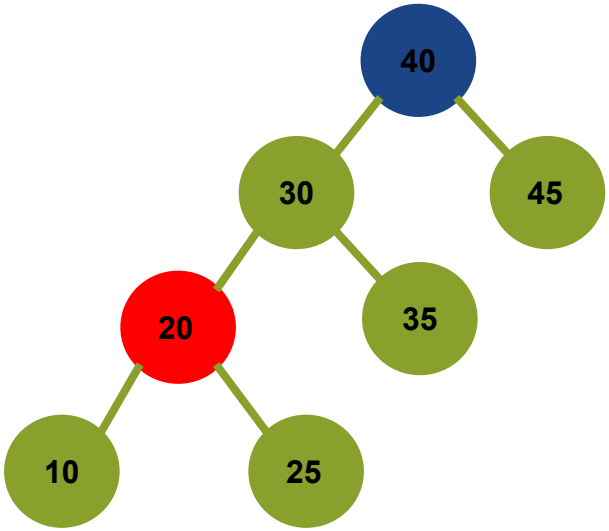
splay tree - zig-zig (double rotation)



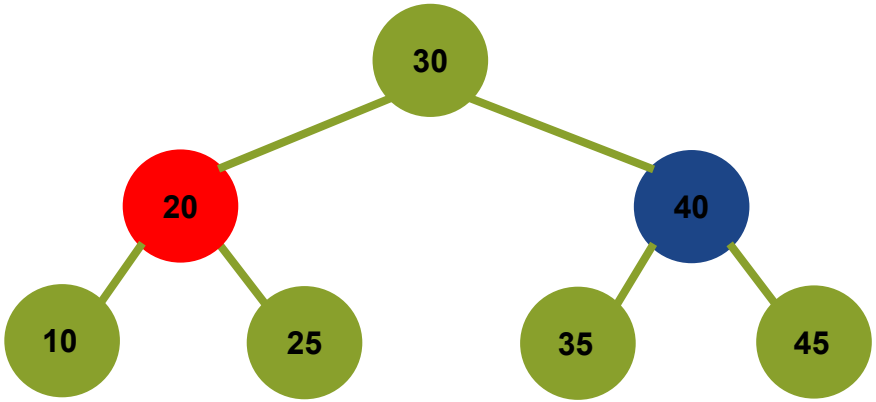
splay tree - zig-zig (double rotation)



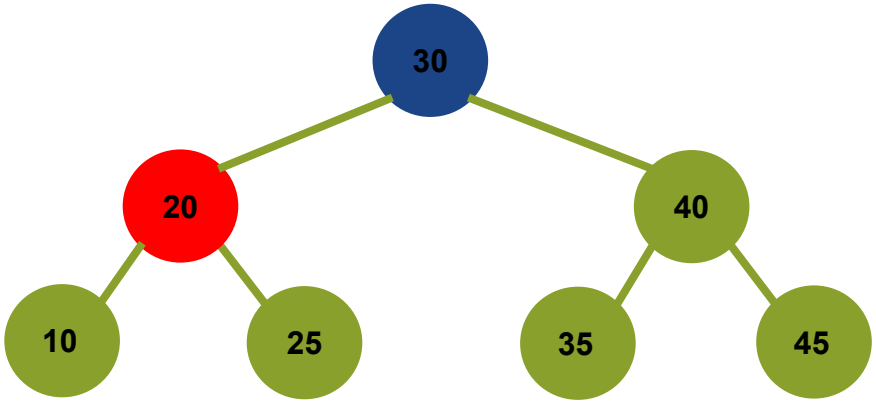
splay tree - zig-zig (double rotation)



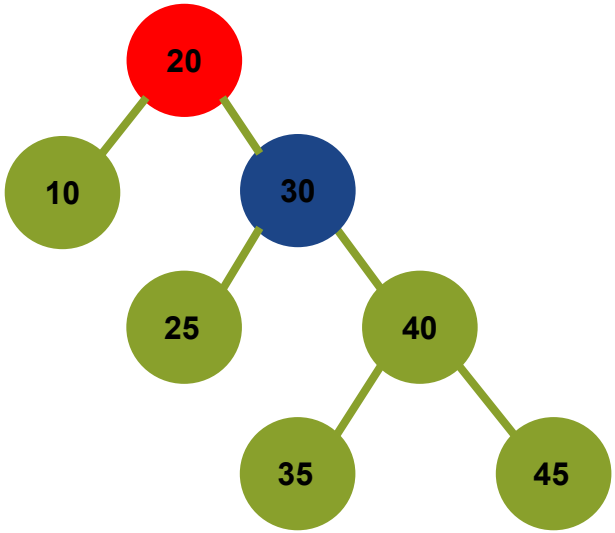
splay tree - zig-zig (double rotation)



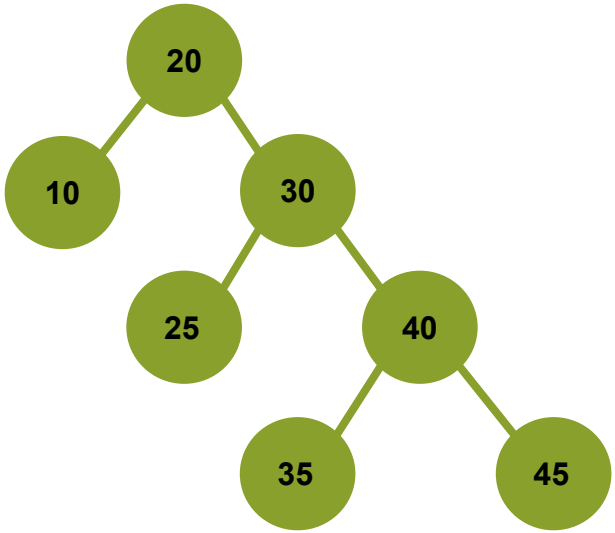
splay tree - zig-zig (double rotation)



splay tree - zig-zig (double rotation)

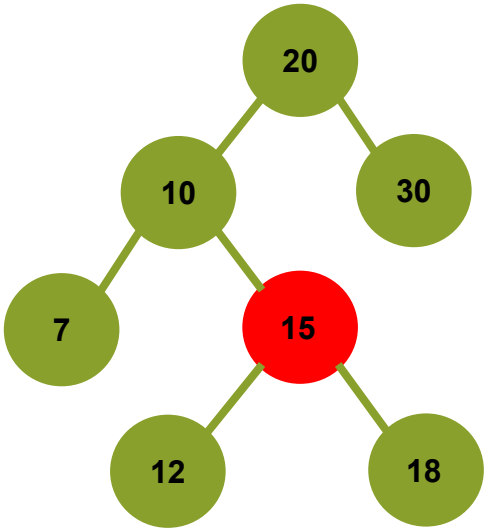


splay tree - zig-zig (double rotation)

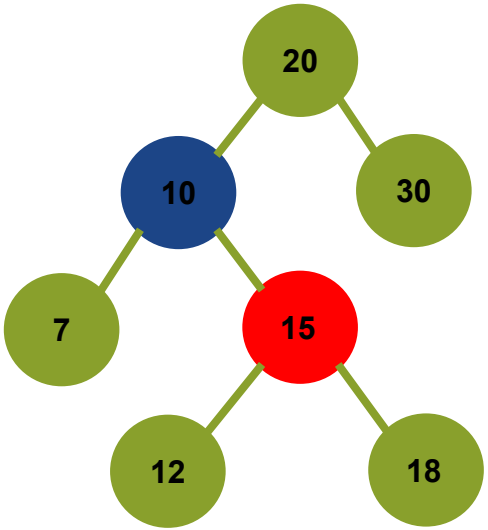


splay tree - zig-zag

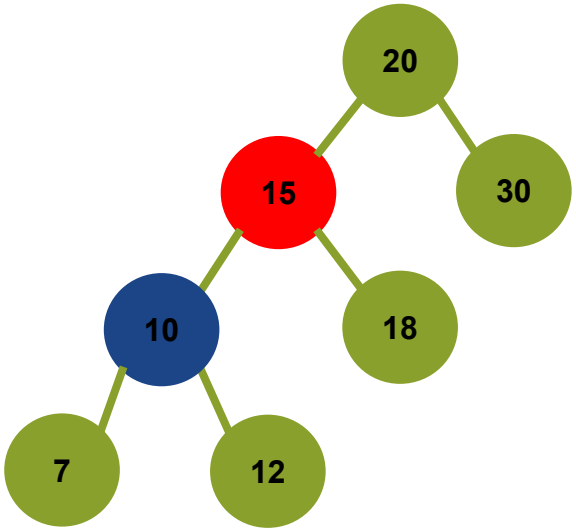
parent of desired node is not the root and node is right(or left) child and parent is a left(or right) child



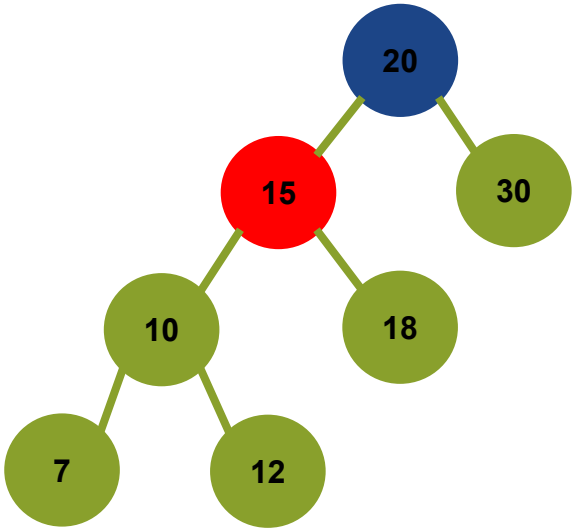
splay tree - zig-zag



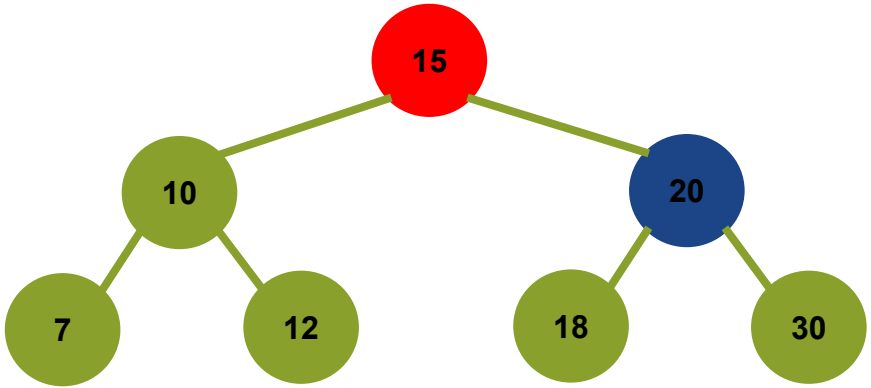
splay tree - zig-zag



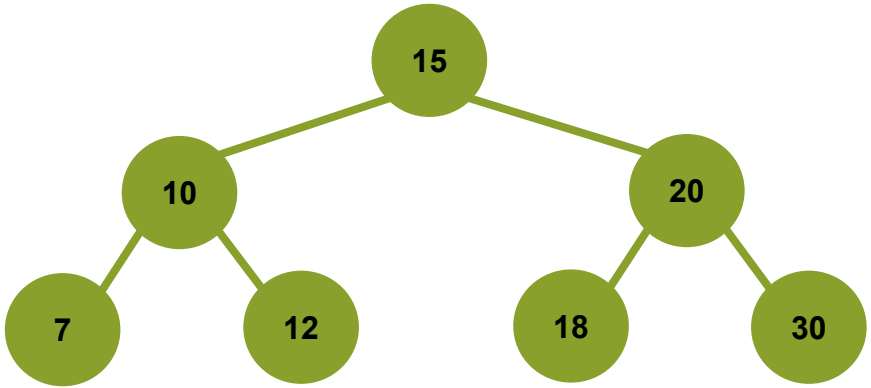
splay tree - zig-zag



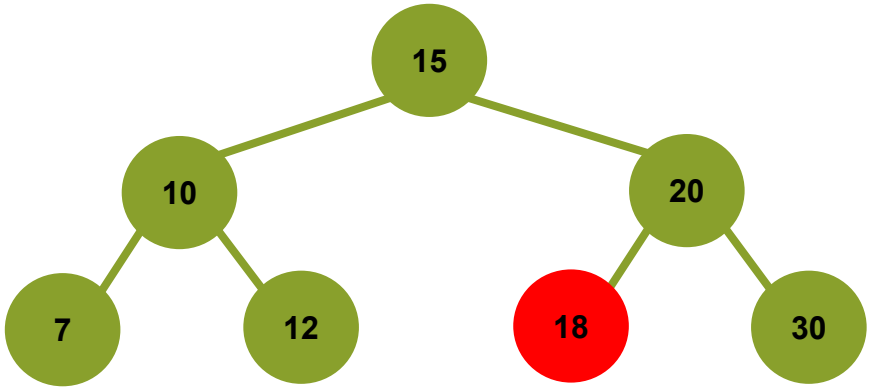
splay tree - zig-zag



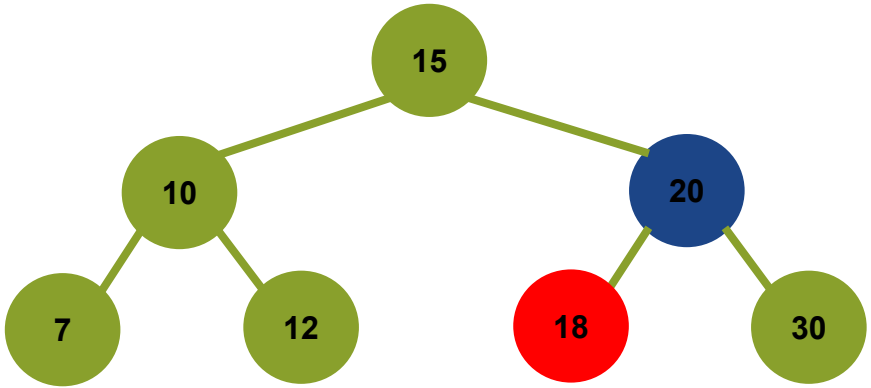
splay tree - zig-zag (double rotation)



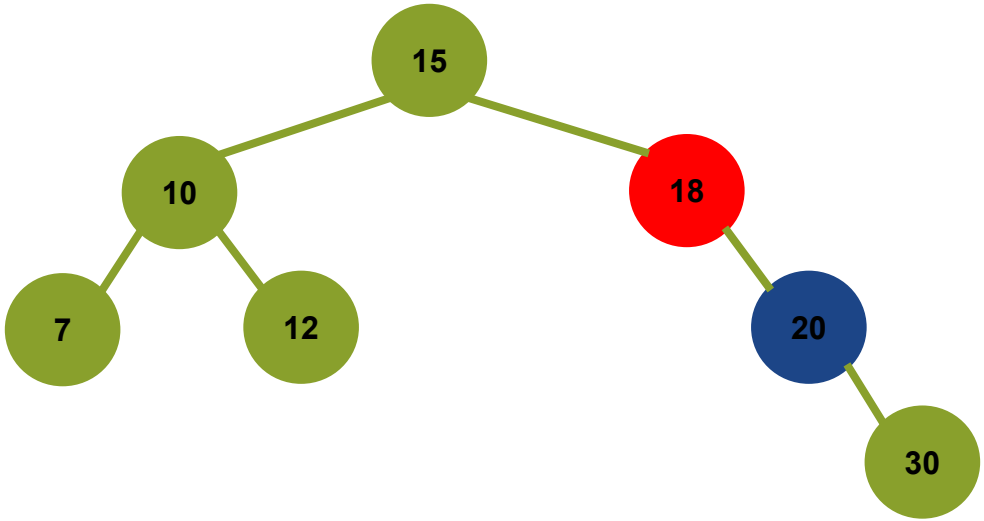
splay tree - zig-zag (double rotation)



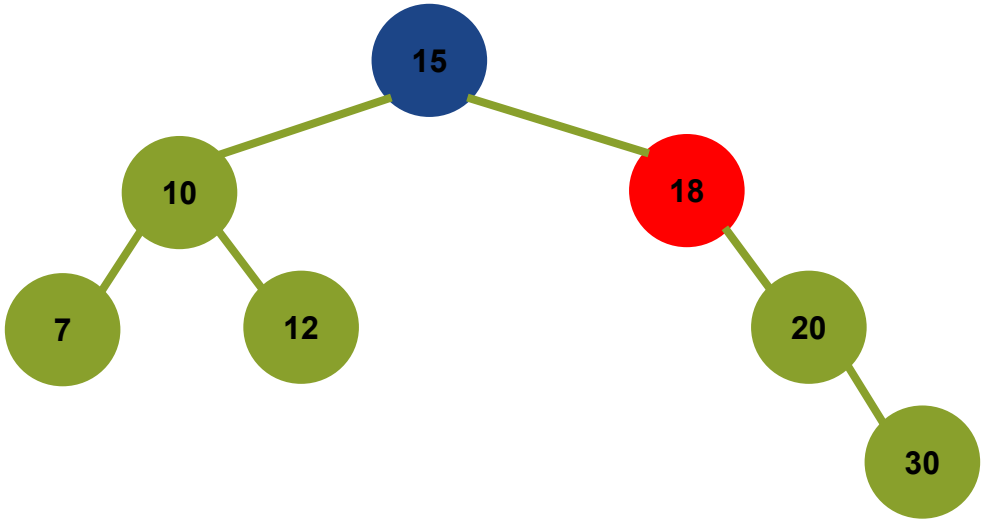
splay tree - zig-zag (double rotation)



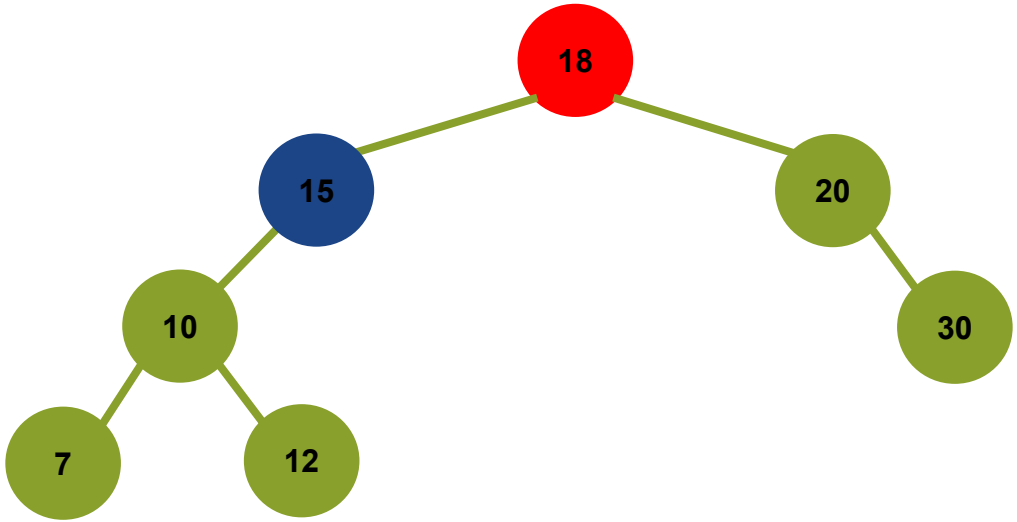
splay tree - zig-zag (double rotation)



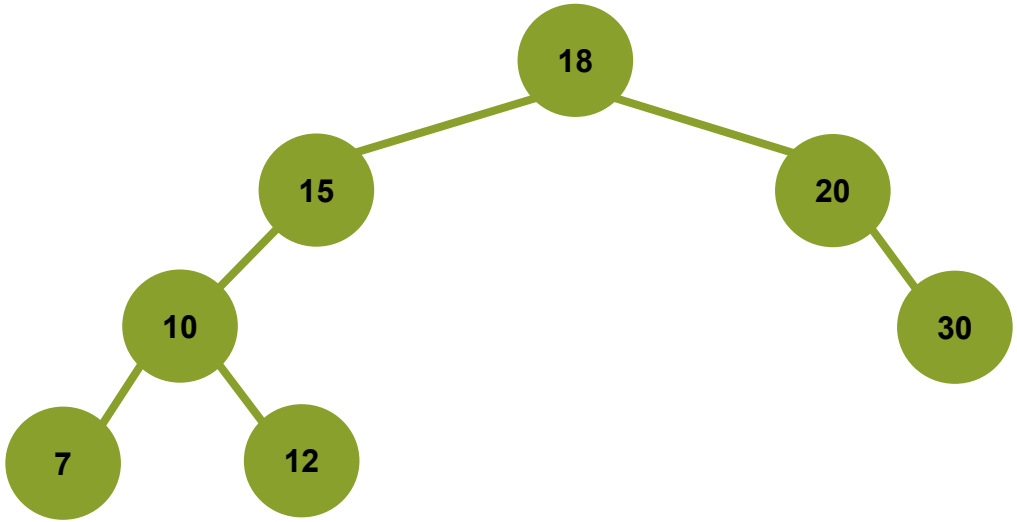
splay tree - zig-zag (double rotation)



splay tree - zig-zag (double rotation)



splay tree - zig-zag (double rotation)

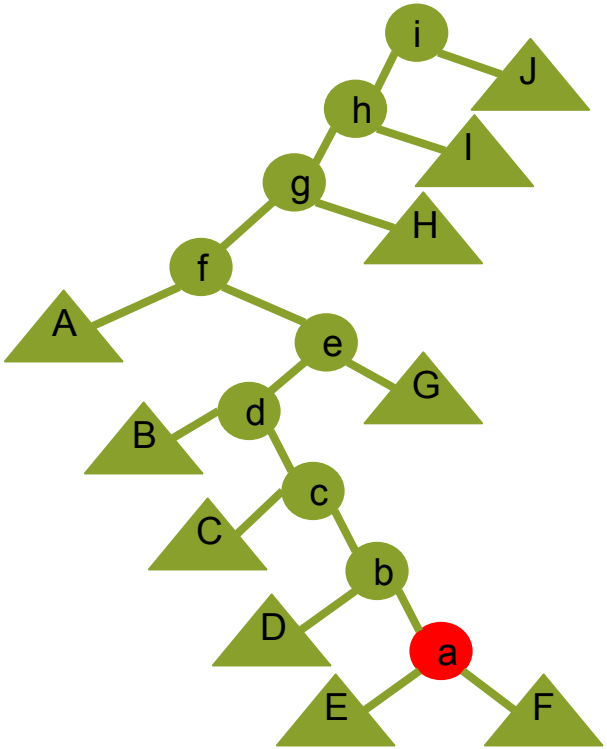


splay tree

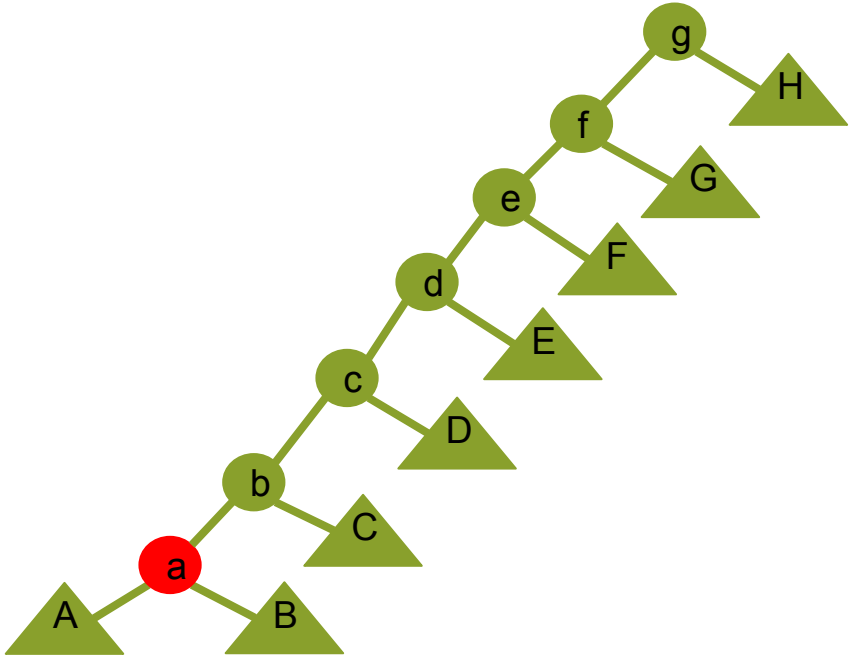
- when the root is to be accessed
  - no rotation occurs
- when the item is not in the tree
  - the most recent non-null node accessed



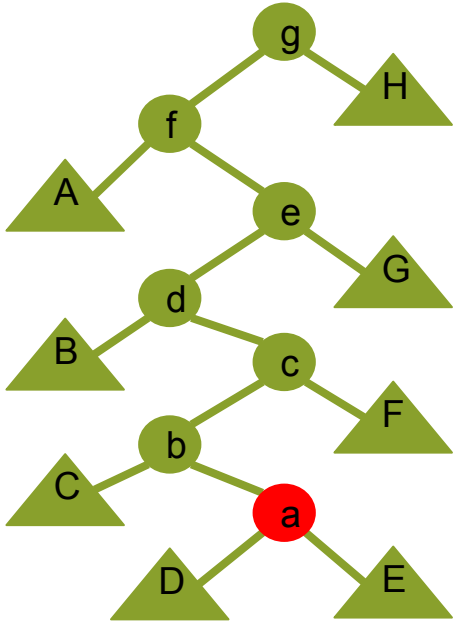
splay tree - exercise



splay tree - exercise



splay tree - exercise



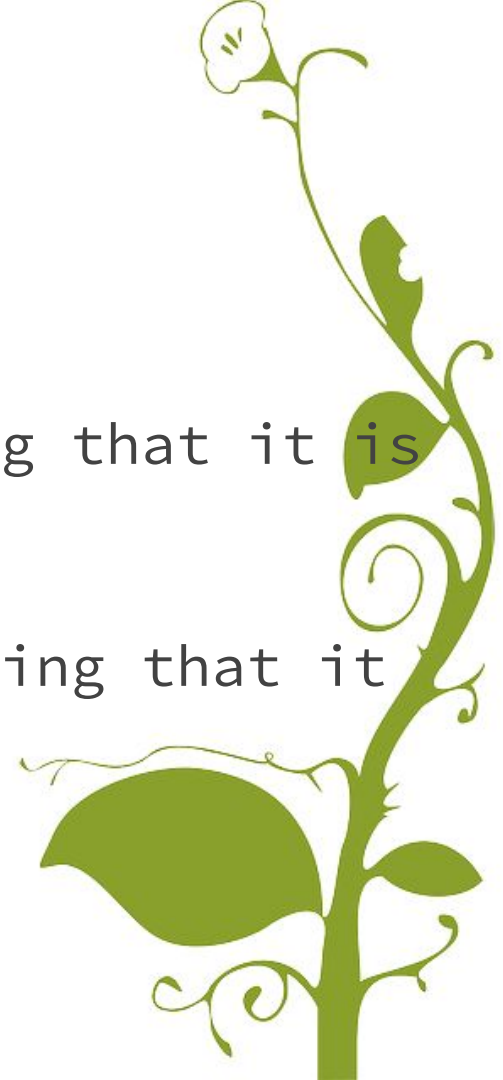
splay tree operations

- access(i)
  - this is the search shown in previous slides
- join (t1, t2)
  - joins 2 trees t_1 and t_2 assuming that all items in t_1 are less than t_2
- split(i)
  - Construct and return two trees t_1 and t_2, where t_1 contains all items in t less than i, and t_2 contains all items in t greater than i.



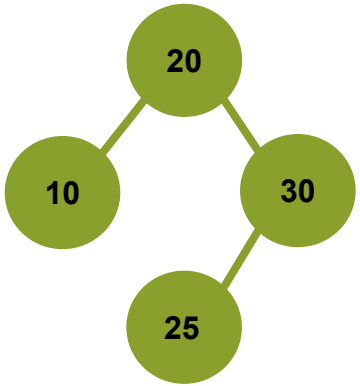
splay tree operations

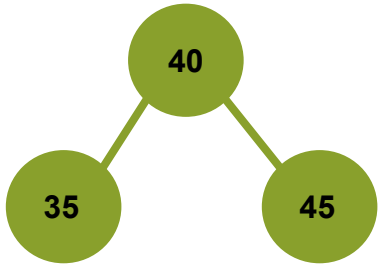
- insert(i)
  - Insert item i in tree t, assuming that it is not there already
- remove(i)
  - Delete item i from tree t, assuming that it is present



splay tree operations - join(t)

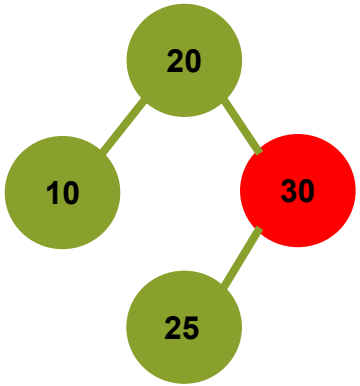
- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree

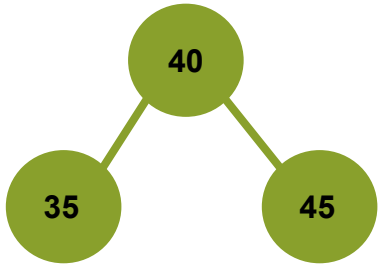




splay tree operations - join(t)

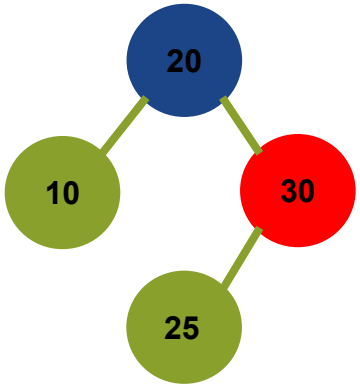
- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree

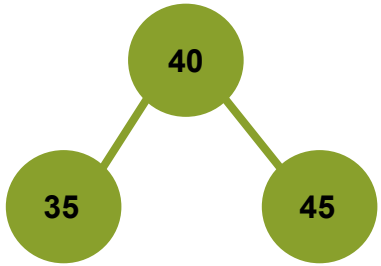




splay tree operations - join(t)

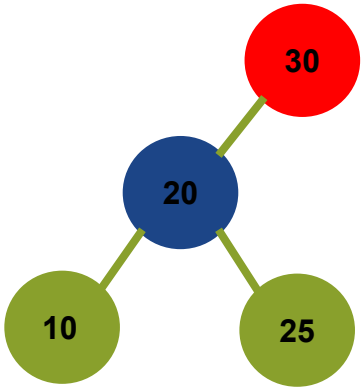
- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree

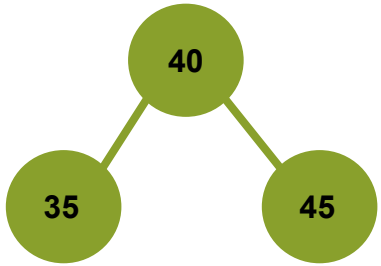




splay tree operations - join(t)

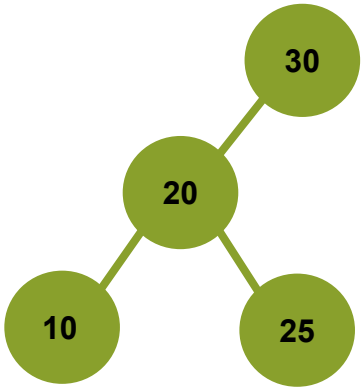
- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree

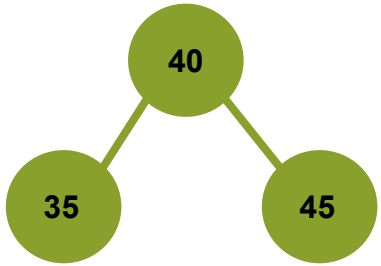




splay tree operations - join(t)

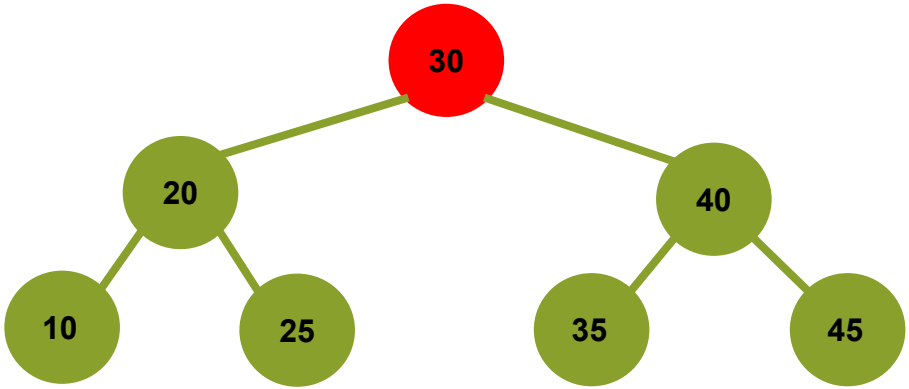
- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree





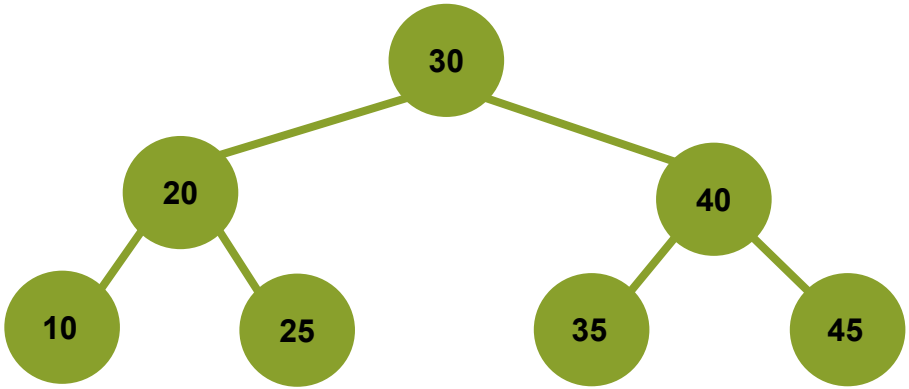
splay tree operations - join(t)

- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree



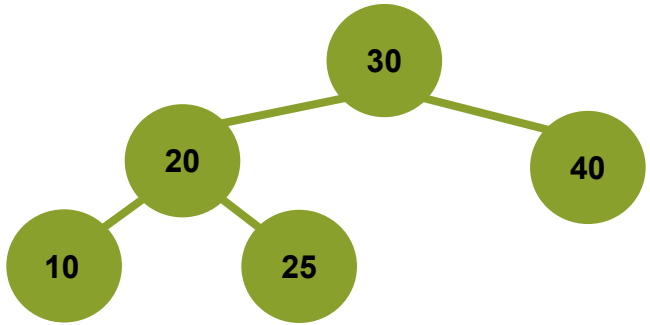
splay tree operations - join(t)

- access the biggest element of the tree
  - this will result with the new root with no right child
- make t the right subtree of the tree



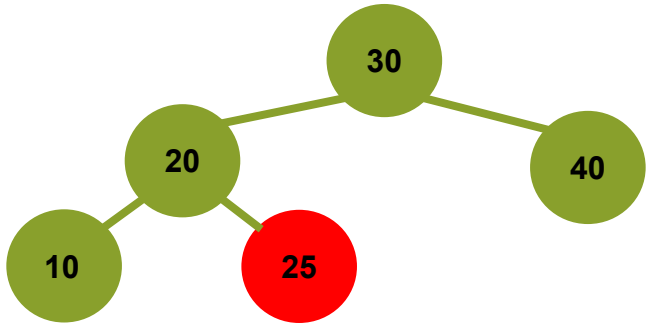
splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root



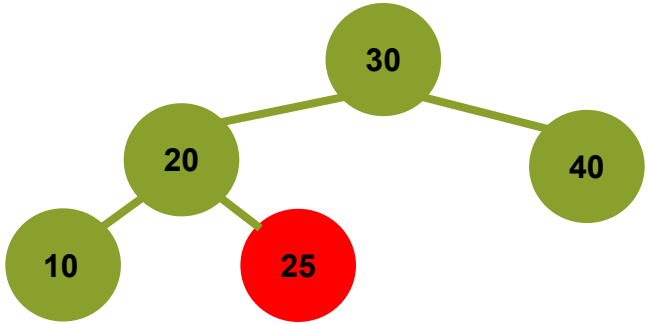
splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root



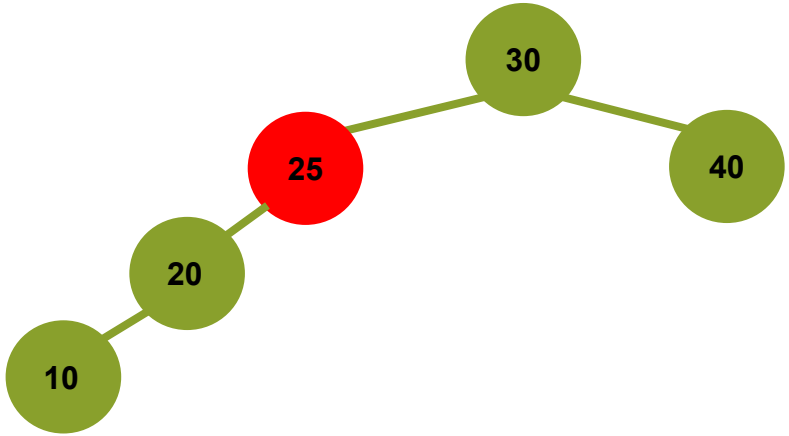
splay tree operations - split(i)

- access i
  - this will create a new root



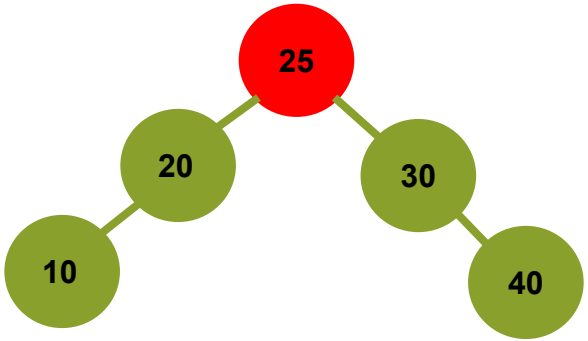
splay tree operations - split(i)

- access i
  - this will create a new root



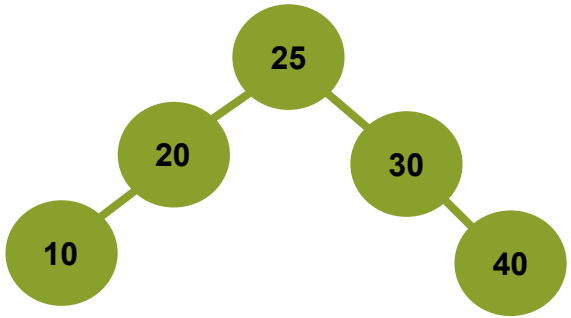
splay tree operations - split(i)

- access i
  - this will create a new root



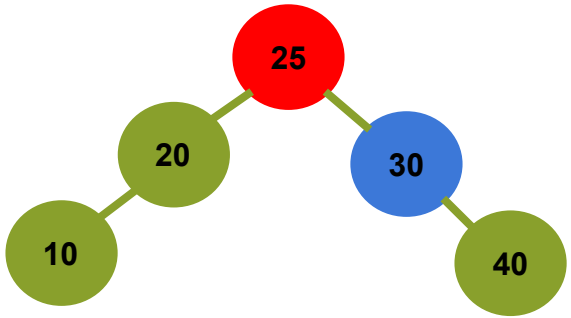
splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root



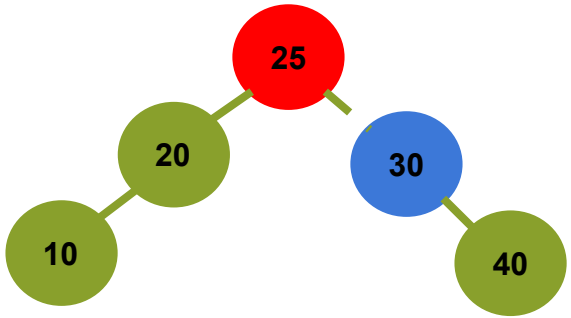
splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root



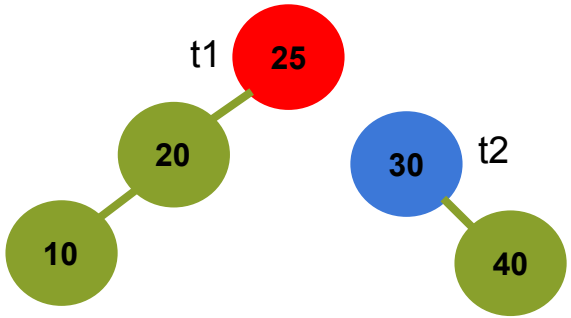
splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root



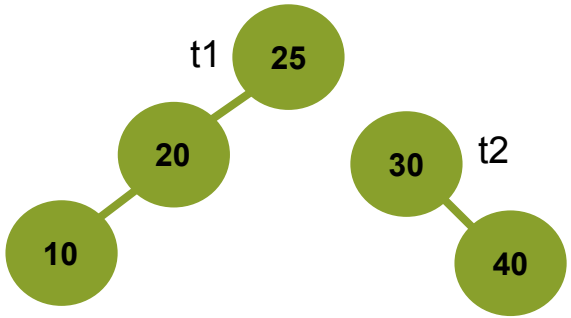
splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root



splay tree operations - split(i)

- access i
  - this will create a new root
- return the two trees by breaking either
  - left link of the new root of the tree
    - i is greater than the root
  - right link of the new root of the tree
    - i is less than the root

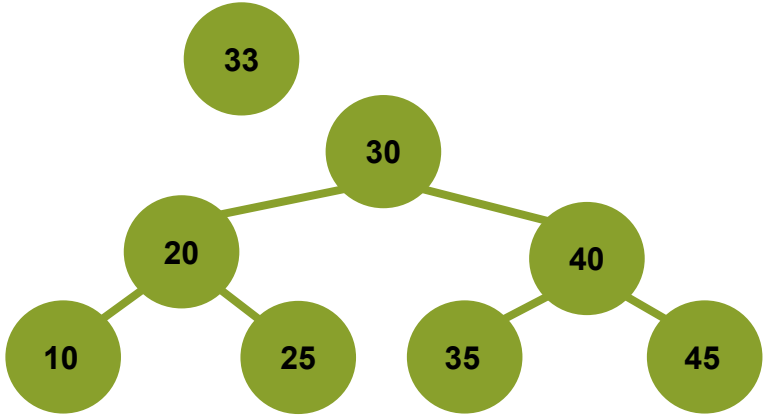


splay tree operations

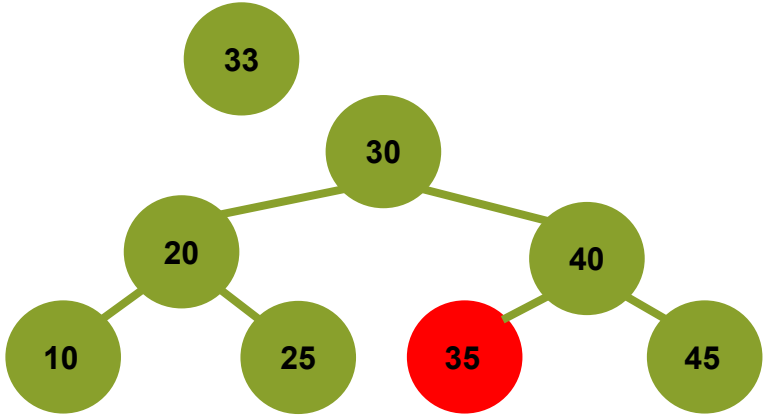
- insert(i)
  - split(i)
    - access(i)
  - use i as the new root of the tree
    - set t1 as left subtree of i
    - set t2 as right subtree of i
    - remember t1 and t2 returned by split



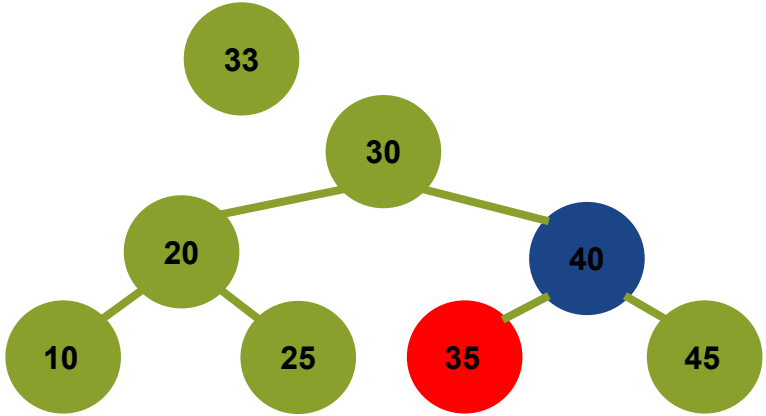
splay tree operations - insert 33



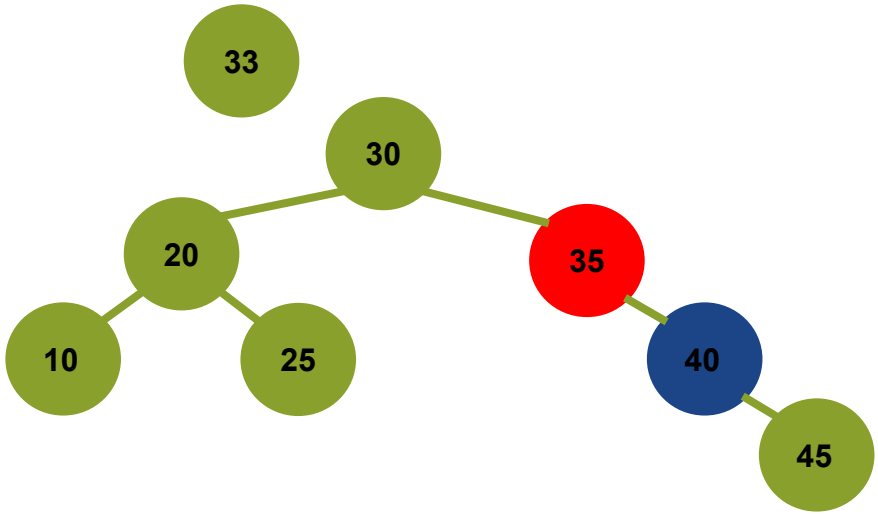
splay tree operations - insert 33



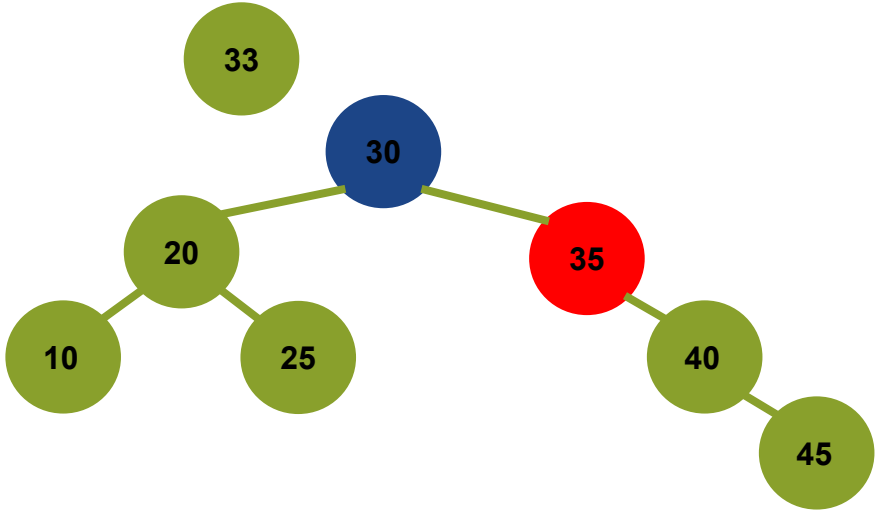
splay tree operations - insert 33



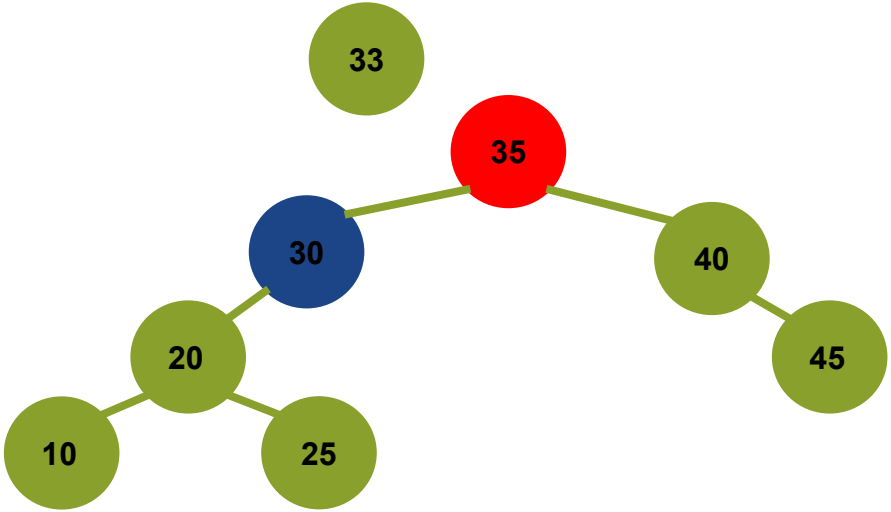
splay tree operations - insert 33



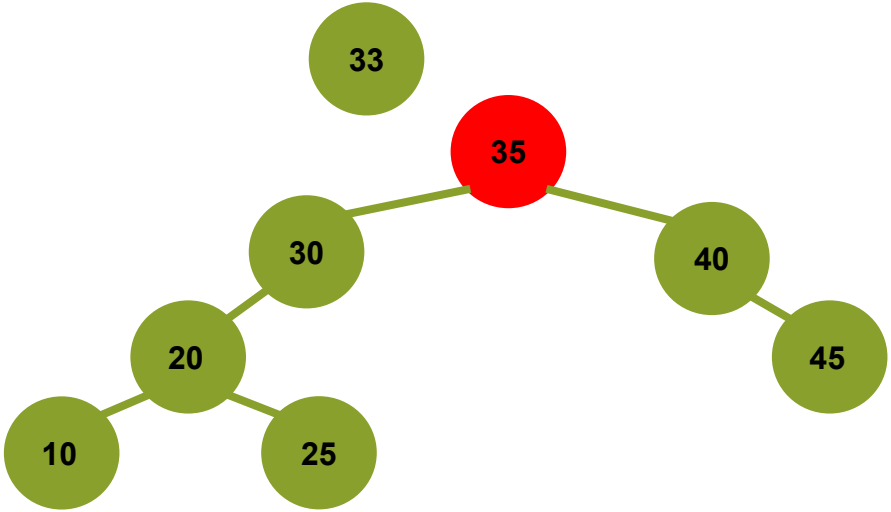
splay tree operations - insert 33



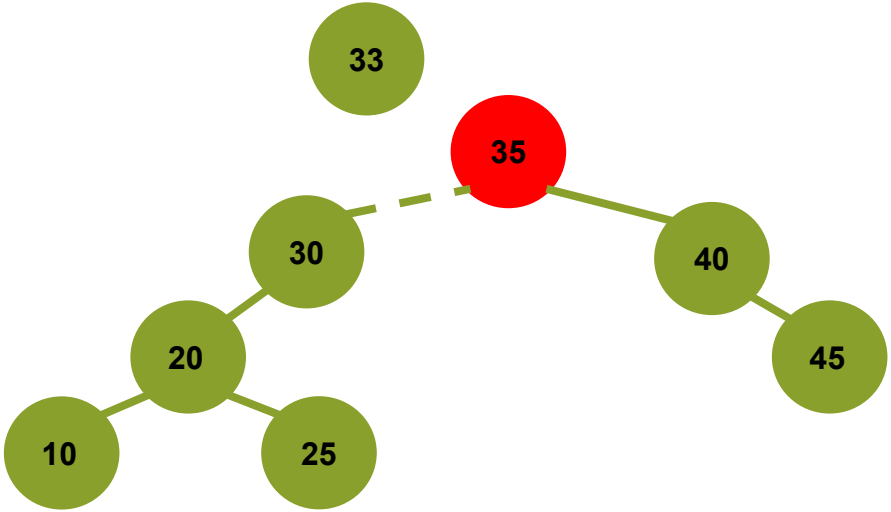
splay tree operations - insert 33



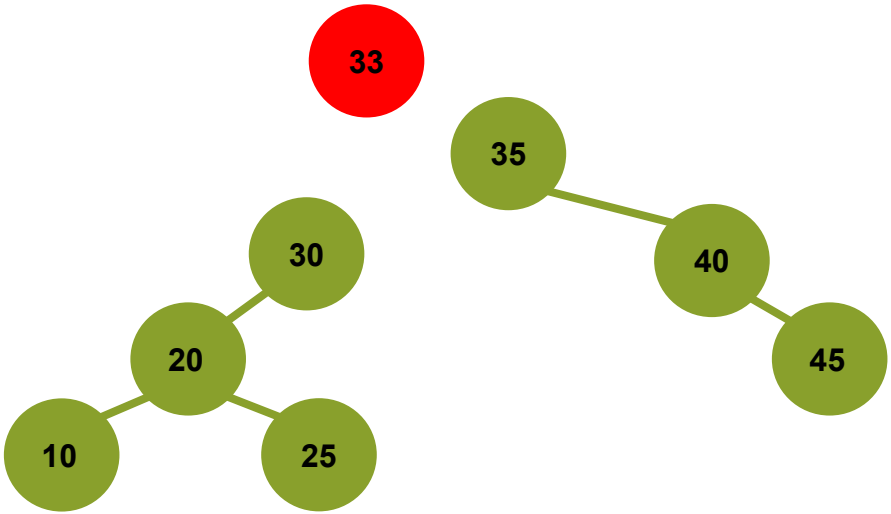
splay tree operations - insert 33



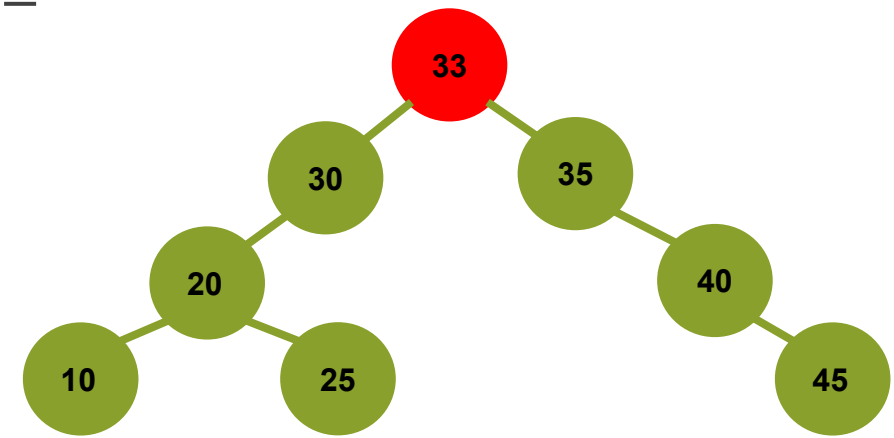
splay tree operations - insert 33



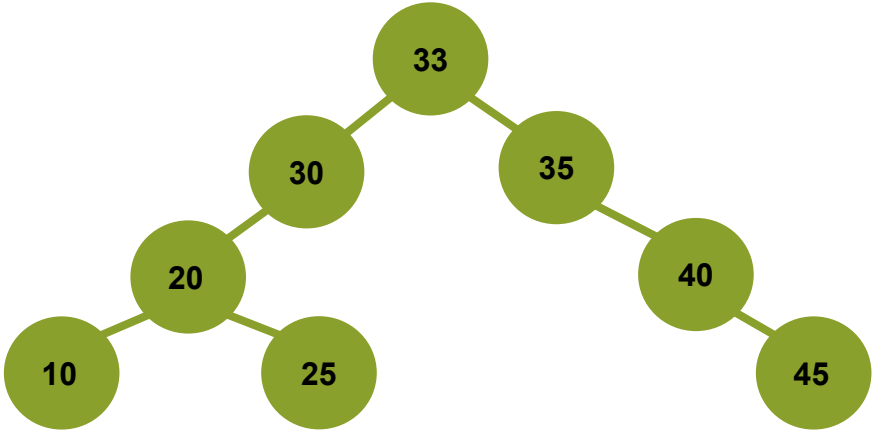
splay tree operations - insert 33



splay tree operations - insert 33



splay tree operations - insert 33

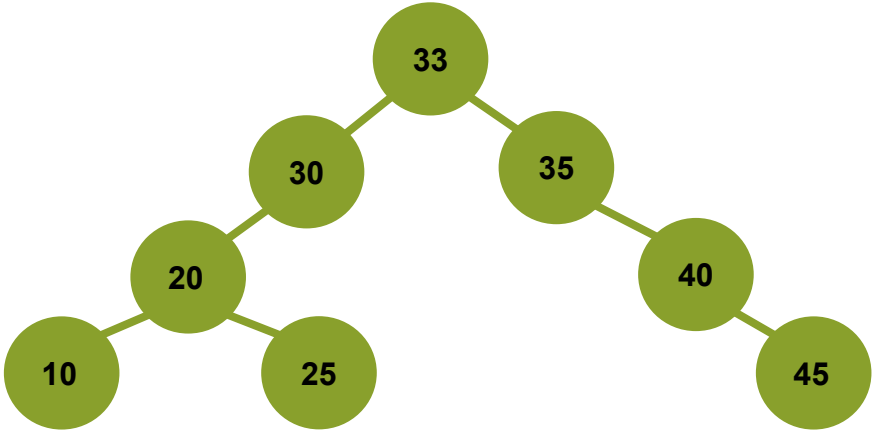


splay tree operations

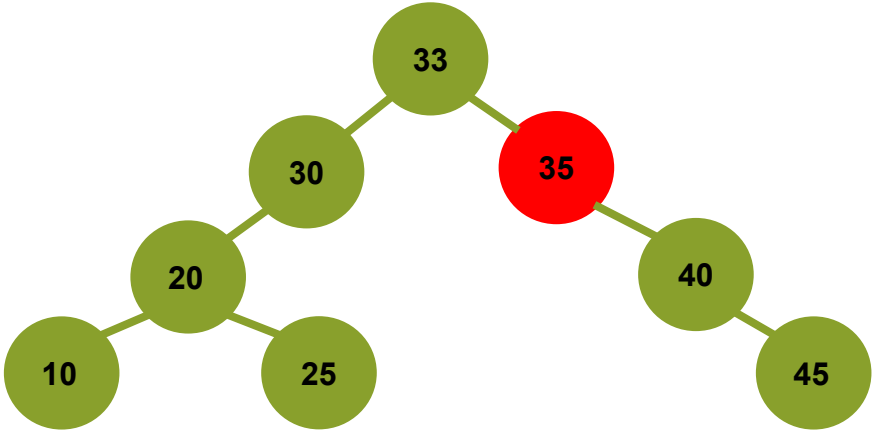
- remove(i)
  - access i
    - i becomes the new root
  - delete i
    - left subtree L
    - right subtree R
    - join(L,R)
      - notice that all items in L are less than those in R



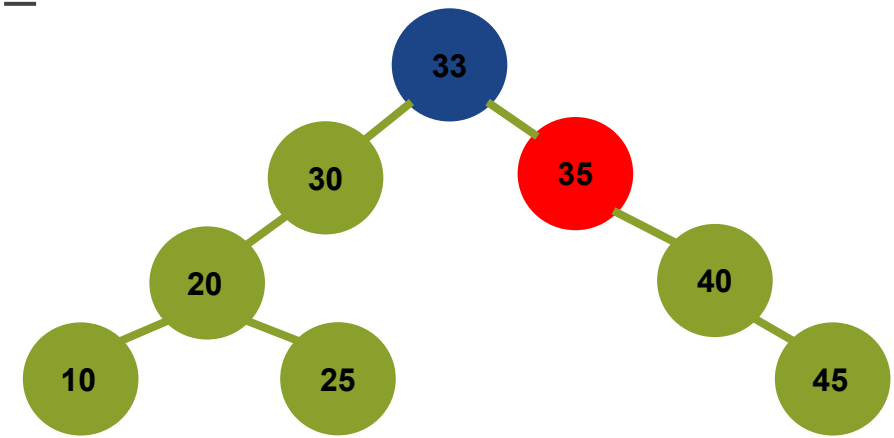
splay tree operations - remove 35



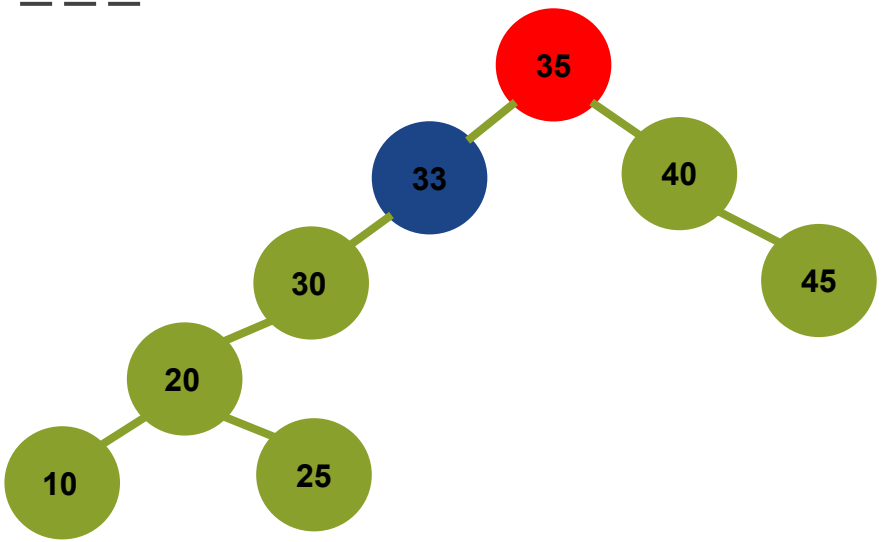
splay tree operations - remove 35



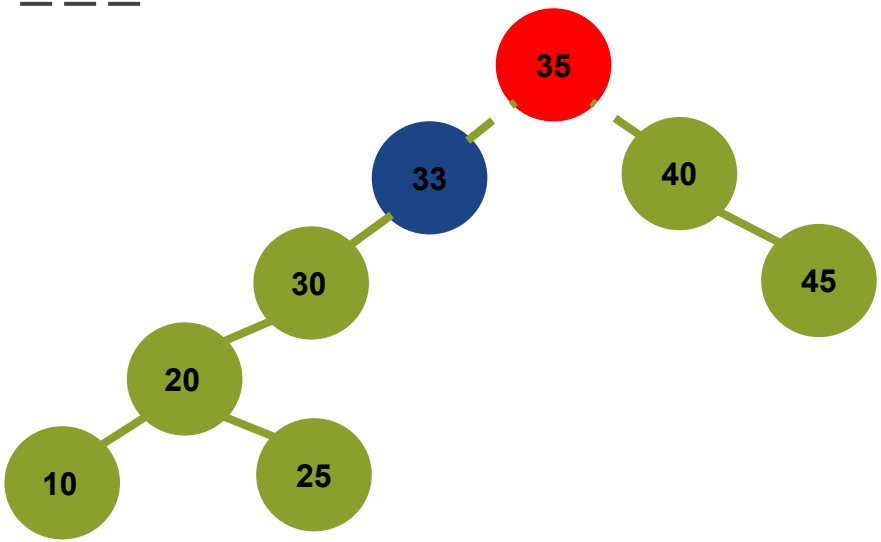
splay tree operations - remove 35



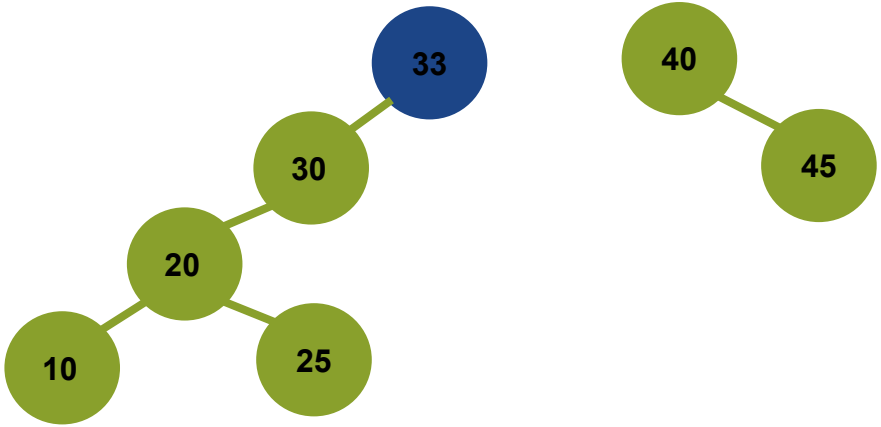
splay tree operations - remove 35



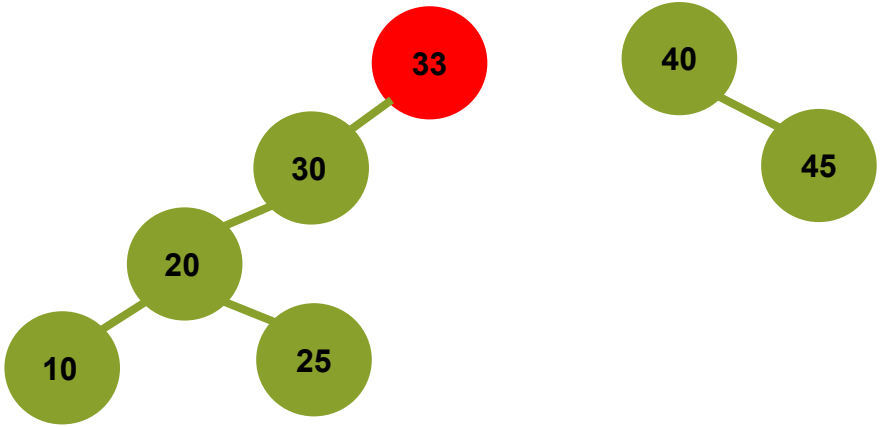
splay tree operations - remove 35



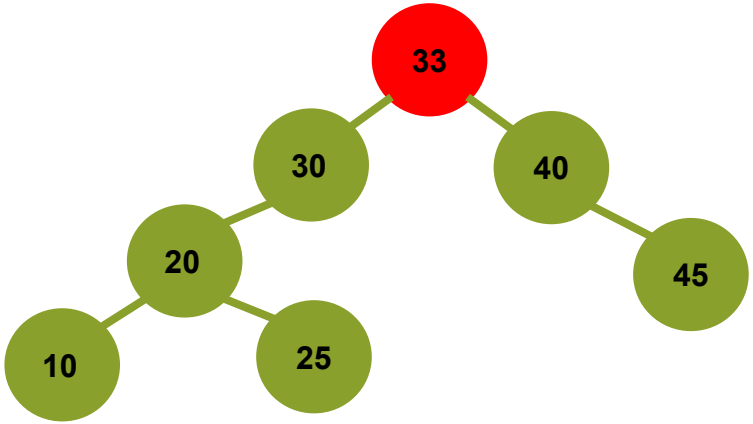
splay tree operations - remove 35



splay tree operations - remove 35



splay tree operations - remove 35



splay tree operations - remove 35

